The Thiele Machine

A Computational Model with Explicit Structural Cost

Devon Thiele

Contents

The point 1

Before the book: the short version 40

I The picture 60

1 How I got here 61

2 What structure is 66

II The axiom 88

3 A record and a toll 89

4 Why the toll isn't arbitrary 103

5 Learning is free, committing is not 136

III The logic 148

6 The counter 149

7 Windows and shadows 161

8 Structural entitlement 181

9 What insight is 200

10 Records over any base: the latch 206

11 The whole direction 227

12 The shadow, exactly 235

13 The diagonal 241

14 Five small models 257

15 The pointer thesis 274

IV The machine 288

16 The smallest honest machine 289

17 Every model 374

18 Built from parts 381

19 Running it 387

20 The CHSH check 393

21 The counter against physics 413

V Checking it, and its edge 422

22 What Coq is, and why 423

23 How to break this 434

24 The claim ledger 442

25 Scope 470

26 Conclusion 476

Appendices 481

A Things I had to learn first 481

B Every assumption in one place 491

C What's mine, and what isn't 495

D Every result and where it is checked 506

E The exact words of twenty-one results 555

F The CHSH derivation, in full 579

G Constructing the quoting operation 601

H Sources and mathematical dependencies 604

The point

The strongest thing in this book first, so you know what the rest is for. On the small machine of Part IV, every program anybody can write, run from a clean start, raises its flag only after a passing check of a claim, a passing commitment to that same claim about the same edition, and then a passing certificate, in that order (Theorem 16.9). And one fixed program, the universal Thiele machine, runs every one of those programs on a host with more counters: it halts exactly when its guest halts, its flag goes up exactly when the guest's does, and its ledger equals the guest's at every guest step and, in between, never gets ahead of the guest's next one (Theorems 16.29 and 16.30). So the guarantee holds for every program, carried through one interpreter, with the cost counted exactly, and all of it is checked in Coq. Putting a rule under every program is an old idea, and I say whose later in this chapter. Carrying it through a universal interpreter with an exact ledger is the piece I didn't find done before, where I looked (Appendix C).

Here's the whole thing before any of the careful parts. I'm going to start with one small picture, take it apart, and keep asking what each piece means, as far down as I can get before I hit something that holds or something I can't get under. I'll tell you which. Then I'll put it back together at the end and you can check whether it still stands.

One warning about how this went, because it explains the shape of everything after it. Most of the time I had the hunch first and found the names later. Over and over, somebody had been here before me. Sometimes that made my day. Sometimes it pissed me off. You'll see both.

Two computers that end up the same

Put two computers on a table. Hand each of them the same list of a thousand numbers.

The first one sorts it. It compares, swaps, walks the list again and again until nothing is out of order. Then it does one more thing: it walks the whole list front to back and checks that every number is no bigger than the one after it. That check could have failed. It passes. Then it tidies up after itself, wipes its scratch work, and leaves the sorted list in memory. (That check compares neighbours, so what it settles is that the output is in order. That the output holds your thousand numbers and no others is a second claim, and it would take a second check comparing the output with the input. When I say sorted below, I mean in order.)

The second one never looks at your list. Somebody typed the sorted version into it earlier, same thousand numbers, same order, and it puts them in memory.

Now wait until both are done and look inside. Same bits in memory. Same bits on the screen. Set it up right, which you can, and it's the same everything, down to the last bit. One of them found something out about your list and stood behind it. The other one is holding identical bits and found out nothing. Wheel them out on a cart, swap them, wheel them back, and ask me which is which. I can't tell you. You can't tell me. And the part that bugs me most: built the ordinary way, neither machine can tell you either.

Sure, while they ran they were different. One did thousands of comparisons, the other did a copy. But the runs are over and the runs are gone, and nobody was being sloppy. Ordinary computers are built to forget how their bits got there, on purpose. Later in this chapter there's the sentence where a physicist said so in 1961, and why he said it costs heat.

Who cares? Anybody who's been handed a sheet that says "all tests passed." Anybody who's looked at an inspection sticker on a windshield: the sticker looks the same whether somebody checked the brakes or somebody sold the sticker. An audit, a proof, a ledger, an AI that sounds very sure of itself: every one of them is a pile of bits that could have come out of either computer, and the question that matters is the one the bits can't answer. Did something establish this, or did it just print it? I'll come back to every one of those at the end of this chapter and say which of them this book actually helps with. Fair warning: fewer than you'd hope.

I'm going to call the difference *established*. The first machine established that your list is sorted. The second has the same answer and established nothing. The rest of this chapter is me taking that one word apart.

Look harder

The obvious answer is: just look harder. Don't look at the end, look at everything. So make the computers as lookable as computers get. Scale them up to the size of a warehouse, every wire a lamp that's lit or dark, every switch something you can walk up to and touch. Nothing hidden in a chip too small to see. Put both of them side by side and walk the floor.

While they run, sure, you'd see the difference. One has a whole wing lighting up, comparing and swapping, and the other just copies. But they finish. The comparing wing goes dark. Now walk the floor and look at every single lamp. The sorted list is spelled out in lit and dark lamps in both, the same pattern, and everything else is dark in both, because both cleaned up.

There is no lamp anywhere that says "a check ran here and passed." Not hidden. Not encrypted. Just not there. You can see everything there is, and the thing I'm after isn't one of the things there is. Looking harder doesn't help, because the problem isn't that you can't see enough. It's that nothing was kept.

That's the shape I want you to carry through the rest: the problem was never visibility. It's keeping.

So keep the trace

Then the fix is obvious, isn't it? Don't let the run go. Keep the trace: every comparison, every swap, the check and its answer, written down as it happens. Wheel the computers out on the cart and back, read the traces, and you'll know which one sorted. Problem solved?

It's the right instinct, and it gets most of the way to this book. But take it apart, because "just keep the trace" has three things wrong with it, and each one is a piece of what comes later.

First, a trace is bits, and bits can be typed in. Whoever typed the sorted list into the second machine can type a trace in too: "compared, swapped, checked, passed," a few thousand lines of it, every one of them plausible. A trace written by a program is something the program produced, and the machine running the program doesn't care whether it's true. So keeping the trace doesn't answer the question. It moves it. Did something establish this trace, or did it just print it?

Second, the way out of that is to take the pen away from the program. Don't let the program write its own trace. Have something underneath it write the trace, one step at a time, as the steps actually happen, so nothing a program does can add a line that didn't happen or erase one that did. Airplanes do a version of this. Flight recorders go back to a prototype David Warren built in Melbourne in the 1950s. He was an Australian scientist at the Aeronautical Research Laboratories, working on jet fuel, and after a string of unexplained airliner crashes he wanted a box that would hold the instrument readings and the voices in the cockpit for whoever came looking afterwards; I know his story from museum and encyclopedia accounts, and I haven't read his own report. The recorders that came out of that idea are fed by the aircraft's own instruments. Nobody's account of events goes into them. And they're built to survive the crash, so the investigators don't have to depend on anyone's memory of what happened. That's the right shape, and it isn't mine. Later in this chapter it gets its name from computer security, a reference monitor, and I'll say outright that a machine built that way would be doing what this book describes. What the book adds is narrower than a rulebook for everything underneath. It's about the records that go up and never come back down: what that something has to keep for good, and what keeping it costs.

Third, you can't keep all of it. A full trace grows with every step and never stops growing, and a machine with finite memory runs out. Even the flight recorder doesn't keep everything. The cockpit voice recorder is a loop. I went looking for how long the loop is, and it has kept growing: for decades it held the last thirty minutes, an American rule in 2008 raised that to at least two hours, and big airliners built since 2021 carry twenty-five hours. Whatever the length, everything older gets recorded over. Every real machine that keeps a trace keeps a window of it, or a fingerprint of it, and forgets the content.

The fingerprint is the sharpest reply, so take it seriously. Fold every new line of the trace into a fixed-size scramble of everything before it, and you've got a hash chain. The timestamps of Stuart Haber and Scott Stornetta, two cryptographers at Bellcore whose story comes later in this chapter, work that way; so do the audit logs the cryptographers Bruce Schneier and John Kelsey designed in 1999; and so do the measurement registers in the security chip a lot of computers carry, the TPM. A running program can only extend those registers, folding a new measurement in. It can't write a value of its own into them, and the ones that matter here start over only at the next reboot. That really does keep something about the whole trace in a few dozen bytes. But look at what it keeps. It keeps a binding. The content isn't in it: to read the history you still need the trace stored somewhere, and the fingerprint only tells you whether a copy somebody hands you is the one that went in. The binding holds only because nobody knows how to find two traces with the same fingerprint,

which is a bet on how hard a search is. No law of nature backs it. That part isn't only my opinion. The computer scientists Moni Naor and Guy Rothblum proved a sharp version of it in 2005, and they even call the thing a fingerprint: keep a small private fingerprint of a big file somebody else stores, check the file by reading only a little of it, and against an opponent with unlimited computing the size of the fingerprint times the amount you read has to be on the order of the whole file. Do even slightly better than that against an opponent with limited computing, and you've shown that one-way functions exist, which is the hardness bet in its barest form. Their setting is a stored file. A growing trace is a different setting, but it's the same bet, with its terms written down. And squeezing traces that keep growing into a fixed number of bits has to land many traces on one fingerprint. That's a merge, hidden behind how hard it is to find which traces share a seat. It's the same merge as the people and chairs further down, where four people have to sit in three chairs. Fingerprints are great. They're not a way out of forgetting. They're a very clever way of forgetting.

So the real question was never "save everything." It's this: what's the smallest thing worth keeping for good, and what does keeping it cost? The answer this book ends up with is the commitment, in the database's sense, a question closed, and the trace drops away. (The cryptographer's sealed envelope also goes by the name commitment. It's a different thing, and both come up later.) Not every comparison the sort made. Just the one fact that a check of this very claim passed and the question got closed. And here's the part that matters most: on a finite machine, keeping even that one fact for good forces the machine to forget something somewhere else. That's the chairs, further down. And keeping things for good fills up, which Part II turns into a count.

So the premise was never that computers can't keep records. They keep records all day. It's that nothing in the rules of an ordinary computer makes the record paid for, permanent, or earned, and those three come in at different places, so here's where. Paid for is the model's one rule: in the Thiele Machine, the step that raises the record has to cost something. Permanent is a premise, the one the chairs need, and one of the conditions Part III puts on a record. Earned, meaning a check of this very claim passed first, isn't in the model's core at all. It's what the step of the small machine in Part IV enforces, proved for that machine and every program on it run from a clean start. One more word rides along with permanent, and it'll come up when I line my record up against other people's: dated. A record that goes up and stays up went up at some particular step. The first raising occurs at a definite step in the execution history. The latch itself does not store that step number. So dated, in this book, means dated in the execution history, and carrying a timestamp is a different property that the record doesn't have. A machine that charges one mark for every

step it takes meets the first rule, and the charging does nothing toward the others, and I'll keep pointing that out.

Computed, stored, established

Three words that get used like they're one.

Computed is about the steps. The first machine computed the sorted list. The second didn't.

Stored is about the bits. Both machines stored it.

Established is neither, and that's the whole problem. It isn't the steps, because they're gone. It isn't the bits, because they're identical. It's something like this: the first machine has a right to lean on "this list is sorted," because it ran a test that could have failed, the test passed, the test was about this very list, and then the question got closed. The second machine has the same sentence and no right to it. That's a true fact about the two of them, and it's written down nowhere, in either one. (Whose right, strictly? A machine doesn't lean on anything, any more than a filing cabinet does. It's the right of whoever reads the first machine to lean on what it says, and the machine is where the grounds for that right would have to be kept. "The machine has a right" is shorter, so I'll keep saying it, and now you know what it's short for.)

Four words I'll hold still from here on. A *check* is the test, the thing that could say no. A *commit* is the closing, the question shut for good, about that edition of that thing. The *record* is whatever stays behind to say so. And *established* is all three, about one thing. That's the working sense. The book cashes it at three depths: a paid closing in the bare model; an object with a checked relation and a paid event, in Part III; and the whole chain on the small machine, in Part IV. Each place says which one it means.

Which raises a fair question. What if the second machine, handed the typed-in list, runs the check on it? Then it has established that the list is sorted. Full stop. It didn't do the sorting and it doesn't need to. The line I'm drawing is between checked and unchecked. Whether the list was computed or copied doesn't enter into it, and "found out nothing" up there was about the copier that never looked. So whatever this book ends up counting, it isn't how hard anybody worked.

People say "a right to lean on it" about a lot of things that turn out to be vibes, so I had to find out whether this one was anything. So I went looking. Here's what I ran into.

The clerk who forgets everything

The picture of a computer I started with is a person with a pencil, a strip of paper and a short list of rules. So I went back to the source, Alan Turing's 1936 paper, to see how far he actually takes it.

He takes it a long way. Back then a "computer" was a job, a human doing sums by hand, and Turing modelled that person. He's careful with the person's mind: whatever the clerk knows, beyond a handful of states, has to go down on the paper. Then he goes all the way. He imagines the clerk getting up, walking off, forgetting everything, and coming back later. To pick up where he left off, the clerk leaves himself "a note of instructions," and Turing says that note *is* the clerk's state of mind, as far as the computation is concerned. Marks on paper and a note. That's the whole state.

My two computers are sitting right inside that. Suppose the clerk didn't write the note. Somebody slipped it under the door, in the right form, with the right instructions. He comes back, reads it, carries on, and every mark he makes from then on is exactly what he'd have made if he'd written it himself. Turing's model can't tell the clerk who did the work from the clerk who was handed a note saying he did. It was never asked to. Turing wanted something you could check without crawling into anybody's head, and a note on a desk is about as checkable as it gets. That's why the paper still works ninety years later. It's also why there's no place in it for the thing I'm after.

So the founding picture of computing is a man who leaves his desk, forgets everything he was doing, and trusts a sticky note. I relate to him deeply.

And now the question nobody needs answered to follow the argument, which is exactly the kind I can't leave alone: who were those clerks? Real people. In the late 1800s the Harvard Observatory hired rooms full of women to do the arithmetic of the sky, cataloguing stars by hand for a fraction of a man's wage, and some of them made discoveries of their own, Henrietta Leavitt's way of telling how far away a star is among them. (I took the wages and the discoveries from Dava Sobel's history of those women, and from the historians at the American Institute of Physics, who put the men's hourly pay at nearly five times the women's.) Rooms like that ran artillery tables in both world wars; that part I took from David Alan Grier's history of human computers. So when Turing modelled "the computer," he was modelling somebody's actual afternoon. Does that change the argument? Not a bit. But it changes the picture, for me anyway: the sticky note was always a person trusting another person's work, and the question of whether the work behind it was really done is older than any machine.

Information doesn't have it either

My next guess was information: one machine got some, the other didn't. That one died fast. Claude Shannon, who turned information into a number in 1948, says on his first page that the meaning of a message is "irrelevant to the engineering problem." His number measures how many messages you *could* have sent. What the one you sent did is outside it, which is exactly right for building telephone lines and exactly no help to me. Same message out of the same set, same number, for both computers. He'd give them the same number if one of them got hit by lightning and came out right by luck.

Fine, forget the set and look at the one message. In the 1960s three people, separately, worked out how to measure the information in one string: Ray Solomonoff, an American who was after a theory of guessing what comes next from what came before; Andrei Kolmogorov, the great Soviet mathematician; and Gregory Chaitin, an Argentine-American mathematician who later worked at IBM. Their measure is the length of the shortest program that prints the string. Same two computers, same string, same number. And then there's the thing in that theory I keep going back to and standing on, to see if it creaks. Running a program can't add this kind of information. What comes out can't hold more than what went in plus the program, give or take a little. Leonid Levin, who studied under Kolmogorov in Moscow and later left the Soviet Union to teach at Boston University, proved a general version of this, and he has a one-liner for it that's too good not to repeat: you can torture an uninformed witness all you want, you won't get information about the crime.

Which means, as far as information goes, sorting your list told you nothing. The sorted list holds nothing that "your list, plus the word *sort*" didn't already hold.

Logic says the same thing in a different coat. The standard logic of knowing, the one the Finnish philosopher Jaakko Hintikka worked out in 1962, has a property people call logical omniscience: if you know some facts, you automatically know everything that follows from them. So by that logic, once you knew every number on your list, you already knew whether it was sorted. Hintikka flagged the problem himself, in the same book, and people have been trying to fix it since. The fixes read like a pile until I asked each one the same thing: when it lets you write *known* next to "sorted," what does it make you write in the margin? Hintikka's margin is empty. Each fix writes one thing there.

The fix that comes closest to me is from 1995, by Sergei Artemov, a logician at the City University of New York as of 2026 who studied under that same Kolmogorov (the 1995 report came out of Cornell). He wanted a logic that could name the proof itself. What I think he's saying: you only get to know a thing if you've built an actual justification for it, and the justification rides along with the claim. That's a

reason traveling with the claim, which is almost a record. Only almost, because it's a way of writing things down and reasoning about them, and nothing in his margin says the reason was ever run, or when. People have taught it to count, too: Ho Ngoc Duc, a logician at Leipzig, in 1997, made knowing take reasoning steps, and Artemov and Roman Kuznets, a logician then at Bern, in 2014, asked how big a justification can get before knowing by it stops being feasible. I took those for rivals at first. As I read them now, Duc counts the steps the knower takes and Artemov and Kuznets weigh what you'd hand somebody to check it, one distance from two ends. It has a computational side, too: Francisco Bavera and Eduardo Bonelli, two Argentine computer scientists who work on what programming languages mean, in 2018 turned justification logic into a lambda calculus whose terms carry, and can inspect, the history of the computation that made them, and Samuel Bucheli, Meghdad Ghari and Thomas Studer, three logicians, Studer the head of the logic group at Bern, put justifications on a time line in 2017, where they persist and change.

Line that up against the margin this book wants: earned, paid for, permanent, dated. Artemov's reason is nearly earned, the steps and the sizes are prices, the time line is a date. Their time line keeps its dates. Mine is dated only in the execution history: the raise happened at a definite step, and the latch doesn't keep the step number. What none of it has is the closing of a question as a step with a price. Their prices are on reasoning, and the Bern justifications change, the one thing a closed question doesn't do. Push on mine, though, and it gives twice. Artemov's margin holds the reason itself, and mine only says a check passed. And their prices grow with how hard the thing was to know. This book charges the step that raises the record, and how long the checking took never enters it, so for a measure of effort, read theirs.

So the two most careful accounts I could find, one about bits and one about knowing, say the same thing: the sort added nothing.

And yet you know something after the sort that you didn't before. You can read the smallest number off the first slot. You can stop worrying. Something happened. That's the clue, and it's the first solid thing I've got: whatever *established* is, it isn't new bits. It's the same bits, *settled*.

The guy who got closest

Somebody had that clue before me, and on that clue he's the closest neighbour I've found, so he gets the most room and the hardest push.

Charles Bennett, a physicist at IBM, wrote a paper in 1988 that starts where I just ended up: information isn't the same as value. His example is an almanac of where the moon and planets will be every day for a hundred years. It has no more

information than the equations it was calculated from, but it saves you doing the calculating. So the value is the work. He called the amount of work a thing carries its *depth*.

Was this my idea, already done, by a guy at IBM? The only way to find out was to find where it parts from mine, because if I couldn't find that place I'd just be renaming his idea. And it's one word: *plausibly*. Depth is the amount of work a thing was *plausibly* made with. Bennett says straight out that you can't know how a message actually came to be; for all you know it came from flipping coins. Depth is read off the finished thing, the way a fossil's age is read off the fossil.

Bring the two computers back. The sorted list in the first one has some depth. The identical list in the second has exactly the same depth, because depth looks at the list and nothing else. A photocopy of a deep thing is just as deep. Depth tells you *some* work was done, somewhere, by someone. It's not a record that *this* machine did it, and it's not a receipt for what doing it cost.

So Bennett's standing right next to the line, on the other side of it, and as far as I can tell he got his own question right. He asked how much work a thing embodies. I'm asking who has the right to lean on it and what they paid to get there. His answer lives in the object. Mine, if I've got one, has to live in the machine.

Statues that walk off

If it isn't in the information, maybe it's in the knowing, so I went and read the philosophers. And there it was again: people had been arguing about the difference between having the right answer and knowing it for a couple thousand years.

Plato's version is a joke about statues. There was a story that the sculptor Daedalus made statues so lifelike they'd walk off if you didn't tie them down. Plato says true opinions are like that: they wander off, and they're not worth much until you tie them down with the reason why. Once they're tied down, they're knowledge, and they stay put. Knowledge tied to a reason, and knowledge that stays. Of everybody I'd have bet on to hand me that shape, Plato wasn't on the list.

Then in 1963 a philosopher named Edmund Gettier broke the tidy version in three pages. Smith and Jones are up for the same job. Smith has good reason to think Jones will get it, and he's counted the coins in Jones's pocket: ten. So Smith figures, with good reason, that the man who'll get the job has ten coins in his pocket. Then Smith gets the job, and, unknown to him, Smith also has ten coins in his pocket. True, believed, backed by a reason. Not knowledge. He was right by luck, because his reason was about the wrong pocket.

That's the push that matters for a machine. Checking one pocket and committing to a claim about another is right by luck. So whatever a machine commits to, the

check underneath has to be about *this very claim*, and I'd better say exactly what that means, because it carries a lot of weight later.

A check is about this very claim when four things hold. It tests the same property the commit names. A cousin of that property won't do. It tests that property on the same thing the commit names, and the thing next to it won't do either. Nothing has written to that thing between the check and the commit, so it's still the same edition the check looked at. And the check is honest: it says yes when the property is true and no when it isn't, on every value it could ever be handed, this one and all the others. Break the first three and you're Smith with his ten coins. Break the fourth and the check is just a rubber stamp. Later in the book I build a machine whose step won't let a commit through unless the first three hold, and whose checks come with a proof of the fourth.

Two notes on those four, because they matter later. The fourth asks for more than it needs. To rule out luck, a check only has to never say yes when the property is false; saying yes every time it's true is a bonus. And the third is a very old bug in computing wearing Gettier's coat. Security people call it time-of-check-to-time-of-use: check a file, use it, and something swaps the file in between (Matt Bishop and Michael Dilger, two computer security researchers at the University of California, Davis, wrote up how to hunt for it in 1996). Databases guard against the same thing by making sure, just before a commit, that nothing they read has been written since; H. T. Kung and John Robinson, two computer scientists at Carnegie-Mellon, called that optimistic concurrency control in 1981. A philosopher's pocket and a sysadmin's race condition, same bug. One limit, said now: a machine can only promise "nothing has written to that thing" about its own memory; for brake pads or a voter roll the third condition becomes a premise about whoever measured (Section 14).

And here's where I went further than this chapter needs, because the obvious next question is: does anything fix Gettier? People tried for years. The first patch is "no false steps in the reasoning": Smith's mistake was believing Jones would get the job, so rule that out. Then a philosopher named Alvin Goldman came back with barns, and he credits the barns to Carl Ginet, another philosopher working on what it takes to know something. You're driving through a county that, unknown to you, is full of fake barns, just painted fronts propped up to look like the real thing. You look at the one real barn in the county and think "that's a barn." True. Believed. Good reason, your own eyes, and no false step anywhere. And still you were lucky: if you'd glanced at the next one over you'd have believed the same thing and been wrong. Every patch gets a new barn, and as far as I can tell the arguing hasn't stopped. Which tells me something I'll keep: whether a check was good isn't only about this one case. It's about what the check would have done on the cases next door. Hold on to that, because it's the next section.

What "could have failed" means

I keep saying the first machine's test *could have failed*. Take that apart, because it's the piece everything else hangs off.

The machine is deterministic. Given your list, the check was always going to pass. So in what sense could it have failed? By "could have failed" I mean this: hand the checking stage an out-of-order candidate output and it says no. I do not mean hand the whole sorter an unsorted input; putting that input in order is its job. The relevant counterfactual is a defective candidate reaching the checker. An ordinary unsorted input reaching the sorter isn't it. Chance has nothing to do with it. The copier that never checks has no corresponding rejection.

Here's the shape. A check is a question asked of every possible input, and it splits them into two piles: the ones it says yes to and the ones it says no to. "Could have failed" just means the no pile isn't empty. A check that says yes to everything has an empty no pile, and it distinguishes no inputs, however long it runs and however official it looks. It can still be sound, when its property holds of everything, and then it's sound and tells you nothing you didn't have. A philosopher, Robert Nozick, made a version of this a condition for knowing, in 1981: you know a thing only if, had it been false, you wouldn't believe it.

Push on it once more, because this is where it really gives. "Could have failed" is a fact about the *procedure*, about what the check would have done on other lists. It isn't a fact about the end state at all. Look at the first machine's memory when it's done: a sorted list. Nothing in those bits says "and whatever made me would have caught a mistake." That's the warehouse again, every lamp visible and none of them saying it. The one thing that makes the first machine's answer worth more is exactly the kind of thing those bits can't hold: the copier ends with the same bits, and neither machine kept anything else. An end state can hold more than the answer, and the only way a machine can own this is to write something down at the moment the check passes, and to have that something mean what it says. More than anything else in this chapter, that's why I ended up needing a record. A better answer wouldn't have done it. Even then the record only carries half of it. It can say a check ran on an edition and passed. That the check would have said no on a bad list is a fact about the checker, and that half rides on a proof about the checker, which somebody outside the machine has to read and accept. Where that kind of chain of reasons stops is the swamp of the philosopher Karl Popper, a little further down.

One more thing that tripped me up and then didn't. Rig a check to pass only on your one list. Is that cheating? No. It's a perfectly honest check of a different claim, "this is that exact list," and it has a big no pile: every other list. It establishes exactly that and nothing more. What a check establishes is whatever it's honest about.

Put that next to the empty no pile and the two fit together: a check establishes what it's honest about. An empty no-pile means the check distinguishes no inputs. It can still be sound for a property that holds on every input; soundness and nontriviality are different requirements. One thing I'll say plainly now, so it doesn't ambush you in Part IV: the small machine I build there doesn't police this. Its step makes sure a commit was checked first, about the same thing, by a checker proved honest. It doesn't stop you asking a question that can't come out no, like "is this number at least zero." Ask one and the whole chain runs, three marks paid, and nothing is established but the chain. The certificate I show off there asks one that can fail.

Settled, whether it's true or not

Then I ran into Charles Sanders Peirce, an American scientist and philosopher, in an 1877 essay called "The Fixation of Belief." He was asking what inquiry is for, and his answer named the thing I'd been running together without noticing. The point of inquiry, he says, is to get out of doubt and settle on a belief. And when someone objects that surely what people want is a *true* belief, he answers like a guy who's watched a lot of arguments end: "as soon as a firm belief is reached we are entirely satisfied, whether the belief be true or false."

That's funny, and then it isn't, because he's right. Settling and being right are two different events. A question can be closed on a wrong answer, and once it's closed it acts closed. Peirce cares a lot about which ways of settling hold up (science wins, for him, because it has to answer to something outside any one person), but the settling itself, the question snapping shut, is there every time, and it never comes with a guarantee.

Karl Popper came at it from the other side. He was a philosopher from Vienna asking what separates science from everything else, and in his 1934 book, the one that came into English as *The Logic of Scientific Discovery*, he says every test has to stop somewhere, at some statement somebody decides to accept, and nothing in the evidence forces you to stop there instead of one step further. You stop when you're satisfied. His picture is a jury: the verdict is a decision about what happened, and the jury deciding it doesn't make it true. Science, for him, is a building on piles driven into a swamp, and the piles stop where somebody's satisfied they'll hold. They never reach firm ground.

A real jury has a problem Popper doesn't mention, because a real jury is twelve people. Lewis Kornhauser and Lawrence Sager, two law professors writing in the Yale Law Journal, noticed in 1986 that a court that votes by majority on each premise of a case and also on the verdict can end up contradicting itself: every judge is consistent, and the court isn't. Christian List and Philip Pettit, two philosophers

writing in Economics and Philosophy, turned that in 2002 into a theorem. No way of combining people's yes-or-no judgments on connected questions meets a short list of conditions that each look harmless on their own and always hands back a consistent, complete group judgment. A group's settling is a rule's settling, and which rule you pick decides which of the honest settlings sticks. I haven't tried to put a group on the machine, and that's where I'd expect the first wall to be.

Popper's swamp sent me further down, because it's the oldest problem in the book and it doesn't have a bottom. Ask why you believe something and you get a reason. Ask why you believe the reason and you get another one. Sextus Empiricus, a skeptic and doctor who lived probably in the second or third century and whose whole method was holding off judgment, wrote down the ancient version, credited to someone called Agrippa, and in 1968 the German philosopher Hans Albert, who with Popper led the school called critical rationalism, gave it a name with a joke in it: the Münchhausen trilemma, after the baron who claimed he pulled himself out of a swamp by his own hair. Every chain of reasons does one of three things. It goes on forever. It loops back on itself. Or it stops somewhere, because somebody decided it stops. People have argued for a very long time about which of the three to live with. So "when do I actually know, and when am I done checking?" doesn't get a clean answer from anybody. What you can do is be honest about where you stopped and why. That's a lot of what the rest of this book is: every place it stops is written down, with its name on it, so you can decide whether you'd have stopped there too.

Lay Peirce and Popper on top of each other and here's what shows through. The check is one act. The stopping is another. The check can run as long as you like and it never *makes* you stop; at some point something decides, and the deciding is its own event. That's the second machine's whole trick: it skipped the check and kept the answer.

And a logician said the part I didn't expect a logician to say. Per Martin-Löf, a Swedish logician whose type theory sits underneath the kind of proof checker that checks this book, says a proof is an act, and once it's done it's done. His words, from lectures he gave in Siena in 1983: "a proof is, not an object, but an act … Only secondarily, and irrevocably, does it become the act that has been performed." That took me a few reads, and the tenses unlocked it: first the act being done, then, for good, the act that has been done. You could say that's true of everything that ever happened. Fine. But it's the two computers again: the first machine's check happened, irrevocably, and the machine right now carries no trace of it. The past can't be undone. The *record* of it can be lost, unless something keeps one. That gap is where this book lives.

Then the intuitionists got there first, which by now I should've expected. The Dutch mathematician L. E. J. Brouwer, who held that a mathematical truth is some-

thing a mind makes evident, argued from 1948 on about an idealised mathematician working in stages, the creating subject. The people after him only came clear to me once I sorted them by three words: the act, the object, and the trace the act leaves behind, which Martin-Löf's sentence has no room for. The logician Georg Kreisel (1967), Austrian-born and then at Stanford, wrote it down as a formula: the subject has evidence for A at stage n. That's a trace, something you can come back and ask about after the act is over. Anne Troelstra (1969), a Dutch logician who studied under Brouwer's student Arend Heyting and became one of the leading authorities on intuitionism, added that the subject never forgets what it has made evident, so the trace never fades. That's how Mark van Atten (2018), a philosopher in Paris who has spent years on Brouwer, credits it; the Stanford Encyclopedia's entry on intuitionism lists the never-forgets axiom among Kreisel's own, and I haven't settled which is right. Van Atten, going through the whole history, reads the formula as "by stage n the Creating Subject has made A evident." Look at the tense. *Has made* is Martin-Löf's "the act that has been performed," reached from the other end. And look at what that carries. It's tied to a stage; at any stage you can ask whether it's happened yet; it never goes away; and it's never wrong. Van Atten even says the subject can't just accept a proof handed to it from outside: it has to go through the steps and rebuild the evidence itself. That's the copier who runs the check. And then there's Göran Sundholm, a philosopher of logic at Leiden. He has long kept a proof as an act apart from a proof as an object (1998), and, as I understand his work, apart from its trace as well. In a 2014 paper he worked through how Brouwer's theory of the creating subject sits beside the thesis that every effective procedure is a program, which is the side this book's machines live on. I know that second paper only by what it sets out to do, so I lean on it lightly.

Set their trace beside my record and the columns nearly all fill: their stage is my dated, their never forgetting my permanent, their never wrong my earned. Their stage is written into the trace, though, and my dated is only a step in the run that the latch doesn't keep. The empty one is the price, and I think I see why. For a mind with unlimited memory, never forgetting costs nothing. A finite machine pays to keep anything for good, and that's the chairs, still to come. So it's the closest thing to my *established* that I've found. The formula is around sixty years old, and the idea under it, Brouwer's creating subject, is older than that. Push on my side from theirs, though, and it gives twice. Van Atten's subject won't take a proof from outside, and my machine takes exactly one: its checker is trusted on a proof about the checker that somebody did outside every run. And their evidence is never wrong because, for Brouwer, made evident is what true means. Mine is right only as far as that checker is honest. Where they differ, the difference is honest: their subject is an idealised

mind with unlimited time and memory, and mine is a machine with a finite table that establishes only the claims its program checks.

Not a switch

So far I've been talking about *established* as if it were a light switch: on or off, the first computer on and the second one off. That bothered me the whole time I was writing it, and for a while I couldn't say why. It felt too small, the way a word can be technically right and still not cover what you're pointing at. I don't trust that feeling on its own, because I've had it plenty of times about things that turned out to be nothing. So I went looking for something outside my own head that would either give the feeling something to stand on or show me I was imagining it.

I didn't know what I was looking for, at first, beyond a rough shape: two things that look exactly the same and differ in how far something about them has been settled. So I started collecting anything that seemed to have that shape, and most of what I collected fell apart as soon as I looked at it closely. How each one fell apart taught me more than the thing itself did, so I'll go through them in the order it happened. Every story below is the version I could piece together from the records I could reach, and where I couldn't reach a record I'll say so. Check me.

The first batch was all hidden things. A pea under one of three cups. A tree, whose age you can't see from the outside. A forged painting that looks right until somebody tests the paint. A glass of gas, where the thermometer shows one temperature and underneath there are uncountably many arrangements of molecules that would all show the same reading. Plato's prisoners in the cave, who see shadows on a wall. The things casting them stay behind their backs (Section 7 tells their story). Every one of these looked promising for about a minute. Then I noticed they all fail the same way: look harder and the hidden thing shows up. Lift the cup. Cut the trunk and count the rings. Run the paint test. Look at the molecules, at least in principle. Turn around in the cave. In each case the hidden part is a fact about the object that better eyes could find. That isn't what the two computers have. You can see every lamp in the warehouse and not find the check.

That failure gave me the first real tool I had, and it's a crude one. Give an observer perfect vision of the visible thing, as much detail as they want, and ask whether they could read the difference off it. If they could, it's something hidden, and that's not what I'm after. If they couldn't, it might be. I want to be honest about how much weight that test can hold. It's a judgment, and I'm the one making it, with whatever blind spots I have. Somebody careful can look straight at the evidence and still read it wrong, because of what they expected to see. So I treat the test as a way of sorting

my own examples. It isn't a verdict, and if you find a case where it sorts wrongly, that tells me something about the test and about me.

The second batch got past that test and failed a different way. A clock face reads three o'clock and doesn't show how many days have gone by; no amount of staring at the face tells you. And a computer can keep a complete history of what it did. That one has a story behind it. In 1961 Rolf Landauer, a physicist at IBM, argued that a computer has to give off heat whenever it throws information away, and that real computers throw information away all the time. Twelve years later a younger colleague of his, Charles Bennett, asked whether they had to. His answer, in a 1973 paper, was no. A machine can write down every step it takes, copy out its answer, and then run itself backward, retracing its steps and wiping the history as it goes, until it holds only the input and the answer and has never thrown anything away. It costs about twice the steps and a lot of scratch paper, and that's all it costs. Bennett thanks Landauer at the end for raising the question in the first place, which I like: the man whose principle says forgetting costs is the one who asked whether you could get out of it. The history it keeps along the way is real extra information, sitting out where anyone could read it, so it isn't a hidden fact of the first batch's kind. And keeping it and unwinding it throws nothing away, so by Landauer's own reckoning it carries no heat floor; the steps and the scratch paper are the whole bill. It can also be run back down. Neither of those two, the keeping that carries no floor and the running back down, fits what happened to the first computer, which did something that can't be undone and paid for it in work. If the standing I'm after could be added for free and taken back for free, it would just be bookkeeping. So that batch taught me what else I needed: something that only moves one way, and costs something to move.

The third batch came closer and still didn't quite fit. In 1782 James Six, a Canterbury man from a family of silk weavers who had taken up watching the weather, described a thermometer for the Royal Society. He wanted the coldest moment of every night, and he'd found that with an ordinary thermometer you had to be looking at it when it happened, because, in his words, "if it be not immediately noticed, it can never after be known." His fix was two little steel markers in the tube that the column pushes up and that stay where they're left. That's a record that only moves one way. Then he spoils it, for my purposes, in the same paper: every evening he wrote the numbers down and slid the markers back with a magnet, "without a touch." A record anybody can reset with a magnet is not the thing I'm after, and Six would have been the first to say he wasn't the first to build one; his paper lists the thermometers that came before his.

Since 1996, starting with oral polio vaccine, vials of vaccine bought through UNICEF have carried a small sticker, a square inside a circle. The square darkens as

the vial takes on heat, and the darkening doesn't undo when the vial cools. When the square matches the circle, the vaccine shouldn't be used. One way, and it can't be reset. But it's chemistry. The reaction in the square is chosen to run about as fast as the vaccine itself spoils, so the sticker is really reading the vaccine's own condition by a proxy. Nobody ran a check; the heat did.

Closer still: the International Vocabulary of Metrology, the reference book that measurement laboratories work from, says a result is traceable only if it "can be related to a reference through a documented unbroken chain of calibrations, each contributing to the measurement uncertainty." That's a lot like what I want, a standing earned by checks, step by step. The same document adds that traceability "does not ensure . . . that there is an absence of mistakes," which is the true-versus-established split written into an international standard. And the top of the chain wasn't fixed either. When the official copies of the kilogram were compared with the prototype in 1889, 1946 and around 1991, they had drifted apart by tens of micrograms over the century, and nobody can say which one moved, because the prototype was the kilogram. In 2019 the kilogram was redefined by fixing a constant of nature, Planck's constant, instead. A chain like that can also lapse: if the calibrations stop, new results stop being traceable. The standing has to be kept up, which my record doesn't.

The rules of chess have the shape too. The current Laws of Chess of FIDE, the international chess federation, say in Article 9.2.3 that two positions are the same only if "the same player has the move, pieces of the same kind and colour occupy the same squares and the possible moves of all the pieces of both players are the same," and they spell out that a king that could still castle and one that can't are in different positions, even on an identical board. That passes the perfect-vision test. And the official text says why it passes: the castling right changes which moves are legal. So the hidden part steers the game: it decides what can happen next. In the machine I'm building, the record never steers. It never touches the ordinary computing at all. Each of these got something right and missed something else.

The one that fit best, and I want to say clearly that it only fit best among the ones I found, was the standing of a mathematical statement. Here's how it went for one statement, as far as I could follow it.

In 1611 Johannes Kepler, the emperor's mathematician in Prague, owed his friend and patron Johann Matthäus Wackher von Wackenfels a New Year's present and, he said, had nothing to give. Wackher, Kepler wrote, loved *nothing*, for the games you can play with the word, so Kepler wrote him a little pamphlet about the nearest thing to nothing he could find, a snowflake, which had landed on his coat as he crossed a bridge. Why six corners? Kepler couldn't say in the end. But along the way he stacked equal balls in his head and asserted that one way of stacking them, the way grocers stack oranges, packs them so tightly that, in the 1966 English translation,

"in no other arrangement could more pellets be stuffed into the same container." He didn't prove it. He said it.

Then the sentence sat. In 1831 Carl Friedrich Gauss, reviewing somebody else's book on a piece of number theory, proved what amounts to the special case of stackings laid out on a regular grid. In 1900 David Hilbert put the general question of packing equal spheres on his famous list of problems for the new century, without mentioning Kepler. In 1953 the Hungarian geometer László Fejes Tóth showed how the whole thing might be cut down to a problem with finitely many numbers in it, and a decade later he guessed that computers might one day finish it. Thomas Hales, a mathematician then at the University of Michigan, started on it in earnest in 1994, and in August 1998 he and his student Samuel Ferguson announced a proof that leaned on a great deal of computer work. The Annals of Mathematics gave the paper to a panel of twelve referees, led by Gábor Fejes Tóth, László's son. They ran seminars on it for years. A journalist, George Szpiro, later reported that by July 2002 they were 99 percent certain it was right; I couldn't find those words in anything the referees wrote themselves. What Szpiro printed in *Nature* in 2003 was the editor's letter as Hales reported it: the referees had "not been able to certify the correctness of the proof, and will not be able to certify it in the future, because they have run out of energy . . . " The Annals printed the human part in 2005, and the whole proof appeared in 2006. Hales, in his own words, "growing out of my frustration, in an effort to bypass the referees," started a project in 2003 to have the entire proof checked by a proof assistant, the same kind of tool that checks this book. It was finished on 10 August 2014.

Lay those dates out on the table and look at what moved. The sentence never changed. Hilbert's version of the question didn't even name the man who first said it. Whether it's true never changed, as far as anyone can tell. What changed was how far it had been established, and it changed in stages, each one paid for by somebody's work: a guess, a special case, a place on a list, a plan, an announcement, a panel's reported ninety-nine percent and their exhaustion, a printing, a machine check. A perfect reader of the sentence can't see which stage it's at. And it only ever moved one way.

Then I tried to break that one too, because by this point I'd watched a lot of good-looking examples fall over, and there's an obvious place to push. Standing can fall. In 1852 a student named Francis Guthrie, colouring a map of the counties of England, noticed that four colours always seemed to be enough to keep neighbouring counties different. The question passed to his old teacher Augustus De Morgan, who wrote to the Irish mathematician William Rowan Hamilton that a student had asked him for "a reason for a fact which I did not know was a fact," and that he didn't know it yet either. In 1879 Alfred Kempe, a barrister who did mathematics on the side and

sang counter-tenor, published a proof, and his description of finding it is wonderful: "After a somewhat arduous search, I have succeeded, suddenly, as might be expected, in hitting upon the weak point …" The journal's associate editor, William Story, added a note saying Kempe had "substantially proved" the theorem, and tightened the counting part of the argument. Eleven years later Percy Heawood, a lecturer at Durham who by one account brought his dog to his lectures, showed that one step of what he called "the now apparently recognised proof" doesn't work. Kempe swaps colours along two chains at once, and swapping the first can spoil the second.

So standing can fall, and I went back and forth on what that does to my picture. Where I've landed for now: the check that got Kempe's proof accepted, Story's note, went through the counting and never examined the double swap. A check that never looked at a step can't have said no to that step. In the sense of the previous few pages, the part that fell was never earned in the first place. I'm aware that's a convenient reading for me, and I'm wary of convenient readings, especially my own. Take it as where I ended up. It isn't a settled answer. It is, though, the reason the machine later in this book only lets its record go up after a passing, sound check and a matching commitment. It supports nontrivial claims whose checks can fail. It does not require every certified claim to be nontrivial.

And then the part I didn't expect. The four-colour statement was true all along. Kenneth Appel and Wolfgang Haken, two mathematicians at the University of Illinois, proved it in 1976 with a computer, and the proof was met with suspicion for years; a cleaner version came in the 1990s; and in 2005 Georges Gonthier, a computer scientist at Microsoft Research in Cambridge, finished a proof of the whole thing checked by Coq, the program that checks this book. So here's a statement that stayed true the entire time while its standing went up in 1879, came down in 1890, went up again in 1976 against a lot of doubt, and went up further in 2005. The fall wasn't a fall of the truth. It was a fall of how far the truth had been established, and it was the right fall to have, because what had been established was less than anyone thought.

Laying all of that out together is what changed my picture. If there are stages, then at each one there's a line, and the line has two sides: not this far yet, and at least this far. Each line amounts to its own yes-or-no question, and the answer to each can only ever change from no to yes, so long as the check behind it could have failed. Maybe the switch wasn't wrong. Maybe it was just incomplete, the first of many lines, and the real shape is a direction with lines drawn across it. I've gone around many times on whether the lines create the direction or the direction comes first and the lines are just places where you cut it, and on whether yes and no are even different from each other until something draws the line between them. I don't have an answer to that. The one part I could actually settle is that a single switch isn't enough once

there are three or more stages, and that is proved (Section 11). Everything else in this paragraph is me thinking out loud.

Flatland, where this hunch started (Section 1), turned out to belong in the first batch. It's a novel Edwin Abbott published in 1884, narrated by a square who lives in a flat, two-dimensional world (Section 7 says more about him and it). When a sphere passes through the flat world, the narrator sees a line whose ends fade wrong and whose length keeps changing, and only when he feels it does he find a circle, a perfect one. Then the sphere tells him it will rise, and he watches it dwindle to a point and vanish, and come back larger and larger. The hidden direction leaks into his view, faintly, and a sharp enough flat observer could in principle work out where the sphere is. In this machine, as far as I can make out, you could move a state all the way along the established direction without changing a single bit of the ordinary machine. That may be exactly why the ordinary machine can't see it. It may also be why, if the direction is worth having, someone has to build it in on purpose and keep it on purpose. Whether keeping it costs anything is what the rest of this chapter tries to find out.

None of these people were writing about what I'm writing about, and none of them is the reason any of this is true or false. Reading them didn't make anything truer. It raised how far I'd established a few things for myself, and I could still be wrong about every one, the way the people who read Kempe in 1880 were. I went through them because I needed something to test the feeling against, and most of what I tested failed. The failures told me what the thing isn't: not hidden, not free, not reversible, not steering. What's left after all that is mostly a list of what it isn't, and I'm still not certain I've described what it is.

Computing already had a word for it

Here's the slightly embarrassing part. Computing already has a word for this and uses it every day. Anyone who's used a database has seen it: COMMIT.

A database does a pile of tentative work, and then there's one moment, the commit, written to a log, after which the change counts. Jim Gray, who more or less built the theory behind that, opens his 1981 paper on it by saying the idea comes from contract law, and his example is a wedding. People negotiate, somebody asks, two people say "I do," and from then on it's binding. If it was a mistake, you don't erase it. You do another transaction to fix it, the way a bad contract gets a second contract. Nobody reaches for a time machine.

The wedding was a philosopher's example first. The Oxford philosopher J. L. Austin, lecturing in 1955, said "I do" *makes* a marriage. It doesn't *describe* one, and sayings like that aren't true or false. They can misfire (the wrong person says it, a

step gets skipped). They can't be false. That's a commit exactly. Writing the commit record doesn't report that a question got closed. Writing it *is* the closing. And a commit can't be false. It can misfire: committed without the check, committed about the wrong pocket. Gettier and Peirce again, from a third direction. Then somebody else reads the record, and for them it stops being a performance and turns into a report: "this was checked and closed." A report can be false. On the small machine in Part IV it can't be, about the machine's own history: if its flag is up from a clean start, some claim was checked and closed, and that claim held of that edition when it was checked. That's the one place in the book where a commit and the truth get tied together, and it took a proof to do it.

Robin Milner, building a proof checker in the 1970s, ran out of memory, so he decided to keep only the finished theorems and throw the proofs away, like a teacher rubbing out the blackboard to make room. The obvious worry is forgery: if the proofs are gone, what stops someone just writing a theorem down? His answer was beautiful. Theorems got their own type in the programming language, and the only way to make one was through the rules of logic. You couldn't forge one because the language wouldn't let you build it any other way. He named that language ML, for Meta Language. I wanted to call my machine the meta machine. I'm choosing to treat that as a coincidence. Calling it a sign would be more my style, so this is more restraint than I usually manage.

Bitcoin is the loud modern version. Its records can't be changed without redoing all the work that came after them, and a computer that drops off the network and comes back trusts the longest chain as the story of what happened while it was gone. That's Turing's clerk again, coming back to a note. The difference is that the clerk's note costs nothing to forge and Bitcoin's costs a mountain of work. It's the clearest case I know of somebody making a settled fact expensive to fake, and they got there by paying for a search. The settling itself carries no price.

That's a different job from the one this book's price does, the mark it charges for raising a record. I'll name that price the toll when I state it, a few pages on, and I'd rather say this now, before you read the toll as a deterrent. One mark deters nobody. Bitcoin's work is tied to something scarce, electricity and hardware; a mark in this book is tied to nothing. Eric Budish, an economist at the University of Chicago, built a model in 2025 of what that tie has to cost. As I read him, for Bitcoin's kind of trust to hold, the honest work has to stay expensive at every moment compared with what an attacker could win, and the bill grows in step with the value being protected. A flat mark can't do that job, and it isn't trying to. What makes a raised record mean something, on the machine where it means anything, is an order rule: check, then commit, then certify. That rule would work just as well at a price of zero. The price is there because, on the premises of the chairs a little further on, a finite machine that

prices its merges can't avoid it. Scaring anybody off has nothing to do with it. The machines I actually build don't meet those premises, so on them the price is a choice made to match (Section 16 says why).

And Bitcoin wasn't first. Pull on its footnotes: three of the eight references in the paper that started it point to Stuart Haber and Scott Stornetta, who in 1991 worked out how to stamp a document with a time nobody could fake later, by chaining each stamp to the ones before it. A company built on their idea, Surety, printed a short fingerprint of its chain in the classified ads of the New York Times once a week from 1995, and it was still doing it in 2018. I found that in a piece the journalist Daniel Oberhaus wrote that year, which says the ad runs under "Notices & Lost and Found." Think about that for a second. To make a record nobody could quietly rewrite, they put it somewhere a million people would keep a copy of without even meaning to. A physicist named Wojciech Zurek has spent decades arguing that the world does the same thing with a measurement, copying the result into so many places that nobody can take it back. I'll get to him properly near the end of Part III.

There are more fences like these: logs that can't quietly change, certificates you can check. Later in the book I take five of them apart. I lean on all of them. They work. I want that said plainly, because the next thing isn't "they're broken."

The next thing is where they sit. The database's commit is bits in a log, kept by a database program, on a machine whose own step will overwrite those bits the moment some instruction says to. Milner's guarantee is a type checker; it holds inside programs written in his language and nowhere else. Bitcoin is a protocol on ordinary computers that will do whatever they're told. Every one of them is a program or a rule, on top of a machine that can't tell my two computers apart. The machine underneath keeps nothing and charges nothing for what got settled.

The obvious comeback: so put the rule one level lower, in the operating system, or what security people call a reference monitor, the small always-on piece everything has to pass through. Yes. Exactly that. A monitor that wouldn't let any program raise its record without paying would be doing the thing I'm talking about. In this book's words it would be *implementing* the model, and I'd count that as agreement. What I care about isn't who builds the fence. It's what the fence has to obey, wherever it's built, and whether that's forced or just a good idea.

That comeback has a history, and it's older than me by a lot. In 1972 James P. Anderson's planning study for the US Air Force described a reference monitor: a piece every access has to pass through, that can't be tampered with, and small enough to check. In 2000 Fred Schneider, a computer scientist at Cornell, argued that a monitor that only watches a run as it goes can enforce only rules whose breaking shows up at some finite point, the kind computer scientists call safety rules. "No certificate without an earlier passing check and a commit" is one of those. And in

2015 Arthur Azevedo de Amorim, a computer scientist then at the University of Pennsylvania, and six coauthors built monitors that check a rule at every single instruction, and proved them correct in Coq, the same proof checker this book uses. So putting a rule under every program, so nothing can get around it, is their move. I didn't invent it. What I'm putting under that move is narrower: a record that has to be paid for, on the small machine one that has to be earned, and a reason, from counting, why a finite machine that prices its merges can't make the price of a record that never comes down zero.

What settling looks like

The chapter on what structure is (Section 2) ends with a pile of things sorted into buckets. Settling a question is that, plus one more thing.

Picture every state the machine could be in as a field of dots. A yes-or-no question about the state splits the field into two rooms: yes and no. That's a sorting, the smallest one there is. And remember the no pile: if one of the rooms is empty, there's no sorting at all, just a field. Now the extra thing that makes it a settlement: once the machine is in the yes room, *no step it can take leads back out*. The door opens one way. A settled question is a sorting plus a door that only swings in.

At its best, what goes through the door is a relation: these numbers are in this order, these two things belong together. There's a chimpanzee named Sultan, later in the book, who fitted two short sticks into one long one and reached the food. Nothing new got added to the cage. Two sticks became one tool. That's what I'd want the door to be for.

But the door doesn't care, and this is the honest edge of the whole picture. Hook the door to a check that says yes to everything and the machine walks through, settled, and wrong. Settled whether true or false, like Peirce said. A door that only swings one way tells you a question was closed. It doesn't tell you it was closed right. Price and correctness are two different books, and I keep both, separately.

One more edge, the one Gray's wedding already hinted at. What stays shut for good is that the question got closed, about that edition of that thing. Whether the claim is still true is another matter. Change the list after the commit and the record still says, truthfully, that the old edition was checked and closed. And if the claim turns out bad, the fix is another act on top, a second record that says "withdrawn." The first one never quietly comes back down. So "entitled to lean on it right now" is three things at once: closed, not withdrawn since, and the thing unchanged.

Why it has to cost

This is the part I'm proudest of, because once you've picked a record that stays, the choosing stops and the counting takes over. And the counting is chairs.

Take any machine with a finite number of states. Its step is a function: give it a state and a move, it gives you back one state. Pick a yes-or-no reading of the state and make it permanent: no step ever turns a yes back into a no. Now suppose some step takes a state that reads no and lands it on one that reads yes. Then that step has to land two different states on one. It *merges*.

Do it with chairs. Draw a box on the floor, three chairs inside, one chair outside by the door. Four people, one per chair: the four states the machine could be in. The three inside already read yes. One step moves everybody at once, each to the chair the rule sends them to. Nobody inside may leave, because yes is permanent. The person by the door has to come in, because this is the step that flips them to yes. Now finish the move, one person per chair. You can't. Four people, three inside chairs: somebody shares. Maybe the newcomer ends up on a lap, maybe two insiders squeeze onto one chair; either way, two people end on one chair, and from where everyone's sitting you can't tell which two. That's the pigeonhole principle, which is old and not mine, and the checked version, with exactly those three conditions (finite, permanent, one flip), is in the chapter "Why the toll isn't arbitrary." No physics in it anywhere. Counting.

(How old is it? I went looking. A French Jesuit, Jean Leurechon, used it in 1622 to argue that two people somewhere must have exactly the same number of hairs on their heads, which is the kind of thing you can be completely sure of and never check. The first name for it I can find came two centuries later, from the German mathematician Peter Gustav Lejeune Dirichlet, who used it in print in 1842 without naming it and in later work called it the drawer principle, the *Schubfachprinzip*; the pigeons came later still. Benoît Rittaud and Albrecht Heeffer, who wrote a short paper in 2014 on what came before Dirichlet, looked for a source for the year 1834 that turns up next to his name and found none, and neither have I. Is 1622 the bottom? I can't tell you. History is like that: it's where you start digging.)

Turn it around and it gets better. A *reversible* computer, one where every step can be run backwards, never merges anything, because a step you can undo never puts two people on one chair. Bennett, the same Bennett, showed in 1973 that any computation can be done reversibly if you give it room to keep its history while it works. But give a reversible computer a finite memory and, however much it computes, it can never raise a flag that stays up. It can always back up to before the flag. Settling needs exactly the kind of step a reversible computer refuses to take. (Where does the merged state go, physically? Out of the machine and into the room,

as heat; the machine and the room together never merge. Section 4 comes back to that.)

Now the physics, as an assumption with its name on it. Rolf Landauer, at IBM, 1961, said that a step that can't be run backwards, that merges, "requires a minimal heat generation." That's the rule people call Landauer's principle. On a finite machine, settling something for good takes a merge, and if Landauer's right, a merge costs heat whenever the machine doesn't already know which of the merged states it was in. So on that assumption, a device that has to settle things correctly without knowing its own state ahead of time can't do it for free.

And here's his next sentence, the one I promised at the start. That heat, he wrote, "serves the purpose of standardizing signals and making them independent of their exact logical history." That's my two computers, in 1961. Ordinary machines spend their merges, and their heat, on *forgetting how the bits got there*, on purpose, so a 1 is a 1 no matter what path made it. That's why the sorter and the copier end up identical, and why the comparing wing goes dark. And the very same kind of step, a merge, is what it takes to close a door for good.

So here's the deepest thing I can say about settling. Not every forgetting settles anything: wiping a scratch pad closes no question. But on a finite machine, every settling forgets. The step that closes a question for good lands two states on one, somewhere, maybe not where you happen to be standing, and afterward nothing can tell which one it was. A verdict keeps the decision and lets go of the arguing. An ordinary computer pays for the forgetting and keeps nothing to show for it. What I want to know is what a machine looks like that pays for the same forgetting and keeps the receipt.

Every assumption in that is a way out, and you should know where they are. Give the machine unlimited memory and it never has to merge; it can write its history down forever, and the chapter "Why the toll isn't arbitrary" builds exactly that machine. Let the flag come back down and the chairs lose their grip. And the heat isn't owed by every run. Landauer's principle, in the general form Part II adopts, charges for what you didn't know, so for one run from a known start, with its controls timed from outside, the premise supplies no positive heat floor (which doesn't show the device gives off none); the floor bites for a device that has to cope with states it can't tell apart, which is how devices get built (Theorem 4.13). The toll still charges its mark on that run, because the toll is a rule about the move, and the move doesn't know who's watching. And Landauer's principle is physics. No proof in this book reaches it. In 2012 a group in Lyon erased one bit held in a tiny bead, kept in place by a laser, and found the average heat it gave off coming down toward Landauer's floor as the erasing was slowed. The principle has serious critics too, the

philosopher John Norton among them. So the counting is proved and the price is an assumption, and I never let those two swap places.

Somebody got there first, goddamn it

Bennett also says looking can be free. His famous move with Maxwell's demon (the whole story's later in the book) is that you can write down which side of a box a molecule is on without paying anything, as long as you write on a blank page. What costs is wiping the page to make room for the next one.

I'd been sitting on this idea that *committing* is what costs, and learning doesn't, and I thought: this one's mine, I finally had an original thought. Then I found Carlo Rovelli's 2024 paper, "The Thermodynamic Cost of Choosing." Nope. Somebody got there first, goddamn it. A real physicist, with a real paper, saying that making a choice that sticks costs at least the same heat Landauer charges for erasing a bit. He even quotes a German saying, *Wahl ist Leiden*, to choose is to suffer, and at that point what are you going to do but laugh.

And then, taking him apart properly, it got good, because he seems to contradict Bennett. Bennett says measuring can be free; Rovelli says any measurement costs. So which is it? Look at what Rovelli calls a choice. A coin in the air hasn't chosen anything; it has to hit the ground and stop. And then the sentence I'd put on my wall: "stabilization must be irreversible, otherwise it is not stabilization." Bennett's free measurement is one that could still be undone; that's *why* it's free. Rovelli's choice is one that's been made to stay; that's why it costs. The way I read them, they're standing on opposite sides of the same door: learning on Bennett's side, committing on Rovelli's. (Bennett's demon still pays in the end, when it wipes the page.) Neither of them puts it that way, so push on it. But if it holds, it's the best sign I've got that the line I keep drawing between knowing and settling isn't something I made up.

Push on it, then, with his paper open, and it gives in four places: his choice is unpredictable, it's a branching where mine is a merge, it's stable only for a while, and he stands against Bennett's free measurement, where I'd have put him beside it (Section 5 goes through all four). So the two-sides-of-one-door picture is my reading. He doesn't draw it, and the piece of it that's really his is the one sentence about stabilizing. That sentence I'll still put on my wall.

So he's my closest neighbour, and here's the difference, exactly. His is physics: a price in heat, at a temperature, for a choice that sticks. Mine is a rule in a model of computation: a charge in the machine's own count, on the step that flips a permanent reading on, with no temperature anywhere in it. Same instinct, different place. I'll take it. It's annoying to be second. It's worse to be alone and wrong. And he's the

closest only on choosing. On copying there's an older neighbour, in the physics of molecules, and Section 5 gets to it.

What the Thiele Machine is

So here's what I built, and which parts I was pushed into and which parts I chose, because you should know which is which.

The Thiele Machine is a model of computation, the way Turing's machine is: math you can reason about. You can't buy it in a box. It isn't a device, a CPU or an instruction set; those are things you build to carry it, and Part IV builds one, as small as I could make it. Unlike a Turing machine, it computes nothing new. It adds what a computation *establishes*, and what establishing it cost. The event it accounts for happens on any computer, whether anybody keeps track of it or not, and the record exists only if something keeps it. It has a few parts. A state. A step that moves the state. A record: a yes-or-no reading of the state that says whether something's been established. A charge on each kind of move, added up on a counter that only climbs. One rule, which I call the toll: the step that flips the record from no to yes has to cost at least one mark on that counter. And a window, what an outside observer gets to see, which is there because the two computers already showed you the problem: the usual window, the bits, can't see the record. The toll's first name, the one I gave it before I understood it, is No Free Insight.

One what, and why one? One mark, in the model's own count. One because it's the least a whole-number count can charge and still not be free, and the toll only says *not free*. The model doesn't say a mark is worth a joule or a dollar; whether it's worth anything physical is Landauer's assumption, named, and nowhere else. And one is just the floor: Part II shows that if you charge one mark per halving, a step that squeezes a lot together pays at least the logarithm of the squeeze, roughly how many times it halves the number of states it can tell apart. Don't read that backwards, though. One mark isn't one halving. A single state flipping into a big room of states that already read yes, with every state in play equally likely, squeezes things together very little, much less than a halving, and the whole-number count rounds that up to one. Read in the worst case over what the machine might be holding, the same flip can remove a full bit; Section 4 has both readings.

What "new model of computation" means here. It adds something to how a machine runs: structure it keeps, and an account of what that structure cost. It doesn't add computable functions. It's an abstract model the way the Turing machine is, but it doesn't hand you a new primitive to compute with; whatever base machine sits under it still does all the computing. When I say "shadow" I mean the particular window I name. Other ways a conventional machine could store things aren't

covered. Enforcing the law, storing it, and letting an observer see it are three separate questions, and the book keeps them separate.

Which leaves a fair question hanging: if it computes nothing new, does it even get to call itself a model of computation? I think so, and not only on my say-so. Computer science has long kept models that compute exactly what Turing's does and differ only in what they count, how long a step takes, what reaching into memory costs, because the counting is the reason to have them. The random-access machine of the computer scientists Stephen Cook and Robert Reckhow (1973) charges each instruction one unit, or a cost that grows with the logarithm of the numbers it handles. The cell-probe model of Andrew Yao, a computer scientist who later won the Turing Award (1981), counts only the memory cells a program reads. The input-output model of the computer scientists Alok Aggarwal and Jeffrey Vitter (1988) counts only the blocks moved between a small fast memory and a large slow one. The bulk-synchronous parallel model of Leslie Valiant, a computer scientist at Harvard (1990), charges for rounds of communication between processors. Closest in kind are the two axioms the computer scientist Manuel Blum gave in 1967 for any measure of computational cost: they name no machine, and say what every cost has to satisfy, the way the toll is a rule every price list on the model has to obey. That's the family the Thiele Machine belongs to, with an axis of its own: what a run established, and what that cost. Part III lays the toll over a register machine of the first kind, and over a Turing machine and a lambda calculus, and none of them computes one function more afterward (Section 10). What each of them gains is a record and a bill.

What was I pushed into? That on a finite machine, a record that never comes down can only be raised by a merge: counting, proved. That merges cost: physics, assumed. What did I choose? To put the count *inside the step*, as part of what a step is. Nothing forces that, and a log off to the side was always the other place it could go. It's my claim about where the account belongs, and the rest of the book is the case for it.

About the name. I named it after myself, on purpose. Half of that is a joke and half of it is a stake in the ground. Turing got a machine, so here's mine. The longer story, meta machines and all, is in "How I got here" (Section 1).

The names work the way Turing's do, and I'm going to be a little annoying about them. *The Thiele Machine*, capital letters, is the model and nothing else. *A Thiele machine*, lower case, is any particular system proved to fit the model, the way one rule table is a Turing machine; a two-box toy on a sheet of paper counts. A Thiele machine is *Thiele-complete* when the machine under the record can compute anything a Turing machine can (under Minsky's 2^n encoding, done outside the machine), for free and without ever touching the record; a record that goes up stays up; and, from a clean start, the record goes up only after a check of some claim passes, by a checker

proved to mean what the claim says, in a language of claims and meanings fixed before any checker is named, then a commitment to that same claim with nothing it's about changed in between, then a certificate, with exactly one mark charged for each of those three acts; and at least one claim on the machine has a check, commit and certificate that raise the record on some inputs and leave it down on others. And a *universal* Thiele machine is one fixed program that runs other machines as guests, with each guest's record earned again by the host's own steps and each guest's toll paid on the host's own ledger. Nothing is left to the guest's good manners.

The first bar, being a Thiele machine at all, is low, and I'd rather you hear it from me. A machine that charges one mark for every step it takes, whatever the step does, obeys the toll for any record you like; a clock is enough to make something a Thiele machine. If Thiele-complete only meant a Thiele machine that can compute anything, a clock bolted to a computer would pass that too, and so would a machine whose record never goes up. That's why Thiele-complete asks for more, and both of those fail it, by theorem: a clock charges for computing, and Thiele-complete says computing is free; and a machine whose record never goes up fails the clause asking for at least one claim whose check, commit and certificate raise the record on some inputs and leave it down on others (Section 16). The small machine of Part IV meets it, and so does the same machine over any properties with an exact test for telling two of them apart and a checker proved to mean what it says, as long as one of them is true of some number and false of another, sorted lists included. And every machine that meets it has the same blind spot built in, also by theorem: the part a Turing machine would call the whole machine, the line it's on and the numbers it holds, can't tell you its record or its ledger. That's about an observer who sees only where a run ended. One who also has the program and the start can work both out.

The two computers, put back together

Here's the two-computer story again, rebuilt as Thiele machines, with the record and the counter written out, so you can watch it happen. You don't have to take my word. The prices are the ones on the smallest machine I build later: one mark each to check, to commit, and to certify (raise the record), and nothing for ordinary work like moving numbers around.

step	**sorter does**	**record**	**counter**	**copier does**	**record**	**counter**
start	nothing yet	no	0	nothing yet	no	0

step	sorter does	record	counter	copier does	record	counter
1	sorts (lots of ordinary steps)	no	0	copies in the typed list	no	0
2	checks "sorted" on this list, this edition: passes	no	1	(no check)	no	0
3	commits to "sorted," about this list, this edition	no	2	tries to commit: nothing checked, refused	no	1
4	certifies	**yes**	3	tries to certify: nothing committed, refused	no	2

At the end, the bits on both machines are still the same sorted list. Everything else differs. The sorter's record is up and its counter reads 3. The copier's record is down, and it's paid 2 for two refusals, because on that machine a refused attempt still costs what it costs. (Read those as two separate attempts. Written as one stored program, the copier would stop at its first refusal and pay 1.) Nothing the copier can do raises its record without first running a passing check of the claim it then commits to. And no outside window that only shows the list can tell you which is which; the difference is all in the part the window doesn't show. Walk that warehouse now and there's finally a lamp that's lit in one and dark in the other.

Now how literally to take that table. It's a picture of the chain, and the chain is what's proved. The small machine's own checks are three questions about one counter: is it zero, is it even, is it at least some number. What its proofs need from the questions is a checker proved to say yes exactly when the answer is yes, and an exact test for telling two questions apart. That's enough for "sorted": a version of the machine that takes any questions with checkers like that proves the same chain once

for all of them, and with a list packed into one counter it certifies a sorted list for three marks and refuses an unsorted one forever (Section 16). Without a proved checker for "sorted," the chain would be honest about a counter and only by convention about your list, which is Gettier's pocket again, one level down.

And a lamp for whom? The record is bits too. It's a lamp on the machine, for anybody walking that warehouse. Hand somebody a printout of it across a desk and you're back at the sticker on the windshield: they have to trust whoever hands it over, or have some way to check the box that says it ran. That crossing is a different problem with its own tools, signatures, attestation and proofs a stranger can check, and this book doesn't solve it. Section 7 says what those tools rest on, and the small machine's record, like any record kept in a machine's state, carries nobody's signature. What the step changes is the question, from "did it run?" to "do I trust the box that says it ran?" That's a smaller question. It isn't no question.

Now the honest half. Take away the small machine's rules and keep only the bare toll, and the copier has an easier move: pay one mark, flip the record to yes, done. The toll is satisfied. The receipt is real. It's a receipt for nothing. The bare toll prices the closing of the door. The check behind it goes unpriced.

What putting it in the step buys

So here's the hardest objection, and I'd rather make it myself. You defined a flag and a charge, then proved the charge gets paid when the flag goes up. That's your definitions agreeing with each other.

On the bare model, that's fair, and the copier's receipt for nothing is the proof. What makes the record mean *established* is the small machine's step, which won't let a commit through unless a check of that very claim passed, meeting the first three of the four conditions from Gettier's pocket, with the fourth, an honest checker, proved about the checker separately. And what it proves isn't about one program. It's this: on *every* list of instructions anybody can write, starting clean, if the record is up, then somewhere earlier a check of some claim, about some edition, passed; then a commit of that same claim; then the certificate; with nothing touching the thing between the check and the commit.

From a clean start, a raised flag guarantees that some claim went through a passing check, a matching commitment about the same edition, and a successful certification. The flag retains that historical fact. The channel names the latest earned commitment, which need not be the commitment on which the flag first rose. The final state does not, in general, identify the claim certified at that first rise or the step at which it occurred. And the flag doesn't say the claim still holds, at the certification or at the end, since the counter can change after the commitment.

That "every" is the whole answer to what the step buys. A log kept by a program gives you a fact about that program: if the program's right, the log's right. The database's guarantee holds for work that goes through the database. A rule in the step gives you a fact about every program the machine can run, the buggy ones and the hostile ones too, because there's no way to run anything on it that isn't a string of its steps. That's what moving the rule down a level changes. Not what can be computed. *Which* programs the guarantee covers: this one becomes every one. Most of the rest of the book is how far that "every" goes and where it stops.

Two limits on that "every," said now so you don't have to find them later. First, it's every program. It isn't every everything. The guarantee covers any list of instructions, run from a clean start, one step at a time, on a step that does what its definition says. Hostile programs, yes. A hostile start (a table of facts nobody checked, a reset), hostile hardware (a soldering iron on the flag's bit, a second writer the step can't see) or a hostile reader, no; Section 25 lists the whole fence. That's the same shape as the database's guarantee, true if the layer underneath is faithful, with a much bigger class of programs inside it.

Second, the flag and the counter on their own are what people who verify programs call ghost state: erase them and every program computes exactly what it did before (Section 6 has where that idea comes from). What isn't ghost on the small machine is the table of checked facts, which the step reads and refuses on, at run time, for every program. And the move of putting a rule under every program isn't mine either. It's the reference monitor's, and its history is a few sections back.

What it isn't

It isn't more computing power. Anything a Thiele machine computes, the machine under its record computes without it. The book proves that over any machine at all, a base-driven record is a switch the machine's own events flip once and nothing flips back; it watches the computing and adds nothing to it. That's a sentence about the record. The checker belongs to the machine under it, and the definition of Thiele-complete doesn't ask the checker to be a program. Hand it a checker that knows which programs halt, and whether a CHECK of "this program halts" traps tells you whether it does, so that machine decides halting (Section 16 has the details). With a checker that is a program, as every checker in this book is, nothing in the machine goes past what a program can do. The record makes nothing faster either: erase the flag and the counter, and every run takes the same steps to the same places. CHECK is another matter. It's a test at unit cost, and one CHECK of "$A \geq m$" settles whether $A \geq m$ in one step, where the two counters alone would count A down. It isn't quantum. It says nothing about which problems are hard.

And an ordinary computer can keep a record like this. Give a Turing machine a spot for the record and a track for the counter, write its rules so it never sets the spot without bumping the counter, prove that, and it *is* a Thiele machine. It's no counterexample (Section 16.16). What an ordinary programmable computer, as built, doesn't do is make its own step enforce a record *for the programs it runs*. When it runs another program, that program's record is just data in memory the program can write, and a buggy program can flip it for free, because the host's step has no opinion about it. Could it be built otherwise? Sure. A universal machine with one fixed table, the kind Turing built, could have that table written so it mirrors each guest's record and charges for the raise, and it would qualify. Nothing about ordinary computers stops that. They just aren't built that way. That gap, between the machine doing the stepping and the program it runs, is where the databases and the proof checkers and Bitcoin built their fences, and it's the gap the universal Thiele machine later in the book closes, first for the programs of the small machine it's built on, and then for every Thiele machine whose workings a program can compute.

There's a cheap version of a universal machine, so here's which one this is. A host with a move that means "run one guest step" runs any guest because I wrote it that way, and the book has one of those, as the simple picture. The universal Thiele machine doesn't get that move. It's one fixed program, written in the small machine's own six kinds of instruction on a host with more counters, and it fetches, decodes and carries out each guest instruction itself, the way Turing's universal machine does. The theorems are about that program: it halts exactly when its guest halts, with the guest's answer in its counters; its flag goes up at some step of its run exactly when the guest's goes up at some step of the guest's, by its own check, its own commit and its own certificate; its ledger equals the guest's at every point where the two runs match; and the machine it runs on is Thiele-complete (Section 16.13). Its guests are the small machine's programs, from every clean start, and those programs compute everything. Add one move that pays a mark and does nothing else, and the same kind of program runs every Thiele machine whose next move, step, prices and reading are computed by programs: it halts when the guest's run stops, its flag goes up at some point exactly when the guest's reading is yes at some state of its run, on a chain whose check is the guest's own reading, and its ledger is the guest's prices plus at most two marks at the raise when the guest's reading starts at no (when it starts at yes, the host has to raise its flag at once and pays three on a ledger of nothing), a surcharge no program on that host, run from a clean start, can avoid when the guest's whole ledger at the raise is below three (Section 16.14). A Thiele machine whose reading no program decides isn't covered, and I say exactly where the edges are.

What's proved, what's argued, and what I just believe

Every claim in this book has a status and I'll tell you which, every time. Here's the whole list at once, so you know what I'm on the hook for.

Checked by a proof assistant, with every assumption printed, for any system with the named parts: anything that obeys the toll pays at least one mark to get from not-established to established (short enough that you'll prove it yourself on paper in Part II). With finitely many yes-states and a record that never comes down, every move that raises it merges states, which is the chairs; consequently every schedule that prices merges charges that move, and every run from no to yes has positive total cost. The price is an assumption with a name. Finding out can be free: you can learn something from a step that merged nothing and cost nothing, so the toll lands on committing. A window that shows a certifying step and a quiet one alike, before and after, can't price certification exactly, however clever it is. And over any machine, a base-driven record is a one-way switch on an event, with two choices left free: the price list and which event.

On the smallest machine I could build, six instruction kinds on two counters: the chain in the table above (check, commit, certify, same claim, same edition), for every program anybody can write, that one included, with claims about any property that comes with a checker proved to mean it and an exact test for telling properties apart, "sorted" included (Section 16). From a clean start, a raised record always went through a passing check, then a commit, then a certificate, of exactly the same claim about exactly the same edition; nothing gets into its list of checked facts without a check writing it. Every run from a clean start that ends with the flag raised has incurred at least three marks of instruction cost, and this bound is attained: check "this counter is zero," commit to it, certify, and when the counter is zero that pays exactly three; when it isn't, the check traps with the flag down and nothing after that ever raises it. The machine doesn't stop you asking a question that can't come out no, and I say so where it matters. It's Thiele-complete in the strong sense from a few pages back, and that sense has teeth: every clock-style machine, one that charges for computing, fails it, and every machine that meets it hides its record and its ledger from its own window, both by theorem. Its computing is that of the two-counter machines of Marvin Minsky, one of the founders of artificial intelligence at MIT, which Minsky proved can compute anything a Turing machine can, under his 2^n encoding, done outside the machine. The two universal programs described under "What it isn't" run on top of it, and what they're proved to do is said there.

Argued only: that an account of computation *should* carry what was established, and what it cost, in how it steps. The proofs show that it hangs together, what it follows from, and what it costs (six instruction kinds and three marks, on the small

machine); whether it ought to be adopted is argued, and it's a claim about what matters, which no proof checker can check.

Guessed at, with the ways to kill it written down: that the event worth recording is certification, because its records are the ones that get copied and relied on all over the place by parties who didn't produce them (the pointer thesis, Part III).

Not proved, and I'll say so every time it's tempting: any physical price for one mark, anything about a physical device running any of this, or anything about which problems are hard.

Which of the proved part is new? Not one big thing, and I'd be suspicious of me if I said otherwise. By my own sorting at the back (Appendix C) it's four smaller ones, each with its neighbours standing right there. The toll derived from finiteness, permanence and a price on merging: the pigeonhole principle, Landauer and Bennett on merging, and the finite heart of the German mathematician Ernst Zermelo's 1896 recurrence argument, about whether a closed box of gas has to come back around to where it started, are all next to it. A window that shows a certifying step and a quiet one alike, before and after, can't price certification exactly: an old fact about functions, the 1956 thought experiments of Edward Moore, at Bell Labs, on what you can learn about a machine you may only watch from outside, and the engineer Rudolf Kalman's observability, the same question asked of control systems. What I put in is a price. A base-driven permanent record, over any machine, is a latch on one event: the 1918 flip-flop and write-once memory are its neighbours. And the small machine, a definition of Thiele-complete that a clock can't meet, and a universal Thiele machine written as one fixed program: a record that's priced, and earned, under the reference monitor's every-program move (Anderson in 1972, Schneider in 2000, Coq-checked monitors from Azevedo de Amorim and coauthors in 2015), carried to every program of the small machine and, with one paid move added, to every computably presented Thiele machine. That fourth one is built and checked; the idea under it isn't new. The first three are small exact statements I didn't find stated before, where I looked. The two-player game check later in the book is careful engineering on known mathematics. It doesn't make a fifth.

At the bottom: what I suspect and can't prove

This is underneath the proofs, so read it with the lights on. None of it is a theorem, and I'm not going to dress it up as one. It's why I care, and I'm telling you going in, so you don't catch me sneaking it in later.

Physics has a word for the moment a question closes: *measurement*. And the people who thought hardest about it keep describing it in this chapter's shape. They ran together for me until I put them on one picture. Flip a coin in a crowded room.

Before it lands, it could go either way. After, it's settled, somebody chalks it on a board, and the result gets copied out into the world, the light and the air, so many times that it can't be taken back and everybody who looks agrees. Five pieces: the question, the landing, the chalk mark, the copies, the agreeing. Each person leans on a different one. Wojciech Zurek has spent decades on how the copying works. Rovelli's coin has to hit the ground, so his piece is the landing.

Niels Bohr, the Danish physicist whose institute in Copenhagen was where quantum mechanics got argued into shape, leans on the mark. In a 1954 talk he said seeing anything at that scale depends on an apparatus that works irreversibly, like permanent marks on a photographic plate: a door that swings one way. I found that line quoted in a recent paper and haven't seen the page in Bohr's book myself. John Wheeler, the Princeton physicist who named the black hole, went further in a 1989 talk in Tokyo and leaned on the question: every physical thing gets its existence from yes-or-no questions answered and registered, boiled down to "it from bit." As I read him, the coin has no face until something asks and something writes the answer down. How literally he meant that, I can't tell. And Peirce, way before any of them, leans on the agreeing: the kind of settling worth trusting is the kind that has to answer to everybody. Laid out like that, they pair up. Bohr and Rovelli lean on a step that only goes one way, in the plate and in the coin. Zurek and Peirce lean on many onlookers who agree, one with light and one with people.

Lay that next to everything above and the parts line up: a check that could have come out either way, a result that's now settled, a price paid in forgetting, a record that stays. I suspect that isn't a coincidence. I suspect the event this book prices on a machine is the same kind of event physics calls a measurement, showing up in a different place. The forgetting is my step. Bohr only says irreversible, and getting from a record that stays to a merge is the chairs, on a finite machine. Push on it and it gives at Zurek. In Zurek's picture the world settles a result by copying it, and copying merges nothing (Section 15 pushes on that). As I read it, that's Bennett's free measurement, writing on a blank page. So physics' own word lands on both sides of Bennett and Rovelli's door: the chalk on the paying side, the copies on the free side. The landing I'd put with the chalk, and when Section 5 pushes on Rovelli's paper it slides toward the free side, the side of finding out. So two pieces already stick out, and I'll leave them sticking out. Which picture a settling on a machine matches is part of the suspicion, and the suspicion doesn't settle it. That's as far as I'll take it here: a suspicion.

Now let me point the biggest gun I can find at it, and that's John Bell, the Northern Irish physicist at CERN whose 1964 theorem comes back in Part IV. In 1990 Bell wrote a short, furious essay against the word "measurement," with a list of words he said have no place in a physics that wants to be precise. On the list: system, apparatus,

environment, irreversible, information, and measurement, which he called the worst of all. I used half of them in the last few paragraphs without even trying. Bell would have taken my suspicion apart in an afternoon, and he'd have been right to, because every piece of it leans on words physics hasn't pinned down. Nothing in this book proves any of it. The proofs are about models and machines, and whenever physics shows up in them it shows up as an assumption with a name on it. The suspicion is why I went looking. It isn't what I found.

Where this goes

So here's the picture, put back together. Two machines can hold the same bits and differ in what they've established, and you can't see it even walking the floor between them, because it was never kept. The difference isn't information; information theory says the sort added nothing new. It isn't depth; depth belongs to the bits, and a copy has just as much. It's an event: a check that could have said no, about the claim, then a question closed for good, with no promise the answer's true. The could-have-said-no part lives in the procedure. Two end states with the same bits and nothing else kept can't tell it apart, so only a record written into the state can carry it. On a finite machine, a record that never comes down can only be raised by a merge, a kind of forgetting, the same kind ordinary computers pay heat for so their bits forget where they came from. If Landauer's right, that costs, for any device that has to settle things without already knowing which state it's in. The Thiele Machine puts the record and the charge in the step itself, and what that buys isn't more computing. It's a guarantee about every program. A log kept by one program covers only that one.

Every time I thought I'd found something new on the way down, somebody had a name for part of it already, and that last sentence is no exception: a rule under every program is the reference monitor's move, from 1972. Fine. What I think is put together here is smaller and more than one thing: the four small things I listed under what's proved, each with its neighbours named. There's no one big one.

And the stuff from the start, the audit, the proof, the ledger, the test sheet, the sticker, the confident AI? Here's the straight answer on which of those this helps. The sticker is a question of whether a piece of paper is genuine, and nothing here touches it. The "all tests passed" sheet fits only if the test is a checker proved to mean what it says, in the machine's own language; a set of tests that can miss a bug certifies "this procedure said yes," which is true and smaller. An audit and a ledger get the test sheet's answer: kept by a program on an ordinary computer, they're bits the program wrote, and they fit only where the step underneath keeps the record and refuses to raise it without the check. A proof is the case this book is built on. A proof checked

by a proof assistant is a check that could have said no, and the checker vouches that the proof fits its statement; whether the statement says what you meant is still yours to read. The AI that talks as if it's sure of itself isn't addressed at all. What the book gives you is a record on the machine itself, with a proof of what a raised one means. Getting that record to somebody across the room who has to believe it is a separate job, and Section 25, under "Outward consequences," says how little of the outside world I've reached. If you came for a tool, that's a short list. If you came for the question, this is where it starts.

The rest of the book is me trying to break all of it. First where I came from, then the word I couldn't define when I started, structure. Then the smallest thing that could carry the bill, on a sheet of paper, with one rule I'll hand you without a reason, and then the chapter where I find out whether I had to pick it. If it breaks there, the book ends there. I'd rather know.

Before the book: the short version

The book in one paragraph. The Thiele Machine is an abstract model of computation. Its core is a state, a step rule, a cost for each move and a yes-or-no certification reading, with one rule: the step turning the reading from no to yes costs at least one unit (I call that rule the toll, and its first name, the one I gave it before I understood it, is No Free Insight; Part II builds it). On top of that core go a counter that adds up the costs and only climbs, and a window that says what an observer sees. The headline result is proved in Coq (a proof assistant: software that checks every step of a proof), with every premise printed. With finitely many yes-states and a reading that is never taken back, every move that raises the reading merges states; consequently every schedule that prices merges charges that move, and every run from no to yes has positive total cost. That's what I call the universal certification result. "Universal" means it holds for every system with those premises; its counting core is classical, and what's mine is the price as a rule on a ledger and the toll that falls out of it (Appendix C). Landauer's principle (the physicist's rule that forgetting costs heat) is a named premise for pricing merges. Around it the book proves that finding out can be free, that a window can hide what was paid, that a base-driven permanent record over any machine is a latch on one event (a switch the event flips once and nothing flips back), and that a system can't decide, by a procedure it can itself represent, every property of its own programs that depends only on what they do. All of that is about any system with the named pieces. Part IV builds one machine that carries the model, six instruction kinds on two counters, on which commitments are earned, and two universal programs on a host with more counters: one runs every program of that machine, and one runs every Thiele machine whose workings a program can compute, each guest's record earned again by the host. The five steps below are the argument in order, with where each stands. "What's proved, what's argued, and what I just believe," back in "The point," sorts the claims by status and names the few that are new, each beside its neighbours.

I didn't invent a machine. I found one. That's how I see this work, and I'll be straight about the difference between that conviction and a theorem before I do anything else.

That sentence walks straight into one of the oldest fights in mathematics, so here's a short trip into it first. G. H. Hardy, a Cambridge number theorist who wrote *A Mathematician's Apology* in 1940 to defend doing mathematics for its own sake, came down on the found side about as hard as anyone ever has: mathematical reality, he wrote, lies outside the people who study it, and the theorems they like to call their creations are really just notes on what they saw there. The German mathematician Leopold Kronecker supposedly split the difference: God made the whole numbers, and everything else is the work of people. Supposedly, because push on that line and it gets thin fast. It comes down through Heinrich Weber, another German mathematician, in a memorial piece written after Kronecker died, pointing back to a lecture from 1886, and that's the only source I could find anybody citing. A great line with one witness. (The first chapter of Part II stops early on to ask how many witnesses a claim ought to need. The answer is more than one.) So does any of this settle whether I found the Thiele Machine or made it up? No. A proof checker doesn't settle it either; Coq checks a proof the same whether you think the theorem was out there waiting or you think you built it. That's why it's labelled a conviction. What it changes is how I hold the thing: like something that was already there, which means I can be wrong about its shape and the shape won't care. And to be exact about what "found" covers: the parts the counting forces on you, the latch shape and the merge. The event to record, the price list, and putting the count inside the step are choices, mine, and "The point" said so where it sorted what I was pushed into from what I chose.

What this book is. An argument, made with that model, about what an account of computation should preserve. Turing's account, and every account built on it, describes a computation by what it reads and writes. I think that account is a shadow, and that there's another axis, holding what the computation established and what that cost. Here's the argument in five steps, each with where it stands.

Holding a sorting costs something; standing behind one might. That's Part I: one word, structure, pushed on with you through as many of the people who wrestled it before me as I could reach. It finds three different acts that people run together. Building and storing a sorting you can read back costs work and space. Forgetting where things came from costs heat, if Landauer is right. And standing behind a sorting, vouching for it, is a third act, and nobody I read had put a price on it. The nearest was Rovelli's price on a choice that sticks, and his is a choice nobody could predict, which Section 5 shows is a different act. Whether vouching has a price is the question the rest of the book takes up.

Establishing is not free. If the step that turns uncertified into certified has to pay, every run from uncertified to certified has paid. Proved, for any system that follows that rule, whatever its states and steps. And the rule isn't arbitrary: on any system

with finitely many states, a certificate nothing takes back can only be switched on by a step that merges two states. That part is pure counting, proved. If merging is charged for, the way Landauer's principle charges for it in its worst case, the rule follows. That part is proved with the charge as a named premise, its critics named, and every premise refusable with a counterexample built. Learning isn't what costs: a measurement that forgets nothing can be priced at zero by an honest price list, and an observer still learns from it. Proved. What can't be free, on those premises, is committing to what you learned in a record the machine can't take back.

Two more things belong to that step. First, which steps a price on merging has to charge, when move names can be told apart: exactly the ones that merge. On a finite machine, every step that writes a record nothing takes back is one of them, and a step that switches a certificate on escapes the price only when the same step switches one off somewhere else. The tidier guess, that the priced steps are exactly the permanent records, is false, and three states show it. No theorem picks out a wider class of events every account must price. With one mark per halving, the step that writes a permanent record costs at least the base-two logarithm of how much it squeezes together (how many times it halves the number of states it can tell apart), and, from an even spread over the states in play, the same bound holds in bits of Shannon entropy, the standard measure of information. Proved, with the price as a premise. Second, where the machine I built stands, and that's a fact Part IV settles. The small machine of Part IV isn't a finite machine of that kind: its counters have no top, so it isn't finite, and some of its free steps merge states too. So the chairs don't reach it. The merge premise asked of the record layer alone does: a move that sends two different states with the same line and counters to one state costs at least one. The small machine's own prices meet that premise, and every price list that meets it charges the flag-raising step, with no finiteness premise (Theorem 4.15). What the chairs do reach is a finite piece: an eight-state machine, four slots and a flag, on which every premise is a theorem and the toll follows outright (Section 4). And the small machine runs that piece: seen through a window that shows only the line number, read modulo four, and the flag, a program of the small machine, after a two-mark setup, does what the eight-state machine does, one move at a time, and from then on pays exactly what the eight-state machine charges (Section 4, with the construction in Section 16.17).

A window can hide what was paid. Two runs can look identical through the usual window and differ in what they established and paid, and nothing reading only the window gets the difference back. The machine I built has such a pair for the window a Turing machine would point at it, the line and the counters, and every machine that meets the definition of Thiele-complete has one through its own window: two runs from one clean start that end looking the same, one certified and one not. Proved,

both. And when two single steps look the same through a window, before and after, and only one of them certifies, no price read off that window charges the certifying step exactly. Proved, for any window with such a pair of steps.

Leaning on a fact takes a paid history. A computation is entitled to lean on a structural fact when it holds the object, a checked relation, and a paid event on its record. Defined, for the model, with a counting fact beside it: narrowing a list of rivals down takes at least the yes-or-no questions that name the survivors. The cost floor that ties those questions to steps a run actually paid for holds for any system with a counter, once somebody supplies three things: the list of rivals before, the list of survivors after, and a tree of yes-or-no questions with enough leaves to cover the narrowing. Then if a run paid at least as many charged steps as that tree is deep, those payments cover the index bits, the questions it takes to point at one entry of the first list less the ones it takes for the second (Section 8). No run is shown to have to pay them. The run doesn't hand back the tree, and nothing in it says why a candidate was dropped; the lists count repeats, and the index bits aren't the number of questions anybody asked. On every Thiele-complete machine, the small machine included, the paid steps are the record moves, each exactly one mark, and a run from a clean start that ends with the record up, having narrowed a list that way, carries an earned chain and pays at least the index bits in record moves (Section 16.17); and when the questions are counted as the run's own checks, the floor needs no tree to be supplied, with a limit stated where it's proved. Entitlement is a license to lean on a claim. It doesn't prove the claim is true.

So the axis belongs in the step. An account of computation should carry what was established and what it cost as part of how it steps. Bolting it on after isn't enough. Proposed. The proofs around it show it hangs together and what it costs; whether it should be adopted is argued, and no test settles it. What's proved around it: over any base machine, a base-driven record is a latch on an event, with two free choices, the price list and the event; no decider whose flip a substrate can itself represent decides which programs admit a shortcut, given a recursion theorem (the fact that a program can act as though it held its own code). "Admits a shortcut" there is a label for any property of programs that two programs doing the same thing always share, and that some programs have and some don't; Section 13 says why the label is all the argument gets to see. A substrate is the model's bare shape, a kind of state and a rule for stepping, and here the extra pieces it's asked for are programs written as states and that recursion theorem. The wall comes by the same self-referring trick, the diagonal, that put a hole in the foundations for arithmetic built by the German logician Gottlob Frege (Part I tells that story). Which event deserves the record is a thesis, with its refutations listed.

Certification is one point I could pin down. It gave me a law I could state, a machine I could build and run, and proofs I could put in front of someone who didn't share my intuition. It's one witness in a larger argument about structure and observation. It isn't the argument.

How the book is laid out. Part I is the picture: how I got here, and what structure is. Part II is the axiom: the smallest thing that could carry a bill, and why its one rule isn't arbitrary. Part III is the logic: the counter, the windows, entitlement, insight, the latch, the whole direction, the exact shadow, the diagonal (the lambda calculus L gets its recursion theorem there by proof), five real systems, and a thesis. Parts I to III never use my machine; every claim in them is about any system with the pieces it names, and the pictures are toys. Part IV builds one machine, the smallest honest machine I could make: two counters, six instruction kinds, a table of checked facts, a ledger and a flag. On it a flag raised from a clean start provably went through a passing check, a commitment and a certification of the same claim, in that order, and it computes everything Minsky's two-counter machines can, so whether it halts is undecidable. There Thiele-complete gets its definition, the one a clock fails, and the universal Thiele machine gets built as one fixed program that runs every program of the small machine and earns its own flag, and then, with one paid move added, every computably presented Thiele machine. The machine that universal program runs on gets Kleene's recursion theorem by proof too, with Rice's theorem beside it; on the two-counter machine itself Rice's theorem holds on every input, Kleene's holds with inputs written as powers of two, for the function the program computes though not for the facts it records, and with plain inputs Kleene's is false, which is proved too. Then a check worth earning, the two-player game and its integer test: a flag raised from a clean start can stand on a passing check of eight counts, and the check passes exactly when every question pair was sampled and a fixed moment matrix is positive semidefinite, which forces Tsirelson's bound, the ceiling quantum mechanics puts on the score of a two-player game, for the tally it's handed. It's there to show the toll charges the same however deep the check is. It isn't a physics tool: it doesn't sort data into quantum and not, and a tally above two doesn't by itself rule out a local model with chance in it. Then what the counter has to do with physics. Part V is how to check all of it and how to break it. The vocabulary I had to learn is at the back.

If you want the argument without the whole tour, read "The point" and this short version, then Section 3, Section 4, Section 10, Section 16 and Section 26. If physics is your thing, add Section 5 and Section 21. If you mean to check me, add Section 23 and Section 24. The appendices are reference: everything in them is there to be looked up when a claim points at it. Nothing gets cut for that. It's a map. Nobody's on a diet.

One more thing before the book starts, because people get it backwards. I used language models to write the code and the proofs in this project, and I trust language models about as far as I can throw them, which is exactly why every formal declaration is checked by a proof assistant that doesn't care who or what wrote it. Coq rejects a wrong proof the same whether a tenured professor or a model hands it over, and it accepted these, with none of my own axioms anywhere in them, re-checkable by the procedure in Part V. Being self-taught and using modern tools didn't make this looser. It's the reason I made a machine check the formal corpus, and then built an auditor to read over the checker's shoulder for every way I might have cheated, on purpose or by accident (Part V says what it looks for). It doesn't trust me either. Good.

I think there's something foundational here. The results in this book have statements, premises, and proof terms. The larger claim is a conjecture. No theorem proves it. I built the machine so that the work could be done in public, with something firmer than my confidence to argue over. If you're the skeptical sort, and I hope you are, the two places to aim are the appendix that sorts what's mine from what isn't (Appendix C) and the chapter of ways to break it (Section 23).

Let me be straight with you

What this project is. The short version gave the argument. This section is the fine print it stands on, starting with the working form of its proposal, structural entitlement: a computation is entitled to lean on a structural fact when it holds the object, a checked relation, and a history that earned it. The model, its laws, and the instructions of any one build are different things, and I keep them apart. The generic flag-raising step pays to set a flag; it checks nothing. A claim that something is actually *true* needs explicit witnesses and a checker proved sound, and some results here are of that kind; the generic accounting law isn't one of them. It's a law about marks.

The universal certification result. The universal certification result is this: when the yes-state set is finite and the reading is permanent, every move that raises the reading merges states. Consequently, every cost schedule satisfying merge pricing charges that move at least one mark, and every run from no to yes costs at least one. This is a theorem over all systems satisfying those premises. It isn't a property peculiar to the implementation in Part IV (`nec_f_perm_flip_merges_yes_finite` and `nec_f_toll_from_merges_yes_finite`, NecFMerge.v). The counting under it is classical, the finite cousin of Zermelo's recurrence argument and folklore among automata theorists; what's mine is reading it as a price on a ledger, and Appendix C gives the neighbours. Three layers stay separate through the book. The first is the structural fact that a permanent raise merges states, which needs no prices at all. The second is the pricing premise, merge pricing, which turns that fact into a ledger bound. The third is the small machine of Part IV, which enforces earned certification on top of both.

The proved core, in three steps. Anything that follows the toll pays something to get from uncertified to certified. A window that shows two states as the same, when they disagree on the counter or on some other question, can't answer that question on every state. Those two are about any system with the named pieces, and so is the third, argued in three lines in Part III: a checker that only sees a window where two states collide can't be both sound and complete about the claim that separates them (Section 7). All three are statements about named laws and named windows, and the third is checked too, over abstract transcripts and then for every Thiele-complete machine (Section 7). An ordinary machine can keep a richer encoding, or enforce the same invariant through its own transition table. What I'm proposing is about why the extra distinctions matter. It isn't a theorem that every Turing-equivalent system

lacks them. And if you want the list of what I think is new, it's the short one at the end of "What's proved, what's argued, and what I just believe" in "The point," four small things, and nothing bigger.

The layers, once. The core definition (Definition 3.1) is a certification system: states, moves, a step, a cost for each move, a yes-or-no reading, and the toll. Everything else is added on top, and each theorem names what it adds. First come extra structures and requirements: a ledger that sums the costs, a window, permanence of the reading, a price on merges. Then an earned implementation, the small machine of Part IV, with a checker proved sound and a provenance theorem. Then an interface, Thiele-complete, that any machine can be read through. Then a universal interpreter for a stated class of guests. The bare definition doesn't ask the step to be computable, so when I say the record adds no computing power, I mean relative to a fixed base.

Substrate versus scaffolding. Two layers run through the book, and confusing them makes the theorems quietly stop meaning what they say. The *substrate* is the shape: a kind of state and a rule for stepping. Different theorems ask that shape for different extra pieces, and the pieces aren't the same. The universal floor (the "pays something" result above) asks for a certification reading and the toll on it. The counter's uniqueness (only one count fits a chosen start and price list) asks for a starting value and a schedule of increments; Part III gives the argument, a short induction that works for any system with those two pieces, and on the small machine of Part IV the ledger is the sum of the charges by theorem (Section 16). The diagonal asks for a way to write programs as states and a recursion theorem for the transformations the substrate can express, and needs no toll at all. "Substrate-level" doesn't mean one fixed plug that every theorem shares. A theorem only gets to use the parts it asks for. Ask it what it needs and it'll tell you, and whatever else is lying around the workshop isn't the theorem's. So each result is stated over exactly the extension it uses. The substrate with the extension a theorem names is what I mean by *the model*, and the name Thiele Machine belongs to the model. The *scaffolding* is everything that holds the shape up so you can touch it, and when I say *my machine*, *the small machine*, *the machine I build* or *the build*, this is what I mean: the machine of Part IV, with its two counters and their versions, its table of checked facts, its channel, trap, ledger and flag, its six instruction kinds and their prices, the host with a register for every whole number, the universal programs that run on that host, and the Coq files that define all of it. The build fills in several extensions at once and borrows, for any one theorem, only the extension that theorem names. Let that line blur and a theorem stated over one extension starts reading as if it carried premises it never uses. "Substrate" never means the build, and it never means the side of a simulation doing the stepping; for that I say "the machine doing the stepping," as against "the program it runs."

So here's how the book keeps the two apart. Parts I to III are about any system with the pieces each result names, and their pictures are toys you can hold (two boxes on a sheet of paper, chairs in a room, a row of dots). My machine isn't one of them; it first appears in Part IV. Every Part III claim that has a machine instance gets that instance in Part IV, with its own checked name, and Part III says so in one sentence at most. When the origin story, a few pages on, says the shape "was the substrate the entire time," it means this layer, the model's shape. It doesn't mean any piece of the build. Same story with two more of its words. When it says "the machine" about the thing it named the Thiele Machine, or calls that thing a meta machine, it means the model; "my machine" is the build, and the build doesn't show up until Part IV. And when it lists what the model asks for (a state, a step, a record, a ledger, a window), read the ledger and the window as two of the extra pieces: a result asks for them only if it uses them.

Part IV has one piece of scaffolding, and I'd rather you know up front how small it is. Section 16 builds the smallest honest machine I could: two counters, six instructions, a table of checked facts, a ledger and a flag. Earned commitments are proved there (a flag raised from a clean start was checked, then committed, then certified, on the same edition, with nothing touching that thing between the check and the commit), and so is universality, through Minsky's two-counter machines and a halting result borrowed from the Coq undecidability library. Then one fixed program, on a host made of the same kinds of instruction with more counters, runs every program of it, and with one paid move added, every computably presented Thiele machine. None of that is the model. It shows the model fits in six instruction kinds.

The names again, this time with every part pinned down, because I'll hold to them for the rest of the book. *The Thiele Machine*, with capitals, is the model and nothing else. *A Thiele machine*, lower case, is any particular system proved to be an instance of the model's core definition, a certification system with its five parts and its toll (Definition 3.1), the way one rule table is a Turing machine. The toys on paper in Part II count, the same way trivial Turing machines do. The small machine counts too, with its toll proved, and Part IV names the theorem. A Thiele machine is *weakly* Thiele-complete when its base is Turing-universal, and that bar is nearly empty: a machine that charges one for every step is a Thiele machine for any record, so a clock bolted to a computer clears it, and so does a machine whose record never rises. So *Thiele-complete* asks for four things (Definition 16.7). A universal base whose moves are free and can't touch the record, and a record that stays up once it's up. A record that, from a clean start, rises only after a passing check of a claim, then a commitment to that same claim with nothing it's about changed in between, then a certificate, with the checker proved to mean what it says: the claims and what they mean come

from a language fixed first, and the proof is that the checker's yes makes the claim true in that language's sense. An exact toll: one mark for each of those three acts and nothing for anything else. And a claim whose check, commit and certificate raise the record on some loaded starts and leave it down on others. Every clock-style machine fails the definition, because it charges for computing, whatever its record, and every machine that meets the definition hides its record and its ledger from its own window, both by theorem. None of the four clauses is there for decoration: for each one there's a machine that meets the other three and fails it, a Thiele machine with a universal base that no reading makes Thiele-complete (Theorem 16.23). The small machine meets it, and so does the same machine over any properties with an exact test for telling two of them apart and a proved checker, as long as one of them can come out both ways, sorted lists included. A *universal* Thiele machine is one fixed program that runs any machine of a stated class as a guest, with the guest's record earned again and its toll paid by the host's own steps. Part IV builds one, U: a fixed program in the small machine's own six kinds of instruction, on a host with more counters, with no step of any guest built in. Its guests are the small machine's programs, so every computation runs on it under Minsky's 2^n encoding, done outside the machine, and it matches each guest's halting, answer, flag and ledger exactly, raising its own flag by its own check, commit and certificate. Then U_P, the same program with one more handler for a paid move that does nothing else, takes as its guests every computably presented Thiele machine, one whose next move (picked by a driver), step, prices and reading are computed by programs, with the reading decided on every state. It matches the guest's halting, raises its own flag on its own earned chain exactly when the guest's reading has said yes, and keeps the guest's prices to within a surcharge of at most two when the guest's reading starts at no, which no program on that host, run from a clean start, can avoid in general. And a computer that runs the small machine, your laptop say, isn't a Thiele machine. It implements one, inside bounds, the way a laptop implements a Turing machine. Push on that comparison, because it's loose, and the looseness is the point. Does a laptop implement a Turing machine? Not exactly. Turing's tape never runs out, and a laptop's memory does; strictly, a laptop is a very large machine with finitely many states, and it behaves like a Turing machine only on computations that fit. Nobody minds, because the bound is known and most computations people care about fit inside it. That's the same deal any device running the small machine gets: its counters stop somewhere, and the theorems speak only for runs that fit. The bounds aren't fine print. They're the whole difference between implementing a model and being one.

How much can a difference in width matter? There's a famous answer, and it's worth the detour. On 4 June 1996 the first Ariane 5 rocket lifted off and broke apart

about forty seconds after its engines lit. The inquiry board, chaired by the mathematician Jacques-Louis Lions, traced it to one conversion in the inertial reference software: a sixty-four-bit floating-point number, an alignment value called BH that's tied to horizontal speed, squeezed into a sixteen-bit signed integer. On Ariane 4, where the software came from, that value could never get big enough to overflow, because of how that rocket flew in its first forty seconds. Ariane 5 flew differently. The value got too big, the backup unit failed on it, the main unit failed the same way about seventy milliseconds later, and the function doing the converting didn't even serve a purpose on Ariane 5; it was kept for commonality with the old rocket. The board's report says the assumption that made the conversion safe had been agreed on and never exposed to outside review. A width premise that holds on one machine is a fact about how that machine gets used. The code doesn't carry it, and yet it travels with the code without anybody noticing. The only defence I know is the dull one: write the premise into every statement that needs it, where the next person will trip over it.

What this project is not. Not a solution to P versus NP, and not a complexity-class separation of any other kind. Not a theory of everything. Not a claim that computation is physics, or that physics is computation. Not a physics result: the game result (the integer check and Tsirelson's bound from the short version) is pure mathematics, and Landauer's principle enters only as a premise with its name on it. Nothing here derives it. Not a quantum computer. Its theorems about quantum strategies are mathematics about vectors and matrices, with no claim about a physical device. Not a verified chip or a verified device of any kind; a checked model and a checked machine, on paper and in Coq. Not a claim to priority on the ideas it builds from. What it does add, with the closest earlier work beside each item and the difference spelled out, is sorted in Appendix C: chiefly the price as a named rule on a ledger and the toll derived from it (on a counting core that is classical), the pricing form of the window fact, the latch form of a record that never comes down, and a checked machine and universal interpreters that carry the toll, whose every-program idea is the reference monitor's. Not closed on every question: the ones I couldn't settle are listed at the end.

What I do and don't know about prior work. I came to this with no formal training in the fields it turned out to need, and I've read what I could reach, with every tool I had. The prior-work map below is orientation. It doesn't pretend to be scholarship. For each piece of this book it says who did the thing it leans on, and what, if anything, this development adds. I'd welcome correction on any row.

There's a law about rows like these, and it's a joke that keeps turning out true. In 1980 the statistician Stephen Stigler proposed that no scientific discovery is named after the person who first found it, and then credited the idea to the sociologist Robert

K. Merton, who had been writing about fights over scientific priority since 1957. So Stigler's law is an example of itself, which is either the best joke in the history of science or a very tidy accident. Is it a law? No. It's a pattern. Nobody proved it. But this book has already tripped over it once, in the opening chapter: the pigeonhole principle wears Dirichlet's name, and Leurechon had it two centuries earlier. And I put my own name on a machine. By Stigler's law, that makes it somebody else's. The table below is my best attempt at finding out whose.

What I compare each time is what a result is *about*. Landauer's bound is about heat, a physical cost under a thermodynamic model. No Free Lunch (an optimisation result, in the list below the table) is about averages over a whole spread of problems. The toll is a floor on one kind of step, the step that turns a certification reading on. Rise-only pricing adds a second rule on top: charge nothing on a step that doesn't turn it on. So the comparisons are about the shape of each result. They aren't claims about who got somewhere first, and they don't show that certification is the only event worth pricing.

Three kinds of exact. The word exact does three jobs in this book, and each job keeps one name. *Rise-only pricing* charges at least one mark on every step that raises the reading and nothing on any other step; its tightest bill, one mark per raise and nothing else, is the *exact schedule*. *Instruction-exact pricing* charges every instruction a fixed listed price. On the small machine of Part IV that's one mark each for CHECK, COMMIT and CERTIFY and nothing for the rest, and the definition of Thiele-complete calls it the *exact toll*. CHECK and COMMIT raise no flag, so the small machine is instruction-exact without being rise-only, and no rule is broken by that. *Simulation-exact accounting* is a host whose ledger equals its guest's ledger at every point where the two runs match, as the universal program U's does. A machine can meet any one of the three and fail the others.

The right-hand column uses words the book only builds later (the latch, the fibre, the universal program, the integer check, the matrix test), so read it as a list of places to go. You aren't meant to follow it yet.

Prior work	What this book leans on	What this development adds
Landauer 1961; Bennett 1973, 1982, 2003; Zermelo 1896; Bocchieri and Loinger 1957; Pin 1992; Brodsky and Pippenger 2002; Ouldridge and Wolpert 2023; Murashita, Funo and Ueda 2014	erasure and merging cost; measurement can be free; recurrence (a step that loses nothing, on finitely many states, comes back around); machines whose moves all reshuffle states recognise only group languages; a language whose smallest automaton has to merge states costs entropy to recognise; a step with no way back has a logarithmic entropy floor	the ledger form of a counting core that's classical (recurrence's finite cousin, folklore for permutation automata) and already priced in entropy for automata: the price as a named rule on a ledger, a certification toll derived from finiteness, permanence and that rule, with merge-or-revoke, the squeeze and entropy forms, and each premise's counterexample built
Maroney 2009; Sagawa and Ueda 2009; Sagawa 2014	thermodynamic bounds that depend on the actual spread over states; a known state's reset has no Landauer floor under control timed from outside; time-symmetric control charges every nonreciprocal move	the certificate-specific entropy bound with its support and permanence premises out in the open; heat still needs the Landauer premise, and no new physical law is proved
Shannon 1948; Hartley 1928	entropy and log-of-count bounds	the same squeeze bound in counted states and, from an even spread, in bits, for a permanent flip
Tarjan 1985; Lyapunov 1892; Nipkow; Charguéraud and Pottier	the potential method, already mechanized	the toll as that method with the flag as potential
Katsumata 2014; Orchard, Liepelt and Eades 2019; Danner and Licata	cost carried as a grade that composes with the computation	a run's cost as a writer cost and as a grade fixed before the run; the counter rides in the state, put there by the step rule, and it is their writer exactly, while their type systems and before-the-run checking of grades are not embedded, by design
Lamport 1978; Floyd 1967; Turing 1949	one-way counters give order and termination	the receipt's three properties (it only climbs, it equals the sum of the charges, it's the only such total) stated and proved for one schedule

Prior work	What this book leans on	What this development adds
Moore 1956; Kalman 1960; Cousot 1977; Codd 1970; Sampath and coauthors 1995; Saboori and Hadjicostis 2007	what a window loses; when an observer can work out a hidden event, and when a secret state stays hidden	the fibre fact with a price in place of a decoder; the small machine's named collision, and the window every Thiele-complete machine is blind through
Smith 2009 (quantitative information flow)	how much an observer learns about a secret from what it sees	none for leakage: the collisions rule out exact decoding through named windows, which isn't zero leakage, and no leakage measure is supplied
Gettier 1963; Necula 1997; Cook 1971	entitlement is not truth; proof-carrying checks	entitlement as object, relation, paid event, with a counted bound
Dwork and Naor 1992; Back 2002 (Hashcash, first proposed 1997)	make establishing something cost, in a way anyone can check cheaply	a comparison, with no claim that the two are the same: proof of work checks a found value against a difficulty target, while the toll charges one mark at the step that certifies and verifies nothing about effort
Eccles and Jordan 1918 (patent filed) and 1919 (paper); Plotkin 1989; Shapiro and coauthors; Abadi and Lamport 1991; Ahman and coauthors 2018	the latch; write-once and grow-only state; history variables; monotonic state whose witnessed facts can be recalled later	the latch form over any base, with its two counterexamples and converse
Cantor, Gödel, Turing, Kleene, Rice; Lawvere 1969	the diagonal	the shortcut wall on any substrate; L's recursion theorem by proof
Bell, CHSH, Tsirelson, Wehner, NPA, Landau, Masanes, Fine; Echenim and Mhalla 2023 and Lean's mathlib (Tsirelson's bound already machine-checked)	the game, its walls, and the matrix test	an integer check on counts that implies the pinned matrix is PSD, which implies the bound
Buterin and Griffith; Wood; TCG; RFC 9162; Necula	five record disciplines	five fragments read through one question: is the write priced

Prior work	What this book leans on	What this development adds
Zurek 2009; Ollivier, Poulin and Zurek 2004; Blume-Kohout and Zurek 2006	pointer states and redundancy	a thesis borrowing the shape, with the gap to the physics stated
Leroy 2009; Klein et al. 2009	the discipline of saying where the proof stops	the list of what's trusted, in one place
Minsky 1961, 1967; Forster, Larchey-Wendling and coauthors 2019, 2020	two-counter machines are universal, and their halting is undecidable, checked in Coq	a machine of six instruction kinds on two counters where earned commitments, no forging and an exact minimum price of three are theorems, with halting reduced to the library's
Forster and Smolka 2017; the Coq Library of Undecidability; Norrish 2011	L as a model of computation in Coq; counter-machine constructions; a compiler from computable functions to counter machines	L's recursion theorem, Rice and halting, checked; the library's compiler used to put every computably presented Thiele machine on the universal host
Turing 1936; Post 1947; Popek and Goldberg 1974; Hamlen, Morrisett and Schneider 2006	the universal machine as one fixed table; hosts that keep guests apart; which policies a monitor can enforce	a definition of Thiele-complete that every clock fails; one fixed program in the small machine's own instruction kinds that runs every program of it with halting, flag and ledger exact, earning its own flag; and one fixed program, with a paid move added, that runs every computably presented Thiele machine, its ledger exact to within a surcharge of at most two marks when the guest's reading starts at no, which can't be avoided in general

Everything in the right column is stated with its premises in the body of the book, and everything in the middle column is cited where it's used. The table is the short form. The long form is Appendix C, which takes each row, says what was already known with sources, and sorts what's added into new result, new proof, mechanization, implementation, synthesis, or conjecture or thesis. Most of the right column is mechanization, implementation, and synthesis.

A few more labels I went and looked up are neighbours. Some of them also sit in the table above, and one, the reference monitor, is the move the small machine's step

makes. I list them so you can see why I don't think the toll is one of them under a new name. Each is a claim inviting correction.

- Linear logic (Girard 1987) treats an assumption as a resource that gets used up. It constrains what a program, or a proof, may consume. The toll constrains the model's step function instead. Same instinct, different layer.
- Foundational proof-carrying code (Appel 2001) checks safety proofs against the smallest trusted base it can: a logic plus the machine's semantics. Here no proof rides along with the program. The step rule writes the receipts as the run goes.
- Session types (Honda 1993) constrain how programs talk to each other so the conversation can't go off script. The toll is about what one computation has paid to establish. Different problem.
- No Free Lunch (Wolpert and Macready 1997): averaged over every problem, whatever an optimiser gains on some problems it gives back on others. That's an average over a whole problem space, and the edge is paid for elsewhere. The toll charges one step in one run, with no refund anywhere.
- Kolmogorov complexity (Kolmogorov 1965): the structure in an object is the length of the shortest program that prints it. That's a property of a finished string, with no steps and no moment of paying. The toll is a law about a step.
- Cook and Levin (Cook 1971; Levin 1973): checking can be cheap while finding looks expensive, and whether finding is really harder is the open P-versus-NP question, a gap in how difficulty grows with size. The toll puts a fixed floor under the one step that commits, however easy or hard the finding was.
- The thermodynamic cost of choosing (Rovelli 2024): settling a binary choice at temperature T gives off at least kT ln 2. That's the closest thing to the toll itself I've found, because it charges a commitment. An object or an average isn't what it prices. But it's physics, indexed to a temperature, and its choice is unpredictable, a branching, and stable only for a while, where the toll's commit is a deterministic merge that stays. The toll charges the commitment in a model of computation, with no temperature in it. Closest neighbour. A different chair.
- Persistent copies (Ouldridge and ten Wolde 2017; Ouldridge, Govern and ten Wolde 2017): making a copy that stays correlated with its original after the two come apart has a thermodynamic bill. The nearest physics I've found to "learning free, committing priced." It's heat for a correlation, in chemistry; the toll is a count in a model of computation.
- Thermodynamics of finite automata (Ouldridge and Wolpert 2023): if the smallest automaton for a language has a letter that sends two states to one, recognising

that language on a device that repeats one local step produces entropy, and if the smallest automaton only reshuffles, recognition can be free. That's the chairs with a physical price on them, and one of their worked examples is the latch's language turned inside out. It's the closest neighbour of all to the road the toll is derived on; Section 4 says exactly what's left over for me.

- CerCo, the Certified Complexity project of Amadio and coauthors, builds a verified C compiler that carries the object code's execution cost back up to annotations on the source. There the cost decorates the program and survives compilation. Here it lives in the model's step rule.
- Reference monitors and execution monitors (Anderson 1972; Schneider 2000; Azevedo de Amorim and coauthors 2015). A small piece every action passes through, enforcing a rule on every program. Schneider argued that a monitor watching a run can enforce only rules whose breaking shows up at a finite point, and "no certificate without an earlier passing check and commit" is one of those; the micro-policies check a tag rule at every instruction, proved in Coq. That's the move the small machine's step makes, so this one is the house itself. What's added is the edition-bound check-then-commit with a paid, permanent record on top, and the derivation of why the payment can't be zero. Kevin Hamlen, Greg Morrisett and Fred Schneider (2006) went on to sort which rules monitors, program rewriters and static checkers can each enforce, and they differ, which is one reason this book doesn't claim that every enforcement mechanism is a Thiele machine underneath.
- Trusted components in replicated services (Chun and coauthors 2007; Levin and coauthors 2009; Veronese and coauthors 2013; Decouchant and coauthors 2022; Clement, Junqueira, Kate and Rodrigues 2012): a small record inside each replica that a replica taken over still can't make say two things, with a proof of exactly what that buys. The record is theirs. The toll isn't in their picture.
- Ghost state (Owicki and Gries 1976; Filliâtre, Gondelman and Paskevich 2014): variables added for a proof that can be erased without changing what the program computes. The flag and the counter are ghost in that sense. The small machine's table of checked facts isn't, because the step reads it and refuses on it, for every program.
- Verifiable computation and proof-carrying data (Parno, Howell, Gentry and Raykova 2013; Chiesa and Tromer 2010): a short proof, checkable by a stranger, that a computation's output is right. That establishes the claim for the stranger, and anyone with the inputs can make one, the copier included. The record here is about one machine's own history. A proof like that is one way to carry the local record out of the box.

- Commitments and entitlements (Brandom 1994): talk as score-keeping, tracking who is committed to what and who is entitled to it. Brandom's entitlements can be defeated and can be passed on by testimony. Here entitlement is priced, step by step, and a copy carries it only as far as the copy is genuine. Same words, different machinery.
- Correspondence assertions (Woo and Lam 1993; Gordon and Jeffrey 2003). In protocol verification, "this end event must be preceded by a begin event for the same thing" is a correspondence assertion, and a type system can prove a protocol keeps it against any opponent. "No certificate without an earlier commit, and no commit without an earlier check of the same claim" has that shape. Theirs is proved about programs by typing. Mine is enforced by the step, which is the LCF comparison again, at the level of single instructions.
- Monotonic parallel composition (Åman Pohjola and Parrow 2016) and persistence in the pi-calculus (Palamidessi, Saraswat, Valencia and Victor 2006). Åman Pohjola and Parrow separate models where a neighbouring process can never stop a step from models that can solve consensus, and test-and-set, an instruction that writes a cell and reports what was there, is on the consensus side. Palamidessi and coauthors compare messages that are used up with messages that stay, and the end of their study is a reduction from Minsky machines. The small machine, put into Åman Pohjola and Parrow's setting with threads pooled on one shared state, has a consensus process in their sense, so it lands on their non-monotonic side; that, and their theorem, are checked (`cs_machine_not_monotonic`, ConsensusSeparation.v). The comparison with Palamidessi and coauthors' persistence stays a comparison.
- Constructor theory (Deutsch and Marletto 2015; Marletto 2016; Marletto, Vedral and coauthors 2022; Violaris and Marletto 2022): physics as which tasks a device that runs in a cycle can do, with irreversible meaning possible while the inverse isn't. It's the nearest physics I know to "committing costs" stated for a cycling device. It's quantum and approximate, and the chairs are classical, finite and exact (Section 4).
- Accountability and fault detection (Haeberlen and Kuznetsov 2009; Küsters, Truderung and Vogt 2010; Treitz and Künnemann 2026): when an outside party can tell who misbehaved. Their windows are this book's windows, and the trust they leave over, an honest log or a trusted step, is left over here too (Section 7).
- Transferability (Easwaran 2009): a deductive proof lets you learn from someone you don't trust because you can check every step, and a probabilistic proof doesn't. That's the argument for carrying the object and re-running the check (Section 7).

And then the silly question that list raises, which I'm going to chase anyway: why is an optimisation theorem named after lunch? Wolpert and Macready borrowed an old saying, "there ain't no such thing as a free lunch," and the saying is older than almost everyone who gets credit for it. American saloons in the 1800s put out a free lunch for anyone who bought a drink, and the drink paid for the lunch. A Washington newspaper was already saying in 1909 that somebody has to pay for a free lunch; a newspaper fable by Walter Morrow had the whole line in 1938; a book by Pierre Dos Utt already had the squeezed form, TANSTAAFL, as its title in 1949; Robert Heinlein's 1966 novel *The Moon Is a Harsh Mistress* made that squeezed form famous; and the economist Milton Friedman put the saying on the cover of his 1975 book of essays on public policy. I'd have credited Heinlein or Friedman. As far as anything I could find goes, neither coined it. Stigler's law again, at lunch. Here's what it changes. No Free Insight is named like a cousin of the saying, and the family resemblance misleads in one way. The saloon's lunch was paid for somewhere else, in the price of the beer, and that's Wolpert and Macready's shape: a gain here, handed back over there. The toll isn't that. Nothing gets paid somewhere else. The step that raises the flag pays, on the spot.

I

The picture

1

How I got here

What I am is curious, past the point of good sense.

I pick a thing up and cannot set it down until I have turned it over enough times to see how it holds together, and this time it did not let go.

I am not a programmer and I was never formally taught any of this.

I was not trying to build a formally verified CPU, and the one I came close to building later, on the way here, is not part of this book.

I would not have known what one was if it bit me.

I was trying to build a 3D renderer.

There was no plan behind it.

Just a pull to make something real with my own hands, and the kind of stubbornness that will not let a thing go until it stands up on its own.

I have worked on it in off hours every day since.

Once I started digging in, two things came clear fast. I knew nothing about computers, and the field was already saturated, every obvious road paved and crowded. So I needed an angle nobody else was standing on, and the angle I had wasn't a strategy. It was just the only way my head works. I did not know what compositional meant. I had never heard of category theory. I called the thing I was building digital DNA, because that is what it felt like in my hands, and I thought about it in the only language I had: gravity pipes and directions and vectors, things that were linear and somehow not linear at the same time, structure that flowed and joined and came apart without my owning one proper word for any of it. I built a custom rendering pipeline out of those intuitions, by feel, throwing out pieces that did not sit right even when I could not have told you the rule they were breaking. And then, the way I always do, I went looking for what I was actually doing, and found out it already had a name. An old one. A deep one. What I had been reaching for in the dark was composition. It was category theory, a whole field I had never once heard of, sitting there describing the exact shapes I had been shoving around. The moment that landed, the rendering pipeline stopped being a rendering pipeline and started turning into a categorical engine, and I started swallowing the real theory as fast as I could find the words for things I had already built.

The method underneath was crude and it held. I wrote a long structured spec in JSON (a plain-text format for describing data) as a context document, fed it to Copilot, and had it build one Python file per concept, one at a time, each with its own tests, and I would not move to the next until I could watch the current one actually run. Category. Object. Morphism. Functor. Monoidal product. It came together in pieces and never in chapters, the way a picture resolves out of static. The Yoneda lemma (a result buried deep in category theory) was the piece that made the rest snap into place, the night I quit borrowing other people's runtimes and started writing my own around it, with Z3 (a constraint solver out of Microsoft Research) wired into the middle.

Then within weeks I found out the engine I had built to draw pictures was general enough to run a physics simulation on the exact same primitives, without my rewriting any of it. I wrote Newton's laws as a program in it, half expecting nothing, and it ran. I want to be careful about what that was and what it was not, because this is exactly the kind of thing that gets misread. It was not me deriving physics, and I am not claiming the machine explains gravity. It was the compositional structure being general enough to express a physics simulation the same way it had expressed a renderer, and that generality is what told me the structure was the point and the pixels never were. Somewhere in those weeks I crossed from debugging why one pixel would not show up to understanding that I had quietly built a categorical computing engine, and that the rendering had only ever been the first thing I aimed it at. The pixel did eventually show up, for what it is worth.

I kept going, because by then I could not not keep going. I built a small custom language for describing scenes (the name for that is a domain-specific language, a DSL), then a program that read the language and caught my mistakes in it (a linter), then a way to type commands at the whole thing from a terminal (a command-line interface, a CLI). The night the engine first ran a program declaratively, meaning I told it what should happen and let it work out the how without my spelling out each step, I told my wife, Julie, I thought I was onto something. She said okay.

Then I hit a wall, and it is the wall this whole document is about. Z3 could settle the small questions and fell over on the exact ones I needed answered, which left me staring down an infinite hallway of guess-and-test with no proof waiting at the end of it, ever. One night, not even working, I was watching a video about Flatland, the old story about a creature living in two dimensions who cannot so much as picture a third. And it landed on me sideways, all at once, the way the real ones always do. I said it out loud: I get it now. There is another axis. Classical computation only ever sees what it can read and write off its own surface, and there is a whole direction it is blind to. It isn't broken. It was just never built with an eye that points that way. That's how it landed that night, and it didn't survive checking in that form:

the direction turned out to be one an ordinary computer can reach, and what the usual rules leave out is the price (Section 3.6 says where the axis ended up). The two computers on the first pages of this book are my attempt to show you that blindness with something you can hold. I opened a new repo that same night and called the thing in it the Thiele Machine. The name began as a working title and became the name of the model. It carries the larger claim plainly: the machine treats structural state as part of computation.

What survived the whole trip from there to here is not the code. Not one line of it. By the time the Coq proofs finally landed, every line of that first renderer had been thrown out and rewritten at least twice, probably more, and good riddance to all of it. What survived was a single architectural commitment I made early and never once let anyone talk me out of: structure is the state itself, the actual thing the machine runs on. It is no metaphor laid over some realer state underneath. Category theory was the first language I found for that structure. It is not what the model is. The model asks for a state, a step, a record, a ledger, and a window, and category theory appears nowhere in that list. It lived on for a while in a bigger machine I built on the way here, a fifty-one-instruction virtual machine with its own chip design, whose partition graph (groups, and arrows between them) was, in different words and different syntax, the same object as the category structure that came together in that first week and first ran morphisms correctly at two in the morning. That machine was useful for building and confusing for everything else, so it's set aside now, kept but out of this book. What building it showed me is that six instruction kinds were enough for everything this book needs, and that the big one was mostly in the way. I did not write that code, not then and not now. I directed the models and refused everything they handed me until the morphisms composed the way the shape said they had to. The morphisms in that early code were objects that remembered where they came from; the records in the small machine I ended up with remember where they came from too, and you can walk up to one and challenge it. The receipt grew teeth. But the shape itself never moved. The shape was the substrate the entire time, and everything else, every renderer and language and proof, was me slowly learning to see what I had already built. By the shape I mean structure held as state. Not category theory. Category theory was just the first language I had for it.

Formal verification was a graveyard of dead ends further on. The hardware came after that, in the bigger machine I set aside, and none of it is in this book. Every step pulled back to show another layer underneath that I had not known was there, and I mean that plainly: I did not know what a Turing machine was when I started, I did not know what Coq was, I did not know most of the words in this document existed. I learned each one the same way, by walking face-first into a wall that needed it and not being let past until I had it. The needing always came first. The learning came after, dragged along behind the needing because it had no say in the matter.

Large language models were my hands the whole way through, and I want the division of labor exact, because it is the thing people most want to get wrong about work like this. The hands had nothing like this to draw on: no reference implementation, no prior project to point at. The ideas, the architecture, the direction, the demand that the thing be true: those are mine, all of them, no asterisk. The symbol-pushing, the actual Coq and the actual math, the models produced and the proof checker judged. The proofs compile or they do not. I refused every one that did not. Coq is an obedient idiot. It checks the proof matches the statement. It does not check that the statement means what you think it means. A theorem can claim "every cat that is also a prime number has whiskers" and the proof can be three characters long and Coq will nod and stamp it. So I read every statement. The proofs you can't fake. The statements you can fake by accident, which is the more dangerous failure mode. I refused both kinds. However the symbols got onto the page, the argument on it is checkable by reading, and it would be exactly as right or as wrong if a million monkeys had typed it. The monkeys could not have refused.

A word on what I actually do, because it runs under everything after it. I do not write code or math by hand. I do not pick constants. I do not grind through derivations. I have never written a line of code on paper or in a notepad in my life. What I do is see, and direct, and refuse. When I take hold of an idea it does not stay a point. It opens up. It becomes a shape I can blow up and turn over and take apart, lay across another shape to find where the two of them touch, run forward and then back to where it started, zoom into until it is the only thing in the world and pull back until it is one dot in a field of other dots, and the whole time it stays wired to everything I have ever connected it to, the right connections and the wrong ones both, all of them live at once. I am not reading that field neutrally, the way a search would. The thing doing the looking is me, the whole of me, my gut and my mood and my stubbornness picking what lights up and which way I move through it, and that is the part you cannot lift out without turning me back into a search engine. So I see where a thing has to fit, the shape it is obligated to have, usually well before I could write it down in a single symbol. Then the model gives me the symbols, the proof checker takes them or throws them back, and I refuse anything that will not compile or does not match the shape I am holding. That refusal is the one thing in

this entire document that is mine and only mine: the demand that it be true, and the throwing-out of everything that wasn't.

I am telling you all of this because it's the engine. I did not come up through anywhere that hands you a foundation and says trust us, this part is settled. No professors, no colleagues, no field full of authority to lean back on when a claim got heavy. From the outside that is a deficit. From the inside it is the whole reason any of this is solid: when I meet a claim I either find the proof or I do not believe it, and that goes double for my own, because my own are the ones I most want to wave through. It is how I am built. It is the only way I know to hold a thing and actually trust that I am holding it.

So everything in here gets built from the ground up. It isn't a courtesy to you. I build it because I do not believe I understand a thing until I have built it with my own hands, watched it stand up on its own, and tried to knock it over. I built this document the exact way I built the machine, brick by brick, refusing each brick until it held weight. That includes the parts you already knew cold before you opened this. I did not skip them for you, and I could not skip them for me.

The vocabulary I had to go look up to do any of this sits at the back of the book, so the argument comes first. If a word stops you, it is there. If you already know this material, you never have to open it. But if you catch something in the basics that is wrong, I want to know, and I mean that as a real ask. It isn't politeness. Nothing in here is correct just because I wrote it. Especially the parts I wrote.

On the name. I wanted to call it the meta machine. Then, in October 2021, Facebook renamed its company Meta: a company taking a term for self-reference as its name and draining the word of meaning in the same motion. So I went one level up, where this machine already lives. The most meta move available, for a machine that runs on meta-level reasoning, was to put my own name on the cover and take the word back. That's the whole of it, a meta move on a meta machine. It happens to be my dad's name too, and my wife's now. So I asked them before I did it. They said go ahead.

2

What structure is

One word. That's this whole section. Here's the word, and you can decide whether to stay. Structure.

I used that word for most of my life without once looking at it, which I suspect is true of most people and most words, and it's fine, right up until you do something stupid. The stupid thing I did was build a machine that keeps a running bill, and the bill has one rule I couldn't talk it out of. You can arrange things however you like for whatever price you name, including free. But the moment you ask the machine to *vouch* for an arrangement, to put its own name on the claim that these things belong together, there's a charge, and it isn't zero. How much, charged in what, why I picked that one, and whether I had any business picking it at all: that's all later, and some of it I'm still not sure of. What I was sure of, standing there with the thing built, was that I'd made something that charges for standing behind structure and I could not have told you what structure was. It didn't care. It can't nod along the way a person does. It needed the word to hold still, and the word had never held still for anyone, least of all me. (The machine itself stays off the page until Part IV, on purpose. Until then, the one rule is all of it that matters, and I want you to meet the rule in a toy you can hold. A machine you'd have to take on trust can wait.)

So I went looking at the people who'd wrestled it before me. Some of them have been dead two thousand years and are still, I'd bet money, better at this than I am. I used everything I had: language models, more than one, books, libraries, my own gut, anything in reach. I don't know if I found enough. I never do. There's always another shelf behind the one I was reading. And being dead and famous is not the same as being right, so I pushed on each of them, and what follows is what I could reach, pushed on as hard as I know how. The one thing I'm asking is that you push on it with me, out loud, at your own kitchen table, because the first time through I was the only one checking, and that's not enough people.

The order I ran into them was a mess. Category theory came first, because I'd built one by feel before I knew it had a name, and everything else came in whatever order the next question dragged it in. I'm going to put them in the order where each one leans on the last, which is mostly the order they happened. And none of it is an oracle. History is where I start digging. It gets retold, smoothed over, and now and

then made up outright (there's a drowning coming up that nobody can place), so every piece of it is a lead to check, and I check the parts I lean on.

2.1 A string, a square, and a drowning nobody can place

Start with the oldest one, and start by doing it. Take a string. A guitar string, a rubber band over an open box, anything that twangs. Pluck it. Now press it down exactly at the middle and pluck half. That's the same note, an octave up, the one that makes a round sound like a round. Press at a third of the way along and pluck the long side: that's the fifth, one of the handful of intervals a bugle can play at all, which is why you hear it all through bugle calls. Half. Two thirds. Small whole numbers, and your ear says yes before your brain does. About twenty-five hundred years ago, in the south of Italy, the followers of a man named Pythagoras are credited with finding exactly that, and it's as true in your kitchen tonight as it was on their hillside.

Here's what they did with it, as best anyone can tell. Nothing survives that's surely from Pythagoras's own hand. The earliest fragments are from a later member, Philolaus, and scholars argue over those. Mostly what's left is Aristotle, one to two centuries later, writing in his *Metaphysics* about them the way you'd write about the neighbours' religion, and he says they took the elements of numbers to be the elements of all things: everything is number. Not that everything can be counted. That the parts of numbers are the parts of all things.

Stop and ask what I asked. What does that even mean? Is a chair a number? Does a chair have a number tucked inside it somewhere? Aristotle doesn't say, and he's reporting a belief he doesn't hold. So don't take it on his word, or theirs, or mine. But before you push on it, know what it was holding up, because you can't test a load-bearing wall without knowing what sits on it. These weren't people doing math for fun on weekends. They were a brotherhood, with rules about what you could eat and when you had to keep quiet, and that plucked string was their scripture. Halve the string, hear the octave: to them that was proof that under the mess of the world there were whole numbers in clean ratios, that the cosmos was a chord and they'd heard it. "Everything is number" wasn't a slogan. It was what they'd organized their lives around. Which is exactly the kind of sentence you should be suspicious of, mine included.

Now push, and push where they loved most: the square. Draw one, with a side one unit long, any unit you like, and draw the line from one corner to the opposite corner. Call that the diagonal. If everything is number, then the diagonal is some whole number of parts long when the side is some whole number of parts, so the diagonal divided by the side is a fraction, a whole number over a whole number. Cancel anything the top and bottom share, the way four-sixths becomes two-thirds,

until they have no common factor left. I'll call the top p and the bottom q, and the letters are purely so I don't have to keep writing "the top" and "the bottom."

You need one fact about squares, and it's the one Pythagoras is famous for. The square built on the diagonal has exactly twice the area of the square built on the side. If you want to see it for yourself, cut out two identical squares, cut the second one along both diagonals into four triangles, and lay one triangle against each side of the first, its long edge on that side. The outline is the square on the first one's diagonal, and it's made of exactly two of the small squares; it's a five-minute job with scissors. So p times p is twice q times q. People write "times itself" as a small raised two, so that's $p^2 = 2q^2$, and that is the only piece of notation in this argument.

Twice anything is even. So p times p is even. An odd number times itself is always odd (three times three, five times five, try a couple), so p itself must be even: it's two times something, and I'll call that something r. Put it back in. Two r times two r is four times r times r, and that equals twice q times q. Halve both sides: two times r times r equals q times q. So q times q is even, so q is even too, by the same step as before. But you already cancelled everything the top and bottom had in common, so they can't both be even. The argument has eaten itself, and when an argument eats itself the thing you supposed at the start is false. There is no such fraction. The diagonal of a square can't be named by any ratio of whole numbers, ever. Aristotle reaches for this very argument in his *Prior Analytics* as his example of proof by contradiction (suppose the diagonal can be measured, and odd numbers come out equal to even ones), and you just did it with a pencil.

There's a story that a member of the brotherhood named Hippasus let this secret out, and his brothers took him to sea and drowned him for it. I went and checked, because it's a good story and good stories are the ones to check. The oldest drownings I could find are in Iamblichus, writing around AD 300, seven or eight hundred years after Hippasus. In Iamblichus the man drowned by name is Hippasus, and what he gave away was how to fit twelve pentagons together into a ball, the dodecahedron. A few pages on, Iamblichus tells it again about whoever first let out the incommensurable, and that time he names nobody. Carl Huffman, who wrote the Stanford Encyclopedia of Philosophy's entry on Pythagoreanism, follows the historian Walter Burkert there and finds no ancient writer who connects Hippasus to this discovery at all. So the version with Hippasus drowned for the diagonal looks like two old stories stitched together later. It's a legend. But notice that the legend exists. Somebody, long after, felt that a man *should* have been drowned for this. That's how much was riding on it. To whoever stitched the story together, the diagonal wasn't a math problem. It was the end of the world.

So the claim is dead, and you killed it. I only handed you the pencil. Now the question that matters: what's left? Look at your square again. The diagonal is still

there. It still stands in a perfectly definite relation to the side; you drew it, and nobody drowned. That relation was real before anyone tried to put a number on it, and it stayed real after the number failed. The relation came first. The number, when there is one, is a name for the relation, and sometimes there isn't a name. I'm going to keep that, and I want you to keep it too, because you're going to watch the same move get made again and again as this book goes on: somebody says "everything is X," and it's true enough to do real work with, and it sounds right when you pluck it, and then somebody draws the diagonal. I don't know yet whether the thing I built survives its own diagonal. That's a real question, and I'll get to it.

2.2 Shapes poured into nothing

About a century and a half after the brotherhood, Plato, in a dialogue called the *Timaeus*, tried to build the whole physical world out of geometry. Fire, air, water and earth were four regular solids (the pyramid, the eight-faced one, the twenty-faced one, and the cube), and the faces of those solids were built from exactly two kinds of right triangle. You can make one of each in a minute: cut a square from corner to corner for the first, and cut an equilateral triangle from a corner to the middle of the opposite side for the second. Everything else, in Plato's story, was a receptacle, a featureless medium that takes on shape the way wax takes a seal. Structure, for Plato, is shape laid into something that has no shape of its own.

Why did he care? He wanted a world made on purpose, by a craftsman working from a plan, and triangles were the plan. Is it true? No, but not for the cheap reason. You can't refute a claim about what things are made of at the smallest scale by picking one up; the smallest scale is exactly where your hands don't reach, and Plato knew that. What sank it is that it made no prediction anyone could check, and the people who eventually did reach that scale, two thousand years on, found atoms, and atoms aren't triangles. The idea was a guess with no way to lose, until it lost. But push on it and two things survive. One: structure is something *laid in*, and the stuff it's laid into doesn't carry it by itself. That's a real idea, and I'd ask you to keep it loosely for now, because later I'll have to ask whether the state a machine holds is the wax or the seal. Two: Plato broke his own claim, in a later dialogue called the *Parmenides*, with an objection that Aristotle later nicknamed the third man, and it's the most useful mistake in this section. Take all the large things. Plato says they share one thing, Largeness. But Largeness is itself large, so now there's a further thing that all the large things *and* Largeness share, and that one is large too, and it never stops. Try it with any property you like; it runs away every time. The lesson, and I think it's right though Plato never quite accepted it himself: the structure of a collection is not one more member of the collection. The plan isn't a brick. Hold on to that one. It'll come back in a place you won't expect.

2.3 Prove it from what you and I both accept

Around 300 BC, in Alexandria, Euclid wrote the *Elements*, and the thing to take from it is not the geometry. It's the method, and the method is the whole reason I trust anything in this book, so give me a paragraph on it. He starts with definitions ("a point is that which has no part"), five postulates (things he asks you to grant, like "to draw a straight line from any point to any point"), and five common notions (things nobody argues with, like "things which are equal to the same thing are also equal to one another"). Then he derives everything else, four hundred and sixty-five propositions, from only those, in order, each meant to use nothing but what came before (he sometimes leans on things he never stated, and later geometers filled those in). Not "trust me." Not "it's well known." Here are the rules you and I both accept; now watch. If you've ever been in an argument that ended with "well, that's just your opinion," Euclid is the man who found a way out of that room, and I'm not embarrassed to say I learned the way out from someone dead for two thousand three hundred years. Four hundred and sixty-five times he does the same trick, and the trick is everything.

And then there's the one rule Euclid seems almost embarrassed by, and it's a hole worth falling into. His fifth postulate, the one about parallel lines, is long and awkward next to the others, and for two thousand years people tried to prove it from the rest so it wouldn't have to be assumed. Proclus, the fifth-century philosopher who wrote a commentary on Euclid's first book, tried. Mathematicians in Persia and the Arab world tried. In 1733 an Italian priest named Girolamo Saccheri spent a whole book assuming it was false so he could watch the contradiction arrive, and he got pages deep into what is now a perfectly good geometry before declaring that it was repugnant and stopping. A century later three people, separately, took the step he wouldn't: drop the postulate, and what you get is other geometries that hold up just as well. Nonsense never arrives. Farkas Bolyai begged his son János to stay away from the problem, warning that it had swallowed years of his own life; the son went in anyway and came out with one of those geometries. Here's the part I keep: they found something real by refusing to believe one of the rules everyone accepted, and checking what happened without it. That's the push, two thousand years long.

What is structure, to Euclid? Whatever the starting rules force. And when are two shapes the same? Here's the one you can do: cut out two paper triangles and lay one on the other. If they coincide, they're equal. That's his fourth common notion, and it's how he proves his fourth proposition, by picking up one triangle and setting it down on another. Sameness is coinciding under a move. Keep that; it's about to get very general, and I didn't see that coming either.

2.4 The statue and the bronze

Aristotle, Plato's student, pulled the forms down out of heaven and put them inside things, which was a bit rude to his teacher and also, I think, correct. His example, from the *Metaphysics*: a bronze statue is bronze plus a shape, and being made of bronze is no part of what it is to be a statue. The same shape in marble makes a different statue with the same form. Matter and form, and form is what the thing *is*.

Push on it. Is that true? Take anything on your desk. Could the same thing be made of something else? A coffee cup, sure. A sandwich, no; a sandwich made of wood is a sculpture of a sandwich, and if you've ever tried to eat one you know the difference matters. So the claim doesn't hold everywhere. It holds exactly where the material was never doing the work, and that's a sharper thing than Aristotle said: structure is what survives a change of material, in the places where the material didn't matter. Why do I care? Because this whole book bets that the record is a form. The material shouldn't matter: the same rule whether it sits on a tape, on a table of counters or on a lambda term. I don't get to just say so. Section 10 is where that bet gets checked, over any base at all, and Part IV builds it once, on the smallest base I could find (Section 16). Aristotle is the one who told me what the bet means.

2.5 Where things are, without numbers

In 1679 Gottfried Leibniz, who co-invented calculus and had opinions about everything, wrote to the physicist Christiaan Huygens asking for something that didn't exist: a way to reason about *where* things are directly, the way algebra reasons about *how much*, without ever laying down coordinates. He called it the analysis of situation. Try it yourself. Describe where the furniture is in the room you're sitting in without using a single number. "The couch is against the wall. The table is between the couch and the window. The lamp is next to the chair." You just did it, and what you used was relations between places and nothing else. Huygens wrote back that November and was not impressed. It took roughly two hundred years for Leibniz's idea to become a field, and the field is called topology, and Huygens is remembered for other things.

What survives the push is the same thing that survived the square, coming at you from the other side. Position is a relation among places. The numbers you lay on top are a convenience, and they're a convenience you can take away and still have the room. I'll be honest about why this one landed for me: the thing I built keeps track of which things belong together, and I spent a long time thinking that was a list of numbers before I understood it was a relation that the numbers were merely naming. Leibniz had that in 1679. Huygens told him no. I'd like to think I'd have sided with Leibniz. I probably would have sided with the famous guy.

2.6 Naming a thing by saying when two are the same

In 1884, in a short book called *The Foundations of Arithmetic*, Gottlob Frege noticed something you can check in your head right now. "Line *a* is parallel to line *b*" and "the direction of *a* is the direction of *b*" say exactly the same thing. There is no fact in one that isn't in the other. But the second sentence has a new object in it, a *direction*, that the first one doesn't. Where did it come from? From the sameness. Once you can say when two things count as the same, you've earned the right to name the thing they share. Frege used exactly this move to say what a number is: the number of things in a pile is whatever every pile you can match up with it, one to one, has in common. That's a residue with teeth: a sameness relation lets you name a new object.

Now push, because Frege pushed too hard, and it cost him. In his big system he also assumed that any condition you can state picks out a collection, the collection of everything that satisfies it. In June of 1902, a young British philosopher and logician, Bertrand Russell, wrote him a short letter. Consider the collection of all collections that are not members of themselves. Is it a member of itself? If it is, then by its own rule it isn't. If it isn't, then it qualifies, so it is. Do it in your head; it flips every time you look at it, like one of those drawings that's a vase and then two faces and then a vase. Frege's floor had a hole in it, and he knew it the day he opened the letter, and his second volume was already at the printer. The repair came from Ernst Zermelo in 1908 and it's the one mathematics still stands on: you may only cut a collection out of a collection you were *already given*. You don't get to sort "everything." You sort a pile. I'll tell you now that this rule shows up again, later, in a place where I had to obey it or fall through the same hole, and I'll point at it when the book gets there.

Frege's reply is worth hearing, because it's the best example I know of how to lose. He wrote back within days. He didn't hide it or argue around it. He said, more or less, that arithmetic was tottering, and in an appendix to the volume that was about to come out he wrote that hardly anything worse can happen to a scientific writer than having a foundation give way right after the work is finished. Then he tried to fix it. If I'm wrong about something in this book, and I'd bet I am somewhere, that's how I'd like to take it.

2.7 Choose the changes, keep what survives

By 1872 there were too many geometries. Euclid's, and ones where parallel lines meet, and ones with no such thing as distance, and nobody could say what made them all geometry and kept them from being a pile of unrelated hobbies. Felix Klein, twenty-three years old and taking up a professorship at Erlangen, published an answer in a program paper that people still call the Erlangen program. Pick a set of

allowed changes: rotations, stretches, whatever you decide. Then a geometry is the study of the properties that *stay the same* under those changes. Pick a different set of changes and you get a different geometry on the very same page. Euclid's, in Klein's reading, is the one where you may slide, turn and flip a shape and scale it evenly, and stretching it more one way than another is off limits. And here's the part I'd underline if this were my book, which it is: in Klein's scheme any group of changes can define a geometry. The choice is free. Nobody tells you which changes to allow.

You can do this one at your desk. Take any drawing and ask what's still true after you turn it upside down (lengths, angles), after you stretch it (lengths are gone by then; which lines cross which is still there), after you crumple it (almost nothing but what touches what). Three answers, three structures, one drawing. The residue: structure is what survives a chosen set of changes, and "chosen" is doing real work in that sentence. There's no structure of a thing from nowhere. You have to say what you're allowed to do to it first. I didn't know that when I started, and I'd been saying "the structure of" things for years.

Klein's move has a descendant in logic, and it goes all the way. In a lecture in London in 1966, published only in 1986, three years after his death, the Polish-American logician Alfred Tarski asked what makes a notion logical, and answered with Klein's recipe at its far end: allow every one-to-one transformation of the world onto itself, and call logical the notions that survive all of them. The more changes you allow, the fewer notions are left standing, and with every change allowed what's left is logic. That's the sentence above, "structure is what survives a chosen set of changes," with the choice made as large as it can be.

2.8 An invariance comes with a bill

In 1918 Emmy Noether, who at the time wasn't allowed to qualify for a university teaching post because she was a woman, and for a while taught courses advertised under David Hilbert's name, proved something that physicists still treat as close to holy. Take a physical system whose laws can be written the way most of physics is written, as "of all the paths the system could take, it takes one that makes a certain total stationary" (the total is called the action, and stationary means a small change to the path doesn't change the total to first order; the path needn't make it smallest, and that's all I'll say about it here). For such a system, every continuous symmetry that leaves the action unchanged comes with a quantity that is conserved. If the action doesn't change when you shift everything in space, momentum is conserved. If it doesn't change when you shift everything in time, energy is. A symmetry isn't decoration. It has a consequence downstream that you can measure with instruments. I can't hand you this one to do at the table; the proof needs calculus I'd have to build

first and I'd lose you and probably me, and the "written as an action" condition is doing real work that I'm not going to unpack here. What I can hand you is the shape. Structure, once it's really there and the setting is the right kind, has consequences you can measure. I'll admit that's the sentence that made me think a machine could carry a bill for structure in the first place, and I'll also admit I don't yet know whether that hope was reasonable. Noether makes it reasonable to ask. She doesn't make it true, and nothing I build in this book is a system with an action.

2.9 All of mathematics as structures

Starting in the 1930s, a group of young French mathematicians began publishing a vast treatise under one made-up name, Nicolas Bourbaki, as if he were a single person. He wasn't. In 1948 he also applied for membership in the American Mathematical Society, and the application was quietly turned down, which I find comforting. (The historian Michael Barany has gone through that application, a second one a year later, and the letters they set off.) Their goal was to rebuild all of mathematics on one idea: a structure is a set of elements "whose nature has not been specified," together with one or more relations on them that satisfy stated axioms. They believed three kinds of structure were the mothers of all the rest: algebraic, order, and topological.

Push on it, and here's what the historian Leo Corry found when he went through the entire treatise looking for the place where this formal definition of structure actually does any work: it "plays no actual role in the presentation of theories within the treatise." Outside the chapter that defines it, almost nowhere. And I found no argument in their writing for why the mother structures number three. And the chapter on category theory, which they promised their readers, never appeared; Corry is where I found that too, and Ralf Krömer's 2006 history of Bourbaki and categories in the 1950s follows the attempt. Why did the definition sit idle? Here's my reading: the thing their definition left out was the *maps*, the ways of carrying a structure from one set to another, and the maps were where the weight was. So I keep two things from Bourbaki. The right thing: relations on elements whose nature doesn't matter, which is Aristotle's bronze in different words. And the warning: a structure stated without its maps won't carry the work. I learned that one the slow way, and the origin story you just read tells it.

Philosophers of mathematics took the same idea further and gave it a name, structuralism. In 1965 the philosopher Paul Benacerraf pointed out that the whole numbers can be built out of sets in two equally good ways that disagree about which set the number three is, and concluded that numbers aren't objects at all: what arithmetic is about is the pattern, and anything standing in the right relations to everything else can play the part of three. The philosophers Michael Resnik

(*Mathematics as a Science of Patterns*) and Stewart Shapiro (*Philosophy of Mathematics: Structure and Ontology*), both in 1997, made the patterns themselves what mathematics is about. The philosopher Geoffrey Hellman (*Mathematics without Numbers*, 1989) kept the talk of patterns and did without the patterns, reading each statement as a claim about what would hold in any system with the right structure, if there were one. The logician Steve Awodey (1996) argued that category theory says it best, a structure being what its maps keep, which is where this section goes next. And the philosopher of science James Ladyman (1998) carried the idea into physics: what a successful theory gets right about the world is its structure. This section's thesis, that structure is the relations and the things in them don't matter, is theirs. None of the formal results takes their arguments as a premise, and the bibliography lists them under the same heading as Bourbaki.

2.10 A set with some named relations

What mathematical logic eventually settled on, with the logician Alfred Tarski giving the field its name, "the theory of models," in 1954, is almost boring, and the boring ones are the ones that last. A structure is a set, called the domain, plus whatever relations and functions a chosen vocabulary names on it. Pick the vocabulary (Klein again: you choose), and a structure is anything that gives those names a meaning. Now I want to say "the smallest one," and the word "smallest" has no meaning until I say what I'm measuring, so here is the ruler: count the vocabulary. One name, taking one thing at a time, with two possible answers. That's a domain with one yes-or-no property named on it. One question, put to each thing. Two honest limits, and I'll give them to you now, so you don't have to find them. A vocabulary with one named constant ("this one, here") is exactly as small by my ruler and it doesn't sort anything, so the claim is "smallest that sorts." Plain "smallest" would be false. And one question only splits the domain into two groups when some things answer yes and some answer no; if they all agree there's one group and nothing was sorted. With those limits said, and with the ruler said: by the count of names, the smallest structure that sorts anything is one question. Someone with a different ruler (fewest symbols to write down, say, or least information) might land somewhere else, and I'd want to see where. Keep it. It's about to turn into the whole section.

2.11 The structure is in the arrows

This is the one I met first, backwards. I'd built it by feel, in a rendering engine, as "digital DNA," and only went looking for its name after the thing already ran. In 1945 two mathematicians, Samuel Eilenberg and Saunders Mac Lane, published a paper that founded a field now called category theory, and they said in plain words that it

was a continuation of Klein, which I'd have loved to know at the time. The move: stop asking what a thing is, and ask how it's carried. A category is some objects and some arrows between them, where arrows can be followed one after another and every object has a do-nothing arrow to itself, plus two rules that make the following dependable: following three arrows in a row gives the same result whether you pair up the first two or the last two, and the do-nothing arrow really does nothing when you tack it onto either end. That's the whole definition. Two rules, and on the page they look like nothing. The two rules are the part people skip, so slow down on them for a second, because each one is there for a reason you can see.

Picture three dots in a row, A, B and C, an arrow f hopping from the first to the second and an arrow g from the second to the third. Composition says there's an arrow taking the whole stride from A to C in one, g after f. Without it, every multi-hop path has to be named on its own, and the structure falls apart into a pile of unconnected pairs, a phone book without a search function. Associativity says that with a third arrow h on the end, pairing up the first two and then adding h gives the same arrow as adding h to the last two first. Both spell the same three-hop path, and if they came out different, two people who built the same chain in different orders would hold two different arrows, which is a bug: the chain is the thing, and how you assembled it isn't. The identity laws say that tacking the do-nothing loop onto either end of an arrow gives back the arrow. If the loop changed things, it wouldn't be the loop. Six terms, objects, arrows, composition, identities, associativity, identity laws, and that's all of Eilenberg and Mac Lane's definition.

Composing relations is older than the definition, too. Who put a name on that move first? I went looking. Augustus De Morgan, writing on the logic of relations around 1860, took the "is" out of the old syllogism, put a general relation in its place, and composed two relations to get the conclusion. The example that gets quoted from that tradition is "x is the lover of a servant of y," which is relational composition with better gossip. When exactly he read that paper to the Cambridge Philosophical Society, and on which pages it was printed, the sources disagree, which is a small reminder that history is where you start digging. Then in 1870 Charles Sanders Peirce, the same Peirce from the opening chapter, carried George Boole's algebra of logic (the arithmetic of true and false that Part IV comes back to, along with the story of Boole himself, a largely self-taught schoolmaster) over to relations, with composition as one of its basic operations, and he's usually counted as the real founder of the theory. So following one arrow after another is older than computers, older than categories, older than Turing's clerk: a nineteenth-century logician's product of relations.

And where did they get the word? Not from mathematics. They borrowed *category* from the philosophers, from Aristotle and Kant, and *functor* (a map from one whole

category to another) from the philosopher Rudolf Carnap, and then gave both words exact meanings none of those philosophers had in mind. So Aristotle walks back into this chapter without being asked: the man who put the form inside the bronze lent his word to the field that keeps track of forms by their arrows. Does that change the laws? No. It's a reminder that a word can start in philosophy and arrive in mathematics with a precise meaning bolted on, which is the thing I've been trying to do with *established* since the first page.

What a thing *is* is how it maps. For now the residue is the one Bourbaki missed: structure lives in the maps. Nothing later in this book needs more category theory than that sentence, and the latch of Section 10, where a record is pinned down by the maps between a machine and the base it extends, is where the sentence does its work.

2.12 Structure is the part that compresses

Play this one with me. I write "th" and you guess the next letter. You said "e," and you were right. Now "the q": you said "u." Congratulations, you just measured the structure of English. In 1948 Claude Shannon measured it properly, in a paper that founded information theory, and found that about half of written English is "determined by the structure of the language." It's the reason a zip file of a long letter is so much smaller than the letter. Be careful what that says. It does not say that every other letter is fixed by the ones before it; you just guessed "e" after "th" and you could have been wrong (think "than"). It says that, on average and over long stretches, about half of what's written could be rebuilt from the rest: the text is roughly twice as long as the choices in it. And he said why that matters in one line that I'd frame if I framed things: the structure "allows one to make a saving in time" (he added "or channel capacity"). The more you can predict, the less you have to read.

Thirty years later Jorma Rissanen, a Finnish information theorist at IBM's research lab in San Jose, turned that saving into a price. His idea is called the minimum description length principle, and it says: the best model of some data is the one that makes the total description shortest, where the total is the length of the model *plus* the length of the data once the model has explained what it can. A model is paid for in bits. It's worth stating only if it saves more than it costs. Peter Grünwald, a statistician who leads a machine learning group at CWI, the Dutch national institute for mathematics and computer science in Amsterdam, in a tutorial you can find online, adds the footnote that keeps this honest: the price of a model depends on the code you chose to write it in, so the price itself needs a principle behind it, and nobody hands you that principle for free.

That's a lot of "length of" to swallow in one paragraph, so do one with me, with numbers small enough to check on the back of an envelope. Every number in it comes from a code I'm about to make up on the spot, and I'll say every rule of the code out loud, because Grünwald's footnote is the whole point: change the rules and every number moves.

Here's the data. A row of thirty-two digits, each a 0 or a 1, and you and I agree in advance that the row is thirty-two long, so nobody has to spend anything saying so:

```
0101010101010101 0101010101010101
```

(The space is only there to rest your eyes.) Now the code. Every description starts with one digit that says which kind of description follows: 0 means "here's the row, written out," 1 means "here's a pattern, then the exceptions to it." Each digit is one yes-or-no answer, one bit, which is what Shannon's unit is. Written out, the row costs that first digit plus the thirty-two digits themselves: thirty-three bits, and there's no model in it at all. A pattern description goes like this. First five digits giving the length of a little block, anything from 1 to 32, because five yes-or-no answers can pick out one of thirty-two lengths. Then the block itself. The rule is "repeat the block until you have thirty-two digits." Then five digits for how many places the row disagrees with the pattern (up to thirty-one, which is plenty here), and five more digits for each of those places, saying which of the thirty-two it is.

Do it for the row above. The block is `01`, two digits long. So the model costs 1 (which kind) plus 5 (the block's length) plus 2 (the block): eight bits. Then what's left to say about the data once the model has said its piece: the row disagrees with "repeat 01" nowhere, so that's five digits saying "zero exceptions" and nothing else. Five bits. Total, thirteen, against thirty-three for writing it out. The pattern pays for itself, with twenty bits to spare. That's Rissanen's sentence happening on paper: a model is worth stating when it saves more than it costs.

Now spoil the row. Flip two of its digits anywhere you like. The model doesn't change, eight bits, but now the leftover says "two exceptions, here and here": five plus five plus five, fifteen. Total twenty-three. Still cheaper than thirty-three; the pattern still pays, it just pays less, because the data has more to say that the model can't. Flip four, and the total is $8 + 5 + 4 \times 5 = 33$, a dead tie. Flip five and it's thirty-eight, and now writing the row out is cheaper than explaining it, and by Rissanen's rule you shouldn't state the pattern at all, even though it's still sort of there if you squint. So "is there structure here?" has turned into a question with a number on each side, and a break-even point you can find yourself: somewhere between four and five spoiled digits, in this code.

And now change the code, because I promised you'd see the numbers move. Say I'd used six digits for each exception's place (wasteful, but nobody stopped me). Then four spoiled digits cost $8 + 5 + 4 \times 6 = 37$, and the tie is gone; the pattern stopped paying one spoiled digit sooner. Same row, same pattern, different verdict, and the only thing that changed was a convention I chose. That's Grünwald's footnote, and it's why I won't let you take any of those numbers anywhere else. They're the price of a description in one made-up code, a length of a written-down message, and nothing more. They are not the charge I opened this book with. That charge, when the book finally states it, is on one step a machine takes. How long anybody's description is doesn't enter it, and if the two ever turn out to be related, that has to be shown. Both of them being counted in whole numbers doesn't get to stand in for the showing.

Now push, because this is where "structure means information" gets born, and it's wrong, and it's an easy thing to believe. Flip a fair coin two hundred times and write down the string. Almost always, there's no shorter way to say that string than to say the whole thing. You can watch it happen in the little code above, cut to thirty-two digits: a coin-flip row disagrees with "repeat 01" in about sixteen places, which would cost eighty-odd bits to list, and even the one block guaranteed to fit with no exceptions, the whole row used as its own block, costs $1 + 5 + 32 + 5 = 43$. The written-out row at thirty-three wins, and no pattern earns its keep. Nikolai Vereshchagin, a mathematician at Moscow State University, and Paul Vitányi, a computer scientist at CWI in Amsterdam, in a 2004 paper on exactly this question, put it without mercy: every bit of a string like that is "meaningless information, random noise." Maximum information. Zero structure. So in this sense structure is the part that compresses, the part where the "th" tells you about the "e," and most information and most structure are different things. Hold that one tightly, because if you built something that charged for information, you'd be charging for the wrong thing, and for a while I wasn't sure which one I'd built.

And now notice that the word just changed meaning on you, and on me. I'm pointing at it so it doesn't slide past. Shannon and Rissanen and Vereshchagin mean *regularity*: a pattern that lets you say something shorter. Tarski, a few pages back, and Paul Halmos, a Hungarian-American mathematician whose short textbook on sets is coming up, mean a set with named relations on it, and those two are not the same thing. Here's the test, and you can run it: number a thousand objects, flip a coin for each, heads goes in group one, tails in group two. That's a perfectly good sorting in the set theorist's sense, every object in exactly one group. And the list of who went where is a thousand coin flips, which is to say it compresses about as well as a thousand coin flips, which is to say not at all. Feed it to a code built like the little one, with the fields widened to fit a thousand, and you get the coin-flip row again,

only longer: no block earns its keep, the written-out list wins, and by Rissanen's rule there's no pattern worth stating. By the set theorist's rule, the sorting *is* the structure, and it's every bit as much a sorting as "evens in one group, odds in the other," which compresses to almost nothing. Same kind of object, wildly different description lengths. So the two meanings don't even rank things the same way: one asks "is there a sorting?" and the answer is yes for both; the other asks "does a short rule say who went where?" and the answer is yes for evens and odds and no for the coins. Structure in the set theorist's sense, none in Rissanen's. I'll keep both meanings, and I'll try to say which one I'm using, and when I slip you'll know what to look for.

2.13 The four parts of every computer

In 1936 Alan Turing described the machine that every computer you've ever touched is a version of, and he did it before any of them existed. It's the clerk with the pencil from the opening chapter, written down as a machine. Write down its parts; there are four. A tape, divided into squares. A head that reads one square at a time and can write on it and move left or right. A finite list of states the head can be in. A table of rules: in this state, reading this symbol, do this. That's all, and two different things are true about it that get run together. Every other recipe anyone has proposed as a general formalization of "a procedure," before or since, computes exactly what this one does; those are theorems, proved one pairing at a time, and Turing himself sketched one of them, the pairing with the logician Alonzo Church's lambda calculus, in an appendix to the 1936 paper. That it also captures everything a person could ever carry out by rote, everything "computable" in the plain-English sense, is a claim, called the Church-Turing thesis, and nobody has proved it, because the plain-English side isn't a mathematical object. Turing himself kept those two apart in the 1936 paper, and so will I. There's a third claim that gets stacked on top, and it's a different kind again: the physical Church-Turing thesis, that every device you could build out of real stuff computes no more than this machine does. That one is a claim about physics. No mathematical proof can settle it. Nobody has built a device that breaks it. Your laptop, your phone, every server in every data center compute nothing a Turing machine can't; what differs between them is speed, space, energy, and cost. Hold on to that last word. Cost is where this book lives.

Now do the thing I did, and do it honestly. Look through those four parts for a place that says "these squares belong together." I couldn't find one. Any grouping has to live in the table, as a rule, or on the tape, as marks, which is to say as bytes like any other bytes. Maybe you can find a slot I missed. I'd want to know. I looked for a long time and I kept not finding it, and not finding something is weaker than proving it isn't there, so I'll be careful about what I claim from that.

In 1972 David Parnas, a software engineer, wrote a short paper that every programmer should be made to read and most haven't, on how to cut a program into modules. He took one small program and cut it two ways. Both worked. But one was easy to change later and the other wasn't, and the easy one could be "much less efficient" if talking between modules cost anything. His point: a module is "a responsibility assignment," a decision about who answers for what, and how you cut decides what every later change costs. If you've ever maintained someone else's code, you've paid Parnas's bill, probably at two in the morning. What survives the push: a division is a commitment. It changes what the work costs. It is no caption, no label stuck on afterwards. I'd like to say the thing I built understood that from the beginning. It didn't. I did, eventually.

2.14 Laying them all down

Now lay every one of those on top of the others and see what shows through. Here are the seven things I think I see. They're my overlay, the places where I see the sources agreeing. The sources never proved it together; some of them never met, and some of them disagree (Bourbaki left out the maps that Eilenberg and Mac Lane built everything on; Shannon's structure and Tarski's are two different words, as I just showed you). You've pushed on each source yourself by now, so you're in a position to tell me if I'm seeing things.

A list of seven slogans would be easy to nod at and impossible to check, so I'm going to carry one thing through all seven and let you watch each one add to the one before it. The thing is the square you drew for the Pythagoreans, side, diagonal and all. Draw a few more on the same sheet: one small, one big, the small one again but tilted, and one stretched out into a long rectangle. You'll want a pile.

Structure is about relations among things. The things themselves aren't it. The diagonal, the bronze, Leibniz's room, Bourbaki's unspecified elements, Tarski's named relations, the arrows. On your sheet, what makes a square a square was never the ink. It's that the four sides match and the corners are square, and that the diagonal stands to the side the way it does whether or not any number names it. Every one of those is a relation among parts. But on its own this first thing is too loose to use, because a sheet of paper is drowning in relations: this square is left of that one, this one is nearer the coffee ring. Which relations count?

Structure is what stays the same under a chosen set of changes. Euclid's triangles laid together, Klein's allowed moves, Noether's symmetry, Frege's direction that every parallel line shares. That's the answer the first thing was missing. The relations that count are the ones that survive the moves you've decided to allow. Slide the small square across the sheet, turn it, and the diagonal still stands to the side exactly

as before; "left of the coffee ring" doesn't make it across. The second thing is a filter on the first.

The same structure can be carried by different stuff. The statue in marble. Two sets that answer the same vocabulary the same way. This is the second thing again with one more change allowed: swap the material. Cut the square out of cardboard, or lay it out in string on the table, and if the relations survive, it's the same square in every way you agreed to care about. Aristotle's bronze is one more thing you agreed not to look at, and the wooden sandwich is what happens when the material was never on that list.

The first act of structure is a sameness that sorts a pile you were already given. Euclid's first common notion, Frege's direction, Tarski's one question, and Russell's letter, with Zermelo's repair that you must start from a pile. This is where the overlay stopped being a list for me and turned into a shape, so slow down here with me. Look at your pile of drawings and say: two of them count as the same when an allowed move carries one exactly onto the other. Watch what happens to the pile. It falls into groups. The small square and its tilted twin go together; the big one sits on its own; the rectangle sits on its own. And it falls cleanly, nothing half in one group and half in another, for a reason you can check. Klein's allowed moves always come with three guarantees: doing nothing counts as a move, every move can be undone, and two moves in a row is again a move. Those three are, in different clothes, "everything is the same as itself," "if this matches that, then that matches this," and "if this matches that and that matches a third, then this matches the third." Put the last two together and you get Euclid's first common notion, "things which are equal to the same thing are also equal to one another." So the second thing, sameness under chosen moves, *is* a sorting of the pile the moment you ask which drawings it calls the same. Frege did it with lines and parallels. Tarski's one question does it in the crudest way there is, by cutting the pile in two. And the pile has to be there first, because a sameness has to be a sameness among something; that's Russell's letter, and Zermelo's repair.

Adding structure rules possibilities out. Every one of Euclid's postulates is a door closed. Half of English is already decided. A conserved quantity limits where a system can go. On your sheet: once the small square and its tilted twin are in one group, you've given up the right to answer any question the moves respect differently for the two of them. Tell me the side of one and the diagonal of the other isn't an open question any more. A sorting is a bundle of doors closed at once.

Held structure gets used without re-deriving it. Shannon's saving in time. Rissanen's compression. Parnas's one-module-at-a-time. Leibniz's analysis of situation, reasoning about where things are straight from their relations, with no coordinates laid down first. This is the fifth thing paying out. The doors you closed are doors

you never have to check again: know which group a drawing is in and you know everything the group shares without getting the ruler out, the same way the row of `01` never needed writing out. With one catch that I'll need a whole section for: it only pays if you kept the sorting somewhere you can read it back.

And structure is always relative to a chosen frame. Klein's group. The vocabulary. The code you compress against. Parnas's criteria. Nobody gets to say "the" structure of a thing without saying from where, and I do it constantly, so watch me for it. On your sheet: allow stretching as one of the moves and the rectangle walks over and joins the squares; allow only slides and turns and it doesn't. Same pile, same ink, different groups, because I chose different moves. The groups were never in the paper. They were in the paper and the choice, together.

So here's how I think they hang together. Relations are what structure is made of. The chosen moves pick which relations count, and a change of material is just one more move. Asking which things the moves call the same sorts the pile. The sorting closes doors, and closed doors are what you get to skip later. And because the moves were chosen, all of it is relative. Read that way, the fourth thing sits in the middle: it's where the first three turn into something you can write down, and where the last three get their start.

That's my reason for starting the next subsection with a sorting, and I want to be exact about what kind of reason it is, because it's mine. None of these people said "start with a partition." Klein would tell you to start with the moves, Eilenberg and Mac Lane with the arrows, Shannon with the probabilities. What I can actually point at is narrower. In the framings about when two things are the same shape or the same kind (Euclid laying one triangle on another, Klein's moves, Frege's directions, Tarski's question) there's a moment where somebody decides what counts as the same, and that decision, written down, is a sorting of a pile. In Shannon, Rissanen and Noether I can't find that moment as cleanly, and I'm not going to bend them until I do. So the overlay doesn't prove that a sorting comes first. It shows me where the framings I trust most keep putting their first foot down, and I'm putting mine there too.

I looked for a framing that denies any of the seven and didn't find one, which could mean there isn't one or could mean I didn't look where it lives. If you know of one, I'd like to see it.

2.15 The smallest piece

Now the sentence I wanted to say at the top and had no right to until here, and even here I have to split it in two, because half of it is a textbook fact and the other half is me. The piece of structure I'm going to start from is a partition.

The textbook half: a partition and an equivalence relation are the same object. That's Paul Halmos, *Naive Set Theory*, 1960, section seven, and you could read that section before dinner. Here it is in words. Take any pile of things. An equivalence relation on it is a way of saying "these two count as the same" that behaves the way sameness should: everything is the same as itself; if a is the same as b then b is the same as a; and if a is the same as b and b the same as c, then a is the same as c. Every such relation sorts the pile into groups, the things that count as the same go together, and every sorting into groups hands you back a relation, "same group." Do one and then the other and you land where you started, in both directions. So a partition and an equivalence relation are one object seen from two sides, and that's the fourth thing on the overlay, the sorting of a given pile, now with a name and a page number.

Here it is in the exact form, since this is the first place the book uses one. A *set* is a pile, $x \in X$ means "x is in the pile X," and a *relation* on X is any rule $\sim$ that, for each pair of things, says whether $x \sim y$ holds. A few more marks show up below, and each is a word in disguise. The buckets are written S_j, with j saying which bucket (finitely many buckets in every example here, though nothing below needs that). A cap, $S_j \cap S_l$, is "what the two buckets share," and $\emptyset$ is the empty pile. And curly braces with a colon, $\{\, y \in X : y \sim x \,\}$, mean "the pile of every y in X for which $y \sim x$ holds"; the theorem writes that pile $[x]$, the things that count as the same as x.

Definition 2.1 (Equivalence relation, partition)**.** A relation $\sim$ on a set X is an *equivalence relation* when it is reflexive ($x \sim x$), symmetric ($x \sim y$ implies $y \sim x$), and transitive ($x \sim y$ and $y \sim z$ imply $x \sim z$). A *partition* of X is a collection of non-empty subsets of X, the blocks, such that every member of X lies in exactly one block. Spelled out, with the blocks written S_j: each S_j is non-empty, no two share a member ($S_j \cap S_l = \emptyset$ when $j \neq l$), and together they cover everything (every member of X lies in some S_j).

Theorem 2.1 (Sameness and sorting are one object)**.** Every equivalence relation on X determines a partition, whose blocks are the classes $[x] = \{\, y \in X : y \sim x \,\}$; every partition determines an equivalence relation, "x and y lie in the same block"; and the two constructions undo each other.

Proof. The classes are non-empty ($x \in [x]$ by reflexivity) and cover X. If z lies in $[x]$ and in $[y]$, then $z \sim x$ and $z \sim y$, so $x \sim y$ by symmetry and transitivity, and then $[x] = [y]$ by transitivity again; so the blocks don't overlap. "Same block" is plainly reflexive, symmetric, and transitive. Going relation to partition to relation returns $x \sim y$ exactly when x and y share a class, which is the original relation; going partition to relation to partition returns the original blocks. □

This is textbook mathematics (Halmos, section 7), and the book uses it as a foundation. I'm claiming nothing new in it. Checked as `nec_t_rel_of_partition_equiv`, `nec_t_blocks_of_equiv_partition`, `nec_t_rel_round_trip` and `nec_t_blocks_round_trip` (NecTPartition.v).

And the smallest structure that sorts, by my ruler in Tarski's setup, the one question, is the same object a third time. Put a yes-or-no question to every thing in the pile. The ones that say yes are one group and the ones that say no are the other. Going the other way, a two-group partition becomes a question the moment you point at one group and call it "yes." The only thing a question has that the partition doesn't is which group got the label. So the set theorist's first step and the logician's first step are the same step, and nobody has to pick a side, which is a relief, because it's an argument you can lose with yourself for a long time.

That's the textbook half. Now mine, and hold it looser, because Halmos says nothing about "smallest" or "first" and I'd be lying if I let his page number carry my sentence. In what I'm going to build, the order goes: a pile, then groups in the pile, then whatever connects one group to another, and the groups come first because a question needs something to be about and a connection needs somewhere to land. That's an order for *my* construction. It is not a law that every account of structure has to start by sorting. You could start with an ordering instead ("this one comes before that one"), or with arrows between the things themselves and no groups at all, and people do, and nothing I've shown you rules that out. So the honest sentence is: a partition is where I chose to start, it is the smallest start by the ruler I gave you, and I chose it because it's the first thing that can be right or wrong about a pile and the last thing I could take away without everything I build on it coming down. If you have a smaller start by a ruler you can state, I want to see it. It would change this book.

2.16 Buckets

So here's the smallest piece, done once, for real. Take everything in front of you and sort it into buckets: nothing in two buckets, nothing left on the floor. That's the whole of the definition. It's the thing you do with a kindergartner and a pile of blocks, and I am aware that I've just marched you past Pythagoras, Plato, Euclid, Frege and Noether to arrive at a pile of blocks. Keep the blocks anyway. They're about to cost you something.

Ten blocks. Red ones in one bucket, square ones in another, everything else in a third. Now a red square turns up, as they do, and you've got your first crisis: it belongs in two buckets and the rule says one. So this isn't a partition yet, and you have to decide which bucket wins. That decision is the whole thing. A partition is a pile of small rulings like that one, and once you've made them all, the pile is holding

a fact it wasn't holding a minute ago. The formal version says the same thing with fewer words and more squiggles, and you've already seen it: it's the definition of a partition a few pages back (Definition 2.1). Non-empty buckets, nothing in two, nothing on the floor.

Here's the part that isn't kindergarten. Why would that fact cost anything? Watch what it does to one question. Before the buckets, "are these two in the same group?" means working it out: go find the first one, go find the second one, apply the rule to each, compare. After the buckets, it's a glance: read the first one's bucket, read the second one's bucket, done. And the glance costs the same when the pile goes from ten blocks to a billion. That's Shannon's saving in time and Parnas's one-module-at-a-time, the sixth thing on the overlay, showing up in a kindergarten.

Now push on it, because that paragraph sells more than it should, and I want you to see where. The partition didn't give you the glance. A *table* gave you the glance: a written-down line for each block saying which bucket it went in. The partition is the fact; the table is one way of holding the fact so it can be read back fast, and you had to build the table, and you have to store it, and it has to be the kind of thing you can read in one look (a bucket number, say; a list of every block in the bucket would have you searching through it again). Anybody who's used a database has met this table; it's what an index is, and an index is a thing you build, store, and keep up to date, and none of that comes free. Take the table away and keep the partition and you're back to working it out. And sometimes there was never a search to save: if the rule is "evens here, odds there," you can answer "same group?" by looking at the last digit of each, no buckets needed. So here's what survives: holding a sorting in a form you can read back is worth something, and building that form cost something, and those are already two different events. Building the table, storing it, checking that it's right, and deciding to *rely* on it are four different things. The question this book is about lives in the gap between the third and the fourth, and nothing in the kindergarten has shown you yet that there's a charge in that gap. It's shown you there might be a place for one.

So who, in everything I read, says what the bill is? I found three answers that come close, and I'll be exact about how close, because this is the edge of what I know. Rolf Landauer, in 1961, and Charles Bennett after him, say that any step which takes two possible situations and makes them one, erasing a bit or merging two paths a computation could have taken, has to push some disorder out into the world, which in a real machine means heat. I mean that literally. It's the place, of all I've found, where a fact about information turns into a fact about heat, though at room temperature the amount for one bit is far too small to show up on anybody's electricity bill. In 2012 a lab in Lyon erased one bit, held in a single tiny bead, and measured the average heat it gave off coming down toward Landauer's floor as the

erasing was slowed, and other labs have found the same since. Not everyone accepts the principle in general, and I'll come back to the people who don't. Is sending blocks to buckets a step of that kind? Push, because it depends on what you keep. If you keep each block and write its bucket next to it, you've lost nothing; you can read the block right back off the pair, and Bennett's whole point in his 1973 paper was that a computation which keeps its inputs can be run without any such step at all. If instead you throw the block away and keep only the bucket label, then two different blocks can leave the same label behind, and that is a many-to-one step, the kind Landauer charges for. Both are things people call "sorting into buckets." Only the second loses a distinction. So the bucket by itself doesn't hand you a Landauer bill. The *forgetting* does, when there is forgetting, and whether a given machine forgets is a question about that machine. It has to be answered on the page, and I won't wave at it here. Rissanen says a model is paid for in the bits it takes to state, and Grünwald says the price depends on the code. Parnas says a bad cut is paid for later, at two in the morning. And that's where it stops. Rovelli, back in the opening chapter, prices a choice that sticks, and his choice is one nobody could predict, which Section 5 shows is a different act. I couldn't find anyone who says that the moment you *vouch* for a grouping, the moment you stand behind "these belong together," has a price of its own. Just arranging them that way is a different act. That could mean nobody says it. It could mean I didn't find the person who does. I honestly can't tell you which.

So that's where I am at the end of one word. Holding a sorting in readable form is worth something and cost something, forgetting where things came from costs heat, and standing behind a sorting is a different act from either, and nobody I found has priced it. (Rovelli's price is on a choice nobody could predict, and that's a different act again. And one price sits closer than any of these, which Section 3 puts beside the toll: Landauer's reset of a bit to one whatever it held, which is what raising a flag that stays up does on a machine that might already have it up. It prices the step and says nothing about vouching.) You can walk back through all of that and tell me where it breaks. Maybe there's no such bill. Maybe I'm about to spend a book chasing a charge that nobody owes, which would be an extremely me thing to do. The next two sections are me trying to find out whether that charge is real on any machine at all, mine or anybody's, and if the answer comes back no, you should stop reading there. I would have stopped writing there too.

II

The axiom

3

A record and a toll

I left you with a question I couldn't answer: something pays for the glance, and I couldn't find anyone who says the moment of vouching for a grouping has a price of its own. Before I go looking for that price in anything complicated, I want to try the opposite. Build the smallest thing that could possibly carry such a bill, on paper, with a pencil, write the bill into its rules, and see what follows from it. If nothing follows in something with five parts, nothing is going to follow in something with five thousand. I want to find that out on page one. Page four hundred is too late.

One thing about where the buckets went, because you'll notice they vanish for a while. Part I ended on vouching for a grouping, and what I'm about to build prices vouching for anything at all: a flag goes up, and the rule doesn't ask what the flag is standing behind. A sorting is the first kind of claim I care about. It isn't the only kind, and the book comes back to what a flag stands behind in Section 8, and on a machine in Section 16, where a flag stands on a checked claim. Until then nothing leans on the buckets. The floor in this section, the reason for it in the next, and the latch in Part III all work for any yes-or-no reading whatsoever, partition or no partition. When the buckets come back, in Section 8, it's as a theorem: a reading that says a grouping has been checked and committed sees the grouping only through its partition, and what it entitles is exactly the groupings that refine it (Theorem 8.5). That's what "structural" in this book's subtitle means at the level of a theorem.

But first the word, because I've got a new one and I don't trust it either. Record. Receipt. Certificate. Vouch. People have been making marks that other people are supposed to rely on for a very long time, and I wanted to know what they thought they were doing, and what it cost them, and whether any of them ever said the thing I need said. So, same deal as before: here's what I could reach, pushed on with you, and I'll tell you where the search ran out.

3.1 Clay, seals, and seven witnesses

Start a little over five thousand years ago, in the Near East, with people who counted sheep with little clay tokens. To send a count across a distance without the courier changing it, they put the tokens inside a hollow clay ball, pressed the shapes of the tokens onto the outside while the clay was soft, and sealed it. The archaeologist

Denise Schmandt-Besserat has spent a career on these, and she puts the practice at around 3300 BC; the ball is called a bulla. Here's the push, and you can do it with a cookie tin: what makes it a record is the outside. It shows what's inside, and you have to break it to get in. Opening it destroys it. Tampering shows. That's a real idea and it will show up again in a form you won't expect: a record is worth something only if it can't be silently undone.

Now law, which is where people get serious about who's allowed to say what happened. The book of Deuteronomy, chapter nineteen: nobody is convicted on the word of one witness; it takes "two or three." The Roman jurist Gaius, writing in the second century, says a will the praetor will honour is one sealed by seven witnesses. Seven. Why seven? Gaius doesn't say. The legal historian Henry Maine, in 1861, had an answer: the older Roman will was a mock sale, done in front of five witnesses, a man holding the scales, and the buyer of the estate, seven people in all, and when the ceremony fell away the seven seals kept the count. Maine's is the only account of it I found, so treat it as his. Gaius's own text does show they kept those old formalities around a will, and it isn't hard to see why anyone would: one person can lie, and a will moves a fortune, and the dead man can't object. So the record isn't the paper. It's the number of independent people who have put their own seal, their own name, their own risk, next to the claim. Try it: write "paid five dollars, signed A" on a napkin and ask a friend to forge it. Takes them ten seconds. Now require two other people to initial it at the moment of payment and each keep a copy. Your forger needs two accomplices or has to match two copies held in two other pockets. The claim didn't change. What stands behind it did.

Which brings me to seals, and the medieval historian Michael Clanchy, whose book is called *From Memory to Written Record*. England between the Norman Conquest and the early 1300s was learning to trust writing. Until then it had trusted people who remembered. The lesson was painful: seals were forged, charters were forged, and historians today have a whole craft of telling real charters from fake ones because the fakes are so many. A seal is only as good as whoever holds the stamp. Keep that one too, because in 2011 a certificate authority called DigiNotar found out, at the cost of its existence, that a mathematically perfect signature is worth exactly what the custody of the key is worth, and intruders had taken control of the machines that signed. Nine hundred years, same failure. I find that oddly comforting.

And then the obvious next question: if seals could be forged, what caught the forgeries? Not the seals. The most famous forged record in European history is the Donation of Constantine, a document in which the emperor Constantine supposedly handed rule over Rome and the western empire to the Pope. It was most likely written in the eighth century, some four hundred years after Constantine died, and it was leaned on for centuries. Around 1440 a scholar named Lorenzo Valla took it

apart by its Latin: the language belonged to a much later age, and it used words, "satraps" among them, that no Roman of Constantine's day would have used for his officials. (Valla wasn't even first. Nicholas of Cusa, a German churchman and philosopher who later became a cardinal, had already called it a forgery in 1433, in *De concordantia catholica*, a long book on how the Church and the empire ought to hang together. I know that from the reference works, and I haven't read the book. Valla's is the version people remember, which is a pattern that keeps turning up.) Look at what kind of check that was. The question wasn't "is the stamp real." It was "could these words have been written when they say they were." A record can pass the first and fail the second, which is a hint that what I keep calling one thing, a record, is doing more than one job.

3.2 Two columns and a stamp

Here's the one I'd never have guessed mattered. Bookkeeping. Around 1300, the ledger of a Florentine company called Farolfi, kept at its branch in Provence, shows the earliest complete example anyone has found of a trick merchants were working out: write every transaction twice, once where the money came from and once where it went, and the two columns have to balance. The treasurers of the city of Genoa kept their ledger for 1340 that way, and accounting historians still count it among the most famous government account books of the Middle Ages kept in double entry. And in 1494 a friar named Luca Pacioli printed the method in a mathematics book, and it spread. That's double-entry bookkeeping, and it's easy to take it for just a tidy habit.

Do it for one day. Keep two columns of every dollar that moves, and total them at night. Now have a friend secretly change one entry in one column. The totals disagree; you catch it. Now have them change the matching entry in both columns. The totals agree. You've caught nothing. Push on that and you've found what double entry actually is: a trap for slips. A careful liar walks straight through it. Did Pacioli see that? I can only get at it secondhand. The scholar Forrester, in a paper on the University of Strathclyde's accounting-history pages, reads him as reporting that in many places the books were taken to the local consul to be authenticated before a single entry went in. The stamp is a witness again, from outside. The two columns are the discipline; the stamp is the claim that someone other than you will stand behind the fact that these are the books. No other set gets that stamp. Keep both halves separate in your head. One is a method. The other is a commitment.

And then I found the place where this got turned into a motto. The Royal Society, founded in London in 1660 and still running, picked as its motto *Nullius in verba*: take nobody's word for it. (Where did they get it? From somebody's words, naturally. It's

half a line of the Roman poet Horace, from a verse letter to his patron, saying he isn't bound to swear by the words of any master. So the society that takes nobody's word for it took its motto from a poet and cut the line to fit. That's not a contradiction, and it's a nice picture of the method: borrow what's useful, say whose it was, and don't swear by it.) The historian Steven Shapin wrote a paper about what that meant in practice, and the answer is more interesting than the slogan. Robert Boyle, pumping the air out of a glass globe to see what happens to a candle or a bird, had three problems: the machine, the report, and the witnesses. He solved the third by having gentlemen present who'd sign their names, and the second by writing the report so richly that a reader who wasn't there could picture the scene and count as a witness anyway. Shapin calls that virtual witnessing, and he notes, dryly, that the air pump was very expensive, seventeenth-century big science. So there *was* a cost. It just wasn't a cost of declaring the fact. It was the cost of the instrument and of getting enough people into the room. Hold on to that distinction. The rest of the book hangs on it.

3.3 A demon, and the step that pays

Now the strangest entry, and the one that gets closest. In 1867 the physicist James Clerk Maxwell, in a letter to his friend the Scottish physicist Peter Guthrie Tait, imagined a "very observant and neat-fingered being" sitting at a tiny door between two halves of a box of gas, letting fast molecules through one way and slow ones the other, so that one side heats up and the other cools with no work done, which is supposed to be impossible. The being got called Maxwell's demon, and physicists have been arguing about it for a hundred and fifty years, which should tell you something.

In 1929 Leo Szilard, a young Hungarian physicist working in Berlin, cut the demon's box down to a single molecule, and that made information part of the accounting. The tempting move, and the one Szilard's own accounting made, is to put the whole bill on finding out which side the molecule is on. Bennett separated finding out from restoring the memory for another round: measuring onto a blank record can, in principle, be thermodynamically reversible, and resetting an unknown record is a different operation. That does not make a real measuring device free to build or run. It says that learning and resetting do not carry the same unavoidable entropy cost, and I'm about to pick a step to charge, so I need that distinction before I do.

Bennett's version of the rule, in a 2003 paper you can read, is that what must cost is "the erasure of a bit or the merging of two computation paths." Erasing, or taking two possible situations and making them one. And in the same paper he

lists copying unknown data onto a blank place to write among "the most important logically reversible operations," the kind he says can, at least in principle, be run thermodynamically reversibly. So, in the one place in all of this where a record is priced by physics, the unavoidable erasure charge belongs to *forgetting* the unknown record. Making a record carries no such charge of its own. Read that twice. It's not what I expected either.

Making a record on a blank place is one operation. Landauer's 1961 paper names another, and it's the one this book ends up charging. Take one bit whose value you don't know and set it to one, whatever it held. Landauer's name for that is "restore to one," and it's the reset he described, two possible situations made into one, the kind of step he said has to give off heat. Now raise a flag that stays up, on a machine that might be holding the flag down or might be holding it up already. Down goes to up, up stays up, and the two become one. That's restore to one. So the step the one-bit toll charges is Landauer's restore to one, and Landauer priced it, in heat, as a reset. What I can claim is the narrower thing: which premises about a machine (a finite memory, a flag that stays up) make the certifying step a reset, so that Landauer applies to it. That's the next section, and Section 21 comes back to it with the numbers.

I'll put the rest of the lineage quickly, because it all points the same way. Regulators who make brokers keep records on media that can't be rewritten (the SEC required exactly that, write once and read many, from 1997 until 2022, when it allowed an audit trail as the other way to comply) are pricing the hardware. The act itself stays free. Haber and Stornetta in 1991, the timestamping that Bitcoin's footnotes lead back to in the opening chapter, chained time-stamped records together with hashes so that changing an old entry breaks every entry after it (and Ralph Merkle's hash tree, from his 1979 Stanford thesis on secrecy and authentication, lets one short value vouch for a great many records); try it with five lines and a made-up sum, change line two, and watch every number below it move. That makes a change *visible*. It doesn't make it cost. Proof-of-work, which the computer scientists Cynthia Dwork and Moni Naor proposed in 1992 to make junk mail expensive and which Bitcoin later chained, is the one framing where a record genuinely costs, and what it charges for is a search: how many tries it took to find a lucky number. An empty block costs the same work as a full one. So the work prices the search. The moment anything becomes certified costs nothing extra. Robert Floyd and Tony Hoare, two computer scientists asking how you'd prove a program does what it's supposed to, attached claims to points in a program in 1967 and 1969 ("whenever you get here, this is true"), and the claim is backed by a proof that's paid for off the machine. And the certificates of complexity theory, which go back to Stephen Cook's 1971 work on NP, are strings designed to be cheap to check: hand me the certificate and I can confirm

the claim quickly. Whether they're also expensive to *find* is the famous open question of the field, P versus NP, and nobody knows; what's settled is only the cheap half. That's the opposite of a price on vouching: the whole design goal is that checking a claim costs nearly nothing. And from statistics, a ledger that runs the other way: Dean Foster and Robert Stine's alpha-investing (2008) runs a long string of tests against a budget of allowed false alarms, where every test spends some of the budget and every discovery, a rejection that stands, pays some back. That's the other way round from a toll on vouching: the test pays, and a discovery that stands gets paid.

3.4 What they all agree on, and who they rule out

Lay them down, and this time what shows through is not one thing. It's five different jobs that the word "record" has been doing, and I'd been letting one word carry all five. Each has a corpse next to it, and the corpses are different too.

Durability: the record can't be silently undone. The bulla, the write-once disk, the hash chain, Bennett's card that has to be wiped. The corpse is the editable log, and the company that kept its debts off the books: Enron hid its debt in side entities set up to keep it off the balance sheet, which is how the Justice Department described the scheme in 2002, and its auditor, Arthur Andersen, shredded the paper. Andersen was convicted for the shredding that year, and the Supreme Court threw the conviction out in 2005 over how the jury had been instructed; either way, those books recorded nothing. But notice this job doesn't need a witness at all. A copy will do: a hash chain nobody signed is still hard to change quietly, so long as somebody else holds the latest link, because a chain with no copy anywhere can be rewritten from end to end and recomputed, and nobody would know.

Authenticity: the record came from who it says it came from. Seals, the signature only a key-holder can make, the consul's stamp. The corpse is the forged charter and DigiNotar. This job is about custody of a stamp, and it's a different job from durability; a perfectly authentic entry can still be erased, and a perfectly durable one can still be a forgery.

Evidence: enough independent people stand behind the claim that lying is expensive. Two witnesses, seven seals, Boyle's gentlemen. It rules out anonymous assertion, and it rules out "the universe keeps the receipt": information spread into the environment is not a record anyone can read or rely on. This one is about numbers of people and their risk. Paper has nothing to do with it.

Accounting: the record checks itself for slips. The two columns. And here's a thing to keep straight: the second column is *not* an outside witness. It's you, checking you. It catches a dropped digit. A careful liar gets past it, as you saw. It belongs on this list as its own job, separate from the seven seals.

Cheap checking: a record whose correctness can be confirmed for much less than it cost to make. Cook's certificates, the signature, the hash. This one isn't universal, and I thought it was. Whether checking needs to be cheaper than making depends on who you're worried about and how much a wrong record costs you; plenty of records are checked by redoing the whole thing, and that's fine when the thing is small.

Now the thing none of them says. Not one of these jobs is pricing the act of certifying. Bennett prices merging and erasing, which a certifying step might or might not be. Proof-of-work prices a search the designer chose. Bookkeeping prices a discipline. Witnesses and seals price showing up and guarding the stamp. I looked, with everything I had, and that's where it ran out. If the bill exists as a bill for certifying, nobody in this lineage wrote it down, and that's either because it isn't there or because the one person who did write it down is on a shelf I never found. The step itself has a price on it already: raising a flag that stays up, from a state that might already have it up, is Landauer's restore to one, and he priced that as a reset in 1961. What nobody I found wrote down is when a certifying step has to be that reset. So I'm going to have to try it myself, on the smallest thing I can build, and let you watch. And I'll have to be honest about which of the five jobs the small thing even attempts, because the answer is going to be none.

3.5 The smallest thing that could carry the bill

Take a sheet of paper. Draw two boxes and label them NO and YES. That's the reading, a single yes-or-no question asked of the state, and I'm going to be careful about what the question is, because after five thousand years of seals and witnesses the word "vouch" is carrying a lot and this toy carries almost none of it. The question is only: "has the designated flag been set." Not "is some claim about a grouping true." Not "did anyone check." There is no claim in this toy, no evidence, no checker, nobody standing behind anything. There is a flag. Whether a flag can ever be made to *mean* that something was checked and can be relied on is a real question, and it's not this section's; I'll build toward it later, and until then please read YES as "the flag is up" and nothing more. Where the state is: which box you're in. That's the whole state, one bit. Then draw a line under the boxes for tally marks. The marks sit outside the state. They're the running total you keep beside it as you play, the way you'd keep score beside a board game.

Now give yourself three moves. STAY: you don't move, no mark. WORK: you don't move either (it's a stand-in for every ordinary step a machine takes), no mark. VOUCH: from NO you move to YES; from YES you stay in YES; either way you make one tally mark, because in this toy the price belongs to the move's name; where

you were standing doesn't enter into it. Play. Start in NO, make any sequence of moves you like, and stop in YES. Count the marks. Now try to reach YES with zero marks. You can't, and you can see exactly why: the only move that crosses from NO to YES is VOUCH, and VOUCH makes a mark. Now erase the mark from VOUCH's rule and play again. YES for free, every time. That's the entire content of what I'm about to call a rule, and I want you to see how little it is before I dress it up: I've decided that the move which crosses the line makes a mark, and nothing else about the machine forces that. It's a choice. The next section is about whether I had to make it.

Why tally marks, by the way? Because they're about the oldest receipt there is. The real ones, the English treasury's notched sticks split down the middle, get their own story in Part III (Section 6), fire and all. The tally on your paper is nothing like those. Nobody holds the other half. You keep it, beside the board, and you could rub a mark out and nobody would ever know. Hold on to that, because by the end of this section it's the reason the toy does none of the five jobs a record does.

Here is the same toy, stated once, carefully, because this is the one place in the book where I state it and everything later points back here. The thing on your paper has five parts.

A set of states. (Here: NO and YES, and nothing else.)

A set of move names. (STAY, WORK, VOUCH.)

A step: for each state and each move name, where you land.

A cost for each move name: a whole number of marks, zero or more. (Zero, zero, one.)

A reading: for each state, yes or no.

That's it. Add the one rule below and I'll call it a certification system; in the checked record the rule is a sixth field, so nothing counts as one without it. And I want to be exact about what's not in it. There's no counter stored inside the state that's forbidden from going down; the marks are the side tally from the board game, and the tally only ever grows because the costs are whole numbers, zero or more. (A machine could keep the total inside its state too; that's an extra part beyond the five, and Section 6 builds it.) There's no window, no observer who sees only part of the state; that's a later question and it gets its own section. And the cost depends only on the move's name; where you are when you make it doesn't matter, which is a real simplification, and I'll flag it where it counts.

One rule, which I'll call the toll. For every state and every move: if the reading is NO before the move and YES after it, the move's cost is at least one mark. That's all. It says nothing about any other move. It says nothing about how big a mark is in anything you could measure. One mark is one mark; what a mark is worth, why

the smallest charge is one (why not two, or a half?), and whether I had any right to decide that, come next, and I'm not sure of all of it. The middle question has a short answer I'll give away now. What the next section forces, on its premises, is that the crossing costs more than nothing. Costs here are whole numbers of marks, and the least whole number above nothing is one. So the positivity is forced, and the "at least one" comes from counting in whole marks.

And one theorem, which is the only thing the toy actually proves, and you've already proved it by playing. Take any run of moves that starts in a NO state and ends in a YES state. Then the marks it made add up to at least one. Here's the argument in words, and it's short enough that you can hold the whole thing. Walk the run move by move, reading the box at each step. It read NO at the start and YES at the end, and every move either leaves the reading alone or flips it, so at some move it went from NO to YES. The toll says that move cost at least one. Every other move cost zero or more. Add them up: at least one. Notice what the argument did not need. It didn't need the reading to *stay* YES afterward; if a later move flipped it back to NO and another flipped it to YES again, that's two tolls, and the argument still goes through. It didn't need the machine to be finite. It didn't need to know what any move does besides what the reading says before and after. It needed exactly one thing: the toll.

That's the whole theorem. Here it is once more in the exact form, because from here on the book carries two layers side by side: what I mean, in words, and what's stated and checked, in symbols you can hold me to. The symbols are few. $\mathbb{N}$ is the whole numbers 0, 1, 2, and so on. S and I are just names for "the set of states" and "the set of move names." Writing $\text{step} : S \times I \to S$ means "hand it a state and a move name and it hands back a state." A *run* is a list of move names, $t = [i_1, \ldots, i_n]$, played from a start s_0, one after another. The double arrow $\Longrightarrow$ reads "then": the thing on its left forces the thing on its right. And "by induction on the run," in the proof, is the formal name for the way you'd check it by hand: settle the empty run, then show that one more move at the front can't break what you've settled.

Definition 3.1 (Certification system)**.** A *certification system* consists of a set S of states, a set I of move names, a step function $\text{step} : S \times I \to S$, a cost $\text{cost} : I \to \mathbb{N}$, and a reading $\text{cert} : S \to \{\text{no}, \text{yes}\}$, subject to one rule, the *toll*: for every state s and move i,

$$\text{cert}(s) = \text{no} \text{ and } \text{cert}(\text{step}(s, i)) = \text{yes} \implies \text{cost}(i) \geq 1.$$

The run of $t = [i_1, \ldots, i_n]$ from s_0 ends at $\text{run}(t, s_0)$, the result of applying the steps in order, and its total cost is $\text{cost}(t) = \text{cost}(i_1) + \cdots + \text{cost}(i_n)$, with the empty run costing 0.

Defined as the record `CertificationSystem` (UniversalCertificationCost.v).

A word on the label before the first result. I call a result an *observation* when its proof does little more than unfold the definitions, and in an axiomatic cost model that's what the definitions are meant to do: the work is in choosing them, so the results that follow from them in a few lines are labelled that way and the theorems with more in them stand out.

Observation 3.1 (The universal floor)**.** In any certification system, every run that starts at a state reading no and ends at a state reading yes has total cost at least 1: if $\mathrm{cert}(s_0) = \text{no}$ and $\mathrm{cert}(\mathrm{run}(t, s_0)) = \text{yes}$, then $\mathrm{cost}(t) \geq 1$.

Proof. By induction on the run. If t is empty, the run ends where it starts, at a state reading no, so the hypothesis "ends at yes" can't hold and there's nothing to prove. Otherwise $t = i :: t'$ (the first move, then the rest); let $s_1 = \mathrm{step}(s_0, i)$. If $\mathrm{cert}(s_1) = \text{yes}$, the first move is a crossing, the toll gives $\mathrm{cost}(i) \geq 1$, and the rest cost at least 0, so $\mathrm{cost}(t) \geq 1$. If $\mathrm{cert}(s_1) = \text{no}$, the rest of the run starts at no and ends at yes, so by the induction hypothesis $\mathrm{cost}(t') \geq 1$, and $\mathrm{cost}(t) = \mathrm{cost}(i) + \mathrm{cost}(t') \geq 1$. Nothing else is used: not finiteness, not that the reading stays yes, not what any move does beyond the reading before and after. □

Where to check it: the record's six fields are `cs_state`, `cs_instr`, `cs_step`, `cs_cost`, `cs_cert`, and the toll `cs_cert_costs`; the theorem is `universal_nfi_any_substrate`, in the same file; and the checked proof is the induction above, step for step. That name says "any substrate," and it says too much: what the theorem quantifies over is every certification system, a substrate with the toll built in as a field, so it's about every system that has the toll and about no other. The same theorem under a name that says so is `every_toll_system_pays_the_floor`, in the same file; the old name stays because other files use it. One honest edge: the theorem does not assume that a YES state exists. If no state ever reads YES, no run meets the theorem's hypothesis, so it's true of every run and tells you nothing about any of them, which is a mathematician's way of saying it's true and useless. The toy on your paper has a YES box, so it isn't useless there. It would be natural to assume the system has to have at least one NO state and one YES state; the checked statement has no such clause, and that's on purpose, because a theorem that applies to no run is still a theorem and the toy is where you supply the runs.

The theorem also runs backwards, and that direction is one line. Suppose every run from a state reading no to a state reading yes costs at least one, whichever state you start from. A single move is a run of length one, so a move that takes no to yes costs at least one, which is the toll. The floor from every start and the toll are the

same rule (`nec_f_floor_iff_a2`), and nobody has to choose between them. From one fixed start that reads no, the floor asks for the toll only at the crossings you can reach without paying anything (`nec_f_floor_at_iff_free_reach`); a crossing you can only reach by paying has already cost at least one by the time you get there.

And the bound of one is the smallest case of a count. Under the toll a run costs at least the number of times it takes the reading from no to yes, with no permanence assumed (`nec_f_cost_ge_rises`). That bound for every run is again the same rule as the toll (`nec_f_rises_iff_a2`). And it is reached for every count: on a one-bit machine whose set costs one and whose reset is free, setting and resetting k times from no takes the reading from no to yes k times and costs exactly k (`nec_f_bit_rises_exact`). All of these are in NecFFloor.v.

And one more edge, because it's the first thing to bite when you ask a machine to stand behind more than one thing. The definition has *one* reading in it. One question, one flag, one toll on its crossing. Once the flag is up, a second claim has nowhere to go: raising a flag that's already up doesn't move anything. So a second claim needs a reading of its own, and a third claim a third: one reading per claim, each with its own crossing. Part III builds that properly, as a family of flags, one per threshold, with a step that raises any of them charged at least one (Theorem 10.11). A step that raises three at once still owes at least one, and nothing in the toll makes the charges add up flag by flag. Charges that add need the composition assumptions for independent machines. The same theorem sends the bill for finiteness: a record held in k bits that only ever turns bits on can take at most $k + 1$ different values along any one run, so it can go up at most k times in that run. (Across different runs it can end on any of the 2^k words.) A finite machine can keep only so many things up for good in its whole life. That's the opening chapter's "you can't keep all of it," turned into a count.

3.6 What this does and doesn't buy

I want to be very plain about how small this is, because the temptation is to look at the word "theorem" and feel something has been established about the world. Nothing has. What's been established is that *if* you adopt the toll, *then* the flag can't be raised for free, in any system with those five parts that obeys it, no matter what the states and moves are. The "if" is doing all the work. I chose the rule. A reasonable person could ask why, and the only honest answer right now is "because that's the thing I wanted to be true," which is not a reason.

Notice also what it doesn't ask about how strong the thing is, in either direction. A system is *Turing-complete* when it can imitate a Turing machine, and no stronger than one when a Turing machine can imitate it. The theorem asks neither. Stronger than a Turing machine, weaker, dead even: none of it reaches the floor. Bring the five

parts, charge the toll on the crossing, and you're in scope. The floor doesn't check what you can compute, because it was never about what you can compute. It's about what a move is allowed to do for free.

Which brings up the objection a theorist makes in about four seconds, so I'll make it first. The toll is a floor, and a floor is easy to meet. Charge one mark for every move, whatever it does, and the toll holds for any reading you like, since every crossing is a move and every move costs one. So any computer with a step counter beside it, and any yes-or-no question at all about its state, is a certification system. A clock passes. Nothing about that is a loophole. It's what the definition says, and it means the definition, on its own, doesn't tie the marks to certifying anything. It only says a crossing can't be free. (It's also why "a base that can compute anything plus a flag that obeys the toll" would be a cheap thing to call Thiele-complete, and why the definition that gets that name, in Part IV, asks for free computing and an earned record, which no clock meets; Section 16.) Two things later in the book do the tying. One is a ceiling: demand that a run's total never exceed the number of times the flag went up, and together with the floor exactly one bill survives, one mark per raise and nothing on any other step (Section 7); a clock fails that the first time it charges a step that raised nothing. The other is order, on the smallest machine I build: there, from a clean start, the flag can't go up unless a passing check of a claim came first, then a commitment to that same claim, and a clock can't fake that, because it's about which moves happened, and a clock only makes marks (Theorem 16.9).

And the other side of that: nothing is in scope just for being a computation. A Turing machine's run isn't an instance of this theorem because it's a run. It becomes one when somebody supplies all five parts for it, says which states read yes, and proves the toll for that cost. Until then the theorem says nothing about it, and I won't pretend it does.

That's also where the other axis from the Flatland night ended up, once I checked it. It isn't a direction an ordinary computer can't reach. Hand a Turing machine a reading, charge the toll on its crossing, and it's in scope like anything else. The possibility was always there. What the usual rules leave out is the price.

The same goes for a system with no reading at all. If nothing in it answers "is the flag up," then nothing in it can say, from the inside, that anything is entitled to be relied on. Whatever entitlement it ends up with gets carried in from outside, by somebody who signs their name to it, and nothing in the system priced it. Smuggled entitlement isn't entitlement for the system, no matter whose name is on it. Be careful which way that cuts, though. Whoever reads the signature can be perfectly entitled on their own account; that's what an auditor has, and it's honest as long as it's called the auditor's. It isn't the system's. On the smallest machine I build later, the reason to believe its checker says yes only when the claim is true is exactly that kind: a

proof, checked outside any record the machine keeps, and I'll say so when I get there (Section 16). What outside entitlement never is, is something the system itself earned.

And here's a question I'll only start on here: bolt two certification systems together, and do you get one? Run them side by side, each move being a move of one or the other, and add up the marks. It depends on the reading you give the pair. Take a reading that can only go up when one of the parts' readings goes up, like "either flag is up" or "both are": then any crossing of the pair is a crossing of a part, that part's toll charged it, and the pair obeys the toll. Take "the first flag is up and the second is down," and it doesn't: that reading turns on when the second flag falls, and nothing charges a fall. The first two readings are checked: read through "either flag is up" the pair is a certification system whose floor is one. Read through "both are," a run that starts with both flags down and ends with both up costs at least two, and two is reached when each part can raise its own flag for one (Section 18). From a start with one flag already up, one raise is enough, so two is a floor for the clean start only. The third is still an argument on this page. A related composite is checked too, a host machine running a guest, read through "the host's own flag or its copy of the guest's" (Theorem 16.39). Whether every way of building one machine out of others keeps the toll is a bigger question, and Section 18 takes it as far as I could: some ways do, some provably don't, and it says which.

So why bother with the toy at all? Not because the lineage was looking for it. The bulla was after durability, the seven seals after evidence, the consul's stamp after authenticity, the two columns after catching slips; none of them was trying to put a price on the moment a flag goes up, and I shouldn't pretend they were. The reason is narrower. Everywhere in that lineage where somebody *did* locate a cost, it sat on a step next to the flag. The flag itself went unpriced as a flag: Szilard on the measurement, Bennett on the wiping, proof-of-work on the search, Boyle on the instrument and the room, Foster and Stine on the test. One entry prices the crossing's own shape, and I put it beside the toll on purpose: Landauer's restore to one, which is the one-bit toll step, priced as a reset whatever the bit is standing for. The flag's job never came into it. The toy is the first thing I could build where the cost sits on the crossing itself, and it proves that if you price that step, the floor follows everywhere, for any machine with the five parts. That's not an answer. It's the question made exact enough to be wrong. And of the five jobs a record does, the toy attempts none of them. Not even durability: the flag can be lowered by any move you care to write, and the marks that don't go away are kept by *you*, the scorekeeper, beside the board. The machine keeps none of them. What the toy has is a scorekeeper's receipt for the crossing. Whether the machine can be made to keep that receipt itself, where a step could reach in and change it, is Section 6's question. Authenticity, evidence,

accounting, cheap checking: none of them is in here, and I'll say so again when it's tempting to forget.

And here's what I'll owe you by the end. Whether the toll is a choice or a necessity: the next section, and I'll tell you now that the answer surprised me. What a mark is worth in anything physical: later, and only partly. What an observer who sees only a window can and cannot recover, including whether they can even tell which step paid: its own section. And whether a full-size machine, the kind you could actually run a program on, can be made to obey the toll without cheating, and what that machine looks like, and what it costs to vouch for structure in it: that waits for Part IV, on purpose, and takes most of the rest of the book, and I'm still not sure I pulled it off.

For now you have a sheet of paper with two boxes, three moves, one rule you were handed without a reason, and one theorem you proved yourself in about a minute. Don't let go of it. It's the smallest version of everything that follows, and if it's wrong here, it's wrong everywhere.

4

Why the toll isn’t arbitrary

I handed you a rule without a reason. The move that raises the flag makes a mark, because I said so. If that’s all it is, this book is a long way of describing a preference, and you should treat it the way you’d treat someone telling you their favourite colour is a law of nature. So this section is me trying to find out whether I had to pick it. I’ll tell you the shape of the answer up front, because I don’t want you reading forty paragraphs to find out whether there’s a point: on a machine with a finite memory, where the flag once raised stays up, the step that raises it has to forget something, and forgetting is the one step Landauer’s principle puts a price on, when what’s forgotten wasn’t already known (that last clause takes a lot back, and I’ll show you exactly how much). Three “ifs” in that sentence. Every one of them is a door you can walk out of, and I’m going to show you each door, because the doors are where the actual content is.

4.1 A closed room with no spare chair

Do this one with real objects, because the whole section rests on it and it’s about ninety seconds of work. Draw a box on the table and call it the certified room. Put three chairs inside the box and one chair outside, by the door. Four people, one per chair; the people are the states your machine could be in, and the three inside are the ones whose flag is already up. One move of the machine moves everybody at once: each person stands up and goes to the chair the rule sends them to. Two rules. First: nobody inside the room may end up outside, because the flag, once up, stays up. Second: the person by the door must end up inside, because this move is the one that raises their flag.

Now try to finish the move with one person in each chair.

You can’t. Three people start inside and have to stay inside, so they take the three inside chairs between them, with nobody left out, because nobody may leave. The person from the door has to come in. There is no fourth chair inside. Somebody shares. Two of the machine’s possible states land on one state, and after the move you can no longer tell, from where everyone is sitting, which of the two it was. The move forgot something.

That's the whole argument, and I want to name the piece of mathematics that did the work, because it's old and it's not mine. If you put things into fewer boxes than there are things, some box gets two. People call it the pigeonhole principle; it's the one from the opening chapter, where Jean Leurechon used it in 1622 to argue that two people somewhere have the same number of hairs, and the first name for it I could find came from Dirichlet, who used it in 1842 and later called it the drawer principle. It's the kind of fact a child accepts and a mathematician spends a career finding new uses for. Here the use is: a step that forgets nothing is a one-to-one shuffle, a one-to-one shuffle of a finite set of chairs hits every chair, so the inside chairs are all taken by insiders, so there's no room for the newcomer. Finite. Permanent. Flip. Those three and nothing else, and the conclusion is "two states became one."

The checked version of the chairs is a theorem a few lines down, and it needs one word first. "Injective" is the mathematician's word for "forgets nothing": a step is injective when two different starts always give two different ends. The theorem says exactly what the chairs said and takes exactly the three premises: a duplicate-free list that names every state (finite), a reading that no step ever turns off (permanent), and one state that reads NO before the step and YES after it (the flip). There is no price in it anywhere. It is pure counting. I'll come back to what counts as physics, but notice that nothing so far has been.

Here are the words, exactly. For this whole section the setting is the five parts from Section 3 without the toll: states S, move names I, a step, a reading, and (later) a cost.

Definition 4.1 (Finite, permanent, injective)**.** The machine is *finite* when there is a list of states with no repeats that contains every state. The reading is *permanent* when no move ever turns it from yes back to no: for all s and i, $\mathrm{cert}(s) = \text{yes}$ implies $\mathrm{cert}(\mathrm{step}(s, i)) = \text{yes}$. A move i is *injective* (forgets nothing) when different starts give different ends: $\mathrm{step}(a, i) = \mathrm{step}(b, i)$ implies $a = b$. A move that is not injective *merges*.

Defined as `finite_states`, `permanent` and `step_injective` (PermanentCertification.v).

Theorem 4.1 (A permanent flip merges)**.** On a finite machine with a permanent reading, if $\mathrm{cert}(s) = \text{no}$ and $\mathrm{cert}(\mathrm{step}(s, i)) = \text{yes}$, then the move i merges.

Proof. Suppose i were injective. Let C be the list of states that read yes; it is finite and has no repeats. By permanence, $\mathrm{step}(\cdot, i)$ sends every member of C into C. Being injective, it sends the members of C to members of C with no two landing on the same one, so its images are as many as C itself, and a list of that many distinct

members of C is all of C. So the yes-state $\mathrm{step}(s, i)$ equals $\mathrm{step}(c, i)$ for some c in C, and injectivity gives $s = c$, so s reads yes. It reads no. That's the contradiction, and it's the chairs. □

Checked as `permanent_flip_is_not_injective` (PermanentCertification.v). The proof there uses the standard-library fact that a duplicate-free list of n members inside a list of n is the whole list, which is the pigeonhole in the form Leurechon would recognise.

Is that new? No. It has been sitting in two places for a long time, one in physics and one in the theory of automata, and a physicist will recognise it on sight, so start there. In 1876 Josef Loschmidt pointed out, against Ludwig Boltzmann's account of how a gas settles down, that the laws of mechanics run just as well backwards: reverse every molecule's velocity and the gas retraces its steps, so mechanics alone can't make entropy only ever climb. Twenty years on, in 1896, a young assistant of Max Planck's in Berlin pushed harder. He took Henri Poincaré's recurrence theorem of 1890, which says a bounded mechanical system comes back as close as you like to wherever it started, again and again, and used it to argue that nothing in such a system can climb forever: give it time and it comes back down. The assistant was Ernst Zermelo, the same Zermelo who'd patch Frege's floor twelve years later. Boltzmann answered the same year, not very politely, that the times you'd have to wait for a return are so enormous that nobody would ever see one. (Jos Uffink, who wrote the Stanford Encyclopedia's entry on Boltzmann's statistical physics, calls that reply drenched in sarcasm and bitterness, and calls Zermelo's own paper a careful logical argument.)

Now look at the chairs again. A step that forgets nothing, on a finite set, is a reshuffle: it moves everybody around in loops, and every loop comes back to where it started. A flag that goes up and stays up is exactly a quantity that climbs and never comes back down. So a reshuffle can't carry one, which is Zermelo's objection with the infinities taken out. On a finite set the loops are enough, and Poincaré's theorem isn't needed. And the wait is at most as many steps as there are chairs. Nothing enormous. Zermelo was aiming it at the second law. Flags never came into it. The quantum version came in 1957, when P. Bocchieri and A. Loinger proved the same return for a closed quantum system whose energies come in separate levels, so going quantum isn't a way out of the room either, not for a closed system that never sheds anything.

The second place stings more, because it's in computer science. Somebody got here first, again. Picture a finite machine that reads letters, one move per letter. If every letter just reshuffles the states one for one, automata theorists call it a permutation automaton, and the sets of inputs such machines can pick out are called group

languages. Jean-Éric Pin's 1992 paper on reversible automata, machines whose letters act as one-to-one maps, sorts out exactly which languages those machines recognise, and in that world the chairs are folklore. Here's the short version. Start at a state that reads no, and collect every input that ends at a yes. Because yes is permanent, adding letters to the end of a collected input keeps it collected. A group language with that property is either empty or everything, and everything would include the empty input, which ends where it started, at no. So it's empty: no run of reshuffles raises the reading. That's Theorem 4.1 with the moves as letters. Pin's paper is where I'd send you to check the automata side. It isn't about flags or prices, and I'm not claiming it says anything about them.

And if you like your facts in the most general clothes going, category theorists have this one too. Treat the moves as a monoid acting on the states, and a permanent reading as a set of states the moves never leave. In the topos of that monoid's actions, the truth value of "reads yes" at a state is the set of move sequences that carry it into the yes-states. Every move can be undone exactly when the monoid is a group, and exactly then is that topos Boolean, which makes every such truth value all or nothing. Morgan Rogers lays out that fact in his 2019 and 2021 papers on toposes of monoid actions. On a finite set, a move that merges nothing can be undone, so it's the chairs again, in a third language. Does it change anything here? No. It's the same fact wearing a bigger coat, and I mention it so nobody thinks the chairs are mine.

Going quantum doesn't get around the automata version either. Alex Brodsky and Nicholas Pippenger (2002) showed that a quantum finite automaton that measures only once, at the very end, accepts with bounded error exactly the group languages, the reshuffle kind. One that's allowed to measure after every letter, and to stop on what it sees, does better: it accepts every piecewise testable language, and "has seen a b somewhere" is one of those. A measurement that can stop the machine is a record that stays, and it's the extra thing that lets the bigger class in.

The third place is the closest, and it isn't old at all. In 2023 Thomas Ouldridge and David Wolpert derived the thermodynamics of a finite automaton built the way real computers are: the same physical step run again and again, each time touching only the part of the machine that changes. They sorted languages into two kinds by the smallest automaton that recognises each one. If that automaton has a letter that sends two states to one, every automaton for the language has such a letter too, and on their derivation recognising the language on a device like that produces entropy, waste no cleverer design gets back, unless the inputs are so finely arranged that only one of the merged states ever actually turns up. If the smallest automaton only reshuffles, they find the language can be recognised with no entropy produced at all. One of their worked examples is the strings with no b in them, which is my flag turned inside out: the runs that raise it are exactly the strings that have seen a b. So

the chairs, a real price on them, and the escape when you already know which chair you're in are all in their paper, three years before this book. Their price is derived from the physics of the device. Mine is assumed.

There's a fourth neighbour, and I'd call it a cousin, because it speaks another language. Constructor theory is David Deutsch and Chiara Marletto's programme for stating physics as which transformations a device that runs in a cycle can and can't bring about, and in it a task is irreversible when it's possible and its inverse isn't. Marletto's version of thermodynamics (posted in 2016) leans on exactly that: tasks that are possible while their transposes aren't. Marletto, the physicist Vlatko Vedral and coauthors (2022) made an irreversible task out of the quantum homogeniser, a machine of reservoir qubits that interact by partial swaps, and ran it with photons. Maria Violaris and Marletto (2022) then argued that the same machine, run in a cycle, can turn a pure qubit into a mixed one and can never turn a mixed qubit back into a pure one, even though every interaction in it is reversible, and that this suggests erasure may cost something beyond Landauer's bound. The chairs are a classical, finite, exact cousin of that sentence: a set of states a step can't leave, a task that's possible, and no way back. Theirs is quantum and approximate and aimed at physics' price; mine is counted, and assumes its price. Deutsch and Marletto's paper on information (2015) has a neighbour of the chairs argument in its section 8.6: measuring one observable disturbs another irreducibly, and nothing can undo the disturbance afterwards while the outcome stays in an information variable. That's a second, theory-independent way of saying that a record that stays makes the step irreversible.

So what I can claim here is narrower again, and I'd rather say exactly how. Is even the pricing form Ouldridge and Wolpert's? In physics, on their derivation, yes: a merge the language can't avoid costs entropy. What I didn't find in their paper is the price as a named rule about a move, one a machine charges on a ledger whether or not anybody's physics is right (put a price on merging and the flip has to pay it, which is the next subsection), and what I get out of that rule: the toll, as a theorem about the ledger, the merge-or-revoke split for a single move, the exact match between "forced to be priced" and "merges" when move names can be decided equal or different, the squeeze bound counted in whole marks and its entropy form, a built machine walking out of each door, and all of it checked. Those are the forms I didn't find stated anywhere I looked, and Appendix C sorts them the same way: the counting core classical and somebody else's, the pricing form only partly mine. The core move is older than the computer, it had two homes before it got to mine, and its physical price had a third.

4.2 Now charge for forgetting, and the toll falls out

Here's the step I'd been circling since the kindergarten buckets. In the last section I told you Bennett's rule as he states it: what has to cost is erasing a bit or merging two paths a computation could have taken. I didn't ask you to believe it. I'm not going to ask you now either. I'm going to write it down as a rule you can accept or refuse, in the smallest form that does any work: *any move that forgets something costs at least one mark.* Any move that is not injective has cost at least one. That's it. Note what it does not say. It says nothing about certifying. It says nothing about flags. It's a price on forgetting and only on forgetting.

Put the two together. The room says: on a finite machine with a permanent flag, the move that raises the flag forgets something. The price says: a move that forgets something costs at least one. So the move that raises the flag costs at least one. That is the toll, the rule I handed you without a reason, and now it has one. The proof is two lines: the first theorem says the flipping move merges, the premise says merges are priced, done. I'll keep saying "premise," because that's what it is, and because the next section of this book is about what happened to some very smart people who put a price where a premise goes and then treated it as a law of nature, and about half of this one is me trying not to do that.

Premise 4.2 (Merge pricing)**.** Every move that merges costs at least one: for all i, if i is not injective then $\mathrm{cost}(i) \geq 1$. (Checked name: `merging_steps_priced`.)

Observation 4.2 (The toll from finiteness, permanence, and merge pricing)**.** On a finite machine with a permanent reading whose cost satisfies merge pricing, every move that turns the reading from no to yes costs at least one. That is, the machine with that cost is a certification system in the sense of Definition 3.1, and the universal floor (Observation 3.1) applies to it.

Proof. A move that flips the reading merges, by Theorem 4.1; merge pricing charges it at least one. □

Checked as `a2_from_merging_price_and_permanence`; the packaging of such a machine as a certification system is `certification_system_from_merging_price`, and the run-level floor for it is `permanent_certification_trace_floor` (PermanentCertification.v).

The theorem asks for more than it uses. Only the yes-states need to be finitely many: with a permanent reading and finitely many states reading yes, the rest of the machine as large as you like, merge pricing still gives the toll (`nec_f_toll_from_merges_yes_finite`). And merge pricing asks for more than the toll needs. Under those

same two conditions the toll holds exactly when every move that flips the reading somewhere and merges is priced (`nec_f_a2_iff_flipping_merges_priced`). Merge pricing gives that (`nec_f_merging_priced_weakens`), and the eight-state machine described later in this section, with its jump made free, meets the toll without merge pricing (`nec_f_a2_without_merge_pricing`). All four are in NecFMerge.v.

One more way to see the same thing, because it answers a rival I'd otherwise owe you. The rival says: leave certification alone. Price *erasure*, and keep an honest flag that reports when something was erased. Fine; but a system can set its certificate at cost zero with its erasure flag reporting nothing, exactly because the certifying move merged two states and the flag didn't say so (`commit_without_erasure_system_is_not_honest`). Make the erasure flag honest, so that it reports every move that forgets, and on a finite machine with a flag that stays up the erasure law sends the same bill the toll does (`honest_erasure_accounting_implies_a2`). The toll isn't a rival to honest erasure accounting. It's what honest erasure accounting says about the certifying move.

So, is the toll arbitrary? Here's the honest version. The toll is a consequence of three things: a finite memory, a flag that stays up, and a charge on forgetting. If you accept all three, you don't get to refuse the toll; it's forced. If you refuse any one of them, the toll goes back to being my preference. So the question "did I have to pick it" has turned into three smaller questions, which is what progress usually looks like: a better set of things to argue about. Let me take the three doors in order, and for each one, build the thing that walks out through it, because a premise you can't see the counterexample to is a premise you're taking on faith.

4.3 Door one: keep everything, and nothing ever has to merge

Take a strip of paper that never runs out of boxes. The machine's whole state is the strip. Each move writes a 1 in the next empty box, and the flag is up as soon as any box has a 1 in it. Check the three premises. Permanent: once there's a 1, there's always a 1. Flip: the empty strip reads NO, and after one move it reads YES. Forgets nothing: every move can be undone by erasing the last 1, so two different strips never become the same strip. So here is a machine that raises a flag, keeps it up, and never merges anything. The room argument has no grip on it, and the toll doesn't follow. The checked version is `unbounded_history_escapes`, and its state is a list of bits of any length, which is the formal way of saying the strip has no end.

If the endless strip feels like a cheat, it's a famous one. David Hilbert, in a lecture course in the winter of 1924–25, told a story about a hotel with infinitely many rooms, every one of them taken, where a new guest still gets a room: everybody moves from room n to room $n + 1$, and room 1 comes free. That's the chairs argument losing its

grip, told as a joke about a front desk. And here's the part worth checking: Hilbert never published it. The hotel became famous because George Gamow put it in a popular book in 1947, and for a long time it wasn't clear Hilbert had told it at all. The historian of science Helge Kragh first guessed that Gamow had made it up. Then Tilman Sauer, a historian of science who had co-edited a volume of Hilbert's lectures on the foundations of physics, pointed him to Hilbert's lecture notes, which had just been printed in 2013, and in 2014 Kragh took his guess back in a paper called "The True (?) Story of Hilbert's Infinite Hotel." Keep the question mark. Even Kragh's own paper puts the lecture in January 1924 in one place and January 1925 in another. Still, this time a story everybody repeated turned out to have a real lecture behind it. I've learned not to count on that happening. Bring it back to the table. In the room, the person from the door had nowhere to sit because the chairs ran out. In Hilbert's hotel they never do, so everybody shifts one over and nobody has to share. The strip machine is that hotel. The argument stays the same. What the hotel shows you is that it was about running out of chairs all along.

What walked out the door is the finiteness premise. Now cut the strip to eight boxes and run it for real: eight boxes, nine moves. On the ninth move there's nowhere to write. To keep going you have to reuse a box, and the moment you reuse a box two different histories end on the same strip, and you're back in the room with a shared chair. Keeping the whole history is exactly the move Bennett made in 1973, when he showed that any computation can be done without forgetting if you let it keep a copy of everything it did; Landauer, back in 1961, had already written that for a machine able to run a program that doesn't stop, the space to hold every intermediate result "cannot be there." The history escape is real. It costs you a memory that never fills. Every machine you can buy has a memory that fills.

Bennett wasn't the first there, or the last. Yves Lecerf, in a short note to the French Academy of Sciences in 1963, had already shown that reversible Turing machines can simulate arbitrary ones, and the note went largely unnoticed. The computer scientist Tommaso Toffoli (1980), and Edward Fredkin and Toffoli together (1982), both then at MIT, built reversible logic: gates and circuits that compute any function and lose nothing, by carrying constant inputs in and extra outputs out. And the computer scientist Kenichi Morita (1996) proved that reversible two-counter machines are universal, so even a machine as small as the one in Part IV can compute everything with every move one-to-one. Every one of them keeps what it doesn't forget somewhere: a history, garbage outputs, a counter that codes the past. The strip is that.

One thing I have to say plainly, because the file is more careful than I'd like to be. The theorem shows the strip machine has the other two properties and forgets nothing. The sentence "and this machine is infinite" is a second theorem, `nec_f_`

`history_infinite` (NecFPaperToys.v): no finite list holds every state of the strip. The first theorem contains no cost at all. "Certifies for free" is written in a second theorem, `history_escape` (AxMerge.v): there the strip is counted as a number, each move adds one at a cost of zero, nothing merges, merge pricing holds, and the toll fails. So the words and the files line up, in three theorems.

Which state is being counted. Whether a move merges depends on what the state keeps. The strip keeps its history, so nothing merges; cut it to eight boxes and the reuse merges. Section 21 proves the general form (Theorem 21.3): keep the previous state alongside the current one and every fixed-instruction step is one-to-one. So a merge price is a fact about a move on a stated state space. Each price derived in this book is derived on the whole state its definition gives, and three things go with it: which state that is, what lies outside it (nothing outside it is charged), and whether the move is one primitive instruction or a composite run of several. Relabelling the states one-to-one changes no price, since it sends merges to merges and one-to-one moves to one-to-one moves. Adding history to the state, or projecting part of the state away, changes the machine being modelled, and the price can change with it.

4.4 Door two: let the flag come back down

Take one coin. The rule is: turn it over. Heads is YES, tails is NO. Every move is undoable (turn it over again), so nothing is forgotten. Tails goes to heads: that's a flip, NO to YES. And heads goes to tails: the same move takes the flag back down somewhere else. Two states, finite as it gets, and the move that raises the flag forgets nothing, so a schedule that charges only for forgetting is allowed to charge it nothing. The file states that the one move forgets nothing (it is injective) and its on-then-off behaviour (`revocable_certificate_escapes`); that the coin is finite and that the price may be zero are written in `toggle_escape` (AxMerge.v), which prices the turn at zero, meets merge pricing, and fails the toll. The thing that walked out the door is permanence.

Now here's a fact that I think is sharper than the premise it replaces, and it was the thing in this section that surprised me. Drop permanence entirely. Keep only finiteness. Take any move that raises the flag anywhere. Then one of two things is true: the move forgets something, or the same move lowers the flag somewhere else. There's no third option, and the proof below is a search: go through the finitely many YES states looking for one this move turns to NO. Find one, and that's the revocation; find none, and the move is permanent at least for itself, which is all the room argument needs. So a flag that goes up for free is a flag the same move can take back. A flag that nothing takes back is paid for. You can't have a free raise that stays raised, not on any finite machine, not with any price schedule that charges for

forgetting. That's a narrower sentence than "every certificate costs." It's true, and I'll take it. The broad one would only be mine.

Theorem 4.3 (Merge or revoke)**.** On a finite machine (no permanence assumed), if $\mathrm{cert}(s) = \text{no}$ and $\mathrm{cert}(\mathrm{step}(s, i)) = \text{yes}$, then either i merges, or there is a state t with $\mathrm{cert}(t) = \text{yes}$ and $\mathrm{cert}(\mathrm{step}(t, i)) = \text{no}$.

Proof. Check every yes-state on the finite list for one that i sends to no. If there is one, that is the second alternative. If there is none, then i keeps every yes-state at yes, which is permanence for the single move i; the argument of Theorem 4.1 uses permanence only for the move it is about, so it applies, and i merges. □

Checked as `flip_merges_or_revokes`, with the single-move form of the room argument as `permanent_at_flip_is_not_injective` (PermanentRecordPricing.v).

I'll give you the three-card version too, because it shows that a move can be forced to be priced without writing anything permanent, and I don't want you to think the picture is tidier than it is. Three cards, T0, T1, T2, with a star on T1. The rule: T0 goes to T1, T1 goes to T2, T2 goes to T1. T0 gains the star (a flip). T1 loses it (a revocation). T0 and T2 both land on T1 (a merge). So this move is priced, by the forgetting rule, and yet it writes nothing permanent; it takes a star away in the same breath. "The priced moves are exactly the permanent records" is false, and the file proves it false with these three cards. What's true is one direction: on a finite machine whose move names can be decided equal or different, every permanent raise is a priced move. The other direction has the three cards standing in it. And "priced" here has an exact meaning worth one sentence: a move is *forced to be priced* when every schedule that charges for forgetting charges it, and that holds exactly when the move forgets something, with no finiteness needed at all. A way to tell two move names apart is enough.

Definition 4.3 (Forced to be priced)**.** A move i is *forced to be priced* when every cost function satisfying merge pricing (Premise 4.2) charges i at least one.

Defined as `forced_priced` (PermanentRecordPricing.v).

Theorem 4.4 (Forced price is merging)**.** If any two move names can be decided equal or different, then a move is forced to be priced exactly when it merges. No finiteness is needed.

Proof. If i merges, merge pricing charges it at least one, whatever the cost function. If i does not merge, define the cost that is 0 on i and 1 on every other move (this is where telling move names apart is used). It satisfies merge pricing, since every merging move is some move other than i and costs one, and it charges i nothing, so i is not forced. □

Checked as `forced_priced_iff_merges` (PermanentRecordPricing.v).

Corollary 4.5 (Permanent records are forced)**.** On a finite machine whose move names can be decided equal or different, a move that never turns the reading off and turns it on somewhere is forced to be priced. (By Theorem 4.3 it merges; by Theorem 4.4 it is forced.) The converse fails: the three cards are forced, because they merge, and they revoke.

Checked as `forced_priced_iff_merges`, `permanent_record_write_is_forced_priced`, and the three-card counterexample `forced_price_without_permanent_record` (PermanentRecordPricing.v).

4.5 Door three: forget for free

The third door is the easiest to describe and the hardest to argue about, so I'll describe it and then spend the rest of the section on the argument. Take two sheets of paper, one blank and one stamped. One move: stamp whatever you're handed. Finite, two states. Permanent: stamped stays stamped. Flip: blank goes to stamped. Forgets: both sheets end up as the same stamped sheet. Everything the room argument needs is here, and the move merges. Now set the price of that move to zero. The toll fails, and you can watch it fail: the flag went up for nothing. That's `free_merge_escapes` (PermanentCertification.v), and what walked out the door is the charge on forgetting. (The same file shows that a price of one on that move satisfies all three premises, `priced_reset_satisfies_premises`, so the premises can hold together; door three is about whether they must.) Nothing in counting stops you from setting that price to zero. Only a claim about the world does. The nearest such claim is Landauer's, and I have to be exact about the gap between the two: the premise as written charges *every* forgetting move, whether or not you knew which state you were in, and Landauer's own heat, as the coin under the cup below will show, is owed only for what you didn't know. So the premise is Landauer's claim taken in its worst case over what the machine might be holding, and that's the version the counting needs.

Here is what Landauer said, in 1961, as exactly as I can put it. I went past the summaries and read the paper itself. A logical operation with no single-valued inverse (a step that forgets, in this book's words) must produce heat, "typically

of the order of kT" per operation, where k is Boltzmann's constant and T is the temperature; and for the specific case of resetting one bit whose value you don't know, the minimum is kT times the natural log of 2, about 0.69 kT. I want you to hear two things in that. First, "of the order of": he was not handing out a number to three decimals. Second, and this surprised me, he did not measure anything. The paper is argument, with models. He writes that he can "show, or at least very strongly suggest" the conclusion, and that his reasoning gives "no guarantee that this minimum is in fact achievable." That's the founding paper of the whole subject, and it's a careful man saying "I think this is right and here's why." I like him more for it.

Was it measured, eventually? Yes, and only recently. In 2012 a group at the École Normale Supérieure in Lyon, Antoine Bérut and Sergio Ciliberto's lab working with Eric Lutz, trapped a single glass bead, two millionths of a metre across, in a double well made of laser light, and used the two wells as the two values of a bit. Then they erased the bit, by lowering the barrier, tilting the trap, and raising it again, many times over, slower and slower, and measured the heat each time. The average heat came down toward Landauer's kT ln 2 and sat there as the erasure got slower. On average. Single runs went under it; the bound is about the average, and any one erasure can beat it. And if you only succeed in erasing half the time you owe nothing, which the paper's formula shows and which I think is the most honest line in it. Others have done it since with different devices. So: predicted in 1961 by argument, seen in a laboratory fifty-one years later.

And is it settled? No, and I'd be lying by omission if I left that out, because the people who doubt it are not cranks. John Norton, a philosopher of physics, has argued since 2005 that the usual proofs of the principle quietly assume what they're trying to prove, and that the kinds of process they assume, combined, could be made to break the very law the principle is supposed to guard. An earlier objection, made in 1999 by Norton and his Pittsburgh colleague John Earman, also a philosopher of physics, was that the principle can't be independent of the second law. Charles Bennett, who did more than anyone to build the subject, rated that one "in my opinion the one of greatest merit," and then disagreed with it. Others replied to Norton; Norton replied back. The Stanford Encyclopedia entry on the topic, written by a philosopher of physics who works on exactly this, ends without a verdict. I've read what I could of both sides and I'm not qualified to referee them, and I'm not going to pretend the laboratory settles a question about what the principle means in general, because measuring one bead doesn't do that.

So here's what I did with it, and I think it's the only honest thing to do. The charge on forgetting is written into the checked work as a *named premise*. It's a definition the theorems take as an assumption, and nowhere is it proved as a general fact, because

it isn't provable from counting; for particular machines with particular price lists it's checked (the stamp at cost one, above), which is a different thing. If Norton turns out to be right, every theorem in this section still holds; they just have one premise fewer that the world agrees to. If Landauer's side turns out to be right, the premise is physics. Either way, nothing in this section says a device does anything until someone measures the device. The counting is mine to prove. The physics is a door I've labelled, and what opens it is a measurement. A proof can't.

Labelled, though, isn't the same as unplaced, and a hedge isn't a position, so here's mine. Earman and Norton's objection is a dilemma. Either an information-theoretic exorcism of the demon assumes the second law, which they call the *sound* horn, and then it adds nothing the second law didn't already give; or it rests on a new principle tying information to entropy, the *profound* horn, and nobody has given that principle independent grounds. A named premise is the sound horn, on purpose. That's where Bennett ended up in 2003, and Norton's "Too Good to Be True" (written in 2025, published in 2026) says the careful literature almost always accepts that horn, and argues that whatever succeeds there is ordinary thermodynamics relabelled. I'll take that. Ordinary thermodynamics, relabelled onto a ledger, is all this section asks of the physics. And on the counting I'm on Norton's side outright. His 2018 paper "Maxwell's Demon Does Not Compute" exorcises the demon with Liouville's theorem and nothing else, no information and no computation: dynamics that keeps every patch of states the same size can't crowd a big set of states into a small one. The counting core of this section, that a step losing nothing can't carry a no-state into a set that stays up, is that point in finite, discrete form. And if you want the best case for the premise itself, the one I found is James Ladyman, Stuart Presnell, Anthony Short and Berry Groisman (2007): from one of the standard statements of the second law, the one named for Kelvin, they derive that a logically irreversible operation can't be carried out thermodynamically reversibly by any of their L-machines. That's the merge-pricing premise with an argument under it, and if you want to argue with the premise, that's the paper to argue with.

One question a physicist asks before any of that, and it's a good one: the laws underneath never merge anything. Classical mechanics carries each exact state forward to exactly one next state and could carry it back again, and Liouville's theorem adds that a patch of states keeps its size as it moves; quantum mechanics is unitary, which is one-to-one too. So when this section says a step merged two states, where did the difference go? Out of the machine. The merge is in the machine's own description, the states its reading is about. The machine together with everything around it still runs one-to-one, and the distinction the machine forgot survives. It gets pushed out into the surroundings, mostly as heat. The surroundings are door one's endless strip. That door is always open for the universe as a whole and shut

for the machine, because the machine isn't the universe and its memory fills. Read that way, Landauer's premise is the claim that pushing a distinction out of a finite machine into its surroundings costs heat, and that's all it claims. Which also means the bill is heat *handed over*. None of it has to be heat *wasted*. Takahiro Sagawa's 2014 analysis says that a step that forgets can, in principle, be run slowly enough that no entropy is produced overall: the heat leaves the machine, the room's entropy goes up by what the machine's went down, and nothing is lost that didn't have to be. So on the premise the forgetting step can't be free, but it doesn't have to be wasteful. When I call it a cost, I mean a transfer you can't avoid. None of it is a loss you could have engineered away.

4.6 How much: the price grows with the squeeze

The toll says "at least one mark." That's a floor for raising one flag. The room argument says more if you let it. Say there are m people already inside and k more come in through the door in one move. (That's a different k from the one in Landauer's kT; when the two meet, below, Boltzmann's constant gets a little B hung under it, k_B, so you can tell it from a head count.) All m plus k of them end up in m chairs. That's a squeeze, by a factor of (m + k) over m, and if you accept a slightly stronger version of the forgetting price, that each mark pays for at most one *halving* of the number of distinguishable states, then the move costs at least the number of halvings in that squeeze, which is the base-two logarithm of (m + k)/m.

Back to door three's stamp, with four sheets this time: three blank, one stamped, and the stamp-everything move. Four states go to one. Before the move, to say which sheet you had took two yes-or-no questions (is it one of the first two? then which of those two?); after the move, it takes none. Two bits gone. Two halvings. So that move costs at least two marks, and two marks is enough: a cost of exactly two satisfies the halving rule (both facts are checked for these four sheets, `quad_stamp_needs_two` and `quad_stamp_cost_two_is_priced`). The general statement is m + k at most 2 to the cost times m, and next to it in the file sits a logarithm form that rounds both logarithms up and is weaker for it (three states and one newcomer round to the same two bits on both sides, and the rounded form says nothing). The halving premise, like the one before it, is a definition used as an assumption. The units are halvings. To turn a halving into joules of heat you multiply by $k_B T \ln 2$, and that multiplication is physics; the counting stops before it.

That little k_B deserves a short trip of its own, because it sits exactly where counting turns into physics, and its name is a small history lesson. Ludwig Boltzmann's grave in Vienna has $S = k \log W$ carved on it (that k is the same k_B, before anyone hung a B under it): entropy is a constant times the logarithm of the number of ways the parts

could be arranged. The idea is his. The constant, and the equation written that way, came from Max Planck, in his work on radiation around 1900. In his Nobel lecture in 1920 Planck said the constant "is frequently termed Boltzmann's constant, although to the best of my knowledge Boltzmann himself never introduced it," and guessed that Boltzmann never believed anyone could measure it accurately. I'd assumed Planck named it after Boltzmann. His own words point the other way: it was other people calling it that. Who first did, I couldn't pin down; one recent paper points to Paul Ehrenfest in 1906, the year Boltzmann died, and I haven't checked that myself. Ehrenfest was an Austrian physicist who had done his doctorate under Boltzmann in Vienna. So the most famous constant in this corner of physics wears the name of a man who didn't write it down. Does that change the bill? No. But look at the job the constant does in that equation. Take it away and what's left is the logarithm of a count, which is a number of halvings, which is what the squeeze bound hands you. The constant is the exchange rate, and whether the world honours that rate for a merge is the premise. No theorem here settles it.

Exactly, with one new piece of notation: for a list D of states, write $|D|$ for how many it has, and $\mathrm{img}_i(D)$ for the number of *different* states the move i sends them to.

Premise 4.4 (Halving price)**.** Each mark pays for at most one halving: for every move i and every list D of states with no repeats, $|D| \leq 2^{\mathrm{cost}(i)} \cdot \mathrm{img}_i(D)$. (Checked name: `compression_priced`. It implies merge pricing, since a merge sends two different states to one and so needs $\mathrm{cost}(i) \geq 1$.)

There's a plainer way to read that premise, and it's worth having because it says what the premise is *about*. Look at one landing spot of move i and count how many states land there; call the biggest such count the move's worst pile-up. The premise holds exactly when every pile-up has at most $2^{\mathrm{cost}(i)}$ states in it, which is to say the cost is at least the base-two logarithm of the worst pile-up. One direction: take D to be one whole pile, which lands on a single spot, and the premise says the pile has at most $2^{\mathrm{cost}(i)}$ members. The other: any list D splits by where its members land, each part fits inside one pile, so $|D|$ is at most the number of landing spots times the worst pile-up, which is the premise. So the halving price asks a move to pay for the worst it could ever do, whichever state it's actually handed. That's worst case on purpose, and it has a home in physics. In the branch of thermodynamics that prices a single run that has to succeed every time, with no averaging over many runs, Philippe Faist, Frédéric Dupuis, Jonathan Oppenheim and Renato Renner derived in 2015, in that framework, what they give as the least work any logical process needs, and when every input might occur and no error is allowed, their bound comes to $k_B T \ln 2$ times exactly that logarithm. Take an AND gate as the example: three inputs land on 0, so it needs at least $\log_2 3$, about 1.6, halvings' worth of work (two marks,

in whole numbers). The premise asks for that bound of a device that has to work whatever it's handed, and it counts it in marks. Joules don't come into it.

Theorem 4.6 (The squeeze bound)**.** On a finite machine with a permanent reading and a cost satisfying the halving price, let m be the number of states that read yes, and let a move i turn k different no-states to yes. Three numbers, then: m counts every yes-state of the machine, reachable or not; k counts a list of distinct no-states that the move turns to yes, and the move may turn others too; and $\mathrm{cost}(i)$ is the price the cost assigns to the move. Then

$$m + k \;\le\; 2^{\mathrm{cost}(i)} \cdot m, \qquad \text{and, when } m \ge 1, \qquad \mathrm{cost}(i) \;\ge\; \log_2 \frac{m+k}{m}.$$

The counting inequality holds as stated. When $m \ge 1$, it is equivalent to the displayed logarithmic lower bound.

Proof. Put the m yes-states and the k newcomers in one list; it has $m + k$ different members. By permanence every yes-state lands on a yes-state, and by hypothesis every newcomer does too, so all $m + k$ land among the m yes-states: $\mathrm{img}_i \le m$. The halving price gives $m + k \le 2^{\mathrm{cost}(i)} \cdot \mathrm{img}_i \le 2^{\mathrm{cost}(i)} \cdot m$. □

The four sheets are the case $m = 1$, $k = 3$, where the bound is reached. With $k = 1$ the bound says $\mathrm{cost}(i) \ge 1$, which is the toll again (`a2_from_compression_price_and_permanence`). Checked as `permanent_flips_compression_bound`, with the rounded-logarithm form `permanent_flips_log_bound` and the counting-only cousin `permanent_flips_collapse_at_least` (at least k states collapse, with no price premise at all).

The bound is reached for every $m \ge 1$ and every cost c. Whenever $m + k \le 2^c m$, a finite machine with a permanent reading and the halving price has m yes-states and a move of cost exactly c that turns a listed k distinct no-states to yes. The move turns every other no-state of that machine to yes as well, $(2^c - 1)m$ of them in all. No cost meeting the halving price on that machine charges the move less than c (`nec_f_squeeze_tight_every_k` and `nec_f_grid_cost_least`, NecFSqueeze.v). So the least whole-number cost that meets the bound is exactly $\lceil \log_2 \frac{m+k}{m} \rceil$ (`nec_f_least_cost_exists` and `nec_f_least_cost_is_ceiling`, NecFSqueezeLog.v).

Four more facts about the halving price, all checked, and the last one is a limit. First, the plainer reading above is a theorem on a finite machine: the halving price holds exactly when no move has a landing spot with more than $2^{\mathrm{cost}(i)}$ states landing on it (`ent2_compression_priced_iff_fibres`). Second, for the eight-state machine described later in this section that's a theorem about its own price list: no move has a

larger pile-up than 2^{cost}, so the halving price holds, and the toll follows from it (`ent2_frag_compression_priced`, `ent2_frag_toll_by_compression`); and the price list is the least that meets the halving price, since any price list that does charges every move at least what this one charges (`ent2_frag_cost_minimal`). Third, the squeeze bound is met with equality there: its stamp turns on the four states whose flag is down, beside the four whose flag is already up, and $4 + 4 = 2^1 \cdot 4$ (`ent2_frag_bound_attained`). Fourth, and this is the limit, the small machine of Part IV as a whole does not meet the halving price, whatever procedure you use to decide whether two of its states are equal, because one of its free moves, the one that takes one off a number the machine holds, lands two different states of the whole machine on the same state (`ent2_small_not_compression_priced`; the pair is in Theorem 16.42). That the halving premise fails there is a theorem (CompressionSmall2.v). What does hold there is the same premise asked of the record layer alone, and it gives the toll on that machine too (Theorem 4.15, below).

4.7 The same bill, written in bits, and when it's zero

There's one more way to say all this, and I include it because it's the way physicists say it and because it hands you the one case where the bill is exactly zero, which I find more instructive than any case where it isn't.

Claude Shannon, in 1948, gave a number to "how much you don't know": if something could be in any of n states and you have no idea which, your uncertainty is log base two of n bits, and that's the most it can be for n states. If you push your uncertainty through a step that forgets nothing, it stays the same; through a step that merges two states you might have been in, it drops. Now run the room. Before the move, you might be in any of the m + k states in play. After it, you're in one of m. So your uncertainty afterward is at most log2 of m, whatever you believed before. If you started with no idea at all, every one of the m + k states equally likely, your uncertainty was exactly log2 of (m + k), and the move removed at least log2 of (m + k)/m bits: the same number as the halving count, now in Shannon's units. Landauer's principle, in the form physicists write it, then says a step that removes that many bits from the state gives off at least $k_B T \ln 2$ of heat per bit. Put the two together and the flip gives off heat, and the exact statements below say how much and under which side conditions, one of which (the temperature has to be taken as zero or above) is easy to skip and shouldn't be.

Why does Shannon's number share a name with the physicists' entropy at all? There's a story about it. Shannon wanted to call it uncertainty, and the mathematician John von Neumann told him to call it entropy instead, for two reasons: the same formula already lived in statistical mechanics under that name, and nobody really

knows what entropy is, so in an argument he'd always have the upper hand. It's a great story. It comes from Myron Tribus, who spent years tying information theory to thermodynamics, who said Shannon told it to him in 1961, and who printed it, with Edward McIrvine, in *Scientific American* in 1971. Then in 1982 an interviewer, Robert Price, asked Shannon about it, and Shannon said he didn't think von Neumann had told him any such thing. So it's a legend with one witness, and the person the witness quoted doesn't remember saying it. What doesn't hang on the story: the two formulas really do have the same shape, and it still takes Landauer's premise, a named assumption, to turn a count of bits into heat.

Exactly, with the notation built first. A *spread* (probabilists say distribution) is a way of giving each state a chance $p(x) \geq 0$, with the chances adding to 1. Its Shannon entropy, in bits, is

$$H(p) \;=\; -\sum_{x:\,p(x)>0} p(x)\,\log_2 p(x),$$

where the big sigma means "add up, over every state x with a chance above zero," and each term is a state's chance times the bits it would take to single that state out. That comes to $\log_2 n$ when n states share the chance evenly and less when the chance is lopsided. Pushing a spread through a move i gives the spread of where you land: $q(y) = \sum_{x:\,\mathrm{step}(x,i)=y} p(x)$. The bits the move *removes* are $H(p) - H(q)$.

Start with the half of the claim that costs nothing, Bennett's half. A move that forgets nothing sends each state somewhere different, so the chances just get carried to new addresses, one for one. Nothing piles up and nothing spreads out. The uncertainty is the same number before and after.

Theorem 4.7 (A move that forgets nothing removes no bits)**.** Let the states be finite, listed once each, and let f send states to states with $f(a) = f(b)$ only when $a = b$. Then for every assignment of chances p, the pushed spread q has $H(q) = H(p)$.

Proof. An injective map on a finite list of distinct states is a reshuffle of that list: every state is hit exactly once. So $q(f(x)) = p(x)$ for every x, and the sum defining $H(q)$ is the sum defining $H(p)$ with its terms in a different order. □

Checked as `step_entropy_invariant_if_injective` (PermanentCertificationEntropy.v). That's why a price on removed bits can leave every reversible move free, which is the shape the toll needs: the bill lands only where something was forgotten.

Theorem 4.8 (Entropy after a permanent flip)**.** On a finite machine with a permanent reading, let m be the number of states that read yes, and let a move i turn $k \geq 1$ different no-states yes. For any spread p whose chance lies on those $m + k$ states, the spread after the move has $H(q) \leq \log_2 m$. For the even spread on the $m + k$ states, the move removes at least $\log_2 \frac{m+k}{m}$ bits.

Proof. After the move, all the chance sits on the m yes-states (permanence for the old ones, the hypothesis for the new). There is at least one yes-state: the image of a state that flips. Thus $m \geq 1$. Write $q_1, \ldots, q_r$ for the positive output probabilities, where $r \leq m$ and their sum is 1.

For $y > 0$, $\ln y \leq y - 1$: the derivative of $y - 1 - \ln y$ is $1 - 1/y$, so its minimum is 0 at $y = 1$. Apply this with $y = 1/(mq_j)$, multiply by q_j, and add:

$$-\ln m - \sum_{j=1}^{r} q_j \ln q_j \;\leq\; \sum_{j=1}^{r} (1/m - q_j) = r/m - 1 \;\leq\; 0.$$

Dividing by $\ln 2 > 0$ gives $H(q) \leq \log_2 m$. The even input spread has $m + k$ probabilities each $1/(m + k)$, hence $H(p) = \log_2(m + k)$. Subtracting the output bound gives the claimed loss $\log_2((m + k)/m)$. □

Checked as `permanent_step_entropy_ceiling` and, for the even spread, `permanent_flip_uniform_entropy_drop` (PermanentCertificationEntropy.v).

Premise 4.5 (Landauer's principle, as a premise)**.** A step that lowers the entropy of the machine's state by ΔH bits, at temperature T, gives off heat $Q \geq k_B T \ln 2 \cdot \Delta H$, where k_B is Boltzmann's constant. (Checked name: `landauer_heat`, with $k_B T$ as one variable.)

Corollary 4.9 (Heat floor of a permanent flip)**.** Under Premise 4.5, with $k_B T \geq 0$, the flip of Theorem 4.8 from the even spread gives off at least $k_B T \ln \frac{m+k}{m}$; with $k_B T > 0$ that is strictly positive.

Proof. Theorem 4.8 gives an entropy loss of at least $\log_2((m + k)/m)$ bits. Multiply by the nonnegative factor $k_B T \ln 2$ and use Premise 4.5. The identity $\ln 2 \, \log_2 x = \ln x$ gives the displayed floor. Here $m > 0$ and $k > 0$, so $(m + k)/m > 1$ and its logarithm is positive; if $k_B T > 0$, the heat floor is strictly positive too. □

Checked as `permanent_step_entropy_ceiling`, `permanent_flip_uniform_entropy_drop`, `entropy_le_log_support`, `permanent_flip_heat_floor`, and `permanent_flip_heat_positive` (PermanentCertificationEntropy.v).

How big is that? Take the case where the squeeze is two, one state flipping in beside one that was already yes, so $m = k = 1$ and the floor is $k_B T \ln 2$. At room temperature, about 300 kelvin, that's roughly three zeptojoules: three thousandths of a billionth of a billionth of a joule. And that's the *biggest* bill a single flip gets from this corollary. With $k = 1$ the floor is $k_B T \ln \frac{m+1}{m}$, and it shrinks as more states already read yes: with a thousand of them it's about a thousandth of $k_B T$, some seven hundred times less than the case of two. So, for an even spread over the states in play, one mark is not one halving. It's one halving exactly when the squeeze is two; for any thinner squeeze and an even spread, the physics asks for less than a halving, and the toll still charges a whole mark, because marks come in whole numbers (that's the rounding the entropy road to the toll leans on, a little further down). Read in the worst case over what the machine might be holding, the way the halving price reads a move, it's at least a full bit every time. A permanent flip merges two states (Theorem 4.1); put half the chance on each, and the move removes a full bit, whatever m is. Checked as `merge_pair_removes_a_bit` and `permanent_flip_spread_loses_a_bit` (PermanentCertificationEntropy.v). So the gap between the mark and the bits is a fact about the even spread. In the worst case the flip removes at least the full bit the mark pays for. Whatever the size, it's a bill only if you grant the premise; nothing on this page measured anything. And even granted, it's a bill nobody will ever see on a meter: real logic gates spend many thousands of times more than this on every switch, even the frugal ones. One 2023 tally of the numbers puts switching a single CMOS gate at a hundred attojoules at the low end, over thirty thousand times this. The point of the number isn't that certifying is expensive. It's that, on the premise, it isn't zero.

Physicists have a name for a step with no way back, and that logarithm has turned up in their work before. Yûto Murashita, Ken Funo and Masahito Ueda (2014) studied processes where some histories of the reversed process have no forward history at all. They call that *absolute irreversibility*, and they derived that the entropy such a process produces averages at least $-\ln(1-\lambda)$, where λ is the chance of those orphaned reverse histories. Their first example is a gas let out of a box that held a fraction l of the room, where the bound comes to $\ln(1/l)$. Now picture the even spread of my flip played backwards: the chance sitting on the m yes-states spreading back out over all $m + k$, a gas let out of a box holding $m/(m+k)$ of the room. Their bound for that is $\ln \frac{m+k}{m}$, the same logarithm as the heat floor, in units of $k_B T$. So is my squeeze a special case of their bound? No, and the reason is worth a minute, because the match is a good trap. The even spread gives a real chance to every state the flip starts from, so no forward history is missing: the flip itself isn't absolutely irreversible at all, and their λ for it is zero. The heat floor is ordinary Landauer heat, the bill for squeezing a spread over $m + k$ states into m of them, the same kind of

bill as any reset into a smaller set. The two logarithms match because a squeeze's Landauer heat equals the entropy produced by the free expansion that undoes it, and that free expansion, the flip played backwards, is the process their bound is about. A mirror. Neither result descends from the other. Their idea does reach the flip, just somewhere else: on a device whose rates never change, raising the flag from a state it knows, which turns up at the end of this section. Gonzalo Manzano, Gülce Kardeş, Édgar Roldán and David Wolpert (2024) carried absolute irreversibility over to computations, with one-way transitions and machines that stop at random times, worked on finite automata, and the same $-\ln(1-\lambda)$ term sits in their second-law bounds.

The even spread is the easy case, and you might worry the heat depends on picking it. It doesn't. Any spread that gives a real chance to every yes-state and to the state that flips will do.

Corollary 4.10 (Positive heat for any full spread)**.** On a finite machine with a permanent reading, let move i turn a no-state s yes. Take any spread p that gives positive chance to s and to every yes-state. Under Premise 4.5 with $k_B T > 0$, the heat the move gives off is strictly positive.

Proof. After the move, s and the yes-states all land among the yes-states, and there is one more of them than there are yes-states, so two distinct states with positive chance land on the same state. That merge removes a positive number of bits (Theorem 4.14 below), and a positive temperature times a positive number of bits is positive heat. □

Checked as `permanent_flip_full_support_entropy_drop_positive` and `permanent_flip_full_support_heat_positive` (PermanentCertificationEntropy.v).

There's a third road to the toll here too, and it's the same shape as the merge road, with bits in place of marks for forgetting.

Definition 4.6 (Entropy-priced)**.** On a finite machine, a cost is *entropy-priced* when, for every move i and every spread p, the cost of i is at least the bits the move removes: $\mathrm{cost}(i) \geq H(p) - H(q)$, where q is p pushed through i.

Theorem 4.11 (The toll from entropy pricing)**.** On a finite machine with a permanent reading and an entropy-priced cost, every move that turns the reading from no to yes costs at least one. So the machine with that cost is a certification system (Definition 3.1), and every run from a no-state to a yes-state costs at least one.

Proof. Let move i turn s yes, and let m be the number of yes-states; $m \geq 1$, since $\text{step}(s, i)$ is one. Take the even spread on the m yes-states and s. By Theorem 4.8 with $k = 1$, the move removes at least $\log_2 \frac{m+1}{m} > 0$ bits, so the cost of i is a whole number above a positive number, which makes it at least one. The run-level floor is the universal floor (Observation 3.1) applied to that system. □

Checked as `a2_from_entropy_price_and_permanence`, with the packaging `certification_system_from_entropy_price` and the run-level floor `entropy_priced_trace_floor` (PermanentCertificationEntropy.v).

So there are three roads to the toll: merge pricing, the halving price, and entropy pricing. They don't compete. They're one premise written three ways, and that's checked too. Ask of a move the most bits it can remove from any spread at all, which is Landauer's principle taken in its worst case. That most is exactly the base-two logarithm of the move's worst pile-up: no spread loses more, and the even spread on the biggest pile loses exactly that much. A cost in whole marks that is at least that logarithm is a cost c with no pile bigger than 2^c, and that's the halving price. Merge pricing is the same premise asked about two states at a time.

Theorem 4.12 (One premise, three forms)**.** Take a finite machine whose states can be decided equal or different, and a move i. (1) If no state has more than $K \geq 1$ states landing on it under i, every spread loses at most $\log_2 K$ bits through i; and if some state has exactly K states landing on it, the even spread on those K loses exactly $\log_2 K$ bits. (2) A cost is entropy-priced (Definition 4.6) exactly when it satisfies the halving price (Premise 4.4). (3) A cost satisfies merge pricing (Premise 4.2) exactly when it satisfies the halving price for every list of at most two different states, and exactly when it satisfies entropy pricing for every spread whose chance sits on at most two states. The first of those two equivalences uses no finiteness, only the test for equal states.

Proof. (1) Split the bits removed into one term per state, as Theorem 4.14 below does, and bound each term with $\ln y \leq y - 1$ at $y = q(\text{step}(x, i))/(K\,p(x))$. Adding up, the total is at most $\log_2 K$ plus a multiple of $\frac{1}{K}\sum_x q(\text{step}(x, i)) - 1$, and that sum counts each state's chance once for every state landing beside it, at most K times, so the extra part is at most zero. For the even spread on a pile of K, the chance starts on K states, $\log_2 K$ bits, and ends on one, which is zero bits. (2) If the cost is entropy-priced, take the even spread on any pile: by (1) the cost is at least $\log_2$ of its size, and a whole number c at least $\log_2 n$ has $n \leq 2^c$. That every pile is at most 2^c is the halving price (the plainer reading above). The other way, (1) with $K = 2^c$ says no spread loses more than c bits. (3) Two different states that land together have one image, so the

halving price for that pair reads $2 \leq 2^{\text{cost}(i)}$, which is $\text{cost}(i) \geq 1$; a pair that doesn't land together, or a single state, asks nothing. A spread on two states loses at most one bit, and none unless they land together; the even spread on two that do loses exactly one. □

Checked as `entropy_drop_le_log_fibre`, `entropy_drop_of_fibre`, `worst_case_entropy_drop`, `entropy_priced_iff_compression_priced`, `merging_priced_iff_pair_halving`, `merging_priced_iff_pair_entropy` and `one_premise_three_forms` (PricingOnePremise.v). So the premise under all three roads is one premise, Landauer's in its worst case over what the machine might hold, and I wrote it three ways because each way makes a different theorem short. Merge pricing asks it only about pairs, which is why that road gives the toll and stops there; the squeeze needs the whole premise.

Now the zero. Put a coin under a cup without looking. Rule: set it heads up. You might have had tails; now you have heads; one bit of what you didn't know is gone, and that's the bit the principle charges for. Now lift the cup, see heads, put the cup back, and apply the rule again. Nothing you didn't know changed. No bit was lost. And the principle, applied to a state you already knew, demands nothing: the premise holds exactly when the heat is zero or more, which is to say it sets no floor at all. The principle charges for *uncertainty removed*. The shape of the move doesn't enter. This is the generalized form of Landauer's principle, worked out by Owen Maroney in 2009, in a paper he wrote at the Perimeter Institute in Canada (he has been a philosopher of physics at Oxford since 2011), and by Sagawa in 2014, and Landauer himself had the seed of it in 1961: for a row of bits all already ONE, he wrote, there's no need to carry out any operation, so no entropy changes and no heat is given off, and resetting a row all ZERO to ONE involves no entropy change either, because neither row carries any information.

Theorem 4.13 (A known state gets no heat floor from the premise)**.** On a finite machine, if the spread puts all its chance on one state s, then any move removes zero bits, and Premise 4.5 for that move holds exactly when $Q \geq 0$.

Proof. A spread on one state has entropy 0, and so does its image, which is again on one state. So $\Delta H = 0$ and the premise reads $Q \geq 0$. □

Checked as `known_state_step_removes_no_entropy` and `known_state_flip_forces_no_heat` (PermanentCertificationEntropy.v).

Be careful with that one, because it's easy to overread. "The premise sets no heat floor for a known state" does not mean a stamping machine run on a known-blank sheet gives off no heat. A fixed device that merges states dissipates on known input

too; Bennett calls that wasteful, because entropy goes out into the room with nothing taken from the data. What the theorem says is only that the *premise* sets no floor there. The floor comes from what you didn't know. That's a sentence about the bill. The device is a separate matter. And notice that the toll still charges the known-state stamp one mark, because the toll is a rule about the *move*, and the move doesn't know what the observer knows. That's the worst-case gap from door three showing up again, and it's the price of having a rule a machine can follow without asking who's watching.

The physics of all this has a standard review, the physicists Juan Parrondo, Jordan Horowitz and Takahiro Sagawa's "Thermodynamics of information" (2015), which sets Landauer's bound, its generalized form and the costs of measurement and feedback side by side. And the gap between a device and the spread it runs on has a physicist's number on it. Artemy Kolchinsky and David Wolpert (2017) showed that a fixed process built to dissipate as little as possible for one spread over its states pays extra when it runs on another, and the extra is how much closer the process brings the two spreads, measured in relative entropy; the extra was later named mismatch cost. That's the worst-case gap again, priced for an actual spread, where the toll prices the move.

There's a second piece of fine print on that theorem, and it's about who works the controls. The generalized principle can charge a known state nothing because it lets the step be driven by a protocol that changes in time: lower this barrier, tilt that well, raise the barrier again, each at the right moment. Something outside the step keeps that time, an outside clock or an outside hand, and the premise doesn't bill it. Take the hand away and the bill changes. Paul Riechers, Alexander Boyd, Gregory Wimsatt and James Crutchfield (2020) analysed what happens when the control is *time-symmetric*, the same played forwards or backwards, which covers a clocked pulse and any device whose rates never change. On their analysis, every move that doesn't come back to where it started when you apply it twice, which they call *nonreciprocal*, costs at least $k_B T \ln(1/\varepsilon)$ on top of the change in free energy, to run with error at most ε. A flip that stays up is always nonreciprocal: apply it twice from the no-state and you're still at yes. So, if they're right, under that kind of control the known-state stamp isn't free after all, and the more reliable you want it, the more it costs. Their class is bigger than mine, though. Erasure is nonreciprocal, and so is a counter of three or more values that steps forward and wraps around, which merges nothing, so their bound is about what the controls are allowed to do, records or no records. So read Theorem 4.13 with its control written in: Landauer's premise sets no positive heat floor for a known state when the step can be driven by a protocol timed from outside it. The same goes for Theorem 4.4. On a machine whose move names can be decided equal or different, the moves every

merge-pricing schedule must charge are exactly the merging moves, and that's true against merge pricing; measured against time-symmetric control, the forced moves would be, on this argument and unchecked, the nonreciprocal ones, the wrapping counter included. Which moves the physics forces a price on depends on what the controls can do.

A deterministic run from a known start has a known state at every step. For such a run, the adopted entropy-loss premise reduces to $Q \geq 0$: it supplies no positive heat lower bound. That does not establish that a particular device attains zero heat. The chairs guarantee a collision among the already-yes states together with the entering no-state. They do not guarantee that the entering state takes part in the collision (a four-state machine where it doesn't is `nec_f_collision_avoids_s`, NecFMerge.v). For an input distribution supported only on no-states, this counting argument alone therefore forces no positive entropy loss. Whether other states in that support merge depends on the move. The operation-level ledger toll still follows under merge pricing.

So here's what this section actually earns. A device with finitely many states that has to raise a flag and keep it up correctly *from every state it might be in*, including states where it can't tell whether the flag is already up, can't do it for free at any temperature above zero, on Landauer's premise; that's the design envelope, the worst case. A particular run from a start it knows can, on the same premise, as long as something outside the step does the timing. What stays forced in every case is the toll as a rule about the move, because the rule can't ask what the run knows. Wherever this book says that settling something for good can't be free if Landauer's right, it means the device sentence. It never means the run sentence.

Here's the general case, and it's the one that shows you where the bits go. Split the bill state by state. Each state x you might be in carries its chance $p(x)$ along to where it lands, $\mathrm{step}(x, i)$, and finds whatever other chance landed there too. If nobody else landed there, its term is zero. If others did, its term is the difference between how crowded its landing spot is and how much of the crowd it brought.

Theorem 4.14 (The bits removed, one term per state)**.** On a finite machine, for any move i and any chances $p(x) \geq 0$, with q the pushed spread,

$$H(p) - H(q) \;=\; \sum_{x:\, p(x)>0} p(x)\,\big(\log_2 q(\mathrm{step}(x,i)) - \log_2 p(x)\big).$$

Every term is zero or more, so a move never adds bits. If two different states $x \neq x'$, both with positive chance, land on the same state, the sum is strictly positive.

Proof. Group the sum for $H(q)$ by where things land: the chance $q(y)$ at a landing spot is the sum of the $p(x)$ that land there, so $H(q) = -\sum_{x:\,p(x)>0} p(x) \log_2 q(\mathrm{step}(x,i))$. Subtract from $H(p)$ term by term. Each term is non-negative because the landing spot holds at least what x brought, $q(\mathrm{step}(x,i)) \geq p(x)$, and the logarithm keeps order. If x and x' share a landing spot, that spot holds at least $p(x) + p(x') > p(x)$, so the term for x is strictly positive. □

Checked as `entropy_drop_as_point_sum`, `entropy_drop_nonneg`, and `entropy_drop_pos_of_support_merge` (PermanentCertificationEntropy.v).

4.8 What you're entitled to, and what you're not

Let me total it: what have you earned the right to conclude, and what is still a door?

You've earned this: on any machine with finitely many states, a move that raises a flag either forgets something or takes a flag down somewhere else. Pure counting, no physics, no premise beyond finiteness, and the half of it where nothing comes down is the four chairs. You've earned this: on a finite machine, if a flag, once up, stays up, then raising it forgets something, and a move that raises k states collapses at least k of them. Counting again. And you've earned this: if forgetting is charged for, the toll follows, and if forgetting is charged per halving, the size of the toll follows too; each of those derivations hands back a system with the five parts and the toll, so Section 3's floor applies to it as it stands. That's the rule from Section 3, now derived, with every premise named, for any finite machine with a flag that stays up. In that section I only declared it.

Here's what you haven't earned. I'm listing it so you don't assume it. You haven't earned that forgetting costs anything in the world; that's Landauer, it's measured for beads and argued over for the general case, and in the checked work it's an assumption with a name. And even granting it, Landauer charges a run that knows where it is nothing at all (the lifted cup), so what the physics forces is a floor on a device that has to work whatever it's holding. Any one run can come in under it. You haven't earned that any machine you'd actually want to build is of this kind, either, and there are two easy ways to miss. A machine that keeps its running total inside its own state, as a number with no ceiling, has infinitely many states, so the finite premise fails for it as a whole. And you can't patch that by giving the total a ceiling. A total that only climbs, held in finite memory, eventually either stops at its top, and then two different histories leave the same top value, which is a merge, or rolls back over to zero, and then it doesn't only climb. So any running total a finite machine keeps for itself has a horizon: it's honest up to its width and no further. And a machine whose price list charges the step that raises the flag, but lets some other step merge two states for nothing, raises the flag honestly and still doesn't charge

for forgetting in general, so the merge-pricing premise fails for it. Either one puts a machine outside these theorems as stated, though a finite piece of it can still sit inside. The machine I build later (Section 16) misses in both ways. It keeps counters with no ceiling, so it's infinite outright, and some of its ordinary steps, counting a counter down among them, can land two different states on one for free. So the chairs, as stated, don't reach it: they force the toll only on finite machines that price every merge, and it isn't one. It doesn't need the chairs, though, and that comes after the eight-state piece below.

A finite piece can still sit inside, and here's the smallest one I could build where every premise is a theorem. None of them is left as a hypothesis. Four slots times a flag, eight states. Three moves: stamp (set the flag, go to the next slot, cost one), jump to slot a (cost two), next slot (cost zero). It's finite (`fin_finite`) and permanent (`fin_permanent`); the next-slot move forgets nothing (`fnext_injective`); the stamp and the jump merge (`fcertify_merges`, `fjump_merges`); merges are priced (`fin_merging_priced`), and priced per halving (`fin_compression_priced`). So this section's two conclusions hold on it outright (`fin_a2_from_merging_price`, `fin_a2_from_compression_price`, FiniteCertMachine.v). The counting lemma underneath, that a duplicate-free list with at most K preimages per image has at most K times as many members as images, is proved generically (`fiber_bound_compression`). Merge pricing alone would take any price of at least one for the jump. The jump's price of two is what the halving price asks, since the jump squeezes four states into one. One more fact about it, also checked: each move's price is exactly the number of halvings in its largest squeeze. For a move that costs c, no state is landed on by more than 2^c states, and some state is landed on by exactly that many (stamp squeezes at most two states into one, jump to a slot squeezes four, and next-slot squeezes nothing).

The machine Part IV builds runs this eight-state machine. Seen through a window that shows only a line number, read modulo four as the slot, and a flag, a short program of that machine, once a two-instruction setup has run, does what the eight-state machine does, one move at a time, and pays exactly what the eight-state machine charges (Section 16.17 has the construction and the exact statements). The machine as a whole is still not merge-priced: some of its free steps merge states, so the finite piece is a piece. The toll on the whole machine comes another way.

Finiteness did one job in the chairs. It forced the step that raises a permanent flag to merge two states. On the machine of Part IV nothing has to force that, because the merge is in plain view. Its CERTIFY takes a state with the flag down and the same state with the flag up to one place, since a flag that's already up just stays up. Its COMMIT overwrites the channel, so two states that differ only in what the channel names land on one state. Both are checked. So any price list that charges every merge

of that machine's own step charges the flag-raising step, and finiteness never comes into it.

That leaves the other miss, the free merges. Clause (c) of Thiele-complete, in Part IV, asks for base moves at zero, and a base move merges (DEC, counting a counter down, lands two states on one), so full merge pricing and clause (c) can't both hold on that machine. They don't have to. Split a state into its base part, which is what a two-counter machine would show (the line number and the two counters), and its record part, which is everything else: the versions, the table of facts, the channel, the trap latch, the ledger and the flag. Then price only the merges the record layer makes.

Premise 4.7 (Record-layer merge pricing)**.** A move that sends two different states with the same base part to one state costs at least one.

The merges among base parts are the ordinary computer's business, the ones any two-counter machine makes, and whatever heat they cost is the ordinary computer's heat. Merge pricing implies this premise, so it asks less, and on the part of the state that's new it's still Landauer's premise.

Theorem 4.15 (The toll on the machine I built)**.** For the small machine of Section 16, with no finiteness premise: (1) CERTIFY merges, and so does COMMIT of every claim, and each joins two different states with the same line and counters. (2) Every cost that satisfies merge pricing for that machine's step charges CERTIFY and every COMMIT at least one, and obeys the toll, so the machine with that cost is a certification system. (3) No INC, DEC or HALT sends two different states with the same line and counters to one state. So the machine's own prices, base moves at zero and each record move at one, satisfy Premise 4.7, and merge pricing fails for them. (4) Every cost that satisfies Premise 4.7 obeys the toll, so the machine with that cost is a certification system.

Proof. (1) Take a state whose channel holds a commitment, once with the flag down and once with it up. CERTIFY raises the first, leaves the second up, moves both on a line and charges both one, so they land together. For COMMIT, take a table holding the claim about the current version, once with an empty channel and once with the channel already naming that claim; COMMIT writes the same name in both. (2) Only CERTIFY raises the flag, and it merges. (3) A base move keeps the table, the channel and the trap latch, adds nothing to the ledger and leaves the flag alone, and when it writes a counter it moves that counter's version up by one; so two states that agree on line and counters and land together agree on everything. A trapped state keeps

everything but its ledger, and a live one stays live, so a trapped state and a live one never land together. Merge pricing fails because of the free DEC of Theorem 16.42, whose two states differ in a counter. (4) Only CERTIFY raises the flag, and by (1) its merge is a record-layer merge. □

Checked as `frag_small_certify_merges` and `frag_small_not_merge_priced` (FragmentSmall.v), and `rm_commit_merges`, `rm_merge_price_charges_record_moves`, `rm_toll_from_any_merge_price`, `rm_cert_system_from_merge_price`, `rm_merge_price_gives_record_price`, `rm_certify_record_merge`, `rm_commit_record_merge`, `rm_base_moves_no_record_merge`, `rm_small_record_priced`, `rm_toll_from_record_price`, `rm_cert_system_from_record_price` and `rm_record_price_and_exact_toll` (RecordMerge.v).

So this section reaches the machine I built. On that machine the toll is forced by Landauer's premise asked of the record layer, and the machine's own price list, clause (c) included, is one the premise allows. There's a second way out, which I note and didn't take. Morita's reversible two-counter machine, from the history earlier in this chapter, runs every step backwards and is still universal: with its input encoded, it computes anything a Turing machine can. Put the record layer on a base like that and every base move is one-to-one, full merge pricing charges the base nothing, and clause (c) holds with merge pricing as it stands. I haven't built that machine or checked it.

And you haven't earned anything about *what* the flag means. Four chairs don't know what they're certifying. The bill is for raising a flag that stays up. Whether raising that flag ever amounts to standing behind a structural fact, the "vouching" I keep promising and keep not delivering, is a question for the machine of Part IV, and here it has moved exactly this far: on a finite machine that charges for every merge, if that flag stays up, it was paid for.

One last thing, because it should nag at you. The toll is forced by the premises, but the premises were chosen, and "permanent" in particular was chosen by me. Why should a certificate stay up? Why not let it come down, pay nothing, and be honest about it? That's a fair question, and merge or revoke is the start of an answer (the three-card machine only showed that the converse of Corollary 4.5 fails): on a finite machine that charges for every merge, a flag that goes up for free is a flag the same move takes back somewhere, and a record that can be silently taken back is the thing the bulla and the hash chain and the write-once disk were all built against. "Permanent" here is exactly that: the flag can't be lowered. The toy in Section 3 didn't have it; the lineage in Section 3 was built around it; I chose it. Whether that was the right job to pick out of the five is something you can argue with me about, and I'd want you to.

And "permanent" is an idealization, which I should say before a physicist does. No physical record lasts forever. A bit held behind an energy barrier leaks over it at some rate, however slow, and a closed finite system comes back around, which is Zermelo's point from the chairs. The theorems in this section need permanence on every state, including states the machine never visits, forever. Weaken that to "the flag stays up on the states that actually turn up, for as long as anyone cares," and merge-or-revoke tells you what goes wrong: the move could take the flag down somewhere nobody ever looks, and then it isn't forced to merge at all. A version of the toll stated in chances and time spans would be a different theorem, and I don't have it.

One piece of it I can hand you, and it's checked. Take one move and repeat it from one start, forever. On a finite machine the states that run passes through are finitely many, so it falls into a loop. If the move forgot nothing on those states, it would be a reshuffle of them, and a reshuffle brings every one of them back around, the start included, where the flag was down. So if the flag goes up along that run and stays up from some point on, the move merges two states *the run itself visits*. The shared chair is one the machine actually sits in. And it doesn't touch the known-state point above: a merge among states the run visits still costs Landauer nothing if the run knew where it was and something outside timed the step. Checked as `nec_f_orbit_merges_visited` and `nec_f_repeat_merges_visited` (NecFRepeat.v), for any map iterated from a start whose flag is down, with the flag up at every step from some point on; no permanence premise is used.

Runs that mix different moves split two ways, and the answer is short enough to give here, checked like the last one. If the moves are picked from outside, in any order at all, the answer is no. Take two states, one reading no and one reading yes. Move a swaps them; move b does nothing. Run a once and then b forever: the flag goes up at the first step and stays up for good, and neither move merges anything anywhere. The swap would take the flag down from the yes-state if it ever ran there; on this run it never does, and that's exactly the weakened permanence this paragraph is about. But if the machine is closed, picking its next move from its own state, the run is the orbit of one map, the step with whatever move the state picks, and the single-move argument applies to that map as it stands. Now say the moves come in a pattern that eventually repeats, one word w of moves over and over. Suppose the flag is down at some step after the repeating starts, and goes up and stays up after that. Count the repetitions from a step where the flag was down (start w at that letter if you have to) and run the same argument on the composite of w's moves, at the states where such a repetition begins. It merges two of those states, both visited; follow the two through the letters of w, and the first letter where their paths meet merges two states the run visits at that letter's place in the pattern. So:

moves picked from outside, no; a closed machine or a repeating program, yes (`nec_f_outside_moves_no_merge`, `nec_f_closed_merges_visited` and `nec_f_word_merges_visited`, NecFRepeat.v; the last finds the letter and the two repetitions, from a flag down at the start and up at every repetition from some point on).

And one limited piece of the chances-and-time-spans version, set in continuous time and checked in all but one step, because it shows exactly where the known-state escape stops. Take a device with no outside hand at all: a little random process hopping among finitely many states at rates that never change, with the usual physical conditions (every hop has a reverse hop, and the rates obey local detailed balance), able to get from any state to any other. Left to run, it settles into a long-run spread π and spends stretches with the flag up and stretches with it down; call their mean lengths T_{up} and T_{down}. Start it in a no-state you know. Then the entropy it produces from then on is at least $\ln\bigl(1 + T_{\text{up}}/T_{\text{down}}\bigr)$, in units of k_B. The argument is three lines once the pieces have names. With fixed rates, the entropy produced is at least the drop in relative entropy between the device's spread and π; that's half of Massimiliano Esposito and Christian Van den Broeck's split of entropy production (2010). A start known to be in state s has relative entropy $-\ln \pi(s)$, which is at least $-\ln \pi(\text{no})$, where $\pi(\text{no})$ is the long-run chance of the flag being down, and the relative entropy runs down to zero as the device settles. And the long-run chances of up and down stand in the ratio of the mean stretch lengths, because the crossings up and the crossings down balance, so $-\ln \pi(\text{no}) = \ln\bigl(1 + T_{\text{up}}/T_{\text{down}}\bigr)$. Read it this way: each doubling of how long the record stays up, against how long it sits down, costs about one more $k_B \ln 2$, and a record that stays up forever on a fixed-rate device costs without bound. That's Murashita, Funo and Ueda's absolute irreversibility landing on the flip at last, with $1 - \lambda$ the long-run chance of the starting state. It isn't new physics: nothing in it needs a record, since an autonomous reset to a fixed state obeys the same bound, and the time-symmetric-control result above covers the same ground with an error rate in place of the time ratio. And it isn't the theorem I said I didn't have. It's a first honest step toward it, and it pins down where "the premise sets no heat floor for a known state" needs its outside clock. Hold that argument to what it says. The entropy is added up from the moment the device sits in the known no-state s until it has settled, over one whole relaxation. The starting spread is all on s, and π is the device's long-run spread. The inequality rests on fixed rates, finitely many states, every state reachable from every other, a reverse for every hop, and local detailed balance to read the entropy as heat. It's one bound on one relaxation, and it doesn't charge each certification again. It also isn't the permanence of the finite-state theorems on a noisy device: on this device the flag comes back down, and T_{up} and T_{down} are average stretches. The discrete-time core is checked, for a finite chain whose steady spread π is positive and satisfies detailed

balance (RelaxationEntropy.v): the entropy produced in one step is exactly the drop in relative entropy to π, and never negative (`re_sigma_is_drop` and `re_sigma_nonneg`); from a known start s the entropy produced in the first T steps is $-\ln \pi(s)$ minus the relative entropy left at step T, so it never exceeds $-\ln \pi(s)$ and reaches it in the limit exactly when that remainder goes to zero (`re_known_start_total` and `re_known_start_bound`); $-\ln \pi(s) \geq -\ln \pi(\mathrm{no})$ for a start among the no-states (`re_known_start_vs_set`); the flow from the no-states to the yes-states equals the flow back (`re_flux_balance`); and with each mean stretch taken as its mass over that flow, $-\ln \pi(\mathrm{no}) = \ln(1 + T_{\mathrm{up}}/T_{\mathrm{down}})$ (`re_stretch_ratio`).

That's the bound, and two things under it were still promises when I stated it: that the device actually settles, and that "mean stretch length" means what it sounds like. Both are checked now, and here's what each one says in plain words.

Settling first. A chain doesn't have to settle. It can get stuck in a corner, or cycle round forever. What rules that out here is a condition named for Wolfgang Doeblin, a German-born French mathematician, son of the novelist Alfred Döblin, who took his doctorate at the Sorbonne in 1938 for work on chains like this one and died in June 1940, at twenty-five. In words: from wherever the device is, its next step lands on each state with at least some fixed fraction δ of that state's long-run chance. No state is ever out of reach in one step, and none is favoured so heavily that the others starve. Under that condition each step shrinks the distance between where the device is and where it's going by the factor $1 - \delta$, so the distance dies off geometrically and the spread converges to π at every state (`rc_contract` and `rc_converges`, RelaxationConvergence.v). Here's the payoff for the bound. The entropy produced from the known start stays under $-\ln \pi(s)$, and it also gets there: after T steps it falls short by at most $(4/m)(1-\delta)^{2T}$, where m is the smallest long-run chance of any state (`rc_known_start_gap` and `rc_known_start_limit`). So $-\ln \pi(s)$ is the exact price of the whole relaxation, and the bound is reached.

The same story holds when the device hops at random moments, at fixed rates, with no clock ticking. At every instant the relative entropy to π falls at exactly the rate entropy is being produced, and that rate is never negative (`rcd_D_derivative`, `rcd_sigma_nonneg` and `rcd_D_monotone`, RelaxationContinuous.v); add the rate up between two times and you get exactly the drop in relative entropy between them (`rcd_produced_window`). For settling, Doeblin's condition becomes a condition on rates: the rate of hopping from any state x to any state y is at least c times $\pi(y)$, for some fixed $c > 0$. Then entropy is always being produced at least c times as fast as the relative entropy still left, and that forces the relative entropy down at least as fast as e^{-ct} (`rcl_sigma_ge` and `rcl_D_decay`, RelaxationContinuousLimit.v). One wrinkle needed its own proof. A known start puts all its weight on one state and none anywhere else, and the logarithms in the relative entropy go bad at zero. The

check shows the relative entropy is still continuous at the starting instant, because $u \ln u$ goes to zero as u does (`rcl_D_at_zero`). So adding up the production from the start out to infinity gives exactly $-\ln \pi(s)$ (`rcl_known_start_total`).

I should say what Doeblin's condition costs.It asks for every hop to be possible directly, in one step or at a positive rate. A device whose states reach each other only through other states can still settle, and the standard theory of these chains says the conclusion holds for it too, but the proofs here don't cover that case. For a chain that ticks, the condition holding for some fixed number of steps would be enough for the same argument; I've checked it for one step.

Then the stretches.The bound used the fact that the long-run chances of up and down stand in the ratio of the mean stretch lengths, and I took the mean length of a stretch with the flag up to be the long-run weight of the up-states divided by the flow into them. That's a theorem, and it's checked in steps. Start each stretch the way the flow starts them, count the steps until the chain leaves the up-states, and the average is exactly $\pi(\mathrm{up})$ over the flow, again under Doeblin's condition (`rs_mean_stretch`, RelaxationStretch.v). The proof is a telescoping sum: the long-run weight that has stayed up for k steps splits into the stretches that started just now and the weight that stays up one step more.

With the stretches measured in seconds,when the device hops at random moments, it's checked too, and the route is different. Ask, for each up-state, how long on average the device takes to leave the up-states from there. Call that its leaving time. The leaving times obey one plain rule: from any up-state, the rate out of it times its leaving time, minus the rates into the other up-states times their leaving times, comes to exactly one second. Average the leaving times over the way the flow starts stretches, multiply by the flow, and the rule plus the balance of the steady spread (as much weight flows into each state as out of it) rearrange the sum into exactly $\pi(\mathrm{up})$ (`rsc_mass_over_flow`, RelaxationStretchContinuous.v). That the rule has a solution at all needs proving, since nothing promises it in advance, and Doeblin's condition supplies it: run a clock that ticks faster than any rate in the chain, and the leaving times are the limit of a count that only grows and never passes one over $c\,\pi(\mathrm{down})$ (`rsc_leave_exists`). Last, the seconds themselves. Take the chance that a stretch started by the flow is still running at time t, with the chain stopped the moment it leaves the up-states. That chance starts at one and falls at least as fast as $e^{-c\,\pi(\mathrm{down})\,t}$ (`rsc_S_decay`). The time spent up, which is that chance added up over time, comes out to the averaged leaving time minus what's still running, and as time runs on it closes in on $\pi(\mathrm{up})$ over the flow (`rsc_mean_stretch_time` and `rsc_stretch_identity`). Mass over flow, in seconds. The bound's two premises are both checked now, in steps and in seconds.

The next section is about a different door entirely. Forgetting has had its section; the question now is whether *finding out* costs, because the demon's whole trick was that it knew which way the molecule was going, and I owe you a straight answer on whether knowing is free.

5

Learning is free, committing is not

I ended the last section owing you an answer about knowing. The bill so far sits on forgetting: on a finite machine, a flag that stays up was paid for because raising it merged two states. But there's a thing that looks like the opposite of forgetting and feels like it ought to cost at least as much, probably more. Finding out. Every time a machine rules out a possibility, it knows more than it did. If forgetting a bit costs, does learning one cost too? I had a strong feeling about this, and you probably have it too, so here it is first: learning is the expensive part. Most physicists had that feeling as well, for a long time. It's why the demon took a hundred and fifteen years to exorcise. Or didn't. "Exorcise" is the word the people who think it worked like to use. John Earman and John Norton, the doubters from the last section, called their two-part paper on it "Exorcist XIV: The Wrath of Maxwell's Demon," and I read that Roman numeral as a joke about how many times the demon has been declared gone. So hold the word loosely. This section is me finding out where the feeling goes wrong. I'm going to tell you the version where the exorcism worked, because it's the version that says where the bill goes, and the door from the last section stays open behind it.

5.1 A small being at a small door

You met Maxwell's demon in Section 3, in one paragraph. It gets a whole section here, because the step it seemed to pay for and the step it actually pays for are what this section is about.

In December 1867 James Clerk Maxwell, who had published his equations for electricity and magnetism two years before, in 1865, and to my mind had as good a claim as anyone to being the most important physicist alive, wrote a letter to his friend Peter Tait describing a problem he couldn't get rid of. Take a box of gas at one temperature, with a wall down the middle and a tiny door in the wall. Now imagine, he said, a "very observant and neat-fingered being" sitting at the door, who can see individual molecules coming. When a fast one approaches from the left, it opens the door and lets it through. When a slow one approaches from the right,

same. Otherwise it keeps the door shut. It never pushes anything; it only opens and closes a frictionless door at the right moments. After a while the right side is hot and the left side is cold, and you've got a temperature difference you could run an engine off, made out of nothing but *looking*. That's supposed to be impossible. It's the thing the second law of thermodynamics says can't happen, and the second law is the law physicists are least willing to give up; Arthur Eddington, the English astronomer, wrote in a 1928 book of popular lectures that a theory found to contradict it has nothing for it but to collapse in deepest humiliation, and nobody has had to test that yet.

Maxwell put it in his textbook in 1871, and it got named Maxwell's demon, though not by Maxwell. William Thomson, later Lord Kelvin, called it a demon in *Nature* in 1874, and Maxwell, in an undated note to Tait headed "Concerning Demons," tried to take the name back: "Call him no more a demon but a valve . . . " Nobody listened, which is probably for the best, because "Maxwell's valve" would not have lasted a century. But keep his correction, because it's exactly right and it's the whole problem: the being does no work. It opens and shuts a door. So physicists spent the next century trying to find where the being was cheating. I want you to feel the pull of the obvious answer, because I felt it and so did most of them: the being has to *see* the molecules. Seeing costs. Case closed. In 1929 Leo Szilard built a one-molecule version of the box and did the arithmetic, and said the second law survives if finding out which side the molecule is on costs at least as much entropy as the being gains. In 1951 the French physicist Léon Brillouin made it concrete: to see a molecule you have to bounce light off it, and the way the philosopher of physics Jeffrey Bub sums up the result, finding out that way costs at least k ln 2 of entropy for every bit, which is kT ln 2 of heat, Landauer's number before Landauer, and there was your toll on looking. For thirty years that was the answer. Information costs because getting it costs.

That's a price on one way of looking, and it doesn't show that every possible way of making a record has to pay it. A device can burn energy without that being a law.

Szilard is worth one more minute, for a reason that has nothing to do with the demon. Four years after the one-molecule engine, in September 1933, by his own later telling, he was waiting at a traffic light on Southampton Row in London, having just read a newspaper report in which Ernest Rutherford, the physicist who had announced the atomic nucleus in 1911, called the hope of getting power out of atoms moonshine, and as the light changed it occurred to him that an element which gave off two neutrons when it swallowed one could keep a reaction going by itself. That's the nuclear chain reaction, thought up crossing a street. It's his memory of himself, and Richard Rhodes's history of the bomb opens with it, in a first chapter called "Moonshine," which is where most people meet it, so hold it as the man's own story.

A timestamp it isn't. Does it change anything about the demon? No. What it adds is a picture of the man: the same mind looked at one molecule in 1929 and one neutron in 1933, and both times the question was what a single small step forces downstream.

5.2 Bennett moves the bill

In 1982 Charles Bennett, who'd already shown in 1973 that computation can be done without forgetting, took the demon apart one step at a time. The being looks at a molecule and writes down which side it's on. Is writing that down a merge? Picture the being's notepad, which starts blank. After looking, the notepad says "left" or "right." Two possible worlds before (molecule left, pad blank; molecule right, pad blank). Two possible worlds after (left, "left"; right, "right"). Nothing merged. Two in, two out, and you can read the before off the after. Copying what you see onto a blank page forgets nothing, and a step that forgets nothing is a step Landauer doesn't charge for. Brillouin had priced one particular way of looking (bouncing light) and took it as the price of getting information at all. Bennett gave a way of looking that, on his account, doesn't pay it.

So where is the bill? The being has one notepad and a lot of molecules. After the first molecule the page is full. To do the next one it has to wipe the page. And *wiping* takes two possible worlds (pad says "left"; pad says "right") to one (pad blank). That's the merge. That's the step with no way back. The demon pays every cycle, and what it pays for is making room to see again, and the amount it pays is at least what it gained, and the second law sleeps soundly. That's the rule from his 2003 notes I quoted in Section 3: erasing and merging cost, and copying onto a blank place to write doesn't.

Section 3 separated making a record from resetting it. Now you can see exactly which step pays, and why the other one was tempting, because looking *feels* like the active part. It's the thing the being does. Wiping feels like housekeeping. The housekeeping is where the money goes. If there's a sentence in this section I'd want you to carry out of it, that's the one, and the idea in it is Bennett's.

5.3 Two different things called "narrowing"

Here is what I had to get straight in my own head before any of this made sense, and I'll give it to you as a picture because that's how I eventually got it.

Draw the machine's possible states as a row of dots. Now draw a second row above it: the dots that *you*, standing outside, still think it might be in. Those are two different rows. The first is a fact about the machine. The second is a fact about you, about what you've seen and what you've ruled out. Both rows can shrink. They shrink for different reasons, and only one of them costs.

The machine's row shrinks when two of its states land on the same state. That's a merge. That's what the price charges. Under the halving rule from Section 4, a run can't shrink the machine's own row by more halvings than it paid for. That's the theorem just below, and it's the Section 4 squeeze bound run forward along a whole program. That section took it one step at a time. No new idea; the same chairs.

Exactly. Write $\lceil \log_2 n \rceil$ for the index-bit count of n: the smallest whole number k with $2^k \geq n$, the number of yes-or-no questions it takes to pick one of n things. It is 0 at $n = 1$, and I set it to 0 at $n = 0$ too, a convention for empty lists, since the logarithm of zero isn't a number. Wherever I write an ordinary logarithm without the rounding, its argument is positive. For a run t and a list D of starting states, let $\mathrm{img}_t(D)$ be the set of states the run can end in, starting from somewhere in D, and write $|\mathrm{img}_t(D)|$ for how many there are (in Section 4 the symbol for one move already meant the count; here I need the set and its size both). Let $\mathrm{cost}(t)$ be the run's total cost.

Theorem 5.1 (The machine's own narrowing is priced)**.** If the cost satisfies the halving price (Premise 4.4), then for every run t and every list D of starting states with no repeats,

$$\lceil \log_2 |D| \rceil \;\leq\; \mathrm{cost}(t) + \lceil \log_2 |\mathrm{img}_t(D)| \rceil.$$

Proof. By induction on the run. A single move of cost c shrinks a list with no repeats by at most a factor 2^c (that's the halving price), which costs at most c in rounded logarithms; the states reachable after the rest of the run from the shrunk set are exactly those reachable after the whole run from the original. Add the costs. □

Checked as `run_narrowing_priced_log` (KnowledgeNarrowing.v). The run itself is the witness; nothing else is handed in. That will matter in Section 8, and I'll say why at the end of this section.

Your row shrinks when you see something that some of your candidates couldn't have shown you. You cross those candidates off. Nothing happened to the machine. It didn't merge anything; it didn't know you were watching; it would have done the same thing with nobody in the room. The physicist Edwin Jaynes had a name for mixing these up, and I think it's one of the most useful names in all of science: the mind projection fallacy, treating what you know about a thing as if it were a property of the thing. Your list of candidates is in your head. The machine's row is in the machine. The halving price is about the machine's row, and crossing a candidate off yours doesn't move it.

And here I have to be fair to Jaynes, because he'd argue with where I just took his phrase. In a 1965 paper on Gibbs's and Boltzmann's entropies he wrote that

entropy is "an anthropomorphic concept" (a phrase he credited to the physicist Eugene Wigner), and by that he meant a property of the particular experiments you or I choose to do on a physical system. On his account the system alone doesn't carry it. So for Jaynes the second law is about what's known too, and the line I'd have liked to draw, that the second law has opinions about the machine and none about you, is exactly the line he'd refuse. Who's right is one of the oldest fights in statistical physics, Jaynes's reading against Boltzmann's, and I can't settle it. I don't have to. The spreads in Section 4's entropy theorems are spreads over the states the device might be in when the step runs, and every theorem there holds whether you read those chances as ignorance or as physics; "the floor comes from what you didn't know" is the same sentence in either reading, with "you" being whoever built the spread. What I need from the fallacy is only this: the two rows are different rows, and only one of them is the machine's.

5.4 The demon on a sheet of paper

Build it, because the whole claim is four dots and two arrows and you should see it merge or not merge with your own eyes.

The machine has a hidden bit (think of it as the molecule's side) and a display (the notepad). Four states: write them as pairs, hidden then display. 00, 01, 10, 11. Two moves.

MEASURE copies the hidden bit onto the display, by adding it in the way that a light switch adds: if the hidden bit is 1, flip the display; if it's 0, leave it. Draw the arrows. 00 goes to 00. 01 goes to 01. 10 goes to 11. 11 goes to 10. Look at the arrowheads: every state has exactly one arrow landing on it. Nothing merged. MEASURE forgets nothing.

WIPE sets the display to 0 whatever it held. 00 goes to 00; 01 goes to 00 as well. 10 goes to 10; 11 goes to 10 as well. Two arrows land on 00 and two on 10. That's a merge, twice over. Under any price that charges per halving, WIPE costs at least one. MEASURE is allowed to cost zero.

Now be the observer. You can see the display and not the hidden bit. What you're shown, the display here, is what I'll call a *window*: the part of the state an outside observer gets to see. Section 7 is about windows in general; here the window is just the display. The display starts blank, so your list of candidates is {00, 10}: two possibilities, one bit you don't know. The machine runs MEASURE. You look at the display. If it shows 1, the only candidate that could have produced it is 10. If it shows 0, only 00. Your list went from two to one. You learned the hidden bit. The machine's price for that move: zero. Its own row of possible states went from {00, 10} to {00, 11}, still two; nothing it could have been got merged with anything else.

You narrowed. It didn't. That's the measuring half of Bennett's argument on four dots, and WIPE's charge is the wiping half. So the demon just broke something: one bit gained, nothing charged. Here's the thing it broke, written out so you can see exactly where the crack runs: for every list of starting candidates, every run, and every actual start in the list, the index bits the observer gains (log2 of the candidates before minus log2 of the candidates after, both rounded up) are at most what the run was charged. Four dots, and that sentence is false.

Definition 5.1 (What an observer knows)**.** Fix a window w from states to what can be seen. Given a list Ω of candidate starting states, a run t, and the actual start $s_0 \in \Omega$, the observer's *knowledge* $K(\Omega, t, s_0)$ is the list of candidates $s \in \Omega$ whose run shows the same thing through the window at every step as the actual run does.

Defined as `knowledge` (KnowledgeNarrowing.v).

If that definition looks familiar, it should. It's the standard way logicians of knowledge set it up: what you know is whatever holds of every case you can't tell apart from the actual one, given everything you've seen (Fagin, Halpern, Moses and Vardi's *Reasoning About Knowledge*, 1995, builds a whole subject on it, runs and observations included). And it's the logically omniscient kind from the opening chapter, the kind where knowing some facts means knowing everything that follows from them, for free. I'm using it on purpose, flaw and all. This section is about learning, and learning in that sense is exactly the thing that's allowed to be free. Settling is the separate act with a price, and nothing in this definition settles anything.

Theorem 5.2 (An observer can learn for free)**.** There is a four-state machine (the demon), a cost satisfying the halving price, a run of cost 0, and candidates with $|\Omega| = 2$, for which $|K(\Omega, t, s_0)| = 1$. So the claim "for every Ω with no repeats, every t, and every s_0 in Ω: $\lceil \log_2 |\Omega| \rceil - \lceil \log_2 |K(\Omega, t, s_0)| \rceil \leq \text{cost}(t)$" is false for that machine and that cost. In the same machine, every cost satisfying the halving price charges WIPE at least one.

Proof. The machine and run are the four dots above: MEASURE is a bijection, so cost 0 satisfies the halving price; the observer's two candidates show different displays after MEASURE; WIPE sends two pairs of states to one each, so the halving price needs its cost ≥ 1. □

Checked as `observer_narrowing_can_be_free`, `demon_observer_learns`, `demon_machine_spread_kept`, `measure_forgets_nothing`, `wipe_merges`, and `wipe_costs_at_least_one` (KnowledgeNarrowing.v).

The demon is the picture. The smallest case is smaller. The claim as written already fails on a machine with two states and every move free, because the observer's very first look tells the two apart, and it never fails on a machine with one state (`nec_f_two_state_observer_free`, `nec_f_one_state_observer_priced`, NecFNarrowing.v). That's why the interesting version is learning *after* the first look, which is where the three states of Theorem 5.3 come in. And WIPE's price needs less than the halving rule: merge pricing alone charges it at least one (`nec_f_wipe_under_merge_pricing`).

I have to be exact about one thing the file is exact about. The halving rule is a floor. It says merges cost at least this much; it does not say anything costs zero. So what the theorem exhibits is *one* price schedule that meets the halving rule and charges MEASURE nothing. A different schedule could meet the rule and still charge MEASURE; nothing in counting forbids overcharging. The claim is "learning is not *forced* to cost." It isn't "learning is free on every machine," and on a given machine a schedule may still charge it. That's weaker than the section title. The title is the short form you'd say before checking the fine print, so the title stays and this paragraph stays next to it.

The physicists had this freedom on the books long before me. Paul Fahn (1996), doing the entropy accounting of Szilard's one-molecule engine step by step, argued for a symmetry between the measuring step and the erasing step: on his account, which of the two you hand the bill to is partly a choice of bookkeeping, and the total is what's fixed. Sagawa and Ueda (2009) derived a bound on that total, with the split depending on the memory. So, on their accounts, "a schedule could charge MEASURE instead" is a known freedom in how the books are kept. I didn't find a loophole. And the other reading is very much alive. Ruth Kastner (2025) argues that Brillouin was essentially right after all: once the physics of a real measurement is put back in, it's measurement that pays and foils the demon, erasure drops out of the bill, and Bennett's free copy fails as physics. The theorem above is about price lists, so it stands either way. What hangs on her is whether a real demon can look for nothing, which is the same door the next paragraph's doubter pushes on.

And the doubter from the last section cuts this side too, so I owe it to you, since I leaned on him there. In 2013 John Norton argued for a no-go result: at the scale of molecules, thermal jiggling keeps any process that's supposed to be thermodynamically reversible from ever actually finishing, and finishing it takes more entropy than Landauer's amount. If he's right, Bennett's free measurement isn't free in practice either, because it's one of those processes. The theorem above survives that, because it's about price lists: a schedule that meets the halving rule isn't forced to charge the copy, whatever a real device spends. What wouldn't survive is the physics I've been hanging on it, that a real demon can look for nothing. If Norton is right, the line

between learning and committing keeps its counting and loses its physics. I can't referee that one either.

5.5 Who holds the pen changes the bill

Now the part that I think is the most beautiful fact in here. You find it by playing with the four dots. Thinking about them won't get you there. Run MEASURE *twice*. The first copies the hidden bit onto the display. The second adds the hidden bit in again, and adding a bit twice with a light-switch add gets you back where you were: the display is blank again. Check the arrows: 10 to 11 to 10. The display went full and then blank, and nothing merged, and the price was zero both times. So a display that was blank before the first MEASURE *can* be put back to blank for free, by a second MEASURE. Just not by WIPE.

What's the difference? MEASURE *consults the hidden bit* when it clears the display. It knows what's written there, because it has access to the thing it was copied from, and so it can undo the copy. Nothing gets erased. WIPE doesn't look. It sets the display to 0 regardless, and "regardless" is exactly the many-to-one. Bennett has the general form of this in the 2003 notes: he lists the "erasure of one of two copies of data known to be identical" among the logically reversible operations, because you're removing a duplicate, and no information is destroyed. And it goes further than Bennett in work that came after him: the quantum physicist Lídia del Rio and colleagues argued in 2011 that the cost of erasing a system depends on how much the eraser already knows about it, down to zero and in the quantum case past zero. The bill for clearing a record is sent to whoever clears it, and it's smaller the more they know. Try it with a card under a cup and a sticky note. Write the colour on the note, then erase the note *without lifting the cup*: two possible notes become one blank, a merge. Now do it again, but lift the cup, check the note matches, and then erase: nothing unknown was destroyed. Same blank note. Different bill.

5.6 The smallest machine that teaches for free, and the one that can't

Two more small facts, and then the point. The demon measures what you learn by comparing the start to the end, but an observer's very first look already rules things out before any step runs, so the fairer question is: after that first look, can a *step* teach you more, for free? Yes, and the demon still does it. And the smallest machine that can is three states in a ring: U goes to V, V goes to W, W goes to U, and the window shows "true" only at W. Start with candidates {U, V}; both show false. Take one step. From U you now see false; from V you now see true. One step told them apart, and a ring forgets nothing, so a zero price meets the halving rule, and the file takes it.

Two states can't do it, ever, with any arrows, and this is the one that needs no price premise at all. If a machine has only two states and you can't tell them apart at the first look, then the window shows the same thing on both, and those two are *all the states there are*, so the window shows the same thing everywhere, forever, and you never learn anything. Try to draw a counterexample. You'll draw the same two dots a few times and give up, and the giving up is the proof. Three is the smallest number of states on which two states that look alike at the first look can come apart at a later look, and that's what teaching is.

Theorem 5.3 (Three states are the minimum for free teaching)**.** Call a machine a *free teacher* when it has a cost satisfying the halving price, a list of candidates with no repeats containing the actual start, and a run of cost 0 after which the observer knows strictly more than after the first look, $|K(\Omega, t, s_0)| < |K(\Omega, [\,], s_0)|$. There is a free teacher with exactly three states, and there is none with two or fewer.

Proof. Three: the ring $U \to V \to W \to U$ with the window true only at W, candidates $\{U, V\}$, one step, cost 0 (a ring is a bijection, so cost 0 satisfies the halving price). Two or fewer: two candidates that agree at the first look are, on a two-state machine, all the states there are, so the window takes one value everywhere and no later look can separate them. This half never uses the price at all. □

Checked as `free_incremental_narrowing_with_three` and `no_free_incremental_narrowing_below_three` (KnowledgeNarrowingMinimal.v), and the demon-against-the-first-look form as `demon_refutes_incremental`.

5.7 What the toll prices, said at last

Now I can say, more precisely than I could two sections ago, what the mark in Section 3 is a mark for. Not for learning. What you know can be narrowed at no cost, with nothing merged, on some machines under some honest price lists, and the demon is one. Not for looking, not for copying, not for finding out. The mark is for the one thing in Section 4 that couldn't be done without merging, on a machine with finitely many states: raising a flag that stays up. Learning is the demon writing on a blank pad, which merges nothing. What the demon pays for is the wipe, a step with a shared chair. The mark in Section 3 is paid for the same reason, because raising a flag that stays up also has a shared chair, but it's a different step: the demon machine has no flag, and its file prices merges; commitments never appear in it. Same chair. Different room.

This is also where I settle up with Rovelli, my closest neighbour from the opening chapter, because I put him and Bennett on opposite sides of one door, and I owe you

how much of that is mine. All of the reconciliation is mine. Rovelli's paper doesn't make it, and in four places his own framing points somewhere else. He reserves "choice" for an outcome that is stable and *not predictable*, and he rules out a thermostat because it runs on a predictable mechanism; the commit in this book is a step a rule fixes in advance, so by his own definition his result doesn't cover it. His choice is a branching, one possibility splitting into two, and he calls choosing in a sense the inverse of erasing, because branching is the inverse of merging; the toll comes from a merge, the opposite shape. His stability is for a while: he says stabilization means something only on times that aren't too long next to the time it takes things to thermalize, and past that nothing stays chosen; my permanence is forever (and the last section owned up that forever is an idealization). And he doesn't stand beside Bennett at all: he holds that any measurement is dissipative, credits the point to the philosopher of science Hans Reichenbach, and doesn't cite Bennett. So what Rovelli and I share is a sentence, that a choice that sticks costs something like Landauer's bill. There's no shared mechanism. Push on it and his coin hitting the ground, a fact gained from something nobody could predict, sits closer to what this section calls learning than to committing. I still think it's the same instinct. That's my reading, and he doesn't make it; you now have everything you need to disagree with it.

And on one point Rovelli isn't the nearest physics. The nearest I've found is in biology. Thomas Ouldridge and Pieter Rein ten Wolde (2017) derived what they call the fundamental costs of making a *persistent* copy of a polymer, one whose sequence stays correlated with its template after the two have physically come apart, and of destroying one. On their account a copy like that is a state out of equilibrium, and making it has a thermodynamic bill they measure efficiency against. A companion paper the same year, with Christopher Govern, does the same kind of accounting for the molecules a cell uses to read out the state of its receptors, which is a measurement in all but name. The way I read them, the copy that has to stay is the one with the bill, and that's the closest thing in physics I've found to the line this section draws between finding out and committing. The differences, so it doesn't get borrowed for more than it is: theirs is heat, in chemistry, for a correlation between two molecules, and their title has the destruction of copies in it too, so their copies don't last forever; mine is a count in a model of computation, for a reading that never turns off, with no temperature anywhere. Same instinct again, from a different direction.

The chemistry has the sharp form of it, too, and it came two years later. Jenny Poulton, Pieter Rein ten Wolde and Thomas Ouldridge (2019) studied minimal models of polymer copying, and in those models, when the copy grows at equilibrium, with the weakest chemical driving, a persistent copy's accuracy and the efficiency of making it both vanish; only driving the growth out of equilibrium, paying, buys a copy that still carries information about its template after the two let go. That's

finding out against committing, in chemistry, with the bill attached, and I'd rather say plainly that biology had the distinction before I did. What stays different is the kind of thing the record is: theirs is a correlation in a chemical ensemble, mine is a reading enforced by a step in a model of computation.

Two more neighbours have "learning" and "commit" in their titles, so the slogan shouldn't be read against them by accident. Sebastian Goldt and Udo Seifert (2017) did the stochastic thermodynamics of a neural network learning a rule, and argued from it that learning has a thermodynamic cost: on their account, the entropy the network produces bounds how much it can learn. That doesn't contradict this section, and here's why. Their learner isn't copying onto a blank page. Its weights are a driven thermal system, jostled by noise and dragged toward what the data push them to, and what they price is that physical process. This section's free learning is narrower: one way of finding out, a copy onto a blank, is *allowed* to cost nothing. Nothing here says every way of learning is cheap, and theirs isn't. From statistics, Aurélien Garivier, Tor Lattimore and Emilie Kaufmann (2016) gave a proof that in a two-armed bandit with Gaussian rewards, a strategy that explores first and then commits for good ends up, as the horizon grows, with at least twice the regret of the best strategy that keeps deciding as it goes. That's the closest published sentence I've found to "committing isn't free" with a number on it. Their price is reward lost, which is a different unit from a mark on a ledger, so it's a cousin. A twin would have to count marks.

So here's exactly what "No Free Insight," the name I gave the toll before I understood it, covers, because I don't want the name doing work it hasn't earned. Insight in the sense of *finding out* can be free. The insight the toll prices is the kind you've *committed to*, written into a record the machine can't silently revise. Whether that narrower thing deserves the name is a fair thing to argue about; I kept the name because it's the question I started with, and because the narrowing is the finding. I don't count it as a retreat.

It also tells you how to read a bound that's coming. Section 8 counts what it takes to narrow a list of candidates down to a shorter one: at least as many yes-or-no questions as it takes to point at the survivors among their rivals, a counting fact that holds for anything with a list in it. Turning that count into a *bill*, saying a run actually paid that many marks, is a different claim, and when the book makes it, for any certification system in Section 8, the payment goes in as a hypothesis: a decision tree (a branching chart of yes-or-no questions, which Section 8 builds) no deeper than the number of paid steps the run took is handed in, along with the lists themselves and the fact that the tree's leaves cover the narrowing. This section is the reason it has to go in that way. The narrowing alone doesn't force anyone to pay; the demon just

showed you a narrowing that cost nothing. So wherever narrowing gets a price in this book, the payment is a premise, and I'll keep calling it one.

And one more boundary. This whole section is about a machine that's finite and deterministic and a watcher who crosses candidates off a list. There's no quantum anything in it. This narrowing has nothing to do with "observation collapses the state": the narrowing here is bookkeeping in the watcher's list, the machine never notices, and the result would be the same if the watcher were a filing cabinet. In 2010 a group in Tokyo (Toyabe, Sagawa, Ueda, Muneyuki and Sano) did run a real demon, a tiny particle walked up a staircase of potential by feedback on what a camera saw, and the energy it gained was paid for by the information the camera took, with no violation of anything. The demon is real, it works, and it pays. Where it pays is the part that took a century.

What I still owe: everything so far says "at least one mark" and I've been keeping the marks on the side of the page, a running total you tally as you go. What happens when the machine keeps that total itself, inside its own state, where a step could in principle reach in and change it? That's the counter, and it's the next thing I have to build.

III

The logic

6

The counter

Everything so far has kept the marks on the side of the page. You played the toy, you made a tally as you went, and the tally lived in your hand. The machine never saw it. That's fine for a toy. For anything that's going to run on its own, with nobody tallying, the total has to live somewhere the machine can reach, which means the machine can, in principle, reach it. So this section is about what a counter has to be before you're allowed to call it a receipt. I mean the counter of *any* machine whose state carries a running total that every step bumps by whatever a price list says that step costs. Section 3's five parts don't need one (there the total was a sum you did on the side), so this is one piece added to the five, and the facts below are about anything that has it. I'll give you the answer first, because it's short, and then spend the section on why each word is there: a receipt is a total that lives in the state, that only climbs, that equals the sum of what was charged, and that nothing else could have been.

6.1 A stick, split down the middle

The oldest receipt I could find that you can still hold is a piece of wood. From about 1100, under Henry the First, until 1826, the English treasury recorded what it was owed on tally sticks: a strip of hazel, notched across with cuts of set widths (the treasury's own handbook from around 1180, the *Dialogue of the Exchequer*, gives them: a thumb's width for a hundred pounds, a swollen barleycorn for a pound, a bare cut for a penny), then split lengthwise so that each half carried the same notches. The payer kept one half, called the stock, which is one story for where the word stockholder comes from. The treasury kept the other. When the time came to settle, you laid the two halves together, and only the two original halves fit, because wood splits along a grain nobody can copy. Add a notch to your half and the other half shows a gap where the partner should be. Try to remove a notch and you find that wood can't be uncut. When they finally stopped using them, the clerks burned centuries of old tallies in the furnaces under the House of Lords, overheated the flue, and burned down the Palace of Westminster, which I mention because it's the best fact I picked up for this book and because it tells you how many of them there were.

Do it: notch a strip of card once for every coin you hand a friend, split it, keep half each. Try to cheat, either direction. The three things you'll find are the three things I want. The total is the notches, added up, and nothing else (I'll call that conservation). You can add a notch but you can't take one away (it only climbs). And two halves that were notched together must agree; a half that disagrees is a forgery, and the wood says which one (there is only one honest count). The treasury didn't have the words. They had the stick.

Economists read the same stick another way, and they're not wrong either. Alfred Mitchell-Innes (1913) made the exchequer tally his star witness that money is credit: the crown paid the people it owed by "raising a tally" on some branch of its revenue, a wooden acknowledgment of debt. A receipt to me, money to him, and it's the same piece of wood. Narayana Kocherlakota (1998) later proved a version of his point: in a wide class of economies where nobody can be held to a promise, anything money lets people do, a memory of everyone's past dealings lets them do too. His title says it in three words: money is memory.

6.2 Why it has to climb

Start with the second property, because it's the one that does the most work and the one people fight hardest. Why can't the counter ever go down?

Because of what the toll is for. The floor it buys, back in Section 3, says a run from NO to YES costs at least one. That sentence is a *floor*: it says the run paid at least this much. If the counter could come back down, a run could pay the toll, get its flag, and then take the payment back, and the floor would be a floor on nothing; "this run cost at least one" would be true of a run that ended having cost nothing. So a counter that can fall is an accountant you can lean on to revise the books, and a floor stated in terms of it isn't a floor. The counter only climbing is not a flourish. It's what makes "at least" mean something. This ledger records cumulative charges, each a whole number of marks and none negative, so it never decreases. A refundable balance, one that pays money back, would be a different accounting object with a different invariant, and the floor here says nothing about it. (In Section 3's abstract theorem the total is a sum of whole numbers and climbing comes for free; this paragraph is about why a total *stored in the state*, where a step could in principle write it, has to be designed so that no step ever lowers it.)

The same rule turns up in two places that have nothing to do with each other, pointing in opposite directions, and that's a big part of why I trust it. In 1978 Leslie Lamport asked how a set of computers that pass each other messages, with no shared clock, could ever agree on what happened before what. His answer is a counter per machine with two rules: add one before you do anything, and when a message

arrives carrying the sender's count, jump your own counter above it (never below your own). Those counters, he showed, respect cause and effect: if this caused that, this has the smaller number. And they do it "with no actual timing mechanism," his words; they're clocks only in name. Try it with three people passing notes and you'll see it work in two minutes, and then let one of them reset their counter to zero after reading a note, and watch a reply come out numbered *before* the note it replies to. A counter that goes down can't be leaned on for order.

Where did Lamport get it? In the notes he keeps on his own papers, he says the timestamps came from an earlier note by Paul Johnson and Bob Thomas on keeping copies of a database in step, whose algorithm wasn't quite right: it let things happen in an order that broke cause and effect. What let him see the fix, he says, was special relativity, which teaches that the one order of events everybody agrees on is the order of what could have caused what. And he passes on a line from Jim Gray, the same Gray as the commit at the start of the book: Gray had heard two opinions of the paper, that it was trivial and that it was brilliant. Lamport's answer is that he can't argue with the first and isn't inclined to argue with the second. His account stops at relativity, so the chase does too. The picture it leaves is a good one: a number that's allowed to climb and never to fall is enough to write "this came after that" with no clock anywhere.

Now the other direction. In 1949 Alan Turing, and in 1967 Robert Floyd, each asked how you prove a program *stops*. The answer, both times: find a quantity that goes *down* on every step and can't go down forever (a whole number that can't go below zero, say). If every step lowers it and it can't fall forever, the steps can't go on forever. Up-only gives you order. Down-only gives you stopping. They're different rules (one needs the jump-above-the-message step, the other needs a floor it can't fall below). They have the same shape, seen from the two ends: a one-way quantity is something you can build on, and a two-way quantity is a number that happens to be written down somewhere.

And the real-world case where a counter was allowed to go down, and got gamed exactly there: Ethereum, one of the largest blockchains, charges "gas" for every step of a program, and for years it *refunded* gas when a program freed up storage. Someone noticed you could fill storage when gas was cheap, hold it, and cash the refunds when gas was dear, and a token was built to do exactly that, and the real work in a block could reach nearly twice the block's stated limit. In 2021 a change called EIP-3529 cut the refunds sharply (none for self-destruct, a smaller one for clearing storage, and a cap of one fifth of the gas used). The written reasons are the gas token and those overfull blocks. I read both as the same lesson: a counter that goes down was being leaned on at the place where it went down. Ethereum's gas comes back in Section 14 as one of five small models of systems people actually run,

and it's the only one of the five that prices a write at all, and it's instructive that the part they had to fix was the part that could fall.

6.3 Conservation: the total is the charges and nothing else

The first property is the plainest one. The counter after a step is the counter before plus what that step was charged. Add up a whole run and the middle values cancel out, the way a sum of differences collapses to last minus first, and the total is the sum of the per-step charges. Why state it?

Because it's the property that fails quietly. Say a machine charges one for raising a flag, and also, somewhere in its error handling, forgets to add the charge when the step fails halfway through. Or charges two in one code path and one in another for the same move. Or has one step that writes the counter directly. Then the counter is *mostly* the sum of the charges, and "mostly" is the whole gap between a receipt and a note that says "trust me, about this much." Conservation is the sentence that there is no "mostly": every step, every outcome, every error, adds exactly its charge, and a step that fails is charged the same as one that succeeds, because the charge is on the attempt. For a machine built the way I'm describing, that sentence is the rule the counter is updated by, so it can't fail by definition. What can fail is the claim that a real machine *is* built that way, and checking that means going through every move it has, the failure branches included, one at a time, which is dull and is the whole job. The run-long form, counter at the end equals counter at the start plus the sum of the charges, then follows from the one-step rule by the cancelling sum; it's below. It's the kind of theorem that looks boring and is the one you'd most want if you were going to rely on the number. One thing it doesn't say: that every mark bought something. The ledger is exact about what was spent. What the spending was worth is another question, and whether a mark bought a real narrowing is Section 8's question, and it needs a witness on top of the ledger entry.

6.4 Only one honest count

Here's the property that surprised me by being provable at all. Suppose you fixed the schedule: this move costs one, that move costs zero, and so on, for every move. And suppose someone else keeps their own books: their own function from states to totals, which starts at zero and, on every step, goes up by exactly what the schedule says. Could their books disagree with the machine's counter, somewhere, on some state you can actually get to?

No. And the proof is the tally stick. Both totals start at zero. On every step both go up by the same amount. So on every state you can reach by steps from the start, they're equal. That's it; it's an induction, the one-step-at-a-time argument from

Section 3's floor. The mathematician's word for this is *initiality*: the counter is the one bookkeeping that all the others, if they're honest about the schedule, turn out to be. Given the schedule, the count is forced. Two things about that sentence I had to be careful with, and they're worth saying before the symbols. It doesn't assume the other books climb: climbing follows from the schedule, since the schedule never charges a negative amount, so "only one honest count" and "it only climbs" aren't two separate demands on the other bookkeeper. And it holds on *reachable* states only. A state you can't get to from the start is one where it says nothing about whether the other books agree, which is the right thing for a receipt to say.

Here are the properties exactly. First the piece being added to Section 3's five, a counter and a price list, and then the two facts about it. The counter gets the Greek letter μ (mu), which is what the checked work calls it.

Definition 6.1 (A counter kept by a price list)**.** A *counter machine* has a set of states, a set of moves, a step function apply that takes a state and a move, a start state s_{init}, a schedule c giving each move a cost in $\mathbb{N}$, and a counter μ from states to $\mathbb{N}$, such that $\mu(s_{\text{init}}) = 0$ and $\mu(\text{apply}(s, i)) = \mu(s) + c(i)$ for every state s and move i, with no exceptions. A *run* of a list t of moves from s applies them in order, and $c(t)$ is the sum of their costs. A state is *reachable* when it is the start or is $\text{apply}(s, i)$ for a reachable s and some i.

Theorem 6.1 (Conservation)**.** In a counter machine, for every state s and every list t, μ at the end of the run of t from s equals $\mu(s) + c(t)$. In particular μ never decreases along a run.

Proof. Induction on t. For the empty list both sides are $\mu(s)$. For a list i followed by t', the one-step rule gives $\mu(\text{apply}(s, i)) = \mu(s) + c(i)$, and the induction hypothesis from $\text{apply}(s, i)$ adds $c(t')$. Since every $c(i) \geq 0$, the counter never falls. □

Theorem 6.2 (Only one honest count)**.** In a counter machine, let B (for "books") be any function from states to $\mathbb{N}$ with $B(s_{\text{init}}) = 0$ and $B(\text{apply}(s, i)) = B(s) + c(i)$ for all s and i. Then $B(s) = \mu(s)$ for every reachable state s.

Proof. By induction on why s is reachable. At the start both are 0. If $s = \text{apply}(s', i)$ with $B(s') = \mu(s')$, then $B(s) = B(s') + c(i) = \mu(s') + c(i) = \mu(s)$, the last step by the counter's rule. □

Those two proofs are the whole of it, and they're about any counter machine. Coq checks them over arbitrary states, as `nec_f_conservation`, with `nec_f_counter_monotone` for the climb, and `nec_f_initiality` (NecFCounter.v), and here they're on the page, where you can check every line. The machine I build later is one of these, and on it the run form is a checked theorem: the ledger after any list of instructions is the start plus the sum of their costs (Theorem 16.5). The uniqueness half holds even with the books' rule asked only of reachable states (`nec_f_initiality_reachable_rule`).

I want to flag what initiality does *not* do, because the fact is smaller than the word. It doesn't pick the prices. The schedule is an input: somebody decided that raising the flag costs one and adding two numbers costs zero, and nothing here says that decision was right (Section 4 is where I argued the floor isn't arbitrary; the rest of a schedule has to be defended by whoever writes it). It doesn't turn marks into anything physical. And it isn't the category theorist's "initial object," though the word is borrowed for a related reason.

There *is* a universal property nearby, and I'll say exactly how far it reaches so the borrowed word doesn't reach further. Take any other machine that understands the same moves (call it the *target*), and pick a state of it to start from. Walking a list of moves through it, one at a time, is the one and only way to send lists to its states that starts where you said and keeps in step as the list grows by one. Programmers call that walk down a list, one item at a time, a *fold*, and I'll borrow the word. That's an induction on the list from its end, the same shape as the proof just above, and it never looks at the other machine's reading or its prices, which tells you how much the toll is doing here: nothing. It's a fact about *lists of moves*; states haven't come into it yet.

Coming down from lists to the states of the machine running them is where it gets interesting, and where the borrowed word stops. Two different lists can leave a machine in the same state. If they leave the target in two different states, there's no single place to send that state. That's the whole obstruction, and it's exact: the map from states exists on the reachable ones precisely when equal states always come from lists that agree on the other side. Call that the *descent* condition, because it's what lets the fold come down from lists to states. A toll doesn't force it. Take the one-bit machine whose set costs one and whose reset is free. It obeys the toll, yet a single set and a reset followed by a set leave its bit in the same state. A target that keeps its whole history as its state tells those two runs apart, so nothing comes down. So the fold is unique on lists, and that's what the word "initial" earns. It doesn't make anybody's states initial among everything that obeys the toll, and I won't say it does. The fold and the descent condition are checked over any machine: the fold is the one such map (`nec_f_fold_unique`), descent holds exactly when each reachable

state goes to one target state (`nec_f_descent_iff_functional`), and a map the fold factors through gives descent (`nec_f_factor_gives_descent`), all in NecFCounter.v; the history-keeping machine is the example that shows where descent fails (`nec_f_descent_fails_for_history`).

And the borrowed word has a home. I didn't build it, so I'll point at it. That lists of moves are the starting object, and that a machine's behaviour factors through its reachable states exactly when a condition like descent holds, is the theory of minimal realization. Joseph Goguen put it as a universal property in 1973, in a paper whose title says it outright, "Realization is universal," and Thomas Colcombet and Daniela Petrişan redid automata minimization in the same functorial terms in 2020. Their machines can have any output at all, which is the honest point: the fold and the descent condition never use the record, so the toll is doing nothing here, just as I said.

The nearest classical parallel to "only one honest count" is in thermodynamics. The mathematical physicists Elliott Lieb and Jakob Yngvason showed in 1999 that once you fix which states of a system can be reached from which by adiabatic processes, and that relation meets a short list of axioms, there is an entropy, a function that goes up exactly along the relation, and it is unique once its zero and its unit are chosen. Their uniqueness, like this one, comes after the fixing: the order is the input there, the price list is the input here, and in both the count is forced by it.

6.5 The toll is an old trick with a new name

Now a thing I first took as a let-down. It would be nice to have a new kind of cost argument. This isn't one. The toll is a case of a technique that algorithm people have used since 1985 and that a mathematician studying the stability of motion had, in a different costume, in 1892. I'm telling you it's old so you don't think I invented it.

The trick is called the potential method, and here's the whole of it. Give every state a number, call it the potential, that measures how much "owing" is stored up in that state. Then look at each step and ask: does the cost of the step, plus the change in potential it causes, ever come out negative? If it never does, you've got a bound: any run costs at least the amount the potential *dropped* from start to finish, because the drop had to be paid for by the steps along the way. The computer scientist Robert Tarjan set it out in 1985, crediting the potential view to Daniel Sleator. Tarjan's use goes the other way (potential as prepaid credit, to bound a run from above): he used it to analyse data structures that are occasionally expensive and usually cheap; and the decreasing-function idea underneath it is the one the Russian mathematician Aleksandr Lyapunov used in 1892 to prove that a system settles down. Do the coin version, which runs Tarjan's way, as a ceiling: a stack of coins, PUSH adds one and

costs one, POP-ALL removes them all and costs as many as it removed, potential equals the number of coins. PUSH costs one plus one potential; POP-ALL of k coins costs k and drops the potential by k, net zero. So any sequence from an empty stack costs at most twice the number of pushes, however many expensive pops are in it. That's the method. Three lines.

Push on that 1892 for a second, because a date like that hides a lot. Lyapunov wrote the idea up as his doctoral thesis, in Russian. A French translation came out in 1907, made by a French naval engineer, Édouard Davaux, and corrected by Lyapunov himself. The English one came out in 1992, a hundred years after the original, and it was translated from the French: a translation of a translation. So for most of the twentieth century a reader with only English met the idea secondhand or not at all. Did the algorithm people of 1985 get it from him? I can't tell you, and nothing here needs them to have. Whether anybody walked from one to the other on purpose is a question I'd like answered. What I do have is a picture: the same idea, a number that only goes one way along a run, turned up in a thesis about whether a moving system settles down and in a paper about data structures, close to a century apart.

Now point it at the flag.Potential: one if the flag is down, zero if it's up. A step from down to up drops the potential by one, so for the step's cost plus potential change to be non-negative, the step has to cost at least one. That's the toll, exactly, and the two are the same statement: the toll holds for a machine if and only if every step has non-negative amortized cost under the flag potential ("amortized cost" is just that sum, the step's cost plus the change in potential). Then the universal floor from Section 3, "any run from NO to YES paid at least one," is just the potential method's standard conclusion, the drop in potential from one to zero, and every system with the five parts that obeys the toll is an instance (the checked record carries the toll as a field, so that's every certification system). Two more facts fall out of the same picture. Under the toll, a run raises the flag at most as many times as its total cost; each raise uses up a unit and a unit isn't used twice. And if you demand the upper-bound version too, that the potential before a step covers its cost plus the potential after, then the two together hold exactly when raising the flag costs exactly one, everything else costs zero, and the flag never comes down; that's the sharpest version of the schedule, and notice that Section 3's toy doesn't meet it: the toy's VOUCH move, played from YES, makes a mark although it raises nothing, which the floor allows and the exact version forbids. (These two theorems let the price depend on the state as well as the move's name, which is wider than Section 3's per-name cost; a per-name cost is a special case.) Two older tools sit beside these, named for completeness: running a list of moves in a "writer" that accumulates a number returns the final state and the run's cost (`run_writer_is_run_and_cost`), and a per-move schedule is a "grade" fixed before the run

(`run_graded_is_run`); neither embeds those frameworks' full type systems, and the files say so. The writer is the move the programming-language researchers Norman Danner and Daniel Licata make in their cost semantics, a translation that carries a cost alongside a program's result (Danner, Licata and Ramyaa, ICFP 2015; Danner and Licata, *Journal of Functional Programming*, 2022). The grade is Shin-ya Katsumata's: a computation carries its resource as a grade, and the grades compose as the computations do ("Parametric effect monads and semantics of effect systems," POPL 2014). Dominic Orchard, Vilem Liepelt and Harley Eades put grades into the types of a working language ("Quantitative program reasoning with graded modal types," ICFP 2019). And the potential method went into types before any of those: the computer scientists Martin Hofmann and Steffen Jost (POPL 2003) gave a type system that attaches potential to data and predicts, before a first-order functional program runs, how much heap it can use, which is where type-based amortized analysis begins; the multivariate work of Jan Hoffmann, Klaus Aehlig and Hofmann (2011) carried it further. Read against them, the counter is a grade carried in the state by the step rule. No type carries it. And what does that difference even mean? A cost that lives in a type gets checked before the program runs, by whoever checks the types; a cost that lives in the state gets written during the run, by the step itself. Same number, different moment of truth.

So is mine an embedding of their frameworks, or something weaker? Half of it is exactly theirs, and the other half is left out on purpose. The counter *is* their writer: a step that moves the rest of the state without looking at the counter, hands back its cost, and has the cost added on is a computation in the writer over whole numbers with addition, and running a list of moves in that writer gives back the final state and the total cost, nothing more and nothing less (`run_writer_is_run_and_cost`). Every system in this book whose step never reads its own counter has that shape. What doesn't come along is the part of their frameworks that works before anything runs: the type systems that carry grades, Danner and Licata's bounding relation and the recurrences they extract for a program's cost. That isn't a hole waiting for a patch. It's the moment-of-truth difference I just named. Putting the count in the step is choosing the other moment, so the grade here is written by the step at run time and never checked ahead of it.

How close do the two moments come? Close enough to say exactly, in a limited form, and it's checked. Take a small structured language over a machine with this kind of counter: sequencing, branches whose two arms carry the same grade, and paid acts each graded with their charge, and grade every program the way Katsumata's discipline does, adding grades along the program. On loop-free programs the two agree on the nose: on every run that doesn't trap, the counter rises by exactly the grade. The argument is conservation set beside the grade rule; the counter's rise

is the sum of the charges along the run, and the grade rule adds the same charges in the same order. Loops break the agreement, and in one direction only. A loop that runs a paid check once for every unit in some register, where the check always passes and is charged one, costs that register's value on entry, and no fixed whole number is an exact grade for it; a grade that's a function of the inputs, the kind cost-aware type theories like calf (Niu, Sterling, Grodin and Harper, 2022) allow, gets exactness back. So the counter carries their semantics exactly, agrees with fixed grades wherever a fixed grade can be exact, and where a run's cost depends on its data it holds the exact number that a fixed grade can only bound. Checked as `nec_f_grade_exact`, with the small machine as an instance in `nec_f_grade_exact_small`, and for the loop `nec_f_loop_cost`, `nec_f_loop_no_fixed_grade` and `nec_f_loop_input_grade` (NecFGrade.v). What has no counterpart in the step is the checking ahead of time: a type checker turning a program away before it runs.

There's one more neighbour, closer than any of those, and I'll name it before somebody names it at me. A number the step keeps up to date and nothing else ever reads is what program-proof people call an *auxiliary variable*, or *ghost* state. Susan Owicki and David Gries added auxiliary variables to parallel programs in 1976, because some true facts about such a program can't be proved without one, under the rule that an auxiliary variable never steers the program. Jean-Christophe Filliâtre, Léon Gondelman and Andrei Paskevich, in 2014, worked out when ghost code can be erased without changing what the program computes. By that test this section's counter is ghost whenever no step consults it, and Section 10's base-driven records are built exactly that way: the base moves the same whatever the ledger and the record say. So what does putting it in the step buy, if the number is ghost anyway? Two things, as far as I can see. The number is there while the machine runs, where a window can show it (Section 7), and not only inside a proof somebody wrote afterwards. And the rule that keeps it is one rule for every run of every program. An annotation covers only the one program somebody happened to prove. Whether a record stays ghost depends on what else the step does with it; the machine in Part IV has a part that isn't ghost, its table of checked facts, and that's where I'll say why. One last relative, from the graph side: Section 3's floor is a cheapest-path bound in a system with weights on its moves. Every path from the no-states to the yes-states crosses the line between them, and every edge across that line weighs at least one. Weighted automata go back to Marcel-Paul Schützenberger in 1961, and priced timed automata, which add clocks, were set out in 2001. The floor is about the smallest case of what they compute.

Exactly, for any certification system with states S, moves I, step, per-name cost, and reading:

Definition 6.2 (Potential, amortized cost)**.** A *potential* is a function $\Phi : S \to \mathbb{N}$. A step has *non-negative amortized cost* under Φ when $\mathrm{cost}(i) + \Phi(\mathrm{step}(s, i)) \geq \Phi(s)$. The *flag potential* is $\Phi_{\mathrm{cert}}(s) = 1$ if $\mathrm{cert}(s) = \mathrm{no}$ and 0 if it reads yes.

Defined as `amortized_nonneg` and `cert_potential` (CostSemanticsComparison.v).

Theorem 6.3 (The toll is the potential method)**.** (1) The toll holds exactly when every step has non-negative amortized cost under Φ_{cert}. (2) For any Φ with non-negative amortized cost and any run t from s, $\mathrm{cost}(t) + \Phi(\mathrm{run}(t, s)) \geq \Phi(s)$. (3) Hence, under the toll, a run from a no-state to a yes-state costs at least $1 - 0 = 1$.

Proof. (1) Check the four combinations of the reading before and after: only no-to-yes has Φ_{cert} dropping, by one, so the inequality there is exactly $\mathrm{cost}(i) \geq 1$; in the other three it holds for any cost. (2) Induction on the run, adding the per-step inequalities so that the middle potentials cancel. (3) Apply (2) with Φ_{cert}, which is 1 at the start and 0 at the end. □

Checked as `a2_iff_nonnegative_amortized_cost`, `potential_telescoping`, `nfi_by_potential`, and `certification_system_is_potential_method` (CostSemanticsComparison.v), with the state-dependent-cost results `flips_le_cost` and `a2_and_aara_iff_exact` (CostFrameworks.v).

So what's new, if the method isn't? Only where it's pointed. The potential method is a way of proving things about costs. What this book adds is a claim about *which* potential, the flag, and about what happens to that claim when you can only see a window of the state. The method is Tarjan's and Sleator's. The reading is mine, and I'm about to find out what it can and can't survive.

The flag potential has relatives in physics-flavoured mathematics too, which I found after the fact. Resource theories study what you can turn into what when some operations are free: Bob Coecke, Tobias Fritz and Robert Spekkens gave the general definition in 2016, Fritz worked out its algebra in 2017, and Tomáš Gonda and Spekkens catalogued the numbers that never go up under free operations, which they call monotones, in 2023. Read with "certified" as the resource and one mark as the currency, the flag potential is the kind of cost monotone they build, measured in that currency, and the floor is its simplest case. Does that make the toll a resource theory? Only in the trivial way: the conversion costs exactly one mark and that's the whole theory. It's a neighbour to know about. I don't count it as a result.

6.6 What the counter is, said once

A receipt is a total that lives in the state, only climbs, equals the sum of the charges on every path including the failures, and is the only such total for its schedule. Each of those had a home before this book: the stick for a total that lives in the state, Lamport's clocks for climbing, Turing's and Floyd's ranking functions for the same one-way rule run downward, the telescoping sum for conservation, the induction for the single honest count. None of them is physics. None of them says what the charges should be. What they say is that once you've chosen the charges, the counter is the arithmetic. It's no second opinion. And the toll on top of it is the potential method with the flag as potential, which is to say a trick from 1985 wearing the question from Section 2.

Here's one thing the counter can't do just by existing: if you can only see *part* of the state, can you tell what the counter is? Can you even tell, from outside, which step paid? The stick works because you can lay the two halves together. What if you're only ever shown one half, through a window, and the window doesn't show the notches?

One piece of that has a short answer, and I'll give it now so the next section can start from it. A window, for now, is just the part of the state an outside observer is shown (Section 5's display was one, and Section 7 is about windows in general). Call a window *counter-complete* when some rule, applied to what the window shows, returns the counter on every state; *flag-complete* likewise for the flag. Say a window *forgets the counter* when changing the counter and nothing else never changes what the window shows, and likewise for the flag.

The short answer: a window that forgets the counter is never counter-complete, whatever else it shows, and a window that forgets the flag is never flag-complete. Take any state and make two copies of it, one with the counter set to 1 and one with it set to 2, everything else held still. The window shows the same thing for both, so any rule gives the same answer for both, and one of the two answers is wrong. The flag is the same argument with yes and no. On the stick: a window that shows you the wood's colour and its length, and hides the notches, can't tell you what's owed, and staring harder at the colour won't help. (The argument needs a state space where the counter can be changed on its own, the rest held still; that's what "forgets" quietly assumes. The machine I build later is such a space, and it has a checked pair of its own: two runs with the same window, one paid and one not (Theorem 16.14).)

Be careful what that doesn't say. It says neither part can be dropped. It doesn't say a window showing both is the smallest window that works, because nothing here orders windows, so there's nothing for it to be smallest in. And it's about reading the counter back. Which step paid is the harder question, and it's the next section's.

7

Windows and shadows

I left you with the stick split in half and asked what happens if you can only ever see one half through a window that doesn't show the notches, and with a harder question under that one: can you tell, from outside, which step paid? This section is about windows. It's a section where the mathematics is so small that I'm going to state it in full and then spend most of the pages on what it's *not*, because the smallness is the trap: the fact is a one-liner and it's tempting to climb over it with cleverness, and you can't, and seeing why you can't is the content.

7.1 A cave, and the people who got good at it

This one is Plato's, and the part I need is near the end. In the seventh book of the *Republic*, Plato has Socrates, the main speaker of the dialogue, describe prisoners chained in a cave since childhood, facing a wall, with a fire behind them and people carrying objects along a walkway between the fire and their backs. All the prisoners have ever seen is the shadows on the wall. To them, Socrates says, the truth would be nothing but those shadows. Further on, Socrates reads the story out himself: the cave is the world of sight, and the climb out of it is the soul's climb toward what's really there. So a real world hidden from anyone who only looks is in the story on purpose, and Plato put it there. It still isn't the part I need.

What I need comes a little before that reading, when one prisoner, freed and taken up into the daylight, thinks back to the cave. The prisoners, Socrates says, hand out honours to whoever is quickest at spotting the shadows as they pass and best at guessing which comes next. Read that again. The prisoners *get good at it*. They build a whole science of shadows, with experts and prizes, and it works, on shadows. What they can't do is say anything about a carried object that leaves no mark on the wall. Not because they're stupid. Because the information isn't on the wall. Skill inside a window doesn't reach past the window, no matter how much skill. That's the fact. Plato dressed it as a lesson about education, and it is one, but strip the dressing and it's a sentence about functions, and it's going to be the whole section.

7.2 The shadow of a cube

Do this one tonight with a lamp. Hold a cube face-on to the light: its shadow is a square. Now hold up a square of card the same size: same shadow. A person who sees only the wall can't tell you whether what's casting the shadow has any thickness at all. Ask them, and they'll tell you, truthfully, that they can't know, and if they're honest they'll also tell you that no amount of staring at that square will change it.

Now turn both a quarter turn. The card's shadow collapses to a line. The cube's stays a square. Now the wall-watcher knows. What changed? Not their cleverness. The *window*. A second look from a different angle is a bigger window than one look, and what bought it was the turn. Edwin Abbott wrote a novel about a window like this in 1884, *Flatland*. A Flatlander never sees a shape. Abbott's narrator says it in his first section: nothing was visible to them, or could be, except straight lines. Put a coin flat on a table and lower your eye to the table's edge, and the coin turns into a line; that's the whole view. What gives a Flatlander anything more is fog. A line that's near is bright and a line that runs off into the fog goes dim, so a trained eye can read a triangle or a pentagon off how fast the brightness fades toward the ends. The training takes time and money, and in the story only people with money get it. That's a window that got bigger, and it was paid for in time.

Then a sphere comes through the narrator's plane. What his eye gets is a line whose ends fade too fast for any flat figure he knows, changing size in a way no flat figure could. He only settles that it's a circle by feeling it, and the circle grows from a point to its widest and shrinks back to nothing as the sphere rises through. From one slice he can't put a sphere together. From the slices one after another, seen and felt and argued over, he starts to. The escape from a window is never "think harder." It's always "a different window," and a different window that answers the question always has to show apart two states the old one showed alike; that's Theorem 7.1 below, applied to the new window. In every story here, getting that window cost something: a turn, a second look, training, time. What a change of window costs in general is a separate question, and no theorem in this section prices it.

Who was Edwin Abbott? Not a mathematician by trade. He was headmaster of the City of London School for twenty-four years, a clergyman, and the author of more than forty books, a Shakespeare grammar among them. He put *Flatland* out under the name "A Square." His full name was Edwin Abbott Abbott, two A's, and the pen name may be a joke on that, A squared; nice if true, but take it as a lead. Here's the part I like best, from a sketch of his life on the geometer Thomas Banchoff's pages: *Flatland*, by far the most read thing he wrote, isn't mentioned at all in his obituary in the London *Times* or in his entry in the *Dictionary of National Biography*. The official record of the man was a window too: it showed the headmaster and the grammarian

and left out the one book people still read. That changes nothing in the argument. It's the argument, acted out by an obituary. (*Flatland* already turned up in Section 1, where the hunch behind this book started, and in the opening section, The point, which first told the sphere's visit.)

7.3 The whole fact, in one paragraph

Here's the mathematics, and I promise it's all of it. You have a set of states, the real things. You have a window, which is a function that takes a state and gives you what you can see of it: the shadow, the slice, the half-stick without notches. And you have a question about the state, which is also a function: it takes a state and gives an answer. The thing you want is a *decoder*: a rule that takes what you can see and gives the right answer to the question, every time. When does one exist?

Think about two states that cast the same shadow. The decoder sees the same shadow for both, so it gives the same answer for both. So if the question has *different* answers on those two states, the decoder is wrong on at least one of them, and there's nothing it can do about it, because it never saw the difference. That's the whole fact. If a decoder exists, then the question must already give the same answer on every pair of states that look the same through the window. Mathematicians call the set of states that cast one particular shadow the *fibre* over that shadow (think of the shadow as a point on the wall and the fibre as the bundle of all the things that could have cast it), and the sentence is: a decoder exists only if the question is constant on every fibre. That's the theorem just below, and the "only if" is the direction that bites. The other direction, "if the question is constant on fibres then a decoder exists," is true too, but it needs you to be handed, for every possible shadow, one actual state that casts it, so you can look up the answer there, and the handing is not free either, since someone has to supply a representative for every shadow. (You can always fill a gap with a default answer. The default doesn't recover what was really there.)

Exactly. Let S be any set of states, V any set of views, $w : S \to V$ a window, and $q : S \to A$ a question with answers in any set A. A *decoder* is a function $d : V \to A$ with $d(w(s)) = q(s)$ for every state s. The question is *constant on fibres* when $w(s) = w(s')$ implies $q(s) = q(s')$.

Theorem 7.1 (Decoders need constant fibres)**.** If a decoder exists, the question is constant on fibres. Conversely, if the question is constant on fibres and you are handed, for every view v, a state $r(v)$ with $w(r(v)) = v$, then $d(v) = q(r(v))$ is a decoder.

Proof. If $w(s) = w(s')$ then $q(s) = d(w(s)) = d(w(s')) = q(s')$. Conversely, $d(w(s)) = q(r(w(s)))$, and $r(w(s))$ has the same view as s, so by constancy its answer is $q(s)$. □

Checked as `decoding_requires_fiber_constancy` and `selected_representatives_give_decoder` (ObservationPolicy.v), for arbitrary types.

Try the smallest case, by hand, because it'll keep coming back. A state is a pair: a position, 0 or 1, and a hidden flag, up or down. The window shows the position only. Two moves from (0, down): one sets the position to 1 and raises the flag; the other sets the position to 1 and leaves the flag alone. Both land on position 1. Now write down every possible decoder from "what I see" to "is the flag up": there are exactly four functions from two positions to two answers. Check each against both results. None of the four gets both right, because both results show position 1 and one has the flag up and one doesn't. That exhaustive check is the theorem on this example; the general statement is the one-line argument above, and that's what's checked. There's no fifth function hiding somewhere.

7.4 People who found this on their own

The fact is so plain that it got discovered separately in every field that has to deal with seeing part of something, and I want to lay them beside each other, because that's the kind of evidence I trust: people who'd never met kept walking into the same wall.

In 1956 Edward Moore, working on the small switching machines that became computer hardware, asked when an experimenter who can only feed a machine inputs and watch its outputs can tell two of its internal states apart. His answer gave a precise meaning to "indistinguishable from outside," and a theorem with a sting in it: even when every pair of states can be told apart by *some* experiment, there are machines where no single experiment tells you which state you started in. What you can always do is run an experiment after which you know which state you're in *now*. In 1960 Rudolf Kalman, whose filter went on to aerospace and industry, asked the same question for continuous systems, called it *observability*, and gave a test (a matrix rank, which you can compute) for whether watching the output over time pins down the internal state. His unobservable states are exactly a fibre: states whose difference never reaches the output, and never will, however long you watch. That second half took me longest to take in. More samples of the same window only repeat the same fibre. Try it on a car: if your only instrument is the speedometer, two cars a mile apart at the same speed read the same forever, and nothing on that dial will ever tell you which mile marker either one started at. Read the mile markers instead and, over time, you recover both position and speed. Same car, different window.

Where did Kalman's filter go first, before aerospace and industry at large? Toward the Moon. In the autumn of 1960 Kalman called an acquaintance, Stanley Schmidt, at NASA's Ames Research Center, and came to visit, not knowing what Schmidt's group was working on. The group, a branch with eight people doing the analysis, was stuck on how a spacecraft going around the Moon and back could work out where it was from what its crew could measure through an optical sight: a few angles to the Earth and the Moon. That's a window problem with a capsule in it. The state is position and velocity, the view is a handful of angles, and the job is to keep a running best guess at the state from a sequence of views. Kalman presented his new filtering paper to them, and with real work (their problem wasn't linear and his theory was) it fit, and it went into the feasibility studies for navigating Apollo. Leonard McGee, of Schmidt's group at Ames, and Schmidt, telling the story in 1985, call it a fortunate sequence of events and say the solution might have gone undiscovered but for that meeting. What it changes: it shows the limit from the useful side. The filter didn't make the angles show more. It made a sequence of views, over time, pin down the state, and it can only do that when the state is observable from those views, which is Kalman's own condition. That's the mile markers. A cleverer speedometer would never have got there.

In 1977 Patrick and Radhia Cousot built a whole discipline of program analysis, called abstract interpretation, on throwing information away *on purpose*. You run the program over coarse descriptions of its values, and the exact values never enter. And you accept that the answer you get has to be true of *every* real value the coarse description could stand for. That last sentence is fibre constancy as a design rule. And in 1970 Edgar Codd, inventing the relational database, made programs independent of how the data is stored, and the relational systems that followed gave users *views*, deliberately partial windows onto the stored data. A decade after Codd, in 1981, two database researchers, François Bancilhon and Nicolas Spyratos, gave the view-update problem an exact semantics. The problem is this: if a user changes what they see through a view, which of the many underlying states that cast that view should the database move to? Several are legitimate. The fibre, from the writing side.

Two more fields have the fact as a founding definition. In the theory of knowledge, what an agent knows is modelled by a partition of the possible states into cells, the states it can't tell apart, and it knows a fact at a state exactly when the fact holds at every state in that state's cell. The economist and game theorist Robert Aumann used that model in 1976 to prove that two people who start from the same prior, and whose estimates of an event's chance are common knowledge, can't agree to disagree; the computer scientists Ronald Fagin, Joseph Halpern, Yoram Moses and Moshe Vardi built a whole logic of knowledge on the same partitions in *Reasoning About Knowledge* (1995). Theorem 7.1 is that model with the cells written as fibres:

a window knows the answer to a question exactly when the question is constant on its cell. And in computer security, the computer scientists Joseph Goguen and José Meseguer defined *noninterference* in 1982: one group of users doesn't interfere with another when nothing the first group does changes what the second can see. A window that can't see the counter is the counter not interfering with the window's view, and the clerk below is what that costs.

And Parnas, from Section 2, hid things on purpose: a module is characterised by the design decision it *hides* from its callers. Hiding is a window. The caller can't later read back what was hidden, including the things it later wishes it could check, and that's the price side of the coin whose benefit side Parnas was selling. None of these people were doing philosophy. They were trying to build filters, databases, compilers, and modules, and the wall is the same wall in each.

One more, from information theory. Oliver Kosut, at Arizona State, and Jörg Kliewer, at the New Jersey Institute of Technology, two information theorists, asked in 2018 when a receiver on a channel with a possible attacker, and no shared key, can always either get the message right or notice the attacker. The answer is all or nothing: the rate is zero, or it's everything the channel would carry with no attacker at all, and which one depends on a condition they call *overwritability*, that the attacker can make what the receiver sees look exactly like an honest transmission of some other message. That's a printer, a way to put the same thing in the window without the event behind it. The fibre fact is their condition in its plainest form, with the probabilities taken out.

And one from machine learning, where the two computers turn up with a lawyer attached. Four machine-learning security researchers at the University of Toronto and the Vector Institute, Anvith Thudi, Hengrui Jia, Ilia Shumailov and Nicolas Papernot, asked in 2022 how a company could prove a trained model had forgotten someone's data, and found that the same model can come out of different training sets, so the finished model can't show whether a point was used or removed. The only claim they found you can audit is about the procedure that was run. Same wall: the finished state is a window, and what happened is in the run.

Philosophers who study testimony, which is the trade of being told things, have the same wall from the other side. Read one after another they blurred into a list of opinions for me, and the list didn't tell me anything until I built a picture that showed where each one sits. So here's the picture first.

Somebody passes you a note across a room. It says the answer to question seven is 42. What crossed the room? Ink. Whatever made the writer believe 42 stayed on their side: the working-out, how careful they were, whether they knew or guessed. Pull that apart and there are five things in play: the claim on the paper, the writer's head, whatever process got the writer there, the trip across the room, and your head

at the other end. Every position I found puts its weight on one of those five. Once I saw that, they stopped being a list and became a map, and the map has connections the list hid.

The two ends of it are Locke and Burge, and they're the same dial turned opposite ways. The philosopher John Locke, in the first book of his *Essay* in 1690, says believing something because someone with authority said it isn't knowing it. If the reason didn't cross the room, you don't have it. Tyler Burge, a philosopher of mind and knowledge at UCLA, argued in 1993 that you're entitled to accept what you're told, if it's presented as true and it makes sense to you, unless you have a reason not to. Locke says only ink crossed, so you've got nothing. Burge says the ink comes with a default, and the default holds until something knocks it over. I get the pull of Burge. Nobody could get through a day checking every note. But a default anybody can trigger by writing something sensible is a free record, the thing this book keeps refusing, so on that dial the book sits at Locke's end. And sitting there has a price: staying there means carrying the check across the room every time.

Jennifer Lackey, a philosopher at Northwestern who has spent much of her work on testimony, cuts the wire between the writer's head and what arrives. Her case, from 2008, is a teacher who doesn't believe the theory she's required to teach, and her students come away knowing it anyway. I went back and forth on whether those students really know it, and I haven't settled it. But look at where the case cuts. If it's right, what arrives at your end never had to be believed at the other end, and the writer's head drops out of the map. Sanford Goldberg, also at Northwestern, moves the weight one more place over, onto the process. In 2010 he argued that whether your word justifies me can depend on how reliable the processes behind your word were, which sit outside my head. That one took me a few reads. What I think he's saying is that part of my reason for believing you lives in you, in machinery I can't see, and he's fine with that. I'm not, quite, and why I'm not is why this book exists: machinery I can't see is machinery I can't check. Christoph Jäger and Nicholas Shackel, philosophers at Innsbruck and Cardiff, built a model of degrees of belief in 2025 in which the writer knowing is neither necessary nor sufficient for the reader to know. That lands where Lackey's teacher landed, from a completely different direction, a story on one side and a calculation on the other. Agreement like that I trust more than either piece alone, as long as I remember a model only shows what can happen inside its own rules.

Then two sets of people who drew arrows. Wesley Holliday, a logician then at Stanford, put testimony into a logic of belief change in 2009, with an authority graph recording whose word each agent takes and rules for what hearing a speaker does to a belief. I've only read a summary of Holliday's paper, so I'm taking that on somebody's word, which is a funny thing to admit in this paragraph. Security

people had drawn the same kind of arrow two decades earlier. In the logic of authentication of Michael Burrows, Martin Abadi and Roger Needham (1990), two computer scientists at Digital's research lab in Palo Alto and one at Cambridge, if I believe an authority has jurisdiction over a statement and I believe the authority holds it, I believe it. Set that beside Burge and it's his default with one change: the authority is named before the note arrives. That one change is the whole difference. Burge's default trusts whoever is writing. Theirs trusts only a writer you picked in advance.

Kenny Easwaran is the one who goes after what crosses the room itself. He's a philosopher who works on probability and on how mathematicians know things, and in 2009 he took up a question I'd never have thought to ask. The philosopher Don Fallis had pressed it in a series of papers: why will mathematicians accept a proof that's long and complicated, or partly carried out by a computer, and turn down a probabilistic one that can be far more certain? That looked backwards to me at first. More certain should win. Easwaran's answer is a property he calls *transferability*. A deductive proof puts the reason on the paper. Every step crosses the room, so you can learn from the writer without trusting the writer. A probabilistic proof leaves the reason behind and sends only the writer's word about what the dice did. Once I saw it that way it stopped looking backwards, and it's the cleanest statement I found of what this book does: make the reason cross the room, carry the thing, and run the check on your own side. That reading is mine. His subject is proofs passed between mathematicians, so if it's wrong, it's wrong on me.

Now push on my own spot on the map, because it gives. The book carries the check across the room, but the check runs on a checker, and I trust the checker because it was proved honest once, outside every run. That's a jurisdiction rule. It's Burrows, Abadi and Needham's arrow with the checker as the named authority, so I'm not sitting at Locke's end as cleanly as I'd like. What I can say is that my authority was picked in advance, that it's a single named thing while Burge trusts every writer, and that its honesty is a proof you can rerun yourself. That's a smaller trust than Burge's, and a different kind from Goldberg's, because the machinery is out where you can look at it. It isn't zero, so I'm pointing at it here myself.

Only Burrows, Abadi and Needham write about machines passing messages, and even they write about the computers as if the computers held beliefs. The rest are about what survives the trip from one head to another. Either way it's the copier's problem, with philosophers' names on it and, in one case, computer scientists'.

Distributed-systems people hit the wall too, and one pair of them measured how far it goes. Their papers were a pile of acronyms to me until I put them all in one room. People at desks work from one rulebook and pass slips to each other. Nobody

sees into anybody's head; all anyone gets is the slips handed to them. Somebody may be cheating. What can the honest ones do about it with nothing but slips?

Andreas Haeberlen, then at the Max Planck Institute for Software Systems in Saarbrücken, and Petr Kuznetsov, then in Berlin, asked in 2009 which faults an outside observer can detect at all. As I read their answer, it sorts cheating by where it sits. A cheat can break the rulebook without changing any slip. A cheat can write a slip an honest player could have written too, for all anyone holding it can tell. A cheat can fail to send a slip that's due, an omission fault, or send one the rulebook would never produce, a commission fault. The first two, they argue, no algorithm can detect. The last two can be caught, and they put a floor under how many messages the catching costs, which I liked for obvious reasons. My first guess was that the classes ranked how bad a fault is. They don't. They rank how far the fault gets toward a slip, and the first two are a fibre: an honest room and a crooked one that hand out the same slips. That's the fibre fact with an auditor attached.

Their group's PeerReview system (Haeberlen, Kouznetsov, the same Kuznetsov spelled another way, and Peter Druschel, the institute's founding director, 2007) is the ladder up to that wall: every node keeps a secure record of the messages it sent and received, and a checker replays the record against a reference implementation, a copy of the rulebook it can run, which exposes the faults a correct node got to see. That's close to the whole-state escape from the three escapes below, built: ship the log, and the checker re-runs it. Close, because a log of slips stands in for a whole state only if the rulebook can rebuild what a desk did from them, and only if the checker's copy of the rulebook is the right one.

The next three ask what you can say once you're at the wall. Ralf Küsters, Tomasz Truderung and Andreas Vogt, three security researchers at Trier, asked what a verdict should hold. They gave in 2010 a general definition of accountability, a verdict naming who misbehaved, and argued, with a proof inside their own definitions, that verifiability, the property that an outsider can check a result, is a restricted form of it. The way I read the restriction: verifiability says something went wrong, and accountability points at a desk. Radha Jagadeesan, Alan Jeffrey, Corin Pitcher and James Riely, programming-language people at DePaul in Chicago and, for Jeffrey, Bell Labs, asked when the verdict comes. They built in 2009 an operational model of systems that enforce a policy after the fact by audit, and studied the trade between the power of the auditor, the cost of the audit and what's asked of the agents. Robert Künnemann, Ilkan Esiyok and Michael Backes, at the CISPA security center in Saarbrücken, in 2019 made the idea a mechanized check, with Certificate Transparency (CT), the public logging of the certificates web browsers trust, among their examples. As I understand it, they did it by turning accountability into a

property of the trace, the whole run. That's Thudi and his coauthors' wall again: what happened is in the run.

In September 2026 Timo Treitz, a student at Saarland University, and Künnemann gave CT the full treatment (Section 14 has my fragment of it) and found, in their model, that plain CT gives accountability only if the log is assumed honest, an assumption the SCT auditing extension (SCT is the signed certificate timestamp a log hands out) can drop and the gossiping extension can't. In my room, CT hands PeerReview's log of slips to a third party to keep. An honest log is the commitment bit from the three escapes below, with a name on the contract: somebody's promise that the bit is set when, and only when, the claim holds, trusted at the step.

Turn the room on me and it gives twice. A flag raised for free that never reaches the window is Haeberlen and Kuznetsov's first class, and no auditor outside catches it; that's the clerk's trouble, next. My way out is PeerReview's, and it trusts a copy of the rulebook too: the checker, proved honest once, outside every run. And every verdict in this book is accept or reject on one claim. It never points at a desk. In Küsters, Truderung and Vogt's terms that's the restricted form, and the rest of accountability isn't here.

7.5 The clerk with two photographs

Now point it at the counter, because that's why this section is here. Suppose you want to bill for raising the flag, exactly: at least one mark for every step that raises it, and nothing for a step that doesn't. And suppose the person doing the billing is a clerk who sees only a *window* on the state, a photograph before the step and a photograph after. The clerk's bill is a function of the two photographs. Can the clerk bill exactly?

Not if the window has the wrong kind of blind spot. Here's why, and it's the cube again. Suppose there are two steps that give the clerk the same before-photo and the same after-photo, and one of them raised the flag and the other didn't. (The two-move toy above is exactly such a pair; so are "a stamp goes onto a document inside a closed drawer" and "someone opens and closes the drawer.") The clerk has one bill for that pair of photographs. If the bill is at least one, it overcharges the step that didn't raise the flag. If it's zero, it undercharges the step that did. Any bill that never misses a raise bills something that wasn't a raise, and no bill from those photographs is exact. The fix is the plain one: put the flag in the photograph. If the window shows the reading, then "one when the reading goes from down to up, zero otherwise" is exact, and such a window has no pair like the one above. That's enough. It isn't the only way: a window can hide the flag and still have no colliding pair, in which case a bill from the photographs can be exact; what ShadowPricing.v

proves the other way is only that a colliding pair rules exact pricing out. And "exact" here means rise-only pricing: at least one on every step that raises the flag, nothing on any step that doesn't. That's all the clerk's problem needs, and the bill "one for a raise, zero otherwise" meets it (`step_price_is_exact`). So it's exact about *which* steps get charged. It isn't exact about *how much*, and it doesn't pick out one bill: "two for a raise, zero otherwise" meets it too. Pinning the bill down takes one more demand, a ceiling: over any run, the total never exceeds the number of raises. With the floor and that ceiling there's exactly one bill left, one per raise and zero otherwise; that's an if and only if, proved for any system whose bill is decided step by step (`exact_commitment_pricing_characterization`, CommitmentPredicateAdequacy.v). These are stated for any states and any steps. The window version, the cube theorem with a price in place of a decoder, comes next.

Same clerk, now in symbols, so you can check every word I just used. Take the five parts of Section 3 with a window $w : S \to V$, and say a step (s, i) *raises* the flag when $\mathrm{cert}(s) = \text{no}$ and $\mathrm{cert}(\mathrm{step}(s, i)) = \text{yes}$. A *price* is any function of a state and a move. It *meets the floor* when every raise costs at least one, and *never overcharges* when every non-raise costs zero; together, it is *exact* (about which steps are charged; the amount is pinned only by the whole-run ceiling above). A *window price* is one computed from the two photographs only: $\mathrm{price}(s, i) = b(w(s), w(\mathrm{step}(s, i)))$ for some bill $b : V \times V \to \mathbb{N}$. A *collision* is two steps (s_1, i_1) and (s_2, i_2) with $w(s_1) = w(s_2)$ and $w(\mathrm{step}(s_1, i_1)) = w(\mathrm{step}(s_2, i_2))$, the first a raise and the second not.

Observation 7.2 (No exact window price across a collision)**.** If the window has a collision, every window price that meets the floor charges at least one on some step that is not a raise; so no window price is exact. If instead the reading is a function of the view, $\mathrm{cert}(s) = r(w(s))$ for some r, then the window price "one when r goes from no to yes, zero otherwise" is exact, and the window has no collision.

Proof. Across a collision both steps get the same bill $b(v, v')$. The floor makes it at least one because the first step is a raise; the second step, not a raise, pays it. If the reading is r of the view, the window price reads the raise directly, and two steps with the same photographs either both raise or both don't, so there's no collision. □

Checked as `shadow_floor_overcharges`, `shadow_cannot_price_exactly`, `window_showing_reading_prices_exactly`, and `window_showing_reading_has_no_collision` (ShadowPricing.v), for arbitrary types.

So the exact price of raising the flag lives in the step itself, or in a window with no colliding pair, and a window that shows the flag is the plainest such window. A window that contains a pair like that one can't carry it. Whether a particular machine

has such a pair, for a particular window, has to be checked machine by machine and window by window; the theorem only tells you what a pair does once you've found one.

There's a tidy way to say what the best window prices are, and it isn't mine. Give each pair of before-and-after photographs the highest charge of any step that casts that pair, and you have the cheapest window price that never undercharges; give it the lowest, and you have the dearest one that never overcharges. Those two are the "there is" and "for all" of logic taken along the window, which F. William Lawvere, an American mathematician best known for his work in category theory, the mathematics of maps between structures, showed in 1969 ("Adjointness in foundations") are the two adjoints of pulling a statement back along a map. Four researchers at the University of Calgary, Robin Cockett, Isabelle Geng, Carlo Maria Scandolo and Priyaa Srinivasan (2023), used the same move, as Kan extensions, to extend resource measures along a map. In those terms the theorem says that an exact window price exists exactly when the two coincide on every pair of photographs, and a collision pulls them apart. That restatement is mine, and nothing checks it.

Control engineers have been hunting pairs like that for a long time, under other names. Discrete-event systems is the field of machines that jump from state to state on events, some of which an outside observer sees and some of which it doesn't. In 1995 five engineers, Meera Sampath, Raja Sengupta, Stéphane Lafortune and Demosthenis Teneketzis of the University of Michigan and Kasim Sinnamohideen of Johnson Controls, asked when such an observer can always work out, from what it sees, that a hidden fault event has happened, within a bounded number of further steps. They called that *diagnosability* and gave a test for it on finite machines. Anooshiravan Saboori, then a doctoral student at the University of Illinois, and Christoforos Hadjicostis, the engineer who supervised his doctorate there, asked in 2007 the question the other way round, for secrets: a system is *opaque* when an observer can never be sure it's in a secret state, because some run outside the secret always looks the same. Read the raise as the hidden event and the window as what the observer sees. Then a collision is the raise failing to be diagnosable with no delay allowed, and a window that can't tell the flag is the flag's state being opaque to it (Theorem 16.15, later, is that, for the small machine). That's my reading; they don't put it that way. Their versions are richer than mine: they allow delays, look at whole runs, and come with algorithms. What this section adds is only the price: what a bill computed through the window does across one such pair.

Why can't the clerk just watch the step itself? A clerk who does, who sees the whole state before and the move, bills exactly with no trouble at all: "one for a raise" is a function of exactly those two things, and it's the step price above. So the collision bites an observer who doesn't see the step, or who's handed an account of the step

and can't trust the account. That second kind is the opening chapter's trace problem come back. If the names of the moves arrive in a log the program wrote, the program can write whatever names it likes, and a clerk reading that log is reading one more window, one the program painted. Seeing the move means being underneath the program, where the step is.

7.6 The three escapes, and what each one costs

Suppose you're the person outside, and you want to be convinced that a run paid. The window you've got doesn't show the counter. Here's how bad that is, in the smallest setting I can build. Take a counter machine from Section 6, give its states a position and a flag as well, and give it a window that shows the position and nothing else. Now take two runs from the same start, with the position at 0, the flag down and the counter at 0. Run A makes one move that raises the flag, costs one, and lands at position 1; its counter reads 1. Run B makes one move that does nothing but step to position 1, at cost zero; its counter reads 0. Call the pair of photographs, position 0 then position 1, the *transcript*. Both runs leave exactly that transcript. A *verifier* is a rule that takes a transcript and says accept or reject. It is *sound* when every transcript it accepts comes only from states where the claim holds, and *complete* when every transcript from a state where the claim holds is accepted. The claim is "the counter is 1."

No verifier on these transcripts is both, and the proof is three lines. Suppose one were both. Run A ends with the counter at 1 and leaves the transcript, so completeness says the verifier accepts it. Now soundness, on that same accepted transcript: every state that leaves it satisfies the claim, and run B's end state leaves it. So run B's counter is 1. It is 0. That's the cube again, with a verdict in place of an answer: a verifier sees only what the window kept, and two states that the window can't tell apart get one verdict between them.

Now dress the transcript up. Take any richer kind of transcript, with a way to squash one down to its bare photographs, and say a verifier on the rich ones *only looks at the bare part* when two rich transcripts that squash to the same photographs always get the same verdict. If some rich transcript left by run A and some rich transcript left by run B squash to the same photographs, then no verifier that only looks at the bare part is sound and complete either: it gives those two the same verdict, completeness makes that verdict accept, and soundness then lies about run B. Notice what that needs. The pair has to survive into the richer kind of transcript; a design whose extra part tells A from B doesn't meet the premise, and the argument says nothing about it. So it doesn't classify every transcript design. What it says is that a verifier that only reads what the bare window keeps fails on that pair.

Economists got to the same wall by another road. Jerry Green and Jean-Jacques Laffont (1986) asked what a mechanism can do when people can claim about themselves only what their situation lets them back up, and Jesse Bull and Joel Watson (2007) built that into mechanism design as *hard evidence*. In their terms, an audit can tell apart only the cases the evidence on offer tells apart. That's the cube with a ledger in it.

What can you do? Here are three things that work, and nothing says they're the only ones; as far as I can tell, cryptography uses all three. Each rests on something you have to trust, and I'll say what, because "a price" in the counter's sense is not what any of them costs.

You can be shown the *whole state*. Then there's nothing to decode; you read the counter off the state. The price is trust: you have to believe the state you were shown is the state there was.

You can be shown a *commitment*: the ordinary window plus one extra bit that is set when, and only when, the claim holds (here, that the counter reads exactly one). Reading that bit gives a verifier that is sound and complete for that claim, *if* the bit is binding (set only when the claim holds) and honest (set whenever it holds). The trust is in those two "ifs," which are a contract about who can set the bit. The contract can be met, and a bit anyone can set breaks it, which is the one-line version of why an unsigned log is just another window. Nothing in that picture has a hardness assumption in it; whether a real commitment scheme meets the contract is a question for cryptography, and this book doesn't take it up. (A clash of words I should own: "commitment" here is the cryptographers' word, something bound to a claim, which in their schemes also hides the claim until it's opened, and this bit doesn't bother hiding anything. The opening chapter's commit is a different act, the closing of a question. Same word, two jobs. The sealed envelope has an exact theory of its own. Three information theorists, Andreas Winter, then at Bristol, and Anderson Nascimento and Hideki Imai, then at Tokyo, found in 2003 the best rate at which a noisy channel can carry commitments, and the cryptographers Claude Crépeau, Rafael Dowsley and Nascimento (2020) worked out how that rate drops as the committer gets some control over the noise, which is the opening chapter's "take the pen away from the program" with a formula on it.) In 1999 the cryptographers Bruce Schneier and John Kelsey worked out how to make a log that an intruder who takes over the machine can't quietly rewrite, and the honest scope in their paper is the honest scope here: it protects what was written *before* the break-in, through keys, and nothing after. Two cryptographers at the University of California, San Diego, Mihir Bellare and Bennet Yee, had named that property *forward integrity* in a 1997 report and proved their scheme has it. Point it at an AI agent that keeps its own log of what it did, and the line moves to the very start: an agent that writes the wrong thing

from its first step is on the far side of the break-in for the whole log. A 2026 preprint on tamper-evident audit trails for such agents, built from hashes anchored on a public blockchain, is even called Agent Flight Recorder (by Laurent Bindschaedler, of the Max Planck Institute for Software Systems, and Quentin Botha and Christoph Siebenbrunner, of the Vienna University of Economics and Business), which is the opening chapter's word, and the opening chapter's fingerprint paragraph is the part of this book that applies to it.

Or you can *ask*. Interaction: the outside party gets to put a question and gets an answer that pins the thing down. The price is the honesty of the answerer; a liar's answer is another shadow. Moore's experimenter chose inputs; the zero-knowledge proofs of the cryptographers Shafi Goldwasser, Silvio Micali and Charles Rackoff in 1985 are the refined form, where a verifier is convinced by a conversation designed to cut exactly the line between true and false and nothing else, with a mathematical guarantee against a cheating prover. I mention zero-knowledge to say what I'm *not* claiming: the asking in this section is one round, with one fixed question ("tell me your counter"), and nothing in it models a cheating prover; the honesty is assumed, and nothing enforces it. That's a toy of the real thing, and I'm not going to let the real thing's reputation stand behind the toy. (Joseph Halpern, Yoram Moses and Mark Tuttle, computer scientists who work on reasoning about knowledge in distributed systems, gave zero knowledge a knowledge-based analysis in 1988, which is where I'd start for the logic of it.)

The real thing even has a cave in it, which after Plato I can't resist. In a paper for the 1989 CRYPTO conference called "How to Explain Zero-Knowledge Protocols to Your Children," two cryptographers, Jean-Jacques Quisquater of Philips Research and Louis Guillou of the French telecommunications lab CCETT, with their families and Tom Berson of Anagram Laboratories on the byline, told it as Ali Baba's cave: a ring-shaped tunnel with a magic door at the back that opens only for a secret word. Peggy goes in down one side while Victor waits outside; then Victor walks to the fork and shouts which side she should come out of. If she knows the word, she always can. If she doesn't, she gets away with it only when Victor happens to call the side she went in. Play it at the table with a coin for Victor's shout: one round, a cheat gets through half the time; twenty rounds, about one time in a million. And here's the part that belongs in this section. In the story, Victor ends up convinced, but a video of the whole thing convinces nobody else, because a staged run, where the two agreed beforehand which sides he'd shout, makes exactly the same recording. The recording is a window, and the real run and the fake one cast the same shadow. That's the cube in the cryptographers' favourite bedtime story, and it's why the real thing has to ask live, many times, against a prover who might be lying: everything my one round leaves out.

Each escape is a window with something added: the whole state, a bound bit, an honest answer. None of them is "a smarter verifier on the old window." Cleverness lives on the wrong side of the projection. What's true of any machine is the cube.

Here are the three constructions exactly, for the pair above, with the relations written down. The relation matters: it says which state a transcript is allowed to describe. If it quietly assumes a truthful answer, then a verifier sound for it is sound only against honest answerers, and it says nothing about a liar. Write A and B for the states runs A and B end at, μ for the counter, and τ for their shared transcript.

Definition 7.1 (Three richer transcripts and their explanations)**.** Keep the states A, B and their shared bare transcript τ, with $\mu(A) = 1$ and $\mu(B) = 0$. Write $E_0(s, t)$ for $t = \tau \ \wedge \ (s = A \ \vee \ s = B)$. The claim is $\mu(s) = 1$.

A *full-state transcript* is a finite list t of states. Define $E_S(s, t)$ to mean that t is nonempty and its last entry is s. The verifier rejects the empty list; otherwise it accepts exactly when the last entry's counter is 1.

A *commitment transcript* is a pair (t, b) of a bare transcript and a Boolean. For any chosen explanation relation E_C, the *commitment contract* is the pair of implications, for all s, t, b:

$$E_C(s, (t, b)) \wedge b = \text{true} \implies \mu(s) = 1,$$

$$E_C(s, (t, b)) \wedge \mu(s) = 1 \implies b = \text{true}.$$

These are binding and honesty, respectively. The verifier accepts exactly when b is true. One relation meeting the contract is $E_H(s, (t, b)) \iff E_0(s, t) \wedge (b = \text{true} \iff \mu(s) = 1)$.

A *response transcript* is a pair (t, r) of a bare transcript and a natural number. Define $E_I(s, (t, r)) \iff \mu(s) = r$. The verifier accepts exactly when $r = 1$. This relation imposes no condition on the bare component t; the response alone carries its entire truth condition.

Each construction assigns verification the constant abstract cost 1.

Theorem 7.3 (Three conditional escapes)**.** The full-state verifier is sound and complete relative to E_S. For every E_C satisfying the commitment contract, the commitment verifier is sound and complete relative to E_C. The response verifier is sound and complete relative to E_I. Each has cost 1 under the stipulated cost assignment. The honest relation E_H satisfies the commitment contract; the unchecked relation $E_U(s, (t, b)) \iff E_0(s, t)$ does not.

Proof. For the full-state verifier, an explaining state is precisely the last entry. Reading that entry's counter therefore accepts exactly when the explaining state's counter is 1. An empty transcript is rejected and has no explaining state.

For the commitment verifier, acceptance says b is true. Binding then gives the claim for every explaining state, which is soundness. Conversely, honesty says that any explaining state satisfying the claim has a true bit, which the verifier accepts. The equivalence in E_H gives both implications directly. Under E_U, however, B explains (τ, true) even though $\mu(B) = 0$. That violates binding.

For the response verifier, explanation gives $\mu(s) = r$. Acceptance adds $r = 1$, so $\mu(s) = 1$. Conversely, $\mu(s) = 1$ and explanation give $r = 1$, hence acceptance. The three cost claims follow from the assigned constant. □

Checked, for any pair like A and B, over arbitrary types of states, claims, transcripts and explanation relations; nothing in it uses more about the two states than their claim values and their shared transcript. The collision argument is `ver_collision_blocks`, the richer-transcript one is `ver_no_factor` (a verifier sound and complete for the claim under an explanation relation cannot be one that only looks at the bare part, once two transcripts with the same bare part are explained, one by a state with the claim and one by a state without it), and the three escapes are `ver_full_state_escape`, `ver_commitment_escape` (under the contract, with `ver_honest_meets_contract` and `ver_unchecked_breaks_contract` for the two relations), and `ver_response_escape` (VerifierSmall.v). The cost 1 in the definition above is not in any of those statements; it is a number I wrote down, and no proof reads it.

On a real machine the collision is a theorem, and it doesn't depend on the toy. Give a run from a loaded start (Part IV's word for a start at line 1 with the two counters set to a and b, written $(1, a, b)$) the *bare transcript* made of a, b and the base window the run ends with, and say a state *explains* a bare transcript when some run from that start ends at it showing that window.

Theorem 7.4 (No bare verifier on a Thiele-complete machine)**.** Let a machine be Thiele-complete through an interface (Definition 16.7). (1) No function from bare transcripts to Booleans is both sound and complete, under that explanation relation, for the claim "the record is up." (2) There is a clean loaded state such that no function from bare transcripts to Booleans is both sound and complete for the claim "the ledger is at least 3 above what it was at that start." (3) For the small machine of Part IV, whose moves CHECK, COMMIT and CERTIFY and whose counter move DEC are defined there: a run CHECK, COMMIT, CERTIFY from the loaded start $(1, 0, 0)$, which ends with the ledger at 3, and three DECs of a zero counter from the same

start, which end with the ledger at 0, end with the same window, and no function on bare transcripts is sound and complete for "the ledger reads 3."

Proof. Observation 16.26(2) gives a loaded state and two runs from it that end with the same window, one with the record up and one with it down, and, from one clean loaded state, two runs with the same window, one whose ledger rose by at least 3 and one made of free base moves that leaves the ledger where it started (that theorem's proof builds it; `complete_two_runs` states it). Both end states explain the same bare transcript. A verifier complete for the claim accepts that transcript, since a state where the claim holds explains it; a verifier sound for the claim then makes the claim true of every state that explains it, including the one where it's false. That's (1), and (2) is the same with the ledger: the paid run's end state satisfies the claim, and the free run's doesn't. For (3) the two runs are written out in the statement: three moves land on line 4 in both runs with both counters at 0, and the same two-state argument applies. □

Checked as `ver_complete_no_record_verifier`, `ver_complete_no_ledger_verifier` and `ver_small_no_ledger_verifier` (VerifierSmall.v), the first two from `complete_hides_record` and `complete_two_runs` of ThieleCompleteWindow.v. Put with the three escapes, the whole picture is one statement for every Thiele-complete machine, `ver_corollary`, and for the small machine `ver_small_corollary`: the bare-transcript verifier doesn't exist; the collision survives into any richer transcript that keeps it, so no verifier that only reads its bare part works either; a verifier on full states exists; one on the bare transcript with a committed bit exists, sound and complete when the bit is honest, and the unchecked bit breaks the contract; and one on a truthful response exists. The collision is a premise of the richer-transcript half, and the contract is assumed; no fourth way is claimed to be impossible, and no hardness is proved.

In the three escapes the collision is gone, and no trick did it: the extra information pulls A and B apart. The full-state transcript shows the counter outright. The honest commitment for A carries a true bit and the one for B a false bit. The honest responses are 1 and 0. In none of the three is the verifier asked to tell identical inputs apart, which was the impossible job.

Now the limit, and I'd rather say it before you find it. A list of states has no authentication built into it, so nothing in it stops a liar from handing over whatever the verifier wants to see. The commitment contract is assumed, and the unchecked bit above is what happens when the assumption is dropped. The response relation only allows a truthful counter value, so asking a question here does not buy resistance to a cheating prover. And the cost 1 is an abstract price I wrote down. It doesn't

measure what it takes to authenticate a state, check a cryptographic proof or send a response. What these three show is which extra distinctions a verifier can use. They are not an adversary model, a security parameter or a hardness reduction.

So what would it take to make that commitment bit binding for real, against somebody who wants to cheat? Cryptographers had to face that one in the open, and the place I'd point is a short paper the computer scientist Manuel Blum, whose Turing Award was for complexity theory and its use in cryptography, presented at the CRYPTO meeting in 1981, "Coin Flipping by Telephone." His setup: Alice and Bob have just divorced, they live in different cities, and they want to flip a coin over the phone for who gets the car, with nobody they both trust to do the flipping. If Alice flips and announces, Bob has to take her word. If Bob calls it first, Alice can lie about how it landed. Blum's way out is for one side to commit first: send something that fixes the choice without showing it, and open it only after the other side has called. And what keeps that commitment binding isn't anybody's promise. It's a bet that a certain kind of big number is hard to factor. Push on that and you're standing on the honest edge of this subsection. In Blum's protocol, binding is earned, on the condition that factoring is hard. Here it's written into the contract and assumed outright, and the unchecked bit above shows what's left without it. That's why I won't say these escapes beat a cheater. Blum's paper is the kind of thing it would take, and nothing like it is in this chapter.

Now point the three escapes back at the record this book is about, because that's where they bite hardest. A record kept in a machine's state is bits too. Hand someone outside a copy of those bits with a note saying "the flag is up," and they're reading a window: the run that really raised the flag and a box built to print that it did cast the same shadow. So putting the record in the step answers "was this earned?" for whoever can see the state and trusts the step that wrote it. For anybody else it moves the question, from "did it run?" to "do I trust the box that says it ran?" Crossing that line takes one of the escapes: the whole state, from a box you already trust; a bound bit, which in practice means a signature over the record or a quote from measured hardware (and Section 14 shows a quote vouches only for what was measured); or asking, against a box that might lie, which is the cryptographers' business. One more relative deserves its name here, *verifiable computation*: the box hands back its answer with a short proof that the answer is right, which a checker can verify without redoing the work (Pinocchio, built in 2013 by the computer scientists Bryan Parno, Jon Howell, Craig Gentry and Mariana Raykova), and proof-carrying data passes proofs like that along a whole chain of parties who don't trust each other (the computer scientists Alessandro Chiesa and Eran Tromer, 2010). That's the strongest export there is, and notice what it exports: the claim. The history stays behind. The proof shows the answer is right for that input. It doesn't show which machine did

the work, or when, and any box with the input can make one. Nothing in this book supplies any of these escapes. The model's record is local to the machine that keeps it, and the small machine of Part IV keeps its record in its state with no signature on it at all.

7.7 What this isn't

Three things this isn't, and I'd rather take them off the table now.

It isn't mysticism. Nothing here says the window is false or the state is unknowable. The wall is a function, the objects are its inputs, and the only claim is that the function isn't one-to-one on the thing you asked about. A richer window recovers it. Plato's prisoners could have turned around.

Rice's theorem is the neighbour this looks most like. It isn't Rice's theorem, though it rhymes with it. The logician Henry Rice, in a paper of 1953, proved that no program can decide any interesting property of what another program *does* from its code, and a cousin of the fact here is that a run which hasn't halted yet looks, so far, exactly like one that never will. But Rice is about what's *computable*. The cube is about what's *there*. The cube theorem holds for a toy with two states and nothing to compute; the obstruction is missing information, and cleverness doesn't come into it. I'd be overclaiming if I let the famous theorem lend its weight to the small one.

And it isn't "with enough data you can recover anything." Kalman's unobservable states stay unobservable for all time. The question is never how much of the window you've got. It's which window.

The counter lives in the state, and whether anyone outside can read it depends entirely on what they're shown, and a window with a pair like that in it can't price the flag exactly, with any rule, ever. Which raises the question I've been pushing in front of me since the buckets. Fine, the flag went up and was paid for and I can see it in the right window. What does that *entitle* me to? A paid flag is a paid flag. What's the claim behind it, and when am I allowed to lean on it?

8

Structural entitlement

I keep writing the word "vouch" and then taking it back. Let me stop doing that and say what it would take. A reading went from NO to YES, the toll charged the move that did it (Section 3), maybe the reading stays up (Section 4 says what it costs to make it stay), and the right window shows it (Section 7). Now: what is a later step *allowed to do* because of that? That's a different question from "is the thing true," and the gap between those two is the subject of this section, and I'd argue it's the subject of most of the arguments people have about evidence, so give me the long way round.

8.1 The coin that moved

Here's an experiment you can run on a friend, and I'd ask you to actually run it, because the feeling of it is the point. Put a coin under one of three cups while they watch. Say, cup one. Send them out of the room. Move the coin to cup three. Then, and this is the trick, put a *second* coin under cup one. Bring them back and ask: "Is there a coin under cup one?"

They say yes. They're right; there is. And they have a reason: they watched you put it there. True, and believed, and justified by what they saw. And it's not knowledge, and you can feel that it isn't, because their reason points at a coin that left. The coin that makes them right is not the coin they're thinking of.

That's the trap from the opening chapter, done with cups: Edmund Gettier's three-page paper from 1963, "Is Justified True Belief Knowledge?", which broke a definition of knowledge in about the time it takes to read it. The definition was that you know something when it's true, you believe it, and you've got good reasons. Gettier took it from two philosophers of the 1950s, Roderick Chisholm and A. J. Ayer, and the only place he mentions Plato is a footnote saying Plato "seems to be considering" something like it in the *Theaetetus*. I'd assumed people had leaned on that definition since Plato. As far as I can find, they hadn't. The Stanford Encyclopedia's entry on the analysis of knowledge calls the idea that it was ever widely held "a bit of a convenient fiction," and Julien Dutant, a philosopher at King's College London, wrote a 2015 paper called "The Legend of the Justified True Belief Analysis" arguing that the older tradition asked for something stronger: a mark on the belief that can't

be wrong. I've read only summaries of Dutant's paper, so take that last part as his, secondhand. Gettier's two cases, and the cups, show a belief that passes all three tests and still isn't knowledge, because the reasons and the fact aren't connected; they just happen to agree. Bertrand Russell had a related case in 1948: somebody looks at a stopped clock at the one moment it happens to be right, and they're right about the time, and they know nothing.

Is 1963 the bottom? Not even close, and 1948 isn't either. The Stanford Encyclopedia's entry on the analysis of knowledge, which is where I'd send you to check me, gives a case from the eighth-century Buddhist philosopher Dharmottara: you're looking for water on a hot day, you see water ahead, it's a mirage, and when you get there you happen to find water under a rock. And one from Peter of Mantua, a logician of the late fourteenth century who died in 1400 and whose *Logica* survives in manuscript: Plato is running right next to you, you take him for Socrates, so you believe Socrates is running, and Socrates is in fact running, in Rome, where you can't see him. Older still? Maybe. This is where the trail goes cold, which isn't the same as where it ends. What it changes is how much I trust the shape. People in different centuries, languages and traditions kept building the same trap by hand: a reason that points at one thing, and a truth that happens to be sitting on another.

What's missing is a wire from the reason to the fact. Ask your friend what they'd need, to be *entitled* to "there's a coin under cup one," and they'll tell you: lift the cup. Not "remember putting it there." Lift it, now, and look. That lifting is an event, it happens on the record, and after it the reason and the fact are the same object.

8.2 Plato nearly had it

Who was Theaetetus, though? The dialogue is named for a real person, and I wanted to know who. He was a mathematician. In the dialogue he's a teenager, and he tells Socrates that his teacher Theodorus had been showing them, one case at a time, that the side of a square of area three, or five, and so on up to seventeen, can't be named by any fraction: the Pythagoreans' diagonal from Section 2, done again case by case. The young man's idea was to stop going one case at a time and find one word for all of them. Euclid's tenth book, on lengths that share no common measure, is thought to be largely his work, and he's credited with being the first to study the eight-faced and twenty-faced solids, the two Plato assigned to air and water in the *Timaeus*. The dialogue opens with him being carried home to Athens from the army at Corinth, badly wounded and sick with the dysentery going round the camp, barely alive; the usual date is 369 BC. And the conversation itself has come down as a record. Euclid of Megara (not the geometer) says he took notes as soon as he got home, filled them out later, and had Socrates correct them whenever he came to Athens, and

he has the result read aloud. So the dialogue about what turns a true belief into knowledge survives as a checked record of what was said. That changes nothing in the argument. I just like that the book about knowledge is named for the man who took the diagonal and made it general.

I want to be fair to the *Theaetetus* here, because it's easy to be quick with the ancients and they deserve better. Socrates, in that dialogue, doesn't just say "true belief plus reasons." He tries three different senses of what the "account" behind a true belief could be: a statement you can say aloud; a list of the thing's parts; and, third, the mark that sets this thing apart from everything else it could be confused with. He tries each, and each fails, and the dialogue ends with nobody knowing what knowledge is, which is one of the more honest endings in philosophy. But the third sense is the one I'd underline. An account as *the thing that distinguishes*: the test that gives the real case a value no rival gives. He couldn't make it work because he had no way to say what the account had to be attached *to*. Two and a half thousand years later, here's this book's guess: the thing it has to be attached to is a state, and the account is an object sitting in it. I'm not claiming that settles Plato. I'm claiming he was pointing at the right thing, and that's my reading of the dialogue, which you can check against it in an afternoon.

Push on that once more, because Socrates gives his own reason for dropping the third account, and it isn't the one I just gave. If the account means *knowing* the mark that sets the thing apart, then knowledge has been defined as true belief plus knowledge of something, and the word you were trying to define is back inside its own definition. A circle. Does a check get out of it? It gets the word out: a test that ran and found the mark isn't defined in terms of knowing. Whether the test was any good is still a real question, and it's the question the rest of this section puts a price on. So I'd say the circle moves. It doesn't vanish. It moves somewhere you can look at it, which is the most I'd claim.

8.3 The shape, said once

So here's what I mean by entitlement, and it has three parts, and I'll try to earn each.

A run is entitled to a claim when three things are present. First, an *object*, carried in the state: a grouping, a relation between groups, a counter, a package of constraints. That's a thing you could print. A belief doesn't count. Second, a *checked relation* between that object and the claim: a test that was actually run, with its result on hand. "A test exists" doesn't count. Third, an *event on the record* that says this object was put there and paid for, so that anyone auditing the run can point back at the moment the claim was marked.

Said as a definition, so the rest of the section has something to point at:

Definition 8.1 (Entitlement)**.** A run is *entitled* to a claim when three things are on hand: an *object* carried in the run's states; a *checked relation* between that object and the claim, meaning a test that was run and whose result is on the record; and a *paid event*, a move of the run at which the object was put in place and which the price list charged at least one. In a certification system (Definition 3.1) a move that takes the reading from no to yes is such a move, because the toll charges it. This is the book's word; there's no Coq definition of it.

Without the third part, the object may be sitting right there and the claim may be true, and the run still hasn't *earned* it. That's the cups: the coin is under cup one; the reason is disconnected from it; nobody lifted the cup. Put it on Section 3's sheet of paper, the one with the NO box, the YES box and the tally line, and grow the toy by one thing: the state is now the box you're in plus whatever is written on the sheet beside it. Next to the boxes, write a note: "coin under cup one, checked." Writing the note is a WORK move, and WORK costs nothing. The note is an object, it's on the paper, and it might even be true. What it doesn't have is a crossing on the record: no move took the reading from NO to YES when the note went down, and there's no mark under the line for it. The note is raw data, forged in one write, and the five parts allow a system that lets you write one, because nothing in them says what an ordinary move may put in the state. A note put down by the move that crosses has a receipt, and the receipt is the event. One limit on that word "receipt": the crossing records that something was paid for at that moment. What the crossing move actually looked at before it crossed is up to whatever the move does, and the five parts don't say.

And here's the part to hold on to. Nothing in the five parts makes a later move *read* the reading. The step function is handed the state and a move name, and it's allowed to ignore the YES box completely; plenty of systems will. So when I say a run is *entitled*, I mean what someone auditing the run may conclude from the reading and the receipt. The system doesn't have to enforce it, and unless somebody builds it to, it won't.

This is a good place to line up the three things the word *established* has had to mean so far, because they blur easily, and a blur here would make the book look like it proves more than it does. The yardstick is the opening chapter's four conditions for a check being about this very claim: the same property, the same thing, the same edition, and an honest check.

What's asked	Bare toll	Entitlement	Earned certification
A check ran	not required	a test result on the record	a passing check before the commit, on every run from a clean start
Same property	not asked	not asked	enforced by the step
Same thing	not asked	not asked	enforced by the step
Same edition, nothing written between	not asked	not asked	enforced by the step
Honest check	not asked	not asked	proved of the checker, once, outside the record
Could have failed	not asked	not asked	not policed
Paid for	at least one mark	at least one mark	at least three marks for the chain

After the column of questions, the first column is Section 3's bare toll: a paid crossing, and nothing says a check happened at all (Section 9 has a remark saying exactly that). The second is this section's entitlement. The third is the small machine of Part IV, whose step won't let a commit through without a passing check of the same property, on the same thing, at the same edition, for every program from a clean start (Theorem 16.8, and Theorem 16.9 for a run that ends certified), whose checker comes with a proof that it says what it means (Lemma 16.1, Theorem 16.11), and whose certified runs from a clean start cost at least three (Observation 16.16). Even there two things are missing, and the first is in the table, where you'll see it. The step doesn't police whether a check could have failed: a property every value has is a perfectly legal check, sound and useless. And in all three columns the status belongs to whoever reads the record. The machine doesn't know anything. What its flag means is assigned from outside, by whoever says what each property means.

Why does the honest check carry so much? The philosopher Linda Zagzebski, who works on knowledge and the virtues of a careful mind, argued in 1994 that any account of knowledge whose justification leaves even a small gap between having good reasons and being right lets a Gettier case through: take a case where bad luck makes the reasons point the wrong way, then add a stroke of good luck that makes the belief true anyway. The first three conditions plus an honest check close that gap by construction, and only one half of honesty is needed for it, the half that says no when the property is false. Saying yes whenever it's true is extra. A check that can be wrong, a test suite that sometimes passes broken code, puts you straight back in

her gap, and then the record certifies something smaller and still true: this procedure said yes.

And who certifies that the check is honest? Not the record. Honesty is a fact about what the check does on every input, and the opening chapter already said a finished state can't hold that kind of fact. It has to be carried by a proof about the checker, done once, outside every run, and nothing on the record pays for it. So an established claim is two things stuck together: a run property the record carries and a program property a proof carries, and the proof is entitlement for the auditor. The machine gets none from it. Section 16 shows where that chain of reasons stops on a real machine.

The engineering version of this was already built, in 1996 and 1997. The computer scientist George Necula, then at Carnegie Mellon, called it *proof-carrying code*: when you ship a program to a host that doesn't trust you, you ship a *proof* with it that the program obeys the host's safety rules, and the host *checks the proof that arrived* before running anything. Not your reputation. Not "a proof exists somewhere." The one in the box. A quarter century earlier Stephen Cook's certificates had the same bones, from Section 3: a claim comes with a witness that's cheap to check, and the checker checks *that* witness. And the Scottish philosopher David Hume, in 1748, arguing about miracles in his *Enquiry Concerning Human Understanding*, laid out the thing the receipt replaces. He weighed testimony against evidence, and wrote that "a wise man, therefore, proportions his belief to the evidence." The way I read him, the whole of his argument is a method for how much to trust a witness when you have no receipt. The move in this book is to stop weighing. A claim on the record is either backed by its object and its paid event, or it's testimony from outside, and a run that leans on testimony hasn't earned it itself. It can inherit it, but only as far as the copy it leans on is genuine, which is Section 7's problem of a record shown from outside.

Robert Brandom, a philosopher at the University of Pittsburgh, built a whole account of language on two words this book leans on, commitment and entitlement, and his *Making It Explicit* (1994) is where I'd send anyone who suspects I invented the pair. In his picture, as I read it and as his reviewers describe it, people talking keep score on each other: what each speaker is *committed* to, what they've put themselves on the hook for, and what each is *entitled* to, what they're licensed to claim. You can be committed without being entitled (a reckless assertion), many claims count as entitled by default until someone challenges them, and entitlement can be borrowed: I can claim a thing on your say-so and defer to you when I'm challenged. Lay mine beside his and the differences are the content. His score is kept by the other people in the room; here it's a reading in the state. His entitlement often comes by default; here it's an event that has to happen and is charged. And his deferral is my testimony, which I've just let back in on a leash: borrowed entitlement is only as

good as the copy is genuine. The leash has an older formal cousin, the jurisdiction rule of Burrows, Abadi and Needham's logic of authentication (Section 7), where you take a statement on an authority's word only if you already believe the authority has jurisdiction over it. It came first, presented in 1989 and printed in 1990, years ahead of Brandom's book of 1994, and its words are *believes* and *jurisdiction* where his are *commitment* and *entitlement*.

There's a whole field that would want a word here too, the logic of belief revision, which asks how a reasonable mind should change what it believes when something new comes in. Its nearest neighbours are worth naming, and I couldn't keep them apart until I put them back on the table with the cups. Your friend believes the coin is under cup one. They can learn something that doesn't clash with that, say cup two is empty, and just add it. They can lift cup one, find nothing, and have to change their mind about a table nobody touched. Or you can tell them you moved the coin. The last two felt alike to me at first. They aren't. In one the friend was wrong. In the other the friend was right, and then the table changed.

Krister Segerberg, a Swedish logician at Uppsala, built a logic of irrevocable revision in 1998: accepting something so that it can't be given up. That's the firmness question taken to its end. Isaac Levi, a philosopher at Columbia who spent a career on how beliefs get fixed and unfixed, split adding a belief into a deliberate kind and a routine kind (I cite his 1991 book; the split goes back to his 1980 one). Deliberate is reasoning your way there. Routine is where you take on whatever a procedure you committed to in advance hands you, the way you'd accept a reading off a thermometer you've already decided to trust, or what's under a cup when you lift it. What I think the split rests on is when the trust goes in: once, before the data comes. A passing CHECK is routine in exactly that sense, with a checker proved honest once, outside every run.

Neil Tennant, a logician at Ohio State, and Gösta Grahne, a computer scientist at Helsinki, I first filed as two separate complaints. They're one move made from opposite sides. The target is the best-known impossibility in that field, the Swedish philosopher Peter Gärdenfors's 1986 theorem that a natural rule for conditionals, the one his paper calls the Ramsey test, can't live with rational revision. Tennant argued in 2008 that it goes through only by treating some additions as revisions: the friend who learned cup two was empty gets counted as having changed their mind. Grahne, working on updates and counterfactuals, argued in 1998 that Gärdenfors's theorem bites only revising, so the friend whose coin was moved is out of its reach. Side by side, they squeeze the theorem from both ends into one case: being wrong about a table nobody touched.

The line this book leans on hardest is between two kinds of change. Revising is changing what you believe about a world that stayed put. Updating is changing it

because the world moved. Two computer scientists, Hirofumi Katsuno and Alberto Mendelzon, drew that line for knowledge bases in a 1991 paper on the difference between updating a knowledge base and revising it; database researchers had been working on updates with incomplete information since the mid-1980s, Arthur Keller and Marianne Winslett among them. Grahne's result stands on it. And look at what the friend sees: an empty cup one is the same empty cup whether they misremembered or you moved the coin. The evidence alone can't tell revising from updating; that's a fibre. What tells them apart is a count of how often the cups were touched. The machine in Part IV keeps that count as an edition number for each thing, and its version rule treats every write as an update: a write moves the edition number, so every fact about the old edition stops counting for the new one.

And Alexandru Baltag, Bryan Renne and Sonja Smets, three logicians at Amsterdam, gave a logic in 2014 that keeps apart three things Gettier's cases blur: justified belief, explicit knowledge and conclusive evidence. The friend who watched has justified belief; the friend who lifted the cup has conclusive evidence. Explicit knowledge I read as knowing with the justification written out beside it, the way the record keeps its check, and I'm less sure of that one. Put them together and they say something exact about this book's record. A record that's never wrong and never corrected would be a dogmatist's dream if its sentences were about the present. They aren't. Each says a property held of a thing at one edition, and that sentence stays true after the edition number moves on. The record can be infallible and never corrected only because its sentences are edition-stamped. Strip the stamps, and the first write would turn it into a liar that can't take anything back.

Push on that and it gives twice. The record is Segerberg's irrevocable acceptance every time, with no revising in it. If a check was dishonest, the only fix that fits a record like this is a second record on top saying "withdrawn," the way a revocation list works, so the infallibility leans on the checker, proved once, outside. And the record is never wrong partly because it only talks about past editions. Whoever reads it wants now, and getting from "held at that edition" to "holds now" crosses every update since. The record doesn't make that step for you.

And the logicians the opening chapter named in passing, the ones who taught the logic of knowing to count. Here they are properly, and this time I want to see what each price rests on. Ho Ngoc Duc, in 1997, gave the logic of knowing an operator for "true after some train of thought": an agent doesn't know every consequence of what it knows now. It can come to know any of them by taking steps of reasoning. Steps. I took that at first for the thing I was after, until I saw which way it points. His operator points forward, at what the agent could come to know. A record points back, at what was checked. One arrow, read from opposite ends. And Sergei Artemov and Roman Kuznets, in 2014, turned logical omniscience into a question of cost: a

logic escapes it when every valid claim that something is known comes with a proof whose size is bounded by a polynomial in the size of the claim. The usual modal logics of knowledge can't pass that test unless NP equals coNP, which most people would bet against, and justification logic passes it for knowledge stated together with its justification, because the size of the justification pays for the search. That last clause took me a few reads. What I think it means is Cook's certificate from the proof-carrying-code paragraph above: put the justification inside the claim, and the hard part, finding it, was paid before the claim was written. Their price rests on an open bet, and it's a price on looking. A price, of a sort. Steps and a price are both there, then, in their own form. What I still don't find in them is the one I care about: a single act that closes a question for good and is charged as that act. Their steps are steps of reasoning, and their price is on finding a proof. Neither is on settling. I couldn't get past the journal's paywall to Artemov and Kuznets's paper, so my account of it comes from how other people describe it.

Push on mine the same way and it gives. A check is one of Duc's trains of thought, so my act is the last step of his train, where the walking stops. What I've added is a direction and a charge on stopping. And the charge rests on a choice. I put it inside the step, and nothing forced me to.

Three things entitlement is *not*. It isn't permission: an access control system says who may write to what, and that's granted from outside and says nothing about truth; a process with permission to write the note still has no entitlement. It isn't "true, therefore usable": that's the cups. And it isn't "sound by luck": a shortcut that gives the right answer because the inputs happened to cooperate is the stopped clock. The count below has one condition aimed at it, and I'll tell you what the condition says and how much it actually catches.

8.4 What narrowing costs: count the rivals

Now the number, because a book about a bill has to say how much. Here's the idea in a form you can do with three playing cards. Shuffle cards marked 1, 2, 3 face down. Sort them using only questions of the form "is this one bigger than that one?", and count the questions. There are six orders the cards could be in. Each question has two answers, so after one question you've split the six into at most two piles, after two questions at most four, and you need to get down to one. Four is less than six, so two questions can't always do it, and three can, and you can draw the tree of questions on paper and see it: six leaves, three deep. The number of questions you need is at least the base-two logarithm of the number of possibilities you started with, rounded up: that's log2 of 6, which is a bit under 3, rounded to 3. Donald Knuth, a computer scientist at Stanford, gives this, in the volume on sorting of his

The Art of Computer Programming, as the lower bound for all comparison sorting, and the shape of the argument is just: a tree f questions deep has at most 2^f leaves, so it can't tell apart more than 2^f cases.

And then the obvious next question: is the floor always the price? Try five cards. There are 120 orders, log2 of 120 is a bit under 7, so you need at least seven questions, and seven is enough, though not by the obvious method. Finding the way is a good evening with a deck of cards (Knuth has it, under the name merge insertion, from the mathematicians Lester Ford and Selmer Johnson in 1959). Now twelve cards. They can be in 479,001,600 orders, log2 of that is about 28.8, so the count says at least 29. Ford and Johnson's method takes 30, and in 1965 Mark Wells, a mathematician at Los Alamos who built a programming language for exactly this kind of combinatorial search, showed by a computer search that nothing does better: twelve cards need 30. The floor was 29. The price is 30. So the count is a floor and only a floor; it never promises the floor can be reached, and the bound later in this section has to be read exactly the same way.

Ralph Hartley, an electronics researcher at the lab that became Bell Labs, writing in the Bell System Technical Journal in 1928, before Shannon, measured information exactly this way: the logarithm of the number of possibilities, no probabilities, just a count. That's the measure here, and I'll say plainly it's Hartley's. Shannon's would need a distribution over the list, and there isn't one. There's just the list's length.

Here's the same count with something you can hold. Put three coins under three cups. Each coin is heads or tails, so there are eight possible rows under the cups. Someone hands you two answers: heads under the first cup, tails under the second. Lift the first cup and compare. A matching answer leaves four possible rows. Lift the second and compare. A match leaves two. The last coin can still be either way. The list really shrank: eight, four, two. Nothing in that count needed a claim about which way the last coin faces.

This is a parlour game, and an old one. In *A Christmas Carol*, in 1843, Scrooge's nephew Fred plays a game called Yes and No at his Christmas party: he thinks of something, and everybody else asks questions he may only answer yes or no. He's thinking of an animal, a disagreeable, savage one that growls and grunts and lives in London, and the answer is Uncle Scrooge. Most people know the game as twenty questions, and twenty is a good number for it: each yes-or-no answer can at best cut the field in half, and twenty halvings take a million or so possibilities, $2^{20} = 1{,}048{,}576$, down to one. Push on that, though. The halving only happens because Fred answers truthfully about what's actually in his head. Lie once and the count still shrinks; it shrinks around the wrong animal. That's the cups again: the four and the two are real because you lifted the cup and compared. Being told what was under it wouldn't have made them real.

Now put it on a run. Say a run comes with a list of states that are "still on the table," the candidates, and a shorter list, with some ruled out. Call the first the prior and the second the posterior, and I want you to hear those as *a list you could print*. They aren't "the space of possible worlds." Picture the ruling-out as observations, each observation that ruled something out being one branch point in a tree like the card tree. And here's the claim: if that tree is no deeper than the number of paid steps the run took, and the tree has enough leaves to cover the narrowing in a way I'll spell out in a moment, then the counter rose by at least log2 of the prior's length, rounded up, minus log2 of the posterior's length, rounded up. At least as many marks as *index bits* the list lost, where the index bits of a list are the yes/no questions it takes to point at one entry of it, the card count again. That's the bound.

"Paid step" needs its own sentence. It means a step the price list charges at least one for. In a certification system the toll guarantees that every crossing of the reading is one; a price list is free to put a floor under more steps than that (a look, a send, a check), and the count takes every paid step there is. The tree is a bare shape, branches and leaves with nothing written on them, and "no deeper than the number of paid steps" is a count along the run that happened. It isn't a walk down the tree. A paid step doesn't have to have asked any question at all.

Notice, too, what the bound does and doesn't say about the run. It takes the two lists, the tree and the covering as given. It doesn't say the run produced the posterior from the prior; nothing in it reads the lists off the run. What it says is that, given lists, a tree and a covering that someone supplies, the paid steps of the run cover the rounded drop in list length. That's Section 5's warning come due: watching can narrow for free, so the payment can't be dug out of the narrowing. It goes in as a premise, the tree's depth against the paid steps, and I'll keep calling it one. At the end of the section the run makes the posterior itself, from a prior and the tests it passed (Theorem 8.4), and then the premise is a share each test keeps.

8.5 What "narrower" buys: a stronger test

Before the count, one link that's pure logic, and Plato's third account is the whole of it. Take a list of candidates and an observation, a function that says what each candidate looks like from outside. The list's *membership test* accepts an observation when some candidate on the list shows it. Shrink the list, and the test can only get pickier: anything the short list's test accepts, the long list's accepts too. The word for that, and it runs backward from English, is *stronger*: the stronger test is the one that lets *fewer* things through.

Pickier, though, isn't the same as strictly pickier, and the cards show the gap. Take all six orders as the prior, and as the posterior the three where card 1 sits left of card

2: 1-2-3, 1-3-2 and 3-1-2. First observation: "is 1 left of 2?" Every survivor says yes, and the removed 2-1-3 says no, so the prior's test accepts yes and no, the posterior's only yes. Strictly stronger. Second observation, same two lists: "is the first card a 3?" Now 3-1-2 says yes and the other two survivors say no, and among the removed orders 3-2-1 says yes and 2-1-3 says no. Both tests accept yes and no. Same strength, even though three orders are gone. What made the difference is a removed candidate whose observation *no survivor shares*. That's Plato's mark, the thing that sets this case apart from everything it could be confused with, turned into a function. Narrow the list and keep one distinguishing removed candidate, and the test is strictly stronger; narrow it without one, and the test may not have moved at all.

8.6 The count, exactly

Here are the words and the count, for any system with Section 6's counter. A *list* is a finite list of candidates, repeats allowed, and $|\Omega|$ is the length of the list Ω, repeats counted. $\lceil \log_2 x \rceil$ is the base-two logarithm of x rounded up: the smallest $k \in \mathbb{N}$ with $2^k \geq x$. At $x = 0$ it is 0, the convention for empty lists set in Section 5. And $a \dot{-} b$ is subtraction on whole numbers that stops at zero, so $3 \dot{-} 5 = 0$.

Definition 8.2 (Decision tree)**.** A *decision tree* is a leaf, or a branch with two subtrees. Its depth is 0 for a leaf and one more than the deeper subtree for a branch; its leaf count $L(T)$ is 1 for a leaf and the sum of the two counts for a branch. Nothing is written on a tree: no question, no answer, no candidate.

Lemma 8.1 (A tree is no wider than two to its depth)**.** For every decision tree T, $L(T) \leq 2^{\mathrm{depth}(T)}$, and so $\lceil \log_2 L(T) \rceil \leq \mathrm{depth}(T)$.

Proof. Induction on the tree. A leaf has one leaf and depth 0, and $1 \leq 2^0$. A branch with subtrees of depths d_1, d_2 has at most $2^{d_1} + 2^{d_2} \leq 2 \cdot 2^{\max(d_1, d_2)}$ leaves, which is 2 to its own depth. The second half: $L(T) \leq 2^{\mathrm{depth}(T)}$ means $\mathrm{depth}(T)$ is one of the k with $2^k \geq L(T)$, and the rounded-up logarithm is the smallest of them. □

Checked as `decision_tree_leaves_le_pow2_depth` and `decision_tree_log2_up_leaf_bound` (DecisionTreeBound.v). Both are stated over the tree type alone, a leaf or a branch, with nothing about any machine in them.

Theorem 8.2 (Paid narrowing, counted)**.** Take a system with a counter μ and a price list c that obey conservation (Theorem 6.1): each step raises μ by exactly its charge. Let Ω and Ω' be lists with $|\Omega'| > 0$, let T be a decision tree with $|\Omega| \le L(T) \cdot |\Omega'|$ (the tree *covers* the narrowing), and let t be a run from s ending at s' in which at least $\mathrm{depth}(T)$ steps are charged at least one each. Then

$$\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil \;\le\; \mu(s') - \mu(s).$$

Proof. The rounded-up logarithm respects order, so the covering gives $\lceil \log_2 |\Omega| \rceil \le \lceil \log_2 (L(T) \cdot |\Omega'|) \rceil$. It is also subadditive on products of positive numbers, $\lceil \log_2 ab \rceil \le \lceil \log_2 a \rceil + \lceil \log_2 b \rceil$, because $2^{\lceil \log_2 a \rceil} \cdot 2^{\lceil \log_2 b \rceil} \ge ab$. So

$$\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil \;\le\; \lceil \log_2 L(T) \rceil \;\le\; \mathrm{depth}(T),$$

the last step by Lemma 8.1. By hypothesis at least $\mathrm{depth}(T)$ steps of the run were charged at least one, and no step is charged less than zero, so the sum of the charges is at least $\mathrm{depth}(T)$. By conservation that sum is $\mu(s') - \mu(s)$. □

For any system with a counter that obeys conservation it is two checked theorems, put together by hand; nothing in Coq joins them. `nec_e_cs_count_any` (NecEEnt.v) proves it over any certification system with the sum of the charges of a run in place of the counter's rise, and without needing $|\Omega'| > 0$; and `nec_f_conservation` (NecFCounter.v), stated over any counter machine kept by a price list, says that sum is the counter's rise. A system that is both gets the theorem. The bill form is also part of Theorem 8.3 below. The tree half is the checked lemma above; the rest is arithmetic on three whole numbers, the two lengths and the leaf count, and on the counter. Notice what's missing from the hypotheses: anything tying the lists to the run. That's the honest shape of the bound.

Three honest small prints, all of them visible in the statement. Both logarithms round up separately, and the subtraction stops at zero, so when the prior and the posterior round up to the same number of index bits (four candidates narrowed to three, say) the bound says nothing. The lists are plain lists, repeats allowed, so "index bits" is a difference of rounded list lengths with repeats counted; a list can get shorter by dropping a duplicate without losing a single candidate, and "a candidate was ruled out" and "the list got shorter" are two different claims. And the difference isn't the number of questions anyone asked.

The covering condition, $|\Omega| \le L(T) \cdot |\Omega'|$, is the nearest thing the count has to an anti-stopped-clock clause, and I have to be exact about how near. Picture each survivor standing for a little crowd of the prior's candidates, itself included, a crowd no bigger than the tree has leaves. Then the condition says every candidate in the

prior fits in some survivor's crowd. That limits how much can be deleted per leaf you paid for. It doesn't say why anything was deleted. So the condition rules out deleting too much for the tree you've paid for. It doesn't rule out deleting for no reason. A run can satisfy every hypothesis of the count and have ruled nothing out on purpose; the count prices the size of the drop. It says nothing about the honesty of it. And the covering can't just be dropped. On the small machine one yes-or-no question with a big constant in it narrows 2^n candidates to one, n index bits, for a ledger of three, so for any $n \geq 4$ the bound is false without it (Section 16.17).

Here's a side trip worth taking, because it looks like it breaks the count and doesn't. Two information theorists, Rudolf Ahlswede and Gunter Dueck (1989), noticed that asking "is it this one?" is a different job from asking "which one is it?" If each receiver only wants to know whether the message is theirs, a noisy channel used n times can serve a number of receivers that grows like two raised to two to the nC, with C the channel's capacity, where naming the message only gets you two to the nC. That's identification, and on its face it says checking can be exponentially cheaper than naming. But their tests are allowed to say yes wrongly once in a while. Make the yes sound, a test that never says yes to the wrong message, which is the only kind of check this book lets raise a record, and the double exponential collapses. The argument is two lines of pigeonhole. I argue it here, and nothing checks it. Each receiver's test, to be any use, says yes on some output its own message can produce, and soundness means no other message can produce that output; so those outputs are all different, one per receiver, and there can't be more receivers than outputs, $|Y|^n$ for an output alphabet Y. Single exponential, the same order as naming. So for a check that never lies, the card count is the right scale after all, and the cheap trick needs a check that can be fooled.

8.7 What you've got

A paid flag is not a true claim; Section 7 left it at "a paid flag is a paid flag," and Section 9 will say it flatly. What a paid flag with an object and a checked relation behind it gives you is *entitlement*: anyone auditing the run may treat the claim as established, and can point at the event where that became legitimate. (Nothing in the five parts makes the system itself read the flag; I said that above and I'm saying it again because it's the sentence most likely to get lost.) Entitlement costs at least the index bits of narrowing it commits to, given lists, a tree and a covering somebody supplies, and given a run that paid for the tree's depth. The cups say why entitlement and truth are different. Plato says what the checked relation has to do: distinguish. Necula says how to build it. Hartley says what to count.

Put the three pieces on one hypothesis list (narrow the list with a test that separates, find the paid event, count the bill against a tree the run paid for) and you get a single statement, structural entitlement, once over any certification system and once, under the longer name structural entitlement represented, for every Thiele-complete machine, the small machine included (Section 16.17). The first one reads as follows.

Theorem 8.3 (Structural entitlement, over any certification system)**.** Take a certification system (Definition 3.1), a state s that reads no, and a run t from s that ends in a state that reads yes. Take a type of candidates, a type of observations with an exact equality test, an observation obs of each candidate, and a *decoder* d that turns a run into an observation (what its record shows). Take lists Ω and Ω' of candidates and a decision tree T such that: (H1) Ω' strictly narrows Ω: every member of Ω' is in Ω, and some member of Ω isn't in Ω'; (H2) some dropped candidate w, in Ω and not in Ω', shows an observation different from every survivor's; (H3) the decoded run $d(t)$ is shown by some survivor; (H4) $|\Omega'| > 0$ and $|\Omega| \le L(T) \cdot |\Omega'|$; (H5) at least $\mathrm{depth}(T)$ steps of the run are charged at least one each. Then (1) the membership test of Ω' is strictly stronger than the membership test of Ω; (2) some step of the run took the reading from no to yes, and that step was charged at least one; and (3) the sum of the charges of the run's steps is at least $\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil$.

Proof. (1) is the cards' argument: the dropped w shows something no survivor shows, so the posterior test says no where the prior test says yes, and every survivor was in the prior, so nothing the posterior accepts is refused by the prior. (2) The run starts at no and ends at yes, so at some step the reading goes from no to yes, and the toll charges that step at least one. (3) is the proof of Theorem 8.2 with the sum of the charges in place of the counter's rise: the covering and Lemma 8.1 bound the index bits by $\mathrm{depth}(T)$, and the sum of the charges is at least the number of steps charged at least one. On a system that obeys conservation the sum of the charges is the counter's rise. □

Checked as `ent_narrowing_strengthens`, `ent_cs_count` and `ent_cs_entitlement` (EntitlementSmall.v); the tree lemma underneath is also there, as `ent_leaves_le_pow2` and `ent_log2_leaves_le_depth`. Theorem 8.2 is the same count with the counter's rise, so for a system that obeys conservation it's the special case. Three things in the hypotheses feed nothing, and I'm naming them here so you don't have to find them: (H3) isn't used by any of the three conclusions; from (H1) only the inclusion is used, and the gap goes unused; and in (H2) the words "not in Ω'" go unused, since a candidate whose observation differs from every survivor's can't be a survivor itself. And (H4)'s $|\Omega'| > 0$ is used in that proof but can be dropped too: the

covering forces it once the prior has a member (`nec_e_cs_entitlement_stronger`, NecEEnt.v).

A statement like that is true by packaging: it takes the hypotheses in and hands the conclusions back, and the right word for it is a checklist. A checklist for leaning on a claim. It discovers nothing about leaning. The version in which the run itself does the narrowing, Theorem 8.4, comes at the end of this section.

And a checklist isn't nothing. On 30 October 1935, at Wright Field, northeast of Dayton, Ohio, the Army was evaluating Boeing's new four-engine bomber, the Model 299. Shortly after take-off it plunged back onto the field and caught fire; that's how the wire report of that day has it, and the later histories add that it pitched up and stalled first, which is what elevators locked in a raised position would do. Two men died of their injuries, the Army's pilot, Major Ployer Hill, and Boeing's chief test pilot, Leslie Tower. The airplane was fine. The crew had left the gust locks on, a locking mechanism worked from inside the cockpit that holds the control surfaces still while the plane sits on the ground. There's a famous line that a newspaper called it too much airplane for one man to fly. I couldn't find that line in any paper from 1935; the earliest place I could find it is a New Yorker article from December 2007 by Atul Gawande, a surgeon who writes about how people get complicated work right, and he names no newspaper, so take it as a lead. What I can find is what came after. The answer was a card: a list of things to check, in order, every time, and the crews of the airplane worked through written checklists for each phase of flight. The most careful history of it I found is by Walter Boyne, a retired Air Force colonel and a former director of the National Air and Space Museum. By his account it wasn't the first written routine for a pilot; those go back before the First World War. And it's this crash, he says, that made the checklist the way a crew flew. The airplane went on to be the B-17. Nothing on that card was a discovery. Every item on it was something the pilots already knew. What the card did was make sure none of it got left out when it mattered. That's what putting entitlement's hypotheses on one list does: it finds nothing new about leaning on a claim, and it makes the hypotheses impossible to drop quietly.

8.8 When the run does the narrowing

Theorem 8.3 takes the lists in. Here is the version where the run makes the posterior, and the count stops being an item on the list and becomes a consequence. It needs one thing a bare certification system doesn't have: which steps are tests, and what each asks. So a reading names them. For each instruction it gives either nothing or a yes-or-no question about the state the run is in when it gets there, and a test step *passes* when its question says yes there. The candidates are states the run could have

started from. A passed test is asked of a candidate by running the same moves from the candidate up to that step and asking the same question there. The *posterior* is the candidates that pass every test the run passed, asked that way, in the order the run asked them.

Theorem 8.4 (Structural entitlement, with the narrowing made by the run)**.** Take a certification system and a reading that names its test steps, every test step charged at least one. Take a state s that reads no, a run t from s that ends in a state that reads yes, and a list Ω of candidate starts that contains s. Let Ω' be the posterior t makes from Ω, and let b_i and a_i be the numbers of candidates standing just before and just after the ith test t passed. Then (1) Ω' sits inside Ω and contains s; (2) every candidate of Ω that isn't in Ω' fails, asked that way, a test t passed; (3) $|\Omega| \cdot \prod_i a_i = |\Omega'| \cdot \prod_i b_i$; (4) if every test t passed keeps at least a 2^{-k} share of the candidates standing before it, $b_i \leq 2^k a_i$, then $\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil$ is at most k times the number of tests t passed, that number is at most the number of steps charged at least one, and the index bits are at most k times the sum of the charges; and (5) some step of t took the reading from no to yes and was charged at least one.

Proof. (1) The run passes its own tests from s, and a candidate is kept only if it passes them all. (2) A candidate dropped failed one of them. (3) Each test turns a standing list of b_i into one of a_i, and the products telescope. (4) Each test keeps at least $b_i/2^k$, so $|\Omega| \leq 2^{kn}|\Omega'|$ with n the number of tests passed, and the rounded logarithms follow as in Theorem 8.2. Each test passed is a test step, charged at least one. (5) is Theorem 8.3(2). □

Checked as `ent_cs_run_entitlement` (EntitlementSmall.v), with the counting underneath as `ent_stages_product` and `ent_stages_share_bits`. Part (3) is the exact count: take base-two logarithms and $\log_2 |\Omega| - \log_2 |\Omega'|$ is the sum, over the tests the run passed, of $\log_2(b_i/a_i)$, the bits each one took. Part (2) answers the complaint I made about the covering, that it doesn't say why anything was deleted: here everything deleted was deleted by a test the run passed and paid for. Two premises stay, the prior list and the share. The share is needed. On the small machine one CHECK of "$A \geq 15$," asked of sixteen candidates, keeps one, a 2^{-4} share, and narrows by 4 index bits on a run of 3 record moves (Proposition 16.46); with $k = 1$ part (4) would say $4 \leq 3$. The same statement on every Thiele-complete machine, where the tests are the CHECK moves and the bill is the ledger, is Theorem 16.45.

8.9 A grouping, checked and committed

Part I ended on buckets, and Part II dropped them on purpose: the toll prices any yes-or-no reading. This is where the word in this section's title gets attached to a partition in a theorem. Take a pile, a finite list of items, and a *grouping*, a bucket label for each item. Its partition is "same label," and the classes of that relation are a partition in the sense of Definition 2.1. The data is a *key* for each item, and a grouping is *sound* for a key when any two items it puts in one bucket have the same key. The grouping says "these belong together," and the key says whether they do. One grouping *refines* another when every bucket of the first sits inside a bucket of the second.

The *bucket machine* holds a key, a table of checked groupings, a channel, a record, a trap latch and a ledger. SET replaces the key for nothing and throws away the table and the channel, which were checked against the old key. CHECK g costs one and passes when g is sound for the current key, and writes g in the table. COMMIT g costs one and passes when the table holds a grouping with the same partition as g, and puts g in the channel. CERTIFY costs one and passes when the channel holds a grouping, and the record then keeps that grouping with the key it was checked against. A CHECK, COMMIT or CERTIFY that fails traps, and a trapped machine only pays. The *reading for a grouping* g is: the record holds a grouping with g's partition. That's the reading "this grouping has been checked and committed."

Theorem 8.5 (A committed grouping entitles exactly its refinements)**.** (1) The classes of "same label" under any grouping form a partition. (2) Two groupings with the same partition of the pile have the same reading at every state of the bucket machine. (3) For groupings g and h of the pile, soundness of g for a key gives soundness of h for that key, for every key, exactly when h refines g. (4) On every run from a clean state (table and channel empty, record down, trap down), if the reading for g is up, the record holds a grouping and a key for which g is sound, and so is every grouping that refines g; and for every grouping h that doesn't refine g there is a key for which g is sound and h isn't. (5) The bucket machine pays the toll, and a run from a clean state that ends with the record up has raised the ledger by at least three.

Proof. (1) "Same label" is an equivalence relation, so Theorem 2.1 applies. (2) The reading asks only for a grouping with the same partition, and having the same partition is symmetric and transitive. (3) If h refines g, two items h puts together g puts together, and soundness of g gives them the same key. Conversely, take g itself as the key: g is sound for it, so h is, and that says h refines g. (4) Along any run, every grouping in the table is sound for the current key, the one in the channel is too, since it has the partition of one in the table, and the one the record keeps is sound for

the key kept with it; SET empties the table and the channel when the key changes. Then (3), and its converse with g as the key. (5) Only CERTIFY raises the record, and it costs one. A raised record needs a passing CERTIFY, which needs a grouping in the channel, which needs a passing COMMIT, which needs a grouping in the table, which needs a passing CHECK. □

Checked as `pr_grouping_partition`, `pr_reads_structural`, `pr_entitled_iff_refines`, `pr_reading_entitles`, `pr_reading_entitles_only`, `pr_toll` and `pr_reading_costs_three` (PartitionReading.v), all closed.

So "structural" has a theorem under it. The reading sees a grouping only through its partition, which is item (2), and what a raised reading entitles is read off the partition too: every finer grouping, and nothing else. Refinement, the order on partitions, is the entitlement order. It's the three parts of entitlement from the start of this section in their smallest case: the object is the grouping, the checked relation is soundness for the key, and the paid event is the CERTIFY, on top of a CHECK and a COMMIT at one mark each. What it doesn't do is make a later move read the record. Like everything in this section, it says what someone auditing the run may conclude.

Before the machine of Part IV, I owe you the word the toll is named after, the "insight" in "No Free Insight" (Section 5), because I've been using it as if it meant one thing.

9

What insight is

I named the rule in this book "No Free Insight," and the name ran ahead of any careful account of what insight is. The book has been narrowing it ever since: not learning (Section 5), not looking, not copying. It's time to say what's left, and to say it in a way that someone who studies actual insight, in actual heads, wouldn't laugh at. So who are those people, and what did they find? Here's what I ran into, pushed on as I go. Where a story reached me secondhand, I treat it as a lead to check.

9.1 Sultan and the sticks

In 1913 the psychologist Wolfgang Köhler went to Tenerife to run a research station with a colony of chimpanzees, and the First World War then kept him there until 1920, five years longer than planned, with the chimpanzees to watch. The famous one is Sultan. Food is outside the cage, out of reach; two sticks are inside, each too short, but one fits into the end of the other. Sultan tries each stick, fails, sulks, plays with the sticks, and then, by Köhler's account, at a moment you can see, joins them and reaches the food. Köhler's words were *Einsicht* and *einsichtig*, which came into English as "insight," and the whole school of psychology he belonged to, the Gestalt school, said that's what thinking is: a *restructuring*, the moment two sticks stop being two sticks and become one tool. Stacking up tries until one works was, for them, something else. Karl Duncker gave the human version, in German in 1935 and in English in 1945, five years after he died at thirty-seven; he'd spent his last years working with Köhler at Swarthmore. You can set it up yourself: a candle, a box of tacks, matches, and the task of fixing the candle to the wall. Almost nobody sees that the *box* is a shelf, until they do. And then he found that if you hand people the box *empty*, with the tacks poured out beside it, they solve it far more readily. Same objects. What changed was which relation among them was on the table.

I like both of these for the same reason: the insight didn't add an object. Sultan had both sticks the whole time. The box was there the whole time. What arrived was a *relation*, which is to say a piece of structure in Section 2's sense, and the arrival was sudden. Hold that, because the thing the toll prices also arrives in one step, and at its best it's also a relation.

9.2 It isn't a feeling

But here's the finding I'd hand anyone who thinks insight is a feeling, and it comes from someone who measured it. In the 1980s the psychologist Janet Metcalfe asked people to solve problems while rating, every fifteen seconds, how "warm" they felt, how close to the answer. On ordinary problems, algebra for instance, the warmth climbs steadily and peaks at the answer, which is what you'd expect. On *insight* problems, riddles of the candle-box kind, the warmth stays flat, flat, flat, and then the answer arrives, and if anything the people who felt warmest beforehand were *more* likely to be wrong. She put that last finding in the title of a 1986 paper: premonitions of insight predict impending error. The feeling of being close is not a gauge of being close. Try it yourself: write a number from one to seven every fifteen seconds while doing a long sum, then while doing a riddle, and compare the curves.

And the most famous insight story there is fits Metcalfe uncomfortably well. Archimedes, a mathematician who died in 212 BC, asked by Hiero, the king of Syracuse, whether a goldsmith had cheated him on a crown, gets into a bath, sees the water rise, and runs home naked shouting "eureka," I've found it. That comes from Vitruvius, a Roman writer on architecture, writing about two centuries later. Is it true? Galileo Galilei, then a young Italian mathematician, at twenty-two, in 1586, took it as known that Archimedes caught the goldsmith. What he didn't buy was the method. Water spilling over the side, as Vitruvius tells it, was to him "a crude thing, far from scientific precision," and he guessed some writer had set down a rumour. So he wrote a little tract, *La Bilancetta*, proposing a balance weighed in air and in water instead, which he thought was probably what Archimedes actually did, since it rests on Archimedes's own results. Whether there was ever a bath, or a shout, nobody can check now. What survives is a story two centuries late, wrapped around a real man and probably a real problem. Notice what the legend kept: the bath, the shout, the running home naked. The feeling. What it left shaky is the part that would count as the check, the test that could have caught the goldsmith. The bit everybody remembers is the bit that tells you least.

So any account of insight that keys on the sensation, the "aha," is keyed to something that the data says is unreliable. Whatever insight is, it isn't the feeling of having it. That's a relief, because a system of states and moves doesn't have feelings, and a word that only meant a feeling would be cheating if I hung it on one. The version this book can price has to be keyed to an *event*, and the question is which.

9.3 What happens when a computation learns

Here's the split, and I'll say what the toll does with each part, and then I'll say who decided the split. Keep Section 3's sheet of paper in front of you: the NO box, the YES box, the tally line, and the three moves STAY, WORK and VOUCH.

The first thing: *raw observation*. Something comes in and gets written down. A counter ticks. Nothing is claimed about what it means, nothing is marked as resting on it, and the reading doesn't move because of it. Later moves can read what it wrote, but that reading is a move of its own. This is the demon's pad in Section 5, and it's what the statistician Dennis Lindley measured in 1956 as the information provided by an experiment, the words of his title, and defined as the expected drop in your uncertainty: a real quantity, and a quantity about what was *seen*. It can be free. The toll says nothing at all about a move that doesn't cross from NO to YES, so a price list may charge it zero: on the paper, STAY and WORK cost nothing, and Section 5's watcher narrowed a whole row of candidates without paying a mark. "Can be" is the honest word. A price list is allowed to charge for looking anyway, and whether looking costs anything in the physical world is a different question that this sheet of paper doesn't touch.

Lindley's measure has a back story, and it runs through Bletchley Park. I. J. Good, who was Alan Turing's main statistical assistant at Bletchley Park in 1941, wrote in 1979 that Turing had worked out the expected weight of evidence an experiment would give, to judge one of the codebreaking procedures before running it, and so partly anticipated Lindley by about fifteen years, in work that stayed secret. Turing measured evidence in *bans*, logarithms to base ten, with a tenth of one called a deciban, and the name came from the town of Banbury, where tens of thousands of the sheets used in that procedure were printed. It's the same base-ten unit other people call the hartley, after the Hartley from the last section. And Good adds that Turing didn't know the logarithm he was using had already appeared in 1878, in a paper by Charles Sanders Peirce, the same Peirce from the opening chapter, who'd called it the weight of evidence. Peirce is as far back as Good takes it. What it changes: nothing in the argument, but it shows how well trodden this first kind of thing is. For a century and a half careful people have measured what an observation tells you, and every one of those measures is about what was seen. That's the side of the line the toll leaves alone.

The second thing: *certified insight*. Some move turns the reading from NO to YES. Here is where the toll lives. In any system with the five parts that obeys the toll, that move costs at least one mark, and every run that starts at NO and ends at YES pays at least one somewhere along the way, which is Section 3's universal floor. What's being charged for, at its best, is exactly the Gestalt thing: a relation, committed to.

One tool, claimed, out of two sticks. "At its best," because Section 3 was careful about what YES means: the flag is up, and nothing more. A crossing that writes down which relation it's committing to carries Sultan's tool with it. A bare crossing, VOUCH on the paper, commits to no relation at all, and it pays exactly the same.

The third thing isn't a kind of event. It's an observation about the price: it's *flat*, whatever is certified. Say the YES state also shows something striking, a deep relation, a result nobody could have guessed, Sultan's two sticks become one tool. The toll charges the crossing at least one. It doesn't charge the striking crossing more, because the floor's proof never looks at anything but the reading before and after. On the paper: VOUCH from NO makes one mark whether you were thinking of Duncker's box or of lunch. Meaning is not cost. The depth of what's certified adds meaning. It doesn't add a mark.

The candle box has a footnote that says the same thing from the other side. In 1962 the psychologist Sam Glucksberg ran Duncker's problem with money on it: some people were told they could win five or twenty dollars depending on how fast they solved it. With the tacks in the box, the people chasing the money did worse than the people who weren't. With the box empty the problem was easier for everybody, and the accounts I've read don't agree on what the money did there, so I'll leave that half alone. Paying more didn't buy the insight, and on the hard version it got in the way. That's a fact about heads. It says nothing about records, and I won't push it further than that. But it's a nice mirror for the paper. On the paper a deeper crossing isn't charged more, and in Glucksberg's room a bigger prize didn't bring the crossing sooner. In neither place does the price measure the depth.

Here they are, exactly, for any certification system (Definition 3.1). Runs are written as lists, the way Section 3's proof wrote them: $[\,]$ is the empty run, and $i :: t$ is the run that starts with move i and carries on with the run t.

Definition 9.1 (Certified insight event, raw observation)**.** In a certification system, a step from state s by move i is a *certified insight event* when it turns the reading on: $\mathrm{cert}(s) = \text{no}$ and $\mathrm{cert}(\mathrm{step}(s, i)) = \text{yes}$. A move i is *raw observation* when no step by it, from any state, is a certified insight event.

Remark 9.1 (What the toll prices, and what it doesn't)**.** Read off Definition 3.1 and Observation 3.1, and checked on the board as a certification system (`nec_f_board_raw_free`, `nec_f_board_floor` and `nec_f_board_floor_flat`, NecFPaperToys.v). (1) The toll puts no bound on the cost of a raw-observation move; on Section 3's board STAY and WORK are raw observation and cost 0. (2) Every certified insight event is charged at least 1, by the toll, and every run from a state reading no to one reading yes has total cost at least 1, by the universal floor. (3) The price is flat: the floor's

proof uses nothing about the end state but its reading, so adding any hypothesis about what else the end state shows leaves the bound at 1 and cannot raise it. A system that wants to charge more for a deeper certification has to write that into its price list, move by move; the toll doesn't do it for anyone.

So the account comes down to two facts and a warning. Looking can be free. Turning the reading on can't. And putting something remarkable behind the reading doesn't make it cost any more.

Who decided the split? Not the costs alone. The toll prices exactly one thing, the crossing. A price list is free to put a floor under more moves than that (a look, a send, a check) and free to leave other moves at zero, so the sorting into raw observation and certified insight is by whether the reading moves, and that sorting is this book's. The part that's *forced* is the floor under the crossing, and Section 4 is the argument for it; anything a price list charges beyond that is a choice, and any concrete machine owes you its whole table so you can see which choices it made.

9.4 Test, then commit, and nothing forces the second to follow the first

The philosopher Karl Popper said a theory earns its standing by being *exposed*: you make a conjecture, you try to kill it, and what survives a real attempt counts for something, where "real" means the test could have failed. His word for that earned credit is *corroboration*. I've leaned on that idea through this whole book, and here's where a certification system has a version of it, and here's where the version is weaker than Popper and I have to say so.

Add a fourth move to the paper: CHECK. It looks at something (a sum, a coin under a cup, a formula against a row of values), and it can come out pass or fail, and it doesn't move the box. Charge it whatever you like; charging it on the attempt, pass or fail, fits Popper, because what made his test count was that it was really run and could have failed. That's the test. It's raw observation in the sense above, because it never turns the reading on. VOUCH is the commit. So the honest protocol is two moves: test, then commit, and the costs are separate, one for the checking work and one for the crossing.

But nothing in the five parts makes the second move wait for the first. The step is handed the current state and a move name; the toll looks only at whether the reading crossed. Play VOUCH from NO with no CHECK anywhere before it, and you land in YES and make one mark, the same box and the same mark as the player who checked first. The two tally lines differ by exactly what the CHECK was charged, and if you charged it nothing they don't differ at all. The mark for the crossing is the

same mark either way. The receipt says a commitment was paid for. It does not say it was earned. Popper's corroboration needs the test; the toll doesn't.

You can build a system that makes the commit wait. Let CHECK leave a mark in the state when it passes, and write the step so VOUCH only crosses when that mark is there. That's a design, written into the step function, and it's a perfectly good one. Nothing in the five parts or the toll asks for it. That's a real gap between the word and the bare system, and the only honest thing is to leave it visible: the toll prices the commit, and whether the commit was preceded by a test is a fact about the run, or about a design somebody chose. The five parts don't guarantee it.

Remark 9.2 (Nothing makes the crossing wait)**.** Read off Definition 3.1, and checked on the board as `nec_f_board_no_wait` (NecFPaperToys.v). The toll constrains $\mathrm{cost}(i)$ for moves that cross, and nothing else; the step function is any function of the current state and the move. So a certification system may have a move that crosses from a state no check has touched, and the toll still holds. On Section 3's board extended with CHECK, the run [VOUCH] from NO crosses with one mark, and so does [CHECK, VOUCH]. Making a crossing depend on an earlier check is a condition on the step function, which a particular system can impose and the definition does not.

9.5 What the word means

Insight, in the sense this book prices, is a relation committed to, in a move that flips a reading from off to on, and that move costs at least one mark, in any system with the toll, for any relation, whatever its depth. (And one edge from the paper: a bare crossing that commits to nothing pays the same mark, so "a relation committed to" is what the crossing is at its best. The toll doesn't guarantee it.) Three boundaries, each from a different place. From Metcalfe: it isn't a feeling, and a system's version has to be an event. From the proof: the price is flat; a shallow commit and a deep one both cost at least one, and cost is not depth. A system can pay heavily to check a trivial claim, and Duncker's box, a real insight, is still one small commit. And from the two-move protocol: a paid flag proves nothing on its own. Price and correctness are separate books.

Which leaves a suspicion I'd have in your place. Everything so far has been one reading on one sheet of paper, with a rule I chose and a count I bolted on. Is the shape of that record something I picked, or is it what any base-driven record has to look like, on any machine at all? That's the next section: take *any* machine that steps, bolt a record onto it, and see what shape it's forced into. How a concrete machine hangs actual claims on its parts, a checked property of one counter at one edition, and where those sit in the state, comes in Part IV.

Records over any base: the latch

For the whole book so far I've been building up a thing with a flag and a counter, and a fair reader is entitled to suspect that the whole apparatus is special pleading for one machine, the one I keep promising to build. So here's the section that answers the suspicion with no particular machine in it at all, and sees what's left. Take *any* machine that steps deterministically: a Turing machine, a register machine (a few numbered boxes that each hold a whole number, and a program that adds, subtracts and jumps), a lambda calculus (Alonzo Church's way of computing by nothing but plugging expressions into expressions; Section 13 builds one), anything. Call it the base. Now bolt a record onto it, under rules I'll state, and ask: what can the record be?

The answer is one shape, and I'll show you the shape first because it's a thing you can build from a cup and a die.

10.1 A cup and a die

Put a cup upside down on the table. The rule: the first time a coin lands in front of it, turn it right side up. There is no rule for turning it back over. Now roll a die, and put a coin down on every six. Roll twenty times. The cup turns over on the first six and stays turned over. Write down which roll did it. That number, the roll that turned the cup, is where the record happened, and the cup afterward is the record.

That's a *latch*. Electrical engineers have had the circuit since 1918, when two British physicists teaching engineering in London, William Eccles, a radio pioneer, and Frank Jordan, applied for a patent on a two-valve arrangement that holds whichever state you last put it in; it became the flip-flop, and the registers and caches in the device you're reading this on descend from it. Cut its reset wire and what's left has one input, "set," and the rule *new state = old state or set*: once set, it stays set. The cup is that circuit: a flip-flop with its reset wire cut off.

(Eccles and Jordan didn't call it a flip-flop. Their 1919 paper calls it a *trigger relay*. Where the newer, sillier name came from I can't source, so I'll leave it at that.)

Is a flip-flop with its reset wire cut off only a picture, or did anybody ever build one on purpose? Somebody did, and for a reason that's very close to this book's. In the mid-1950s Wen Tsing Chow, an engineer at the Arma division of the American Bosch Arma Corporation in Garden City, New York, was asked by the United States Air Force, in the accounts of his career I could find, for a more flexible and secure way to hold the fixed numbers, the targeting constants, inside the guidance computer of the Atlas missile. What he and his Arma colleague William Henrich patented (filed in 1957, granted in 1962, under the plain title "Storage matrix") was a grid with a connection at every crossing, where you set the numbers by taking out the connections you didn't want, physically or by burning them out with too much current. The word *burn* is right there in the patent, and people still say they burn a chip. The descendants are called programmable read-only memories, PROMs, and a fuse blown in one stays blown: there's no current you can send to grow it back. A whole memory built out of cups with no reset wire, because somebody wanted numbers nobody could quietly change later. Does that prove anything here? No. The theorem below doesn't care what a cup is made of. But it's good to know the shape wasn't invented for a sheet of paper. Somebody needed it, for stakes a good deal higher than a die.

Now the claim, stated loosely and then exactly: *every base-driven record, over any base, is a cup with no reset wire, turned by some event the base passes through.*

10.2 What "base-driven" has to mean

Here's where the content is, and I'll give you the five rules, because the theorem is only as strong as they are.

A machine *extends* a base when there's a map from the bigger machine's states down to the base's, and that map starts in the base's start states, reaches every one of them, moves in step with the base (one step up top is one step down below, through the map), and agrees about halting. The bigger machine may carry as much hidden state as it likes beyond the base. That's the "any base" part: the base doesn't get to see the extension, and the extension doesn't get to change what the base does.

The extension carries a *record*, a yes/no reading, and a *ledger*, a counter. It's a *base-driven record* when five things hold. One: the record's next value depends only on the base's current state and the record's current value. (I'll call this *driven by the computation*. It says the record answers to what the machine is doing. A clock on the wall or someone's hand doesn't get a say.) Two: the ledger never goes down (Section 6). Three: the step that turns the record on costs at least one; that's the toll (Section 3). Four: once on, the record stays on; that's permanence (Section 4). Distributed-systems people have a name for a predicate like that. The computer

scientists K. Mani Chandy and Leslie Lamport, in their 1985 paper on taking a consistent snapshot of a distributed system, called a property *stable* when once it holds it holds forever after, and stable properties are the ones a snapshot can be trusted to detect. Five: some run from a start state actually reaches a step that turns the record on; otherwise the record is decoration.

Here are the words exactly. On/off is true/false, $\vee$ is "or" (on when either side is on), and next^k is k steps.

Definition 10.1 (Base, record-carrying machine, cover)**.** A *base* B is a set of states with a step function next_B, a set of starting states, and a halting predicate. A *record-carrying machine* M is a set of states with a step function next_M, a set of starting states, a *record* rec from states to Booleans, a *ledger* μ from states to whole numbers, and a halting predicate. The cost of a step is $\text{cost}(m) = \mu(\text{next}_M(m)) \dot{-} \mu(m)$. A *cover* of B by M is a map π from states of M to states of B that sends starting states to starting states, has every starting state of B as the image of some starting state of M, commutes with the steps, $\pi(\text{next}_M(m)) = \text{next}_B(\pi(m))$, and agrees about halting: m halts exactly when $\pi(m)$ does.

Definition 10.2 (Base-driven record, latch)**.** M with cover π is a *base-driven record* over B when (R1, driven by the computation) there is one function f such that, for every state m, $\text{rec}(\text{next}_M(m)) = f(\pi(m), \text{rec}(m))$; for every state m, (R2, ledger) $\mu(m) \leq \mu(\text{next}_M(m))$; (R3, toll) if $\text{rec}(m)$ is off and $\text{rec}(\text{next}_M(m))$ is on, then $\text{cost}(m) \geq 1$; (R4, permanence) if $\text{rec}(m)$ is on, so is $\text{rec}(\text{next}_M(m))$; and (R5, a reachable write) some starting state m_0 and some k have $\text{rec}(\text{next}_M^k(m_0))$ off and $\text{rec}(\text{next}_M^{k+1}(m_0))$ on. An *event* is a function h from base states to Booleans, and its *latch* is the step $(b, r) \mapsto (\text{next}_B(b),\ r \vee h(b))$. *$M$ factors as the latch of h* when, for every state m (reachable or not),

$$(\pi(\text{next}_M(m)),\ \text{rec}(\text{next}_M(m))) = (\text{next}_B(\pi(m)),\ \text{rec}(m) \vee h(\pi(m))).$$

Now the theorem. Under those rules, the base state and the record move *exactly* as a latch: the record switches on at the first base state where some event holds, and never off, and the event is read off the extension's own rule by asking it what it does when the record is off (stated exactly below). That's a statement about what the base and the record do together. The hidden state the extension carries can't push the record anywhere (R1) doesn't allow, since the record's next value is a function of the base state and the record's own value. The theorem doesn't say the whole extension is a bare latch machine, and it doesn't make hidden state show up anywhere in particular: an extension can carry a field that touches neither the record nor the

ledger. And the event is pinned down only on base states that some state with the record off covers; on a base state with no such state above it, $f(b, \text{off})$ is whatever (R1)'s function says, and nothing in a run ever consults it.

I promised the content was in the definition, and here's what that means: the proof is a short case split on whether the record is already on, and it uses only *two* of the five rules, "driven by the computation" and "permanent." The ledger, the toll, and the reachable-write clauses play no part in it. A latch is what you get from "answers only to the base" plus "never comes back down," full stop. The other three rules matter for the *converse*: for any base and any event the base can reach, the latch of that event, priced at one unit per write, *is* a base-driven record. Put the two together and you've got: base-driven records are exactly latches. One shape, no others, over any base.

Observation 10.1 (The record is a latch)**.** Let B be any base, π a cover of B by M, and suppose M's record is driven by the computation (R1), through a function f, and permanent (R4). Then M factors as the latch of the event $h(b) = f(b, \text{off})$. In particular every base-driven record factors that way; (R2), (R3) and (R5) are not hypotheses.

Proof. The base half of the equation is the cover's step clause. For the record, split on $\text{rec}(m)$. If it is on, permanence (R4) makes the next value on, and on $\vee\, h(\pi(m))$ is on. If it is off, (R1) makes the next value $f(\pi(m), \text{off}) = h(\pi(m)) = \text{off} \vee h(\pi(m))$. Only (R1), (R4) and the cover are used; (R2), (R3) and (R5) are discarded. □

Checked as `record_axis_is_latch_r1_r4` (StructuralClockSync.v), from (R1) and (R4) alone, and as `record_axis_is_latch_holds` (StructuralRecordAxis.v) for every base-driven record, with the definitions `BaseMachine`, `BaseCover`, `computation_driven`, `HonestBaseExtension` (the Coq name for a base-driven record; in this book "honest" is kept for checkers) and `latch_factorization` (StructuralCoreAnyBase.v), and the clauses (R2) to (R5) as `ledger_carried`, `rc_a2` (StructuralCore.v), `record_permanent` and `reachable_record_write` (StructuralCoreCover.v).

The latch's nearest neighbours are in verification, and they're old. The computer scientists Martín Abadi and Leslie Lamport, then at Digital's research lab in Palo Alto, added *history variables* to specifications in 1991: extra state that records what has happened and never steers the system, used to prove that one specification implements another. A history variable that only ever turns on is the cup. Temporal logic has the same record as an operator. In the past-time temporal logic that Orna Lichtenstein, Amir Pnueli and Lenore Zuck set out in 1985, "once h" holds now exactly when h has held at some point so far, and a runtime monitor keeps it in one

bit with the update $r' = r \vee h$. Klaus Havelund and Grigore Roşu, two computer scientists working on runtime verification, synthesized monitors of exactly that kind for safety properties written in that logic in 2002. That update is the latch of Definition 10.2, word for word. What the observation adds is the other direction: a record driven by the computation that never comes down can't be anything else.

One thing about that proof deserves a second look, because it cuts both ways. It never asks what the base is. The cover decides that: whatever the map sends down is base, and the record's rule is allowed to read all of it; whatever stays up top is extension, and (R1) keeps the record's rule from reading it. Draw the line low, so the base is only the computation's own working state, and (R1) is a real demand. Draw it high, put every table and every refusal bit into the base, and (R1) is easy to meet, and the latch shape comes out exactly the same. So the theorem holds wherever you draw the line, and that's exactly why it can't tell you where a record's meaning lives. The meaning lives in what the event h reads, and that's in the base by construction. The pieces above the line, the record and the ledger, are the ones that never steer anything; the pieces that make a flag mean something sit below it, where this theorem doesn't look. So when the machine in Part IV arrives, ask it first which parts it puts above the line.

Theorem 10.2 (Every reachable latch is a base-driven record)**.** Let B be any base and h any event. Let Latch_h have states (b, r, x), starting states $(b, \mathrm{off}, 0)$ with b starting in B, step $(b, r, x) \mapsto (\mathrm{next}_B(b),\ r \vee h(b),\ x + \delta)$ with $\delta = 1$ when r is off and $h(b)$ is on and $\delta = 0$ otherwise, record r, ledger x, halting as b halts, and cover $(b, r, x) \mapsto b$. Suppose some starting b_0 and some k bring Latch_h from $(b_0, \mathrm{off}, 0)$, in k steps, to a state whose record is still off and whose base state satisfies h. Then Latch_h is a base-driven record over B.

Proof. (R1) holds with $f(b, r) = r \vee h(b)$. (R2): each step adds 0 or 1. (R3): a step that turns the record on has r off and $h(b)$ on, so it adds 1. (R4): on $\vee\, h(b)$ is on. (R5) is the supposition. The supposition asks for the first time h holds on some run; any run from a start that reaches an h-state has such a first time, though the checked statement uses only the first-time form. □

Checked as `latch_core_honest` (RecordAxisDiscrimination.v).

A reversible base needs one more thing to carry a record, and the next theorem says exactly what. If the base never merges two states, keep every earlier record value in a growing history beside the latch, and the whole step merges nothing either; but on a finite state space a permanent record can't be written by a step that merges

nothing at all. A step that merges nothing is called *injective*: no two states go to the same next state. And $r :: H$ is the history H with r put on the front, as in Section 9.

Theorem 10.3 (Records on reversible machines)**.** (1) Let B be a base whose step is injective. The machine with states (b, r, H, x), step $(b, r, H, x) \mapsto (\mathrm{next}_B(b),\ r \vee h(b),\ r :: H,\ x + \delta)$ (δ as in Theorem 10.2), starting states with r off, H empty and $x = 0$, has an injective step, and under the supposition of Theorem 10.2 it is a base-driven record over B. (2) On a finite state space, a single injective step with a permanent reading never turns that reading from off to on.

Proof. (1) Equal images have equal base parts, so equal base states by injectivity; equal histories $r :: H$, so equal records and histories; and equal ledgers once the equal δ is subtracted. That it is base-driven is the proof of Theorem 10.2 with the history carried along. (2) is Theorem 4.1 of Section 4 with a single move. □

Checked as `history_latch_injective`, `history_latch_honest` and `finite_reversible_cannot_write` (RecordAxisDiscrimination.v), the last from `permanent_flip_is_not_injective` (PermanentCertification.v).

So a base that never merges can still carry a record, if it's allowed to grow a history that never ends. A machine with finitely many states can't carry one without merging: on a finite machine the record's first write is a merge, and Section 4 already put the price on that.

Process-calculus people ran into the same thing from the other side. Vincent Danos and Jean Krivine, two theoretical computer scientists in Paris, wanted to pin down what a transaction is. They took in 2005 a reversible version of CCS, Robin Milner's calculus of processes that talk to each other, where every step can be rolled back, and modelled a transaction as a process that wanders freely through everything that can be undone until it performs an action declared irreversible, which is the commit. The way I read it, they start from steps that all come undone and ask what has to be added before anything sticks, and the theorem above starts from a record and asks what the step under it must be. In their calculus, as in the theorem, a permanent effect needs a step with no way back, and the calculus has to say which steps those are. The word I tripped on was *declared*. Theirs are labelled. Mine, I thought, are picked out by counting, since on a finite machine the first write merges whatever anyone calls it. Push on that and it gives. I choose the event h, and on a base that grows its history forever the write merges nothing (Theorem 10.3), so there my permanence is declared too.

There's an arrow of time in that theorem, and I should say where it came from, because I put it there by hand. Every starting state has the record off. That's the only

reason a raised record means the event happened: it was down at the start, and only h lifts it. Turn the cup up before the first roll and it tells you nothing about the die. So the clean start is an assumed ready state, a past hypothesis in miniature. The past hypothesis is the physicists' name for an assumed low-entropy beginning of the universe, which several of the accounts below lean records on. Whether every trusted record leans on it is their argument, and I'm not settling it. Nothing in this book derives one; every theorem here about records assumes its small version and starts from there.

When I went looking for who else had found the arrow hiding in the start, I got five papers that read like five unrelated answers. They lined up once I asked each one the same thing: what does the cup being down at the start rest on? There are two places to put it. You can put it at the beginning, as a fact about how things started. Or you can put it in the structure of what causes what. David Wolpert, then a physicist at Los Alamos and the same Wolpert as the free-lunch theorem, sorted memory systems in 1992, and one of his kinds works only because it begins in a prepared state set up by a process that runs one way, which is how it leans on the second law. That's the cup put down by somebody before the game, at the first place. Wolpert and Jens Kipper, a philosopher at Rochester, put in 2024 that sorting on a formal footing, and they say plainly that they have no formal proof this kind can only remember the past. I trust them more for that sentence. What I think they're saying is that a prepared start buys you a memory, and getting its direction too takes something they can't yet prove.

Aram Ebtekar, an independent researcher in Vancouver, and Marcus Hutter, who works on the theory of general artificial intelligence at Google DeepMind and the Australian National University, built in 2024 a reversible toy world, imposed a past hypothesis on it, coarse-grained it, and got records of the past and none of the future. As I read their route, the clean start doesn't go straight to the records. It goes through a causal structure first, and the arrows come out of that. Thomas Blanchard, a philosopher of science at Cologne, got in 2024 the same lopsidedness of knowledge out of the causal Markov condition, a condition on causal structure. So Blanchard starts at the second place, which is the middle of Ebtekar and Hutter's road. That took me a while to see, because one is a toy world and the other is philosophy of causation. They go from the start to causes to records; he goes from causes to records. The fit is my reading, and it's mine to defend. And Carlo Rovelli, two years before his paper on the cost of choosing from the opening chapter, studied traces in "Memory and entropy" (2022) and concluded that a memory turns low entropy into available information. He's at the first place, and he says what the beginning gets spent on. Read against the latch, that's the blank the record uses up: the flag can say yes once because it started at no.

So does the latch explain why records point backwards? No. I'll say that plainly, so it doesn't look like it does. The step runs forwards and the runs start clean, and a theorem built on those two can only hand the arrow back. Push further and it gives in a second place. The history that grows forever looked like a way around the clean start. It starts empty, so it leans on the same blank. Under a step that forgets nothing, on a finite set, no permanent record can ever be written, past or future, which is Zermelo again. On the map the book sits at the first place, with Wolpert and Rovelli, and it sits there by assumption. Ebtekar and Hutter at least imposed their start on a world with its own dynamics and watched what came out. I wrote mine into the definition.

Constructor theory, which Section 4 brings in as a neighbour of the chairs, has a paper on time (Deutsch and Marletto, 2025) saying that laws stated as which transformations a device can and can't bring about can't mention time at all, so duration has to come from relations between physical systems, clocks, that have their own dynamics. What I think they're saying is that whenever you want a duration you point at a second system and count what it did. That's the position this book is in by construction. The step count is the only time in it, and it comes from the machine's own steps. The next subsection has a record turned at the fifth tick of a hidden clock, and (R1) throws it out, which read beside them is a rule about which system may be the clock: the one being recorded. Push on it, though. Their laws have no step in them, and the paper's work is getting duration and dynamics out of clocks and timers. I put a step, next_B, into the definition of a base. So I'm in their position only at the far end of their argument, where a clock already exists. The base is my clock, and I assumed it.

Physicists have built this latch on paper too, in quantum mechanics, and it behaves the way the last theorem says. The question that sorted them for me is where the cup puts what it forgets, since a closed quantum system with separate energy levels comes back (Bocchieri and Loinger, earlier) and the theorem left only two ways out: a history that never ends, or a merge. Klaus Hepp, a mathematical physicist at ETH Zurich, wanted a measuring device inside quantum mechanics itself, and studied in 1972 a particle that flips spins in a long chain as it passes, now called the Coleman-Hepp model (I know Hepp's paper only from accounts of it), and John Bell (1975) took it apart: the record is exactly definite only for an infinitely long chain, which is the reversible escape above, a history that never ends, in quantum clothes. Jonathan Halliwell, a theoretical physicist at Imperial College London, used in 1999 a two-state detector that flips once when a particle crosses a point and is kept from flipping back by a large environment, which is a latch with the environment holding the history. His abstract calls it *effectively* irreversible, and I like that he puts the word *effectively* right there where you can see it. And Murray Gell-Mann, the physicist who

named the quark, and James Hartle, of the University of California at Santa Barbara, gave in 1995 the property a name inside their framework of decoherent histories: the permanence of the past, past alternatives that stay settled as the story is extended into the future. They name (R4) for whole histories and leave the cup to others. So Hepp's chain, as Bell read it, is exact and infinite, and Halliwell's detector is finite and only effectively permanent. Mine is finite, exact and permanent at once, and it gets there by letting the first write merge. Push on that and it gives: I never say where the merged state goes. Hepp's chain had to be infinite. Halliwell had to put his in an environment. Mine just goes, and the toll writes a unit in a counter. That's fine in a model of computation, and it's exactly where this stops being physics.

10.3 Both rules do work: the toggle and the clock

A theorem with two hypotheses should have two counterexamples, one for each hypothesis dropped, and it does, and they're the two ways a record goes wrong in the world.

Drop permanence. Take a base that just counts, and a record that flips at every step: on, off, on, off. It's driven by the computation; it's not permanent; and it's not a latch of any event. This is a *toggle*, a piece of working state. A record you can revoke is working state wearing a record's clothes, and if a revocation has to stick, that's a *second* latch ("revoked"), which is how certificate revocation lists actually work: you add an entry; you never delete the certificate. Alin Tomescu, a computer scientist then at MIT, and his coauthors (2019) built append-only authenticated sets and dictionaries for transparency logs, and revocation fits that shape exactly: a revocation goes in once and stays. The pair theorem (stated below) covers two latches that can each read the other; do it with two cups where the second only turns over on an even roll *if the first is already up*, and you'll see the second can never turn before the first.

Dialogue theorists built a version of that two-latch rule for arguments. Give each speaker a row of cups, one for every claim they've made. Everybody I read in this field is answering two questions about those cups: whose hand turns them, and can they come back down. C. L. Hamblin, an Australian philosopher and logician who also came up with reverse Polish notation for early computers, wrote *Fallacies* (1970), and it has a chapter on formal dialectic where each speaker has a *commitment store*, a list of what they're on the hook for, and the rules of the dialogue update it. The speaker doesn't. I know that chapter from other people's accounts of it. That's an answer to the first question, and it's (R1) for arguments: the cup answers to what happened in the dialogue, and the speaker's hand is kept off it. Henry Prakken, who works on AI and law at Utrecht, built in 2005 dialogue games for argumentation on

stores like that and made commitments sticky: a claim can be retracted only when it has been challenged, and claims and backtracking carry a price. He also floats a convention that once a player has retracted a proposition, that player can't take it on again, which is a revocation that stays revoked, the way an entry on a revocation list does. That's an answer to the second question, and it took me a second pass to see that it's two answers. A cup can come down, and coming down costs. And once it has, the taking-back is itself a cup that stays up. Erik Krabbe, an argumentation theorist at Groningen, names in 2001 the awkward part of that: retraction is frowned on, and yet resolving a dispute takes somebody retracting a doubt or a point of view, so the rules have to say when taking a commitment back is allowed. As I read him, he's standing between the two ends of the second question. A store that never gives can't settle a dispute. A store that gives freely isn't a store. The part of all that I'd keep is Hamblin's, that the store belongs to the rules. It doesn't belong to the speaker. Take the pen away from the program. And this book answers Krabbe with a price: a take-back is a second cup, "revoked," and turning it costs at least one like any other write, so a retraction is allowed, permanent and paid for. That's Prakken's convention with the toll attached. Where mine gives is his first condition. He lets a claim come down only when it's been challenged, and nothing in my latch asks why the second cup turned. The pair theorem lets the second cup's event read the base and the first cup, and nothing in it requires a challenge to be there.

The same field has a notion that looks a lot like earning. In the abstract argumentation of Phan Minh Dung, a computer scientist at the Asian Institute of Technology in Bangkok (1995), arguments attack one another, and the *grounded* semantics starts from the arguments nobody attacks and keeps adding the ones they defend. Look at how it's built: things go in and never come out, which is a row of cups turned one at a time. Pietro Baroni and Massimiliano Giacomin, at Brescia, introduced in 2007, and Martin Caminada, then at Aberdeen, revisited in 2014, *strong admissibility*: a set is strongly admissible when every argument in it is defended by others in the set that don't lean back on it, so the support bottoms out. That's the argumentation form of an earned claim. Above the bottom, nothing in it is in by default, which puts it at the far end from Burge's default on the testimony map in Section 7. Caminada argues it plays a vital part in the dialogue games that decide grounded acceptance. I've read Caminada's paper; Dung's and Baroni and Giacomin's I know through what later papers say about them. Notice the bottom, though. An argument nobody attacks gets in on a check that can't fail, and this book says a check that can't fail establishes nothing. Now turn that on my own cups, because it lands. My first write needs only that h holds, and what h reads sits in the base, where the latch theorem doesn't look. I charge a unit for it. A unit is a price. It isn't a check. So my earned records bottom out the way Dung's do, in something let in without an argument, and the one

difference is that mine pays to get in. Whether a machine's earned record certifies exactly the strongly admissible sets is the kind of statement I'd have to prove before I said it, and I haven't.

A record that's allowed to change its mind isn't lawless, though. It's somebody else's subject. Count how many times the cup may be turned. Once, from down to up, is the latch. More than once and you're somewhere else, and the people who went there sort it two ways: by how many turns are allowed, and by how long the game runs. The philosopher Hilary Putnam, then at MIT, and E. Mark Gold, a computer scientist at a research lab in Santa Monica, in back-to-back papers in 1965, looked at questions a machine answers by guessing and revising, where the answer is whatever the guesses finally settle on. Putnam's paper says outright that you're never sure you have the correct answer. That stopped me the first time. What kind of answer can't you ever be sure you have? The way I came to read it: the cup's position at any moment is the current guess, the answer is where it ends up, and nobody watching can tell when it's done moving. Yuri Ershov, a logician in Novosibirsk, sorted those in 1968 by how many revisions they're allowed (that's how his hierarchy is described; I haven't read his paper). Allow one, from no to yes, and you get exactly the sets a program can list. That level I did check, in a modern account of the hierarchy by Rod Downey, Carl Jockusch, Timothy McNicholl and Paul Schupp, four computability theorists, and it's a short exercise once you see it. That's what a permanent record raised by a search catches. Allow two, three, and so on, and every extra change buys new sets: the rungs of the ladder are all different. A record that can come back down lives on that ladder. Now put Putnam's worry on my cup, because it lands. On the bottom rung a yes is final the moment it appears, and a no never is. Down means not yet. A single cup of mine can certify that something happened and can never certify that it didn't; a final no would take a second cup, turned by finding a reason, and that's the pair of latches again. That's what the bottom rung costs, and I'd rather name it. This book stays on the bottom rung. I'm pointing at the ladder. I'm not climbing it.

The ladder doesn't stop at finite time, and the people who followed it past infinity have names for the two kinds of record I just described. A definition that only ever adds, stage after stage, and keeps everything it has at each limit, is an *inductive definition* (the long treatment is a 1974 chapter by Wayne Richter and Peter Aczel, two recursion theorists). That's the latch run through the infinite, a cup that goes up at some stage and is still up at every stage after. Let it take things back, and keep what stays put from some stage on, and you have what John Burgess, a philosopher and logician at Princeton, called in 1986 *quasi-inductive* definitions, which he met in the revision theories of truth that grew up beside the logician Saul Kripke's 1975 theory of truth, which builds up the true sentences in stages. Burgess's paper measures

how complicated the truths of those theories get. I've read its abstract. The place where he names them I haven't. What took me a few reads is that Burgess's rule at a limit is Putnam and Gold's rule, keep whatever the guesses settled on, applied at every limit stage along the way. That's my reading of how the two fit. If it's right, the split between a cup that only goes up and a cup that settles is the same split at finite time and past it. A permanent record is the first kind. A toggle that's allowed to settle is the second. Joel Hamkins and Andy Lewis, two set theorists, let a Turing machine keep running through the ordinals in 2000. At a limit a cell keeps its value if it settled, and reads 1 if it kept flipping back and forth forever, and the trick they use again and again is the toggle: flash a flag on and off every time a guess changes, and at the limit the flag is on exactly when the guess changed infinitely often. A flag that can come back down is how that machine learns what no finite run could tell it. A permanent one can't do that trick, because at a limit a cell that only ever goes up just stays up. So past infinity the toggle is the clever one. A latch tells you a thing happened at least once. Their flashing flag tells you it happened infinitely often. I threw the toggle out a few paragraphs back anyway, and I'd do it again, because before the limit its reading can't be relied on, and nobody using a machine of mine gets to wait for a limit. One finite-time fact belongs beside it, so nobody concludes too much from permanence. Hao Wang, a logician then at Oxford, described in 1957 a machine whose only write is to mark a square and which can never erase one, and it computes everything a Turing machine does. Permanence of a machine's own memory costs it no computing power. What the book prices is a permanent reading raised by a step that has to work whether or not the reading is already up, which is a different thing. So the cost I named a paragraph ago is a cost in what a reading can claim. Computing power doesn't pay it.

Drop "driven by the computation." Take a base that counts, and a record that switches on at the fifth tick of a *hidden clock*, whatever the base is doing. It's permanent. The computation doesn't drive it. This is a *timestamp*. A timestamp is not evidence. It says *when*. It doesn't say *what happened*, and a record switched by an outside party or a clock records that party or that clock and leaves the computation out. Try it with the die: play twenty rolls with the rule "turn the cup after roll seven no matter what," and notice you've recorded nothing about the dice.

StructuralRecordAxis.v proves exactly the facts I named for each: the toggle is driven, isn't permanent and isn't a latch; the clock is permanent and isn't driven. That each also keeps the other rules (the ledger is the step count, the first switch-on costs one, a run reaches it) is true by inspection, and isn't a separate theorem.

Permanent but undriven is a clock, driven but impermanent is a toggle, and a record is the thing that's both. Undriven, for the clock, means that along the runs that actually happen the base state doesn't fix when the record turns; Lemma 10.7

below says why that's the condition, and what (R1) as written adds to it. That's the whole discrimination, and I think it's the most useful single sentence in this book for looking at systems other than the one I build, which is why it's here.

The pair theorem and the two counterexamples, exactly. The base for both counterexamples is the counter: states are whole numbers, the step adds one, every state is a starting state, and nothing halts. Two more bits of shorthand: $\neg r$ is r flipped, and a statement in square brackets, like $[k = 5]$, reads on when it's true and off when it isn't.

Lemma 10.4 (Two records are two latches)**.** Let c_1, c_2 be two records on M, with cover π onto a base B, such that: there is one function F with $(c_1(\text{next}_M(m)), c_2(\text{next}_M(m))) = F(\pi(m), c_1(m), c_2(m))$; the ledger never goes down; a step that turns either record on costs at least one; and each record is permanent. Then there are h_1, h_2 with $c_1(\text{next}_M(m)) = c_1(m) \vee h_1(\pi(m), c_2(m))$ and $c_2(\text{next}_M(m)) = c_2(m) \vee h_2(\pi(m), c_1(m))$ for every m.

Proof. Take $h_1(b, r_2)$ to be the first part of $F(b, \text{off}, r_2)$ and $h_2(b, r_1)$ the second part of $F(b, r_1, \text{off})$, then split on each record as in Observation 10.1. □

Checked as `record_pair_is_two_latches_holds` (StructuralRecordAxis.v).

Lemma 10.5 (The toggle)**.** Let the toggle have states (n, r), step $(n, r) \mapsto (n + 1, \neg r)$, every state starting, record r, ledger n, cover $(n, r) \mapsto n$ onto the counter. It is driven by the computation, with $f(b, r) = \neg r$; it is not permanent; and it factors as the latch of no event.

Proof. At $(0, \text{on})$ the next record is off, which breaks permanence; and any latch gives $\text{on} \vee h(0) = \text{on}$ there. □

Checked as `toggle_computation_driven`, `toggle_not_permanent` and `toggle_not_latch` (StructuralRecordAxis.v).

Lemma 10.6 (The clock)**.** Let the clock have states (b, k, r), step $(b, k, r) \mapsto (b + 1, k + 1, r \vee [k = 5])$, every state starting, record r, ledger b, cover $(b, k, r) \mapsto b$ onto the counter. It is permanent and not driven by the computation.

Proof. Permanence: $\text{on} \vee x$ is on. Not driven: the states $(0, 5, \text{off})$ and $(0, 0, \text{off})$ have the same base state and the same record, and their next records are on and off, so no f of the base state and the record can give both. □

Checked as `clock_record_permanent` and `clock_record_not_driven` (StructuralRecordAxis.v). "Driven" is a condition on every state. The clock makes every state a starting state, so the two states the proof compares are both ones a run can begin at, even though a clock started at $(0, 0, \text{off})$ would never reach $(0, 5, \text{off})$.

That last sentence does more work than it looks, so here it is as a lemma. Start the same clock in sync, at (b, b, off) only. On every state its runs reach, the hidden count equals the base, and the clock is exactly the latch of $[b = 5]$. (R1) still rejects it, because (R1) asks about every state, and $(0, 5, \text{off})$ is a state, though no run from a synchronised start reaches it. So (R1) as written separates the clock from a latch partly by how the cover is drawn and through states nobody visits. The separation I mean is the one on reachable states: the clock with every state a start fails (R1) even when only states some run reaches are counted, since two starting states sit over the same base state with the same record and disagree about the next one. And the latch theorem holds in that reachable form too.

Lemma 10.7 (The clock, on the states runs reach)**.** Say M's record is *driven on reachable states* when (R1) holds for every state reached from a starting state in some number of steps. (1) If M's record is driven on reachable states and permanent, then on every reachable state M steps as the latch of some event. (2) The clock of Lemma 10.6 isn't driven on reachable states. (3) The clock started in sync, with starting states (b, b, off) and the same step, record, ledger and cover, is permanent, isn't driven by the computation, is driven on reachable states, and on every reachable state steps as the latch of $[b = 5]$.

Proof. (1) is the proof of Observation 10.1, run on reachable states only. (2) The states $(0, 5, \text{off})$ and $(0, 0, \text{off})$ are starting states, reached in no steps, and the proof of Lemma 10.6 applies to them. (3) From (b, b, off) every step adds one to both counts, so a reachable state has $k = b$ and its next record is $r \vee [b = 5]$. Over every state, (R1) fails at the same pair as in Lemma 10.6. □

Checked as `record_axis_is_latch_reachable`, `clock_not_reachably_driven`, `sync_clock_record_permanent`, `sync_clock_not_driven`, `sync_clock_reachably_driven` and `sync_clock_reachable_latch` (StructuralClockSync.v).

So "permanent but undriven is a clock" is a sentence about the clock with every state a start, over the cover that hides its count, and it holds there in the strong form: undriven on the very states its runs pass through. A clock started in step with its base is a latch on the runs it has, and (R1) throws it out only because (R1) also looks at states those runs never reach.

10.4 Is there only one such machine? No. One structure, two free choices.

Here's a sentence I have to take apart: "there's only one way to carry a record." Three different things get squeezed into that breath.

First claim: given the toll and a fixed price list, is there one way to keep the books, starting at zero and adding each move's price? Yes, on every state reachable from the start; that's Section 6's initiality, and it assumes the price list.

Second claim: is every base-driven record over a base *the same machine*, charges and all? No, and the counterexample is embarrassing in its simplicity. Take any base-driven record and add one more coin to the bill on every step, whatever the step is. Now walk the five rules. The record still answers only to the base and its own value (R1); the bill never touches it. The ledger still never goes down (R2); it climbs faster, if anything. The step that turns the record on still costs at least one (R3), because the toll is a *floor* and a surcharge only raises the price. Once on, the record stays on (R4), and the same run still reaches the write (R5). So the billed machine is base-driven, and it isn't the machine you started with: wherever the original sat still for free, the billed one pays a coin a tick for sitting still. Do it with the cup and the die. Charge a coin for every roll, six or not. The cup turns over on the same roll as before. You've changed the bill and nothing about the record. A machine that charges for CPU time is base-driven and isn't the one that doesn't.

But the billed machine isn't a different *structure*. It's the same record, steps and halting with a different price list. So the claim worth keeping is sameness *up to the price schedule*: compare two machines by their record, their halting and their steps, through their maps down to the base, and don't compare charges. Fix which event the record is, and the bill stops mattering; a flat surcharge, or any surcharge that depends on the current state, sits on the same structure. That's the first free choice. The walk above is the whole argument, and it's checked over an arbitrary base, with any surcharge that depends on the current state: the billed machine is base-driven (`nec_w_billed_honest`), has the same latch event (`nec_w_billed_same_latch`), steps along with the original on record, halting and base (`nec_w_billed_same_up_to_schedule`), and pays each step's cost plus the surcharge (`nec_w_billed_step_cost`), all in NecWArgued.v.

And then the second free choice, and this one still surprises me. *Which event* is the record? Nothing in the five rules picks it. The converse above already says so, if you read it twice: for any event the base can reach, the latch of that event, priced at one unit per write, is a base-driven record. Do it with the die. One cup turns over on the first six. Another turns over on the first one. Both are cups with no reset wire; both are base-driven. Roll a one first, and until a six turns up the two cups disagree about whether anything has been recorded. So *which permanent reading counts as the record*

is a second thing nobody forces on you. The price list is one free choice. The event is the other. That second choice is the question Section 15 is about: what, if anything, makes certification the event worth recording. "The ledger passed one" is an event too, and so is anything else the base can reach.

Corollary 10.8 (The event is a free choice)**.** Let B be any base and h_1, h_2 two events, each meeting the supposition of Theorem 10.2. Then Latch_{h_1} and Latch_{h_2} are both base-driven records over B. If some base state b has $h_1(b)$ on and $h_2(b)$ off, then one step from $(b, \mathrm{off}, 0)$ leaves the first record on and the second off.

Proof. That both are base-driven is Theorem 10.2 applied twice. The step is the latch's own rule: $\mathrm{off} \vee h_1(b)$ is on and $\mathrm{off} \vee h_2(b)$ is off. □

Checked as `nec_w_event_free_choice` (NecWLatch.v), whose proof is `latch_core_honest` (RecordAxisDiscrimination.v) applied to each event.

Now let the cup wobble. Say whatever turns it over sometimes does and sometimes doesn't, with odds you could write on a card. Does fixing the price list fix the odds? No, and the counterexample is two cups with the same price tag and differently loaded coins. Here's the setup, said carefully. Let a record be a single bit that moves at random: from each value, a list of weighted next values, every weight positive, never empty, never leaving on once on, and with the write from off to on charged at least one. Call such a record a *permanent probabilistic record*. Call two such records *the same schedule* when the same moves are possible and every move is charged the same, and *the same in probability* when the weighted lists are equal.

Theorem 10.9 (The schedule doesn't fix the probabilities)**.** (1) There are two permanent probabilistic records with the same schedule that are not the same in probability. (2) There is a permanent probabilistic record that no deterministic latch on its current value describes.

Proof. (1) From off, one record moves to off or on with weights 1 and 1, the other with weights 1 and 2; from on, both stay on with weight 1; both charge 1 for the write. Same possible moves, same charges, different weights. (2) The first record moves from off both to off and to on, while a latch $r \vee h(r)$ gives one value. □

Checked as `schedule_determines_probabilities_refuted` and `deterministic_latch_handles_branching_refuted` (ProbabilisticRecord.v), with the definitions in ProbabilisticRecordCore.v. These are finite-weight models on one bit with no base underneath; they say nothing about any particular machine.

Third claim: is there one state space, up to renaming, that every base-driven record-carrier has to have? I don't claim it. Nothing needs it.

10.5 Bases from outside, and a relation I have to be honest about

The theorem is stated for any base, and the checked work instantiates it on three models of computation that other people built: a small Turing machine (any program); a register machine of the kind the computer scientists Stephen Cook (of the certificates) and Robert Reckhow described in 1973, with unit-cost indirect addressing (any program); and a lambda calculus (call-by-value, meaning an argument is worked out before it's plugged in, and made deterministic by letting a term with no step left to take step to itself). On each, the latch form holds, by the one generic theorem applied to that base (the checked names are listed with the formal statement below). So a register machine with a base-driven record is a register machine plus a cup. So is a lambda calculus. (There's one more instance, the machine I build later, where a particular flag is shown to be a base-driven record and the same theorem makes it a latch, in Section 16.)

Here are the three bases explicitly. A base needs a state, a next-state function, a set of starting states and a halting predicate; the latch theorem asks for nothing else about its instruction vocabulary.

Definition 10.3 (Three concrete bases)**.** The **tape base** has a finite Boolean list t, a head index h, a program counter q and a natural-number cost field. Reading beyond the list returns false. Writing at h extends the list with false cells if needed, then changes that cell. Moving right increments h and extends the list through the new head position. Moving left decrements a positive h; at $h = 0$, it prepends a false cell and keeps the head at zero. A fixed finite program contains these instructions:

Instruction	Next state
Write a bit	Perform the write and advance q by one.
Move left or right	Perform the move and advance q by one.
Branch to j	Set $q := j$ if the current cell is true; otherwise advance q by one.
Halt	Leave the entire state unchanged.
ClaimTapeIsZero with parameter δ	Advance q by one and do nothing else.

An out-of-range program fetch is Halt. Every state is allowed as a starting state, and halting means the fetched instruction is Halt. The cost field is preserved by every

instruction in this base. In particular, its ClaimTapeIsZero instruction neither erases the tape nor charges the cost field; the name supplies no extra semantics.

The **random-access base** has a register function $X : \mathbb{N} \to \mathbb{N}$ and a program counter q. For a fixed finite program its instructions are:

Instruction	Next state
Constant (i, c)	Set $X(i) := c$.
Add (i, j, k)	Set $X(i) := X(j) + X(k)$.
Subtract (i, j, k)	Set $X(i) := \max(0, X(j) - X(k))$.
Indirect load (i, j)	Set $X(i) := X(X(j))$.
Indirect store (i, j)	Set $X(X(i)) := X(j)$.
Positive branch (j, a)	Set $q := a$ if $X(j) > 0$; otherwise advance q by one.
Jump to a	Set $q := a$.
Halt	Leave the entire state unchanged.

Each register-writing instruction advances q by one; its operands are evaluated in the old state, and all other registers are preserved. A fetch outside the program also leaves the state unchanged. Starting states have $q = 0$ and arbitrary register contents. A state is halted when it fetches Halt or is outside the program. Unit cost here counts instructions. It has nothing to do with the certification ledger.

The **lambda base** uses the terms and weak call-by-value rule defined in Definition 13.3. To compute its next state, first try to reduce the function part of an application. If that part cannot step and is an abstraction, either substitute an argument that is already an abstraction or take a step in the argument. If neither applies, the term cannot step. Variables and abstractions cannot step by themselves, and there is no reduction under an abstraction. The base leaves a term unchanged when it cannot step and calls precisely those terms halted. Every term is allowed as a starting state, including open terms.

These definitions make the three examples bases in the sense of Definition 10.1. Applying the latch theorem to them does not assert universality, an execution-time bound, or a simulation between the three bases.

There's also a relation defined between bases, *weak base equivalence*: two bases are equivalent when some correspondence covers their start states both ways, related states agree on an observation and on halting, and one step on one side is matched by some number of steps, possibly zero, on the other. That "some number of steps" is the *stuttering* idea Leslie Lamport (the same Lamport as Section 6's message counters) put forward in 1983: a fine-grained machine and a coarse one shouldn't count as

different because one takes three ticks to do what the other does in one. The relation carries no cost; a machine that multiplies huge numbers in one step and a Turing machine differ in price. The relation never looks at price, so on what it does look at they can match. It's proved to be an equivalence relation, and the latch form is preserved across it.

There is a limit to that statement. The "preserved across it" proof throws away its equivalence hypothesis; the latch holds on every base regardless, so the preservation is true for a boring reason. And nowhere in the checked work are *two* of the bases actually related by this relation. The relation is defined, it's an equivalence, and it is not used to connect the Turing machine to the register machine to the lambda calculus, or any of them to the machine in Part IV. It would be easy to read that list as if it did. What's proved is that each base, separately, carries a latch. That's the honest sentence.

Definition 10.4 (Weak base equivalence)**.** Bases B_1 and B_2, with observations o_1 and o_2 into one common set, are *weakly equivalent* when some relation R relates every starting state of each to some starting state of the other, and any related x, y have $o_1(x) = o_2(y)$, the same halting, some $n \geq 0$ with $R(\text{next}_{B_1}(x), \text{next}^n_{B_2}(y))$, and some $n \geq 0$ with $R(\text{next}^n_{B_1}(x), \text{next}_{B_2}(y))$.

Theorem 10.10 (Outside bases, and the relation)**.** (1) Weak base equivalence is reflexive, symmetric and transitive. (2) If two bases are weakly equivalent, Observation 10.1 holds over one exactly when it holds over the other. (3) Observation 10.1 holds over each of the three bases in Definition 10.3, for every fixed program where the base has one.

Proof. (1) Reflexive with equality as R and one step against one. Symmetric by reversing R. Transitive by composing the two relations; a step on the first base is matched by n steps on the second, and those n steps by some number on the third, by induction on n. (2) and (3) apply Observation 10.1 directly. In (2) the equivalence hypothesis is discarded. □

Checked as `weak_base_equiv_refl_holds`, `weak_base_equiv_sym_holds`, `weak_base_equiv_trans_holds` (through `weak_match_left_runs` and `weak_match_right_runs`), `weak_equiv_preserves_record_latch_holds` and `record_axis_is_latch_on_tm_holds` (CrossBaseGranularity.v), `record_axis_is_latch_on_ram_holds` (CrossBaseGranularityRAM.v) and `record_axis_is_latch_on_l_holds` (CrossBaseGranularityL.v), with weak base equivalence and the tape base defined in

CrossBaseGranularityCore.v, transitivity stated in CrossBaseGranularityTransCore.v, and the random-access and lambda bases defined in their own files.

10.6 Records with more than two values

A cup has two positions. Some records have more: a counter of how many checks passed, a level that only rises. The same reasoning goes through one threshold at a time. For each value a, "the record has reached a" is a yes/no reading, and each of those readings is a latch. What doesn't go through is squeezing the whole record into one cup. "Rises" needs one word: the values come with an order $\leq$, "is at or below," which needn't compare every pair (that's what makes it a *partial* order), and $[a \leq x]$ is the yes/no reading "x has reached a."

Definition 10.5 (A growing record)**.** Let the record take values in a set A with a partial order $\leq$ decided by a Boolean test. A record ρ on M, with cover π onto a base B, is a *base-driven growing record* when: there is one function f with $\rho(\mathrm{next}_M(m)) = f(\pi(m), \rho(m))$; it grows, $\rho(m) \leq \rho(\mathrm{next}_M(m))$; the ledger never goes down and every step that changes ρ costs at least one; and some run from a starting state reaches a step that changes it.

Theorem 10.11 (Growing records are families of latches)**.** (1) For every base-driven growing record there is an h with

$$[a \leq \rho(\mathrm{next}_M(m))] = [a \leq \rho(m)] \vee h(a, \pi(m), \rho(m))$$

for every value a and state m: each threshold is a latch, whose event may read the current value. (2) The thresholds determine the value: if $[a \leq x] = [a \leq y]$ for every a, then $x = y$. (3) For a growing record, charging every change is the same as charging every threshold that switches on. (4) One latch is not enough: a three-valued record that climbs $0 \to 1 \to 2$ over a one-state base, each climb charged one, is base-driven, and no single Boolean latch with a decoding from the base state and the latch reproduces it. (5) A chain of distinct k-bit words, each word's set bits contained in the next one's, has at most $k + 1$ members.

Proof. (1) Take $h(a, b, x) = [a \not\leq x] \wedge [a \leq f(b, x)]$. If $a \leq \rho(m)$ already, growth and transitivity give $a \leq \rho(\mathrm{next}_M(m))$; otherwise the equation is the definition of h. (2) Antisymmetry, with $a = x$ and $a = y$. (3) Growth plus antisymmetry turn "the value changed" into "some threshold switched on," and back. (4) Over one base state, a latch and a decoding can produce at most two values, and the record takes three. (5) Each strict step up the chain adds at least one set bit, and there are only k bits. □

Two cautions about that. In (1) each threshold's event may read the record's current value, so the events aren't all functions of the base alone. And (3) charges a step when it switches some threshold on: a step that switches on three thresholds at once owes at least one, and nothing here makes the charges add up per threshold.

Checked as `growing_record_decomposes_holds`, `thresholds_determine_record_holds`, `record_price_iff_threshold_price_holds`, `one_latch_refuted` and `chain_needs_bits_holds` (GrowingRecord.v), with the definitions in GrowingRecordCore.v. The order need not be finite; (5) is the finite-memory bound.

10.7 What the base being free buys you

The reason this section exists is to answer the suspicion I started with. The record isn't special pleading for one machine. Over any base, under rules you can read, a record that answers to the computation and never comes down is a latch on an event, and the proof doesn't care what the base is. What a particular machine adds on top of the latch is a *choice of price list* and a *choice of event*, and both are choices. The surcharged cup shows the first: the bill can change and the record not notice. The two cups on one die show the second: the rules are happy with either event. Any machine I build will have to make both choices, and whether its choice of event is the right one is a question I'll get to (Section 15), and won't claim to have settled. Before that, the opening chapter promised more than two positions. Earlier in this section a record that rises got cut into one cup per threshold; the next section asks how much of the rest of the book survives once a record has more places to be than up and down.

11

The whole direction

Back in the opening chapter I said something I couldn't yet back up. I said *established* didn't feel like a switch to me. It felt like a direction with lines drawn across it, and the switch was just the first line. Then I said the one piece of that I could settle was that a single switch isn't enough once there are three or more stages, and I pointed here. So here's where I pay that off, and where I find out how much of the rest survives being pushed on.

The cup of Section 10 has two positions. Everything the book has proved about the toll so far (the floor, the merges, the squeeze, the windows) was proved for a cup. The question for this section is blunt: if the record has more places to be than up and down, how many of those results come along, and which ones break?

11.1 A row of cups

Get four cups and a pencil. Write five stages of a sentence on a strip of paper, left to right: *guessed*, *announced*, *refereed*, *printed*, *machine-checked*. Those are close to the stages the Kepler story moved through in the opening chapter. Hales's referees ran seminars on his proof for years and ran out of energy before they could certify it, so here *refereed* stands for that long reading, which is part of why the last stage exists. You don't have to believe my version of that story to use them. Now put a cup between each pair of neighbours, upside down. The first cup means "at least announced," the second "at least refereed," the third "at least printed," the fourth "at least machine-checked." When the sentence reaches a stage, turn over every cup up to that stage. Cups only ever get turned right side up. None of them has a reset wire.

Two things to notice with the cups in front of you. Each cup by itself is exactly the cup of Section 10: it reads yes or no, and once yes it stays yes. And four cups were enough for five stages. Try it with three. You can't: three cups that only ever turn up, in order, give you at most four different arrangements along the way. I'll prove both of those in a minute, and the proof is about as short as the experiment.

Now a harder strip. Some checks don't line up. A referee can read a proof, and a machine can check it, and neither of those is above the other: a refereed proof hasn't been machine-checked, and a machine-checked proof hasn't been refereed. Draw it as a diamond. *Nothing* at the bottom, *refereed* on the left, *machine-checked*

on the right, *both* at the top. Moving up the diamond is still only ever moving up, but now there are two ways up from the bottom, and the two middle points don't compare. That's what a *partial* order allows: some pairs simply aren't comparable. Put a cup on each point that isn't the bottom, one for "at least refereed," one for "at least machine-checked," one for "both." Each cup is still a latch. The record is now a place on the diamond, and the three cups together tell you which place.

Whatever the shape of the strip, a straight line of stages or a diamond or something with a hundred points, I'll call the set of places the record can be the *direction*, and each question "has the record reached at least this place?" a *line* across it. One line is one cup.

11.2 The order, said exactly

The strip and the diamond each come with an order you can see by looking: whether one place is at or below another. Said exactly, the order is a test. For any two places x and y it answers "is x at or below y?" with yes or no, and I write a yes as $x \leq y$. The test has to obey two rules: every place is at or below itself, and if $x \leq y$ and $y \leq z$ then $x \leq z$. An order that obeys just those two is called a *preorder*. If it also obeys a third rule, that $x \leq y$ and $y \leq x$ together force $x = y$, it's a *partial order*, and that third rule is called antisymmetry. The two-point order, no below yes, is the cup. The whole numbers in their usual order are a straight strip that never ends.

The cups say the same thing in their own terms. For a place a, the cup for a asks "has the record reached at least a?", so with the record at x it's up exactly when $a \leq x$, and I write its answer as $[a \leq x]$. In the row of cups I said the cups together tell you which place the record is at. On the strip and the diamond you can check that with your eyes, and part (2) of Theorem 10.11 already proved it whenever the order is partial, though I hadn't asked what in that proof was doing the work. My first guess was that it held for every order.

Why bother with the bare preorder at all, when every strip on the table is a partial order? Because I wanted to see what the third rule was doing, and the way to see what a rule does is to take it away. Take a preorder with two places, call them *left* and *right*, where each is at or below the other. There are two cups, one asking "at least left?" and one asking "at least right?", and both are up at left and both are up at right. Look at the table and you can't say where the record is. Now build a record that swaps between left and right on every single step and charges nothing. Every rule I'm about to state is satisfied. The record never drops below anywhere it was, because each place is at or below the other. No cup ever moves, so the toll in the next subsection has nothing to charge. The record moved on every step and nothing on the table noticed.

Theorem 11.1 (The cups fix the place, and only with antisymmetry)**.** (1) In a partial order, if two places x and y leave the same cups up, $[a \leq x] = [a \leq y]$ for every a, then $x = y$. (2) In a preorder that isn't a partial order this can fail: there's a preorder with two different places that leave the same cups up, and a record over it, driven by the computation and growing in that preorder, that changes on every step while no cup moves.

Proof. (1) Look at the cup for x. It's up when the record is at x, since x is at or below itself, so it's up at y too, and that says $x \leq y$. The cup for y gives $y \leq x$ the same way. Antisymmetry gives $x = y$. (2) Left and right above, with the record swapping every step. □

Checked as `bp_thresholds_determine` and `thresholds_fail_on_preorder` (AxCore.v), and `indiscrete_loses_record` (AxLatch.v).

So in a partial order the cups are the place, and in the preorder I built they aren't. That second half holds in every preorder that isn't partial, by one line: if $x \leq y$ and $y \leq x$ with $x \neq y$, then a cup that's up at either one is up at the other, by transitivity. So the cups fix the place if and only if the order is partial (`nec_f_same_rung_same_cups`, `nec_f_non_partial_cups_fail` and `nec_f_cups_iff_partial`, NecFCups.v). My first guess was right on every strip I could draw, and it was right because of a rule I hadn't noticed I was leaning on. On a bare preorder, putting two places on the same rung tells the toll not to care about moves between them, and it doesn't. I don't think that's a defect. It's the order telling you what you agreed to count.

11.3 The toll, over the whole direction

One cup had one rule: the step that turns it right side up pays. Now there's a row of cups, or a diamond of them, and a single step can turn over several at once. Here's the rule I'd guess first, and it turns out to be the right one: *a step that turns over any cup pays at least one.* A step that turns over three cups at once still only has to pay one. The rule asks whether a step turned something over. It doesn't ask how many.

What does that rule buy? The same thing one cup bought. Take any run, and suppose some cup is up at the end that was down at the start. Somewhere along the run a step turned that cup over, and that step paid. So the run paid at least one. Count the same way and you get more: the run paid at least once for every step that turned anything over.

Now look at what that argument didn't use. It didn't care whether the cups sat in a row or a diamond or something with a hundred points. It didn't care how many there were. It didn't even use the fact that cups never get turned back down. If

something could knock a cup back over, the argument still works, because all it needs is the one step where that cup went up.

To say it exactly I want to put it in terms of places, so here's the translation. The cup at a place a is up exactly when the record is at or above a. Say a step takes the record from x to y. The cup at y is up afterwards, since y is at or above itself, and it was down before exactly when y isn't at or below x. And if y is at or below x, any cup that's up at y was already up at x. So a step turns over some cup exactly when it lands somewhere that isn't at or below where it started. The same reasoning says a run ends with some cup up that was down at the start exactly when it ends somewhere that isn't at or below where it started.

Theorem 11.2 (The toll over any direction)**.** Take a machine whose record is a place in any preorder, and say a step *turns a cup over* when it takes the record from x to a place that isn't at or below x. (1) Every step that turns a cup over costs at least one exactly when, from every start, every run that ends somewhere not at or below its start costs at least one. (2) When that holds, a run costs at least the number of its steps that turn a cup over. (3) It holds exactly when every cup, on its own, pays the toll of Section 10: for every place a, a step that turns the cup at a from down to up costs at least one. The floor of Section 3 is (1) with one cup.

Proof. (1) and (2) are the walk above. For the other half of (1), a run of one step that turns a cup over ends somewhere not at or below its start. (3) is the translation above. □

Checked as `ax_floor_iff_a2`, `ax_cost_ge_exits`, `ax_view_a2` and `ax_a2_iff_views` (AxCore.v, AxLatch.v), with the one-cup floor as `two_point_nfi`. When the cups only ever go up, as on the strip and the diamond, a step that turns a cup over is just a step up (`ax_exit_is_strict_rise`).

That surprised me a little. I'd filed the floor away as a fact about one bit. Nothing in it was about one bit. It was about one cup going up, and anything you build out of cups has that.

11.4 One line isn't enough

Now the payoff I owe the opening chapter. Could a cleverer single cup do the job of all four? One latch, and some decoding that reads the record's place off the latch and whatever the machine shows? Section 10 already shut that door in the smallest case: over a machine with one visible state, a record that climbs through three places can't be produced by one latch and a decoding, because one latch and one visible state make two outcomes and the record has three.

So try a second guess, closer to the cup's own rule. The cup's rule was "new value is old value *or* the event." The obvious rule for a whole row is "new place is old place *joined with* the event's place," where joining two places means going to the lowest place at or above both. On the row: the event names a place, and you turn up every cup as far as that place. Here's where it breaks. Take the strip $0 < 1 < 2$, a cup at 1 and a cup at 2, over a machine with one visible state, so the event has to be one fixed place. Let the record jump from 0 straight to 2, turning up both cups in one step, and stay put at 1 and at 2. From 0 the event has to turn up both cups, so the fixed place is 2. From 1 the same event would turn up the second cup as well, and the record stays at 1.

What works is giving every cup its own event, and letting the event for a cup look at where the record already is. That's exactly what Theorem 10.11 said, and this is why it had to say it. That leaves how many cups. In the row of cups I asked you to try five stages with three and said you couldn't. Theorem 10.11 (5) already bounded the chain from above. Here's the count behind that again, with the matching example added, and with an arrangement of k cups read as a k-bit word, a one for every cup that's up.

Theorem 11.3 (Lines a direction needs)**.** (1) A list of different arrangements of k cups, each keeping up every cup the one before had up (a chain of different k-bit words in which each word's set bits are among the next word's), has at most $k+1$ members, and for every k there's such a chain with exactly $k+1$. So a straight strip of $n+1$ stages needs exactly n cups. (2) Over a machine with one visible state, take the record on the strip $0 < 1 < 2$ that jumps from 0 to 2 and stays put at 1 and at 2. It grows and is driven by the computation, and no rule of the form "join the record with a fixed place" reproduces its steps.

Proof. (1) Going from one arrangement to the next, which is a different one, turns up at least one more cup, and there are k cups, so there are at most k steps and $k+1$ arrangements. For the matching chain, take no cups up, then the first, then the first two, and so on, packed to the left: that's the row of cups. (2) Joining 0 with a fixed place g gives g, and from 0 the step has to give 2, so g is 2. Joining 1 with 2 gives 2, and from 1 the step has to stay at 1. □

Checked as `chain_bits_exact` (AxLatch2.v) and `chain3_not_join_latch` (AxLatch.v).

So five stages need four cups. One won't do it, and three won't either. Cups that only turn up, that is: n write-once threshold bits. An ordinary binary counter holds $n+1$ stages in about $\log_2(n+1)$ bits, because its bits go down as well as up, and the

theorem says nothing against that. The switch isn't wrong. It's the first line, and a straight strip with more than two places needs more of them, exactly one fewer than it has stages. A direction that branches needs at least as many cups as the steps in its longest strict climb, and it can need more. Put three places above a bottom, none of them above another. The longest climb is one step, and one cup can't tell four places apart.

People got here before me, and from a direction I didn't expect, which was programming for many processors at once. Lindsey Kuper and Ryan Newton's LVars (2013), "lattice-based data structures for deterministic parallelism" in their own title, are memory cells whose value only climbs in an order, where a read can't ask for the value and can only ask whether it has climbed past a threshold. That's a cup. What the cup buys them took me a couple of reads. As I understand it, writers running side by side can push the value up in a different order on every run, and a cup that's up stays up whatever the order was, so a program that only ever looks at cups can't see the others' timing. Amin Timany and Lars Birkedal (2021), coming at it from proofs about programs, show in Coq, as I read their paper, that any preorder at all can be carried as monotone ghost state in a program logic. That's the closest thing I've found to the decomposition into lines being checked by somebody else. The match is my reading, and if it's off, that's on me.

The two rest on the same thing, a value that only climbs and a question that only asks whether it's past a point, and they use it for opposite jobs. In an LVar the threshold read steers the program. Ghost state, as far as I understand the term, lives in the proof, and the running program never reads it. Here the record is never allowed to steer anything, so I sit with the ghost state. Push on that and it gives a little: Section 2's glance, the whole reason to keep a record, is a read. What keeps me on that side is that a glance worth trusting gives the answer the long way would have given. Their purposes aren't mine. But the shape is theirs first. What I can't find in either is a price on crossing a line.

11.5 Why crossing a line has to cost

Part II showed that the cup's toll isn't an arbitrary rule. It's forced by three premises, and the argument that ties them together was the chairs: on a machine with finitely many states, a record that goes up and stays up has to seat two states in one chair at the step that raises it (Theorem 4.1), and a step that seats two in one chair is a step that forgets which one it came from. Does that survive the whole row of cups? Mostly, and the parts that don't survive are the interesting ones.

The room needs a flag, and now there are many cups, so the first job is to pick one. My first guess was the cup for the place where the record starts. That goes nowhere:

the starting state already has that cup up, so it's already inside the room, and there's nobody at the door. The cup that works is the one the step turns over, the cup for the place a where the record lands. It's up after the step, since a is at or above itself, and it was down before, which is what turning a cup over means. So the room is every state with the cup at a up, and the state the step starts from is the one at the door. If the move keeps everybody in the room inside it and seats nobody twice, the insiders fill the inside chairs between them and there's no chair left for the newcomer. So one of two things happened. Either two states shared a chair, or somebody who was in the room got moved out, which means their cup at a went back down. I'll call that second thing a *revocation*.

Theorem 11.4 (Turning a cup over merges)**.** On a machine with finitely many states and a record in any preorder: (1) if a move turns a cup over at some state and lands the record at a place a, then either the move merges two states, or there's a state with the cup at a up that the same move takes to a state with the cup at a down, which is a revocation; (2) if the record only grows, so that no cup ever goes back down, the move merges; (3) if the record only grows, the toll holds exactly when every move that turns a cup over at some state, and merges, costs at least one, so merge pricing gives the toll.

Proof. (1) Gather every state with the cup at a up. There are finitely many. If the move keeps each of them there and is injective, it maps that collection into itself without two landing on one, so it hits all of them, including the state it sends the starting state to; injectivity then says the starting state was in the collection already, and it wasn't, because its cup at a was down. So either some state in the collection has its cup at a knocked down, or the move isn't injective. (2) Growth rules out the knocking down. (3) If the toll holds, every move that turns a cup over somewhere pays, merging or not. The other way, a move that turns a cup over merges by (2), so the hypothesis charges it. □

Checked as `ax_exit_merges_or_revokes`, `ax_growth_exit_merges` and `ax_a2_iff_flipping_merges_priced` (AxMerge.v), with `ax_a2_from_merge_price` for the derivation. It's the room again, once for every cup.

Each premise there can be knocked out, and I wanted to see each one go, because a premise you've never seen fail is one you don't understand yet.

Knock out finiteness. Take the whole numbers as the states, which gives a row of cups that never ends, and let the record be the state itself, one more on every step, at no charge. Every step turns up one more cup. The step never merges anything: different numbers go to different numbers. Merge pricing holds, because there are

no merges to price, and the toll fails on every single step. The room broke at the one place it needed to be finite. Everybody moved one chair along, and there was always another chair. That's Charles Bennett's 1973 escape in miniature, the way I read it: a machine that keeps its whole history never has to forget, so it never has to pay, for as long as it has room for the history. With infinitely many states it always has room.

Knock out growth. Take a two-state machine whose record flips back and forth between two places, so a cup goes up on one step and gets knocked back down on the next. The two states just trade chairs, so nothing merges, and the price list is honest about merges. The step that turns the cup up pays nothing. It takes the other branch of (1): the state that had the cup up gets moved to where it's down. It revokes.

Knock out the price on merges. A two-state machine that stamps every state with yes turns the cup up wherever it was down, merges at the stamp, and charges nothing. Of course the toll fails; nobody asked the merge to pay.

Theorem 11.5 (Three ways out)**.** (1) There's a machine with infinitely many states, a growing record and an honest merge price on which every step turns a cup over and the toll fails. (2) There's a two-state machine with an honest merge price whose record doesn't grow, which turns a cup over and fails the toll. (3) There's a two-state machine with a growing record whose merges aren't priced, which fails the toll.

Checked as `history_escape`, `toggle_escape` and `free_merge_escape` (AxMerge.v). So the toll over a whole direction rests on exactly the three things the cup's toll rested on: finitely many states, a record that doesn't come down, and merges that pay. Nothing extra got smuggled in when one cup became a row, and nothing the one cup needed got left behind.

I'd half expected the diamond to need something new, some extra rule for the two middle cups that don't compare. It didn't. Every cup gets its own room, and every room is the old one.

12

The shadow, exactly

Section 7 put the machine behind a window and asked what an outsider could tell. It found that two states can look the same through the window and differ in what matters, and that a price computed from the window can't be exact if the window has a collision (Observation 7.2). That was for one cup. With a whole row of cups in hand I can ask the sharper version of the same question, which is also the one I actually care about. Take the ordinary machine, the one with no cups and no ledger, and let it look at itself. What's the shape of everything it can't see, and what does the toll cost in terms of that shape?

12.1 The ordinary machine is a shadow

Take any machine with a record and a ledger, and sweep both off the table, the cups and the counter. What's left is a machine that steps the same way and knows nothing about either, provided the step's effect on everything else never depends on the cups or the counter: sweeping them off has to commute with the step. A system whose control reads its record or its ledger has no shadow in this sense. Where the premise holds, call what's left the *shadow*. Over any one state of the shadow there's a pile of full states, one for each place the record could be, with its own cups up, and each value the ledger could hold, all of them casting that same shadow. Section 7 already called the pile over a point the *fibre* over it. The shadow is one particular window, the one that shows everything except the cups and the ledger.

The shadow has its own fibres, and every move has piles of its own too: the states a move sends to one landing spot. I'll call those the move's *landing piles*. A fibre is everything that looks the same before anybody moves. A landing pile is everything that ends up in the same spot after one move. Keep the two kinds apart, because the whole section turns on how they meet.

12.2 Pricing from the shadow

First the window question, over a whole row. A *window price* charges each step from what the window shows before and after it. It meets the floor if it charges at least one for every step that turns a cup over, and it's *exact* if it also charges nothing for

any step that turns no cup over. A *collision* is two steps that look the same through the window, where one turns a cup over and the other doesn't.

With one cup, the fix in Section 7 was to put the cup in the window. So my first guess for a row was that the window has to show some of the cups. Try the smallest version: a record in the whole numbers, and a window that shows only the first cup, the one that asks "at least one?" Let the record rise on some steps and hold still on others. Once it's past one, that cup is up before and after both kinds of step, so a step that turned over a cup further along looks exactly like a step that turned over nothing. That's a collision. So then, I thought, the window has to show every cup. That's wrong too, from the other side. Take a record in the whole numbers with two moves, one that adds one and one that leaves it alone, and a window that shows only whether the record is even, which on the row is whether an even number of cups are up. You can't read the record off that window, and no flat charge is exact, since some steps turn a cup over and some don't. But the evenness changes on exactly the steps that turn a cup over, so charging one exactly when the evenness changed is exact.

Theorem 12.1 (Window prices over a direction)**.** (1) If the window has a collision, every window price that meets the floor charges at least one for some step that turns no cup over, so no window price is exact. (2) On a machine with finitely many states and moves, whose views can be told apart by a yes-or-no test, an exact window price exists exactly when the window has no collision. (3) Along any run, a window price that meets the floor charges at least the number of steps that turn no cup over but look, through the window, like a step that does. When it can be decided, for each pair of views, whether some step showing them turns a cup over, some window price meets the floor and, among the steps that turn no cup over, charges one on exactly those, so that number is the least overcharge. (4) A window can price exactly without showing where the record is: there's a record in the whole numbers with a move that raises it and a move that leaves it alone, and a window that shows only whether the record is even; the evenness changes exactly on the rises, no charge that is the same for every step is exact, and charging one exactly when the evenness changes is exact, although no rule reads the record off the window. (5) There's a record in the whole numbers and a window that shows only the one cup "at least one," and that window has a collision.

Proof. (1) The two colliding steps get the same bill. The floor makes it at least one because of the step that turns a cup over, and the other step pays it too. (2) Without a collision, "does some step that looks like this turn a cup over?" is a question about what the window shows, and on a finite machine you can answer it for every pair of views; charge one exactly when the answer is yes. (3) A step that turns no cup

over but looks like one that does gets that step's bill, which is at least one by (1)'s argument; add along the run. For the other half, charge one exactly when the answer to (2)'s question is yes, which needs that question to be decidable, and count the overcharge. (4) A rise changes the evenness and the other move doesn't, so the window sees exactly the steps that turn a cup over; a flat charge either bills nothing for a rise or bills something for a step that turns nothing over; and 0 and 2 look the same through the window. (5) The first cup, as above. □

Checked as `ax_window_collision_overcharges`, `ax_window_no_exact_price`, `ax_window_exact_iff_no_collision`, `ax_overcharge_lower`, `ax_overcharge_tight`, `ax_exact_without_position` and `ax_threshold_window_blind` (AxWindow.v).

Item (4) changed how I think about this. I'd assumed a window had to see the cups to price them. It doesn't. It has to see them *move*. A window that knows nothing about where you are, and knows exactly when you've gone somewhere, can send the right bill every time.

12.3 What a priced move destroys

Now the fibres. Part II showed that the step raising a permanent flag on a finite machine seats two states in one chair, and Theorem 11.4 showed the same for a step that turns any cup over, once the cups only go up. Which two? It matters a great deal. If they're in different shadow fibres, then the ordinary machine lost something and the record didn't. If they're in the same fibre, the ordinary machine lost nothing, and what got destroyed was only ever off the table: two states that cast one shadow, told apart by their cups or their ledger, crushed into one.

My first guess was that it's always the second kind, because the cups are where everything happens. It isn't always. It depends on whether the ordinary machine forgets anything on its own.

Theorem 12.2 (The toll prices a collapse inside a fibre)**.** On a machine with finitely many states and a growing record: (1) a move that turns a cup over somewhere sends two different states to one landing spot; (2) if the shadow's step for that move forgets nothing, sending different shadow states to different shadow states, then those two states are in the same shadow fibre, and every landing pile of the move lies inside one fibre; (3) without that premise it can fail: there's a two-state machine whose two states sit in different fibres and whose shadow step forgets, so the merge at the step that turns the cup over lands two states from different fibres and no fibre loses a point. (4) The halving price, which says a move of cost c shrinks any collection of states by at most a factor of 2^c, holds exactly when every landing pile has at most 2^c states, on a machine of any size.

Proof. (1) is the room of Theorem 11.4, with growth. (2) If two states land on one spot, their shadows land on one shadow spot, and the shadow's step tells shadows apart, so the two states cast the same shadow to begin with. (3) The stamp from the three ways out, with each state as its own shadow: both states land on yes, and so do their shadows. (4) is the worst pile-up of Premise 4.4: one landing pile is a collection that lands on a single spot, and any collection is at most the number of spots it lands on times the biggest pile. Neither half needs the machine to be finite. □

Checked as `ax_exit_collision`, `ax_fibre_collapse_exit`, `ax_step_fibre_in_fibre`, `shadow_merge_escape`, `ax_compression_iff_fibres`, `ax_priced_spots` and `ax_spots_priced` (AxShadow.v, AxInfinite.v).

So on a machine with finitely many states and a record that only grows, when the ordinary machine forgets nothing, the toll is a price on crushing a fibre: on taking two states that cast one shadow, which only what's off the table told apart, and making them one. The last of those premises is the one I just showed you failing.

12.4 How much gets destroyed, on average

The halving price is a worst case. It prices the largest landing pile a move can have. The next question is the average one: if I don't know which state the machine was in, cups and all, but I have some spread of chances over the states, how much do I lose, on average, about which state it was, once I know where it landed? That's what Shannon's conditional entropy measures (Section 4 built the entropy from scratch, so I won't do it again here). Write it in bits.

Theorem 12.3 (The halving price in bits bounds the average loss)**.** Take any spread of non-negative weights over the states, uniform or not. (1) If every landing pile has at most N states, the conditional entropy of the state given its landing spot is at most the total weight times $\log_2 N$ bits. (2) So on a machine with finitely many states and the halving price, the average information a move destroys is at most its cost in bits, times the weight. (3) The bound is reached: the even spread on a pile of 2^c states loses exactly c bits. (4) On a machine of any size and for any spread, even one whose entropy is infinite, the same bound holds for every finite family of landing piles of a priced move.

Proof. Inside one pile, Gibbs's inequality: the entropy of a spread over n states is at most $\log_2 n$, with equality for the even spread. Add over the piles, weighting each by its share. The halving price makes n at most 2^{cost}, and $\log_2 2^c = c$. □

Checked as `ax_fibre_gibbs`, `ax_loss_le`, `ax_priced_loss_bits`, `ax_priced_loss_tight` (AxProb.v) and `ax_inf_loss_bits` (AxInfinite.v). These use Coq's real numbers, and they lean on four standard-library assumptions, all named in Section 22: the two decidability principles that come with the reals, functional extensionality, and excluded middle.

Where does it stop? My first guess was one place, a landing pile with infinitely many states, and that the average would blow up there right along with the worst case. That's half right, and the half that's wrong surprised me. No finite toll can price a move that sends infinitely many states to one spot, and that's a theorem. But the average depends on the spread. Put weight $\frac{1}{2}$ on the first state, $\frac{1}{4}$ on the next, $\frac{1}{8}$ on the next, and so on, and the information lost by crushing the pile never reaches 2 bits, however many of its states you count. Under that spread, two bits of toll would cover the average, though nothing covers the worst case. So I went looking for a spread that does blow up, and it took a nastier one: blocks of states, block k holding 2^{2^k} states and a share $\frac{1}{(k+1)(k+2)}$ of the weight, which adds up to one. Under that spread the loss grows past every bound. No toll covers it, on average or otherwise.

Theorem 12.4 (An infinite pile)**.** (1) A move that sends infinitely many states to one spot isn't priced by the halving price at any finite cost. (2) Under the halving spread $2^{-(j+1)}$, the information lost by crushing the pile, counted over its first n states, is less than 2 bits for every n. (3) Under the block spread, the information lost grows past every bound, so no cost covers it.

Checked as `ax_collapse_unpriceable`, `ax_geometric_loss`, `ax_block_mass_n`, `ax_block_loss_unbounded` and `ax_block_no_toll` (AxInfinite.v).

And heat? Part II already said where the line is, and it's the same line here. Landauer's principle, that crushing c bits' worth of possibilities costs at least $c\,kT\ln 2$ of heat on average, is a premise from physics, and a contested one (Section 4). Nothing in this section proves anything about heat. What the counting does give is this: under that premise, on a machine with the halving price, a move's cost priced at $kT\ln 2$ per unit, times the weight of the piles, is always enough to cover the minimum the premise would demand, for every spread and every finite family of landing piles. Checked as `ax_toll_covers_landauer_minimum` (AxInfinite.v). If the premise falls, that sentence falls with it, and every theorem above it stands, because none of them used it.

12.5 What the shadow is

Put the two sections together and here's the shape I've been circling since the Flatland night. The ordinary machine is the shadow of a machine with a row of cups beside

it. Over every shadow state there's a whole fibre, a pile of states that differ only in their cups and their ledger, and the ordinary machine can't tell them apart, on a machine whose step never looks at the cups. On a machine that pays the toll, a step that turns a cup over costs, whichever cup it is and however many it turns, and on a machine with finitely many states whose record only grows, when the ordinary machine forgets nothing, what that cost pays for is exactly crushing two states of one fibre into one. A window can send the right bill without seeing the cups, if it sees them move. And on a machine with the halving price, where each unit of cost pays for at most one halving, the bill in bits, times the weight of the spread, is never less than what got crushed on average.

What I haven't shown is that any of this is out there in the world, that nature keeps a direction like this over its own shadows. That's a belief of mine, and nothing in this book proves it. What I've shown, and you can check, comes with its premises showing. A machine whose record is driven by the computation and climbs a partial order is a row of cups, one latch for every line. If it also has finitely many states, never lets the record come down, and prices its merges, it has to pay this way. And when its ordinary machine forgets nothing, what it pays for is exactly this. It's less than the Flatland night promised me. It's also the part of it that held when I pushed.

12.6 Shortcuts

Here's the question every machine with a record has to face, and no amount of good design makes it go away. A cup is worth turning over because something later glances at it and doesn't have to work the thing out again: that's Section 2's glance, reading a block's bucket off the table without applying the rule from scratch. Call any move like that a *shortcut*, and call it *sound* when it gives the answer the long way would have given. (That gloss is about cost, leaning on what was done so nothing is redone, and the wall I'm about to show you doesn't look at cost at all; the next section says exactly what it does look at.) So: can a machine *decide*, from inside, whether its own shortcuts are sound? That's a wall, and it's the same wall the logician Kurt Gödel and Turing hit, and I have to show you that it's there before I show you anything that pretends to be on the other side of it.

13

The diagonal

In Section 2 I said you'd watch the same move get made again and again as the book went on: somebody says "everything is X," it's true enough to do real work with, and then somebody draws the diagonal. I owe you the diagonal itself now, because Sections 10 to 12 just handed a record, and a whole direction of them, to *any* system that steps, and the next thing anyone wants from a system with a record is for it to tell sound shortcuts from unsound ones by itself. Before I let anything claim that, I have to show you the wall every such claim runs into, and then show you exactly where a system with a record stands relative to the wall. Not on the far side. Nobody's on the far side.

13.1 Five names and a table

Do this first, with a pencil, because the whole family of arguments is this one trick and you should own it. Write five friends' names down the left of a page. Next to each name, write a five-digit string of zeros and ones, anything you like. Now build a new string: its first digit is the *opposite* of the first digit of the first row; its second digit is the opposite of the second digit of the second row; and so on down the diagonal. Look at what you've made. It differs from row one in position one, from row two in position two, from every row at that row's own position. So it's nobody's string. Your table of five missed something.

With five rows and five digits that's a cute fact. Georg Cantor, a mathematician working out how to compare infinite sets, made the point in 1891 that it works for *any* list, however long, even infinite: hand me a complete list of all infinite strings of two symbols, I'll walk the diagonal flipping as I go, and I'll hand you back a string that's on no row. So there is no complete list. There are more such strings than there are places on any list, and that's where "some infinities are bigger than others" comes from. (He wrote the two symbols as m and w, where I'd write 0 and 1, and that paper wasn't even his first proof of the fact. The first was in 1874, by a harder road through the real numbers, and in 1891 he called the new one much simpler. The diagonal was him finding the easy way.)

Now look at what the argument *did*, because it's going to do the same thing again and again, in new clothes, and I want you to recognise it each time. (Not every name

below is a flipped diagonal in the table sense. Kleene's theorem is the tool that makes the diagonal buildable, Rice's is the general form, the one from Ken Thompson, who built the original Unix operating system at Bell Labs, is an attack that uses the same trick, and the last, Lawvere's, is the theorem that makes them all one.) There was a table: rows are the things being listed, columns are questions, each cell says "does row i do column j." There was the diagonal: the cells where a thing is asked about *itself*. There was a flip: something with no fixed point, like turning yes into no, applied along the diagonal. And the thing that got refuted was *the assumed object*, the complete list. Nothing outside the assumption was touched. Table, diagonal, flip, and the assumption eats itself. Hold those four.

13.2 The same move, again and again

Russell, 1902, from Section 2: the collection of all collections that don't contain themselves. That's Cantor's flipped diagonal with "contains" as the table, and Russell said as much, in *The Principles of Mathematics* in 1903. He got there, he wrote, by running Cantor's argument on a supposed collection of everything, which ought to be the biggest collection there is. What got refuted was the assumed object, the collection of everything, and Zermelo's fix was to stop assuming it.

Jules Richard, a French mathematician who taught at the lycées of Tours, Dijon and Châteauroux, took his doctorate in Paris in 1901 and wrote mostly on geometry, in 1905, in a short note, a letter to the editor of a French science review: some phrases (his were in French) define real numbers ("the ratio of a circle's circumference to its diameter"); there are only countably many phrases, so list the numbers they define in order; run Cantor's diagonal down their decimal expansions and you get a real number no phrase defines, except that you just defined it, in the phrase that describes the diagonal. The lesson I take from that one is the lesson this whole section turns on: keep apart the language a system *talks in* and the language you use to talk *about* it. Mix them and the diagonal bites you.

Richard's paradox has a cousin small enough to do at the table, and it comes with a footnote I can't resist. Russell, the first to put it in print, in 1906, credited it to G. G. Berry, a junior librarian at the Bodleian Library in Oxford. A paradox that has outlived empires, and it came from the library desk. In the version in *Principia Mathematica* it's about syllables. Take the phrase "the least integer not nameable in fewer than nineteen syllables." There are only so many phrases shorter than nineteen syllables, so they name only so many numbers, so some least number is left over, and the phrase picks it out. Now count the phrase: the, least, in-te-ger, not, name-a-ble, in, few-er, than, nine-teen, syl-la-bles. Eighteen. In eighteen syllables it has named the number that can't be named in fewer than nineteen. Same lesson as Richard's, in

a smaller coat: "nameable" is a word about the language, used inside the language, and that's where the bite is.

Kurt Gödel, 1931, with an improvement in 1936 by Barkley Rosser, a logician who had studied under Alonzo Church at Princeton and whose trick lets the theorem assume only that the system is consistent: the big one, and I'll say exactly what it says, because every word of it is carrying weight. Take any consistent formal system with a mechanically checkable list of axioms that can do a bit of arithmetic. Then there are sentences it can neither prove nor refute. The engine is a lemma that lets you build, for any property you can write down, a sentence that the system proves *equivalent* to that property holding of the sentence's own code number. Apply it to "not provable" and you get a sentence equivalent, inside the system, to its own unprovability. Table: sentences by code. Diagonal: a sentence about its own code. Flip: provable to not-provable. Two cautions the Stanford Encyclopedia stresses, and I'll repeat them. The sentence doesn't literally "say" anything about itself. It's provably *equivalent* to a claim about its code, which is less mystical and more exact. And the theorem doesn't say the sentence is unknowable. It says *this one system* can't settle it.

And how did the world take it? Mostly, at first, it didn't. Gödel, twenty-four years old, announced the first theorem during a round-table discussion on the last day of a conference in Königsberg in September 1930, and by the account of Solomon Feferman, one of the editors of Gödel's collected works, the room met it with general incomprehension. Except for one man. John von Neumann pulled Gödel aside to talk about it, kept thinking about it, and a couple of months later saw that the argument gave something more: a consistent system of this kind can't prove its own consistency. He wrote to Gödel with it in a letter dated 20 November 1930. Gödel had already got there. His paper had reached the journal three days earlier, with that second theorem announced in it, and von Neumann left the credit with Gödel. It's the "somebody got there first" shape from the opening chapter, with one difference: here the man who got there second was von Neumann.

Alan Turing, 1936. I'd always taken it that Turing proved the halting problem undecidable, and here I had to go and look. The set theorist Joel David Hamkins and his coauthor Theodor Nenu went back to the 1936 paper and report that the word "halting," the modern statement, and the self-referential proof everyone teaches are all, strictly speaking, absent from it. What Turing proved undecidable were two other questions about his machines (whether a machine prints infinitely many digits; whether it ever prints a given symbol). The second is *equivalent* to halting and the first is strictly harder. So their answer cuts both ways, and they say so: Turing never stated the halting problem, and he essentially proved it, because his second question is a short translation away from it. The name came later. Martin Davis seems to have

been the first to write "halting problem," in a book of 1958, and Stephen Kleene had already stated the problem, without that name, in his 1952 textbook *Introduction to Metamathematics*. The textbook version is still right as mathematics, and here it is, on two index cards. Card A is the Checker: hand it any program card and it announces "halts" or "runs forever." Card B is the Contrarian: "Hand me to the Checker. If it says I halt, run forever. If it says I run forever, stop." Hand B to A. Either answer is wrong. There's no Checker. Table: programs by programs. Diagonal: a program run on itself. Flip: halt to loop.

Stephen Kleene, 1938, the logician whose textbook I just mentioned, and this is the one that turns the paradox into a tool, so slow down. Everything above needs the self-referring object to *exist*: Gödel's sentence has to be buildable, the Contrarian has to be able to "hand me to the Checker." Kleene's second recursion theorem says: for any transformation of programs you can compute, there's a program that behaves exactly as if it had been handed its own source code. Any program may be written as though it already had a copy of itself in hand. He stated it almost in passing, in a paper on something else, and proved it in the last two lines of one of its sections. I have that count from Yiannis Moschovakis, a logician at UCLA and Athens who wrote a 2009 paper on it, expanded in the Bulletin of Symbolic Logic in 2010, called "Kleene's Amazing Second Recursion Theorem," and he calls it a little gem. It's the reason "programs that print their own source" exist in every programming language that can compute whatever a Turing machine can (they're called quines, after a philosopher I'll come to in a moment; try "Print the following twice, the second time in quotes: 'Print the following twice, the second time in quotes:' " and obey it). Self-reference in computation isn't a paradox. It's a construction. The paradoxes happened only when someone assumed an *unrestricted* collection or language; the theorem happens when you build the self-referrer honestly.

And where does the word quine come from? Not from the philosopher himself. Douglas Hofstadter coined it in 1979, in *Gödel, Escher, Bach*, in honour of Willard Van Orman Quine, who had made a long study of self-reference that works indirectly. Quine left a sentence you can take apart at the table, from a 1962 essay on paradox. Here it is, all of it: "yields falsehood when preceded by its quotation" yields falsehood when preceded by its quotation. Do what it says. Take the part in quotation marks, put a quoted copy of it in front of itself, and look at what you've built: it's the whole sentence, word for word. So the sentence says that it itself is false. That's the liar paradox, but look at how it got there: no "this sentence" anywhere, no pointing at itself, just an instruction to quote and copy, the same move as the print-twice instruction above. That's Kleene's trick in plain English, and it's why the paradox and the tool look so alike. The paradox is what you get when the copy asserts

something about its own truth. The tool is what you get when the copy just does something.

Henry Rice, 1953, whom you met briefly in Section 7: the general form. Any property that depends only on what a program *does*, whatever its text, and that's true of some programs and false of others, has no algorithm that decides it for all programs. "Does this program halt" is one such property. "Does this program ever output 7" is another. "Does this program contain an if-statement" isn't, because that's about the text. One standard proof goes straight through Kleene.

And the one that's actually an attack: Ken Thompson, in his Turing Award lecture, published in 1984. Build a compiler that, when it compiles the login program, plants a back door, and when it compiles *a compiler*, plants both habits into that one too. Then delete the trick from the source. The compiled compiler still carries it, and recompiling a clean source with the poisoned binary re-plants it, forever. Reading the source never reveals it. A locksmith teaches an apprentice to cut a spare for every town-hall key and to teach *that* to every apprentice they train, and never to write it in the manual; years later the manual is spotless and every locksmith in town cuts the spare. That's the recursion theorem with a knife in it, and his moral was that you can't trust code you didn't write entirely yourself, which is a sentence I'll come back to in Section 22 when I tell you what the trusted base of this book is.

And Thompson wasn't first either, and he said so. Pull on the lecture's acknowledgment the way the opening chapter pulled on Bitcoin's footnotes: he writes that he first read of the possibility in an Air Force critique of the security of an early implementation of Multics, a time-sharing system from the 1960s, and that he couldn't find a more specific reference to it, and he asks anyone who can to send him one. The critique is Paul Karger and Roger Schell's 1974 security evaluation of Multics, which already described a trap door planted in a compiler that, because the compiler is written in its own language, keeps itself alive. So the knife was in the drawer ten years before the lecture was printed. Then the obvious next question: is there any way out, or is "trust no binary you didn't build" the bottom? There's a partial answer, and it took a while. David A. Wheeler, in a 2005 paper and a 2009 dissertation at George Mason University, worked out *diverse double-compiling*: compile the suspect compiler's source with a second, independently made compiler, use the result to compile that source again, and compare the output bit for bit with the suspect binary. If they match, the binary really is what its source says, unless both compilers were poisoned in the same way. The trust doesn't vanish. It moves, from one compiler to the claim that two unrelated ones weren't rigged together. That's the shape of every stopping point in this book. You don't reach the bottom. You reach a place where you can say exactly what you're trusting.

Bill Lawvere, in 1969, proved a fixed-point theorem that, as I read it, makes all of these one theorem in disguise: if a set can be mapped onto all the functions from itself to some target, then every function on the target has a fixed point; so if the target has a function with no fixed point (a flip), no such covering exists. Cantor, Russell, Gödel, halting, as instances of one shape. Noson Yanofsky retold it in 2003 without the category theory. I mention it because "it's the same wall" is a claim I make below, and I want you to know it's fair at the level of *proof shape*, and that every instance has its own boundary of *who counts as a decider*, and that boundary is exactly where most overclaims live.

13.3 Four things it doesn't give you

Before I point this at a system with a record, here are four conclusions the theorem doesn't reach. Each one looks as if it follows, and I wanted to see exactly where each one stops.

"Gödel shows minds aren't machines." John Lucas, a philosopher who spent thirty-six years as a fellow of Merton College, Oxford, argued it in 1961, in a paper called "Minds, Machines and Gödel." Roger Penrose, the Oxford mathematical physicist who later shared a Nobel prize for his work on how black holes form, revived it twice, in two books, *The Emperor's New Mind* in 1989 and *Shadows of the Mind* in 1994. I'm sure of who argued it and when. Whether they're right is the question, and the standard replies, as far as I can tell, have held up: the Gödel sentence is true *only if the system is consistent*, so a human who "sees" its truth has to know the system is consistent, which is the hard part; and nobody can build the Gödel sentence of a system they can't specify, so humans might be machines too complicated to diagonalise. The theorem has a conditional in it and the argument drops the conditional.

And here's the part that's easy to miss: Gödel himself didn't go as far as Lucas. In a lecture on the day after Christmas in 1951, at a meeting of the American Mathematical Society at Brown University, he drew from his own theorems only an either-or. Either the human mind, even inside pure mathematics, infinitely surpasses the powers of any finite machine, or there are problems about whole numbers, of a certain simple kind, that no proof the human mind can conceive will ever settle. Feferman, writing about that lecture, says Gödel privately believed the first half and still stated only the disjunction, because he didn't have a knock-down argument against the machine side. Gödel wrote soon afterward that he meant to publish the lecture, and never did; the text sat among his papers until after his death. So the man whose theorem gets quoted for "minds aren't machines" stated an either-or and left the deciding to

whoever came next. That's the conditional again, kept, by the person with the most right to drop it.

"Undecidable means unknowable." Two senses share the word: a problem with no single algorithm correct on every instance, and a sentence independent of one particular system. Neither means nobody can know a given answer. Any single instance is decidable, trivially: for any one program, one of "say yes" and "say no" is right. What's ruled out is one *inside* procedure that's right on *every* case.

"Everything interesting is undecidable." Rice is about behavioural properties over *all* programs. It says nothing against checking the text, nothing against deciding for specific programs, nothing against analyses allowed to say "I don't know," nothing against finite-state model checking or type checking. Real compilers are limited by cost and by choosing to be sound-but-incomplete. The diagonal isn't what stops them.

"Self-reference is a paradox." It's a theorem with a construction. Kleene, quines, Thompson.

13.4 Now point it at the record

Here's the question Section 12 left open, with nothing in it but what Parts I to III have built. A system steps; a record rides along and latches on an event (Section 10); when the event is a certification, Section 8 said the reading, with its object and its paid event behind it, is what lets an auditor treat a claim as established, and for a claim that a shortcut is sound, that's what makes the shortcut *admissible*. So: can a system like that have, as one of its own programs, a procedure that correctly decides, for every program, "does this program admit a sound shortcut"? That would be autopilot. Hand it any program and it says yes or no, and it's always right.

That's the question in plain words, and the checked statement is narrower, so here it is before the proof. In the theorem, "admits a shortcut" is any property of programs with two features: two programs that behave alike agree on it, and there's one program that has it and one that doesn't. That's all the argument gets to look at. The diagonal doesn't see soundness. Each system that wants the wall picks its own property, and whether that property lines up with being entitled to soundness in Section 8's sense is a separate fact about that system, checked one system at a time if it's checked at all. A procedure that returns soundness records for some programs, or only for restricted inputs, or with outside evidence, is not what the wall rules out.

So in the theorem "shortcut" is a label. The plain-words gloss at the end of Section 12, leaning on a record so the thing isn't worked out again, is about *how* a program gets its answer, and how is exactly what extensionality throws away: the theorem only gets to see what programs do, and it treats any two that do the same

as one. A question like "is there a cheaper program that gives the same answers?" isn't the theorem's question, and a wall for it would need a different theorem. The nearest thing I know of is Manuel Blum's speedup theorem, from 1967, and it goes somewhere stranger: there are computable functions with no fastest program at all, where every program for the function has another that's faster on all but finitely many inputs. That's a fact about cost. It isn't a wall about deciding, and nothing in this book proves anything in that territory. Each use of the wall also names its sameness. In the substrate theorem it's the substrate's own $\simeq$; on L it's what a term computes; on the small machine's host, in Section 16.18, it's everything a program ends with, the record included. A particular notion of shortcut gets the wall only after someone proves it behaves alike on programs that are the same in the sense in use, and holds of one program and fails of another.

The answer is no, under premises. The proof is the two index cards, and I'll give you the premises because they're the whole content. Suppose there were such a decider, one that says yes exactly when a program admits a shortcut. Build the flip: "if the decider says yes about me, behave like a program that admits no shortcut; if it says no, behave like one that does." If the system has a recursion theorem (a program can be handed its own code), and if that flip is one of the transformations the system can carry out, and if "admits a shortcut" depends only on behaviour (two programs that behave the same agree on it), and if there's at least one program each way, then there's a program that behaves exactly like its own flip, and whatever the decider says about it is wrong. Both cases close in a line each. That's `structural_shortcut_undecidable`, which rules out a decider that says yes if and only if a program admits a shortcut, stated for *any* substrate with those fields. (A substrate here is the bare shape the class asks for: programs, a run that may fail to give back a state, a counter with a step relation it never goes down along, an encoding of programs as data, a notion of "representable" transformation, and the recursion theorem for those. Nothing about what the states are made of. The smallest checked instance is the nat family below.) It's Rice's trick one floor over, and I should be careful with that. Rice's theorem is stated for the functions Turing machines compute. This wall is the same diagonal with different premises: it carries its own recursion theorem and its own list of representable transformations, and it picks the property. So it isn't a corollary of Rice as Rice stated it, and it doesn't show that any property is harder or easier to decide than classical halting. And here's something about its proof that I want you to have: it never touches the counter, the toll, or the step relation. It uses the recursion theorem, representability of the flip, the two witnesses, and extensionality, and nothing else. (The file's header calls the substrates it covers ones that respect the toll. The proof uses neither the toll nor the counter's climbing. It's the diagonal, clean.)

Here's the setting and the wall, with every word pinned down so you can poke at it.

Definition 13.1 (Substrate)**.** A *substrate* has a type of states, a type Prog of programs, and a run map that takes a program and a state and either gives back a state or gives nothing. It has a counter μ on states and a step relation along which μ never decreases; an encoding enc of programs as states with a decoder dec such that $\mathrm{dec}(\mathrm{enc}(p)) = p$; and a predicate Rep on maps $f : \mathrm{Prog} \to \mathrm{Prog}$, the transformations the substrate counts as its own. Two programs are *equivalent*, $p \simeq q$, when $\mathrm{run}(p, s) = \mathrm{run}(q, s)$ for every state s. The last field is the *recursion theorem*: for every f with $\mathrm{Rep}(f)$ there is a program p with $p \simeq f(p)$.

Definition 13.2 (Shortcut predicate)**.** A *shortcut predicate* on a substrate is a property A of programs with two programs y and n such that $A(y)$ holds, $A(n)$ fails, and A is extensional: $p \simeq q$ implies $A(p) \iff A(q)$. For a Boolean function d on programs, its *flip* is $f_d(p) = n$ if $d(p)$ is true and $f_d(p) = y$ if $d(p)$ is false.

Theorem 13.1 (No inside decider)**.** Let a substrate carry a shortcut predicate A with witnesses y and n. Then there is no Boolean function d on programs such that $\mathrm{Rep}(f_d)$ holds and, for every program p, $d(p) = \mathrm{true} \iff A(p)$.

Proof. Suppose d is such a function. Since $\mathrm{Rep}(f_d)$, the recursion theorem gives a program p with $p \simeq f_d(p)$. Split on $d(p)$. If $d(p)$ is true, then $A(p)$ holds by correctness, and $f_d(p) = n$, so $p \simeq n$, and extensionality turns $A(p)$ into $A(n)$, which fails. If $d(p)$ is false, then $A(p)$ fails by correctness, and $f_d(p) = y$, so $p \simeq y$, and extensionality turns $A(y)$ into $A(p)$. Both cases are contradictions. The argument uses the recursion theorem once, the two witnesses, extensionality, and nothing about μ, the step relation, or the encoding. □

Checked as `structural_shortcut_undecidable` (StructuralUndecidability.v; the correctness it rules out is $d(p)$ true exactly when $A(p)$), stated over the class `Substrate` (Substrate.v) and its extension `WithShortcutPredicate`. Every field named above is a premise; the theorem is about every substrate that supplies them.

So the wall is there. Now the honest part: *who counts as a decider*. The theorem rules out a decider whose flip the substrate can itself represent. It says nothing about a decider standing *outside*, and an outside decider can exist for the very same property, correct on every program. That's the boundary of the sentence, and the boundary is Richard's lesson: the language you talk about a system in is a different language

from the one it talks in. You'll see it in miniature in a minute, on the nat family, with a pencil.

Two forms of the wall live in this section, and they're honest in different ways.

The *nat family* (`nat_self_undecidable`): a tiny substrate built so that program two is the Contrarian for a given candidate decider d, and for every d, d is wrong about program two. It's a *family*, one instance per d, and its recursion theorem holds by construction. It's the index cards made formal and nothing more.

Theorem 13.2 (The nat family)**.** Fix any Boolean function d on $\mathbb{N}$. Let the substrate N_d have programs and states $\mathbb{N}$; program 0 returns 1, program 1 returns 0, program 2 returns 0 if $d(2)$ is true and 1 if it is false, and every other program gives nothing; μ is constantly 0; $\mathrm{Rep}(f)$ holds when $f(p) = 1$ for every p with $d(p)$ true and $f(p) = 0$ for every other p; and $A(p)$ holds when program p returns 1 on input 0. Then it is not the case that $d(p) = \text{true} \iff A(p)$ for every p.

Proof. The recursion theorem of N_d holds with the fixed witness 2: for a representable f, $f(2)$ is 1 or 0 according to $d(2)$, and program 2 returns exactly what that program returns. The witnesses for A are 0 and 1. The flip of d is representable by the definition of Rep, so Theorem 13.1 applies to d itself. Directly: $A(2)$ holds exactly when $d(2)$ is false. □

Checked as `nat_self_undecidable` (NatSubstrateInstance.v), with the built-in recursion theorem `nat_recursion_theorem`; the general form, no decider whose flip is representable in N_d, is `nat_structural_shortcut_undecidable` in the same file.

Now push on it once more, because the nat family shows the outside decider in three lines, and you can do them yourself. Stand outside N_d, knowing d. Write down a new decider e: say yes on program 0, say "not $d(2)$" on program 2, and say no on everything else. Check it against A. Program 0 returns 1, so $A(0)$ holds and e says yes. Program 1 returns 0, so $A(1)$ fails and e says no. Program 2 returns 1 exactly when $d(2)$ is false, and that's what e says. Every other program gives nothing, so A fails and e says no. So e is right about every program. Does that break the wall? No, and here's the bit that matters: in N_d a flip is representable only if it says 1 exactly where d says yes, so the flip of e is representable only if e and d agree everywhere, and they disagree at program 2. The correct decider exists; its flip just isn't one of the substrate's own moves. (That one isn't a separate theorem in the Coq. It's three lines and a pencil, and you just did it.) Undecidable from inside and decided cleanly from outside, at the same time, about the same property. That's not a paradox. It's two different people asking.

And the one where the recursion theorem is *proved* from the rules, closed under the global context, which is why it exists: *L*, the weak call-by-value lambda calculus that Yannick Forster and Gert Smolka presented in 2017 as a model of computation in Coq, and which a group at Saarland has since used as the seed of a whole library of undecidable problems. The development here builds its own copy of L from the reduction rules, so the L results below assume nothing from that library. There, `L_recursion_theorem` is a theorem with a proof, Rice's theorem follows (`L_rice`), halting follows (`L_halting_undecidable`), and the diagonal is reproved in the shape of the substrate theorem (`L_structural_shortcut_undecidable`), for any property of programs that depends only on what they compute and that holds of one program and fails of another. The statement mentions no shortcut. That's the diagonal for a fixed model, closed under the global context, with its hypotheses in its statement. It isn't an instance of the generic class (the class wants a run map that always answers, with a state or with nothing, and L's evaluation can run forever without ever answering), so it's the argument *reproved*. The generic theorem isn't *applied* here. Both are true; they're different sentences.

Here's what that string of words means, because L is the strangest thing in this book that computes, and the smallest. The lambda calculus is Alonzo Church's, from the 1930s, and it's a programming language with one idea in it: functions. A term is one of three things. A variable. A function, written λ followed by its body, the body saying what to do with whatever you hand it. Or one term applied to another, written side by side. Running a program is one move, over and over: when a function meets an argument, put the argument into the body where the variable was. No numbers, no memory, no instructions, and in 1937 Turing published a proof that it computes exactly what his machines compute. "Weak call-by-value" is two house rules about the order of moves: an argument is run down to a finished function (a *value*) before it's put in, and nothing runs inside a function that hasn't been handed anything yet. Variables are written as numbers counting how many λs back they were bound (a trick of Nicolaas de Bruijn, who ran the Automath proof-checking project at Eindhoven from 1967 and comes back in Section 22; it means two variables can never get mixed up by sharing a name), and a term is *closed* when every variable in it is bound inside it, so it leans on nothing outside. Last piece: to hand a program to a program, L needs a way to write any term as data, the way a program's source is a string you can pass around. The one used here is the *Scott encoding*, named for Dana Scott, a student of Church's who went on to build mathematical models of the lambda calculus itself. As far as I can trace it, the encoding first appeared in lecture notes of his from Berkeley in 1962 that he never published. All you need about it is that every term t gets one *code*, written $\ulcorner t \urcorner$, and the code is itself a closed value. Two tools get built on top of that in the proof: a *quoting* term Q that takes the code of a

term to the code of that code (a program that prints its own source needs exactly this move), and a *fixed-point combinator*, a term that lets a function call itself, which is how Q works its way through a code of any size.

Definition 13.3 (L, as built here)**.** A term of L is a variable (a de Bruijn number), an application $s\,t$, or an abstraction λs; the values are the abstractions. One step is weak call-by-value, left to right: $(\lambda b)\,v$ steps to b with v substituted when v is a value, a step inside the left part of an application is a step of the application, and so is a step inside the right part once the left part is a value. A term s *evaluates to* v when s reaches v in some number of steps and v is a value; $p \equiv q$ when p and q evaluate to the same values. Every term t has a code $\ulcorner t \urcorner$, its Scott encoding, which is a closed value. A map f on terms is *L-computable* when some closed value s has $s\,\ulcorner p \urcorner \equiv f(p)$ for every p. A term D *decides* a property P of terms when, for every p, $D\,\ulcorner p \urcorner$ reaches $\lambda\lambda.1$ (true) if $P(p)$ holds and $\lambda\lambda.0$ (false) if it fails. P is *extensional* when $p \equiv q$ and $P(p)$ give $P(q)$.

Theorem 13.3 (Recursion, Rice and halting for L)**.** (1) For every closed value s there is a closed term t that reaches $s\,\ulcorner t \urcorner$; hence every L-computable f has a term p with $p \equiv f(p)$. (2) Let P be extensional, with $P(y)$ and not $P(n)$ for closed terms y and n. Then no closed term decides P. (3) No closed term decides halting, the property of evaluating to some value.

Proof. (1) A closed quoting term Q takes $\ulcorner t \urcorner$ to the code of $\ulcorner t \urcorner$; its constructors, recursive program and terminating induction are given in Appendix G. A closed term mkapp takes two codes to the code of their application. Let $A = \lambda x.\, s\,(\text{mkapp}\ x\ (Q\,x))$ and $t = A\,\ulcorner A \urcorner$. One step gives $s\,(\text{mkapp}\ \ulcorner A \urcorner\ (Q\,\ulcorner A \urcorner))$, and the argument reduces to $\ulcorner A\,\ulcorner A \urcorner \urcorner = \ulcorner t \urcorner$. For the second half, take s computing f; then $t \equiv s\,\ulcorner t \urcorner \equiv f(t)$. (2) Given a closed decider D, the flip $\lambda x.\, D\,x\,(\lambda_.\, n)\,(\lambda_.\, y)\,(\lambda z.z)$ is a closed value; on the code of p it reaches n when D answers true and y when D answers false. Take its t from (1). If $P(t)$, then $t \equiv n$ and $P(n)$; if not, $t \equiv y$ and $P(t)$. Both are contradictions. (3) Halting is extensional; apply (2) with $y = \lambda z.z$ and $n = (\lambda x.\, x\,x)(\lambda x.\, x\,x)$, which steps only to itself and so never reaches a value. □

Checked as `second_recursion`, `L_recursion_theorem`, `L_rice`, and `L_halting_undecidable` (LRecursion.v), with the quoting term's specification `Q_spec` and the substrate-shaped form `L_structural_shortcut_undecidable` (no decider whose flip L can compute says true exactly when the property holds) in the same file. The file uses only Coq's standard library.

The construction in that proof isn't mine, and the credit belongs right here. Forster and Smolka's L development of 2017 has the Scott encoding of terms, a closed quoting term with exactly this specification, the code of a term to the code of its code, and a fixed point built the same way: their t is $C \ulcorner C \urcorner$ with $C = \lambda x.\, s\,(A\,x\,(Q\,x))$, so their C is my A, and their A, which builds the code of an application, is my mkapp. They say they follow the proof in the Dutch logician Henk Barendregt's textbook on the lambda calculus (1984), where statement (1) is the full lambda calculus's second fixed-point theorem, and part (2) is what they call Scott's theorem. My file rebuilds L and reproves all of it on the standard library alone; the route is theirs, and Barendregt's before them.

That's the wall on paper, on a family built to fit it, and on one outside model, closed under the global context. The machine built in Part IV fills in this same shape, closed the same way: on its host machine, the one the universal program runs on, which has a counter for every whole number, programs are numbers, Kleene's recursion theorem is a theorem, and so is Rice's, and the fixed point can be taken to agree on what the program computes and also on the ledger and the flag (Section 16.18).

Here's the picture all of that fills in, lined up against Turing's, one column per axis.

Time axis	**Structural axis**
counts: steps	counts: μ
asks: does it halt?	asks: does it admit the shortcut?
the wall: no Turing machine decides halting	the wall: no decider whose flip is representable, under a recursion theorem

Read the table as two uses of one diagonal, each under its own premises. That's all it says. The structural wall never uses μ; the μ entry only says what that axis counts. The nat result is a family indexed by the candidate decider, and L's results are about one fixed model and properties that mention no shortcut. Neither hands you a second fixed undecidable problem, and neither separates Turing degrees: a property can sit behind the wall from inside and be decided cleanly from outside, as the pencil argument on N_d showed. And "admits the shortcut" in the right-hand column is the theorem's sense, any behavioural property with a yes and a no. "Has a cheaper way to the same answer" is a second question, and the column has no wall for it.

13.5 How to kill it

Three ways, and I mean them as instructions.

One: produce, for some substrate with a shortcut predicate, a decider whose flip the substrate can represent and that's correct on every program. That breaks the generic proof at a named step, and I've named the steps.

Two: for the nat family, produce a d that correctly decides the predicate of the substrate built from d.

Three: for L, produce a closed term that decides some extensional property holding of one closed term and failing of another. That breaks `L_rice`, which rests on the reduction rules and Coq's standard library and nothing else.

An outside decider alone doesn't count against any of the three; the pencil argument on N_d is why.

13.6 What this does to the book

It does two things. It keeps me honest: nothing in what follows claims that a system with a record can check its own shortcuts on autopilot, because that's a wall, the same wall, and I showed you the bricks. And it tells you what a record *can* do, which is the thing Part III has been about since the counter: it can carry a shortcut *with its receipt*, so that the question "was this earned" is answered by an event you can point at. No decider has to exist for that, which is lucky, because on a system with a recursion theorem no inside one can. The diagonal says such a system can't automate the judgement from inside, for any behavioural property with a yes and a no. The latch says you can record the commitment. Those are different jobs, and a record does the second.

The diagonal has a famous cousin that seems like it ought to bite a record too, so let me walk right up to it. Gödel's second theorem says a consistent system strong enough for arithmetic can't prove its own consistency, and Martin Löb, a logician answering another logician's question, sharpened that in 1955: if such a system proves "if this is provable, it's true," it already proves the thing itself. So can a record vouch for itself, and does Löb stop it? My first guess was that Löb stops it, since a record looked like a system talking about itself. Here's where I landed instead, and mostly it's a relief. Hold two things side by side: "there's a receipt for this somewhere," and the receipt in your hand. Löb's theorem is about *provability*, the bare claim that some proof exists, the receipt somewhere. A record of the kind this book keeps is a different animal: an event, a check of some claim that passed, a commitment to it, and a certificate, at a definite step of the run. The receipt in your hand. It doesn't say which claim or which step, as the two runs in Section 16.18 show; it says the event happened. Sergei Artemov, the logician from the opening chapter, drew exactly that line in 2001, between implicit provability ("there is a proof") and explicit evidence ("this is a proof"). As I read him, for explicit evidence the reflection

step "if this is a proof of F, then F" is harmless, while for the bare "there is a proof" it's what Gödel and Löb forbid. Why, in my words: a receipt in your hand you can look at, and a receipt somewhere you take on the system's own word. A record sits on the explicit side.

Push on that and it gives, because a receipt is only as good as whoever wrote it. Its trust rests on the checker meaning what it says, and that gets settled once, from outside the machine. I don't think it can be moved inside, and Gödel's second theorem is my reason. The theorem says a consistent system strong enough for arithmetic can't prove its own consistency, and a checker vouching for itself would be doing something close to that. That last step is my reading. The theorem itself stops at consistency. So I didn't get rid of the receipt somewhere. I moved it out of the machine and handed it to whoever reads the record. Try to move that proof inside the system, so it can certify its own checker or a successor's, and you hit exactly the wall Eliezer Yudkowsky and Marcello Herreshoff named the Löbian obstacle in 2013, writing about agents that want to trust their own rewrites. As I understand their draft report, they came at it from the far side, an agent wanting to trust a successor's proofs the way it trusts its own. Same arrow, pointed forward, same wall.

One more thing falls out of the same picture, and it's a short argument with no checked theorem behind it, so take it at that weight. Can a record rest on itself, the way the sentence "this sentence is certified" would like to? Not if it's earned and permanent. The check that a certificate stands on happened while the record was still down, because the record goes up only after that check passes, and once up it never comes back down. So no claim whose truth needs the record already up can be the claim a certificate stands on. Saul Kripke's 1975 theory of truth calls a sentence like that, one that only ever leans on itself, ungrounded, and earned records are grounded in his sense: every one bottoms out in a check that didn't lean on it. Caspar Oesterheld's 2019 work on programs that cooperate by simulating each other needed a grounding step of the same kind to stop an infinite regress. I haven't checked this in Coq, and I'm not claiming more than the paragraph says.

One last distinction, because the two are easy to conflate and conflating them is the easiest way to misread this book. There's a *second* wall, nothing to do with the diagonal. It's Section 7's kind of wall, a window with a colliding pair in it. Two states that differ in their certification reading can look identical through a window that doesn't show that reading, the stamp inside the closed drawer and the drawer opened and shut, and then no function of the window recovers the reading (Theorem 7.1: a decoder has to give one answer on everything the window lumps together). That's missing information. Nothing about it is undecidable; it holds for finite things with nothing to compute; and it sits *beside* Rice. None of Rice is underneath it. The property the diagonal uses (any behavioural property of programs with a yes and

a no) is a different kind of thing from the reading the window hides, and nothing here says the two are the same. Two walls, two axes. The diagonal is about what no inside procedure can decide. The window is about what no outside observer can see. I'll need both, and I'll try not to call one by the other's name again. The machine I build later has a colliding pair of exactly that kind (Theorem 16.14).

14

Five small models

A toll, a counter, two kinds of wall and a latch, and every one of them shown on paper, on cups and cards and dice I made up. I've been building a toy and claiming it's the smallest version of something people actually do. Time to go and look at what people actually do. I picked five systems that keep records other people rely on, with real money or real security behind them, and asked each the one question this book is about: *is there a price on the moment the record is written?* Not on the work around it. On the write. I'll tell you the answer first, because it's a single sentence and it surprised me: of the five specifications, read through the transcriptions I made of them, only one takes a fee at the moment of the write, and that one has a leak at exactly the place Section 6 said a counter leaks, where it goes down.

"Price" needs one meaning for all five, or the answer is whatever I want it to be, so here it is, and every system below gets classified by it. A specification *prices* an event when it attaches one of three costs to it. A *fee*: something taken at the event itself, whatever happens afterward. A *bond*: value put at risk by the event and lost only if some later event happens, a contradiction, say. A *budget*: a finite allowance the event draws down, so that only so many such events can ever happen. A wall isn't a price under this definition. Nobody pays a hash; reversing one is just infeasible. Neither is work a protocol needs and never charges for. The table at the end of the section has a column for each of the three kinds, and what it shows is a finding about the five, because the definition came first.

Three things are easy to blur in a section like this, so I'll name them now and keep them apart all the way through. There's the *real specification*: the document real money and real security run on. There's my *transcription*: one mechanism copied out of that document into Coq, small enough to check. And for some of the five there's a *wrapper*: a tiny abstract model that takes one premise as given and says what follows from it. A claim proved at one level isn't a claim at the level above it. Near the end I'll walk one system, gas, through all three, so you can see exactly where each claim stops.

What I built for each is a *fragment*: a small Coq model of the one mechanism I'm asking about, transcribed from the system's own specification where there was one, and nothing more. Not the system. I'll say for each one what the fragment is and

what it isn't, because "Ethereum has been formalised" is the kind of sentence that gets a book thrown across a room, and rightly.

14.1 Proof-of-stake finality: the price is on going back

Ethereum and chains like it decide which blocks are final by having validators, people who've locked up money (their *stake*), vote on checkpoints: blocks picked out at regular numbered rounds called *epochs*. A vote isn't for one checkpoint. It's a *link* from a checkpoint the validator already trusts, the source, to a later one, the target, and every chain starts from one first block, its *genesis*, which counts as trusted from the start. In the design Vitalik Buterin and Virgil Griffith published in 2017, called Casper, a checkpoint is *justified* when validators holding two thirds of the stake vote a link to it from a justified source, and *finalized* when it is justified and a two-thirds link runs from it to its direct child. The record is "this checkpoint is final."

What's priced? Two rules: don't vote for two different targets in the same epoch, and don't cast a vote whose span, source to target, surrounds the span of another of your votes. Break either and your deposit can be destroyed, which is called *slashing*. The theorem they prove, called accountable safety, says that if two *conflicting* checkpoints are ever both finalized, at least a third of the stake broke a rule and can be slashed. So the price is on *reversal*, on making the record contradict itself. The write itself, finalizing, takes no fee: honest validators finalize all day and nobody loses anything. In the terms from the start of the section, Casper's price is a bond. The deposit stands behind every vote, the finalizing votes included, and it's taken only if the record is later made to contradict itself.

Do it with three validators. Let A and B vote one branch, B and C vote a competing branch. Each pair is two of three. Both branches finalize. Now read the rule: B cast two different votes with the same target epoch. B is the third that gets slashed; A and C are clean. That's what a fork costs and who pays, and the checked work has exactly this instance (`casper_fork_exists`, `casper_fork_slashable`), so the theorem's premise isn't vacuous: forks of that kind can be built, and the theorem fires on them.

The fragment: a transcription of a published Coq proof of Casper's accountable safety (the casper-proofs development from Runtime Verification and the University of Texas at Austin, itself based on Yoichi Hirai's proof in Isabelle, another proof checker; proofs redone on the standard library), with validator sets as predicates and quorum intersection as a premise. A *quorum* is a set of validators big enough to act, here two thirds of the stake, and *quorum intersection* is the arithmetic of the three-validator example: two thirds plus two thirds is four thirds, so any two quorums share at least a third. The fragment takes the overlap as given. It doesn't do that

arithmetic. No deposits, no rewards, no fork choice (the rule for picking a branch to build on), no connection to anything else in this book. `accountable_safety`: a finalization fork implies that a set of validators the size of that overlap, at least a third of the stake, is slashed. And `finalization_without_slashing`: a one-validator chain finalizes and nobody's slashed, which is the "write is free" sentence in a degenerate setting, and only there.

Here's the fragment, small enough to read in one sitting. Every name below is from the transcription.

Definition 14.1 (Casper setting and votes)**.** A *setting* has a type of validators, a type of block hashes with a parent relation and a genesis hash, and two classes of validator sets, the "two-thirds" sets Q_1 and the "one-third" sets Q_2. It carries two premises: *quorum intersection*, for any two sets in Q_1 some set in Q_2 lies inside both; and *at most one parent* per hash. A *state* says, for each validator, target hash h, target epoch t and source epoch u, whether that validator voted the link (h, t, u). A link from p at epoch u to h at epoch t is *justified by* q when $q \in Q_1$, every member of q voted (h, t, u), h is the $(t - u)$-th descendant of p, and $t > u$. Genesis is justified at epoch 0, and a block reached from a justified block by a justified link is justified. Block h is *finalized* at epoch v with child c when c is a child of h, h is justified at v, and the link from h at v to c at $v + 1$ is justified. A *finalization fork* is two finalized blocks $h_1 \neq h_2$, neither an ancestor of the other. A validator is *slashed* when it voted two different targets with the same target epoch (a double vote), or voted links (h_1, t_1, u_1) and (h_2, t_2, u_2) with $t_1 > t_2$ and $u_2 > u_1$ (a surround vote). The state is *quorum-slashed* when some set in Q_2 consists of slashed validators.

Theorem 14.1 (Accountable safety)**.** In every setting (so under quorum intersection and at most one parent), every state with a finalization fork is quorum-slashed.

Proof. Let the fork be h_1 finalized at v_1 and h_2 at v_2. If $v_1 = v_2$, the two links into epoch $v_1 + 1$ are each voted by a set in Q_1; quorum intersection gives a set in Q_2 that voted both; their targets differ, since equal children would give h_1 and h_2 the same child and so, by at most one parent, $h_1 = h_2$. That set double-voted. If $v_1 \neq v_2$, say $v_1 < v_2$, walk down the chain of justified links that justifies h_2, by strong induction on the epoch gap. Every block on that chain is an ancestor of h_2, so none descends from h_1. Some link in the chain starts at or below v_1 and lands above v_1. If it lands at $v_1 + 1$, it and the finalization link of h_1 have distinct targets at that epoch, and the intersection set double-voted. If it starts at v_1, its source is a justified block at v_1 other than h_1, and the links that justified those two blocks give a double vote the same way.

Otherwise it starts below v_1 and lands above $v_1 + 1$, so it surrounds the finalization link of h_1, and the intersection set made a surround vote. □

Checked as `accountable_safety` (CasperFFG.v), a transcription of the casper-proofs Core/AccountableSafety file at a pinned commit with proofs redone on the standard library; validator sets are predicates, and both premises are fields of the setting record, so they are hypotheses of every theorem. The record reading is `conflicting_records_are_priced` (CasperRecordReading.v).

Theorem 14.2 (The premise can hold, and the write is free)**.** (1) In the three-validator setting where each validator has stake 1, Q_1 is the sets of stake at least two thirds and Q_2 the sets of at least one third, the state where A and B vote one branch and B and C the other has a finalization fork; B is slashed, A and C are not, and $\{B\}$ is in Q_2. (2) In the one-validator chain setting, one vote finalizes genesis and no validator is slashed.

Proof. Both are computations on fixed finite data. In (1) B voted two different targets at epoch 1, which is the double vote; A's and C's votes are one link at epoch 1 from source 0 and one at epoch 2 from source 1, which neither double-vote nor surround. □

Checked as `casper_fork_exists` and `casper_fork_slashable` (CasperForkWitness.v), and `finalization_without_slashing` (CasperRecordReading.v).

One thing that means for the record itself: finality is revocable at a price, the price is named, and a final block can hold a false statement.

And the obvious next question: is the 2017 design what Ethereum actually runs? Partly, and the paper's own title is the clue. It's called *Casper the Friendly Finality Gadget*, and its first sentence says it's a finality system that *overlays* a chain still choosing its blocks by proof of work. A gadget, bolted on top. In 2020 a larger group, Buterin among them, wrote up *Gasper*, which joins Casper's finality to a fork-choice rule called LMD GHOST, and on 15 September 2022, in the switch people called the Merge, Ethereum stopped choosing blocks by proof of work at all and went over to proof of stake. So the checkpoints and the two slashing rules above are one part of a system that holds real money today, and the transcription is of that part's safety proof as the 2017 design states it. Everything between the two, the fork choice and the rest of the running protocol, is the gap between my transcription and the running system, which I walk through near the end of this section with a photocopied recipe and a kitchen. Does any of that move the price? No. Finalizing is still free for an honest validator. What costs is contradicting yourself.

14.2 Gas: the one that charges a fee at the write

Every operation on Ethereum costs *gas*, paid in ether by whoever sent the transaction. The yellow paper, Ethereum's formal specification, gives the reason in one line: every computation is fee-bearing so the network can't be abused and halting questions are sidestepped. The record I care about is a nonzero slot in the shared storage, which every full node in the world (every computer keeping its own complete copy of the chain) has to keep until it's cleared. Clearing a slot is allowed, and the refund rules below exist to reward it.

And here it is: writing a fresh storage slot from zero to nonzero costs twenty thousand gas, by the table in the paper's appendix, plus twenty-one hundred for touching a *cold* slot, one the transaction hasn't touched yet (after the first touch it's *warm*). As I read it, the point is to discourage storage growth, since storage grows every node's database. That's a fee on the write, at the moment of the write, and it's the only fee in the five. Gas has a budget too, in the sense from the start of the section: every block has a gas limit, and the gas its transactions use can't exceed it, so only so many writes fit in a block.

Where does a number like twenty-one hundred come from? Somebody picked it, so the obvious next question is who, and why, and the answer has a date on it. In September 2016, just as Ethereum's developers were gathering in Shanghai for Devcon 2, their developer conference (the third one, since they started counting at zero), attackers started filling blocks with an operation that was cheap in gas and expensive in fact: it sent every node off to read data from its disk, and nodes fell behind the chain. The first repair, a fork that October called Tangerine Whistle, raised the prices of the operations the attack had leaned on. The cold-slot charge came later, in a proposal by Vitalik Buterin and Martin Swende called EIP-2929, switched on in the Berlin upgrade of April 2021, and the motivation it gives is still those 2016 attacks. So twenty-one hundred isn't a law of nature or a measurement of anything deep. It's a scar: a price set after somebody found the place where the old price was wrong. That doesn't change the argument; the write is still priced at the moment of the write. It does change how I read the rest of that table: some of its numbers, at least, are somebody's answer to "what did this cost last time," which is a record of a different kind.

Now the leak, and it's Section 6's leak. Set a slot and then clear it *in the same transaction* and you get a refund. Count it through: first write twenty-two thousand one hundred; clearing it costs a hundred and credits a refund of nineteen thousand nine hundred; net, twenty-three hundred. That's `revoked_write_nearly_free`, and `persistent_write_priced` is the other half: leave it set and you pay at least twenty thousand. The write is priced. The *revoked* write nearly isn't. But I have to add what

the fragment leaves out, because twenty-three hundred on its own misleads: the realized refund is capped at one fifth of the gas the whole transaction used (that's the 2021 change from Section 6, the one that ended the refund-hoarding token's advantage). So a real set-then-clear pays a lot more than twenty-three hundred. How much more? The cap is a fifth of everything the transaction used, and that includes the twenty-one thousand every transaction pays just for existing. Take a transaction that does nothing but set and clear the slot: it uses 43,200 gas, so the cap is 8,640, so it pays 34,560, which is 13,560 on top of the base. (Look at the slot alone and forget the base, which the rule doesn't do, and you'd get 17,760.) Either way it's far above twenty-three hundred. The counter-level net is twenty-three hundred. The bill is not.

Here's the arithmetic the way the specification has it, so you can redo it with a pencil.

Definition 14.2 (One slot in one transaction)**.** A slot run records the slot's value at the start of the transaction (original), its current value, whether it has been touched (warm), the gas charged so far, and the refund counter. A store of a new value charges 2100 if the slot is cold, plus 20000 if the slot still holds its original value, the new value differs, and the original is 0 (2900 if the original is nonzero), and otherwise 100. It adjusts the refund counter by the specification's rules; in particular, when the new value equals the original value 0, the refund grows by $20000 - 100 = 19900$. The run starts fresh: original and current 0, cold, gas and refund 0. The *net* charge is gas minus refund.

Theorem 14.3 (Pricing the persistent write and the revoked one)**.** (1) For every list of stores from a fresh slot, if the slot ends nonzero then the net charge is at least 20000. (2) Storing 1 then 0 leaves the slot at 0 with net charge 2300; storing 1 alone has net charge 22100.

Proof. (1) By induction on the list, with the invariant: while the original value is 0, the net charge is at least 0 when the current value is 0 and at least 20000 when it is not. Each store keeps it, which is a finite case split on whether the slot is warm and which of the original, current and new values are equal. A fresh slot satisfies it. (2) Direct computation: $2100 + 20000 = 22100$ for the first store, 100 for the second, refund 19900, and $22200 - 19900 = 2300$. □

Checked as `persistent_write_priced` and `revoked_write_nearly_free` (EVMStorageGas.v), transcribed from the execution specification's storage instruction and gas table at a pinned commit. The end-of-transaction refund cap of one fifth is not in the fragment.

The fragment: one slot, one transaction, the gas and refund arithmetic transcribed from the Ethereum execution specification at a named commit. No stack, no call context, no other slots, and *no refund cap*; the file says so.

Then the wrapper, which is a different kind of object, and the walk near the end is about exactly that difference. It's a *gas schedule* in the abstract: any states, any steps, a cost per step, a yes/no commitment flag, and a charge predicate saying which steps get billed. It formalises no EVM (the Ethereum virtual machine) at all. It is Section 7's result with the whole-run ceiling, the one that pins the amount as well as which steps pay, under a new name, and the file says that too. Its two halves are worth reading separately. Leave one committing step unbilled, and a one-step run commits for nothing (`undercharged_opcode_admits_free_commitment`), so the floor fails outright; one missed commit is enough. Bill one step that doesn't commit, and the schedule charges more than the number of commits on that one-step run (`overcharge_breaks_exactness`), so the ceiling fails. And the class isn't empty: in any system, the bill that charges one exactly on a step that raises the flag and nothing on any other meets both halves (Section 7's step price).

And the wrapper itself, which is the cleanest thing in this section and has the least Ethereum in it.

Definition 14.3 (A gas schedule)**.** A *gas schedule* has states, instructions, a step, a cost $c(s, i) \in \mathbb{N}$ that may depend on the state, a Boolean commitment flag on states, and a Boolean charge predicate $\chi(s, i)$, with two premises: a charged step costs at least 1, and an uncharged step costs 0. A step *commits* when it takes the flag from false to true. For a run t from s, let $c(t)$ be its total cost and $n(t)$ its number of committing steps. The schedule has the *floor* when $c(t) \geq n(t)$ for every run, and *no overcharge* when $c(t) \leq n(t)$ for every run. It has *unit pricing* when $c(s, i) = 1$ if $\chi(s, i)$ and 0 otherwise.

Theorem 14.4 (Gas schedule exactness)**.** For every gas schedule: it has the floor and no overcharge if and only if $\chi(s, i)$ holds exactly when the step from s by i commits, and it has unit pricing. Moreover, (1) if some committing step from an uncommitted state is uncharged, that one-step run costs 0 and the schedule fails the floor; (2) if some charged step does not commit, the schedule overcharges.

Proof. The equivalence is Section 7's characterization with the whole-run ceiling, under another name. From right to left, each step's cost is 1 on a committing step and 0 otherwise, so a run's cost counts its commitments. From left to right, apply the floor and no overcharge to one-step runs: a committing step must cost at least 1, so it

is charged; a charged step costs at least 1 and may not exceed its commitment count, so it commits and costs exactly 1. Parts (1) and (2) are those one-step runs read the other way. □

Checked as `gas_schedule_exactness`, which proves both directions, `undercharged_opcode_admits_free_commitment`, `undercharged_opcode_breaks_certification_floor` and `overcharge_breaks_exactness` (GasMetering.v); `gas_schedule_exactness` is `exact_commitment_pricing_characterization` (CommitmentPredicateAdequacy.v) carried over unchanged.

Gas prices storage growth and computation. It says nothing about whether what's stored is true.

14.3 Attestation: hash chains, and a quote about the past

A trusted platform module, the small chip on most laptop motherboards, keeps a handful of registers that can only be *extended*: the new value is a hash of the old value together with a digest (a short hash) of whatever software just loaded. You can't set them; you can only fold more in. A *quote* is the chip signing those register values together with a number the asker chose, a *nonce*, so the asker knows the quote is fresh. Intel's SGX does the equivalent for an *enclave*, a walled-off region a program runs in. The record is the extended register.

What's priced? On the registers, nothing in the specification: no fee, no bond, no budget. Reversal isn't expensive. It's *infeasible*: you'd have to invert a hash or steal a key. That's the difference between a price and a wall, and it's easy to blur, which is why the definition at the start of the section leaves walls out. What stands behind a cost can come in three currencies. It can be economic, a fee or a deposit the protocol takes (Casper and gas are here). It can be cryptographic, a reversal made infeasible by a hardness assumption, which makes it a wall with no exact price on it (attestation and transparency are here). Or it can be physical, energy spent or hardware worn, which none of these five specifications charges as a fee or a bond; the nearest thing is the chip's write budget, below. And the record's permanence is only as long as the register lives: a platform's registers are cleared when it resets, so the permanence is per boot.

Why the registers, then, when something that lasts would make a better record? Because the quote is what a remote asker reads, and the registers are what it signs. The same chip has longer-lived records, and they're closer to the cup than anything else in this section. Version 2.0 of its specification has storage slots of a kind it calls a bit field, changed only by a command that ORs new bits into the old value: a bit can be set and never cleared, which is $r \vee h$ from Section 10, on a laptop motherboard. It has counters that can only be incremented, and a counter's first increment starts

it at the largest value any counter on that chip has ever held, so deleting one and making a new one can't wind it back. And those slots are rationed. Bryan Parno and his coauthors, building a system called Memoir in 2011, noted that the specification of the day only required a counter to be able to tick once every five seconds, and that the chip's storage was expected to survive about a hundred thousand writes in its whole life, so writing once a second would wear it out in under twenty-eight hours. Memoir writes to it twice per boot. Permanent, finite and rationed: the permanent record has a budget, in exactly the sense from the start of the section, and it sits on these slots and counters; the registers the quote signs have none. It's the budget of Theorem 10.11's chain of k-bit words, which can only climb k times. Phones and small controllers go one floor further down, to fuses like the PROMs in Section 10: a version number kept as a row of fuses, one more blown at each security update, and boot code that refuses anything older. An industry guide on it, Rob Smart's 2021 explainer for PSA Certified, a security certification scheme for chips and devices, tells designers to make sure there are enough fuse bits to last the number of updates expected over the device's lifetime, which is part (5) of that same theorem, written as a design rule.

What all of this is for is *rollback*. Memoir's example is a password checker restored to an old saved state to undo a user's password change, and in 2017 a system called ROTE did the same job for Intel's enclaves by having several machines vouch for each other's counters, with the stated result that the only way left to break it is to reset every one of them to its start. In 2016 Raoul Strackx and Frank Piessens's Ariadne got the cost of that job down to flipping a single bit of non-volatile memory per state update, which they call the theoretical lower limit. Notice who the attacker is in every one of these: whoever gets to put an old state back. A bad program doesn't come into it. A record kept by the step can't see that attacker, which is why every guarantee in Part IV about a record's earned history is stated from a clean start, and a restored start is outside all of them. The per-step toll still holds after a restore. It just can't tell you what happened before it.

Replicated services put the same kind of record somewhere else: inside each replica, where a replica that's been taken over still can't make it say two different things. Byung-Gon Chun and coauthors' attested append-only memory (A2M, 2007) is a log a replica can only add to; Dave Levin and coauthors' TrInc (2009) shrinks that to a trusted counter; MinBFT (Giuliana Veronese and coauthors, 2013) uses a trusted counter to reach agreement with $2f + 1$ replicas when f of them can lie, where the usual count is $3f + 1$; and Damysus (Jérémie Decouchant, David Kozhaya, Vincent Rahli and Jiangshan Yu, 2022) has a small trusted service, the Checker, that keeps a monotonic counter of where the protocol is. Allen Clement, Flavio Junqueira, Aniket Kate and Rodrigo Rodrigues (2012) worked out, with proofs, what such a

record buys, and as I read their result it's a good warning for this book. On its own, stopping a replica from saying two different things doesn't lower the $3f + 1$ bound in an asynchronous network at all. Add signatures that a third party can check, and it turns any protocol that survives crashes into one that survives liars, with no extra replicas. The way I read it, a record helps the others only as far as they can check what it says, which is Section 7's point from the other side.

There's a fourth currency under those three, and it's the one every permanent record pays in. It's the logical kind: Section 4's room of chairs applies to any of these records. On a finite state space, a step that switches a record on somewhere, and never switches it off anywhere, forgets something. Forgetting alone doesn't single out a record write: any step that overwrites a field able to hold two values forgets something too. What singles it out is where the forgetting lands. Take the states the record already reads yes on, together with the one the step switches on. The step sends every one of them to a yes state, and it can't keep them apart, so what it destroys is a distinction on the record's own yes side: two states it already vouched for, or the state it just raised and one that was up already. Any overwrite merges states somewhere; a permanent write is forced to merge them on that side. That needs no premise about money, keys or heat. The economic, cryptographic and physical payments each need a premise of their own about their kind (a fee rule, a hardness assumption, Landauer's principle), and they hold exactly where that premise holds.

Theorem 14.5 (A permanent write forgets on its own yes side)**.** For any states S, instructions I, step and Boolean reading: suppose a duplicate-free list names every state, the instruction i never turns the reading from on to off, and i turns it on at some state s. Then the step by i is not injective. More: the step by i sends every state that reads on, and s, to a state that reads on, and it is not injective even on those states.

Checked as `permanent_write_has_logical_payment` (PricingPhysicsAudit.v), from `permanent_at_flip_is_not_injective` (PermanentRecordPricing.v); the proof is Theorem 4.1 with permanence required only of the one instruction i. The second half is `permanent_flip_merges_on_yes_side` (PermanentRecordPricing.v): if the step were one-to-one on the states that read on together with s, it would map the states that read on onto themselves, and then s would land where some state that reads on already lands.

What the quote can do is say what was *measured*. It can't say the measured code is correct, and it can't say anything about runtime state that was never folded in. A correctly measured buggy program attests beautifully. The fragment proves that exact limit: one register, a hash about which nothing is assumed, a quote as a pair,

and no function of the quote decides a runtime flag the quote ignores (`quote_cannot_attest_unmeasured_state`), which is Section 7's cube again. And a companion shows that a sign-and-verify interface *alone* gives no authenticity (a scheme where verify always says yes passes the interface), so a cryptographic premise has to be added; the fragment doesn't model signatures, formats, or a freshness check (the nonce is just a field the verifier fixes).

The same limit, turned outward, is the honest edge of the opening chapter's four conditions. Every one of them is about a thing the machine holds: same property, same thing, same edition, honest check. For a claim about the world (the oven is hot, the parcel arrived) there's no edition the machine controls. Whatever reads the world is a window onto it, and "nothing changed between the check and the commit" stops being something a step can enforce and becomes a premise about the sensor. So a record can establish claims about the machine's own state, and claims about the world only through named premises about whatever measured them. A correctly recorded sensor reading certifies the sensor.

Definition 14.4 (Platform and quote)**.** Fix any functions $H : \mathbb{N} \times \mathbb{N} \to \mathbb{N}$ (the extend hash) and $\delta : \mathbb{N} \to \mathbb{N}$ (the digest); nothing is assumed about either. A *platform* is a measurement log (a list of numbers) and a Boolean runtime flag. Its register is the log folded from 0 by H. The *quote* with nonce n is the pair $(\delta(\text{register}), n)$.

Theorem 14.6 (The quote cannot attest unmeasured state)**.** For every nonce n there is no function D from quotes to Booleans with $D(\text{quote}_n(P)) = \text{flag}(P)$ for every platform P.

Proof. Take two platforms with the empty log, one flagged true and one false. Their quotes are equal, since the quote never reads the flag, so D would have to return both values on one input. □

Checked as `quote_cannot_attest_unmeasured_state` (TPMQuoteGap.v). In the same file, `quote_decides_measured_claims` says any Boolean claim about the retained digest can be read off the quote. The companion is `tpm_interface_authenticity_refuted` (TPMQuoteAuthenticity.v): a scheme whose verify always says yes satisfies the sign-and-verify interface while accepting a signature no key produced.

14.4 Certificate transparency: visible, unpriced

The certificates that make your browser show a padlock are issued by hundreds of authorities, and a bad or compromised one can issue a certificate for a domain

it shouldn't. Certificate Transparency makes authorities submit every certificate to public append-only logs built as Merkle trees, so that anyone can check that a certificate was logged and that the log never rewrote its past. It was specified in RFC 6962 in 2013 and revised as RFC 9162 in 2021. I went looking for which one the running logs follow, and it's still 6962: Mozilla's reference for web developers says browser implementations are based on 6962, and as late as November 2025 Chrome's team was only proposing to drop its rule that every certificate carry a timestamp from at least one 6962 log. The fragment I transcribe is 9162's. A Merkle tree is Section 3's hash chain folded into a tree: hash each certificate (those are the *leaves*), hash the leaf hashes in pairs, hash those in pairs, and so on up to one *root* hash. To prove one leaf is in, the log hands you only the few hashes sitting beside the climb from that leaf to the root, the *path*, and you redo the climb yourself. A second check, *consistency*, shows that a bigger tree is a smaller one with leaves added on the end and nothing rewritten. The record is a leaf in the tree.

The data structure has a history of its own in security. The computer scientists Scott Crosby and Dan Wallach, at Rice University, built tamper-evident logs on the same kind of tree in 2009, with a proof that an entry is in the log and a proof that a later version of the log extends an earlier one, each a logarithmic number of hashes; a logger that rewrites its past gets caught by the second.

Priced? Nothing. The RFC says it in its first section: logs let interested parties *detect* misissuance afterward. They don't *prevent* it. And a log that shows different views to different people (a *split view*) can defeat the auditing, so logs are still trusted parties. What it buys is visibility.

Which raises the obvious question: if nothing's priced, why does any authority bother logging? Nothing in the RFC makes it. The answer is a browser. Chrome's team set the rule, enforced from Chrome 68 in July 2018, that every certificate issued after 30 April 2018 had to meet its transparency policy, and a publicly trusted certificate that doesn't is simply not trusted by Chrome: the visitor gets a full-page warning in front of the site. So there is a price after all. It isn't anywhere in the specification. It lives in one vendor's browser, and it's the price of being shown to that browser's users at all. Is that the bottom? Not quite, because it means the strongest lever on this record is a policy one company can change, which is a fact about who holds the stamp. The tree has nothing to do with it. That policy changes nothing in the table at the end of this section, which asks what each specification prices. But it's worth keeping two sentences apart: "the specification prices nothing" and "nothing is priced."

Do the inclusion proof by hand, because it's the most satisfying ten minutes in this section. Take the RFC's own seven-leaf example tree and verify that leaf d3 is in it from the path the log hands you: three hashes and a rule about which side to

hash on, depending on whether your position is odd. Three hashing steps and you land on the root. The checked work runs the RFC's own examples (`rfc9162_example_inclusion_d3` and its siblings, including the awkward lone-leaf case) through a transcription of the RFC's verifier control flow with a symbolic hash. Verifier logic only: no signatures, no gossip (auditors comparing the views they were shown), no misbehaving log.

Definition 14.5 (Inclusion check, RFC 9162 section 2.1.3.2)**.** Given a leaf index m, a tree size N, a leaf hash, a root hash and a path, the check rejects unless $m < N$. Otherwise it walks the path from the leaf with two counters starting at m and $N - 1$, hashing each path element on the left or the right according to the counters as the RFC specifies and halving the counters, and accepts when the second counter ends at 0 and the result equals the root. A *symbolic hash* makes a leaf hash $\ell(x)$ and a node hash $\nu(a, b)$ free constructors, so two hashes are equal only when they are built the same way.

Theorem 14.7 (The RFC's own examples pass)**.** With the symbolic hash, in the RFC's seven-leaf tree with leaves $d_0, \ldots, d_6$ (writing d_k also for its leaf hash $\ell(d_k)$), the inclusion check accepts leaf 3 with path $[d_2, \nu(d_0, d_1), \nu(\nu(d_4, d_5), d_6)]$, and likewise leaves 0, 4 and 6 with their paths; the consistency check from size 4 to size 7 accepts. For every input with $m = N$, the inclusion check rejects.

Proof. The examples are evaluation of the transcribed functions on constructor terms. The boundary case is the first test, $m < N$, failing. □

Checked as `rfc9162_example_inclusion_d3`, its siblings `rfc9162_example_inclusion_d0`, `rfc9162_example_inclusion_d4` and `rfc9162_example_inclusion_d6`, `rfc9162_example_consistency_4_7`, and `rfc9162_inclusion_boundary_safe` (RFC9162Merkle.v). The append-only history is list append, and `ct_extension_preserves_entries` says appending keeps every old entry.

14.5 Proof-carrying code: no price, no permanence

Necula's idea from Section 8: ship untrusted code with a proof that it obeys the host's policy; the host computes what would need to be true and checks the proof with a small checker. The record is the checked proof, and it's held only for the moment of loading. Priced? No. Checking is cheap by design, which is the whole point, and the producer paid to build the proof off the host. Permanent? No; nothing's written. The fragment is a three-instruction memory-bounds language with the policy "every

address is below the limit": the checker accepts exactly when the condition holds, and a certificate implies the condition. One example is checked on top of that: a program that reads address two under a limit of two is rejected. Not the paper's real case study, and no bridge to anything else in this book. And its limit: a program that writes garbage to address zero passes if zero is below the limit. The proof is of the stated policy and nothing else.

And what was the paper's real case study, the one my fragment isn't? Network packet filters: the little programs an operating system runs on incoming network traffic to decide which program gets to see which packet. They run inside the kernel, the most trusted part of the system, and they have to be fast, so they're exactly the code you'd most like to take from a stranger and least like to trust. In the 1996 paper that first set out the idea, Necula and Peter Lee wrote several filters by hand in the assembly language of a DEC Alpha, shipped each with its proof, and reported a one-time cost of 1 to 3 milliseconds to check the proof, after which the filter ran with no checks at all. That's "checking is cheap by design" with a number on it. It changes nothing about my fragment, which still isn't that case study, and it doesn't add a price: the milliseconds are the checker's work, and nothing is being written down for them to be a toll on.

Definition 14.6 (The bounds fragment)**.** Instructions are READ a, WRITE a v, and HALT. For a memory limit L, the policy allows READ a and WRITE a v when $a < L$, and always allows HALT. The *verification condition* of a program is that the policy allows every instruction in it. A *certificate* is a list of per-instruction policy facts, one for each instruction in order. The *checker* tests the policy on each instruction and returns the conjunction.

Theorem 14.8 (The checker is the verification condition)**.** For every limit L and program P: the checker accepts P exactly when the verification condition holds; any certificate for P gives the verification condition; and with $L = 2$ the checker rejects READ 2 followed by HALT.

Proof. The first two are induction on the program and on the certificate, one instruction at a time. The third is a computation: $2 < 2$ fails. □

Checked as `pcc_checker_accepts_iff_vc`, `pcc_certificate_implies_vc`, and `pcc_unsafe_program_rejected` (NeculaPCC.v, definitions in NeculaPCCTarget.v).

14.6 One example, three levels

Before I lay all five side by side, let me walk one of them up and down the three levels from the start of the section, because every claim in here lives at exactly one level and it's easy to let a claim slide up a floor without noticing. Gas, since it's the one place all three line up. Think of a restaurant's real recipe, my photocopy of one page of it, and a cookbook chapter on what any recipe has to contain. I can check my photocopy line by line. That tells you nothing about what the kitchen cooks tonight.

The real specification. Ethereum's execution specification says, in a document that runs real money, what a storage write costs, what clearing it refunds, and how the refund is capped at the end of the transaction. None of what I checked is at this level. What the network actually charges is whatever the clients people run actually compute, and the evidence for that is the clients and the chain. This book has none of it.

The transcription. I copied one mechanism out of that document, one slot in one transaction, gas and refund arithmetic only, at a named commit (Definition 14.2, Theorem 14.3). What the result tells you: *if* the storage rule is what I copied, then leaving a fresh slot set costs at least twenty thousand, and set-then-clear nets twenty-three hundred at the counter. What it can't tell you: anything about the refund cap (not copied, which is why the bill came out so different from the counter earlier), the stack, calls, other slots, or whether any client implements the rule as written. The boundary of the claim is the edge of the copy.

The wrapper. The gas schedule of Definition 14.3 isn't Ethereum. It's any system with steps, a cost per step and a yes/no commitment flag, plus one thing *stipulated*: a charge predicate, with the premise that a charged step costs at least one and an uncharged one costs nothing. That premise is the whole of what the wrapper adds, and it's added by hand; no specification supplies it. Given it, the theorem says the bill is exact, never letting a commit go free and never billing a non-commit, exactly when the charge predicate is the commit predicate at unit price. What that tells you is what exact pricing would have to look like, in any system. It doesn't say Ethereum meets it, and Ethereum doesn't: real gas bills computation that commits nothing, which the transcription shows in numbers. So the wrapper prices a definition, the transcription prices a copied rule, and neither prices the chain.

The other four sit on the same three levels, and only one more wrapper is spelled out in this section. Casper's transcription is the accountable-safety proof above; its wrapper is a small gadget with the stake at risk handed in. A finalize step that risks nothing turns the record on for free, which breaks Section 3's toll (the file calls that free forgery), and one that risks at least one unit has a floor (`nothing_at_stake_is_free_forgery`, `slashing_finality_floor`, PoSFinality.v). It records exposure to loss

(no collateral is destroyed in it), has no Casper checkpoints in it at all, and is priced by stipulation, the same way the gas wrapper is. Attestation, transparency and proof-carrying code have transcriptions that prove what their verifiers do and no price theorem, because their specifications carry no price. Their wrappers, which I don't spell out here, are Section 7's verifier argument told in each system's vocabulary, with a colliding pair behind each one, and that argument is all they carry here.

How to knock down the four claims with the most weight on them, since the book's rule is that every claim gets a way to break it. For the gas transcription: a list of stores from a fresh slot that ends nonzero with net charge under twenty thousand. For the gas wrapper: a schedule that never lets a commit go free and never bills a non-commit, whose charge predicate still differs from the commit predicate at some step. For the finality gadget: a finalizing trace from unfinalized to finalized whose charged stake is zero while the gadget claims a floor. For the attestation fragment: a verifier reading only the quote's fields that decides a runtime flag the quote ignores.

14.7 The same table, five ways

Lay them out by the one definition of price from the start of the section. Casper puts a bond behind the write and takes it on *reversal* (slashing). Gas takes a fee at *the write* (and refunds the revocation, capped), and rations writes with a gas limit per block. Attestation prices *nothing* on the registers the quote signs, and puts a budget on its longer-lived slots and counters. Transparency prices *nothing*. Attestation and transparency make reversal infeasible or visible. Proof-carrying code prices nothing and keeps nothing.

System	Fee at the write	Bond	Budget	What the transcription checks
Casper finality	none	the deposit, taken on a contradicting vote (slashing)	none	a finalization fork means a slashable third
Gas	20000 for a fresh slot, refunded on revocation	none	the gas limit of each block	a persistent write to an empty slot nets at least 20000
Attestation	none; reversal is infeasible	none	none on the registers; a write budget on the non-volatile slots and counters	the quote can't decide what it never measured

System	Fee at the write	Bond	Budget	What the transcription checks
Transparency	none; misissuance is visible	none	none	the RFC's own inclusion examples pass
Proof-carrying code	none, and nothing is kept	none	none	the checker is the verification condition

Each row proves what its own transcription says. Reading a row as a claim about a deployed system takes evidence from that system, and no row shows that certification is the one event every model has to price.

So the thing my toy charges for, the moment the record is written, is charged a fee in one of the five specifications I read, and in that one the designers had to patch the place where the charge could be clawed back. I find that neither a vindication nor a refutation. It's a fact about what people built, and what they built was mostly walls (hash chains, signatures) and penalties for contradiction. Tolls on commitment were rare. Whether a toll on commitment is a *better* design than a wall is not something I can prove, and I won't pretend the five say so. What I read in them, and it's a reading of five fragments I chose, is that the record axis is real in each of them (each has a bit that goes one way) and that the price axis is mostly empty. With one definition of price fixed before the survey, that last part is a count: of fifteen cells in the table, four are filled (one fee, one bond, two budgets), and only gas takes a payment at the moment of the write.

That leaves the question Section 10 raised and Section 15 has to face. Five systems, five different events chosen as "the record": finalized, committed, attested, logged, checked. Is there anything that picks out one event as *the* one worth recording, or is it a free choice every time? I have a guess. It's a thesis, and I'll show you how to kill it.

15

The pointer thesis

Here's where I am. Over any base, a record that answers to the computation and never comes down is a latch on an event (Section 10), and the event is a free choice (Section 10 again, the two cups on one die). Five real systems each picked a different event (Section 14). The toll in this book sits on certification. Why that one? I don't have a proof. I have a thesis, in the sense the Church-Turing thesis is one (Section 2): a claim that ties notions I can define to notions about the world that no definition reaches, so no proof can settle it. The honest shape of a thesis is: here's the claim, here's where I got the idea, here's what's checked, here's what isn't, and here's how to kill it. In that order.

15.1 The claim

Of all the events a machine could put on its meter, the ones its price list charges for and its counter adds up, certification is the one whose records get *copied and relied on by parties who didn't produce them*: checked, stored, and acted on across independent places that share no other state. Copying alone can't be the test. A replicated system copies everything it holds to every node, the gas used in a block header as faithfully as the word final, so rival events get copied too. Nobody who didn't produce them acts on them. If that's true, the choice of certification stops being free. It's the one event the world around the machine keeps redundant copies of and leans on, and that's a reason to price it and leave the others unpriced. So the thesis has two halves, and the ways to kill it at the end aim at both: certification is that event, and where a deployed discipline puts a price on writing a record, the record it prices is one that's copied and relied on in that sense.

There's a second handle on the same question, and it comes from Section 4. The permanent-certificate theorem never uses the word certification for anything special: given Section 4's rule that forgetting is charged, it prices any yes-or-no reading that, once on, stays on, on a machine with finite memory. So it points at a narrower candidate: the events to price are the ones that write a record no later step can take back. Certification is one of them. As an exact answer that candidate is false, because a merging step can also revoke (Section 4's three cards). Whether the records that are copied and relied on are the permanent ones, and whether being copied and relied

on picks out anything the merge criterion doesn't, is what the two handles have to settle between them.

Two holes in that, which I can see from here, so you don't have to find them for me. The first is a missing premise. Even if the world copies certification records more than any other kind, "copied" doesn't give you "priced" on its own. Something has to connect them, and the best candidate I have is a premise about incentives: a record that many independent parties copy and lean on is exactly the record worth forging, so it's the one where a price buys the most protection. That's a reasonable premise. It's also a premise. The thesis's second half states the connection as a claim to be tested; it doesn't explain it, and nothing in this book supplies the explanation. The second hole is a circle. If "certification" just means "the record other people rely on," then of course it's the one that gets relied on and copied, and the claim says nothing. So the event has to be picked out without mentioning reliance: here it's a reading going up for good right after a check of the same claim passed, which you can find in a run without asking who reads it. Whether the world copies *that* event more than its rivals is then a real question, and it could come out no.

That's the whole thesis. Now the rest of what a thesis owes you.

15.2 Where the shape comes from, and how much of it survived

The shape is from physics, and I want to tell you where exactly, because I borrowed a word and I didn't get to keep the whole thing.

Quantum mechanics has a famous problem: a system can be in a blend of states, and when you measure it you find one definite state, and nobody agrees on what "measuring" did. In 1981 Wojciech Zurek asked a sharper question: of all the ways a measuring device *could* end up, why does it end up recording one particular kind of quantity (the position of a pointer on a dial, say)? Why never some blend? His answer, developed over twenty years and summarised in a 2009 paper called "Quantum Darwinism," is that the *environment* decides. The air, the light, the table, everything the device touches, is constantly interacting with it, and only certain states of the device survive that interaction; those are the *pointer states*. And more: the environment selects them, and then it *copies* them. The light bouncing off a dial carries the dial's reading to every corner of the room, redundantly, so that a thousand observers can each read a different scrap of the environment and all agree on where the pointer is, without touching the dial. Objectivity, on this view, *is* redundancy: a property is objective when many independent fragments of the world each carry it.

Zurek isn't the only one behind it. Fernando Brandão, Marco Piani and Paweł Horodecki (2015) published a proof that, as I read it, makes something like it generic: under any quantum dynamics at all, observers who learn about a system only

through its environment get at most classical information, about one and the same measurement of the system. That's their theorem behind the word "objective," and it's a statement about separate fragments, which is exactly the part I didn't keep. Xiao-Liang Qi and Daniel Ranard (2021), as I read their paper, sharpened it into a statement about every possible evolution of system and environment: everywhere in the environment outside a region of fixed size, whatever an observer can learn about the system is classical, read off one fixed measurement. A statement about every evolution and every input is the quantum cousin of what the definitions below ask of an observer, that it record the event in every state, including ones that never happen to turn up.

That's the shape I borrowed. Certification is to a machine's ecosystem (the machine plus everything around it that reads, stores and relies on its records) what the pointer reading is to the room: the thing that gets copied everywhere. It's a good shape and I think it's the right intuition. Here's what survived into the checked work, and I'm going to be exact because the word "redundant" is doing less than it does for Zurek.

Zurek's redundancy is *graded* (each fragment carries almost all the information, with some small loss) and it requires the fragments to be *disjoint*: different observers reading different parts of the environment. The checked definition is the zero-loss case with *no disjointness at all*. An ecosystem is a state type, some number of observers, and one bit per observer; an observer *records* an event when its bit is true exactly when the event holds; an event *proliferates* when every observer records it. Nothing in that says two observers draw on separate parts of the state. They could all be reading the same bit. The file says so itself. So I kept the word and dropped the independence that makes Zurek's version mean "objective." That's a gap between the thesis's prose and that first formal object, and the prose is where the thesis lives. The definitions under the thesis below close part of the gap: they define independent carriers, and the definition turns out to force something (Theorem 15.1).

The independence I dropped is exactly what the thesis's closest published neighbour keeps. Jess Riedel, Zurek and Michael Zwolak (2016) took histories of a closed quantum system, the story a whole sequence of events tells, and asked which of them the world keeps records of in many separate pieces of the environment. As I read their paper, redundant records in separate pieces make a history consistent, so ordinary yes-or-no reasoning about it works, and they pick out a small set of histories from the huge number that are merely consistent: in their words, an objective past. That's "what proliferates is what counts" in quantum form, with the disjointness in it, and it came before this book. Halliwell, in a second 1999 paper, "Somewhere in the universe," had already carried, as I understand him, the link between records and consistent histories from pure starting states to mixed ones, where the records still exist but are necessarily imperfect.

There's a deeper mismatch in the borrowing, and it runs against the book, so it goes here. In Zurek's picture the whole thing, system and device and environment together, evolves by quantum mechanics' rule for closed systems, and that rule never sends two states to one; every step can be run backwards. So, seen from the whole, the environment's copying is just that, copying: the dial's reading gets written onto fresh light, nothing anywhere is forgotten, and it's Door one of Section 4 with the rest of the universe as the endless strip. That's the operation this book says is free, Bennett writing on a blank page. The forgetting shows up only when you look at the dial alone and ignore what the light carried off. Then a blend of pointer readings decays into a plain either-or, and two different blends can decay into the same either-or: two states, one result, a merge in the coarse picture. So the opening chapter's "every settling forgets" fits the dial's own view. From the whole it fails, and the thing the thesis leans on, copying, is the free half of the story. What that does to the thesis: if there's a price, it isn't on the copying. It's on whatever makes the copy final from the dial's side, and the claim above, as written, points at the copy.

A published no-go cuts closer still. Philippe Allard Guérin, Veronika Baumann, Flavio Del Santo and Časlav Brukner (2021) gave an argument that, under assumptions that look natural in quantum mechanics, an observer's own perceptions of her outcome at two different times can't share one reality: no joint chance for the two, depending linearly on the starting state, reproduces what unitary quantum theory predicts for each. A record that persists, for the one holding it, is exactly what gets into trouble once the whole is treated as one-to-one. It isn't a theorem about my machine, which isn't quantum. It's still the strongest published reason I know to hold the word "permanent" loosely when a physicist is in the room.

Which leaves the question to ask of any borrowed shape: has anybody checked the original? For a long time quantum Darwinism was a theory without a lab. Then, in 2018 and 2019, three groups working separately, in Italy, China and Germany, went looking for its fingerprint, as Philip Ball's 2019 report in *Quanta Magazine* tells it. The German one, at Ulm, with Zurek among the authors, watched the spin of a single defect inside a diamond and the carbon atoms around it, and saw the redundancy the theory predicts: the defect's state turned up copied in its surroundings, and it didn't take much of the surroundings before more of them told you nothing new. The other two did it with photons. Is that the bottom? Not by a long way, and the people involved said so. Zurek himself pointed out that the photon experiments make their copies artificially and that the diamond's environment is small, and others noted that all three are schematic versions of a real environment and don't rule out rival accounts. So the shape I borrowed has passed some first tests in a lab, at the scale of a handful of atoms or photons, with the caveats written on it. None of that touches the thesis here, which is about machines and never rested on quantum physics. It

changes the picture: what I borrowed is a lead somebody is checking. It isn't settled, and I don't get to lean on it as if it were.

15.3 What's checked

Definitions, two toys, a theorem about independence, a survey, and two refutations of things that would have been stronger.

The definitions above, plus "unique pointer among rivals": the event proliferates and none of a listed set of rival events does. Uniqueness is *relative to the list*, necessarily, since any event that holds on exactly the same states as a proliferating event proliferates whenever it does: the same fact under another name, so someone has to say which names count as rivals. And the first criterion, as a schema: for every ecosystem in some class, its metered event is the unique pointer among its rivals. The class and the rivals are inputs. Nothing about them is proved. A second set of definitions, after this one, tightens "proliferates" to "copied and relied on" and drops the rival list.

Here's the formal object, small enough to hold in one hand.

Definition 15.1 (Ecosystem, records, proliferates, pointer)**.** An *ecosystem* is a type of states, a number N of observers, and for each observer i a Boolean fragment F_i on states. Observer i *records* an event E (a property of states) when, for every state s, $E(s)$ holds exactly when $F_i(s)$ is true. E *proliferates* when every observer $i < N$ records it. E is the *unique pointer among* a list of rival events when E proliferates and no rival in the list does. Given a class C of ecosystems, and for each member a designated metered event and a list of rivals, the *pointer criterion holds* for C when every member's metered event is the unique pointer among its rivals.

Nothing in the definition makes two fragments read different parts of the state, and "metered" is only a label: the definition has no metering in it.

One bit per observer can't even state the objection a replicated system raises. Every full node of a blockchain holds the whole shared state, so every fact about it, "gas used is at least one" as much as "final," sits on every node. The tightened definitions give each observer whatever it holds, an action it takes on what it holds, and a flag for whether it produced the record.

Definition 15.2 (Copied and relied on, independent carriers, the pointer)**.** An *ecosystem with reliance* is a type of states, a number N of observers, and for each observer i a view V_i (a function from states to some type of things an observer can hold), an action a_i (a Boolean function of the view) and a producer flag. Fix a set R of reachable states. An event E is *copied* on R when for each $i < N$ some Boolean function d of

the view has $E(s)$ exactly when $d(V_i(s))$ is true, for every s in R. Observer i *relies on* E when $a_i(V_i(s))$ true gives $E(s)$ for every s in R, and $a_i(V_i(s))$ is true for some s in R. E is *relied on by parties who didn't produce it* when some observer $i < N$ is not a producer and every such observer relies on E; it is *copied and relied on* when it is both. E is the *pointer* on R when it is copied and relied on, and every event that is copied and relied on holds at every state of R where E holds. An event *a step rule could meter* is one decided by a Boolean function of the state, so a price list can charge exactly the steps that turn it on. The carriers are *independent* under a family of relations P_i (states that agree on observer i's own part of the world) when each view gives equal values on states related by its P_i, and for any two observers $i \neq j$ below N and any states s, t there is a state u with $P_i(u, s)$ and $P_j(u, t)$: the two parts can be set separately.

Two pointers on the same R agree on every state of R, so the pointer is unique with no list of rivals (`pointer_on_unique`). And the definition of independence has a consequence I didn't put in on purpose.

Theorem 15.1 (Independent copies on every state: the event is equivalent at any two states)**.** In any ecosystem with reliance whose carriers are independent and which has at least two observers, an event copied on the set of all states holds at one state exactly when it holds at any other, so it holds at every state or at none.

Proof. Let observers 0 and 1 decide the event by d_0 and d_1. Given states s and t, take u with observer 0's part from s and observer 1's from t. Then the event at s is d_0 of observer 0's view, which is the same at u, so it equals the event at u; and the event at u equals the event at t the same way through observer 1. □

Checked as `independent_copies_everywhere_are_constant` (PointerObservable.v). So separate parts of the world can each carry an exact copy of a real record only on the states the world actually reaches, where the copying made them agree, and that's why the definition asks for copies on a set R of reachable states. Zurek's copies are the same: the light carries the dial's reading because the dial put it there.

Theorem 15.2 (A replicated header: copying doesn't separate, reliance does)**.** Let the states be a block header (a finalized bit, a gas-used count) and three nodes' copies of it, with R the states where every copy equals the header; node i's view is its own copy, its part of the world is that copy, node 0 is the producer, and every node acts exactly when its copy says finalized. Then the carriers are independent; "finalized" and "gas used is at least one" are both copied on R; "finalized" is the pointer on R; and "gas used is at least one" is not relied on by parties who didn't produce it.

Proof. Independence: to give node i the copy from s and node j the copy from t, overwrite node j's copy in s. On R every copy is the header, so each node reads either fact off its copy. Node 1 acts only where its copy, and so the header, says finalized, and acts at the finalized header with no gas used; that state shows gas used fails node 1's reliance. Any event relied on by node 1 holds wherever node 1 acts, which on R is wherever the header is finalized, so "finalized" is the strongest. □

Checked as `ledger_carriers_independent`, `ledger_copying_alone_does_not_separate`, `ledger_finalized_is_pointer` and `ledger_gas_not_relied_on` (PointerObservable.v). It's a sanity check with maps I wrote, and it shows only that the tightened definition can tell the two apart where plain copying can't.

Thesis 15.3 (The pointer thesis)**.** Let C be the class of ecosystems with reliance that faithfully model deployed record-keeping disciplines, each with its reachable states, its observers reading independent carriers in the sense of Definition 15.2, its certification event, and the records its discipline prices. Then for every member of C: (1) the certification event is the pointer on the reachable states, so it is copied and relied on by parties who didn't produce it, and every event copied and relied on in that sense, every event a step rule could meter among them, holds wherever certification does; and (2) every record the discipline prices is copied and relied on by parties who didn't produce it.

The thesis leans on two notions with no precise mathematical definition: what it is to "faithfully model" a system, and which systems count as "deployed." Those two tie the definitions to the world, the way "computable in the plain-English sense" does in the Church-Turing thesis, and no definition can do that job, so the claim is a thesis. The other two terms it needs, independent carriers and the events a step rule could meter, are defined in Definition 15.2. Its formal shape is checked as a schema, `pointer_thesis_schema` (PointerObservable.v), with the class, the reachable states, the certification event and the priced records as inputs; nothing about them is proved.

A toy: three observers on a replicated ledger, where certification proliferates and "work done" doesn't. The file labels it a sanity check, and that's all it is. It isn't evidence.

Theorem 15.4 (The toy)**.** Let the toy have states (a certified bit, a work count), three observers, and every fragment equal to the certified bit. Then "certified" is the unique pointer among the one rival "work at least 1".

Proof. Every fragment is the certified bit, so every observer records "certified" by definition. For the rival, take the state with the bit false and work 1: the event holds and observer 0's fragment is false, so observer 0 does not record it. □

Checked as `toy_cert_unique_pointer` and `toy_work_not_proliferating` (PointerObservable.v).

A survey of twelve events, one checked theorem: for each of the five systems in Section 14, the designated event (finalized, committed, attested, logged, checked) proliferates in the observer map I wrote for that system, and a rival (proposer work, a scratch counter, measurement noise, submission latency, prover steps) doesn't. The five together are one statement (`five_labeled_models_have_selected_pointer`), an instance of the schema above. Plus a symmetric message-authentication code, which does *not* proliferate, and a digital signature, which does. Twelve rows. And a thirteenth that I think is the most important one: swap which bit every observer copies, and the criterion picks the *other* event. The criterion selects whatever the observer map copies. The map is where the argument lives, and I wrote the maps. So the five "confirmations" are five instances of one construction with the fields renamed. I'm saying that twice on purpose, so it can't slip past once.

Theorem 15.5 (The survey, and the swap)**.** (1) In each of the five observer models (proof-of-stake, gas, attestation, transparency, proof-carrying), the designated event proliferates and the designated rival does not; in the symmetric-MAC model the authentic-tag event does not proliferate; in the signature model the authentic-signature event does. (2) Let the states be pairs of bits (first, second), with three observers whose fragments all equal the second bit. Then "second is true" is the unique pointer among the one rival "first is true".

Proof. Each confirmation holds by definition: each observer's fragment is true exactly when the event holds. Each refutation exhibits one state where the event holds and some observer's bit is down. In (2) that state is (true, false). □

Checked as `twelve_candidate_measurements_checked` and `swapped_event_is_pointer_checked` (RecordProliferationSurvey.v, statements in RecordProliferationSurveyTarget.v); the five designated models together are `five_labeled_models_have_selected_pointer` (PointerObservableReductions.v), an instance of the indexed form of the criterion, `pointer_criterion_holds_indexed`.

Two stronger claims, both refuted, and I'd ask you not to merge them because they fail for different reasons.

The first stronger claim: *every forgery-resistant commitment proliferates*. If that were true, certification would be the pointer because it's the commitment. It's refuted in

three observer models (deniable authentication, where only the designated verifier's bit tracks the event; a symmetric MAC, where the key holders share a secret and nobody else can check; an object capability, an unforgeable token that grants access to whoever holds it) and confirmed in two (a public log; a digital signature). Build the first with three observers: give one a copy of the event's bit and the other two a constant false. The one records it; it doesn't proliferate. The checked half is the map computation. Whether my maps are *fair* to the real designs is argued in prose, and you should argue back.

Theorem 15.6 (One blind observer is enough)**.** (1) In any ecosystem, if some observer $i < N$ has $F_i(s_0)$ false at a state s_0 where E holds, then E does not proliferate. (2) Let the deniable model have states (authentic, verifier-accepts), three observers, observer 0's fragment the authentic bit and observers 1 and 2 constantly false. Then observer 0 records "authentic", and "authentic" does not proliferate. The symmetric-MAC and object-capability models also fail to proliferate their events; the public-log and signature models proliferate theirs.

Proof. (1) Recording would require $E(s_0) \iff F_i(s_0)$, and the left side is true while the right is false. (2) Observer 0's fragment is true exactly when the event holds. Apply (1) with observer 1 and the state (true, true). The other two refutations apply (1) to a blind observer of their own model, and the two confirmations hold by definition. □

Checked as `blind_observer_blocks_proliferation` and `labeled_model_verdicts` (PointerObservableCounterexamples.v). The theorem is about the observer maps. That a third party in deniable authentication really cannot be convinced is the modelling claim, and the model has no transcript, key or verifier in it to check that against.

The second stronger claim: *agreement plus accuracy plus locality imply permanence*. That is, if all observers agree, each sees the event exactly, and each updates by the same local rule with no coordinator, then the recorded event can't be undone. Refuted by the smallest thing imaginable: one bit, two observers who both see it, and the rule "flip." All three conditions hold and true becomes false in one step (`toggle_game_refutes_strong_pointer_necessity`). Do it on paper; it takes a minute and it's humbling. What *does* give permanence is durability, "an observer's true view stays true," added as a premise, and then it follows in a few lines (`durable_consensus_implies_permanence`). So durability is the premise doing the work, and durability is Section 4's permanence wearing a different coat. Agreement doesn't buy it. Do the paper version again with one change, the rule "stay true once true" in place

of "flip," and the bit can't fall, because the rule won't let it. The observers agreeing has nothing to do with it. (The machine I build later has a reading that's durable in exactly this sense: its flag never comes down, by its step rule (Theorem 16.5).) None of this rules out a narrower theorem with real assumptions about the network, the adversary or durability; it rules out these two strong forms.

Definition 15.3 (Observation game)**.** A *game* has states, a number N of observers, a Boolean event e, a Boolean view V_i for each observer, and a step map. For $i, j < N$ it has *consensus* when $V_i = V_j$ on every state; *authenticity* when $V_i(s) = e(s)$ for every state and every $i < N$; *coordinator-free update* when one function ℓ on Booleans has $V_i(\text{step}(s)) = \ell(V_i(s))$ for every state and every $i < N$; *durable views* when $V_i(s)$ true gives $V_i(\text{step}(s))$ true for every $i < N$; and a *permanent event* when $e(s)$ true gives $e(\text{step}(s))$ true.

Theorem 15.7 (What buys permanence)**.** (1) It is false that every game with $N > 0$, consensus, authenticity and coordinator-free update has a permanent event. (2) Every game with $N > 0$, authenticity and durable views has a permanent event.

Proof. (1) The toggle game: states are Booleans, two observers, event and both views the identity, step "flip". Consensus and authenticity hold trivially, the local rule is "flip", and the state true steps to false. (2) If $e(s)$ is true, observer 0's view is true by authenticity, stays true by durability, and authenticity turns it back into $e(\text{step}(s))$. □

Checked as `toggle_game_refutes_strong_pointer_necessity` and `durable_consensus_implies_permanence` (EcosystemGame.v, definitions in EcosystemGameTarget.v).

15.4 What isn't checked, and must stay that way on the page

That certification *in deployed systems* is singled out by being copied and relied on across *independent* carriers. The criterion now defines independence; it has no precise definition of "deployed" or "faithfully model," and no cost law connecting being copied and relied on to price. Every correspondence between a Coq observer map and a real system is a sentence I wrote. The thesis is a thesis, and anyone who says the formal object proves it is reading more than is there.

15.5 How to kill it

Three ways, in the order I'd try them.

One: find a deployed discipline that puts a price on writing a record that parties who didn't produce it don't copy and rely on. A real system with a toll on something nobody else leans on. That breaks the second half, "the priced record is the copied and relied-on one."

Two: find a rival event, one certification doesn't imply, that is copied and relied on *as well as* certification in a real ecosystem. Then certification isn't the pointer and the first half's "the one" is wrong. Or do it in a model: show that something like a multiplication count, or the number of times a Turing machine's head turns around, is copied and relied on by the same standard, so the criterion picks out more than one event and decides nothing.

Three: formalise an ecosystem honestly, with independence as Definition 15.2 defines it, where certification *fails* the test. The symmetric-MAC row is already most of the way there for a different claim; push it.

The checked work has already refuted two stronger forms of its own claim, which I take as a sign the method works and as a warning about the main claim.

15.6 The end of the logic

That's Part III, and the end of the logic that started in Part II. Here's what it earned. A toll on raising a flag is forced by three premises (Section 4). Learning isn't forced to cost. Committing is (Section 5). The counter is arithmetic once the schedule is fixed (Section 6). A window with a colliding pair in it can't price the flag exactly (Section 7). Entitlement is an object, a check, and a paid event, and, given lists, a tree and a covering that someone supplies, and a run that paid for the tree's depth, it costs at least the index bits it commits to (Section 8). Insight is a committed relation (Section 9). A feeling doesn't qualify. Over any base a record that is driven by the computation and never comes down is a latch, with two free choices, the price list and the event (Section 10). With more than two places, a record that is driven by the computation and climbs a partial order is a row of cups, each one a latch; the toll over the whole row charges at least one on any step that turns up some cup, and a straight strip of $n + 1$ stages needs exactly n cups that only turn up (Section 11). When the machine has finitely many states, its record only grows, and the ordinary machine forgets nothing, the toll prices crushing two states of one fibre, and a window can price exactly by seeing the cups move (Section 12). No inside procedure whose flip a substrate can represent decides a shortcut property, given a recursion theorem, which the nat family has by construction. L isn't a substrate, and on L the same diagonal is proved again from a recursion theorem it has by proof. What a record can do is carry the commitment (Section 13). Of five specifications, read through my

transcriptions, one charges a fee at the write (Section 14). And which event to record is a thesis with a borrowed shape and a listed set of refutations (Section 15).

The obvious question to ask of all that is whether the toll is the only way. A theorem saying so would be the best thing this book could hold: a short list of requirements that any account of what a step costs has to meet, whatever field it comes from, with the toll and the counter the only accounting left standing. I looked for that theorem, and the answer I can give you is no, and the no is small enough to check on three states. Call them blank, once and twice, with once and twice reading yes. The flip takes blank to once and leaves the other two where they are. The smooth takes both once and twice to twice and leaves blank alone. The smooth forgets something, so the price on merging from Section 4 charges it a mark. It raises nothing, so the toll leaves it free. Now try requirements that name no reading: no charge on a move that changes nothing, none on a move that forgets nothing, the same bill when you rename the states, a total that adds up along a run, a bill that looks only at the step, and one mark on the flip from blank. The toll, read as the bill that charges the flip from blank and nothing else, meets every one of them on this machine. So does the price on merging. They disagree on the smooth, so that list doesn't pick between them. What picks the toll is a requirement that names the record: over any run the total is never more than the number of times the reading went up. With the floor, that leaves exactly one bill, one mark per raise and nothing else (Section 7), and it rules the price on merging out, because the smooth costs that price a mark and raises nothing. All of that is checked on the three states: `nec_f_three_toll_meets`, `nec_f_three_merge_meets`, `nec_f_three_bills_differ`, `nec_f_three_ceiling_picks_toll` and `nec_f_three_merge_fails_ceiling`, with the general uniqueness as `nec_f_record_ceiling_unique` (NecFThreeState.v). The only renaming of the three states that respects the moves is the identity (`nec_f_three_automorphism_id`), so the renaming requirement asks nothing there. So the uniqueness is real and it's small. It holds once you've said which reading you're charging for, and which reading deserves the charge is the pointer thesis, which is still a thesis.

The uniqueness theorems that do exist are about pricing information loss, and they're worth a look because they say what kind of bill the price on merging is. John Baez, Tobias Fritz and Tom Leinster (2011), mathematicians who work in category theory, the mathematics Lawvere's diagonal theorem came from, asked what a measure of information loss has to look like. As I read their paper, they proved that, for maps that preserve a probability measure, Shannon entropy is up to a constant factor the only measure of information loss that is functorial, convex-linear and continuous. János Aczél and Bruno Forte, mathematicians at the University of Waterloo who worked on functional equations, and C. T. Ng, who had taken his

doctorate there under Aczél in 1971, asked in a paper of 1974 which entropies deserve to be called natural, and proved, by their account, that the entropies with a short list of natural properties are exactly the linear combinations of Shannon's and Hartley's, where Hartley's is the log of the count, so even for merges the choice between the average and the worst case is a real choice. Both price merges. Neither prices raises, so neither picks out the toll, and that's the three-state example in a bigger room. On the other side of the fence, Luciano Pomatto, Philipp Strack and Omer Tamuz (2023), economists at Caltech and Yale who asked what learning something should cost, proved, by their account, a uniqueness result for what it costs to *acquire* information, with constant marginal costs. That's a price on producing a signal, a different bill from the free learning of Section 5.

What none of that is: a machine. Every theorem so far quantifies over "any states, any steps," or is about a small copy of somebody else's system, or points forward at a thing I keep promising. That's the abstract part of the book, and it's done. From the next page on I stop saying "any machine" and build one, mine: the least I can make that still shows the whole model, with every part named and every price written down, so that everything above finally has something to bump into, and then one fixed program, run on a host with more counters, that runs every machine of the model whose workings a program can compute. If a statement up there doesn't mean what I think it means, that's where it'll show. Part IV is the thing. I want to see if it holds weight.

IV

The machine

16

The smallest honest machine

Parts I to III never built anything. They said "any states, any steps," and whenever a picture was needed, a board or some chairs or a cup stood in. Now I have to build a machine, and the first worry I have about that is a slightly embarrassing one. I built a big one first, the one from the origin story, with dozens of instructions, some picked for reasons and some picked because I had to pick something, and theorems held of it. What does that show? That the model works, or that I'm good at stacking instructions until the proofs go through? A big machine is a wonderful place to hide a mistake. It's a garage so full of stuff that you could lose a car in it.

So the machine in this book is a small one. The least I can build that still shows the whole model, every piece of it, with nothing waved at. If the idea only stands up with dozens of instructions holding it, the idea belongs to those instructions. The model can't claim it. If it stands up on six, the six are the idea, and you can hold all of them in your head at once, which is more than I can say for most things I've built. Six kinds, to be exact: each takes operands, a counter or a property or a line to jump to, so the instructions themselves are unboundedly many. And smallest, in this chapter's title, is my account of building it. No comparison class, no measure of size and no proof of minimality stands behind the word.

16.1 What the model actually asks for

Here's what Parts II and III say the model adds to a machine. A *record*: a flag that reads no or yes, "certified" or not. A *toll*: the step that raises the flag costs something (Section 3). A *ledger*: the running total of those costs, a whole number that only climbs (Section 6). And Section 10 proved the thing this whole chapter stands on: over any base at all, a base-driven record is a *latch*, a bit that switches on at an event the base decides and never switches off (Observation 10.1). The record watches the base. It doesn't compute anything the base couldn't compute without it.

Push on that for a second, because it cuts the job in half. If the record adds no computing power, then all the computing power comes from the base, and the question "how small can the machine be" splits into two questions that don't touch each other. For the base: what's the smallest thing that can compute everything there

is to compute? For the record: what's the smallest set of pieces that makes the flag *mean* something?

One honest asterisk on "don't touch," because the machine below makes it matter. What's blind is the flag and the ledger: the base never reads either of them. But the pieces that make the flag mean something (a table of checked facts, a slot for a commitment, a trap that stops a refused run) have to sit where the step can read them, which is in the base, and they do steer the run: a failed check stops a program dead. The latch theorem holds wherever you draw the line between base and record, which is exactly why it can't tell you where the meaning lives. On this machine the meaning lives on the base side of the line, in the bookkeeping, and the flag on top reports it.

The record question, the second of those two, is the one Part III left to the machine, on purpose. Section 9 said that testing a claim and committing to it are two different acts, and that nothing in the five parts of the model makes the second wait for the first. Read Section 3's definition again with that in mind: a machine where you pay one unit and raise the flag over nothing in particular obeys every rule in it. That machine is honest about the price and silent about what was bought. A receipt for nothing. The toll alone can't fix that, because the toll is a rule about *cost*, and "was this earned" is a question about *order*: did a check happen, of this claim, before the commitment, with nothing changing underneath in between?

This machine is my attempt to make the order part of the machine, in the smallest place I could find to do it. I'll show you the pieces, then the proofs, then the places it falls short, which there are, and which I'd rather you hear from me.

16.2 Where the counters come from

The base first, and the base isn't mine. It's Marvin Minsky's. In a 1961 paper in the *Annals of Mathematics*, and then in his 1967 book *Computation: Finite and Infinite Machines*, Minsky studied a machine so bare it's almost a joke. Picture a few bins of pebbles and a numbered list of instructions. There are only two kinds of instruction: put a pebble in bin c and go to the next line; or, if bin c has a pebble, take one out and jump to line j, and if it's empty, just go to the next line. That's it. No memory, no tape, no arithmetic beyond plus one and minus one. When the line you're on has no instruction, the machine stops.

What Minsky proved, by hand, is that two bins are enough to compute anything a Turing machine can compute, as long as you're willing to pack the input in a strange way. The trick is to keep many numbers inside one number, as the powers of different primes ($2^x 3^y 5^z$ holds three numbers in one), and to do arithmetic on the exponents by multiplying and dividing with the second bin as scratch space. It's

slow. Exponentially slow. A two-counter machine doing your taxes would finish sometime after the sun burns out. But "universal" is a claim about what's *possible*. Pleasant doesn't come into it, and on that claim two bins of pebbles are as strong as anything.

Who cares? Minsky did, because the question then was where the edge of computation sits and how little machinery reaches it. You should, because it means the base of this chapter's machine hides nothing: if it can compute everything with plus one, minus one and a jump, then nothing about the record can be smuggling computing power in through some instruction I made fancy. And I care because it's the smallest universal base I know of that is already in a Coq library, so the hard part (proving that its halting question is undecidable) is someone else's checked work. I don't have to ask you to take my word for it.

And the pebbles aren't only my picture. They were somebody's actual machine, the same year. In one issue of the Canadian Mathematical Bulletin, September 1961, Z. A. Melzak, a mathematician at the University of British Columbia, described a computer made of distinguishable holes and an unlimited supply of pebbles, with one three-place operation, and a few pages later Joachim Lambek, a mathematician at McGill University, trimmed it to two moves: put a pebble in hole X; take one out of hole X if there's one to take. He called it an *infinite abacus*, and early on he writes "Our method is probably not new," which is a hard sentence to beat. He also tells anybody who distrusts infinity not to worry about all those holes, since any one computation only ever uses finitely many of them. Is that two inventions or one? Lambek says his was a modification of Melzak's, so call it one and a half, with Minsky's as a separate find the same year. And then Lambek went somewhere you wouldn't guess: in 1986, with Philip Scott, a logician who had come to McGill to work with him and later taught at the University of Ottawa, he wrote a book whose first part, as the publisher describes it, shows that typed lambda calculi and a certain kind of category are essentially the same thing. Pebbles to category theory in one career, twenty-five years end to end. Does any of it change the argument? No. The library and I use Minsky's version and Minsky's name. But a bare machine that several people reach for in the same year looks like a shape that was sitting there. It doesn't look much like anybody's invention.

The title of Minsky's 1961 paper is the clue I can't leave alone. It's about "Post's problem of tag." Who's Post, and what's tag? Emil Post was a logician, a fresh PhD from Columbia, who in 1921, on a fellowship at Princeton, was playing with about the simplest rewriting game there is. Take a word of 0s and 1s. Look at its first letter, chop the first three letters off the front, and stick something on the end: `00` if the letter you looked at was 0, `1101` if it was 1. Repeat, and stop when fewer than three letters are left. Try it at the table. Start from `1000`: you get `01101`, then `0100`, then `000`,

then `00`, and it stops. Start from `100`: you get `1101`, `11101`, `011101`, `10100`, `001101`, and then `10100` again, and it goes round forever. Post wanted a rule that tells you, for every starting word, which of those happens. He worked at it for months and later called it hopeless, and as far as I can tell it still is: a hundred years on, nobody knows whether that one little game, with that one rule, can be decided. Stephen Wolfram, the physicist who built Mathematica, wrote a 2021 article on it that tells the Princeton story, and it's where I'd start. In his 1961 paper Minsky proved by hand that tag games in general can compute anything, which is why Post's name is on it. It didn't settle Post's own game. And the chase goes one step further. In 2008 Liesbeth De Mol, a historian and philosopher of computing who wrote her doctoral work on tag systems, turned the famous $3n + 1$ problem (halve an even number, triple an odd one and add one, and ask whether you always get down to 1) into a small tag game of the same kind. So is Post's game secretly as hard as everything, or just stubborn? I can't tell you, and as far as the published work goes, nobody can yet.

None of that changes the machine below. It changes the picture. The two-counter machine came out of a hunt for the smallest thing that can't be predicted, and the smallest things are exactly where the open questions are still sitting.

One word clash, before it bites. Minsky's machine is usually called a *counter machine*, and its bins are *counters*. Section 6 also had a counter: the ledger μ, the number that only climbs. Same word, two completely different things, and I'd love to say I set that up on purpose. In this chapter, *counter* means one of Minsky's two bins, called A and B, and the climbing total is always *the ledger* μ.

16.3 The machine

Here's the whole state. I'll say each piece in words first.

- Two counters, A and B. Each holds a whole number, with no upper limit.
- A *version* for each counter, also a whole number with no limit. It starts at 0 and goes up by one every time its counter is written. Think of it as the counter's edition number: a new value is a new edition.
- A *line number*, which instruction comes next, counting from 1 the way the library's two-counter machine does (more on that choice just below).
- A *fact table*: a list of facts. A *fact* is three things stuck together: a property, a counter, and a version. "A is even, as of edition 3" is a fact. The table has room for sixteen. Why sixteen? It's a pick, with no derivation behind it, and what the pick costs is in the subsection near the end of this chapter on what the machine doesn't do.

- A *channel*: either empty, or holding one fact. This is where a commitment sits.
- A *trap*: one bit, down at the start. When it goes up, the machine has refused something, and it stays up.

Counting lines from 1 looks like the least interesting choice in the chapter, and it's one of the oldest arguments in programming, with a famous referee. Edsger Dijkstra was a Dutch computer scientist, one of the generation that founded the field, and for over forty years he mailed numbered notes to a few dozen colleagues. In 1982 he wrote one of them, EWD 831, titled "Why numbering should start at zero." His case goes through ranges first. Of the four ways to write "2 up to 12" with less-than signs, he picks the one that takes the bottom in and leaves the top out, $2 \le i < 13$: two ranges that meet end to end then share one number written once, and the length is just the top minus the bottom. Start a sequence of N things at zero and it's $0 \le i < N$, and each thing's number is how many things came before it. He reports that programmers on the language Mesa, which allowed all four conventions, found the other three a steady source of clumsiness and mistakes, and he points out that FORTRAN starts at one. Does that settle it? It settles which convention is tidier, by an argument you can check. It doesn't settle which convention somebody else's work used, and that's the trap: the library's two-counter machine, which it calls a Minsky machine, numbers its lines from 1, and so does this machine, because matching it line for line is worth more here than tidiness. Is zero the right place to start counting? Dijkstra's title says it is, and I can't tell you more than that. What his note carries is an argument about which way makes fewer mistakes. That part you can check for yourself, and it's the more useful thing to have.

Everything in that list is the *core*, the part the base moves. Outside the core sit the two pieces from the model: the ledger μ and the flag. The machine never reads either of them to decide what to do next. I'm not asking you to take that on trust. It's how the step is written, and it shows up as a theorem below.

The properties a fact can name come from a tiny fixed language with three words: *is zero*; *is even*; and *is at least n*, for any whole number n you like. Each one comes in two forms. There's the *checker*, a little program that computes yes or no: compare with 0, look at the last bit, compare with n. And there's the *meaning*, the plain arithmetic statement the checker is supposed to be checking: the number is 0; the number is two times something; n is less than or equal to the number. Why both? Because a checker is code, and code can be wrong in ways that look right, and "the checker says yes" is only worth anything once somebody has proved it says yes exactly when the meaning is true. That proof is one line per property, and it's the first block below.

Why can a machine check a claim at all? Because a claim with a precise meaning is something you *evaluate*: plug in the values and turn the crank. George Boole worked out in 1847 that you can do arithmetic on claims this way, treating "and" like multiplication and "or" like a kind of addition. Why multiplication? Try it at the table. Write 1 for true and 0 for false, and multiply: one times one is one, and anything times zero is zero. That's "and" to the letter. Then the stranger thing Boole noticed. Say "x and x" and you've said nothing new; it's just x. In numbers that's $x \times x = x$, and he made it a law of his system, and it's a law ordinary numbers mostly break. Which numbers equal their own square? Two times two is four, no. A half times a half is a quarter, no. Zero, yes. One, yes. That's the whole list. So when Boole wrote his bigger book in 1854, *The Laws of Thought*, he asked you to imagine an algebra whose letters take the values 0 and 1 and nothing else. Every checker on this machine answers in that algebra.

And the 1847 book has a backstory worth a detour, which I found in the Stanford Encyclopedia of Philosophy's entry on Boole. Boole was largely self-taught and spent about fifteen years as a schoolmaster, and what pulled him back to logic, early in 1847, was a loud public quarrel between Augustus De Morgan and the Scottish philosopher Sir William Hamilton over who had first thought of a small technical idea in the old logic of syllogisms. Two well-known men fighting over credit, and a schoolmaster watching from the side picks up the subject and writes the little book this paragraph is about. Few people remember what the quarrel was over. Every checker in this chapter runs on what the bystander wrote.

Then push on it, because Boole's "or" isn't quite the one a checker uses. His plus only made sense for things that can't both hold at once, so "x or y" with both of them true had no meaning in his system. William Stanley Jevons, an English economist who also worked on logic, read "or" as "either or both," which gives "x or x is x," the same trick as before. He wrote to Boole in 1863 saying plus ought to work that way, and in 1864 he put it in a book called *Pure Logic*. Boole wouldn't have it, since it broke the tie to ordinary algebra his whole system stood on. The "or" in every programming language I know is the inclusive one Jevons argued for. Does that change anything about the checker? No; the crank turns the same way. It changes the picture: the "or" a checker uses is the one the man who invented the algebra turned down.

Hanging a checked claim on a point in a computation is an old move too. Tony Hoare, in 1969, built a whole way of reasoning about programs out of it: attach a claim to a *point* in a program ("whenever execution reaches here, this is true"), and reason forward from claim to claim. Robert Floyd had done it two years earlier on flowcharts. Their claims were checked on paper by the person writing the proof.

This machine's version is a property of one counter at one edition, checked by a step while it runs.

And is two years earlier the bottom? Not even close, and the next step down is a paper this book already leaned on. In June 1949, at a Cambridge conference held for the new EDSAC computer, Alan Turing gave a talk three pages long, "Checking a large routine," the same note the counter chapter used for stopping (Section 6). Stopping was the second half of it. The first half is this move, twenty years before Hoare: the programmer should make "definite assertions which can be checked individually," and the correctness of the whole should follow from them. His example program computes a factorial using nothing but addition, and his picture for why you'd bother is one you can do on the back of an envelope. Add up a long column of four-digit numbers and hand someone only the total, and they have to redo the whole sum at one sitting, because the carries tie every column to the one before it. Hand them the total of each column as well, and the job falls apart into small checks they can do one at a time. That's a claim hung on a point in the middle of the work, so the person checking never has to hold all of it at once. The paper got its due mostly later: in 1984 two computer scientists, F. L. Morris of Syracuse University and C. B. Jones of Manchester University, who had worked on it together at Oxford, republished it, with its misprints corrected, as an early program proof by Alan Turing. Theirs is the text I read, and the column of numbers is Turing's own example in it. Does any of that change the machine? No. It changes the date on the idea, and it tells you something I like: the first person to ask how you'd check a program put the claims in the middle of it. He didn't save them for the end.

Now the six instruction kinds.

Instruction	**What it does, when the trap is down**	**Cost**
INC *c*	Adds one to counter *c*, adds one to its version, goes to the next line.	0
DEC *c j*	If *c* is above 0: subtracts one, adds one to its version, jumps to line *j*. If *c* is 0: just goes to the next line.	0
HALT	Nothing. A program that reaches it stops.	0

Instruction	What it does, when the trap is down	Cost
CHECK *p c*	Runs the checker for *p* on *c*'s value. If it says yes and the table has fewer than sixteen facts, writes the fact (*p*, *c*, *c*'s current version) into the table and goes to the next line. Otherwise, trap.	1
COMMIT *p c*	Looks in the table for exactly the fact (*p*, *c*, *c*'s current version), field by field. If it's there, puts it in the channel and goes to the next line. Otherwise, trap.	1
CERTIFY	If the channel holds a fact, goes to the next line, and the flag goes up. Otherwise, trap.	1

The first three are Minsky's machine, plus a HALT so a program can say it's done. The last three are the record's. Read them as a chain: you can only certify with a commitment in the channel, you can only commit to a fact in the table, and the only way into the table is a check that passed. Three links, and each one costs.

One of what? One unit of μ, the same unit as Section 3's toll. Why one? Why would ten be worse? Because one is the smallest positive whole number, which is honestly the whole defence, and Section 4 is the section where I tried to do better than that for the flag. That argument doesn't reach this machine, though (the end of the chapter says why), so here the one is a choice, full stop.

Where does that one come from, though? It's older than Coq by about a century. In 1888 the German mathematician Richard Dedekind published a little book whose title asks what numbers are and what they're for, and in 1889 the Italian mathematician Giuseppe Peano published a booklet, written in Latin, that set out the axioms for the whole numbers in the form people still teach: there's a first number, and every number has a next one, and a few rules about how "next" behaves. I had the two of them down as working independently. I went looking, and the Stanford Encyclopedia's entry on Dedekind says Peano acknowledged Dedekind's priority, which is why the axioms often go by both names. (Peano's booklet is also where the sign $\in$ that you met in Section 2 comes from: he wrote it as a Greek epsilon, for the Latin *est*, "is.") Coq builds its whole numbers exactly that way, out of zero and "the next one" and nothing else, so inside Coq the number three really is S (S (S O)), the next of the next of the next of zero. What that changes here is small and I like it: the one a CHECK costs isn't a number pulled from a hat. It's one application of

the smallest step the number system has. You can't charge less than one S and still charge something.

The zeros matter just as much: computing is free on this ledger. Plus one, minus one and jumping around cost nothing at all. Only the three acts that make a claim cost, and each of them costs whether it passes or traps. Failing is not a discount.

The version rule. Here's why versions exist, with numbers. Start with $A = 4$, version 0. CHECK *is even A* passes, and the table gets the fact (*even*, A, 0). Now INC A: A becomes 5, and its version becomes 1. Now COMMIT *is even A*. The machine looks for (*even*, A, 1), the claim about the current edition. The table only has (*even*, A, 0). No match. Trap. The old fact is still sitting in the table, still true about edition 0, and completely useless for committing to anything about edition 1. A fact is about one edition of one counter, and an edition never comes back, because versions only go up and never wrap. That's the whole trick against stale evidence: you can't check a number, change it, and then commit to the check.

That's the opening chapter's time-of-check-to-time-of-use bug, the file checked and then swapped, and the database answer to it is Kung and Robinson's: at commit time, check that nothing you read was written in the meantime, and back out if something was. The version rule is that check, done by the step, on one counter. It also shows where the guarantee ends. It works because one step does every write and every write bumps a version. A second writer that changed a counter without bumping its version, a second processor or a device writing memory on its own, would walk right around it, and none of the theorems below are about a machine with one of those.

Processor designers put a cousin of this check in silicon. Jensen, Hagensen and Broughton, at Lawrence Livermore in 1987, described a pair of instructions now called load-linked and store-conditional: the load reads a word and the processor starts watching it, and the store goes through only if nothing has written that word since; otherwise it fails and the program tries again. It's tempting to say CHECK is the load and COMMIT is the conditional store, and it's wrong in two places. COMMIT doesn't write the counter it checked, and the thing doing the watching, the fact, sits in a table the step keeps. No single processor holds it. The hardware version does cover the second writer, because the watching happens on the memory everyone shares, which is exactly the case the version rule leaves out.

The real kinship is older, and it's somewhere else. Picture the table and the counters shared by many processors, with CHECK, COMMIT and CERTIFY each done as one indivisible step. Maurice Herlihy (1991), a computer scientist who has spent his career on how many processors can share memory safely, ranked shared objects by how many processors each one can bring to agree on a single value with nobody ever waiting on anybody. Test-and-set, an instruction that writes a cell and reports

what was there, brings exactly two. A sticky bit, the idea of Serge Plotkin (1989), a Stanford computer scientist working on distributed algorithms, is a cell the first writer jams for good, with the first writer's value, and everyone after can read it; it brings any number. I guessed that the shared object would sit at the top with the sticky bit, because a write that kills every fact about a counter, together with a commitment other processors can see, looks like a way to build one. Ranking the object takes its operations, what each one returns, which steps are indivisible, how it starts and which processes run it, and then a proof that every processor decides, they all decide the same value, the value is one somebody proposed, and nobody waits. Here it is, with every instruction one indivisible step and the whole state readable, and the object is at the top. The reason is the order of the fact table. Each processor CHECKs a property that holds of a counter at zero, PZero to propose 0 and PEven to propose 1, then reads the table and decides the bit of its oldest fact. The table only gains entries at the front, and a CHECK that finds it full trips the trap, after which nothing moves, so the first CHECK of the run fixes the oldest fact for good. Every processor that decides decides that bit, the bit is somebody's proposal, and each one decides after two of its own steps whatever the others do (`sc_agreement`, `sc_validity`, `sc_wait_free`, SmallConsensus.v). The version rule and the commitment play no part. The trap alone doesn't do it: it records that something trapped, and it doesn't hand everyone the first writer's value.

Now take the version rule away, let failures change nothing, keep only the properties "at least *n*" so that a checked property only ever turns true, drop the decrement, and make the fact table, the commitments and the flag sets and latches that only grow. I guessed that then the object drops to the bottom rung, the level of plain memory, which is the shared-memory face of what the database researchers Joseph Hellerstein and Peter Alvaro (2020), at Berkeley and Santa Cruz, call CALM, where monotone means nobody has to coordinate. That's proved too, by Herlihy's own kind of argument. Let the updates return nothing and let the whole state be readable. If at every state, of any two updates, one leaves the state alone, or the two commute, or one overwrites the other, then no protocol brings two processors to agreement with nobody waiting (`mc_no_consensus`). The monotone object has that property, with registers beside it, because an instruction that would succeed still succeeds after anything else has run (`mc_mono_interfere`, `mc_mono_no_consensus`, MonotoneConsensus.v). The toll changes neither result. Neither idea is mine: the top is Plotkin's and the bottom is the monotone fact that field already knows.

Process-calculus people have a statement of the same split. Johannes Åman Pohjola and Joachim Parrow, computer scientists at Uppsala University (Parrow is one of the inventors of the pi-calculus, a mathematical model of processes that talk over channels), in 2016 proved, with the proof checked in the Isabelle proof assistant, that

no model whose parallel composition is monotonic, where a neighbouring process can never stop a step, has a consensus process. One of their examples of a model that does is test-and-set, an instruction that writes a cell and reports what was there. Test-and-set isn't a sticky bit: in Herlihy's ranking it brings two processors to agree, and a sticky bit brings any number.

Their setting is small enough to check here whole. A model is any set of processes with a step relation and a way to compose two processes that doesn't care about order or grouping. Composition is monotonic when whatever a process can reach alone, it can still reach with anything set beside it. A consensus process is one where any number of copies, run together, can end up agreeing on either of two outcomes, and once a run has one outcome it can never show the other. Their theorem goes through in a few lines: run half the copies to one outcome and the other half to the other, and if neither half can block the other, the two outcomes meet on the same state (`cs_no_consensus_monotonic`, ConsensusSeparation.v). Then the machine goes into it. A process is a shared machine state with a crowd of threads, each proposing a bit, and composing two processes pools their threads on one state. One thread proposing each bit is a consensus process in exactly their sense, for the outcomes "some thread decided 0" and "some thread decided 1" (`cs_machine_consensus`), so by their theorem that composition isn't monotonic (`cs_machine_not_monotonic`). The small machine sits on their non-monotonic side, checked. What puts it there in the run is the order of the fact table, the same as above; a store that refuses a stale commit for good, as the version rule does, refuses in the same spirit. The shared-memory version of their split is the pair of results above: the small machine's object reaches agreement for any number of processors, and the monotone variant reaches it for none (`sc_agreement`, `mc_mono_no_consensus`).

Security people know the pattern by a third name. A check that blesses a value, and a write that takes the blessing away, is close kin to dynamic taint tracking, where a label rides along with the data and a sanitizing step changes it; the labels go back to the computer-security researcher Dorothy Denning's 1976 lattice model of how information is allowed to flow. How much a monitor like that has to remember is a studied question too: Chamseddine Talhi, Nadia Tawbi and Mourad Debbabi (2008), three security researchers at universities in Quebec, worked out which rules a monitor can still enforce when its memory is bounded. The version rule is one small, specific member of that family, done by the step.

Notice what *isn't* a write. DEC on a counter that's already 0 doesn't change the counter, so it doesn't change the version. It's a test that happens to move the line number.

The fact table. Facts go in and never come out. Nothing erases them and nothing overwrites them. When the table is full, a seventeenth CHECK traps. It doesn't push

an old fact out to make room. That's a choice with a bill attached, and I'll put the bill in the section on what this machine doesn't do.

Traps. Any failure puts the trap up and changes nothing else in the core; the ledger still takes the instruction's cost. Once it's up, every instruction leaves the core alone. A stored program stops at a trap. But I also prove things about *any* list of instructions, programs or otherwise, because an adversary doesn't have to run my programs; they can feed the machine whatever they want. For those lists, an instruction after a trap is still charged its cost, and still changes nothing.

Now the exact version.

Definition 16.1 (Properties, checker, meaning)**.** A *property* is one of: Zero, Even, or $\mathsf{Ge}(n)$ for a whole number n. The *checker* $\mathrm{eval}(p, v)$ computes yes or no: $v = 0$ by equality, evenness by the parity function, $n \le v$ by comparison. The *meaning* $\mathrm{holds}(p, v)$ is the arithmetic statement: $v = 0$; there is an m with $v = 2m$; $n \le v$.

Lemma 16.1 (The checker says what it means)**.** For every property p and every whole number v, $\mathrm{eval}(p, v) =$ yes if and only if $\mathrm{holds}(p, v)$.

Proof. Split on the three properties. Equality with zero returns yes exactly at zero; comparison with n returns yes exactly when $n \le v$. For evenness, the parity checker returns yes at 0, no at 1, and the same answer at $v + 2$ as at v. Induction in steps of two gives yes exactly at $0, 2, 4, \ldots$, the numbers $2m$. Those are precisely the three meanings. □

Checked as `eval_iff` (EarnedCore.v).

Definition 16.2 (Core, state, claim, start)**.** A *core* k holds two counters A, B and their versions (whole numbers), a line number, a fact table (a list of facts, a fact being a triple of a property, a counter and a version), a channel (empty or one fact), and a trap bit. Write $\mathrm{val}(k, c)$ and $\mathrm{ver}(k, c)$ for counter c's value and version. A *state* s is a core, written $\mathrm{core}(s)$, with a ledger $\mu(s)$ and a flag $\mathrm{cert}(s)$. The *claim* p of c in k is the fact $(p, c, \mathrm{ver}(k, c))$. The table's room is 16. The start $\mathrm{start}(a, b)$ has counters a and b, versions 0, line 1, an empty table, an empty channel, the trap down, $\mu = 0$ and the flag down. A state is *clean* when its table and channel are empty and its flag is down.

Definition 16.3 (Instructions, costs, the step)**.** The instructions are INC c, DEC c j, HALT, CHECK p c, COMMIT p c and CERTIFY, with effects on a core with the trap down as in the table above. Passing any check, commitment or certification requires the trap to be down. With that condition, CHECK passes when $\mathrm{eval}(p, \mathrm{val}(k, c)) =$ yes and the table holds fewer than 16 facts, COMMIT passes when the claim p of c is in the table, and CERTIFY passes when the channel holds a fact; a failing one sets the trap and changes nothing else. On a core with the trap up, every instruction changes nothing. Costs: CHECK, COMMIT and CERTIFY cost 1, the rest 0. One step on a state is

$$\mathrm{exec}(s, i) = \big(\, \mathrm{cexec}(\mathrm{core}(s), i),\ \ \mu(s) + \mathrm{cost}(i),\ \ \mathrm{cert}(s) \vee \mathrm{fires}(\mathrm{core}(s), i) \,\big),$$

where cexec is the core's step and $\mathrm{fires}(k, i)$ is yes exactly when i is CERTIFY and passes at k. For any list t of instructions (a *trace*), $\mathrm{run}(t, s)$ applies them in order and $\mathrm{cost}(t)$ adds their costs. A *program* is a list of instructions numbered from 1; a program run fetches the instruction at the line number and executes it, and stops when the trap is up, the line holds HALT, or the line holds nothing.

Defined as `cexec`, `exec`, `run`, `total_cost`, `run_prog`, `start` and `Minimal.EarnedCore.clean_start` (EarnedCore.v).

Read the step line slowly, because the rest of the chapter leans on its shape. The core moves by its own rule and nothing else. The ledger adds the cost. The flag becomes "it was up, or this is a CERTIFY that passed." The core's move doesn't mention μ or the flag anywhere. The base is blind to the record by construction.

A worked run, the shortest one there is. The program: CHECK *is zero A*; COMMIT *is zero A*; CERTIFY. Start at $\mathrm{start}(0, b)$, for any b. Line 1: A is 0, the checker says yes, the table is empty so there's room, and the fact (*zero*, A, 0) goes in. $\mu = 1$. Line 2: the claim about A's current edition is (*zero*, A, 0), and it's right there. Into the channel. $\mu = 2$. Line 3: the channel holds a fact, the flag goes up. $\mu = 3$. Line 4 has no instruction, so the program stops. Three paid acts, in order, a flag that's up, and a channel that names the last earned commitment, which here is (*zero*, A, 0).

Now start the same program at $\mathrm{start}(5, b)$. Line 1: A is 5, the checker says no, the trap goes up, $\mu = 1$, and the program stops right there. The flag is down, and you can run it as long as you like: it stays down. That second run is the point of the first. The same three instructions certify from some starts and refuse from others, because the check could have failed, and from most starts it does. A check like that has a no pile, which is the only kind the opening chapter counts as establishing anything.

Could I have picked an even dumber claim? Sure, and the machine would have let me. CHECK *at least 0 A* passes on every number there is, so CHECK, COMMIT, CERTIFY on it raises the flag for 3 from every start (stop the presses: a number is

at least zero). The step doesn't police that. It checks order, aboutness and that the checker tells the truth; whether the question was worth asking isn't something it looks at. A check that can't fail establishes nothing, and this machine will charge you three and raise the flag for it anyway. Spotting that is the reader's job, with the property in hand, and it's on the list at the end of the chapter.

Others walked into the same line from communication. Three theoretical computer scientists, Oded Goldreich, Brendan Juba and Madhu Sudan (2012), built a theory of communication with a partner you may not understand, in which getting a message across counts only as a way to reach a goal, and the central notion is *sensing*: being able to tell, from what you see, whether you're making progress. The sensing they ask for is *safe*: when there's no progress it says so, and so it never reports a success that didn't happen. Their main result, as I read it, is that safe sensing, with a partner who could help if you spoke its language, is enough for one strategy that reaches the goal with every such partner. A check that can fail and never lies, standing in front of a verdict you can't take back. That's their theorem, in their setting, and it's the same shape.

16.4 It computes everything two counters can

First the base, because if the base can't compute, nothing else here matters.

Definition 16.4 (Two-counter program, compile, window)**.** A *two-counter program* M is a list of instructions, each either MINC c or MDEC c j. A configuration is a triple (line, a, b). One step of M: if the line holds no instruction, M has *stopped*; MINC adds one to c and goes to the next line; MDEC subtracts one from c and goes to line j if c is above 0, and goes to the next line if c is 0. $\mathrm{mrun}(n, M, x)$ is n steps from x, staying put once stopped. The *compile* of M replaces each MINC c by INC c and each MDEC c j by DEC c j. The *window* of a core k is the triple (line number, $\mathrm{val}(k, A)$, $\mathrm{val}(k, B)$).

Theorem 16.2 (Step for step)**.** For every two-counter program M and every core k with the trap down: M has stopped at $\mathrm{window}(k)$ if and only if the program $\mathrm{compile}(M)$ has halted at k; and if M steps from $\mathrm{window}(k)$ to y, then one program step of $\mathrm{compile}(M)$ from k lands on a core with window y and the trap still down. So for every n, n program steps of $\mathrm{compile}(M)$ from k land on a core whose window is $\mathrm{mrun}(n, M, \mathrm{window}(k))$, with the trap down.

Proof. Look up the instruction at the line. If there is none, both sides stop. If it's MINC c, the compiled INC c adds one to c and goes to the next line, which is exactly the window's move (the version also moves, but the window doesn't show versions).

If it's MDEC c j, split on whether c is 0, and both sides do the same thing in each case. Neither instruction can trap. The run statement is induction on n, one step at a time. □

Checked as `simulation_step` and `simulation_run` (EarnedCore.v).

Theorem 16.3 (Halting matches)**.** For every two-counter program M and whole numbers a, b: M stops after some number of steps from $(1, a, b)$ if and only if the program compile(M) halts after some number of program steps from start(a, b).

Proof. The start has window $(1, a, b)$ and its trap is down. Apply Theorem 16.2 after any number n of steps: the two configurations still have the same window, the trap is still down, and one is stopped exactly when the other is halted. The same n witnesses termination in either direction. □

Checked as `halting_correspondence` (EarnedCore.v).

Now the part that isn't mine. Proving that halting is undecidable for two-counter machines is real work, and it's already done, checked, in a shared Coq project called the Coq Library of Undecidability Proofs (presented by Yannick Forster, Dominique Larchey-Wendling and coauthors at CoqPL 2020). The two-counter result in it is Larchey-Wendling's, reached from his and Forster's work on Minsky machines (CPP 2019) by way of FRACTRAN, a model of computation invented by John Conway, the English mathematician best known for the Game of Life, which the library uses as a stepping stone. Their two-counter machine has the same two instructions as Minsky's, with the same rule for zero. Andrej Dudenhefner, one of the library's authors, writes that rule out in his 2022 paper on deciding questions about two-counter machines: DEC on a counter at zero goes on to the next instruction, and otherwise it takes one off the counter and jumps. So the only thing I have to prove is that their machine and my INC/DEC fragment are the same machine, step for step, and then borrow their theorem.

One honest wrinkle about what "undecidable" means in that library, because it isn't the textbook sentence. Every function you can write in Coq is a program, so in that library "decidable" means "some Coq function answers it." Coq by itself can't prove that *no* Coq function answers a question without an extra assumption about what programs are. So the library says the strongest thing it can without one: if anything decided this, then the questions on which a single-tape Turing machine *doesn't* halt could be listed by a program, which is the classic thing that can't be done. That's their convention, and the theorem below inherits it exactly.

Theorem 16.4 (Halting is undecidable for this machine)**.** The library's two-counter machine agrees with the INC/DEC fragment: its step, its stop, and its termination match those of the compiled program, so a library problem (P, a, b) halts if and only if the compiled program halts from start(a, b). Consequently, the question "does program P halt from start(a, b)?" is undecidable in the library's sense: a decider for it would make the complement of single-tape binary Turing machine halting enumerable.

Proof. The agreement is a case split on the library's instruction at the current line, and then induction along the run, ending in Theorem 16.3. The compile map is then a reduction from the library's two-counter halting problem, which the library proves undecidable, and the library's own lemma carries undecidability across a reduction. □

Checked as `mm2_step_iff`, `mm2_stop_iff`, `mm2_terminates_iff`, `mm2_halting_iff` and `earned_core_halting_undecidable` (EarnedCoreLinks.v), the last applying the library's `MM2_HALTING_undec`.

What that buys, and what it doesn't. It buys: the base of this machine is a universal machine in Minsky's sense, and nobody (in the library's sense) can predict in general what it'll do. What it doesn't buy is a fast machine, or one where your input arrives conveniently. The counters start holding a and b, given directly; there's no encoding layer in my files. Minsky's prime-power packing is how you'd get from "any Turing machine" to "two counters," and that construction is his and the library's, and I use it as it stands.

Is "given directly" just a convenience, or does it bite? It bites, and somebody found exactly where. In 1972 Rich Schroeppel, then at the MIT AI Lab, wrote a ten-page memo titled "A Two Counter Machine Cannot Calculate 2^N." He spells out the convention first: a counter machine computing a function starts with n in one counter and zero in the others. Then he gives a proof that a two-counter machine started that way can't compute 2^n. His theorem is checked here, for every program of the small machine (`tc2_no_pow`, Tc2Pow.v) and for every machine with a finite control whose counters are read only through zero and parity above a threshold (`am_no_power_of_two`), by the same chain of stages that proves the theorem about multiplying near the end of the chapter (Theorem 16.60): along an arithmetic progression of inputs the outputs grow by a fixed amount, and powers of two don't. So the packing Minsky needs is precisely the thing two plain counters can't do for themselves. Hand the small machine its inputs straight and there are functions of them it can't compute; hand it the inputs already packed and it computes anything a Turing machine can. That's no contradiction with the undecidability above, which ranges over every

program and every starting pair, packed ones included. The base hides nothing. It just needs its input pre-chewed, and the chewing is somebody else's.

16.5 It's a certification system, and the flag is a latch

Now the record. Three facts first, all of them read straight off the step line.

Theorem 16.5 (Ledger, toll, permanence)**.** For every state s and instruction i, $\mu(\text{exec}(s,i)) = \mu(s) + \text{cost}(i)$, and for every trace t, $\mu(\text{run}(t,s)) = \mu(s) + \text{cost}(t)$. If $\text{cert}(s)$ is down and $\text{cert}(\text{exec}(s,i))$ is up, then i is CERTIFY, it passes at $\text{core}(s)$, and $\text{cost}(i) \geq 1$. If $\text{cert}(s)$ is up, so is $\text{cert}(\text{exec}(s,i))$.

Proof. Read the ledger component of Definition 16.3: it adds exactly $\text{cost}(i)$. Repeated substitution along a trace gives the sum of its costs, or, equivalently, induction on the length of the trace does. The flag's new value is its old value or $\text{fires}(\text{core}(s), i)$. A false-to-true change therefore requires that event, whose definition requires a passing CERTIFY. CERTIFY costs 1. An already true flag stays true under the same disjunction. □

Checked as `mu_conservation`, `mu_conservation_trace`, `only_certify_certifies`, `a2` and `cert_permanent` (EarnedCore.v).

That third clause in the middle is Section 3's toll, for this machine. Which means this machine *is* one of the things Section 3 was about, and Section 3's theorem applies to it without my lifting a finger.

Theorem 16.6 (An instance of the certification system)**.** The states, the instructions, the step exec, the cost and the flag, with the toll of Theorem 16.5, form a certification system (Definition 3.1). So the universal floor (Observation 3.1) holds here: if $\text{cert}(s_0)$ is down and $\text{cert}(\text{run}(t, s_0))$ is up, then $\text{cost}(t) \geq 1$.

Proof. Take the state and instruction sets of Definitions 16.2 and 16.3, their step and cost, and the flag as the reading. These provide the five data fields of Definition 3.1. Theorem 16.5 supplies its one required rule. Applying Observation 3.1 gives the claimed bound. □

Checked as `earned_cs` and `earned_core_floor` (EarnedCoreLinks.v); `earned_core_floor` is `universal_nfi_any_substrate`, the checked name of Section 3's theorem, applied to `earned_cs`. The same floor is proved directly, without the link, as `nfi_floor` (EarnedCore.v).

Then Section 10. Section 10 said: take any base, and any record that answers only to the base and never comes back down, and the record is a latch. This machine has a base: a program together with a core, stepping by the program step. And it has a record, the flag, with the ledger riding along. So the question is whether it's an *honest* extension in Section 10's five-rule sense (Definition 10.2), and if it is, Section 10 does the rest.

Theorem 16.7 (The flag is a latch of the base)**.** Let the base have states (program, core), stepping by the program step, starting at start(a, b)'s core for any a, b, and halting when the program has halted. Map each (program, state) to (program, core). Then the machine, run as a closed machine on (program, state), is a base-driven record over that base, and so its flag factors as the latch (Observation 10.1) of the event "the next instruction is a CERTIFY that passes."

Proof. The rules of Definition 10.2 one at a time. Driven by the computation: the next flag is the old flag or "the next instruction is a passing CERTIFY," which is a function of the base state and the old flag (that's the step line). Ledger: μ only adds. Toll: Theorem 16.5. Permanence: Theorem 16.5. A reachable write: the three-line program of the worked run raises the flag from a start. Section 10's theorem then gives the factorization. □

Checked as `earned_core_honest` and `earned_core_is_latch` (EarnedCoreLinks.v), the latter by `record_axis_is_latch_holds`; the step-level facts are `cert_latch` and `base_blind` (EarnedCore.v).

So far, nothing new. The toll, the floor and the latch are things Parts II and III already promised about any machine of this kind. The next subsection is the part neither of them could give.

16.6 Commitments are earned

Here's the shape before the words. Pick any run from a clean start, any list of instructions at all, chosen by somebody who'd love to cheat. Look at the moment a COMMIT would pass. Then, somewhere earlier in that run, there's a CHECK of exactly the same claim that passed and paid, at the same edition of the counter, and between that CHECK and now, nothing touched the counter. Not "probably." Not "on the programs I wrote." On every list of instructions anybody can write.

To say "nothing touched it" exactly, one word.

Definition 16.5 (Untouched)**.** A list of instructions *mid*, run from a state s, leaves counter c *untouched* when, for every way of splitting *mid* as t_1, then i, then t_2, executing i at the core reached by running t_1 from s leaves both $\mathrm{ver}(\cdot, c)$ and $\mathrm{val}(\cdot, c)$ as they were.

Defined as `untouched` (EarnedCore.v).

Theorem 16.8 (A commitment was checked first)**.** Let s_0 be clean and *pre* any trace after which COMMIT p c would pass. Then *pre* splits as pre_1, then CHECK p c, then *mid*, where: the CHECK passes at the core reached by pre_1; CHECK p c and COMMIT p c each cost at least 1; c's version after pre_1 equals its version after *pre*; and *mid*, run from just after the CHECK, leaves c untouched.

Proof. COMMIT passes, so the claim $(p, c, \mathrm{ver}(c))$ for the current version is in the table. By the no-forging invariant (Theorem 16.12 below), that fact was written by a passing CHECK p c at some point pre_1, with the version c had then; since it's the current version too, the two versions are equal. Versions never go down, and any instruction that changes a counter's value also raises its version, so an instruction in *mid* that touched c would have left c's version higher at the end than at the CHECK. It isn't. So nothing in *mid* touched c. □

Checked as `earned_commitment_provenance` (EarnedCore.v).

Theorem 16.9 (A raised flag was checked, then committed, then certified)**.** Let s_0 be clean and t any trace with $\mathrm{cert}(\mathrm{run}(t, s_0))$ up. Then for some p and c, t is

$$pre_1,\ \text{CHECK}\ p\ c,\ mid_1,\ \text{COMMIT}\ p\ c,\ mid_2,\ \text{CERTIFY},\ post,$$

where the CHECK passes at its state, the COMMIT passes at its state, the CERTIFY passes at its state, c has the same version at the CHECK as at the COMMIT, and mid_1 leaves c untouched. And when the CERTIFY runs, the channel holds exactly the claim that COMMIT made: the fact (p, c, c's version at the CHECK). CERTIFY takes no operand; this is what ties it to the claim.

Proof. The flag starts down, so some first instruction raised it, and by Theorem 16.5 it's a passing CERTIFY. A CERTIFY passes only with a fact in the channel, the channel starts empty, and only a passing COMMIT puts a fact there, so some earlier COMMIT p c passed. Take the last one before the CERTIFY: the fact in the channel is the one it wrote. Theorem 16.8 at that COMMIT gives the CHECK, the equal version, and the

untouched stretch, and the equal version makes the committed fact (p, c, c's version at the CHECK). □

Checked as `earned_certification_provenance` and, with the channel clause, `earned_certification_same_claim` (EarnedCore.v).

And the failure side, so you can see the refusals are clean, with nothing left half-done:

Theorem 16.10 (Unearned steps trap and change no claim)**.** If COMMIT p c would not pass at a core k, executing it puts the trap up and leaves the channel and the table as they were. If CERTIFY would not pass at a state s, executing it puts the trap up and leaves the flag as it was.

Proof. If the trap is already up, the core is unchanged and its trap remains up. Otherwise, the failed COMMIT takes the failure branch of the instruction table: set the trap, leave every other core field alone. A failed CERTIFY takes the same branch, and its fires value is false. The flag update is then its old value or false, which leaves it unchanged. The instruction's charge still applies to the ledger. □

Checked as `unearned_commit_traps` and `uncommitted_certify_traps` (EarnedCore.v).

If that rings a bell, it should. It's Milner's move from the opening chapter: in LCF a theorem is a value that only the rules of the logic can build, so nobody can just print one (Milner and two colleagues on his Edinburgh project, Michael Gordon and Christopher Wadsworth, wrote the system up in 1979, and Gordon, who went on to build the HOL prover at Cambridge, told its history in 2000). Here the thing nobody can print is one bit, and what guards it is one step rule. Milner's guard was a type checker. Same architecture, much lower down.

A provenance theorem is also an itemised receipt, and the itemised receipt has a history that's exactly this gap. James Ritty ran a saloon in Dayton, Ohio, and believed some of his staff were pocketing money. On a steamboat trip to Europe in 1878 he watched a mechanism that counted the turns of the ship's propeller, and wondered whether something like it could count the money instead. In 1879 he and his brother John patented a "Cash Register and Indicator," and they sold a version under the wonderful name Ritty's Incorruptible Cashier. The first model was this book's counter (Section 6) with brass on it: press a key, the amount goes onto a dial showing the day's running total. No drawer, no paper. At closing time you hold the dial up against the cash. And that's all a total can do. It tells you *that* the money is short. It can't tell you which sale went missing or who rang it up. The paper came

later. The patents passed to other owners, first a Cincinnati salesman named Jacob Eckert, and in 1884 to John H. Patterson, who renamed the business the National Cash Register Company, and the story told about that company is that it added a paper roll: one copy kept for the shop's own books, one handed to the customer. That last part I only have from histories of the company and of the machine, so hold it the way you'd hold any company's story about itself. The order is right either way: first a total that can tell you something's wrong, then a list that tells you where. The ledger μ is the dial. Theorem 16.9 is the list: which CHECK, of which claim, about which edition, before which COMMIT. The theorem supplies that list about the run. The machine doesn't keep it as data anyone can read back from the end state, and nothing in the theorem gives a procedure for rebuilding it from there. A fact in the table holds a property, a counter and a version number. That's neither the counter's value back then nor a log of what happened in between.

Read Theorem 16.9 once more for what it *doesn't* say. Between the CHECK and the COMMIT, the counter is untouched. Between the COMMIT and the CERTIFY, it isn't promised anything. You can check A is even, commit to it, add one to A, and then certify. The flag goes up, and what it stands on is a commitment about the edition that was current when you committed. The certificate is historical: it says "this was checked and committed about this edition," and it says nothing about whether that's still true. I think that's the right meaning for a certificate (a birth certificate doesn't expire when the baby grows), but it's a meaning, and you should know which one you're getting. So a certificate doesn't say the claim holds at the CERTIFY, and it doesn't say the claim holds at the end of the run. A CERTIFY that rechecked the version would give a different guarantee. That's a possible extension, and the historical theorem doesn't need it.

16.7 What a passing check means

Provenance says a CHECK happened. It doesn't yet say the CHECK was *right*, and that's a separate question with a separate proof. A machine could have a perfectly ordered chain of check, commit, certify, all standing on a checker that says yes to everything. So: what does a fact in the table actually tell you about the counter?

Theorem 16.11 (Checker soundness)**.** Let s_0 be clean and t any trace, and let k be the core of $\mathrm{run}(t, s_0)$. For every fact (p, c, v) in k's table with $v = \mathrm{ver}(k, c)$, the property holds of the counter's current value: $\mathrm{holds}(p, \mathrm{val}(k, c))$. In particular, if COMMIT p c would pass at k, then $\mathrm{holds}(p, \mathrm{val}(k, c))$.

Proof. Carry one sentence through every instruction: every fact in the table has a version no higher than its counter's, and if its version equals its counter's, its property holds of its counter's value. A clean start has an empty table, so the sentence holds there. A passing CHECK adds a fact at the current version, and the checker said yes, so by Lemma 16.1 the property holds. A write raises one counter's version, so every old fact about that counter falls strictly behind and the second half no longer applies to it. Nothing else changes a value or a version. The sentence holds after every instruction, so it holds at the end. The part about COMMIT is the case where the claim is in the table at the current version. □

Checked as `checker_soundness` and `committed_claim_holds` (EarnedCore.v), through the invariant `sound` and its one-step form `sound_step`.

So, on a run from a clean start, a live fact means exactly this: one of three arithmetic properties holds of one counter's current value. Not "the program is correct." Not "the number is interesting." The property, of the value, now. That's all "checked" means on this machine, and it's enough to be true. Three events stay separate here. The property holding of the value is one. A CHECK passing is another: a CHECK of a property that holds still doesn't pass when the table is full or the trap is already up. A certification is a third, and it needs a COMMIT and a CERTIFY after the CHECK. So a CHECK that didn't pass says nothing about whether its property is false.

So who checked the checker? Nothing on the record did. From a clean start, the record carries the run's half of the story: a CHECK, then a COMMIT of the same claim about the same edition, then a CERTIFY. The other half, that the checker says yes exactly when the property holds, isn't in any run. It's Lemma 16.1, a proof about the little program the step calls, checked once by Coq and priced by nothing on the ledger. So a claim established here stands on two legs: a fact about this run, carried by the record, and a fact about the checker, carried by a proof that whoever reads the record has to accept. That's the Münchhausen trilemma from the opening chapter taking its third horn on purpose: the chain of reasons stops at a proof, named and on the page, where you can decide whether you'd have stopped there too. And the entitlement it gives is the reader's. The machine doesn't know anything. It keeps the order.

The question in that paragraph is older than any checker, and its most famous wording isn't about what you'd guess. "Who will guard the guardians?" gets quoted about governments and police and, lately, proof assistants. Where's it from? Plato's *Republic* has the worry, sort of: in Book III (at 403e), Glaucon, Plato's older brother and one of the people Socrates is talking with, says it would be absurd for a guardian to need a guardian, which is close in words and opposite in mood, a man calling the question silly. The Latin, *quis custodiet ipsos custodes*, is from the Roman satirist

Juvenal's sixth satire, and it's a joke about a husband. His friends tell him to bolt his wife in and set guards on her; the line asks who'll watch the guards, since she'll start with them. And it gets better. In 1899 an Oxford undergraduate, E. O. Winstedt, found extra lines of that satire in a medieval manuscript in the Bodleian Library, lines the other copies lack, partly overlapping the familiar ones, and the famous question sits in that tangle. Scholars have argued ever since about whether those lines are Juvenal's at all. So the most quoted sentence about trusting the checker comes with a doubtful source of its own. Does that change anything here? No. The only answer I know that works is de Bruijn's (Section 22 has it): you never get rid of the last guard, you make the last guard small enough to watch yourself. Lemma 16.1 is one line per property. Euclid had the instinct: five postulates you can hold in your head, and everything else meant to follow from them.

That split, a fixed checker proved sound from outside and a process that only gets to act when the checker says yes, has a name in the writing on self-improving programs. The AI researcher Jürgen Schmidhuber, at the IDSIA lab in Lugano, described a Gödel machine (2003) that rewrites its own code only after its built-in proof searcher has proved the rewrite useful, and Benja Fallenstein and Nate Soares (2015), researchers at the Machine Intelligence Research Institute in Berkeley, survey agents that reason about their successors through formal proofs a fixed checker accepts. Same architecture, aimed at a different question: they want to know when a machine may trust a smarter version of itself, and this machine only wants to know whether a flag was earned. Keeping the checker's proof outside the machine is my design choice. The nearest theorem, in Section 13, is narrower: no procedure the substrate can represent decides, for every program, a property that depends only on behaviour and holds of one program and fails of another. It doesn't say a machine can't check a proof about a checker, or verify a restricted part of itself.

Soundness rules out a lying checker. It doesn't rule out a pointless question: *at least 0* has a perfectly sound checker and an empty no pile, as the worked run showed.

Program checking has a theory of its own, and its founding paper asks for a checker that could have said no. The computer scientists Manuel Blum and Sampath Kannan (1989; journal version 1995) defined a checker for a computational problem: given a program and an input, it may run the program as it likes, and it must say "correct" with high probability when the program is right everywhere and "buggy" with high probability when its answer on that input is wrong. A checker that always says "correct" isn't one. Here the checker is fixed, deterministic and proved sound once, and the part of their demand that survives is the fourth clause of Thiele-complete in Section 16: some claim whose check can fail.

16.8 Nothing gets forged

The proof of Theorem 16.8 leaned on an invariant I promised to show you. Here it is. The fear it answers: could some clever sequence of instructions get a fact into the table without a CHECK writing it? A DEC with a funny jump target, a COMMIT after a trap, something I didn't think of? The answer is checked one instruction at a time, which is the only way I trust myself to answer it.

Definition 16.6 (Earned fact; no forgery)**.** After a trace t from s_0, a fact $f = (p, c, v)$ is *earned* when t is *pre*, then CHECK p c, then *mid*, where the CHECK passes at the core reached by *pre*, f is exactly the claim p of c at that core, and, if f is live at the end (v equals c's version after t), *mid* leaves c untouched. The trace has *no forgery* when every fact in the table after it is earned.

Theorem 16.12 (No forging)**.** For every core k, instruction i and fact f in the table after executing i at k: either f was already in k's table, or i is CHECK of f's property and counter, it passes at k, and f is the claim it writes. Consequently, if a trace has no forgery, so does that trace followed by any one instruction; and from every clean start, every trace has no forgery.

Proof. The first part is a case for each of the six instruction kinds: only a passing CHECK adds anything, and it adds exactly its claim. For the second, a new fact is earned by the CHECK that just wrote it; an old fact stays earned, and if it's still live after the new instruction, its counter's version didn't move, so the new instruction didn't touch the counter and the untouched stretch grows by one. A clean start has an empty table, which has no forgery, and the step carries it along any trace. □

Checked as `facts_step`, `no_forging_step` and `no_forging` (EarnedCore.v).

The closest earlier result is in capability hardware, and it's much bigger. The computer scientists Jack Dennis and Earl Van Horn, at MIT, introduced *capabilities* in 1966: tokens a process has to hold to touch an object, which it can pass on and can't forge. CHERI, a capability extension of real instruction sets built at the University of Cambridge and SRI International, rests on that idea, and in 2020 Kyndylan Nienhuis and twelve coauthors proved in the proof checker Isabelle/HOL, for the whole CHERI-MIPS instruction set, that arbitrary code given some capabilities can't increase them as it runs (up to any change of protection domain). That's nothing gets forged across a full instruction set. Theorem 16.12 is the same shape, one instruction at a time, for six instruction kinds and a table of checked facts.

16.9 A full table traps

Sixteen facts of room, and no eviction. Two halves to prove: nothing ever leaves, and nothing ever overflows.

Theorem 16.13 (Bounded table, no eviction)**.** Every instruction keeps every fact already in the table. If the table holds 16 or more facts, CHECK *p c* puts the trap up and leaves the table as it was. If the table holds at most 16 facts, it still holds at most 16 after any instruction.

Proof. With a raised trap no instruction changes the table. With the trap down, only CHECK can change it, and a successful CHECK adds one fact without removing an entry. Success requires a previous length strictly below 16, so the new length is at most 16. At length 16 or greater that branch is unavailable: CHECK raises the trap and preserves the table. Every other instruction preserves it as well. □

Checked as `facts_keep`, `full_table_traps` and `facts_bounded_step` (EarnedCore.v).

Why trap? Why not make room? Because "making room" means deciding which fact dies, and a fact that can die is a fact whose absence you have to explain later. A table that only grows is a history you can read back, and the provenance proofs above lean on exactly that. The bill for it is real, though, and it's in the subsection near the end of this chapter on what the machine doesn't do.

16.10 The window can't see the record

Section 7 was about windows: a view of a state that shows some of it, and the theorem that a window which can't tell two states apart can't price the step between them exactly (Observation 7.2). Here's the smallest pair I could build where the window is Minsky's view of the machine (line number and the two counters) and the record is everything else.

Run two traces from start$(0, 0)$. The *earned* run is CHECK *is zero A*; COMMIT *is zero A*; CERTIFY. The *idle* run is DEC A 0, three times. In the idle run, A is 0 every time, so each DEC just goes to the next line and writes nothing. Both runs end on line 4, with $A = 0$ and $B = 0$, both versions 0. Through the window, identical. One of them has paid 3, holds a fact, could commit again, and has its flag up. The other has paid nothing, holds nothing, can't commit, and its flag is down. Two photographs, same picture, different people.

Theorem 16.14 (Same window, different record)**.** The earned run and the idle run from start$(0,0)$ end at cores with the same window and the same versions of A and B, and with different ledgers, different flags, different tables, and different answers to "would COMMIT *is zero A* pass."

Proof. In the earned run, CHECK adds $(\mathsf{Zero}, A, 0)$, COMMIT puts that fact in the channel, and CERTIFY raises the flag. The ledger is $1+1+1=3$. No counter is written. In the idle run each DEC sees zero, so it advances the line without writing a counter or raising a version. Its three costs are zero and it adds no fact or commitment. Thus both final windows are $(4,0,0)$ with versions $(0,0)$. The earned table contains the current zero claim and the idle table is empty, giving the stated difference in whether another COMMIT would pass, as well as in the ledger and flag. □

Checked as `receipt_separation` (EarnedCore.v).

Theorem 16.15 (No window oracle)**.** There is no function g from windows to whole numbers with $g(\text{window}) = \mu$ at the end of every trace from start$(0,0)$. Likewise there is no function from windows to yes or no that gives the flag at the end of every such trace, and none that gives whether COMMIT *is zero A* would pass.

Proof. Such a g would have to give both 3 and 0 on the one window the two runs of Theorem 16.14 share. The same for the flag and the commit right. □

Checked as `no_mu_oracle`, `no_cert_oracle` and `no_commit_oracle` (EarnedCore.v).

This is Section 7's shape, made of pebbles: the record lives off the window. Anybody who only watches the counters and the line number, which is everything Minsky's machine *is*, can't tell you what was paid, what was committed, or whether the flag is up. Not because they looked carelessly. Because it isn't there to see.

Two things the same in everything you look at and still two things: philosophy has an old fight about exactly that, and it's worth a detour, because it tells you which side of the fight these theorems stand on. Leibniz said that for real things it can't happen. There are no two individuals indiscernible from each other, he wrote in 1716 to Samuel Clarke, the English philosopher and theologian who was taking Newton's side against him in an exchange of letters, and he told a story to go with it. A clever gentleman of his acquaintance, talking with him in front of the Electress Sophia of Hanover, Leibniz's friend and patron, in the garden at Herrenhausen she had helped lay out, thought he could find two leaves perfectly alike. She dared him to. He "ran all over the Garden a long time" looking, and it was "to no purpose." You can run his

search tonight with a handful of anything, leaves or coins or grains of rice: look hard enough and every pair differs somewhere. Then in 1952 Max Black, a Russian-born philosopher teaching at Cornell, pushed back with a universe you can't build at the table. Nothing in it but two iron spheres, the same size, the same colour, the same everything, some distance apart. Name a difference between them. There isn't one, and there are still two of them. People have argued about Black's spheres ever since, and I'm not going to settle it.

Here's why I don't have to. Leibniz's claim is about *every* property a thing has. The theorems above are about three: the line and the two counters. The two runs aren't indiscernible in his sense at all: one has a fact in its table and its flag up, and the other doesn't, and Coq tells them apart in a line. They're indiscernible to a window, which is a much cheaper thing, and it's the thing that matters for a machine, because an observer of a machine only ever gets a window. Leibniz's gentleman would always find his difference if he could look at everything. It doesn't help him if all he's allowed to look at is three things about each leaf.

There's a horse in this, too, if you squint. In Germany in the early 1900s a mathematics teacher named Wilhelm von Osten showed off a horse called Clever Hans, who answered arithmetic questions by tapping a hoof, and got them right, in front of crowds. Same answers out as a horse that could count: that's the idle run's matching window. Then the psychologist Oskar Pfungst stopped looking at the answers and started changing the conditions, and published what he found in 1907. Put blinders on Hans, or have him asked by someone who didn't know the answer, and the arithmetic fell apart. Hans was reading the questioner: tiny, unintended movements of the head and body as the hoof reached the right number. That's the probe. Pfungst didn't argue with the answers. He asked a question the answers couldn't fake, one that needed the real thing to be there, the way a COMMIT needs a checked fact to be in the table: run COMMIT *is zero A* after each of the two runs, and one passes and the other traps (Theorem 16.14). And the part of the story I like best: nobody was lying. Von Osten was never persuaded by Pfungst and kept showing his horse. A matching window doesn't need a cheat behind it, which is why you want a probe. An accusation is the wrong tool for it.

16.11 The exact price is three

Section 3 said a raised flag costs at least one. On this machine that's true and too modest. A raised flag needs a CERTIFY, a CERTIFY needs a COMMIT, a COMMIT needs a CHECK, and each of those is its own instruction costing one. So at least three. One of what: three units of μ, one per paid act, each one the "one" I picked

back in the table. And three is the number of links in the chain. It isn't a guess that happens to hold.

Observation 16.16 (The minimum price of a certificate)**.** If s_0 is clean and cert(run(t, s_0)) is up, then cost$(t) \geq 3$ and $\mu(\text{run}(t, s_0)) \geq \mu(s_0) + 3$. For programs: if the flag is up after n program steps from start(a, b), then $\mu \geq 3$ there.

Proof. Theorem 16.9 finds a CHECK, a COMMIT and a CERTIFY at three different places in t, each costing 1, and every other instruction costs at least 0. The ledger statement is Theorem 16.5. A program run is the run of the instructions it actually executes, so the program form follows. □

Checked as `certified_run_min_cost` and `program_certified_min_cost` (EarnedCore.v).

The three is three independent units, one for each part of a clean start. A flag that is down buys the first: a run that raises it costs at least one (`nfi_floor`). An empty channel buys the second: with the flag down and the channel empty, a run that raises the flag costs at least two (`nec_s_floor_two`). An empty table buys the third, and that is the theorem above. Leave out any one part and its floor fails, and a cheaper certificate is there: with the flag already up it costs 0, with a commitment already in the channel 1, and with a live fact already in the table 2 (`nec_s_clean_conjuncts_each_buy_one`, `nec_s_min_cost_needs_flag_down`, `nec_s_min_cost_needs_empty_channel` and `nec_s_min_cost_needs_empty_table`). The exact condition is `nec_s_floor3_iff`: every run from a state that raises the flag costs at least three exactly when the flag is down and one of two things holds: the trap is up, or the channel is empty and every fact in the table is about an older version of its counter. And three is paid exactly from every clean start whose trap is down (`nec_s_witness_from_every_clean_start`). All of these are in NecSClean.v, except `nfi_floor`, which is in EarnedCore.v.

Theorem 16.17 (Three is tight)**.** For all whole numbers a and b, run the program CHECK *is zero A*; COMMIT *is zero A*; CERTIFY for 4 program steps from start(a, b). If $a = 0$, it ends with the flag up and $\mu = 3$ exactly. If $a \neq 0$, it ends with the flag down and the trap up, and for every n, n program steps from start(a, b) leave the flag down.

Proof. If $a = 0$, the checker for Zero says yes at 0, so the first instruction adds $(\text{Zero}, A, 0)$ to the empty table and charges 1. No counter changes, so the second instruction finds that same fact, puts it in the channel and charges 1. The third finds

the channel occupied, raises the flag and charges 1. The line is now 4, beyond the three-instruction program, and the fourth program step adds no charge. The final ledger is exactly 3. If $a \neq 0$, the checker says no, so the first instruction sets the trap and leaves the flag down. A program run takes no step once the trap is up, so every later program step leaves the state, flag included, exactly where it is. □

Checked as `earned_run_check_can_fail` and `earned_run_refused_forever` (EarnedCore.v), with the program named `earned_run`, and the lemma `run_prog_trapped` for a trapped run standing still. The version whose check can't fail, CHECK *at least 0 A*; COMMIT *at least 0 A*; CERTIFY, certifies from every start for exactly 3 (`min_cost_tight`, with that program named `witness`). So the floor of three is reachable from any start at all, and that's the machine declining to police trivial checks. On that program the check distinguishes no inputs.

That's the worked run from the start of the chapter, now as a theorem. The floor is 3, and a run whose check could have failed pays exactly 3, so 3 is the price. The bound has no slack in it.

Don't mistake that 3 for a measure of how hard a claim is to certify. It's the same 3 for every claim, easy or hard, because only the three acts that make a claim are priced and the counters work out whatever the check needs for free. A cost of certifying that grows with the claim would have to price the checking itself, and that's a different question with its own literature. Four theoretical computer scientists, Guy Blanc, Caleb Koch, Jane Lange and Li-Yang Tan (2022), studied one version of it under the name the query complexity of certification: how many questions an algorithm has to ask a black-box function to hand back a small certificate for its answer. Their count moves with the function. Mine doesn't move at all.

And that's every piece of the model, standing on six instruction kinds: a base that computes everything two counters can, a ledger that's exact, a toll the universal floor applies to, a flag that's a latch of the base, a window that can't see the record, and, from a clean start, a commitment that can't happen unless a check of the same claim, on the same edition, came first and was right. Every result above is closed: Coq reports no assumptions for any of them. That isn't my say-so. EarnedCore.v and its link file, EarnedCoreLinks.v, each end by asking Coq to print the assumptions of each headline theorem, and the build of Section 22 compiles them and fails unless every one comes back closed. The project-wide audit of Section 22 lists these files too, and every theorem in them comes back closed there as well. The base file uses nothing but Coq's standard library, so you can check it with nothing else installed; the link file adds only the book's own abstract records and the library of undecidability proofs.

16.12 What it takes to be Thiele-complete

The front matter promised a definition of Thiele-complete with teeth, so here it is. First, why it needs teeth. The easy definition goes: a Thiele machine is Thiele-complete when its base can compute anything a Turing machine can (under Minsky's 2^n encoding, done outside the machine). It looks fine, and it's nearly empty. Take Minsky's two-counter machine, charge one mark for every move, and pick any yes-or-no question about its configuration as the record. Every move costs one, so every move that raises the record costs one, so it obeys the toll. That's a Thiele machine with a universal base: a clock bolted to a computer. A machine whose record never goes up passes too, since a toll on a raise that never happens is never broken. Neither one earns anything, and a name both of them get for free isn't saying much.

So the definition below asks for what the small machine actually does, written so that any machine at all can be held up against it. You name a language of claims first, then hand over a reading of the machine in that language, a list of things, and then the reading has to pass four clauses.

Definition 16.7 (Thiele-complete)**.** A *machine* has states, moves, a step, a cost for each move and a yes/no reading of each state, the *record*. A *claim language* for it is fixed first: a type of *claims* with an exact equality test; a *meaning* for each claim, a statement about a state; for each claim, a relation "the thing this claim is about is unchanged" between two states; and a proof that a meaning that holds keeps holding while the thing it's about is unchanged. An *interface* over that language hands over the rest: a *universal base*, meaning a two-counter configuration shown by each state (its *window*), a class of *live* states, a move for each two-counter instruction, and a loaded state for each start $(1, a, b)$; a *checker* that says yes or no to a claim at a state, with a proof that a yes means the claim's meaning, as the language gives it, holds of that state; a set of *clean* states; a *ledger*, a whole number for each state; and a *kind* for each move: a base move, CHECK of a claim, COMMIT of a claim, or CERTIFY. The machine is *Thiele-complete over* the language when some interface over it meets four clauses, and *Thiele-complete* when some interface, with claims, meanings and "unchanged" of its own, meets them. (a) *Universal base.* Each two-counter instruction's move is a base move; from every live state it acts on the window exactly as the instruction acts on a configuration, and the state stays live; each loaded state shows $(1, a, b)$, is live and is clean; no base move changes the record; and a record that's up stays up under every move. (b) *Earned record.* Clean states have the record down. Every run from a clean state that ends with the record up contains a CHECK of some claim c that passes, then a COMMIT of c with the thing c is about unchanged at every state in between, then a CERTIFY, with the record down just before that CERTIFY and up just after it. A passing check means its claim's meaning holds of the state it checked, and a

meaning that holds keeps holding while the thing it's about is unchanged. (c) *Exact toll.* Base moves cost 0, each CHECK, COMMIT and CERTIFY costs exactly 1, and each move adds its cost to the ledger. (d) *Non-vacuity.* Some claim c has a CHECK, a COMMIT and a CERTIFY such that those three moves alone, from a loaded state, raise the record exactly when c's meaning holds there; and c holds at some loaded state and fails at another. A Thiele machine whose base is universal, with no more asked, is *weakly* Thiele-complete.

Defined as `claim_language`, `lang_interface`, `thiele_complete_over`, `thiele_interface`, `thiele_complete_with` and `thiele_complete` (ThieleComplete.v), with the four clauses as `universal_base_clause`, `earned_record_clause`, `exact_toll_clause` and `non_vacuity_clause`, and the easy notion as `weakly_thiele_complete`. An interface over a language becomes an interface of the plain kind by taking the language's claims, meanings and "unchanged" as its own (`lang_ti`), and the four clauses are asked of that.

Why the language comes first. If the interface could name the meanings, it could take "the checker says yes" as the meaning of every claim, and checker soundness would hold for nothing. In the language-first form the meanings are written down before any checker is handed over, so "a checker proved to mean what it says" has the content it has on the small machine: what the claim says is the language's, and the proof is that a yes from the checker makes it true. The plain form stays because the theorems that follow hold for any interface at all, so they hold over every language too.

Read the four clauses as four plain sentences. (a) It computes anything a Turing machine can, under Minsky's 2^n encoding, done outside the machine, and the computing can't reach the record: the base moves are the two-counter machine, they leave the record exactly where it was, and a record that goes up stays up. (b) From a clean start, the record goes up only on top of the small machine's chain: a check that passed, a commitment to the same claim with nothing changed underneath, then a certificate. (c) Computing is free and each of the three acts costs one, so the ledger counts the acts. The steps in between go free. (d) Somewhere there's a claim whose check could have said no. A machine whose every claim is "at least 0" doesn't qualify, and neither does one with nothing to certify.

And look where the checker's honesty sits. In (b), "a passing check means its claim holds" is a field of the interface, proved once, outside every run, exactly where it sat for the small machine (Lemma 16.1), and over a claim language it's proved against meanings the interface didn't write. The definition doesn't ask the machine to vouch for its own checker. The nearest reasons not to ask are Gödel's second theorem, about a system proving its own consistency, and the diagonal of Section 13, about deciding

a property of every program. Neither forbids a machine checking a proof about a checker, so keeping that proof outside is a choice. Nor does it ask the checker to be something a machine could run. Clause (b) wants a yes or no for every claim at every state, sound for the claim's meaning, and nothing in it says that answer has to be computable. Every instance in this book has a checker that's an actual program, so anything those machines certify could be listed by a program that runs them. Inside Coq with no extra assumptions that's all there can be, since every checker you can write there is a program, the same point I made about what "undecidable" means in the library. An oracle gets in two other ways: read the definition as ordinary mathematics, where any function counts, or add an axiom that hands Coq a function knowing which programs run forever. Hand the definition a checker like that and it's still met, and the machine certifies claims no program could ever confirm. Nothing in this book builds one. So "a Thiele-complete machine certifies exactly the claims a program could list" is true only with "and its checker is computable" added, and the definition doesn't require that.

The same scope goes on the front matter's "it isn't more computing power." The record (the flag and the ledger) is never read by the step that computes the next core (`base_blind`), so erasing it changes no run, and it makes nothing faster. The checker is part of the machine under the record. With an oracle checker that machine decides what the oracle decides, since whether a CHECK traps is there for anyone to see; with a checker that's a program, it does what programs do. And a checker that's a program still makes CHECK a test at unit cost.

Theorem 16.18 (CHECK is a unit-cost test)**.** From any start (a, b), the single move CHECK $\mathsf{Ge}(m)$ A costs 1, and the trap latch is down after it exactly when $a \geq m$.

Proof. At a start the latch is down and the table is empty, so the check passes exactly when $\mathrm{eval}(\mathsf{Ge}(m), a)$ says yes, that is when $m \leq a$; a failing check sets the latch. The price table charges a CHECK 1. □

Checked as `check_ge_unit_cost` (ThieleComplete.v).

The two counters alone look at A only through the zero test of DEC, and from $a = m - 1$ and $a = m$ that test can't come out differently in fewer than m steps, so they need about m steps for what CHECK does in one. I argue that; nothing checks it. So the record makes nothing faster, and CHECK, a primitive test, can.

One word needs pinning before the theorems, using the three names set at the front of the book. In clause (c), the *exact toll* is instruction-exact pricing: each of the three acts, CHECK and COMMIT included, costs exactly one, so the ledger counts the acts that make a claim. Back in Section 6, one of the cited results (`a2_and_aara_i`

`ff_exact`) is about rise-only pricing in its tightest bill, the *exact schedule*: one mark on the step that raises the flag and nothing on any other step. Measured against the flag alone, the small machine isn't rise-only, since CHECK and COMMIT raise no flag and cost one each; measured the way (c) asks, it is instruction-exact. Neither is wrong. They answer different questions: does the ledger count the acts, or does it charge the flag and only the flag? Simulation-exact accounting, the third name, comes up with the universal program U below.

Theorem 16.19 (What every Thiele-complete machine has)**.** Let a machine be Thiele-complete through an interface. (1) It obeys the toll and has a universal base, so it's weakly Thiele-complete; its base runs every two-counter program step for step from every live state, and halting from a loaded state matches halting from the configuration it shows. (2) Along any run, the ledger rises by exactly the number of CHECK, COMMIT and CERTIFY moves in it. (3) A run from a clean state that ends with the record up contains at least three of those moves and raises the ledger by at least 3. (4) Some CHECK fails at some loaded state, on a claim that is false there, and some run from some loaded state raises the record. (5) On a run from a clean state, the move that raises the record is a CERTIFY. (6) Every move costs 0 or 1, and some move costs 0.

Proof. (1) A raise of the record isn't made by a base move, by (a), so it's a record move, which costs one by (c). Running a program on the base is induction on its steps, each matched by (a). (2) is (c), added up along the run. (3) Clause (b) finds a CHECK, a COMMIT and a CERTIFY at three different places, and (2) counts them. (4) is (d): the three moves certify exactly where the claim holds, the claim fails at some loaded state, and the check can't pass there because (b) says a passing check means the claim holds; at the loaded state where it holds, the three moves raise the record. (5) Clause (b) puts the record down right before a CERTIFY and up right after it, and the record never comes down, so that CERTIFY is the first and only move where it rises. (6) Every move is a base move or a record move, and (a) supplies base moves. □

Checked as `thiele_complete_is_weak`, `base_runs_every_program`, `base_halting_correspondence`, `ledger_counts_record_moves`, `certificate_costs_three`, `check_can_fail`, `some_run_certifies`, `only_certify_raises`, `complete_costs_at_most_one` and `complete_has_free_move` (ThieleComplete.v).

Theorem 16.20 (The language first, and what a certificate then means)**.** (1) A machine that is Thiele-complete over some claim language is Thiele-complete, so every part of Theorem 16.19 holds of it. (2) If a machine is Thiele-complete through an interface whose claims have an exact equality test, it is Thiele-complete over the language made of that interface's claims, meanings and "unchanged," with clause (b)'s proof that a meaning keeps holding as the language's own. (3) If a machine is Thiele-complete over a claim language L, some interface over L meets the four clauses and has this property: every run from one of its clean states that ends with the record up contains a CHECK of a claim c of L that passed, then a COMMIT of c, and L's meaning of c holds at the state the CHECK saw and at the state the COMMIT saw.

Proof. (1) Take the plain interface the language-first one becomes. (2) The checker's soundness and the keeping proof are the ones clause (b) already supplies. (3) Clause (b) gives the chain; the checker's soundness proof against L gives c's meaning at the state checked; L's keeping proof carries it, through states where the thing c is about is unchanged, to the state committed. □

Checked as `thiele_complete_over_complete`, `thiele_complete_with_over` and `over_certificate_means` (ThieleComplete.v).

Part (2) says where the strength of the language-first form lives: in the naming. Any reading can be turned into a language, so "Thiele-complete over some language" asks no more than Thiele-complete does. "Thiele-complete over L," for an L written down without looking at the checker, says that what the machine certifies is what L says. Every instance below is stated that way.

Each item in the proof names the clause it comes from, and that's all the definition gives. A Thiele-complete machine needn't have a fact table, a CHECK that traps when it fails, a COMMIT that looks anything up, or claims about counters. The definition asks for an earned chain on the runs that raise the record and leaves the rest to the machine. So the small machine's table provenance, its trap, and its channel are facts about that machine, and they don't carry over to every machine that meets the definition. Through a plain interface its claims can be any labels the interface names; over a claim language they are the language's. Either way its checker can be an oracle, as the paragraph above says.

The second half of (6), that some move costs 0, is the one that kills the clock.

Observation 16.21 (A clock is a Thiele machine and isn't Thiele-complete)**.** Take the two-counter machine, let every move cost 1, and take as the record any yes/no question about its configuration whatsoever; or take a flag that switches on, and stays on, the first time any chosen test of the configuration says yes. Either way the

machine obeys the toll and has a universal base, so it's weakly Thiele-complete, and it is not Thiele-complete. More generally, no machine whose every move costs at least 1 is Thiele-complete.

Proof. Every move costs one, so any raise costs at least one: the toll holds for any record. The machine's own moves are the two-counter instructions, which gives the universal base. But a Thiele-complete machine has a free move, by Theorem 16.19(6), and this one has none. □

Checked as `clock_weakly_thiele_complete`, `clock_not_thiele_complete`, `latch_clock_weakly_thiele_complete` and `latch_clock_not_thiele_complete` (ThieleComplete.v).

That's short, and it's short because clause (c) does exactly the job it was written for: it says computing is free, and a clock charges for computing. The other clauses earn their keep too, and that's worth checking, because a definition with a clause nothing needs is a definition hiding something.

Theorem 16.22 (Each clause does work)**.** (1) Take a free two-counter base with a ledger beside it, and one extra move TICK that costs 1 and raises a flag. It obeys the toll, and read with TICK as CERTIFY it meets the universal base clause and the exact toll clause; it is not Thiele-complete, and more generally no machine with a move that raises the record from every state is. (2) Take a free two-counter base and a record that never rises. It obeys the toll and meets clauses (a), (b) and (c); it is not Thiele-complete, and more generally no machine whose record never rises is.

Proof. (1) From a clean state, TICK alone raises the record. Clause (b) would need a CHECK, a COMMIT and a CERTIFY inside that one-move run, and one move can't be three. (2) Clause (d) needs a run from some loaded state that raises the record, and none does. □

Checked as `paid_latch_weakly_thiele_complete`, `paid_latch_meets_base_and_toll`, `paid_latch_not_thiele_complete`, `one_move_record_excluded`, `silent_weakly_thiele_complete`, `silent_meets_base_record_toll`, `silent_not_thiele_complete` and `never_certifies_excluded` (ThieleComplete.v).

The paid latch is the receipt for nothing from the opening chapter, wearing a universal base: honest about the price, silent about what was bought. The definition throws it out.

The silent machine meets three clauses and fails only (d). The paid latch fails (d) as well as (b), so it doesn't show (b) doing work alone, and nothing above says whether

a clock meets (a), (b) and (d). So here's the sharper check, the usual evidence that a definition isn't written around one machine: for each clause, a machine that meets the other three and fails it.

Theorem 16.23 (The four clauses are independent)**.** For each clause of Definition 16.7 there is a machine with an interface that meets the other three clauses and fails that one. Each of the four machines obeys the toll and has a universal base, so it's weakly Thiele-complete, and none of them is Thiele-complete through any interface at all. (a) fails for this chapter's machine with one more move, DROP, free and read as a base move, which lowers the flag and trips the trap latch. (b) fails for this chapter's machine with one more move, FREE, costing one and read as a CERTIFY, which raises the flag on any state whose trap is down, with no commitment behind it. (c) fails for this chapter's machine with every move costing one. (d) fails for the silent machine of Theorem 16.22(2).

Proof. (a) Nothing moves a trapped state's flag, so a run that uses DROP ends with the flag down. A run from a clean state that ends with the flag up never used DROP; it's a run of this chapter's machine, and Theorem 16.9 gives its chain. DROP is free and read as a base move, so (c) holds, and (d) holds as it does for the plain machine. A state with the flag up has it down after DROP. That breaks the permanence in (a), and permanence is about the machine alone, so no interface meets (a). (b) FREE never lowers the flag, costs one and is read as a record move, so (a), (c) and (d) hold as before. From a clean start FREE alone raises the flag, and a run of one move has no room for a CHECK, a COMMIT and a CERTIFY. Through any interface: (d) gives a loaded state from which the record rises, so its trap is down; (a) makes that state clean, so its record is down by (b); and FREE alone raises it, which (b) forbids. (c) Clauses (a), (b) and (d) never mention a price, and they hold as for the plain machine. Base moves cost one, so (c) fails, and every Thiele-complete machine has a free move (Theorem 16.19(6)). (d) is Theorem 16.22(2). □

Checked as `drop_meets_b_c_d`, `drop_fails_a`, `drop_not_thiele_complete`, `drop_weakly_thiele_complete`, `free_meets_a_c_d`, `free_fails_b`, `free_not_thiele_complete`, `free_weakly_thiele_complete`, `paid_meets_a_b_d`, `paid_fails_c`, `paid_not_thiele_complete`, `paid_weakly_thiele_complete` and `thiele_complete_clauses_independent` (ThieleCompleteIndependent.v), with `silent_meets_base_record_toll` and `silent_not_thiele_complete` (ThieleComplete.v). Three of the four are this chapter's machine with one thing changed, and in each case the one clause it fails is the only thing keeping it out. Clause (c) also fixes something it has no business fixing: the size of a mark. Double every price on the small machine and it leaves the class, because its CHECK now costs two and (c) asks for one. A definition

that a change of currency breaks would be a strange one, so here is (c) with the unit left open.

Theorem 16.24 (Any unit)**.** Call a machine *Thiele-complete at unit c*, for a whole number $c \geq 1$, when some interface meets clauses (a), (b) and (d) and this form of (c): base moves cost 0, each CHECK, COMMIT and CERTIFY costs exactly c, and each move adds its cost to the ledger. (1) At unit 1 that is Thiele-complete. (2) Multiply every price of a Thiele-complete machine by $c \geq 1$ and the machine is Thiele-complete at unit c; divide every price of a machine Thiele-complete at unit c by c and the machine, with the same states, moves, step and record, is Thiele-complete. (3) At every unit some move costs 0, so the clock of Observation 16.21, with any reading or any latch as its record, is Thiele-complete at no unit, and neither is any machine whose every move costs at least 1. (4) At unit c, a run from a clean state that ends with the record up contains at least three of the three acts and raises the ledger by at least $3c$; the claim it stands on held when it was checked and still held when it was committed; and the move that raises the record is a CERTIFY. (5) The small machine with every price doubled is Thiele-complete at unit 2 and isn't Thiele-complete.

Proof. (1) $1 \cdot r = r$. (2) Clauses (a), (b) and (d) never mention a price, and the two machines have the same runs. The ledger is multiplied or divided along with the prices, and $(\ell + c\,r)/c = \ell/c + r$ exactly, so the divided ledger still adds each divided price. (3) The move compiled from INC A is a base move, so it costs 0. (4) Clause (b) finds the three acts in the run as before, and each costs c. The other two parts are the unit-1 results on the divided machine, which runs exactly as this one does. (5) is (2) with $c = 2$, and the doubled machine has a CHECK that costs 2, which no Thiele-complete machine has (Theorem 16.19(6)). □

Checked as `thiele_complete_at_one`, `scale_complete_at`, `unscale_complete`, `scaled_has_free_move`, `clock_not_thiele_complete_at_any_unit`, `latch_clock_not_thiele_complete_at_any_unit`, `paid_moves_not_thiele_complete_at_any_unit`, `scaled_certificate_costs_three_c`, `scaled_committed_claim_holds`, `scaled_only_certify_raises`, `doubled_earned_scaled` and `doubled_earned_not_thiele_complete` (ThieleCompleteScaled.v).

So the class doesn't care what a mark is worth. It cares that computing is free and that the three acts cost the same. The clock fails at every unit for the reason it failed at one: it charges for computing. I keep unit 1 in the definition because the small machine is built that way and every count in this chapter is in its marks.

Now the other direction. A definition nobody meets is as useless as one everybody meets.

Theorem 16.25 (The small machine, the generic one and the sorted one are Thiele–complete)**.** (1) This chapter's machine is Thiele-complete, read with INC and DEC as base moves, CHECK p c and COMMIT p c as record moves on the claim "p holds of c," CERTIFY as CERTIFY, the window as (line, A, B), "unchanged" as "same value and same version of c," clean states as clean starts and the ledger as μ. (2) So is the same machine over any property language with an exact equality test and a checker proved to mean what it says, as long as some property is true of one number and false of another. (3) In particular the machine whose property is "this counter, read as a list, is sorted" is Thiele-complete. (4) Every Thiele-complete machine is a certification system (Definition 3.1) and obeys the universal floor; for this chapter's machine, that system agrees step for step and cost for cost with the one of Theorem 16.6, and for the generic machine with the generic machine's own. (5) Each of the three is Thiele-complete over its own claim language, fixed before the checker. For this chapter's machine the language's claims are pairs of a property and a counter, equal when both parts are, the meaning of "p holds of c" is the meaning holds of Definition 16.1 applied to c's value, written without the checker, and "unchanged" is "same value and same version of c." For the generic machine the language is built from the properties, their equality test and what they mean, and the checker isn't part of it; the sorted machine is that case.

Proof. (1) Clause (a) is Theorem 16.2 together with the step line: INC and DEC never touch the flag, and the flag never comes down. Clause (b) is Theorem 16.9 with checker soundness (Lemma 16.1), and a meaning about c keeps holding while c's value doesn't move. Clause (c) is the price table. Clause (d) is Theorem 16.17 with the claim "A is zero," which holds at $(0, b)$ and fails at $(5, b)$. (2) and (3) are the same argument with the property language left open, and the sorted claim holds of the list numbered 18, which is $[1, 2]$, and fails of 20, which is $[2, 1]$. (4) is (1) of Theorem 16.19 and Section 3's floor. (5) The interface over the language is the one of (1), (2) and (3) with the meanings handed over by the language, and the checker's soundness is Lemma 16.1 and its generic form. □

Checked as `earned_core_thiele_complete`, `earned_generic_thiele_complete` and `sorted_machine_thiele_complete` (ThieleComplete.v), (5) as `earned_core_thiele_complete_over`, `earned_generic_thiele_complete_over` and `sorted_machine_thiele_complete_over` (ThieleComplete.v), and `thiele_complete_floor`, `earned_complete_agrees` and `generic_complete_agrees` (EarnedGenericLinks.v).

The same fixed-language form is proved for the priced machine and its counter and sorted-list instances (`priced_thiele_complete_over`, `priced_core_thiele_complete_over`, `priced_sorted_thiele_complete_over`, PricedComplete.v), the multi-register machine (`ent2_mmachine_complete_over`, MultiThiele2.v), and the lifts over a base or its window (`lift_thiele_complete_over`, `lift_window_thiele_complete_over`, LiftCore.v). For a base simulated by finite lists of moves, it also holds for the macro lift and the five-lever example (`sim_base_lifts_over`, `fl_macro_lifts_over`, LiftMacro.v). The claims keep their stated meanings; the new interfaces supply the checkers.

And here's the thing every machine that meets the definition can't help doing. It's Section 7's shape again, only this time it isn't about one machine. It's about all of them.

Observation 16.26 (Every Thiele-complete machine hides its record and its ledger from its window)**.** Let a machine be Thiele-complete through an interface, and let its window be the two-counter configuration its universal base shows. (1) For every state s there's a loaded state and one free base move from it that lands on a state with s's window, with the record down and the ledger unchanged. (2) There's a loaded state and two runs from it that end with the same window, one with the record up and one with it down; and likewise two runs from one loaded state that end with the same window and ledgers at least 3 apart. (3) So no function of the window gives the record at the end of every run from every loaded state, and no function of the window gives the ledger. (4) The same holds of the certification system built from the machine.

Proof. Theorem 16.19(4) gives a loaded state and a run from it that raises the record, and (3) says that run paid at least 3. Say it ends with window (line ℓ, x, y). Two-counter instructions can steer any configuration to any other: empty A, fill it to $x+1$, empty B, fill it to y, then one DEC of A that jumps to line ℓ. Those are base moves, so from the same loaded state they reach the same window, cost nothing, and leave the record down, by (a) and (c). That's two runs, one window, two records and two ledgers, which is (2). A function of the window would have to give both answers on that one window, which is (3). For (1), load $(x+1, y)$ and take just the last DEC. (4) is (3) read through the certification system, whose runs are the machine's. □

Checked as `complete_every_window_printed`, `complete_hides_record`, `complete_hides_ledger`, `complete_no_record_oracle`, `complete_no_ledger_oracle` and `thiele_complete_hides` (ThieleCompleteWindow.v), and `complete_cs_window_blind` (EarnedGenericLinks.v).

The observer in that theorem sees one thing, the final window. An observer who sees the window before and after each step, or the whole trace of windows, or who is also handed the program and the start, is a different observer, and the collision says nothing about them; given the program and the start, the run is fixed and so is its record. The theorem's two runs also start from the same loaded state, so they're reachable witnesses, which is stronger than two arbitrary states that happen to look alike. And "no function of the window gives the record" is all it says. It doesn't say the window tells you nothing about it.

Notice how little that took, and say it out loud so nobody mistakes it for more. It's the fibre fact from Section 7 plus one observation: a universal base can print any window for free. Which is the point. The window here is everything a Turing machine would call the machine, the line and the counters. On a Thiele-complete machine, that's never enough to tell you what was established or what it cost, and it can't be, because computing is free and can't touch the record. A richer window, one that shows the flag, is a different window, and nothing above says it fails.

16.13 One machine that runs them all

The small machine raises its flag only by checking, committing and certifying, and it pays for that. The next question is whether one machine can stand in for all of them, the way Turing's universal machine stands in for every Turing machine. Here's what I mean by that. A *universal Thiele machine* is one fixed machine running one fixed program that runs any machine of a stated class as a guest, with the guest's record kept and the guest's toll collected by the host's own steps. No program running on the host has to keep a convention for it.

And I want it the hard way. The easy way is to give the host a move that means "take one step of the guest," and then "it runs any guest" is true because I wrote it that way. There's a host like that two subsections on, after the two programs, and it's useful as a picture. It isn't Turing's kind of universal. Turing's universal machine has no instruction for "do what the machine on the tape would do." It has the same kind of instructions every other Turing machine has, and one fixed table, and the table does the reading and the deciding and the carrying out. That's the bar here: one fixed program, written in the small machine's own six kinds of instruction, with no step of the guest built into anything.

Turing's own explanation is worth having in your head, because the program below copies its shape. In the 1936 paper the universal machine's tape starts with the machine to imitate, written out as a string of letters, its standard description, and after it the imitated machine's complete configurations, one after another: where its head is, what state it's in, what's on its tape. To take one step, the universal machine

finds the last configuration, looks through the description line by line, comparing, until it finds the line for that state and that symbol, and then writes out the next configuration the way that line says. Then it goes back and does it again. It leaves marks on squares so it doesn't lose its place. Every bit of that is ordinary instructions on an ordinary tape, and nothing in its table knows any particular machine.

Turing's universal program came first on paper, and one that people actually came to use arrived through the side door. John McCarthy, who had started the artificial intelligence project at MIT with Minsky, was writing up his new language Lisp around 1960 and wanted to show it was neater than Turing machines, so he wrote a universal function for it, *eval*, the language describing how to evaluate itself, and argued it was shorter and easier to follow than a universal Turing machine. In his own history of Lisp, that was for the paper, a thing to read; the notation for writing programs as data was made up for the paper with no thought of using it in practice. Then Steve Russell (S. R. Russell in McCarthy's account), a programmer on that MIT project, noticed that eval could serve as an interpreter and coded it by hand, and there was a running language with an interpreter that nobody had planned. McCarthy adds that the surprise froze some decisions he'd made "rather lightheartedly" for the paper. So the trick has a habit: written to be read, it ends up being run. The program below goes the other way round, on purpose. It's written to run, and the proofs check that what runs is what the page says.

The host. The program runs on a machine with the small machine's instructions and the small machine's prices: INC, DEC, HALT for free, and CHECK, COMMIT, CERTIFY for one each, with the same table of sixteen facts, the same channel, trap, ledger and flag. One change: it has a counter for every whole number, each with its own version, where the small machine had two. No new kind of instruction, no new price. Take away the table, the channel and the flag and what's left is old: a machine with as many registers as it needs, adding one, taking one away and jumping on zero, is the unlimited register machine of John Shepherdson and H. E. Sturgis, two logicians who proved in 1963 that such machines compute every partial recursive function.

Definition 16.8 (The host machine)**.** The *host machine* is the machine of Definition 16.3 with one change: it has a counter, with its version, for every whole number r (a *register*), and INC r, DEC $r\ j$, CHECK $p\ r$ and COMMIT $p\ r$ name one register each. Its property language has one property, Slot. Write $\langle m, n\rangle$ for the number $(2n+1)\cdot 2^m$; every positive number is $\langle m, n\rangle$ for exactly one pair, and 0 is no pair. Number the small machine's properties $\mathsf{Zero} \mapsto 0$, $\mathsf{Even} \mapsto 1$, $\mathsf{Ge}(n) \mapsto n+2$. The checker of Slot says yes at a value x exactly when $x = \langle m, v\rangle$ and the small machine's checker says

yes to the property numbered m at v; its meaning is the same with holds in place of the checker.

Defined as the machine of EarnedMulti.v, with `pair`, `unpair`, `pcode`, `heval` and `holds` (UniversalCodes.v).

There's one place the guest's machine shows up in the host. The host's single property knows how to read the small machine's three-word property language: "this number is the pair (property, value) and the property holds of the value." It doesn't know the small machine's step. That's the counter version of Turing's universal machine knowing the alphabet its guests write in.

Two facts that force the design. Why more counters? Because of two facts, and the program looks the way it does because of them.

The first is about reading. On this machine a program that wants a counter's value in its own control flow has to take pebbles out (DEC) and put them back (INC), and every INC, and every DEC on a counter above 0, is a write, so the counter's version goes up. A fact is about one version, so a read done that way kills every live fact about the counter. Not every look is a write: CHECK reads a value without writing it, a DEC on 0 changes no version, and a fresh CHECK after such a read can make a new live fact. But the guest's counters are exactly the ones the host has to read by taking pebbles out, again and again, to decide which way a DEC jumps and to work out a property's answer, and each of those reads kills whatever fact was kept there. So the host can't keep the guest's facts on its copies of the guest's counters.

The second says that even if reading were free, a fixed program couldn't keep them by sending each guest property to one of a fixed, finite list of host properties and checking it on the same register the guest's fact is about. That's all the theorem below rules out, and it's the pigeonhole.

Theorem 16.27 (No host fact on an exact copy of the guest's counter)**.** Take any host property language with an exact equality test and any checker, a finite list Q of host properties, and any translation of the small machine's properties into members of Q whose host check passes whenever the guest's check passes on the same number. Then there are $n \neq m$, with $\mathsf{Ge}(n)$ and $\mathsf{Ge}(m)$ translated to the same host property, such that the guest program CHECK $\mathsf{Ge}(n)$ A; COMMIT $\mathsf{Ge}(m)$ A traps within two steps from every start, while on every host state with the trap down and room in its table, and some register holding a number at least n, the translated CHECK and COMMIT on that register both pass and put the committed claim in the channel.

Proof. A fixed program mentions finitely many properties, so Q is finite, and the translation sends the infinitely many $\mathsf{Ge}(0), \mathsf{Ge}(1), \ldots$ into it: two of them land on the same member. The guest's COMMIT asks for $\mathsf{Ge}(m)$, and its table can only hold $\mathsf{Ge}(n)$, so it traps (or its CHECK already trapped). The host sees one property twice: its CHECK passes, since the guest's would, and its COMMIT finds exactly what that CHECK wrote. □

Checked as `pigeonhole_collision` and `no_exact_copy_host` (UniversalNoCopy.v), with the same statement on the host's certification system as `multi_cs_no_exact_copy` (UniversalInterpreterLinks.v).

So the host keeps the guest's facts somewhere else: in *slots*, extra registers used for nothing but holding facts, one per checked fact, each holding the pair (property number, value) that the guest checked. The pigeonhole doesn't bite a slot, because the property lives in the slot's number. The instruction doesn't have to carry it: CHECK Slot on a slot asks about whatever pair is sitting there, and there are infinitely many pairs. A slot is written only when the program is moving a fact into it. And when the guest writes the counter a fact was about, the program gives every slot about that counter a nudge, one INC and one DEC, which leaves the number alone and pushes the version up, so the host's facts go stale at exactly the moment the guest's do.

The program. Here's U the way Turing would describe it, one register at a time and then one job at a time. Registers 0 and 1 hold the guest's A and B. Register 2 holds the guest's whole program as one number: each instruction gets a number (an opcode paired with its operands, INC as 0, DEC as 1, HALT as 2, CHECK as 3, COMMIT as 4, CERTIFY as 5), and the list of them is packed with $\langle \cdot, \cdot \rangle$, the empty list as 0. Register 3 is the guest's line. Registers 4 to 13 are scratch. Registers 14 and 15 count how many CHECKs on A and on B have passed. Registers 16 to 47 are *mirrors*, sixteen for each guest counter, and registers 48 to 79 are the slots, sixteen for each guest counter. The guest's table holds sixteen facts in total, across both counters, and all sixteen can be about one counter, so the host gives each counter room for all of them. That's a host reservation, and the guest's own capacity stays sixteen in all. Register 80 is a dead end that's never written.

Each guest step goes like this. At line 1 the program copies the program number and the guest's line into scratch, putting the originals back as it goes, because copying a counter on this machine means emptying it into two others and refilling it. If the guest's line is 0, or past the end of the program, it stops. Otherwise it peels instructions off the front of the copy, one per line, until it holds the one at the guest's line, and splits that into opcode and operands. Then a chain of DECs on the opcode sends it to one of six handlers.

- INC c: INC the guest's counter c, nudge every slot about c and zero its mirrors, add one to the guest's line, back to line 1.
- DEC c j: DEC the guest's counter c. If it was 0, add one to the guest's line. If it wasn't, nudge every slot about c, zero its mirrors, and set the guest's line to j. Back to line 1.
- HALT: stop.
- CHECK p c: let k be the count of passed CHECKs on c. If k is below 16, build $\langle$number of p, value of $c\rangle$ in slot k of c, run CHECK Slot on that slot, set its mirror to the number of p plus one, add one to the count and to the guest's line, and go back to line 1. If k is 16, run CHECK Slot on the dead end, which holds 0, which is no pair, so it traps, just as the guest's seventeenth CHECK does.
- COMMIT p c: look through the sixteen mirrors of c for one equal to the number of p plus one. At the first match, run COMMIT Slot on that slot. With no match, run COMMIT Slot on the dead end, which traps.
- CERTIFY: run CERTIFY, add one to the guest's line, back to line 1.

Why the mirrors? COMMIT has to find the slot holding the claim, and it can't read the slot to find out, because reading would make the fact stale. So beside every slot sits a mirror the program is allowed to read: the property's number plus one while the slot's fact is live, 0 once the guest has written the counter. If a guest CHECK fails, the host's CHECK on the slot fails too, since the slot holds exactly the pair the guest checked, and both machines trap.

Count what costs anything. Fetching, unpacking, copying, nudging, branching: all INC and DEC, all free, the way computing is free on the small machine. The only paid instructions are the CHECK, COMMIT and CERTIFY sites, and each guest CHECK, COMMIT or CERTIFY makes the program run exactly one of them. So the host's ledger lands on the guest's, mark for mark. And when the guest's flag goes up, the host's goes up by the host's own CERTIFY, standing on the host's own COMMIT and CHECK on a slot. It isn't copying a bit. It runs the same chain on its own registers.

Definition 16.9 (The program U, and loading a guest)**.** U is the fixed list of 3728 host instructions described above, starting at line 1 and mentioning registers 0 to 80; exactly 69 of them cost anything. For a guest program P and a start (x, y), the *loaded host* $\mathrm{hload}(P, x, y)$ has register 0 at x, register 1 at y, register 2 at the number of P, register 3 at 1 and every other register at 0, every version 0, line 1, an empty table and channel, the trap down, ledger 0 and flag down. Write $\mathrm{grun}(m)$ for m program steps of P from $\mathrm{start}(x, y)$, and $\mathrm{hrun}(n)$ for n program steps of U from $\mathrm{hload}(P, x, y)$.

Defined as `U`, `hload`, `prog_code`, `grun` and `hrun` (UniversalLayout.v, UniversalSim.v, UniversalRun.v, and UniversalCodes.v for `prog_code`), with the 69 paid sites listed exactly by `U_paid_sites` (UniversalLayout.v).

Theorem 16.28 (U keeps pace with its guest)**.** For every guest program P, start (x, y) and number of guest steps m, there's a number of host steps n such that one of two things holds. Either the host is back at line 1 with $m \leq n$ and *related* to the guest: registers 0 and 1 hold the guest's A and B, register 3 holds the guest's line, the guest's trap is down, the two ledgers are equal, the two flags are equal, and the host's table and channel hold, fact for fact, slot facts matching the guest's facts and channel. Or both have stopped, with the same counters, the same trap, the same ledger, the same flag, and matching tables and channels. And from any related pair where the guest hasn't stopped, one guest step is matched by at least one host step that lands related again, or with both stopped.

Proof. The one-step part is a case for each thing the guest can do at its line: stop; INC; DEC, taken or not; CHECK, passing, failing, or with a full table; COMMIT, found or not; CERTIFY, passing or not. For each, the matching stretch of U is proved to land where the case says. The relation carries the bookkeeping that makes the next case go through: each count register equals the number of slots in use for its counter, each slot in use holds (property number, checked value), a slot's fact is current exactly when the matching guest fact is, a live slot's mirror is the property's number plus one and a stale one's is 0, and unused slots, unused mirrors and the dead end are 0. A freshly loaded host is related to the guest's start. The run statement is induction on m. □

Checked as `hload_rel` and `U_step` (UniversalSim.v), and `U_simulation` (UniversalRun.v).

Theorem 16.29 (Halting and answers match)**.** For every P, x and y, the guest's run of P from start(x, y) halts if and only if U's run from hload(P, x, y) halts. And whenever the guest has halted at step m and U at step n, the host's registers 0 and 1 hold the guest's A and B, and the two traps, the two ledgers and the two flags are equal.

Proof. Theorem 16.28: a halted guest is matched by a stopped host, and a host that has stopped can't be back at line 1 with the guest still running, because line 1 isn't a stop. A halted machine stays halted, so the two halting points carry the same values. □

Checked as `universal_halting` and `universal_output` (UniversalRun.v).

Theorem 16.30 (The host's flag is the guest's, and its ledger is exact)**.** (1) The guest's flag is up at some step of its run if and only if the host's flag is up at some step of U's run. (2) For every m there's an n at which the two are related, with $m \le n$, or both stopped, with $\mu(\text{hrun}(n)) = \mu(\text{grun}(m))$. And for every n there's an m with $\mu(\text{grun}(m)) \le \mu(\text{hrun}(n)) \le \mu(\text{grun}(m+1))$: in between matching points, the host's ledger never gets ahead of the guest's next step.

Proof. (1) Flags are equal at matching points and never come down. (2) Ledgers are equal at matching points; inside the stretch of U that runs one guest instruction, at most the one paid site of that instruction runs, so the host's ledger sits between the guest's before and after it. □

Checked as `universal_flag_iff` and `universal_ledger_exact` (UniversalRun.v).

That's a third sense of "exact," and it's the plainest of the three: two ledgers, one number.

Theorem 16.31 (The host earns its own flag)**.** If the host's flag is up after n steps of U from $\text{hload}(P, x, y)$, then the instructions U executed contain, in order, a passing CHECK Slot on a slot, then a passing COMMIT Slot on the same slot, with the slot untouched and its version unchanged in between, then a passing CERTIFY; the slot held $\langle \text{number of } p, v \rangle$ at the CHECK, with $\text{holds}(p, v)$. And the guest's own run from $\text{start}(x, y)$ contains a passing CHECK $p\ c$, then, with c untouched and its version unchanged, a passing COMMIT $p\ c$, then a passing CERTIFY, with c holding v at its CHECK.

Proof. By Theorem 16.30 the guest's flag rises too; find the guest step where it rises, a passing CERTIFY, and the related host point just before it. The guest's channel there names a fact, and the relation says the host's channel names the matching slot fact. The host's own flag rises at its own first passing CERTIFY, and nothing paid runs between the related point and that CERTIFY, so the host's channel hasn't moved: it names that slot fact. The host machine's version of Theorem 16.9 gives the host's CHECK and COMMIT on that slot. The record kept with the slot says what pair it held and when, and the checker's meaning gives $\text{holds}(p, v)$. The same theorem on the guest's run, up to the step where its flag rose, gives the guest's chain on the same p and c, and the record says c held v at its CHECK. □

Checked as `universal_earned` (UniversalRun.v).

Theorem 16.32 (The host is Thiele-complete, and U's halting is undecidable)**.** (1) The host machine, with Slot as its property and one move per instruction, is Thiele-complete (Definition 16.7), and Thiele-complete over its own claim language, whose claims are "Slot holds of register r," whose meanings are written without the host's checker, and whose "unchanged" is "same value and same version of register r"; and U's run from hload(P, x, y) is a run of it. (2) Read as a certification system it obeys the universal floor; a run of U that raises the host's flag costs at least 3 there. The system built from its Thiele-completeness agrees with it step for step and cost for cost, so the same run costs at least 3 on that one too, and the universal floor of 1 holds there directly. (3) U halts from hload(P, x, y) exactly when P halts from start(x, y), so the question "does U halt from this loaded host?" is undecidable in the library's sense of Theorem 16.4.

Proof. (1) The base is registers 0 and 1, run by INC and DEC; the claims are "Slot holds of register r"; the rest is the host machine's own versions of the small machine's theorems, and for non-vacuity, register 0 loaded with 1, which is $\langle 0, 0\rangle$, the pair of Zero and 0, against register 0 loaded with 0, which is no pair. Over the claim language the interface is the same one, with the meanings handed over by the language. (2) is Theorem 16.25(4) and Theorem 16.31. (3) is Theorem 16.29, and the identity map reduces the small machine's halting question, undecidable by Theorem 16.4, to this one. □

Checked as `universal_thiele_complete`, `universal_thiele_complete_over` and `U_run_on_host_machine` (UniversalRun.v), and `multi_cs_floor`, `interp_U_certified_floor`, `interp_U_complete_floor`, `interp_complete_agrees`, `interp_halting_iff` and `interp_halting_undecidable` (UniversalInterpreterLinks.v).

Put it together, plainly. One fixed program, 3728 instructions of the small machine's six kinds, with no guest step built into anything, runs every program of the small machine from every clean start start(x, y). It halts exactly when its guest halts, with the guest's answer in its first two registers. Its flag goes up at some step of its run exactly when the guest's goes up at some step of the guest's run, by its own check, its own commit and its own certificate. Its ledger is the guest's, mark for mark, wherever the two line up. And the machine it runs on is Thiele-complete. Since the small machine runs every two-counter program, U runs every computation, under Minsky's 2^n encoding, done outside the machine. That's the universal Thiele machine of this book, for the small machine's programs; the next subsection carries it to every computably presented Thiele machine.

The fine print, before you take that anywhere. The guests start clean, from start(x, y); the whole-run theorems are about loaded hosts, and only the one-step

theorem is about any related pair. The host's property reads the small machine's three-word language, so a guest with a different property language needs a different host property; the host of the next subsection has one, a single property that reads a universal claim language. Sixteen slots per counter is the guest's whole table size, reserved for each counter in case every fact is about that one. And it's slow: every guest step unpacks the program number from the front, on a machine where copying a number means moving it a pebble at a time.

Enforcement in the step is the reference monitor's move from the opening chapter, Anderson's 1972 move: one mechanism everything has to pass through. What's put together here is a priced, earned record under that move, carried to every guest by one fixed program. The move itself is Anderson's. Kevin Hamlen, Greg Morrisett and Fred Schneider (2006) worked out, in their model, which policies mechanisms like that can enforce at all, and their paper is where I'd go to see why enforcing a property of every run escapes the Rice-style walls that deciding one runs into.

Why make such a fuss about proofs that walk every case? Because the first universal machine had bugs. It's the machine in Turing's 1936 paper, and nobody gets to call Turing careless. In 1947 Emil Post, in an appendix to a paper on a problem of Thue, pointed out that the construction leaned on a convention it hadn't earned, and he patched a small defect in how the universal machine writes out each complete configuration. Around the same time Donald Davies, who joined the National Physical Laboratory in 1947, where Turing was designing a computer, worked through the universal machine's tables and found more mistakes, much to Turing's annoyance, as the accounts of Davies's life have it. (It's the same Davies who later named the *packet* in packet switching.) His corrections were finally printed in a collection edited by Jack Copeland, a philosopher and historian of computing who runs an archive of Turing's papers, *The Essential Turing*, in 2004. They may be some of the first bugs ever written, in a program for a computer that didn't exist yet. That's a lovely claim and a hard one to check, so take it as a lead. I can't give it to you as a fact. What it changes here: a universal machine is exactly the kind of construction where a careful person leaves a hole, *U* has 3728 lines for a hole to hide in, and its theorems are checked by Coq one case at a time, which is the kind of check that catches Davies's kind of slip. It doesn't catch a definition that says the wrong thing. That's why the definitions are on the page, for you.

16.14 Every computably presented Thiele machine, on one host: driven, decided, priced in whole units, carried as a latch

U runs the small machine's programs, and through them every computation, under Minsky's 2^n encoding, done outside the machine. But a Thiele machine from some-

where else has its own states, its own moves, its own prices and its own reading, and running it through the two-counter encoding keeps its computation and nothing else. So the question Turing's machine raises for this book is the bigger one. Take any Thiele machine whose workings can be computed. Is there one fixed program that runs every such machine as a guest, keeping its halting, raising its own flag exactly when the guest's reading would rise, on an earned chain of its own, and keeping a ledger equal to the guest's own prices, with the host's flat ones nowhere in it? There is, within limits I'll list, and here it is.

First the class, said plainly. A *computably presented* Thiele machine is a certification system in Section 3's sense (states, moves, a step, a price for each move, a yes/no reading) together with a *driver* that picks the next move from the current state or says stop, and a way of writing states and moves as whole numbers. "Computably" means four little programs, written in the plainest language of computable functions there is, the one built from zero, plus one, picking an argument, composing, recursion and search: one computes the driver, one the step, one the price, and one the reading, and the reading's program answers yes or no on every state there is. Run from a start s_0, the driver's moves give one run. Its *ledger* after n moves is the sum of their prices, and its *latch* after n moves says whether the reading has been yes at any state so far.

The host gets one thing the small machine doesn't have: a paid move that certifies nothing. On the small machine every charge comes from a CHECK, a COMMIT or a CERTIFY, and those can bill seven: one passing CHECK and six COMMITs of the same claim cost seven and raise no flag. But every passing CHECK takes one of the table's sixteen entries for good, every COMMIT rewrites the channel, and a failing CHECK traps. So the host machine gets a seventh instruction kind, PAY: it costs one, goes to the next line, and changes nothing else; it never raises the flag, takes no table entry and leaves the channel alone. That's why the compiler uses it.

How far the small machine gets without it is proved, and the count is short. Take a run that never traps and ends with the flag down. No CERTIFY ran in it, since a passing one raises the flag and a failing one traps. Every CHECK in it passed, since a failing one traps, and each passing CHECK filled one of the table's sixteen entries, so there were at most sixteen. Every COMMIT in it committed a claim the table holds, and the table holds at most sixteen, so at most sixteen COMMITs can be of claims the run hadn't committed before. That's 32, and the only way past it is a COMMIT of a claim already committed earlier in the run (`pf_bound_start`, PayFree.v). A claim names its counter's version, and the version changes at every write, so a repeat needs its counter left alone since the earlier COMMIT. A guest whose every move writes its counters can't do that, so it runs out of ways to pay after 32 (`pf_no_repeat_boun d`). A simulated program that runs longer than that has to bill somewhere, and PAY

is where. Whether some other host without PAY could do the same job is a question I haven't settled, and I don't claim it.

The definition of Thiele-complete sorts every paid move into CHECK, COMMIT or CERTIFY, so PAY has to be filed under one of them, and it's filed as a CHECK of the empty claim, a claim that never holds. That's a filing for the interface. It isn't what PAY does on the machine: a CHECK that fails on this machine traps, and PAY goes on to the next line. With PAY filed that way the host stays Thiele-complete (`pu_universal_thiele_complete`, UniversalPRun.v), and Thiele-complete over its own claim language, in which the empty claim means nothing (`pu_universal_thiele_complete_over`, UniversalPRun.v), and its floor of three for a flag raised from a clean start still holds with PAY present (`pu_multi_certified_run_min_cost`, EarnedMultiPriced.v).

The program is U with one more handler: a seventh branch on the guest's opcode, for PAY, that runs the host's own PAY once. Call it U_P; it's 3735 instructions, with 70 paid sites, the 69 of U plus the PAY. The guest it runs is a program of the small machine with PAY added, and that guest is where the presented machine goes. Its four programs are compiled into two counters, by the packing Minsky used, with the undecidability library's compiler from computable functions to counter machines doing the hard part; the guest keeps the presented machine's state coded in its counter B, takes one of its moves by running the compiled step, pays the move's price in PAYs, and, when the compiled reading says yes, checks, commits and certifies. The host's one property is fixed too: a claim is either one of the small machine's three words about a number, or "run reading program r on the coded state v," and the host's checker evaluates that with one fixed routine for every presented machine there is. So nothing in U_P knows any particular presented machine. The machine arrives as data, the way Turing's guests arrived on the tape.

A fixed checker that runs whatever program it's handed and always answers needs saying how, so here is the routine. A claim of the second kind carries a number r. The checker decodes r as a counter-machine routine: its instructions, where they sit, which register holds the answer, which register holds the fuel, and how many registers to load. If r doesn't decode to a routine, the answer is no. Otherwise the checker loads the routine's registers from v, reads a fuel number f out of v too (the fuel register's entry in v's prime factorisation), runs the routine for at most f steps, and says yes exactly when by then the routine has left its code with 1 in its answer register. Every input gets an answer within f steps, so the checker is total, and what it means is that bounded run: yes if and only if the routine halts within the fuel the value carries, with answer 1 (`cg_ueval_total` and `cg_ueval_iff`, CompilerChecker.v, through the host's one property `pu_heval`, UniversalPCodes.v). That's how it stays clear of the diagonal. A total evaluator E that agreed with every

total yes-or-no program on that program's code alone would let $g(n) = \neg E(n, n)$ contradict itself at its own code. This checker never evaluates a program on its code alone: the fuel comes with the claim's value. The guest compiled from M runs M's reading routine itself and puts at least the step count it took into the fuel register, and at such a value the checker says yes if and only if the reading says yes (`cg_checker_exact`).

Theorem 16.33 (One host runs every computably presented Thiele machine)**.** Let M be a computably presented Thiele machine, so that a driver picks its next move (one run per start), a program decides its reading on every state, and its prices are whole numbers; and let s_0 be a start state. Load U_P with the guest compiled from M's presentation and s_0 coded in. What the host carries of M's reading is its latch. Then: (1) U_P halts exactly when M's run stops. (2) For every n such that M hasn't stopped before move n, there's a host time t at which the host's register for the guest's counter B decodes to M's state after n moves, the host's ledger is M's ledger after n moves plus a *surcharge*, and the host's flag is M's latch after n moves; and if M stops at n, such a t has U_P halted as well. (3) The host's flag is up at some time exactly when M's reading is yes at some state of its run. (4) If the host's flag is up at time t, the host's own instructions up to t contain a passing CHECK on a slot, then a passing COMMIT on the same slot with the slot untouched in between, then a passing CERTIFY; and the slot held the claim "reading program r says yes of v" for M's own reading program r, that claim holds, and v decodes to a state of M's run where the reading is yes.

Proof. Two layers, each proved on its own. The bottom layer is the compiler: for every presented machine, its guest program's run passes through points that code M's states in order, with the guest's ledger equal to M's ledger plus the surcharge and the guest's flag equal to M's latch, and the guest's own flag stands on an earned chain whose claim is M's reading program applied to the coded state where the reading first said yes. The top layer is U_P, proved the way U was in the previous theorems, with the PAY case added: it keeps pace with any guest of this kind, halting and flag matched, ledger equal at matching points, and its flag earned on a slot. Compose the two. The earned part takes one more step: the host's chain gives a guest CHECK that passed, and a guest run that ends with its flag up has exactly one passing CHECK, the compiler's CHECK of the reading on counter B. So the guest CHECK the host's chain points to is that one, and its claim is M's reading at the first state where it said yes. □

Checked as `presented_universal` (PresentedUniversal.v), whose five parts are also stated separately as `presented_universal_halting`, `presented_universa`

`l_points`, `presented_universal_halt_point`, `presented_universal_flag_iff` and `presented_universal_earned`; the host layer is `pu_universal_halting`, `pu_universal_flag_iff`, `pu_universal_ledger_exact` and `pu_universal_earned` (UniversalPRun.v).

Two things the theorem doesn't say. First, the host's flag is M's latch. If M's reading says yes at some state and no at a later one, the host's flag stays up, so it says the reading was yes somewhere in the run, and it doesn't say the reading is yes now. Second, what the host re-establishes is the reading: the claim it checked is "M's reading program says yes of this coded state," and that's all. A bare certification system needn't tie its reading to anything true about sorting, proofs or the world outside. If M's reading is meant to say a list is sorted, that it does is a separate correctness theorem about the reading, and the host doesn't supply it. The two universal programs also keep their own guests: U runs the small machine's programs over its three words about numbers, and U_P runs computably presented machines over the universal claim language.

Now the surcharge, because that's where the ledger stops being exact, and it can't be helped. The host raises its flag the only way the host machine allows, by a CHECK, a COMMIT and a CERTIFY, and those cost three. If M's first move that turns its reading to yes costs at least three, the host pays the difference in PAYs and nothing more is owed. If it costs less, the host pays three there anyway, which is the larger of three and the move's price. The surcharge starts at that move: while M's latch is down it's zero, so a run that never certifies is matched exactly. Whether anything could do better depends on what M paid before. When M's whole ledger at the raise is below three, no program on the host machine can match it, since a raised host flag costs three from a clean start. When M paid more than that earlier, nothing here says this compiler's surcharge is the least possible: part (1) of the theorem below bounds this compiler, and part (3) is the only lower bound proved.

Theorem 16.34 (Exact, within two, and no better)**.** Let M, s_0 and the loaded host be as in Theorem 16.33, and let M's reading be no at s_0. (1) For every n before M stops there's a host time at which the host's ledger lies between M's ledger after n moves and that plus two, with the state decoded and the flag matched as before. (2) If moreover every move that first turns M's reading to yes costs at least three, the surcharge is zero and the host's ledger equals M's exactly at those times. (3) If M's latch is up after n moves while its ledger is below three, then no run of the host machine from a clean start, under any program at all, ends with its flag equal to that latch and its ledger raised by exactly M's ledger.

Proof. (1) and (2): the guest pays M's prices in PAYs, a unit at a time, except at the move where M's reading first says yes. There it has to run its CHECK, COMMIT and CERTIFY, three units, and it makes up the rest of the move's price in PAYs, so it pays the larger of the price c and three. The price is at least one, because the move raised a reading (the toll), so the excess is at most two, and none when $c \geq 3$. (3): a raised flag on the host machine costs at least three from a clean start, whatever the program, and M's ledger is less. □

Checked as `presented_universal_surcharge_le_two`, `presented_universal_exact` and `presented_universal_no_exact_below_three`, with the corollary for U_P itself, `presented_universal_U_no_exact_below_three` (PresentedUniversal.v). Part (3) is an observation in the sense of Section 3: it reads off Observation 16.16. If the reading is already yes at the start, the host has to raise its flag at once, and that costs three on top of a ledger of nothing.

And one machine carried all the way, by proof, so the class isn't empty. Its states are whole numbers, its one move adds one and costs three, its driver always picks that move, and its reading is "at least three." The four programs are written out from the library's building blocks and proved to compute those four things. Loaded into U_P from 0, the host never halts, its flag goes up, at the host time matching move n its ledger is exactly $3n$ and its flag is up exactly when $n \geq 3$, and the raised flag stands on a host chain whose slot claims the machine's own reading of a state at least three (`pu_demo_computably_presented`, `pu_demo_never_halts`, `pu_demo_flag_rises`, `pu_demo_exact` and `pu_demo_earned`, PresentedDemo.v). That's proved. Nothing executes U_P on that guest.

Now the limits, each of which is in the statement, said plainly.

- **Readings that are decided.** The reading's program has to answer yes or no on every state. A reading a program could only confirm, waiting forever on the no cases, isn't covered, and neither is a reading no program computes at all.
- **One run per start.** The driver picks the move, so the guest is deterministic. A machine whose next move is somebody's free choice gets in only with that choice written into its driver.
- **The record is the latch.** The host's flag is up once M's reading has been yes at any state of the run, which is exactly the latch of Section 10. A reading that goes yes and then back to no is carried as "it was yes once."
- **The ledger is exact up to the surcharge.** When the reading starts at no, at most two, and zero when the first raising move costs three or more; when it starts at yes, three. And no program on this host, run from a clean start, can do better when the guest certifies with its whole ledger below three.

- **Prices in whole units, paid one at a time.** A move that costs a million is a million PAYs. Nothing here is fast, and the compiler's packing is Minsky's, so it's slower still.
- **Proved; never run.** The demonstration machine's numbers come from theorems. Nobody has run U_P on it: the files it lives in aren't extracted, the compiled programs are far too big for any machine to step through, and some of the library's pieces don't compute inside the checker at all. Section 19 says what was run.

16.15 A simpler host, with the guest step built in

There's a much simpler host than U, and I keep it in the book because it's easy to read and it shows the copy rule in one line. Its universality is by construction: it has a move that runs one guest step by calling the small machine's own step. So it isn't the universal Thiele machine of this book. U is. Read it as a picture of what U does the long way, and as a host that starts from any guest state at all, clean or not.

It carries a guest inside its own state: the guest's program as data, the guest's counters, its table, its ledger, its flag. The host has one extra move, run one guest step. In that one move it looks at the guest's flag, takes the step, looks again, and if the flag went from down to up, the host's own copy of that flag goes up in the same move, and the host pays for it.

Definition 16.10 (The host)**.** A *guest* is a program P of this chapter's machine together with a state g of it (Definition 16.3). A *host state* h has six parts: its own core $\mathrm{core}(h)$, its ledger $\mu(h)$, its own flag $\mathrm{own}(h)$, the guest program $P(h)$ held as a list of instructions, the guest state $g(h)$, and the *mirror* $m(h)$, a yes/no copy of the guest's flag. The host has two kinds of move. OWN i runs the instruction i on the host's own core, costs what this chapter's machine charges for i, and raises own exactly when that machine's flag would rise. GSTEP b costs b. If the host's core is trapped it does nothing else. Otherwise it moves the host to its next line and, if the guest's next instruction costs at most b, runs that instruction on $g(h)$; in the same move the mirror becomes $m(h) \vee (\neg\mathrm{cert}(g(h)) \wedge \mathrm{cert}(g'))$, where g' is the new guest state. The host's reading is $\mathrm{own}(h) \vee m(h)$. Its stored program is $U_0 = [\mathrm{GSTEP}\ 1;\ \mathrm{OWN}\ (\mathrm{INC}\ A);\ \mathrm{OWN}\ (\mathrm{DEC}\ A\ 1)]$: one guest step, then two moves that bring the host back to its first line. (The Coq file calls it `U`; I write U_0 here so nobody mixes it up with the universal program U.)

Every instruction of the small machine costs at most one, so GSTEP 1 runs every guest instruction there is. Why charge the budget b? Why not the cost of whatever instruction got fetched? Because charging the budget keeps the price a function of

the move alone, which is what the certification system of Part II asks for: you read the price of a host move off the move, the way you read the small machine's prices off its table, without peeking inside the guest. The bill for that is easy to see. A host that steps a free guest instruction, an INC say, still pays one for the move.

So the host's ledger is closer to a clock than to a receipt for its guests: one unit per guest step, whatever the step was. It never falls behind the guest's own bill (part (3) of Theorem 16.37 below), and it matches that bill only when every guest step happens to cost one. Could a cleverer host have simulation-exact accounting, a ledger equal to its guest's? Not with a price read off the move alone. GSTEP 1 is the same move whether the guest's next instruction is a free INC or a paid CHECK, so one price can't match both. A simulation-exact host has to be built the other way, with no guest step to put a price on: that's U, which pays only on the record moves it runs itself, so its ledger equals its guest's (Theorem 16.30).

And one thing about the definition before the theorems, so you know which of them carry weight. GSTEP runs the guest's instruction by calling the small machine's own step on the guest's state: the guest's step is a primitive of the host. So "the host runs any guest" (Theorem 16.35) is true by construction. The host was built to do exactly that, and the proof is mostly unfolding. The theorems with something in them come after it. From a loaded host, or any state where the two agree, the mirror always equals the guest's flag (Observation 16.36). Nothing raises the mirror unless the guest itself passes a CERTIFY inside a guest step the host paid for (Theorem 16.38). No OWN move, in any host program at all, touches the guest's state or the mirror (the second sentence of that same theorem), and no host move of either kind rewrites the guest's program (`hexec_gprog`, UniversalThiele.v). And the mirror is tied to the guest's earned chain: after any host run from a loaded host, if the mirror is up, the guest's own run from its start contains a passing CHECK, then a COMMIT of the same claim, then a CERTIFY (`host_mirror_earned`, UniversalThiele.v). So this host runs any guest by construction and keeps the guest's record by theorem, as a copy of the guest's flag. It never earns a flag of its own. U does both the long way: it runs the guest with its own instructions, and it earns its own flag.

Theorem 16.35 (The host runs any guest)**.** (1) If the host's core is not trapped, then after n moves GSTEP 1 the guest state is the guest program's own run of n steps from $g(h)$, the guest program is unchanged, and the host is still not trapped. (2) If the host's core is at line 1 and not trapped, then $3n$ steps of the program U_0 leave the guest exactly where n steps of its own program put it. (3) For every two-counter program M and inputs a, b: M halts from $(1, a, b)$ exactly when the host program made of OWN moves of the compiled M halts, from a host whose core starts at (a, b), whatever guest that host carries.

Proof. (1) Induction on n: a host that isn't trapped stays untrapped through a guest step, and with budget 1 the guest step is the guest's own step. (2) One pass of U_0 is a guest step, then INC A and DEC A back to line 1, so it is (1) once per pass. (3) A host program made only of OWN moves is the small machine, core for core, and Theorem 16.3 does the rest. □

Checked as `universal_simulation`, `universal_program_simulation` and `host_thiele_complete` (UniversalThiele.v).

Observation 16.36 (The copy is the record)**.** Say a host state h *agrees* when $m(h) = \mathrm{cert}(g(h))$. Every host move takes an agreeing state to an agreeing state, so every host trace and every stored host program does too, and a freshly loaded host agrees. In particular, if h agrees and is not trapped, the mirror after n moves GSTEP 1 is the guest's flag after n steps of its own program.

Proof. One move at a time. An OWN move touches neither the mirror nor the guest. A trapped GSTEP touches neither. An untrapped one sets the mirror to the old mirror or the crossing, and the guest's flag is a latch (Theorem 16.5), so if the old mirror equalled the old flag, the new mirror equals the new flag. The last sentence is this together with Theorem 16.35. □

Checked as `record_agreement_step`, `record_agreement`, `record_agreement_program`, `hload_agrees` and `simulated_record` (UniversalThiele.v).

Theorem 16.37 (The host's step collects the guest's toll)**.** (1) If h agrees, is not trapped, and the guest step that budget b allows takes the guest's flag from down to up, then the mirror is down before GSTEP b and up after it, the host's ledger rose by exactly b, and $b \geq 1$. (2) In the run of (1) of Theorem 16.35, from a host that agrees, if the guest's flag goes up between its steps n and $n+1$, the mirror goes up between the host's moves n and $n+1$, and that move costs 1. (3) For every host trace t, $\mu(g(\mathrm{run}(t,h))) + \mu(h) \leq \mu(\mathrm{run}(t,h)) + \mu(g(h))$: the host's ledger grows at least as fast as the guest's.

Proof. (1) The guest's flag rises only on a passing CERTIFY (Theorem 16.5), which costs 1, and the budget let it run, so $b \geq 1$; the mirror was the guest's flag, so it was down, and the move sets it to the crossing. (2) is (1) at the matching move. (3) Induction on t: a guest step that runs costs the guest at most the budget the host paid, and every other move leaves the guest's ledger alone. □

Checked as `toll_enforced_by_host_step`, `every_guest_crossing_is_a_host_crossing`, `host_pays_for_guest`, `host_cost_covers_guest` and `simulated_cost` (UniversalThiele.v).

Theorem 16.38 (No free host certification)**.** If the mirror is down at h and up after a host move i, then i is GSTEP b with $b \geq 1$, the host was not trapped, the guest's next instruction was CERTIFY, and the guest's flag went from down to up in that move. A trace made only of OWN moves leaves the mirror and the guest state exactly where they were. And the host's own flag rises only by an OWN CERTIFY that passes on the host's core.

Proof. An OWN move copies the mirror unchanged, and a trapped GSTEP does too, so the rise is an untrapped GSTEP b whose guest step crossed; a crossing is a passing CERTIFY, which costs 1 and needed the budget. The own flag moves by this chapter's step line. □

Checked as `no_free_host_certification_step`, `no_free_host_certification`, `no_free_host_certification_program`, `guestless_runs_leave_mirror` and `own_record_only_by_certify` (UniversalThiele.v).

Theorem 16.39 (The host is a Thiele machine)**.** For every host state and move, if the host's reading goes from no to yes, the move costs at least 1; the same holds for the mirror alone and for the own flag alone. So the host read through own $\vee$ m, the host read through m alone, and the guest class are certification systems (Definition 3.1), and any host trace that raises the host's reading costs at least 1.

Proof. The mirror's half is Theorem 16.38; the own flag's half is the small machine's toll on the host's core. The trace form is Section 3's universal floor. □

Checked as `host_toll`, `host_own_toll` and `host_mirror_toll` (UniversalThiele.v), and `host_cs`, `host_mirror_cs`, `guest_cs` and `host_nfi` (UniversalThieleLinks.v).

Remark 16.1 (Two guests, run)**.** Load the guest CHECK *is zero A*; COMMIT *is zero A*; CERTIFY; HALT with both counters at 0. After two moves GSTEP 1 the mirror is down; after three it is up, the guest's flag is up, the host's own flag is still down, and the host has paid 3. Under the program U_0 the mirror is down at host step 6 and up at step 7, with 3 paid. The guest CERTIFY; CERTIFY traps on its first step: after five moves GSTEP 1 the mirror is down, the guest is trapped, and the host has paid 5 for nothing.

Checked as `demo_certifies`, `demo_program_certifies` and `demo_forgery_fails` (UniversalThiele.v).

That copy can't be faked. The proof walks every host move there is, and the copy only rises in a guest step where the guest itself ran CERTIFY and passed. A host program that never touches the guest leaves the copy exactly where it was. From a loaded host the copy always equals the guest's flag, and the host's ledger always grows at least as fast as the guest's, and a three-instruction loop runs any small-machine program one guest step per pass. Here the mirror, the crossing test and the charge sit in the host's step rule, so Theorems 16.37 and 16.38 hold for every sequence of host moves, including ones that never run a guest. That's the simple version. Everything it gets from its built-in move, *U* gets from ordinary instructions, plus an exact ledger and a flag of its own.

16.16 The machine versus the program it runs

All three hosts in this chapter come down to one distinction, and it's worth saying on its own, because the two halves are easy to run together: *the machine doing the stepping* (what the machine is) and *the program it runs* (what runs on it). The toll of Section 3 is a property of the machine doing the stepping. A rule a program keeps holds as long as the program keeps it.

Here's the picture I keep for it. Play Monopoly on the counter of a real bank. The game has a bank of its own, with its own rules, and one of them is printed in the rules sheet: the Bank never goes broke, and if it runs out of money the Banker may issue more by writing on any ordinary paper. Inside the game that's fine; the players agreed to it. The real bank's teller, standing right there, has no opinion about any of it. Push a thousand in Monopoly money across her counter and her ledger doesn't move, because her rules are about her own money. The game's money is nothing to them. The real bank is the machine doing the stepping. The game is a program it happens to be hosting. A rule that lives in the game holds as long as the players keep to it. A rule that lives in the bank holds whatever the players do.

(And since the game is on the table: its box told an origin story for decades, that an out-of-work salesman named Charles Darrow invented it during the Depression. The board is older. Elizabeth Magie, an American writer and game designer with an economic argument about land to make, patented The Landlord's Game in 1904, with a square track, railroads and a square that sends you to jail, and in 1902, in *The Single Tax Review*, described it as "a practical demonstration of the present system of land-grabbing with all its usual outcomes and consequences." Darrow sold his version to Parker Brothers in 1935, and the company bought Magie's patent for a few hundred dollars, five hundred by the accounts I found. A story told by the company

selling the product is a lead to check, every time. It changes nothing about banks. It's a good reminder of who writes the story on the box.)

A fixed machine can enforce an invariant in its own transition table, and that includes a Turing machine. Give its tape a ledger track and a certification cell, and write a table that never sets the cell without bumping the ledger: that machine obeys the toll in its step relation. It's an instance of the abstract certification system, so it can't be a counterexample to it. What it doesn't do, built the ordinary way, is enforce the toll on the programs it runs. A universal machine whose table was written only to interpret carries the simulated certification bit as plain data on its tape. If the host is programmable, so that other code shares that tape with the guest, any of that code can flip the bit without paying, because the host's step has no opinion about it. So an ordinary programmable computer, as built, doesn't enforce the toll for its programs. That's how it's built. It could. The simpler host above enforces it by having its own step mirror the guest's bit and charge for its crossing (Observation 16.36 and Theorem 16.38), and the universal programs U and U_P are fixed tables that go one better: they earn their own flag, by their own check, commit and certificate, exactly when the guest's would rise, and pay on their own ledger what the guest pays, U exactly and U_P to within a surcharge of at most two when the guest's reading starts at no (Theorems 16.31, 16.30, 16.33 and 16.34). That's a statement about which reading a step rule prices. It isn't a claim that Turing machines can't price one.

Computing ran into this exact gap in hardware, and how it got out is worth the detour. In 1974 two computer scientists, Gerald Popek and Robert Goldberg, asked when a machine can run guests, a whole operating system inside another, efficiently and safely. Their answer is one sentence you could check against any instruction set. Call an instruction *sensitive* if it reads or changes the machine's own settings, the things the host has to keep for itself, and *privileged* if running it outside the host's own mode makes the machine stop and hand control to the host, which is called a trap. For the kind of machine they modelled: if every sensitive instruction is privileged, a host can be built. The guest runs at full speed, and whenever it touches something that matters, the machine itself stops it and lets the host decide. That's a host whose step has an opinion.

Then the obvious next question: did the machines people actually bought pass the test? The most common one didn't. In 2000 John Scott Robin, an Air Force officer, and Cynthia Irvine, a computer-security researcher at the Naval Postgraduate School, went through every instruction of Intel's Pentium and found seventeen that are sensitive and not privileged; POPF, which loads the processor's flags, is the one usually pointed at. A guest could run them and the host would never hear about it. The fixes came two ways: VMware rewrote the guest's code on the fly so the troublesome instructions never ran as written, and in the middle of the 2000s Intel

and AMD added hardware support so they'd trap. Does that change anything here? Not the theorems. Popek and Goldberg were after speed and isolation. A record and a price weren't their question, so their result isn't mine and mine isn't theirs. It changes the picture, though. The universal programs sit where they'd want a host to sit: the host's own steps collect each guest's toll and earn each guest's record again. And an industry learned the same lesson the expensive way: a rule binds the guest only if it lives in the step underneath.

The smallest checked version of "a ledger beside an ordinary machine" is a Turing machine configuration with a counter next to it, where each step adds one, plus one more if it writes a nonblank symbol. The counter doesn't disturb the machine.

Theorem 16.40 (A counter beside a Turing machine changes nothing)**.** Let a Turing machine with transition table δ step a configuration c to $\mathrm{tm}_n(\delta, c)$ after at most n steps (stopping when δ has no entry). Pair each configuration with a counter, and step the pair by stepping the configuration and adding the step's charge. Then for every n, δ and pair (c, m), the configuration part after at most n steps of the pair is $\mathrm{tm}_n(\delta, c)$. In particular this holds from $(c, 0)$.

Proof. By induction on n. For $n = 0$ both sides are c. For $n + 1$, if δ has no entry at c, both runs stop and both sides are c. Otherwise the pair steps to (c', m') where c' is exactly the Turing machine's next configuration, whatever m' is, and the induction hypothesis at (c', m') finishes it. □

Checked as `thiele_simulates_turing_gen` and `thiele_simulates_turing` (ProperSubsumption.v). The "Thiele" in those names is a local record, a Turing configuration and a number. It isn't a Thiele machine in this book's sense, and the file says not to cite it as one simulating Turing machines. A counter beside a machine is the scorekeeper's tally from Section 3: honest about the machine, and kept by whoever holds the pencil.

One limit on all of this, so it isn't read as more. Certification is one event. Other kinds of observable a machine might carry would each need their own event, their own evidence and their own cost law, and nothing here supplies those.

16.17 Parts II and III, run on this machine

Parts II and III proved their counts and their collisions for any system with the named pieces, and they left a promise: that the machine of this part would be put through them. Here it is. The eight-state machine of Section 4 gets run by the small machine, price for price. Structural entitlement, the count of what narrowing a list of rivals costs, is proved for every Thiele-complete machine, with the small machine as

a worked member, a larger search on the host that has a counter for every number, and a run that shows the count's one awkward hypothesis, the covering, can't be dropped. And the time a check saves gets an exact exchange rate, for the part of the machine that has no record. (The no-bare-verifier result is in Section 7, next to the collision it comes from.)

The eight-state machine, run by the small machine. Look at the small machine through a window that shows two things: its line number read modulo four, which I'll call its *slot* (the eight-state machine's word from Section 4, unrelated to U's slot registers), and its flag. Call a state *live* when its trap latch is down, the fact "$A \geq 0$" about A's current version is in its table, and its channel holds a commitment, which is what CERTIFY needs in order to run.

Definition 16.11 (The eight-state machine, compiled)**.** Stamp is the single instruction CERTIFY. Next-slot is the single instruction INC B. Jump to slot a, for a one of $0, 1, 2, 3$, is the four instructions COMMIT $(A \geq 0)$ A; COMMIT $(A \geq 0)$ A; INC B; DEC B a. The *setup* is the two instructions CHECK $(A \geq 0)$ A; COMMIT $(A \geq 0)$ A.

The two COMMITs in a jump change nothing but the line and the channel, and each costs one, which is where the jump's price of two comes from. The INC makes B positive, so the DEC takes one off and jumps to line a for nothing, and B is back where it was.

Theorem 16.41 (The small machine runs the eight-state machine)**.** (1) For every live state s and every move m of the eight-state machine, running m's instructions from s ends in a live state whose window is the eight-state machine's step of s's window by m, and the ledger has risen by exactly m's charge: one for stamp, two for jump, zero for next-slot. (2) The same holds for any list of moves: the window after the compiled list is the eight-state machine's run of the list, the state is live, and the ledger has risen by the sum of the charges. (3) So a run of compiled moves from a live state that takes the flag from down to up has raised the ledger by at least one. (4) From any start (a, b), the setup ends in a live state with window (slot 3, flag down) and the ledger at 2; after it, any compiled list runs as on the eight-state machine from (slot 3, flag down), with the ledger at 2 plus the sum of the charges; and if the flag is up at the end, the whole run, setup included, contains the earned chain of Theorem 16.9.

Proof. (1) By cases on the move. On a live state CERTIFY runs, because a committed claim is on the channel; it raises the flag, moves the line on by one and charges one. INC B moves the line on by one and charges nothing, and it touches only B and B's version, so the fact about A and the commitment stand and the state stays live. In a

jump, each COMMIT finds its fact in the table and succeeds, charging one, and the INC and the DEC are free and end at line a, whose slot is a; the flag isn't touched. (2) is (1) over the list, by induction. (3) The eight-state machine's own toll (Section 4) says a run of its moves that takes the flag from down to up is charged at least one, and (2) carries that charge across. (4) CHECK ($A \geq 0$) passes at every value of A and the COMMIT after it passes, which takes line 1 to line 3, slot 3, with two marks paid; then (2) applies. A raised flag was raised by a CERTIFY, and Theorem 16.9 gives the chain. □

Checked as `frag_runs_finite_machine`, `frag_pays_finite_price`, `frag_runs_finite_trace`, `frag_certification_paid`, `frag_setup_live`, `frag_program_runs` and `frag_program_earned` (FragmentSmall.v), with the room argument for the eight-state machine itself as `frag_fin_toll` and `frag_cert_system` in the same file. Three reservations, said now. The setup's claim is true of every value, so its CHECK can't fail; it's there because the small machine won't CERTIFY without a checked, committed claim. It isn't there as a test. And the jump's price of two is two paid COMMITs, a choice a program makes that the machine allows and doesn't force. The window is a projection, too: the small machine's full state keeps far more than eight states' worth.

And the whole small machine isn't the eight-state piece it runs. It merges some states for free.

Theorem 16.42 (The small machine's merges)**.** (1) A move of the small machine that takes the flag from down to up is CERTIFY; CERTIFY merges two different full states into one (a state with the flag down and the same state with the flag up land on the same state), and it costs at least one. (2) DEC A 2 also merges two different full states, and it costs nothing. So it is false of the small machine that every move that merges is priced.

Proof. (1) Only CERTIFY changes the flag, and it costs one. The two states with the flag down and up, and a committed claim, are the same state after CERTIFY. (2) Take a core at line 1 with A at 1 at version 0, and the same core with A at 0 at version 1. DEC A 2 takes the first to $A = 0$ at version 1, line 2, a write; it takes the second to line 2 with A still 0 at version 1, because DEC on zero is no write. Same state, price zero. □

Checked as `frag_small_certify_merges`, `frag_small_certifying_step_priced_merge`, `frag_small_dec_free_merge` and `frag_small_not_merge_priced` (FragmentSmall.v). It's the same fact as the last item of the list of what this machine doesn't do, which comes near the end of this chapter, now with names: the chairs

argument needs a price on every merge, this machine's schedule puts the floor on the flag only, and that's why the eight-state piece is a piece. The record layer closes the rest: asked only of merges between states with the same line and counters, the merge premise holds for this machine's own prices and forces the toll on it with no finiteness premise (Theorem 4.15).

Structural entitlement, for every Thiele-complete machine. Theorem 8.3 was about any certification system and handed the run's payments in as a bill. For a Thiele-complete machine (Definition 16.7) the bill is known. Call the CHECK, COMMIT and CERTIFY moves of a run its *record moves*; each costs exactly one, so the ledger rises by the number of record moves, and clause (b) says what a raised record must contain.

Theorem 16.43 (Structural entitlement represented)**.** Let a machine be Thiele-complete through an interface. Take a clean start s_0 and a run t from it; a type of rivals and a type of observations with an exact equality test; an observation obs and a *representative* observation r of each rival; a decoder d from runs to observations; lists Ω, Ω' of rivals and a decision tree T. Suppose (M1) Ω' strictly narrows Ω, as in Theorem 8.3, and (M2) some dropped rival shows an observation different from every survivor's; (M3) the run ends with the record up and the decoded run $d(t)$ is shown by some survivor; (M4) $\mathrm{depth}(T)$ is at most the number of record moves in t; (M5) $|\Omega'| > 0$; and (M6) the covering is by fibres: there is a map F from rivals to lists of rivals such that every rival x of Ω lies in $F(u)$ for some survivor u with $r(x) = r(u)$, the lengths of the lists $F(u)$ over the survivors add up to at least $|\Omega|$, and no $F(u)$ is longer than $L(T)$. Then (1) the membership test of Ω' is strictly stronger than that of Ω; (2) t contains the earned chain of clause (b), with the meaning of the claim true at the CHECK and at the COMMIT; (3) $\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 |\Omega'| \rceil$ is at most the number of record moves of t; (4) the number of record moves of t is exactly the ledger's rise; and (5) the ledger rose by at least 3.

Proof. (1) is Theorem 8.3(1). (2) Clause (b) applied to a run from a clean state that ends with the record up gives the chain, and a passing check means the claim's meaning holds, which keeps holding while the thing it's about is unchanged. (3) The fibres cover: summing their lengths gives $|\Omega| \leq L(T) \cdot |\Omega'|$, and Theorem 8.3's count finishes it, with the record moves, which are the paid steps, in place of the charged steps. (4) is clause (c) added up. (5) is Theorem 16.19(3). □

Checked as `ent_representation` (EntitlementSmall.v), whose Coq statement is printed in Section E. It's a checklist, as I said in Section 8, and the small print is the same kind. (M3)'s second half feeds no conclusion: only the record's being up

is used. (M1) is used only for the inclusion. Its gap feeds nothing. And the first clause of (M6), the representative observation, isn't used by the count either. What the machine adds to Theorem 8.3 is conclusions (2), (4) and (5): the payment is the machine's own three acts, each at exactly one, and a raised record carries the earned chain. The packaged form, a record of the fifteen things a shortcut has to supply, with a theorem that every inhabitant lands in the bound, is `ent_shortcut` and `ent_every_shortcut_lands_here`. The count with the tree thrown away, in the form I use below, is `ent_complete_tree_bound`.

Two further statements, about the run's own questions and about the record clause.

Theorem 16.44 (A floor with no supplied tree)**.** Take a machine and an interface that meets the exact-toll clause (c) of Definition 16.7, so each record act costs exactly one. Call the *questions* of a run the claims of its CHECK moves, and say two rival states *answer alike* when every one of those claims gets the same verdict at both (each verdict taken at the rival itself, as it starts; the states the run passed through don't enter). If, for a list Ω of rival states and a number $m > 0$, the rivals of Ω that answer alike with any one rival, itself included, are never more than m, then $\lceil \log_2 |\Omega| \rceil \mathbin{\dot{-}} \lceil \log_2 m \rceil$ is at most the number of CHECK moves of the run, which is at most its number of record moves, which is the ledger's rise.

Proof. A run's questions give each rival an answer list, and answer lists are lists of Booleans of length equal to the number of CHECK moves, so there are at most 2^c answer classes among the rivals, c the number of CHECK moves. The classes have at most m members each, so $|\Omega| \leq 2^c \cdot m$, and the rounded logarithms follow. The rest is clause (c). □

Checked as `ent_questions_floor` (EntitlementSmall.v). The tree has gone: it is the run's own questions. There are limits, and they're the ones in the statement. The verdicts are taken at the rivals as they start. The states the run had reached when it asked don't come into it, and a free branch, DEC on zero, can still narrow a list for nothing. So this prices a narrowing that the paid checks explain, and no more.

The next statement takes the verdicts where the run took them. A rival is a state the run could have started from; a CHECK the run passed is asked of a rival by running the moves before it from the rival and checking the same claim at the state that reaches. Nothing is supplied but the prior.

Theorem 16.45 (Structural entitlement, with the narrowing made by the run)**.** Let a machine be Thiele-complete through an interface. Take a clean start s_0, a run t from it that ends with the record up, and a list Ω of rival starts that contains s_0. Let the posterior Ω' be the rivals that pass every CHECK t passed, each asked as above, in the order t made them, and let b_i and a_i be the numbers of rivals standing just before and just after the ith. Then (1) Ω' sits inside Ω and contains s_0; (2) every rival of Ω that isn't in Ω' fails, asked that way, a CHECK that t made and passed; (3) $|\Omega| \cdot \prod_i a_i = |\Omega'| \cdot \prod_i b_i$; (4) if every CHECK t passed keeps at least a 2^{-k} share of the rivals standing before it, $b_i \le 2^k a_i$, then $\lceil \log_2 |\Omega| \rceil \dot{-} \lceil \log_2 |\Omega'| \rceil$ is at most k times the number of CHECKs t passed, that number is at most the record moves of t, the record moves are the ledger's rise, and so the index bits are at most k times the ledger's rise; (5) t contains the earned chain and the ledger rose by at least 3; and (6) with no share assumed, the index bits are at most the sum over the CHECKs t passed of $\lceil \log_2 \lceil b_i / a_i \rceil \rceil$.

Proof. (1) The run passes its own checks from s_0, and a rival is kept only if it passes them all. (2) A rival dropped failed one of the tests, and each test is a CHECK of t at the prefix where it passed. (3) Each test turns a standing list of b_i into one of a_i, and the products telescope. (4) Each test keeps at least $b_i/2^k$, so $|\Omega| \le 2^{kn} |\Omega'|$ with n the number of tests, and the rounded logarithms follow. The tests are among the run's CHECK moves, which are record moves, and clause (c) makes the record moves the ledger's rise. (5) is clause (b) and Theorem 16.19(3). (6) is (4) with 2^k replaced at the ith test by the smallest power of two at least $\lceil b_i / a_i \rceil$; every a_i is positive because s_0 is never dropped. □

Checked as `ent_run_entitlement` and `ent_run_round_bits` (EntitlementSmall.v), with the counting underneath as `ent_stages_product`, `ent_stages_share_bits` and `ent_stages_round_bits`. Part (3) is the exact form of "index bits are the sum of what each test took": take base-two logarithms and $\log_2 |\Omega| - \log_2 |\Omega'|$ is the sum of $\log_2(b_i/a_i)$. Parts (4) and (6) are what survives rounding to whole bits. Here the run does the narrowing, every candidate dropped was dropped by a check the run paid for, and what stays a premise is the prior and the share. The share can't go.

Proposition 16.46 (One CHECK can take four bits)**.** Take the sixteen starts $(a, 0)$ with $a = 0, \ldots, 15$, and the run CHECK $(A \ge 15)$ A; COMMIT $(A \ge 15)$ A; CERTIFY from $(15, 0)$. It raises the record. Its one passing CHECK keeps one rival of sixteen, so the posterior is $(15, 0)$ alone and the index bits are 4, while the run has 3 record moves. So index bits at most the record moves, the bound of Theorem 16.45(4) read with $k = 1$, fails here.

Checked as `ent_share_needed` (EntitlementSmall.v). The share here is 2^{-4}, and with $k = 4$ part (4) reads $4 \leq 4$. It's the example of Section 8 that showed the covering can't be dropped, seen from the run's side: one test can take many bits, and the bill covers the bits per test times the tests.

Theorem 16.47 (What the record adds)**.** Let a machine be Thiele-complete through an interface. (1) From a clean start, the record is up at the end of a run exactly when the run contains the earned chain, and a run with the record up raised the ledger by at least 3. (2) Conclusions (1), (3) and (4) of Theorem 16.43 hold without a clean start and without the record being up, for a machine that only meets the exact-toll clause (c): the hypotheses (M1), (M2), (M4), (M5) and (M6), and the decoded run being shown by some survivor, are all it needs.

Checked as `ent2_record_adds`, `ent2_upgrade_iff` and `ent2_observed_repres entation` (EntitlementMore2.v). So the record doesn't make narrowing cost more. It adds the earned chain and a floor of 3, and that's all it adds: a run that narrows without ever raising the record is still bound to pay its index bits, and the floor of 3 no longer applies to it.

Weights, and the partition form. Two more pieces of the same count, both checked. First, suppose the rivals aren't equally many: give each a whole-number weight, a multiplicity, and call the sum of the weights of a list its *mass*. The count survives with masses in place of lengths. If the prior's mass is at most $L(T)$ times the posterior's, the rounded index bits of the two masses differ by at most depth(T); so for any certification system a run that paid at least that many charged steps has a bill at least the drop, and for a Thiele-complete machine the representation theorem goes through with masses and the same five conclusions. On such a machine the ledger rises by the same number from every start, the number of record moves, so the expected rise over a weighted prior is its mass times the record moves, and the drop in rounded bits times the prior's mass is at most that expected rise, with no averaging argument. The weights are whole numbers, cross-multiplied, and no probability enters. Checked as `ent2_weighted_bits`, `ent2_wreduction_covers`, `ent2_weight ed_cs`, `ent2_weighted_representation`, `ent2_rise_independent`, `ent2_expecte d_rise_exact` and `ent2_weighted_expected_bound` (EntitlementMore2.v). Second, the covering by fibres has a plainer form that doesn't need the clause that the fibres add up to the prior. Say the rivals are *partitioned* by the representative observation when every rival looks like some survivor under it and no observation class, counted inside the prior, has more than $L(T)$ members. The fibre lengths then add up to at least the prior, because every rival is in the class of some survivor, so a partition gives the covering, and the count holds under it. The converse is false: there are coverings that aren't partitions. Checked as `ent2_cover_sum`, `ent2_partition_old_iff`, `ent`

`2_partition_to_reduction`, `ent2_partition_bits` and `ent2_reduction_not_partition` (EntitlementMore2.v).

A worked member on the small machine. Eight rivals: the starts (a, b) with $a \in \{0, 1, 2, 3\}$ and $b \in \{0, 1\}$, counter A at a and counter B at b. The run is CHECK (Even) A; CHECK ($A \geq 2$) A; COMMIT ($A \geq 2$) A; CERTIFY, from $(2, 0)$. The program raises the flag from a rival exactly when a is even and at least 2, so exactly from $(2, 0)$ and $(2, 1)$, and the posterior is those two. The decoder reads one "passed" for each CHECK in the run's receipts; the representative observation is "B is 0," which isn't constant; the fibres are the four rivals that share B; the tree is the complete tree of depth 2.

Proposition 16.48 (The eight-start member)**.** For every a and b, the run raises the flag from (a, b) exactly when a is even and at least 2. All hypotheses of Theorem 16.43 hold, with eight rivals narrowed to two: the test of the posterior is strictly stronger; the index bits are $3 - 1 = 2$; the run has 4 record moves and the ledger rises by exactly 4; and the earned chain is in the run. The run's own questions (Theorem 16.44) are its 2 CHECKs, so the floor is met with equality. With the two CHECKs alone, the record stays down, the ledger rises by 2, and the bound $2 \leq 2$ still holds.

Checked as `ent_small_certifies_iff`, `ent_small_posterior_is_certified`, `ent_small_instance`, `ent_small_questions` (EntitlementSmall.v) and `ent2_observed_small_instance` (EntitlementMore2.v), with `ent_earned_complete_with` saying the small machine meets the definition through its own interface. It's a real narrowing. The cups of Section 8 aren't sitting under it in a costume, and the posterior is exactly the set of rivals the run certifies, so here a proof ties the lists to the run. Nobody supplies them. Read with Theorem 16.45, the run's two passing CHECKs, replayed from the eight rivals, leave four and then two, the same two, so the posterior is the one the run makes; each keeps half, so $k = 1$, and the index bits, 2, are at most $1 \cdot 2$ and at most the ledger's rise of 4 (`ent_small_run_narrowing`, EntitlementSmall.v).

A search through n bits. The same shape at a size you can't enumerate, on the machine of Definition 16.8, the one with a counter for every number, with the small machine's property language of Zero, Even and At-least-n, read at registers, in place of Slot. That machine is Thiele-complete too (`ent2_mmachine_complete`, MultiThiele2.v). Hide $n = k + m$ bits, one in each of registers 0 to $n - 1$. The program asks k questions, one per bit of the first k: CHECK "at least 1" of register i for a 1, CHECK "is 0" for a 0. Then it COMMITs the first claim and CERTIFYs. The prior is every n-bit value; the posterior is the 2^m values that begin with the answers; the tree is the complete tree of depth k.

Theorem 16.49 (A search through n bits)**.** For $1 \le k \le 16$ and any m: from a world holding a value v of at least k bits, the program certifies exactly when v begins with its answers, so the posterior is exactly the set of rivals it certifies from; the test of the posterior is strictly stronger than the prior's; the earned chain is in the run; the prior has 2^{k+m} members and the posterior 2^m, so the index bits are exactly k; the run has exactly $k+2$ record moves and the ledger rises by exactly $k+2$; and the run's own questions are k CHECKs, so Theorem 16.44 is met with equality. With the COMMIT and the CERTIFY left off, the questions alone narrow 2^{k+m} values to 2^m, the record stays down, and the ledger rises by exactly k. And no run from a clean start that raises the record contains more than 16 CHECKs before its CERTIFY, so with 17 questions no start certifies.

Proof. Each question is a CHECK of whether one register holds a given bit, which passes exactly when it does; a failing CHECK traps. So the program certifies exactly when every answer is right, which is when v begins with the answers. The counting is the count of bit strings, and the representation theorem applies with every hypothesis a theorem. The cap is the fact table: it holds sixteen facts, never evicts, and a CHECK on a full table traps (Theorem 16.13), so a seventeenth CHECK before the CERTIFY ends the run in a trap. □

Checked as `ent2_search_certifies_iff`, `ent2_search_posterior_is_certified`, `ent2_bit_search`, `ent2_search_lands`, `ent2_search_questions_floor`, `ent2_search_observed`, `ent2_questions_cap` and `ent2_no_search_past_16` (BitSearch2.v, BitSearchMember2.v, BitSearchObserved2.v). The sixteen belongs to the machine. The program has no say in it: it's the same ceiling as the item "Sixteen checks per run, ever" in the list of what the machine doesn't do, near the end of this chapter, and it's why this search has $k \le 16$ however many bits are hidden. Unasked bits can be any number. The lists, the tree and the fibres are supplied here, as in the representation theorem, and tied to the run only for this member. The covering still prices only the size of the drop. Its reason goes unpriced.

The covering can't be dropped. It's a hypothesis, and one might hope it's a nuisance one could remove. It isn't, and the small machine shows it. Fix $n \ge 4$, take the 2^n starts with A below 2^n as the rivals, and run the three-move chain on the claim "$A \ge 2^n - 1$": CHECK, COMMIT, CERTIFY. It certifies exactly from $A = 2^n - 1$, so the posterior is one start, n index bits are gone, and the ledger rose by 3. So for $n \ge 4$ the claim "the index-bit drop is at most the ledger's rise" is false for this run, and no tree of depth at most 3 covers the narrowing, whatever the representative observation. One property with a big constant in it carries n bits in a single yes-or-no question; what the bound prices is a narrowing whose classes are balanced, a tree

with enough leaves. A narrowing of any other shape is outside it. Checked as `ent2_uncovered_posterior`, `ent2_uncovered_claim_false` and `ent2_no_cheap_covering` (CoveringNeeded2.v).

The time a check saves. The question is whether counter A is at least m. There are two ways to find out. The *free* way uses base moves only, INC and DEC, which cost nothing. But a base move reads A one unit at a time: a DEC takes a unit off A or finds it empty. After n moves, a run that started with A at least n has never seen A empty, so it can't have behaved differently for different such A except by a constant shift in A itself. The *paid* way is one CHECK ($A \geq m$): one move, a ledger of one, and the line it ends on says which way it went. Certifying the answer takes the whole chain, CHECK, COMMIT and CERTIFY.

Theorem 16.50 (The time tax)**.** (1) No program of two-counter instructions that runs fewer than m moves has its final line number and its final B determine whether $A \geq m$ at every start; in particular none tells $A = m - 1$ from $A = m$. (2) A program of m DECs and INCs (a ladder: DEC A and on success go on to the next pair, otherwise INC B) decides it in exactly m moves, from the line number and B, and costs 0. (3) The paid chain of three instructions raises the flag exactly when $A \geq m$ and has a ledger of exactly 3, whatever m is. (4) A single CHECK ($A \geq m$) decides it in one move from the line number, with a ledger of exactly 1, whatever m is. (5) So if a ledger unit is worth λ moves, the paid route to deciding costs $1 + \lambda$ and the free route costs at least m, and the paid route is cheaper exactly when $1 + \lambda < m$; the paid route to certifying costs $3 + 3\lambda$.

Proof. (1) Take the starts $(m - 1, b)$ and (m, b). When $n < m$ both have $A \geq n$, so in the first n moves no DEC finds A empty in either run, and the two runs differ only by the shift of one unit in A. They end at the same line with the same B, though $A \geq m$ holds of one and not the other. (2) Run the ladder: each pair either takes a unit off A and moves on, or finds A empty and falls through to the INC. The line after m moves is $2m + 1$ exactly when all m units were there. It's all DEC and INC, so it costs 0. (3) is the price table: CHECK ($A \geq m$) passes exactly when $A \geq m$, a failing CHECK traps and the flag stays down, and the three moves cost three. (4) is the same price table with one instruction: a passing CHECK goes on to the next line and a failing one traps. (5) is arithmetic on (1) to (4). □

Checked as `ent2_free_run_shift`, `ent2_free_cannot_decide_fast`, `ent2_ladder_decides`, `ent2_machine_free_cannot_decide`, `ent2_machine_ladder_decides`, `ent2_ladder_free`, `ent2_chain_decides` and `ent2_time_tax` (TimeTax2.v), the first three for two-counter configurations and the next three for the same programs

compiled and run on the machine, with the certifying comparison $3 + 3\lambda < m$ inside `ent2_time_tax`, (4) as `nec_e_time_tax_one_paid_move`, and the lower bound and the one-move decider together as `nec_e_time_tax_free_needed` (NecESearch.v); the deciding comparison $1 + \lambda < m$ is arithmetic on those. The exchange is sharper than a comparison of two particular programs: the free side is a lower bound over every program of INC and DEC, so no choice of program is involved, and the paid side is constant. The limits: the bound is for programs that make no record move, INC and DEC with falling off the end as stopping. It doesn't cover every Thiele-complete machine, because the definition doesn't forbid a base move that changes a counter by more than one unit, and such a machine is a different machine, one the bound doesn't speak for. And the two routes put their answers in different places, the free one in the line number and the paid one in the flag; this compares two ways to answer one question. It doesn't compare two certifications.

16.18 The diagonal, on this machine's programs

Section 13 left the diagonal standing on any substrate and on a little family built to fit it, reproved it on L, which isn't a substrate because its evaluation can run forever, and said what a machine has to supply to fill in the same shape, two things: programs that can be handed to programs as numbers, and a recursion theorem for the transformations it can carry out. This machine's universal program reads a whole guest program out of one register, with a numbering of programs that goes there and back (Definition 16.9), and halting is undecidable here in the library's sense (Theorem 16.4). What's left are the two classical results that turn a universal machine into a place where the diagonal bites: Rice's theorem and Kleene's recursion theorem. They come with a wrinkle I should put first, because it decides which machine they're about.

The wrinkle is the one from earlier in this chapter, at the end of the subsection on what two counters compute. The small machine takes its inputs straight into two counters, and Schroeppel's memo gives a proof that two plain counters can't even compute 2^n from n. Rice's reduction has to keep a program's input safe while a copy of a halting instance runs. Kleene's construction has to run an evaluator while keeping its input. Two counters have nowhere to keep it. So the two theorems land where they're true and where I could prove them. The host machine of Definition 16.8, the one with a counter for every number that U runs on, is a fine programming system: its programs are numbers, an input sits in register 1, and there's room to spare. On it both theorems hold in full. On the two-counter machine itself the story splits, and I'll tell it at the end of the subsection: Rice's theorem holds on every input, by a trick for keeping the input that I'll show you; Kleene's holds when inputs are

written as powers of two, the way Minsky packed them; and on plain inputs Kleene's is false, and that's a theorem too.

The host machine's programs are numbered the way U numbers a guest's. Write $\langle m, n\rangle$ for $(2n+1)\cdot 2^m$. INC r is $\langle 0, r\rangle$, DEC r j is $\langle 1, \langle r, j\rangle\rangle$, HALT is $\langle 2, 0\rangle$, CHECK r is $\langle 3, r\rangle$, COMMIT r is $\langle 4, r\rangle$, CERTIFY is $\langle 5, 0\rangle$, and a program is the list code: the empty program is 0, and an instruction x followed by a program t is $\langle x, \mathrm{code}(t)\rangle$. Every number decodes to some program, and decoding a program's code gives the program back. Say a host program *outputs* y on input x when, started with x in register 1 and every other register 0, with a clean table, it halts with y in register 0.

Rice first, because it's the shorter road. Henry Rice, whom you met in Section 7, proved in his 1951 doctoral thesis, and published in 1953, that no program decides any property of what programs *do* unless every program has it or none does. Properties of how a program is written aren't covered. The road from halting is a single detour. Take a property of what programs do, a program y that has it, and a program n that doesn't. Given an instance of two-counter halting, a Minsky program H with starting counters a and b, build a host program that first loads a and b into two fresh registers, then runs a copy of H moved onto those registers, then empties both, then runs y. If H halts on (a, b) the built program behaves like y and has the property. If H runs forever the built program never stops on any input, and it behaves like the program that loops, which lacks the property (if the looping program has it instead, swap the roles of y and n). Everything the built program does before y is INC and DEC, which pay nothing and touch no fact, channel, trap latch or flag. So a decider for the property would be a decider for halting, and the library already says what that would cost: the complement of single-tape Turing-machine halting would become something a program could list, the same synthetic sense of "undecidable" as Theorem 16.4. Two words carry the weight, and they're the ones to check in the statement: what "behaves the same" sees, and where the program starts.

Definition 16.12 (Behaving the same, on the host)**.** A host program *ends* in a state when it halts there, from the start with x in register 1 and the rest clean; a trap counts as stopping. Two host programs *behave the same* when for every input x, any state one ends in has a state the other ends in that agrees with it on every register's value, the fact table, the channel, the trap latch, the ledger and the flag, and the same the other way round. The line number and the versions aren't compared. A property of host programs is *extensional* when behaving the same carries it from one program to the other.

Theorem 16.51 (Rice's theorem on the host machine)**.** Let Π be an extensional property of host programs that some program has and some program lacks. Then Π is undecidable in the library's sense.

Proof. The reduction above. The prefix before y leaves every register but the two fresh ones as it found them, and empties those, so when H halts the built program ends in whatever state y ends in from the same input, with the same ledger, table, channel, trap and flag. When H doesn't halt, the built program stops on no input. The reduction is from the complement of halting, so no excluded middle is needed; the proof splits on the decider's own answer for the looping program. □

Checked as `sm_host_rice`, with the reduction's two halves as `sm_rice_prog_halts` and `sm_rice_prog_diverges` (SmHostRice.v). A property that depends only on the partial function a program computes, whether it stops and what it leaves in register 0, is extensional, so it's covered; the files also derive that case again from the recursion theorem below, as `sm_host_rice_fun` (SmKleene.v).

On the small machine itself the same road works from the clean start, and only from there. Say two two-counter programs *behave the same from the clean start* when, whatever state one halts in from counters $(0, 0)$, the other halts in a state with the same counters, trap latch, ledger and flag, and the same the other way round. The fact table isn't compared, because the prefix moves the versions of A and B by fixed amounts and the facts carry versions. There is no input to keep, since the program starts at $(0, 0)$, and H's instance is loaded straight into A and B.

Theorem 16.52 (Rice's theorem for the small machine, from the clean start)**.** Let Π be a property of two-counter programs that's carried from a program to any program that behaves the same from the clean start, held by some program and lacked by some other. Then Π is undecidable in the library's sense. In particular "halts from the clean start" is undecidable.

Checked as `sm_guest_rice` and `sm_guest_halting_clean_undecidable` (SmGuestRice.v). That's the clean start only. Every input is harder, because the standard reduction has to keep the input safe while the halting instance runs, and two counters have nowhere obvious to keep it. The way through is at the end of the subsection.

Kleene's recursion theorem is the deeper one, and it's a construction, which is Section 13's point about Kleene's little gem (Moschovakis's phrase): the content is the building, and the contradiction in Section 13 was just the last line. It says: for every transformation F of programs that some program can carry out on numbers,

there's a program p that computes the same thing as $F(p)$. The shape of the building is the old one. From the program that carries out F, build an *evaluator* that reads a number, treats it as a program's code, specialises that program to its own code, runs F's program on the result, and runs whatever program comes back. The specialising step is what logicians call s-m-n: fix one input of a program and get a new program that needs one input fewer. Then the witness is the evaluator specialised to its own code. Run on any input, it builds its own code, hands it to F's program, gets the code of $F(p)$, and runs that.

Here's each piece on the host machine, with its exact premise.

Theorem 16.53 (s-m-n, and a universal program for host programs)**.** (1) For a number c and a host program V, let spec(c, V) be c copies of INC 2 followed by V with its jumps moved to follow them. Then for all x, y, spec(c, V) outputs y on input x if and only if V, started with x in register 1 and c in register 2, halts with y in register 0. (2) There is a host program W made only of INC, DEC and HALT such that, for all c, x, y, W started with x in register 1 and c in register 2 halts with y in register 0 exactly when the host program numbered c outputs y on input x.

Proof. (1) The prefix writes register 2 c times and nothing else, so it moves that register's version from 0 to c. A fact records a version, so after the prefix every later version of register 2, and the version in every fact the run records about register 2, is c more than in a clean run. A CHECK or a COMMIT reads versions only to ask whether two versions of the same register are equal, and adding c to both keeps the answer, so the two runs take the same steps. The run is V's run from the stated start, moved down by c lines. (2) is the evaluator. A finite algorithm decodes a number into a program, runs it step by step on given registers, and, searching over the number of steps, returns the output of the first halting run. I wrote that algorithm out in Coq, turned it into a term of the lambda calculus L, and ran the library's compiler from L to Minsky programs, which gives a program of two-counter instructions that a host program can be read off from, instruction for instruction. The tools supply the program that computes the evaluator. The fixed-point argument below is not theirs. □

Checked as `sm2_smn_all` (SmSmnAll.v), an if and only if statement that carries no premise on register 2, and `sm_host_universal` (SmKleene.v); the narrower form of (1), with the premise, is still there as `sm_smn`. The numbering is `sm_hcode`, `sm_hdecode` and `sm_hdecode_hcode` (SmCodes.v). Be exact about one thing: U is not that universal program. U runs two-counter guests, and a two-counter guest given its input plainly can't run the evaluator. With inputs written as powers of two one can,

as Theorem 16.59 near the end of this subsection shows. W runs host programs, and it can, because it's a host program on a machine with registers.

Theorem 16.54 (Kleene's recursion theorem on the host machine)**.** (1) For every host program T there is a host program e such that for all x, y: e outputs y on input x if and only if, for some d, T outputs d on the number of e, and the program numbered d outputs y on x. (2) So for every transformation F of host programs that some host program carries out on numbers (on the number of p it outputs the number of $F(p)$, for every p), there is a host program e such that for all x, y, e outputs y on x if and only if $F(e)$ does. (3) The same on numbers: for every function f on numbers that some host program computes, some number e has the program numbered e and the program numbered $f(e)$ output the same on every input.

Proof. (1) Let E be the following program of INC and DEC, built the same way as W from one finite evaluator, with T's number t written in. On inputs x and c it computes the number of $\text{spec}(c, P_c)$, where P_c is the program numbered c, runs T on that number to get d, and then runs the program numbered d on x. Let c_0 be the number of E, and let $e = \text{spec}(c_0, E)$. By (1) of Theorem 16.53, e on input x does what E does on x and c_0. And E on c_0 builds the number of $\text{spec}(c_0, P_{c_0})$, which is the number of e, since P_{c_0} is E. So e hands its own number to T, gets d, and runs d on x. (2) and (3) are (1) with T carrying out F or f. □

Checked as `sm_second_recursion`, `sm_kleene` and `sm_kleene_codes` (SmKleene.v). That fixed point is of the *function*: e and $F(e)$ stop on the same inputs and leave the same number in register 0. Their ledgers, flags and fact tables can differ, because the marks e pays are the evaluator's. I'd like a fixed point that agrees on the record too, and the next two theorems say how much of that can be had. The ledger and the flag can be had. Every register can't.

Theorem 16.55 (The recursion theorem, up to the record)**.** Let F be a transformation of host programs that some host program T carries out on numbers. Then there is a host program e such that, for every input x, any state e ends in has a state $F(e)$ ends in that agrees with it on register 0, the trap latch, the ledger, the flag, the number of facts in the table, and whether the channel is empty, and the same the other way round. So e stops on x exactly when $F(e)$ does, and the ledgers are equal exactly, with no surcharge. T may itself check, commit and certify.

Proof. The construction is Theorem 16.54's, with a different evaluator. The first step is to see that a record is small. From a clean start, a host program that stops has a record I can read off five numbers, in this order: what it left in register 0; how many

facts the table holds; what is left of the ledger once the facts, the trap and the flag are paid for, which I'll call the commits; 1 if the flag is up and 0 if not; and 1 if the trap latch is up and 0 if not. The record obeys a short rulebook. The table holds at most sixteen facts. The latch is a single bit. A commit needs a fact behind it, a certify needs a commit, and the channel is empty exactly when no commit was paid. The ledger is the facts, plus the commits, plus the flag, plus the trap, and that's all of it. Those are the only properties of a record the replay will use, and I proved them by following the plain machine alongside the real one, one step at a time.

The second step is five blocks, one for each number. Each block reads the code of T, runs T on the number of the specialised program, runs the program that comes out on x, and returns its number. All of that happens on plain data: the blocks are programs of INC and DEC, built the way W was, from an evaluator written out in Coq, turned into a term of L and compiled to Minsky programs. So if T has marks of its own, they are simulated and never paid.

The third step is the payment. The evaluator puts 1 in a fresh register, which satisfies the host's one property, and CHECKs it once for every fact. It COMMITs to that claim once for every commit. It CERTIFYs once if the flag is up. It copies the first number into register 0. And if the trap latch is up, it CHECKs a register that holds 0, which fails, and the host traps. Everything before the payments is INC and DEC and costs nothing, and the payments cost what the record costs, so the ledger is exact. Specialise the evaluator to its own number, as before. □

Checked as `sm2_kleene_obs` (SmFixedPoint.v), with the five numbers as `sm2_final_numbers` and the evaluator's specification as `sm2_ev_spec` (SmTally.v) and the replay program as `sm2_VG_forward` and `sm2_VG_halt` (SmFixed.v), where the trap latch is up exactly when the fifth number is 1 and the flag exactly when the fourth is positive; `sm2_kleene_fun` gives back the version that compares only the function. Two limits, and the second one is a theorem. First, e pays after it has computed everything. It is not the way U runs a guest, mirroring every move: e pays the counts, CHECKs, then COMMITs, then the CERTIFY, whatever order $F(e)$ used, so the order of the record's events isn't preserved, and for a program $F(e)$ that never stops, e never stops and pays nothing, though $F(e)$ may pay without bound. The relation says nothing about states in the middle of a run. Second, no fixed point agrees on every register, or on what the facts name.

Theorem 16.56 (No fixed point of full behaviour)**.** There is a transformation F of host programs, carried out by a host program, such that no host program e behaves the same as $F(e)$ in the sense of Definition 16.12.

Proof. Let $F(p)$ be the one-line program INC r, with r one above the number of p. A host program carries this out on numbers; I got it from the same route through L. None of it was written by hand. A program's number is at least every register it names: the number of an instruction is above its register, and the code of a list is above every instruction in it. So e can't name r, the register $F(e)$ writes. On input 0 the run of $F(e)$ ends with 1 in register r, and if e stops at all, it ends with 0 there. If e doesn't stop, then it already differs from $F(e)$, which does. Either way they don't behave the same. □

Checked as `sm2_no_exact` and `sm2_reg_bound` (SmNoExact.v). A variant of F that also CHECKs register r and commits to it gives $F(e)$ a fact and a commitment about r that e can't have, so no fixed point agrees on the fact table alone, or on the channel alone, or on the versions of the registers (`nec_e_no_fixed_point_facts`, `nec_e_no_fixed_point_versions`, NecEHost.v). What survives is what Theorem 16.55 states. Whether one more fixed register could be added to that list I haven't settled either way, and I don't claim it.

And with the recursion theorem proved, the wall of Section 13 goes up on this machine without a premise left over. Call a property of host programs *functional* when it depends only on the partial function a program computes.

Theorem 16.57 (The diagonal on the host machine)**.** Let Π be a functional property of host programs that some program has and some program lacks. (1) Read "f is representable" as "some host program carries f out on numbers." Then no function d from programs to Booleans whose flip is representable is right about Π on every program: this is the argument of Theorem 13.1, reproved for host programs, with its recursion premise a theorem here. (2) No host program decides Π outright: there is no program D that, on the number of every program, halts with a number in register 0 that's 1 exactly when the program has Π.

Proof. (1) The flip, send programs that d calls good to the program lacking Π and the others to the one having it, is carried out by a host program by hypothesis. Theorem 16.54(2) gives a program e that computes what its image computes; Π holds of e exactly when it holds of the image, and the image lacks it exactly when d calls e good, so d is wrong at e. (2) From a deciding program D build a host program that computes D's flip on numbers, by the same route through L and the Minsky compiler, reading the Boolean off D's runs with the standard library's unbounded search; then (1) applies. This one is an outright negation. The synthetic notion doesn't come into it. □

Checked as `sm_no_inside_decider` and `sm_no_host_decider` (SmKleene.v, SmDecider.v).

Rice and Kleene are the ingredients the diagonal of Section 13 asks a substrate for, and the wall they put up doesn't care about the record: the base does the computing and the flag watches. A program's ledger and flag are among what "behaves the same" sees on the host, so a nontrivial property of those is undecidable too, in the library's sense of Theorem 16.51, and the fixed point is up to the record, as far as the theorems above say and no further.

Now the two-counter machine itself, which I promised at the start of the subsection: Minsky's two bins of pebbles and nothing else. Rice, first. Say a two-counter program *behaves the same* as another on a set X of inputs when, on every input n in X, started with n pebbles in bin A and none in bin B, one stops exactly when the other does, and then they agree on both bins, the trap latch, the ledger, the flag, and on which property and which counter each recorded fact and the channel are about. The versions are left out: they say which write a fact is about, and the trick below moves them.

The trick is the whole problem. Rice's reduction has to keep the program's input safe while a copy of a halting instance H runs, and there's no third bin to keep it in. My first guess was bin B, and that dies at once, because H needs both bins. The second guess is Minsky's own packing: keep several numbers in one bin as powers of different primes, H's two counters as the exponents of 2 and 3, and the input as the exponent of 5. That dies more quietly. To put the input there, the program would have to turn n pebbles into 5^n, an exponential, and Schroeppel's memo, from earlier in this chapter, gives a proof that two plain counters can't even build 2^n from n. I had no way to build 5^n.

What works is to stop keeping the input beside the work. Turn n pebbles into $6n + 5$, which is only adding, and then multiply in H's counters as powers of 2 and 3. H runs by halving the pile and splitting it in thirds and building it back up, and the $6n + 5$ rides along in the middle of every one of those piles. It's odd, and counted off in threes it leaves two over, so no halving or thirding ever reaches into it.

Theorem 16.58 (Rice's theorem for the two-counter machine, on any inputs)**.** For any set X of inputs, a property of two-counter programs that's carried from a program to any program that behaves the same on X is undecidable in the library's sense, unless every program has it or none does, and then it's decidable. X can be every input, the powers of two, or $\{0\}$.

Proof. For every input: given a two-counter instance H starting from counters a and b, build a program that turns the input of n pebbles in bin A into $(6n+5)\cdot 2^a\cdot 3^b$, runs H inside the exponents of 2 and 3, divides out every 2 and every 3 when H stops, takes away 5, divides by 6, and then runs the program that has the property. Because $6n+5$ shares no factor with 2 or 3, the coding never mistakes it for an exponent, and the input comes back whole. If H halts, the built program behaves the same as the one with the property on every input; if not, it never stops on any input. The reduction is from the complement of halting. □

Checked as `tc_rice` and `tc_rice_dichotomy` (TcRice.v).

So $6n+5$ is the place to keep the input that two counters seemed not to have. It isn't beside the work. It's inside it, in a factor the work never touches. I like that one more than I can justify: the input hides in the leftover that every halving and thirding walks straight past.

Kleene needs more than keeping an input. It needs a program that runs a program. So write a number x as 2^x pebbles and read a result as the exponent of 2, the way Minsky did. With inputs written that way, there's one two-counter program, call it U_2, that, started on $2^x\cdot 3^e$, stops with 2^y exactly when the program numbered e gives y on x; specialising a program to a number is exact; and the recursion theorem holds. (U_2 is a different program from the universal host program U, which runs on the machine with a register for every number.) Checked as `tc_universal` and `tc_smn` (TcPacked.v).

Theorem 16.59 (Kleene for the two-counter machine, inputs written as powers of two)**.** (1) For every transformation F of programs that some program carries out on program numbers, there's a program e that computes the same function as $F(e)$, with inputs and outputs written as powers of two. (2) The fixed point is of the function only: there's such an F for which no e behaves the same as $F(e)$ once the properties and counters of the recorded facts count.

The counterexample in (2) is short. Let $F(p)$ check that bin A holds at least the number of p plus one. A program can only record facts about properties it names, and every property it names is below its own number, so no e records what $F(e)$ records.

Checked as `tc_second_recursion`, `tc_kleene` and `tc_no_fine_kleene` (TcPacked.v, TcNoFine.v).

Even with Schroeppel in hand, my first guess about Kleene was that the powers of two were a convenience, and that plain inputs would only take more work. They don't take more work. With plain inputs, Kleene's theorem is false. A transformation

is *carried out* if some program, started on 2^n where n is the number of a program p, stops with 2^m where m is the number of $F(p)$.

Theorem 16.60 (No plain recursion on two counters)**.** (1) No program of n lines stops with $c \cdot x$ pebbles in bin A, started on x pebbles in bin A and none in bin B, for every x, where $c = N! + 1$ and $N = 16(n+1)(2n+1)^{16}$. (2) So there's a carried-out transformation F such that no program e computes the same plain function as $F(e)$.

Proof. (2) from (1): let $F(p)$ multiply bin A by c for the length of p. A program carries F out by the same route through L as before, and a fixed point would multiply every input by a number fixed by its own length. For (1), count the control states of a program of n lines: its line, the trap latch, whether the channel holds a commitment, and for each of at most sixteen facts its property, its counter and whether it still concerns that counter's current version. There are at most $4(n+1)(2n+1)^{16}$, and N is four times that, so c shares no factor with any number up to N. Now suppose a program does multiply every input by c. On a large input, a bin far fuller than anything a CHECK names is seen only two ways: whether it's empty, and whether its pebbles pair off evenly. So the run breaks into stages, each a bounded cycle that moves the small bin up by some g and the full one down by some m, both at most N. Follow one input and the same input plus the product of the m's through the same stages, and compare the outputs: the growth is c times the product of the m's on one side and the product of the g's on the other. A number with no factor up to N can't divide a product of numbers up to N unless it is 1, and c is at least 2. □

Checked as `tc2_no_mult` (Tc2Mult.v), from `am_no_multiplier` (Tc2Chain.v), which says the same for any machine with finitely many control states whose counters are read only through zero and parity above a threshold, and `tc2_plain_recursion_false` (Tc2Plain.v), which gives a carried-out F with no e equivalent to $F(e)$ on plain inputs, all closed.

The case split in the stages is decided by running for a bounded number of steps, so no excluded middle goes in. For a long time I couldn't read Schroeppel's own memo. I went looking again and found the scan. As I read it, his route is close in spirit: he trades a two-counter machine that starts with one counter empty for a machine with one register that can only add, multiply and divide by fixed numbers and take one away, each of those built out of one of the program's loops, and that's the stages here under another name. I haven't followed his proof line by line; this one is checked. The c it uses is astronomically bigger than anything a real multiplier needs, and finding the best bound is a question I leave alone.

So on two bins: Rice holds on every input. Kleene holds with inputs written as powers of two, for the function. It fails for the facts. And with plain inputs Kleene fails outright. The last one is the answer I didn't see coming. The powers of two weren't a convenience. On two bins they're the difference between a recursion theorem and none.

That failure has a setting computability theorists know. The mathematician Hartley Rogers, in his 1967 textbook, calls a numbering of the computable functions *acceptable* when every other numbering can be translated into it by a computable map, and in every acceptable numbering Kleene's theorem holds. Read on plain inputs, the two-bin programs number something smaller than all the computable functions, since none of them computes 2^n from n (Schroeppel's theorem, Section 16), so they fall outside Rogers's class, and his route to the theorem isn't open there. The computer scientists James Royer and John Case (1994) studied that territory: programming systems for classes of functions smaller than all the computable ones, which can't be acceptable, and what can still be programmed and proved in them. And the logician and puzzle writer Raymond Smullyan's two books on recursion theory and self-reference (1993, 1994) take the fixed-point theorems apart axiomatically, so you can see which property of a numbering each one leans on. The plain-input failure is a small, checked instance on the map they drew.

16.19 What the universal machine covers, and what it doesn't

Here's what that covers, said plainly, a sentence at a time. The guests of U are this chapter's programs, every one of them, from every clean start start(x, y). That's the stated class. The whole-run theorems don't cover a guest handed facts in its table, a commitment in its channel or a raised flag at the start; the one-step simulation, by itself, works from any related pair. It's the same kind of class Turing's universal machine has: a fixed instruction set, and a finite program held as data, here packed into one number the way Turing packed his into a description number. Each guest keeps its computation, its halting, its answer, its record and its prices, exactly: the host halts when it halts, its flag goes up when the guest's would, on a chain of its own, and its ledger is the guest's.

The guests of U_P are every Thiele machine whose driver, step, prices and reading are computed by programs, with the reading decided on every state. Each keeps its computation and its halting; its reading becomes the host's flag as a latch, raised on an earned chain whose claim is the guest's own reading; and its prices become the host's ledger, exactly, except for a surcharge of at most two at the move where the reading first says yes, when the reading starts at no. The surcharge is zero when that move costs three or more, and it can't always be avoided: when the guest certifies

with a ledger below three, no program on that host, run from a clean start, matches its ledger exactly (Theorems 16.33 and 16.34). Picture a clerk who'll stamp anybody's receipt. He copies it onto his own forms, bills at your rates to within two, and the stamp at the bottom is one he earned by checking your own reading. He didn't copy it.

What's outside the class is outside the theorems. A machine whose reading no program decides (an oracle checker, say, which the definition of Thiele-complete doesn't rule out) doesn't come in. Nor does a machine whose next move is nobody's computation. Nor does a guest that starts with a table of facts nobody checked: the guests start where their presentation says, and the host from a clean start.

And here's why insisting on no built-in step is the point. A plain interpreter on an ordinary computer runs guests too, but then the guest's record is data the interpreter's program updates by convention, which is exactly the position of the ordinary programmable computer from the front matter and Section 16.16. That's a fact about how such interpreters are built. It says nothing about what they can do: U and U_P are fixed interpreters, and their record isn't kept by convention. Nothing in them raises the host's flag except the host's own CERTIFY, and the host's step won't let that pass without a COMMIT, or the COMMIT without a CHECK on a slot nobody has touched. The guest's record isn't data the host updates. It's a record the host earns again, under the same rule, on its own registers.

16.20 What it doesn't do

Here's where it falls short, in plain words, because a small machine is only honest if its edges are as visible as its middle.

- **The check price is flat.** CHECK costs 1 whether it's asking "is this 0" or "is this at least a billion." The property language is fixed and each property is one evaluation, so "pays for its work" here means that flat price, the same however hard the question was. A machine that charged by the difficulty of the check would be a different machine, and I haven't built it.
- **DEC on zero isn't a write.** It doesn't change the counter, so it doesn't change the version. That's the right call for versions (nothing changed), but it means DEC on zero is a free way to move the line number without leaving any mark. The idle run in the window section uses exactly that.
- **After a trap, a trace still pays.** On programs, a trap stops the run. On arbitrary traces, instructions after a trap are each charged their cost and none of them changes the core. I chose to keep charging so that the ledger stays the plain sum

of what was issued (Theorem 16.5 has no exceptions); the price of that choice is that a trapped machine can still be billed for nothing.

- **"Checked" means what the property language can say.** On this machine that's *is zero*, *is even* and *is at least n*, and a live fact means one of those holds of one counter's current value. It doesn't mean the program is correct, or that anything follows from the property, or that the property was worth checking. So can a proved machine of this kind check "sorted," the claim the opening chapter's table runs on? Yes, through an encoding. A copy of this machine, instruction for instruction, with its property language left open, has provenance, checker soundness, no forging and the floor of three proved once for every property language that comes with an exact equality test and a checker proved equal to its meaning. Give it "the list this counter encodes is sorted" as a property and it certifies a sorted list for three and refuses an unsorted one forever (`sorted_run_certifies`, `sorted_run_certifies_iff` and `sorted_run_refused_forever`, EarnedGeneric.v). The copy is written out again in its own file, and no theorem derives it from this one; putting the two side by side is how you check they're the same machine. What makes that a check of the list is the proof that the checker means sortedness. Without that proof, "sorted" would be a number some unconstrained code wrote into a counter, and the record would be honest about the counter and silent about the list.
- **The certificate is historical.** Between the COMMIT and the CERTIFY, the counter can change, and the flag still goes up. The flag says a claim was checked and committed about an edition. It says nothing about whether the claim is still true.
- **One latch, and a channel that moves on.** The flag records that an earned chain happened, once. After that it stays up, and later CERTIFYs change nothing. The channel is the part that names a fact: with the flag up from a clean start, it names a fact that some earlier passing COMMIT committed (`certified_channel_earned` and `channel_names_last_commit`, EarnedCore.v). But a later commitment replaces it, so at the end it names the last earned commitment, which needn't be the one the flag rose on. Two runs from start$(0, 0)$ show it, worked from the instruction table. First run: CHECK *is zero A*; CHECK *is zero B*; COMMIT *is zero A*; CERTIFY; COMMIT *is zero B*. Second run: the same five instructions with the last two swapped, so COMMIT *is zero B* comes before CERTIFY. Both end in the same state: both counters 0 at version 0, both facts in the table, the channel holding (*zero*, *B*, 0), the trap down, a ledger of five and the flag up. In the first run the flag rose at step four, on the commitment about *A*. In the second it rose at step five, on the commitment about *B*. So the flag is an ever-certified latch: it says an earned chain happened at some step, and the end state doesn't say which

claim or which step. A receipt naming the claim the flag first rose on would need that claim kept where nothing can overwrite it; a receipt with a time on it would also need the step number kept; and several certifications that each last would need a record richer than one flag. None of those is built or proved here. They're extensions. Several claims need a latch each.

- **Sixteen checks per run, ever.** Facts never leave and the table holds sixteen, so a run gets at most sixteen passing CHECKs in its whole life, and the seventeenth traps. A certificate needs only one, so this costs nothing for the theorems, but it's a hard wall for any program that wants to keep checking forever, and it's this machine's small version of the opening chapter's "you can't keep all of it." A CHECK that repeats a claim already in the table, same property, same counter, same edition, passes again and takes another entry. So the wall is sixteen passing CHECKs, and sixteen different claims is a different count. Removing duplicates, evicting old facts or collecting dead ones would each be a different design, and each would need its own invariants proved.
- **The clean start matters.** Provenance, soundness, no forging and the price of three are all stated from a clean start. Hand the machine a table pre-loaded with facts nobody checked, and of course none of that follows. The theorems are about what the machine produces. What you smuggle in at the door is outside them. Spell the door out once: the theorems cover any list of instructions anybody can issue, from a clean start, on a step that runs as written, with nothing else writing to the machine. They say nothing about whoever sets or restores the start, about a reset, about a device that misbehaves, or about whoever carries the flag's value to you.
- **The universal programs have stated guests.** U runs every one of this machine's programs from every clean start start(x, y), with halting, answers, flag and ledger matched (the flag over the whole run, the ledger at matching points). U_P runs every computably presented Thiele machine, with its reading decided by a program, its record carried as a latch, and its ledger matched to within a surcharge of at most two when its reading starts at no, and of three when it starts at yes. When the guest certifies with a ledger below three, no program on that host, run from a clean start, matches its ledger exactly. A machine outside that class, with a reading no program decides, isn't covered by either. The simpler host with the built-in guest step pays one per guest step whatever the step was, so its ledger is at least the guest's and usually more; U's is exact, and U_P's is exact up to the surcharge.
- **Slow, and fed directly.** Universality is Minsky's, with the inputs placed straight in the counters. Getting from an arbitrary Turing machine to two counters goes

through his exponential packing, which my files don't redo; the compiler behind U_P uses the library's version of it, and nothing about it is fast.

- **The step doesn't police trivial checks.** CHECK *at least 0* passes on every value, so CHECK, COMMIT, CERTIFY on it raises the flag for three from any start (the note after Theorem 16.17). A check that can't fail establishes nothing, and the machine raises the flag for it anyway. The step checks order, aboutness and an honest checker, and it leaves "was that worth asking" to whoever reads the record.
- **The chairs don't reach it, and the toll is forced another way.** Section 4's chairs argument forces a toll on a finite machine that pays for every merge. This machine is neither. Its counters have no ceiling, so it's infinite. And its free instructions merge states: take a core at line ℓ with c at 1 and version w, and a core at line $j-1$ with c at 0 and version $w+1$, everything else alike, and DEC c j sends both to line j with c at 0 and version $w+1$, for nothing. So full merge pricing fails here, and clause (c) needs those free moves. What holds is merge pricing asked of the record layer alone: a move that sends two different states with the same line and counters to one state costs at least one. This machine's own prices meet it, CERTIFY's merge is a merge of that kind, and so every price list that meets it charges the flag-raising step at least one, with no finiteness premise (Theorem 4.15). That premise is Landauer's, named and assumed. And the exact price of one, where the premise asks for at least one, is still a design choice.

16.21 What this chapter adds up to

So Part IV has one machine, and it isn't the model. It's a Thiele machine in the front matter's sense, an instance of the certification system with the toll proved: `earned_cs` (Theorem 16.6). It's Thiele-complete, in the sense of Definition 16.7, the sense no clock meets (Observation 16.21 and Theorem 16.25): it computes whatever a Turing machine can, under Minsky's 2^n encoding, done outside the machine, its computing is free and can't touch the record, and, from a clean start, its record is earned. And the universal Thiele machine runs on a host made of its own kinds of instruction: one fixed program, U, with no guest step built in, that runs every one of its programs with halting, flag and ledger matched exactly and earns its own flag (Section 16.13); and the same program with one paid move added, U_P, that runs every computably presented Thiele machine, carrying its reading as a latch, its prices to within two when its reading starts at no, and a flag earned on the guest's own reading (Section 16.14).

If you ever find yourself wondering whether a property of this machine is the model's or just the machine's, hold it up against Section 3's certification system, the latch of Section 10 and the four clauses of Thiele-complete. What follows from those holds of every machine that meets them; the rest (a table of sixteen facts, three properties, a flat price of one) is this machine's choice. Six instructions, and a seventh for a host that has to bill somebody else's prices. If the model needs more than that, this is where it'll show.

What's left of Part IV: whether any of this depends on counters (Section 17), how machines are built from parts (Section 18), whether any of it actually runs (Section 19), then a check worth earning, and then what any of this costs against physics.

17

Every model

Here's the worry I couldn't put down after the last chapter. Everything in it was built on Minsky's two counters. The definition of Thiele-complete names a two-counter window in clause (a). The universal program runs two-counter guests. So is all of this a fact about counters? Pick a different machine to start from, a Turing machine with its tape, a register machine, Alonzo Church's lambda calculus, and does the record still go on the same way, or did I just find a trick that happens to work for one toy?

I chased it with a picture: a box with a panel bolted to its front. The box computes, with levers for moves and a window showing what clause (a) asks to see, two numbers and a line number. The panel is the last chapter's earned layer: version tallies that click when the box writes, a rack for facts, the flag, and the three buttons CHECK, COMMIT and CERTIFY at one mark each. The worry is whether that panel only bolts onto the one box I built.

If it did, it would be a curiosity. The whole point of the opening chapter was that the second computer is missing something *any* computer could have. So this chapter tries to break that, from both directions: the panel onto other boxes, and other boxes run inside mine.

17.1 From any base up

Start with the direction I care about most: the panel onto a box I didn't build. The box can be any machine at all that has a universal base in the sense of clause (a), and in the picture that's two things. It shows a two-counter window. And for each two-counter instruction it has a lever that acts on the window as that instruction does. What else is inside, what its states look like, I don't get to know.

My first guess was to keep whatever the box came with. Plenty of machines have a meter of their own, or a light on the front that means something's done, so leave those where they are, bolt the panel on beside them, and see what happens. It can't work, and the definition says why. If a lever can cost something, computing isn't free. If a lever can switch on the box's own light, a record went up with no CHECK behind it. Thiele-complete rules out both. So the meter and the light come off, along

with everything else the box already had: its costs, its record, all of it, except the states, the levers and what each lever does to the state.

Then bolt on a panel cut down from the small machine's, and nothing else: one version number for the whole box that every lever raises, which is the tally, a table with room for at least one fact, and three new moves, CHECK, COMMIT and CERTIFY, at one mark each, with every lever free. Call that the *lift* of the machine.

Theorem 17.1 (Every universal base lifts)**.** (1) The lift of any box with a universal base, over a claim language with an exact equality test and a checker proved to mean what it says, is Thiele-complete, as long as the claims its CHECK can ask include one that's true at one loaded state and false at another, and the fact table has room for at least one fact. (2) The claims "this register is at least n" and "the program is at line n," read off the window, always include such a claim, so with those claims every universal base lifts. (3) Conversely, every Thiele-complete machine has an interface meeting the four clauses under which its base moves, taken alone, form a machine with a universal base. (4) The panel leaves the box alone: from a state whose trap is down, a run of lever pulls in the lift moves the box exactly as the box's own run does, and the trap stays down.

The proof of (1) is the small machine's proof with the bins swapped out for whatever the box has. That's the part I'd braced for, and it turned out to be nothing. The chain CHECK, COMMIT, CERTIFY never looked at anything but the tally and the claim. The levers never touched the panel. Neither side of the proof ever needed to know it was looking at pebbles.

Checked as `lift_thiele_complete`, `lift_window_nonvacuous`, `lift_window_thiele_complete`, `lift_reduct_universal` and `lift_projection_run` (LiftCore.v, LiftConverse.v), all closed.

So the earned layer isn't something two counters do. It sits on top of any machine that can compute, and it never needed to know which one. I'd half expected to be rewriting the proof once per model.

17.2 What never lifts

A theorem that says the panel fits every box is one I don't trust until I've watched it fail to fit something. So I went looking for boxes it won't turn into a Thiele machine, and the first one I tried was the one I was surest would work.

A Turing machine, with one fixed table. Each lever does one row of the table. That looked like the obvious base to me, and it isn't one at all. Clause (a) asks for a lever for every two-counter instruction, and there are infinitely many two-counter

instructions, since a jump can go to any line. A fixed table has finitely many rows, so it has finitely many levers, and it runs out of levers long before it runs out of instructions. The first two items below are that failure in two forms, and the others fail for other reasons. Each is proved.

Theorem 17.2 (What doesn't lift)**.** (1) A box with only finitely many levers has no universal base, and no box with a universal base is finitely branching, meaning that from every state all its levers together lead to only finitely many different states. (2) The lift of a finitely branching box is never Thiele-complete, whatever claims and fact table it gets. (3) Neither is the lift of a box whose step ignores the state it's in, where each lever sends every state to the same place. (4) On a record axis where every place is at or below every other, there is no Thiele-complete machine at all. (5) The lift's own non-vacuity clause holds exactly when the fact table has room and some claim is true at one loaded state and false at another.

Checked as `lift_finite_moves_not_a_base`, `lift_ub_not_finitely_branching`, `lift_finite_branching_not_complete`, `lift_stateless_not_complete`, `lift_ax_indiscrete_no_machine` and `lift_canonical_nonvac_iff` (LiftConverse.v, LiftAxis.v), all closed.

So my first test case failed, and at first I read item (1) as saying a Turing machine can't be a base at all. It says something narrower. A Turing machine *with one fixed table* can't be one. A classic model comes in the other way round, with one lever for each whole program, run to the end. Pull it and the box runs that program from start to stop in a single move. That's how the register machine below comes in, and it's the honest reading of "a Turing machine is a base" too. The levers are the programs. They aren't the rows.

That still leaves a worry about clause (a) itself. It asks for one lever per two-counter instruction, which is a fact about how the two-counter machine is presented, and a box that does each instruction in several pulls looks shut out for a reason that has nothing to do with computing. It isn't shut out, and the fix is to let the pulls be grouped. Call a box a *base by simulation* when it has a window reading a two-counter configuration off each state, through any encoding at all, a class of live states, a finite list of its own levers for each two-counter instruction, and a live loaded state showing $(1, a, b)$ for each start, such that from every live state the list acts on the window exactly as the instruction acts on a configuration and the state stays live. The *grouped box* has the same states, and its levers are finite lists of the box's levers, a list pulled in order.

Theorem 17.3 (The one-lever presentation loses nothing)**.** (1) Every box with a universal base is a base by simulation. (2) Every base by simulation is a universal base of its grouped box, and a run of the grouped box is the box's run along the joined lists. (3) So for every base by simulation, the lift of its grouped box, with the window claims and a fact table with room for at least one fact, is Thiele-complete. (4) The box with five levers, add one to A, add one to B, add one to a jump target, and for each of A and B "if it's above 0, take one off and jump to the target, otherwise go to the next line, and put the target back to 0 either way," is a base by simulation, has no universal base, and its grouped box lifts to a Thiele-complete machine.

Proof. (1) Each list has one lever. (2) The grouped box's step is the box's run along the list, so the simulation clause of a base by simulation is clause (a) for the grouped box. (3) is (2) with Theorem 17.1. (4) INC r is one pull; DEC r j is j pulls of the target lever, then the decrement lever for r, from a live state where the target is 0. The box has five levers, so item (1) of Theorem 17.2 says it has no universal base. □

Checked as `ub_to_sim_inhabited`, `sim_to_macro_inhabited`, `sim_macro_runs`, `sim_base_lifts`, `fl_has_sim_base`, `fl_no_universal_base` and `fl_macro_lifts` (LiftMacro.v), all closed.

So clause (a) can be read by simulation, and nothing is lost: a box that simulates two counters in finite runs of its own moves, through any window, is a universal base the moment its moves are grouped, and it lifts. Item (1) of Theorem 17.2 is about the presentation, and the five-lever box is the proof that a finite lever set is not what fails. What this doesn't give is a Turing machine's rule table running a tape: the five-lever box is a register box, and the next subsection is still how Turing machines come in.

Item (3) is a box with no memory. Pull a lever and it lands wherever that lever always lands, whatever happened before. So a window on it can't tell INC A followed by INC A from INC B followed by INC A, and a box that can't tell those apart can't keep two counters in its window.

Item (4) is the trouble with bare preorders from Section 11, where the cups stopped telling you the place, showing up one level higher. If every place on the axis is at or below every other, the flag has no up to go to. No step ever turns a cup over, nothing ever has to pay, and the earned chain has nothing to earn.

17.3 The same thing, in seven models

Now the other direction, the one the worry was really about. Put boxes with different insides in a row: one with a tape, one rewriting lambda terms, one with bins. Is a

panel riding on one of them the same as a panel riding on another? The honest way to ask is whether the boxes agree on what they compute.

My first idea was to wire the boxes to each other myself, two at a time, with a relation that says one box's run keeps pace with another's. I even have such a relation. Section 10 defines weak base equivalence, and then has to admit that it never connects any two of the bases it was built for. Doing that box by box would be years of work, and somebody has already done the hard part, in a form that doesn't need my relation at all. The Coq Library of Undecidability Proofs, built over several years by Yannick Forster, Dominique Larchey-Wendling, Gert Smolka and many others, proves in Coq that seven models of computation compute the same relations on numbers: the weak call-by-value lambda calculus L, Turing machines, two kinds of Minsky machine, binary stack machines, John Conway's FRACTRAN, and the μ-recursive functions. Conway's model I know only as a step on the library's road to two counters (Section 16) and by its place on that list, so I'll say nothing about it beyond that. I didn't re-prove the agreement. I lean on their proof. Their proof is checked by the same proof assistant as mine, so leaning on it costs me nothing in certainty. What I proved is the joint where my boxes meet theirs.

Theorem 17.4 (The base doesn't matter)**.** (1) A relation on numbers that's computable in any one of those seven models is computable in all seven. (2) The box that keeps a whole two-counter configuration in one number, with one lever for each instruction, has a universal base, its step is computable in all seven models, and its lift, with a table of sixteen facts, is Thiele-complete. (3) A register machine of the kind Cook and Reckhow described, where each two-counter instruction compiles to at most six of its own instructions, has a universal base too, and its lift is Thiele-complete. (4) Any box on numbers with a universal base lifts to a Thiele-complete machine, and its step, computable in one model, is computable in all seven.

Checked as `lift_num_models_agree`, `lift_ex_in_all_models`, `lift_ex_thiele_complete`, `lift_ram_thiele_complete`, `lift_numeric_base` and `lift_classic_bases` (LiftModelsAll.v, LiftModels.v), all closed, with the library's own equivalences carrying the part about seven models.

So the answer to the worry is that the record doesn't care. A box with a universal base, described in any of the models people have used to say what computing is, takes the same panel, becomes the same kind of machine, and pays the same three marks for the same three acts.

And the trip goes back down. So far the panel went onto other boxes. Now put another box, panel and all, inside mine. Section 16 proved that the fixed program U_P on the small machine's host runs every computably presented Thiele machine. In

terms of the seven models, that reads: if the four relations that describe a machine (its driver, which picks the next move or says stop, its step, its cost and its reading, on codes) are computable in any one of the seven, then the machine is computably presented, and U_P runs it, halting when it halts, with its flag, and with a surcharge of at most two when its reading starts at no.

Checked as `lift_computable_runs_on_U` (LiftHeadline.v).

So a Thiele machine described in lambda calculus runs, under one fixed program, on the small machine's host. That's model independence in both directions: up, by lifting any universal base, and down, by running anything presentable on one fixed program. When I saw both arrows close I stopped worrying that I'd built a trick for one toy. A trick for one toy wouldn't survive being carried both ways.

17.4 Why two counters

If the box doesn't matter, why did I build on two bins? Because I wanted the smallest box that holds up, and the obvious smaller one has a single bin. Nothing earlier in this book says one bin is too few. So before I'd let myself say two is the fewest, I tried one.

Watch what a program on one bin can see. While the bin isn't empty, it can't tell how full it is: a stretch of lines that never finds the bin empty does exactly the same thing from a fuller bin, with the extra pebbles riding along. So the only news a one-bin program ever gets is the bin running dry. Between two of those moments it's just walking its own lines, and among more moments than there are lines, two are on the same line, which is the pigeonhole again. Put those together and a one-bin program either stops within a count you can write down in advance, or never stops at all.

Theorem 17.5 (One counter is too few)**.** (1) A program P on one bin, started with n pebbles in it, halts exactly when it has stopped within $k(n + k + 1)$ steps, where k is the number of lines in P. So whether it halts is decidable. (2) Halting for the book's two-counter machine is undecidable. (3) So no translation from two-counter programs to one-counter programs keeps halting, unless the complement of Turing-machine halting could be listed by a program.

Proof. (1) Two facts and a count. The two facts are the walk above: the extra pebbles ride along, and among more moments than there are lines, two are at the same line. Counting with those two facts gives the bound. (2) The library's undecidability of Minsky-machine halting, carried over to the book's instruction set line by line. (3) A translation that kept halting would make two-counter halting decidable by (1). □

Checked as `lift_oc_halts_bound`, `lift_oc_halts_dec`, `lift_cm_halting_undecidable`, `lift_no_oc_translation` and `lift_no_oc_translation_undecidability` (LiftOneCounter.v, LiftModels.v), all closed.

So two counters are the fewest in exactly this sense: one counter can't compute everything, because its halting can be decided, and two can. I like where the answer came from. The pigeonhole that left no spare chair in the closed room is the same one that says why the box has two bins.

17.5 What this chapter doesn't say

A panel that bolts onto anything is an easy picture to hear too much in, so here are four things the chapter might be heard to say, and doesn't. First, it doesn't prove "a lift is Thiele-complete exactly when its base is universal" as one equivalence. It proves the upward direction, the converse for every Thiele-complete machine's own base moves, and the two kinds of base that never lift. Second, it doesn't contain a Turing machine's rule table running a tape. The Turing machines here are the library's, and they enter through what they compute. Third, the lift's tally is one version number for the whole machine. It doesn't keep one per register, the way the small machine keeps one per counter. That's coarser, and it's enough for every theorem above. Fourth, it doesn't show that a lifted machine is computably presented in general. The trip down needs the four relations, and for a particular machine somebody has to supply them.

All of it is one closed statement too, if you want the whole chapter in one place. Checked as `lift_model_independence` (LiftHeadline.v).

I went into this chapter looking for the place where the bins were doing secret work for the panel. I didn't find one. Either there isn't one, or I didn't look hard enough, and the four items above are where I'd send anybody who wants to look harder.

18

Built from parts

Back in Section 3 I asked a question and only got halfway: bolt two certification systems together, and do you get one? Everything real is built out of parts. A computer is a processor and memory and a disk, a bank is branches, a proof is lemmas. If the record and its toll only held for one machine at a time and fell apart the moment you put two together, they'd be a property of toys.

So put two Thiele machines on a bench, each box with its own panel, and wire them up the three ways anybody wires machines. Side by side, with no wire between them at all, so that every move is a lever pulled on one box or the other. End to end, with the first box's bins tipped into the second's when the first is done. And one inside the other, with the inner box running as a guest on the outer one. Some of it holds. Some of it provably doesn't, and the places where it doesn't were where I learned the most.

18.1 Side by side

Two boxes on the bench, no wire between them. A move of the pair is a lever pulled on one box or the other. The question is what the pair's record should be, and my first answer was the wrong one: a single count of how many certifications the bench has made. One number, easy to read. I'll come back to why it fails, because it's in the list of things that break. What works is to keep both panels showing. The pair's record is the pair of records, a place for the left machine and a place for the right one, compared one side at a time.

Since that record has more than two places, "Thiele-complete" in this chapter means the definition of Section 16 with the record allowed to be a place along a direction, the way Section 11 set it up. The old definition had a single bit there, and at two places this one is exactly the old one. Checked as `ax_tc_two_point_iff` (AxTwoPoint.v).

Theorem 18.1 (Side by side)**.** (1) The cost of any run of the pair is the left machine's cost of its moves plus the right machine's cost of its moves, and the order in which levers on the two boxes were interleaved doesn't change where the pair ends up or what it paid. (2) The pair pays the toll exactly when both machines do. (3) Read

through "either flag is up," two certification systems side by side are a certification system with floor one; read through "both flags are up," a run that starts with both flags down and ends with both up costs at least two, and two is reached when each part can certify for one. (4) Two Thiele-complete machines side by side are a Thiele-complete machine. (5) In that pair, call a side's *floor* the place on its record axis where its clean states sit. A run from a clean start that ends with each side at or above a point that isn't at or below that side's floor makes at least six record moves and raises the ledger by at least six, and six is reached. (6) The pair of records is the product in the precise sense: for any machine mapping into both, preserving records, there's exactly one map into the pair that does both jobs.

Most of that is what you'd guess from the bench. Each box keeps its own ledger and nobody else pulls its levers, so the bills add, and it can't matter which box you happened to work on first. The six in (5) is two certifications of three marks each, one per box, with no discount for doing them on the same bench.

Checked as `cmpz_cost_prod`, `cmpz_interleaving_invariant`, `cmpz_prod_a2_iff`, `cmpz_cs_or_floor`, `cmpz_cs_and_floor`, `cmpz_cs_and_tight`, `cmpz_prod_tc`, `cmpz_prod_joint_certificate`, `cmpz_prod_joint_certificate_tight` and `cmpz_prod_universal` (CzProd.v, CzCounter.v, CzCS.v, CzProdTC.v, CzCat.v), all closed. The machines that preserve cost and record form a category with these pairs as products, and the pairing is a symmetric monoidal structure, with the pentagon, triangle and hexagon laws checked (CzCat.v).

That last sentence is the most formal way I know of saying "side by side composes, and nothing about the order of the parts matters." It's a lot of category theory to confirm what the bench told me in a glance, and I'm glad to have it anyway. A glance is where I'd have missed something.

18.2 One after the other

Now wire them end to end. Run the first box, and when it's done, tip its two bins into the second box's start, for free, and run the second. That's how programs get chained everywhere, and my first thought was that the tipping couldn't matter. It's plumbing. Pebbles go from one bin into another, nobody checks anything, and plumbing has no business on a record. That turned out to be half right, and the wrong half is item (4).

Theorem 18.2 (One after the other)**.** (1) Running one Thiele-complete machine and then handing its two counters to another through free loads gives a Thiele-complete machine. (2) For any two programs of the two-counter machine, it computes the composite: some schedule leaves the first machine's window where the first program takes the input, and the second's where the second program takes that result. (3)

The hand-off keeps the toll when every state it can load into sits at one level of the second machine's record. (4) If some load lands the second machine at a record that isn't at or below another load's, the hand-off loses the toll: it's a certification nobody paid for.

Checked as `cmpz_seq_tc`, `cmpz_seq_composes`, `cmpz_ld_a2_if` and `cmpz_ld_unclean` (CzSeq.v, CzLoad.v), all closed.

Item (4) is the one to remember. The tipping is free, so it had better not tip in anything that counts as established. If pouring pebbles into the second box can land its panel somewhere that isn't at or below where some other pouring would land it, the second machine gets a raised record without ever having checked anything. That's a forged certificate wearing the costume of plumbing. My first thought was right about the pebbles. It was wrong about where they land.

18.3 What doesn't compose

Here's the list of things I tried to build on the bench that fell apart, each with a proof that it does. Every one of them started as a reasonable-looking wiring job.

Theorem 18.3 (What breaks)**.** (1) Read the pair through "both flags are up" as a single flag, and any run that raises it from a clean start makes at least six record moves; the definition of Thiele-complete asks for a claim that three moves can certify, so through the pair's own interface that reading fails the non-vacuity clause. (2) Two clocks side by side are weakly Thiele-complete. They aren't Thiele-complete, so composing the easy notion composes nothing. (3) Give the small machine a second writer that changes counter A without raising its version, and the run "check that A is 0, let the other writer change A, commit, certify" raises the flag, so the earned-record clause fails when the machine is read the small machine's way, with the second writer as a base move. (4) Two windows that each price their own machine exactly can collide as a pair, so exact pricing isn't preserved by putting machines together. (5) Count the pair's certifications with one number, and the count can't tell which side certified. The pair of records can.

Checked as `cmpz_and_costs_six`, `cmpz_and_not_complete`, `cmpz_or_clocks_not_complete`, `cmpz_shared_raises`, `cmpz_shared_not_earned`, `cmpz_exactness_not_preserved` and `cmpz_sum_record_loses` (CzProdTC.v, CzShared.v, CzWin.v, CzCounter.v), all closed.

Item (1) is one flag for two boxes. It costs what two certifications cost, six record moves, and the definition wants a claim that three can settle, so a single flag over both panels, read through the pair's interface, is too expensive to meet the definition.

That's this interface. Nothing here shows that no other interface on the pair could meet it, and Theorem 18.1(4) shows the pair meeting it through a richer record.

Item (3) is the one with the most teeth, because it's the opening chapter's bug, the file checked and then swapped. On the bench it's a second hand reaching into a bin while the box isn't looking. Two writers, one value, and a check that's out of date by the time anybody leans on it. The small machine's version rule is exactly the fix, and this is what happens without it: the commitment goes through on a claim about a thing that changed underneath it. Database people and the people who build processors have been fighting this one since at least Kung and Robinson's paper of 1981 and the Livermore report of 1987, and the opening chapter called it time of check to time of use. Here it's a theorem about the record, and the version number is what makes it go away. The second hand can still reach in. With the version, its reach shows on the tally, and the COMMIT traps.

Item (5) is my first guess from the side-by-side wiring, coming back to fail on schedule. It's why the pair's record compares one side at a time. Squash the two records into one number and the record forgets who did the work, and a record that forgets who did the work isn't one anybody can lean on.

18.4 One inside another

The last way to wire the bench: put one box *inside* the other, as a guest on a host. Part of this the book already has. Section 16 showed the fixed program U_P runs any computably presented Thiele machine, and the small machine's own programs, with the record earned again by the host. What's new is putting those links in a row, a box in a box in a box, and asking what the outermost one pays.

My first guess was that it could pay less. A host that's clever about how it runs the guest might skip some of the guest's bills. The first item below puts a floor under that: the host pays at least one for each step where the guest's record went up. It's a floor on rises, and it doesn't keep the guest's whole bill.

Theorem 18.4 (Nesting)**.** (1) Let a host that pays the toll simulate a guest faithfully, with a record map that reflects the order. Say a run of the host *follows* a run of the guest when it splits into one stretch of host moves for each guest step, each stretch carrying the image of the state before that step to the image of the state after it. Every run of the guest has a host run that follows it, every such host run ends on the image of the guest's end state, and every host run that follows a guest run costs at least the number of the guest's steps that turned a cup over. A host run that only ends on the image of the guest's end state can cost less. (2) In each link the surcharge is 0 while the guest's record is down, and the bounds below hold at the points where it's

up. A small-machine guest on U_P is carried with surcharge exactly 0. A computably presented machine whose reading starts at no is carried, through its compiled guest, with surcharge at most 2, and exactly 2 once its reading has first said yes when moreover every move costs at most one. One whose reading starts at yes is carried with surcharge exactly 3. (3) Links compose, and their surcharges add: a tower of k links has surcharge between the sums of their lower and upper bounds. (4) A tower of presented machines, each level running the next on U_P, adds exactly 2 for every level above the first, on top of the first level's own surcharge, provided every level's host run is itself presented by the next.

Checked as `cmpz_follow_exists`, `cmpz_follow_reaches`, `cmpz_follow_cost_ge_exits` and `cmpz_end_only_not_enough` (CzFollow.v), with `cmpz_sim_exit_cost` and `cmpz_sim_cost_ge_exits`, `cmpz_priced_link`, `cmpz_host_link`, `cmpz_host_link_le_two`, `cmpz_host_link_exact`, `cmpz_host_link_start_up`, `cmpz_link_compose`, `cmpz_tower` and `cmpz_tower_presented_exact` (CzCat.v, CzLink.v, CzTower.v, CzSelf.v), all closed.

Item (1) is the cleanest sentence in the chapter. However you build the outer box, as long as it pays its own toll and doesn't lie about the inner box's record, it pays at least one for every step where the inner record climbed, on every run that keeps pace with the inner box step by step. The keeping pace is needed. A guest that turns a cup over and then turns it back ends where it started, and the host can reach that image by doing nothing at all, for nothing. The step by step floor is the guest's rises paid one by one; the end alone can't see them. That's a floor on the climbs, and it isn't the inner box's whole bill. A guest can charge a hundred for one rise while a host charges one for the matching rise, and both pay their tolls. Keeping the whole bill takes a further demand on the simulation, that it carry the guest's costs too, and for U_P that's the separate ledger theorem of Section 16.14, with its surcharge.

The premise in (4) is a real limit, and I want to say exactly what it is. Two links are unconditional: from a presented machine to its compiled guest, and from that guest to U_P running it. Going deeper needs the host's own run, at each level, to be presented as a machine the next level up can run, and the host keeps a register for every whole number, which doesn't have a one-to-one code in a single number the way a presented machine needs. So the box that has to fit inside the next one has more states than there are numbers to name them. For three levels and more, the theorem says what happens *if* each level is presented. It doesn't claim that it always is.

18.5 What this adds up to

Wired side by side, end to end, or one inside the other, a Thiele machine on the bench stays a Thiele machine, and the tolls add the way money adds. What breaks, breaks for reasons you could name before you saw the proof: a single flag for two claims costs two claims' worth, a clock is still a clock, a shared value without a version can't be trusted, and a free hand-off can't be allowed to carry a certificate. None of those is a surprise in hindsight. All of them are now theorems, and the composition question from Section 3 has an answer. Two certification systems side by side are one under both readings I checked, "either flag" and "both flags." The "both flags" reading still fails the definition through the pair's interface, the second writer breaks the earned-record clause through the small machine's own reading, the clocks and the two counterexamples fail outright, and simulation in general has a proved floor.

Three of my first guesses on this bench were wrong: the single count, the harmless tipping, and the clever host that pays less. The last two leaned toward a record that says more than anybody paid for, and the first toward one that says less than it should. I didn't see any of the three coming. The proofs did.

19

Running it

Everything so far is theorems. A fair reader is entitled to ask the rude question: does any of it run? The Coq definitions are drawings, exact to the last part, and every theorem in this book is about the drawings. A proof that the drawn machine does something isn't a machine doing it, and I'd be suspicious of a book about a machine that never once got built and switched on.

So this chapter builds it twice from the same drawings, once by Coq's own machinery and once by hand, switches both on, and compares them step by step. It says exactly what ran, exactly what didn't, and what you'd have to trust to believe the parts that did. Here's the short version first.

What	Status
The small machine	run: Coq, extracted OCaml and Python agree on every step
The host machines, with and without PAY	run, the same three ways
U and U_P on tiny guests	run, extracted, against the Python guest
U or U_P on a guest that certifies	proved, not run
Source programs through the compiler	run as far as the host program
The two-counter guest and U_P running it	proved, not run
The presented machine carried by U_P	proved, not run

19.1 Taking the coat off

The first build is made by Coq itself. Coq can turn a definition into a program in an ordinary programming language. It calls that *extraction*. In the drawings every part comes wearing a coat of proof, and extraction takes the coat off: the proof parts are thrown away and what's left is code that computes the same function. I extract to OCaml, an ordinary programming language that Coq's extraction supports directly. Two files do it. One extracts the small machine, the host machine with a register for every whole number, the host with PAY, both universal programs U and U_P written

out instruction by instruction, and the loaders. The other extracts the compiler below and its runner.

Two things about that build are worth knowing before you believe a word of the rest. First, the code that runs is the code the theorems are about. Coq won't extract some of the files directly (a technical limit on files that rename modules), so the extracted functions are copies with plain names, and each copy is proved equal to the original, by computation or by a short proof, inside Coq. The two universal programs are printed by Coq as literal lists and proved equal to the programs in the theorems. Checked as `rlz_host_program_is`, `rlz_phost_program_is` and `rlz_hos t_run_prog_is` (Realize.v and RealizePriced.v).

Second, whole numbers, and here my first instinct was the wrong one. The quick way to build is to map Coq's whole numbers onto the computer's own machine integers, which are fast. They're 63 bits wide here, and the guest codes U reads go up near 2^{100}, so they'd wrap around without a word of warning. Coq's whole numbers never overflow. A machine word does, and the book already told you what that cost Ariane 5 (Let me be straight with you). So the extraction maps Coq's whole numbers to an OCaml library of unbounded integers, and each arithmetic operation is mapped by hand to that library's version of it. Those are the hand-written substitutions in the trusted path: one line that maps Coq's whole numbers onto the library's integers, a few that map Coq's Booleans, decisions, lists, pairs, options and its one-value type onto OCaml's own, two that map Coq's "and" and "or" onto OCaml's, two in one file that map the hosts' one-value property types onto OCaml's unit, and the arithmetic operations, seventeen in one file and thirteen in the other. Each arithmetic line carries a one-line justification next to it: addition is addition, subtraction stops at zero the way Coq's does, division by zero gives zero the way Coq's does. The rest is Coq's own definitions. The hand-written code outside that is two drivers, which read input, print output, and count steps while they call the extracted step function in a loop.

What you're trusting, if you trust a run: Coq's extraction, the OCaml compiler and the C tools under it, the integer library, those mapping lines, the two drivers, and the Python code that does the comparing. None of that is proved. Coq's extraction, the OCaml compiler, the C tools and the integer library are old and used by a lot of people. The mapping lines, the drivers and the comparing code are mine, and they're small enough to read in one sitting. That's a reason to trust it. It isn't a proof.

19.2 A second machine, written by hand, to check the first

Switch the first build on and let it run, and what have you learned? Less than I first thought. A run of extracted code only tells you the extracted code agrees with itself.

If the extraction, or one of the integer mappings, or a driver had slipped somewhere, a run on its own would never show it. So there's a second build, made by hand: an implementation of the small machine and both hosts written separately in Python straight from the Coq definitions, with every function naming the definition it copies. Then the two builds are made to argue.

They're compared on the *whole* state after *every* step: the program line, every counter and register and its version, the fact table in order, the channel, the trap latch, the ledger and the flag. Against Coq itself, computing the same steps inside the proof assistant, the Python machine is run on every small-machine program of one or two instructions over a 26-instruction alphabet and every program of three over a 9-instruction one, 1,431 programs, each run from several starts (4,266 runs in all), 300 random ones, and a few thousand more on each host. Against the extracted OCaml, the same exhaustive and random small-machine sets again, U and U_P on tiny guests, and the runs below. They agree on every step of every one. I also broke the machine on purpose, giving it a fact table one too small, a free CHECK, a commitment that ignores versions, and a write that doesn't bump the version, and each break gets caught. A second set pins the boundaries this book draws in words, and most of those runs succeed: two runs that end in the same state with different first certifications, a check that can't fail certifying from every start, six commitments billing seven without a flag, a write after the commitment and a flag over a claim that's now false, sixteen copies of one CHECK filling the table, a CERTIFY costing one at a ledger already at two, and a merge between two yes-states that leaves the entering state alone.

That's evidence. It isn't proof, and Section 22 says why Dijkstra would want me to say so. What it is, is three routes to the same numbers: Coq computing inside itself, the code Coq extracted, and a machine written by hand from scratch. When they agree on every step of thousands of programs, a slip in the extraction, the integer mappings or the drivers would have to be matched by the same slip in a machine written separately by hand to hide. I can imagine that happening. I can't imagine it happening often. The comparison doesn't test whether the definitions say the right thing, since the hand-written machine copies them. That's what having them on the page is for.

The same holds for how it was made. The Python machine was written from the same Coq definitions, by the same process as the Coq itself, the one the short version at the front of the book describes (I wrote the code and the proofs with language models). So it guards against a slip in the extraction, the integer mappings or the drivers. It doesn't guard against a misreading of the definitions: whatever reading went into the Coq went into the Python with it.

19.3 U on a real guest

Now the universal program, which is where I expected trouble. The guest I wanted to run first was the obvious one, a guest that goes the whole way: CHECK, COMMIT, CERTIFY, flag up, run inside U on the built machine. Here's why that's not what ran. U keeps the whole guest program as one number, a number that grows like two raised to the sum of the codes for the guest's instructions (for a guest of one instruction it's exactly a power of two), and it reads that number the only way a counter can, one unit at a time. So its speed depends on the code, and the code grows fast: a guest with one CHECK in it has code 2^{24}, and a guest with the whole chain has a code near 2^{100}.

So I came down to what could finish. Here's what ran, on the extracted program, compared against the Python guest at the end: the host stopped exactly when the guest did, and showed the guest's counters, latch, ledger and flag.

guest	host	host steps	time
count A down from 2	U	25,953,317	
one passing CHECK	U	402,654,059	618 s
one failing CHECK	U	201,327,155	305 s
one passing CHECK	U_P	402,654,059	617 s
one failing CHECK	U_P	201,327,155	313 s

The first row's count is pinned exactly: if the build changed so that it moved by one step, the comparison would fail. The times are from one ordinary computer and are only there to give you the scale. That's about twelve host steps for every unit of the guest's code, for every guest step, which is exponential in the size of the guest. A guest with one CERTIFY in it would have code 2^{32}, about fifty billion host steps, and at the measured rate of about 650,000 steps a second it would take something like twenty-two hours, nearly a day. The guest I wanted, with the whole chain, has a code near 2^{100}, which at the same twelve steps a unit is more than 10^{31} host steps a guest step; at the measured rate, with this encoding on that computer, that's around 10^{18} years. So U on a guest that certifies is proved. It isn't run. I'm not going to dress that up. Runs this long only fit in memory at all because the host's register store can be squeezed during the run without changing any state, which is a theorem too. Checked as `rlz_sched_sound` (RealizeCompact.v).

Why so slow? Because a fixed program on a machine of counters has to keep the guest packed into numbers, and counters only count. Whether every universal program on counters has to pay this much is a real question, and this book doesn't answer it. What it claims is exactly what the table shows. I'll admit the table stung a little. About ten minutes to watch one CHECK pass is a long time to stare at a machine you built to be small.

19.4 A compiler, all in Coq

Running guests one CHECK at a time doesn't tell you whether the built machine can do anything you'd recognise as a program. You'd want to hand it gcd, or a sort, written the way people write them, and get the answer out, and writing gcd by hand in INC and DEC is no way to find out. So the last piece in the drawings is a compiler. The source is a small language you could write by hand: whole-number variables, addition, subtraction that stops at zero, comparisons, *if*, *while*, and procedures that return several results, with the rule that a procedure can only call ones defined before it, so there's no recursion. Every stage is a Coq function with a Coq proof: the procedures are inlined, the result is flattened to assignments and jumps, that is compiled to Minsky's counter machine through the library's own compiler, that to a program for the host machine, that to a two-counter guest, and the guest to a run of U_P.

Theorem 19.1 (The compiler is correct)**.** For every source program with no recursion, and every input: the program computes y exactly when the runner (the Coq function that gets extracted) halts with y for some amount of fuel; exactly when the compiled host program halts with y; exactly when the two-counter guest halts with y; and exactly when U_P running that guest halts with y, with its first counter at 0, its ledger at 0 and its flag down. So compiling a program computes its answer and certifies nothing: the compiler is a route to answers, and a certification has to be written into the source as a CHECK, a COMMIT and a CERTIFY. The runner also takes exactly as many steps as the host program does, with the rest of its fuel left over, and a run that doesn't halt has used all its fuel.

Checked as `cmp_final` (CmpFinal.v), with the runner as `cmp_exec_machine`, `cmp_exec_sound`, `cmp_exec_complete` and `cmp_exec_unhalted` (CmpRun.v) and the source language's own interpreter as `cmp_interp_sound` and `cmp_interp_complete` (CmpLang.v), all closed. The statement says nothing about how many steps the compiled program takes. It's about answers.

The runner and the compiler are extracted, and the build has been run on a set of programs I picked to be annoying: gcd, factorial, a primality program written with procedures, an Ackermann tower, a sorting network on a list packed into one number, and a tiny interpreter for another machine written in the source language. Then 220 random programs that halt, 160 random programs with loops that might not, programs that never halt run for a fixed budget, and programs mutated on purpose to make sure the comparison notices. Each answer is compared with plain Python arithmetic, an independent Python interpreter of the source language, Python machines running the compiled programs step for step, and, for sixteen of them, Coq

computing the same thing itself. Ackermann's function at $(3, 5)$ is 253; the source program takes 64,768 operations to get there, and the compiled host program takes about 119 million steps. The extracted runner does a few million host steps a second, so that's well under a minute.

That's about 1,800 host steps per source operation for Ackermann, and up to about a hundred thousand for multiplication by repeated addition, because every addition copies a running total one unit at a time. Counters are unary, and unary is slow, and the slowdown grows with the values. How it grows with the size of the program I haven't measured. The last two stages, the two-counter guest and U_P running it, are proved. They aren't run, for two reasons: the files that build the guest aren't extracted, and even if they were, U_P on a compiled guest would be slow for the same reason U on a certifying guest is. Still, I like that 253. It came out of nothing but INC and DEC, and a proof that they mean what the source meant.

19.5 What wasn't built

No chip. The drawings were never cut into metal. Earlier work on this project went into hardware, a chip design for a much bigger machine, and none of it was carried over: there's no circuit for the small machine, no simulation of one at the level of gates, and no board ever ran any of this. The small machine exists as Coq definitions, as extracted OCaml, and as a Python program written by hand, and those three agree on every step anybody has checked. That's the whole of what "it runs" means in this book.

I'd love to see it in metal some day. But builds that can be held against each other on every step are worth a lot to me. A board with nothing to hold it up against isn't, and that's the trade this edition made.

20

The CHSH check

The small machine checks whether a number is zero, even, or at least something. That's a fine way to show the chain, and a dull thing to certify. Here's a check worth earning, built from zero, and then the one thing the small machine's host does with it, which is smaller and more exact than "it checks for quantum behaviour," a sentence that's tempting and wrong. The short version, so you know where this is going: the check never computes the famous number and never writes down the famous bound. It tests seven facts about integers in eight counts. If they pass, a certain five-by-five table has a property called positive semidefinite, and that property forces the famous number under the famous bound. The wall is a consequence of the check. It never goes into it. Everything else is explaining those sentences.

But first, why is a quantum game in a book about records at all? Not for the physics. Nothing in this section tests nature, and the check can't tell honest play from a made-up tally (the guarantee paragraph below says exactly how). It's here for three reasons, all about the record. It's a check whose content is striking enough to tempt anyone into thinking that certifying it should cost more than certifying something dull, and on the small machine it doesn't: a flag raised from a clean start costs at least three, and exactly three at the cheapest, whatever the claim (Section 16). It sets free observation against a priced check: a round of the game is written down for nothing, and judging the pile is one of the three paid acts. And it's an integer test small enough to be a property in the small machine's sense, a checker that's a little program and a meaning that's a statement, with a proof that the two agree, which is exactly what Section 16's generic machine asks of any property before it lets a flag stand on it.

20.1 The game

Two players, Alice and Bob, in separate rooms. A referee flips two coins and hands Alice a question x, zero or one, and Bob a question y, zero or one. Alice answers a, zero or one; Bob answers b. No talking during the game. They may agree on a plan beforehand, as detailed as they like. They win a round when their answers *differ exactly when both questions were one*. In words: if either question is zero, match; if both are one, differ.

Try to win every time. A plan is a list: what Alice says to x equals zero, what she says to x equals one, and the same two for Bob. Four bits. Now write the four requirements the rounds impose: a0 equals b0 (both asked zero); a0 equals b1; a1 equals b0; a1 differs from b1. Chain the first three: a0 equals b0 equals a1, and a0 equals b1, so b0 equals b1. But the fourth says a1 differs from b1, and a1 equals b0 equals b1. Contradiction. One of the four must break. So any plan loses at least one of the four question pairs, and if the referee asks all four equally often, the best a plan can do is three out of four. Seventy-five percent. Do it the long way too: there are sixteen plans (four bits), and if you write them all out and count wins you'll find eight win three rounds and eight win one, and none wins four (the broken requirements always come in odd numbers, which you can see by adding up the four equations and keeping only whether the totals come out even or odd, with "equal" as 0 and "differ" as 1: each of the four answers appears twice on the left, so the left sides add to an even number, and the right sides add to one). Sharing a coin before the game to pick a plan at random doesn't help; a mixture of numbers no larger than three-quarters is no larger than three-quarters. Arthur Fine, a philosopher of physics at the University of Washington, much of whose work is on the physicist Albert Einstein and the quantum theory, proved in 1982 that the whole "pre-agreed answers, no influence between rooms" story is *exactly* the mixtures of those sixteen lists, one scoreboard for all four questions at once; the sixteen are the whole classical story. That one I don't have to take on his word: the form of his theorem that speaks of the four averages is checked in this book, and the end of this section says how.

The number. Recode answers zero and one as plus one and minus one. For each question pair, let E be the average, over many rounds, of Alice's answer times Bob's. The CHSH quantity is S equals E00 plus E01 plus E10 minus E11. For one fixed plan each win adds one to S and each loss subtracts one, so S is wins minus losses: a three-win plan has S equal to two, a one-win plan has minus two. With equal question frequencies, the win rate is one half plus S over eight. So "seventy-five percent" and "S at most two" are the same fact, and the second is the one that got the name.

The ceiling is easy to reach and costs nothing to write down. Tally one round per question pair with both players answering zero every time: after the recoding every answer is plus one, so all four correlators are one. That's the plan that answers zero to everything, which matches on every pair and so wins the three pairs that ask for a match, $S = 1 + 1 + 1 - 1 = 2$. On the small machine's host a tally like that is a few INCs on counters, and INC is free. Nothing in it says the rounds were played; anybody can tally anything for nothing. The other side, that no finite mixture of plans beats two, is Theorem F.2 in Appendix F, one direction of Fine's result. The full

equivalence, in the form that speaks of the four averages, is checked too, as `nec_e_fine_iff`; the last part of this section states it.

20.2 Where it came from, and why anyone cared

So who started this, and why would anybody care whether two rooms can win a coin game? The names come in a line, and the line didn't mean much to me until I saw that each of them moved the same thing a notch: from an argument about what's real, which you can only agree with or not, toward a pile of counts a referee can judge. Each one rests on a different piece of the game above.

The trail starts in 1935, with Einstein, by then at the Institute for Advanced Study in Princeton, and two younger colleagues there, Boris Podolsky and Nathan Rosen, asking whether quantum mechanics describes everything that's real about a pair of particles. Their argument: if measuring one particle lets you predict the other's result with certainty, the result must already have been fixed, and the theory leaves that out. What they rest on is the plan in each player's pocket, a pre-agreed list of answers, the plan above. And the first thing worth checking is whether the paper even says what Einstein meant. By his own account, not quite. Podolsky did the writing, and that June Einstein wrote to Erwin Schrödinger, the Austrian physicist whose 1926 wave equation is at the heart of quantum mechanics, that the essential thing had been "smothered by formalism." The person who lays that letter out is Arthur Fine, the same Fine whose theorem closed the sixteen plans above, in his encyclopedia entry on the argument. So Fine stands at both ends of the line: he checked the founding paper against what its author meant, and he proved exactly what the plans can do. Even the founding paper is a lead to check. It isn't a verdict.

For thirty years that was philosophy. In 1964 John Bell turned it into arithmetic: if each particle carries pre-agreed answers and nothing travels between the rooms, certain averages can't exceed a fixed number, and quantum theory predicts they do. What that rests on took me a few reads. His inequality rules out the *pair* of assumptions together, called local realism (answers exist before you ask, and no influence across the gap). Either one alone survives it. And where does a paper like that get published? In a brand-new journal called *Physics Physique Fizika*, which charged authors no page fees and in fact paid them by the word, ten cents a word, and which folded in 1968 after four volumes, or three. I went looking for that count, and the sources don't agree on it: four is the number the American Physical Society's archive of the journal gives, and Iulia Georgescu, writing about the journal in *Physics World* in 2026, says three. She's also where the ten cents comes from. As Andrew Whitaker tells it in his biography of Bell, the journal gave out no free copies, so Bell spent his payment buying copies of his own paper to mail to other physicists.

A result nobody else holds a copy of hasn't left the room yet, and Bell paid to send his across. In 1969 four physicists, John Clauser, then a graduate student at Columbia, Michael Horne and Abner Shimony at Boston University, and Richard Holt at Harvard, rewrote it as the four-average form above, S at most two, which could be tried in a lab with imperfect equipment. It's called the CHSH inequality, after their initials. That notch rests on what a real lab can deliver. Shimony was in philosophy and in physics at Boston University both, which is the whole line in one man.

Then the obvious next question: did anybody actually try it? Stuart Freedman, a graduate student at Berkeley, and Clauser, by then a postdoc there too, tried it in 1972. The physicist Alain Aspect's group at the Institut d'Optique in Orsay, with two colleagues there, Jean Dalibard and Gérard Roger, did it in 1982 with the measurement settings switched while the photons were in flight, which in the game's terms, as I read it, is the referee handing out the questions too late for any note about them to cross between the rooms. Before that, in 1975, Aspect went to see Bell at CERN to tell him what he planned, and by Aspect's own account Bell heard him out in silence and then asked, first thing, whether he had a permanent position. Aspect did. Bell's reason, as Aspect tells it, was that most physicists would take the experiment for a waste of time, or take him for a crackpot. I read that as the toll in its oldest form: a check that could have come out the other way costs somebody something to run, and Bell wanted to know who'd pay. And in 2015 three groups of experimental physicists (Bas Hensen's in Delft, with electrons 1.3 kilometres apart; Marissa Giustina's in Vienna; Lynden Shalm's at NIST, the National Institute of Standards and Technology, in Boulder) closed the two main escape routes at once. In 2022 Clauser, Aspect and the Austrian physicist Anton Zeilinger shared the Nobel Prize "for experiments with entangled photons, establishing the violation of Bell inequalities and pioneering quantum information science." The 2015 tests came out of that line of work, and Zeilinger was one of the authors of the Vienna one. So in under fifty years the subject went from a question you needed a permanent job to risk to a Nobel. The lab number sits above two. A remaining escape, that the referee's coins were secretly correlated with the particles, is a philosophical one and I'm not going to pretend it's closed.

Now put this book's check on the same line, because that's where my position gives. It sits past the far end, at the counts. Those fifty years went into making the counts worth judging, rooms really apart, questions really late, escape routes really shut, and my check starts after all of it. It's handed eight counts and can't tell a lab's tally from one somebody wrote down for free. So the check is a referee of counts, and everything that made these counts worth refereeing happened upstream of it.

What quantum play does: if the players share an entangled pair and choose what to measure from their question, the best they can do is S equals two times the square root of two, about 2.83, a win rate of about eighty-five percent. Boris Tsirelson, a Soviet mathematician who later taught at Tel Aviv University, in a 1980 paper called "Quantum generalizations of Bell's inequality," proved that that's a *ceiling*: quantum correlations violate Bell, but only up to there. That ceiling, and the state that reaches it, are checked for strategies in finite dimension, with real amplitudes (`qs_tsirelson` and `qs_tsirelson_reached`, QuantumStrategies.v) and with complex ones (`qc_tsirelson` and `qc_tsirelson_reached`, QuantumStrategiesComplex.v); and the ceiling holds in any dimension at all, infinitely many levels included (`ta_tsirelson`, TsirelsonAlgebraic.v; the next part says how). What's checked below, besides, is the algebra.

And the wall above that, which is the one I find most instructive. In 1994 the physicists Sandu Popescu and Daniel Rohrlich asked: what if you only demand that nothing can be *signalled*, that Alice's answers on their own have the same odds whatever Bob is asked, and the other way round? Then you can write down a "box" (the PR box, after their initials) that wins *every* round (its answers differ exactly when both questions are one, by fiat, each answer alone a fair coin), S equals four, and nothing about it lets either room send a message. So relativity alone doesn't explain the quantum ceiling; the PR box obeys relativity and beats 2.83. The order is: two for plans, 2.83 for quantum, four for no-signalling.

And the box has the same kind of history as everything else in this book: somebody got there first. Popescu and Rohrlich's initials stuck, but the big review of the field by Nicolas Brunner and four coauthors credits boxes like it to Peter Rastall, whose 1985 paper on locality and Bell's theorem appeared in *Foundations of Physics*, and, separately, to Leonid Khalfin, a physicist at the Steklov Institute's Leningrad branch, and Tsirelson himself, both in 1985, nine years before. Then the question that really matters: if relativity isn't what stops things at 2.83, what is? People have tried candidates. In 2005 the computer scientist Wim van Dam, who works on the theory of quantum computing, argued, in a paper on the implausible consequences of such boxes, that with enough PR boxes, Bob could work out any yes-or-no function of both their inputs after Alice sends him a single bit, which would make the whole question of how much two parties need to talk collapse, and that's a strong hint the boxes are too good to be real. In 2009 Marcin Pawłowski, a physicist at the University of Gdańsk, and five coauthors proposed a principle they called information causality: Bob can't get more information about Alice's data than the bits she actually sends him. They argue that ordinary and quantum physics both obey it, and that any theory that beats 2.83 breaks it. How much of the rest of the quantum boundary it reaches is something I haven't checked. Which principle fixes the whole quantum

boundary, if any one does, this book won't answer. The check below is a fence at the middle step.

20.3 Five sentences that are easy to say by accident

Here they are, each with what I can stand behind beside it.

"Faster than light." Nothing travels. Each room alone sees fair coins; the correlation shows up only when you lay the two columns side by side afterward. "Quantum means random." The average S is fixed by the state; what's random is each single answer; the plans are the less random side. "A Bell violation proves nonlocality." It rules out local realism, both assumptions together, and every real test has loopholes with names. "A tally above two proves nonlocality." No: S is a *population* average, and one finite tally can land above two by luck. Four rounds, one per question pair, with coin-flip answers, reach S equals four one time in sixteen. That gap between a population bound and a finite sample is the problem Yanbao Zhang, Scott Glancy and Emanuel Knill, at NIST in Boulder, work through for real Bell tests (Physical Review A, 2011 and 2013), and the book doesn't solve it. The way they solve it is worth knowing, because it has the shape of this book's record. Turn each round into a small bet that no local plan can expect to win, even a plan that remembers every earlier round, multiply the bets as the rounds come in, and the French mathematician Jean Ville's inequality, from his 1939 thesis, one of the works that started the theory of martingales, says the chance the product ever climbs past one over δ is at most δ. So a flag can go up for good the moment the product crosses, with its error stated up front. It needs one thing the small machine doesn't have: settings drawn underneath the program, where the program can't choose them. And how much can one round tell you? Wim van Dam, Richard Gill, a mathematical statistician who later held the chair in that subject at Leiden, and Peter Grünwald (2005) measured exactly that, in bits of evidence against every local plan. Of the two-party proofs they compare, CHSH comes out the strongest, "much stronger" than the others in their words, at about 0.046 of a bit per round, a few hundredths, so a convincing record takes hundreds of rounds. Four won't do. The checked work proves only the narrow thing, that a tally above two is not explained by *one fixed plan replayed every round* (`chsh_violation_rules_out_locally_factorizable_coupling`), and that narrow thing is what the sentence has to stay. And "the PSD test proves the data is quantum." (PSD is short for positive semidefinite, the property of the five-by-five table from the short version; the next part builds both.) It doesn't, and the direction matters. Passing implies the five-by-five table, pinned in a way the next part explains, is PSD, and that implies S squared is at most eight; both are proved. The correlators of a PSD table like that are the correlators of some quantum strategy. That's Tsirelson's representation theorem, and it's proved here: the strategy is built on two systems of

eight levels each (`tr_representation`, TsirelsonRepresentation.v). The exact edge of the quantum set for this game was found by Lawrence Landau, whose 1969 Berkeley thesis was on the quantum theory of local observables, and Lluís Masanes, then at the University of Barcelona (next part), and that's proved here too (`am_arcsine`, ArcsineBoundary.v). Failing says nothing about whether the play was quantum: the check is complete for the pinned matrix, the quantum set is wider than that, and quantum behaviour can fail it. Every one of the sixteen plans fails it (`nec_e_check_refuses_all_plans`), and I'll show you why. What it guarantees is in one paragraph, once the check is on the host machine.

20.4 The five-by-five, and what positive semidefinite means

Here's the mathematics the check rests on, and it's one idea: a table of numbers *could be a table of dot products*. Take a two-by-two symmetric table with a and c on the diagonal and b off it. It's positive semidefinite (PSD) when a and c are at least zero and a times c minus b squared is at least zero. Check two by hand: ones on the diagonal and one half off, the determinant is three quarters, passes; ones on the diagonal and 1.2 off, the determinant is negative, fails. The reading: PSD means the table could be the dot products of some arrows, and 1.2 is too big a dot product for two arrows of length one. That's the whole idea, generalised to bigger tables.

Now the table. Rows and columns labelled I, A0, A1, B0, B1 (the identity, Alice's two questions, Bob's two). Diagonal all ones. The I row and column zero off the diagonal (that's "zero marginals": the table takes each answer alone to average to zero, a fair coin; the counts record no marginals and the check doesn't test this). A0 with A1 zero, B0 with B1 zero (no round asks Alice both questions, so no tally holds that average, and the table sets it to zero). Those zeros are choices. They pin the table to one slice, and I'll come back to what they cost. And the four Alice-Bob cells hold the four correlators E. That's a five-by-five with four free numbers in it. It splits into the one in the corner and a four-by-four block made of an identity and the two-by-two grid E of correlators, and PSD of that block is the same as saying *E shrinks every vector* (E is a contraction), which is three inequalities: one minus E00 squared minus E10 squared at least zero; one minus E01 squared minus E11 squared at least zero; and their product at least the square of (E00 E01 plus E10 E11).

Run it. All four correlators 0.6 except E11 at minus 0.6, S equals 2.4: the two diagonal terms are 0.28 and 0.28, the cross term zero, passes; above two, below 2.83. All four at one over root two (E11 negative): diagonals zero, cross term zero, passes with no slack, S equals two root two exactly, the 2.83; nudge any entry's size up and a diagonal goes negative. The all-ones plan (one, one, one, one): first diagonal is one minus one minus one, negative, *fails*, though S equals two and it's a perfectly legal

classical play. The PR box (one, one, one, minus one): fails the same way. A fence at the middle step, and a fence that keeps out every one of the sixteen plans: each has all four correlators at plus or minus one, so the first diagonal is always one minus one minus one. Of the classical plays, only diluted mixtures of plans get through.

That point, one over root two in size in every entry, is four real numbers, and no tally of rounds ever lands on it. A tally's correlators are ratios of whole numbers, and the point needs each of them to be one over root two, which would make root two a fraction, and you killed that fraction with a pencil in Part I. That point and its mirror image, every sign flipped, are the only places the bound is reached (Remark F.2 in the appendix). So a passing tally always has S squared strictly below eight. It can get as close as you like: the (17, 3) tally in the five examples below has S squared equal to 7.84, and tallies of the same shape with a fraction nearer one over root two get nearer still. Two root two is the least bound the check can promise, and no tally the check accepts ever reaches it. Hold on to that corner; it comes back.

Picture what quantum play can reach as a field, and every test as a fence. In 2007 Miguel Navascués, Stefano Pironio and Antonio Acín, in a paper called "Bounding the Set of Quantum Correlations" (NPA, after their initials), came at the edge from outside, resting on necessary conditions. In their terms a table like this one is the first rung of a ladder: bigger tables of expected products, each PSD, each a necessary condition for quantum behaviour when its entries are left free, each rung tighter, and (their 2008 paper) converging to the quantum set (strictly, the set from commuting operators, which can differ from the tensor-product set in general scenarios). Each rung is a fence with the whole field inside it. The check's table is the first rung with the marginals and the two within-party entries pinned to zero. Pinning is a stronger demand than the rung makes, so passing puts you inside the first rung, and a quantum point can still fail. My fence cuts across the field. Lawrence Landau in 1988 and Lluís Masanes in 2003 came at the edge itself, for this two-questions-each case only. I'd have drawn a flat wall at 2.83. The exact boundary they found is a curved surface (the arc-sine form) touching 2.83 at one point. It's checked here: a table has a PSD completion exactly when its entries lie between minus one and one and their arcsines pass four inequalities (`am_arcsine`), so those are exactly the quantum tables (`am_quantum_arcsine`, ArcsineBoundary.v).

So the obvious next question: if every rung is tighter than the last, does some rung finally land exactly on the quantum set and stop? My first guess was yes, at least here. A 2026 paper by Anubhav Chaturvedi, a quantum-information theorist at the University of Gdańsk, says no: no *finite* rung of the ladder captures the whole quantum set even in this simplest case. And for the four correlators alone, a 2008 paper by the same Navascués, Pironio and Acín works out that the first rung already lands on Landau and Masanes's curved edge. That looked like a contradiction

until I saw what each one fences. Landau and Masanes draw the edge for the four correlators alone. Chaturvedi's is the full behaviours, each room's own odds included. So there's the corner again from the other side: tallies never reach it, and, if his proof holds, rungs never close on the field. I wanted that to be one fact. It's two. Mine is root two again, about which points a count can hit. His is about the fences. His result backs, from outside, the caution this book keeps about the ladder, that no finite rung may close on the whole quantum set. It says nothing about my own check, though. My check's caution, that a quantum point can fail it, comes from the pinning, and his paper never pins anything.

20.5 The check, in integers

Eight counts: for each of the four question pairs, one counting rounds where the answers matched and one where they differed. A round is one bump to one count; nothing about the round is tested, and no marginals are recorded. Then the check reads the eight and tests seven integer facts. For each pair, let N be same plus different and D be same minus different (so E is D over N). The seven: the four N are strictly positive (every pair was sampled; a pair with no rounds is rejected outright, and never read as a correlator of zero); A, which is N00 squared N10 squared minus D00 squared N10 squared minus D10 squared N00 squared, is at least zero; B, the same with the other column, is at least zero; and with C equal to D00 D01 N10 N11 plus D10 D11 N00 N01, C squared is at most A times B. Those are the three contraction inequalities with the denominators cleared. No division, no reals, no square root, no S. Try it: counts (same, different) of (4, 1), (4, 1), (4, 1), (1, 4) give E of 0.6, 0.6, 0.6, minus 0.6; N is five and D is three (minus three for the last); A is 175, B is 175, C is zero; passes. That's the first example above, the S equals 2.4 one, in integers.

The theorem chain, in words: a passed check gives PSD of the five-by-five built from the counts; contraction and PSD are equivalent, both directions; and PSD forces S squared at most eight. The names are in the statements below. The underlying algebra is one identity: four times the sum of the four correlators squared, minus S squared, is a sum of six squares (`chsh_gap_is_sum_of_squares`), so if the row sums of squares are at most one, S squared is at most eight (`tsirelson_squared`), and the point one over root two reaches it (`tsirelson_tight`). That's Tsirelson's bound as *algebra*, with no Hilbert space (the mathematical home of quantum states) in it.

Here is the chain as three statements. The derivations are in Appendix F; this is what they prove.

Definition 20.1 (Correlators from the counts; the pinned matrix)**.** For eight counts and a question pair xy, let $E_{xy} = (\text{same}_{xy} - \text{diff}_{xy})/(\text{same}_{xy} + \text{diff}_{xy})$, read as 0 when both are 0. For reals E_{xy}, $M(E)$ is the symmetric five-by-five matrix on I, A_0, A_1, B_0, B_1 with ones on the diagonal, E_{xy} in the cells (A_x, B_y) and (B_y, A_x), and zero everywhere else (Definition F.5). M is *PSD* when $v^{\mathsf{T}} M v \geq 0$ for every real vector v, and $S(E) = E_{00} + E_{01} + E_{10} - E_{11}$.

Theorem 20.1 (The integer check implies the pinned matrix is PSD)**.** If eight counts pass the seven integer facts, then $M(E)$ for their correlators is symmetric and PSD.

Proof. The four positivity facts make every denominator positive, so each E_{xy} is D_{xy}/N_{xy}, and dividing the other three facts by positive squares turns them into the three contraction inequalities (Lemma F.15). Contraction is equivalent to PSD of $M(E)$ by a Schur complement on the four-by-four block (Corollary F.13). □

Checked as `column_contractive_check_witness_sound` with `npa_psd_iff_column_contractive` (Theorem F.16 and Corollary F.13 in the appendix).

Theorem 20.2 (PSD forces the Tsirelson bound)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$: if $M(E)$ is symmetric and PSD, then $S(E)^2 \leq 8$.

Proof. A PSD matrix with unit diagonal has every three-by-three principal minor at least zero. On rows A_0, B_0, B_1 that minor is $1 - E_{00}^2 - E_{01}^2$, and on rows A_1, B_0, B_1 it is $1 - E_{10}^2 - E_{11}^2$, so the four squares sum to at most 2. The identity $4\sum_{xy} E_{xy}^2 - S^2 =$ a sum of six squares then gives $S^2 \leq 4 \cdot 2 = 8$. □

Checked as `npa_psd_implies_tsirelson_bound`, with the row bounds `npa_psd_zero_marginal_implies_row_bounds` (CHSHColumnCheck.v), and the identity `chsh_gap_is_sum_of_squares` (TsirelsonFromAlgebra.v); this is Theorem F.14 in the appendix.

Corollary 20.3 (A passed check means the bound)**.** If eight counts pass the seven integer facts, their correlators satisfy $S^2 \leq 8$. In fact $S^2 < 8$, and passing tallies come as close to 8 as you like.

Proof. Theorem 20.1 supplies positive semidefiniteness of the pinned moment matrix for these four correlators. Theorem F.14 in the derivation appendix applies to every such matrix and gives $S^2 \leq 8$. Its quadratic form calculation is printed there. The score is a ratio of whole numbers, and no such ratio squares to 8, so the bound is

strict. The tallies whose four correlators are $n/q, n/q, n/q, -n/q$ with $2n^2 \le q^2$ pass, and their scores $4n/q$ come as near $2\sqrt{2}$ as you like. □

Checked as `nec_e_check_strict`, with `nec_e_check_never_tsirelson` for $|S| < 2\sqrt{2}$, `nec_e_no_sqrt8` for the fractions, and `nec_e_check_sharp` for coming as close as you like (NecEChsh.v).

Remark 20.1 (Sound, and incomplete)**.** The pinned matrix is stricter than the quantum question it is usually read against. PSD of $M(E)$ gives the correlators a PSD completion, since the zeros are one choice of the two free entries (`zero_marginal_implies_elliptope`, ElliptopeCompletion.v). The converse fails. The all-ones plan has a PSD completion (`deterministic_strategy_elliptope`), and every deterministic plan is a quantum strategy (`qs_deterministic_plan`, QuantumStrategies.v); yet its minor on rows A_0, B_0, B_1 is $1 - 1 - 1 = -1$, so $M(E)$ is not PSD there, and a tally of one matching round per pair gives $A = 1 - 1 - 1 = -1$ in the integer check, so the check fails. The second failure is a named theorem on the machine: the tally of one matching round per pair is refused forever (Proposition 20.6). Every deterministic plan fails the same way. The tally of one round per pair fails the check for each of the sixteen plans (`nec_e_check_refuses_all_plans`), so does a plan replayed over any number of rounds once pairs 00 and 10 have been sampled (`nec_e_check_refuses_deterministic`, NecEChshInt.v), and no deterministic plan's pinned matrix is PSD (`nec_e_deterministic_not_pinned_psd`, NecEChsh.v). So a pass proves the bound, and a failure proves only that the tally's pinned matrix isn't PSD or a pair went unsampled; it says nothing about whether the play was classical or quantum. The free tally from the start of this section is exactly such a tally: it reaches $S = 2$ and the check throws it out.

None of that algebra is new, and I don't want it to read as if it were. Tsirelson's bound has been formalised before, in Isabelle/HOL by Mnacho Echenim and Mehdi Mhalla, computer scientists at Grenoble (2023), and in Lean's mathlib, and Navascués, Pironio and Acín already analyse this very matrix by Schur complement and discuss the level above it. What I think is mine here is smaller: an integer decision procedure on the counts, and a checked link from "the check passed" to PSD to the bound.

The picture under the check is older still. Tsirelson showed, in a longer paper of 1987 after his 1980 note, that the correlations two separated quantum parties can reach are exactly the tables of inner products of unit vectors, one vector for each question on each side. With the marginals pinned to zero the pinned matrix is that picture, and by its Schur complement it is PSD exactly when the two-by-two table of correlators, read as a matrix, has operator norm at most one. The physicists

Reinhard Werner and Michael Wolf's survey of Bell inequalities (2001) sets out that correlation-matrix view for two-party inequalities in general.

20.6 The check, earned on the host machine

Now put it on the machine. A check on the small machine is a property: a checker, which is a little program that says yes or no, a meaning, which is a statement, and a proof that the two agree. The version of the small machine with its property language left open (Section 16, Theorem 16.25) takes any property language with an exact equality test whose properties come with those three things, and for every one of them a flag raised from a clean start stands on a passing CHECK of one claim, a later COMMIT of the same claim with its counter untouched in between, and a later CERTIFY, the three acts costing one each. So here's the plan for the game. Pack the eight counts into one counter, as a list, the way the sorted-list machine packs a list into one number (Section 16). Let the checker be the seven integer facts read off the unpacked counts. And let the meaning be what the facts are for: every pair was sampled, and the pinned five-by-five is positive semidefinite. Then the proof the generic machine asks for, that the checker says yes exactly when the meaning holds, is the chain above with the denominators cleared in both directions, and a flag raised on that machine from a clean start means: these counts, at this edition of their counter, passed the check, somebody committed to that with nothing changed underneath, and the correlators they give satisfy $S^2 \leq 8$. Here is that, stated exactly, and it's stronger than the chain above in one way I want to say once and plainly. The earlier theorems said that a passed check makes the pinned matrix PSD. The check is also complete for it: it passes *if and only if* every pair was sampled and the pinned matrix is PSD, because clearing the denominators works in both directions. The statement is over the machine of Definition 16.8, the small machine with a counter for every number, with PCHSH in place of its one property, Slot. A CHECK reads one counter, which is why the eight counts have to travel in one.

Definition 20.2 (The packed tally, and the property PCHSH)**.** Read a whole number as a list in the list code (0 is the empty list, and $2^x(2y+1)$ is x followed by the list y), and take as its *tally* the first eight entries, in the order $\text{same}_{00}, \text{diff}_{00}, \text{same}_{01}, \text{diff}_{01}, \text{same}_{10}, \text{diff}_{10}, \text{same}_{11}, \text{diff}_{11}$, a missing entry reading 0. Every number is some tally. The property PCHSH has the checker "the tally passes the seven integer facts" and the meaning "every question pair was sampled, and the pinned matrix $M(E)$ of the tally's correlators is symmetric and PSD."

Theorem 20.4 (The check is exact)**.** (1) The seven integer facts hold of a tally if and only if every pair was sampled and $M(E)$ is symmetric and PSD. (2) For every whole number v, the checker of PCHSH says yes at v if and only if the meaning holds of v's tally. (3) Reading the number that packs a tally gives that tally back. (4) If the facts hold, the score of the tally satisfies $S^2 \leq 8$ and $|S| \leq 2\sqrt{2}$; in fact $S^2 < 8$ and $|S| < 2\sqrt{2}$, and passing tallies come as close to 8 as you like.

Proof. (1) The forward half is Theorem 20.1. For the backward half, PSD gives the three contraction inequalities (Corollary F.13); with every N positive, multiplying them through by the squares of the denominators gives $A \geq 0$, $B \geq 0$ and $C^2 \leq AB$, which is Lemma F.15 read the other way. (2) is (1) with the packing, since every number is a tally. (3) is the code and its decoder being inverse. (4) is (1) forward, then Theorem 20.2; the strict form and the closeness are Corollary 20.3. □

Checked as `small_chsh_check_iff`, `small_chsh_clear_denominators`, `small_chsh_psd_iff_contractive`, `small_chsh_psd_tsirelson`, `small_chsh_check_tsirelson`, `small_chsh_eval_iff` and `small_chsh_tally_of_code` (SmallChshCheck.v, SmallChshMachine.v). The non-strict bounds of part (4) are a single checked theorem, `small_chsh_check_tsirelson`, with both bounds in one statement. The strict form of part (4), and the closeness, are checked as `nec_e_check_strict`, `nec_e_check_never_tsirelson` and `nec_e_check_sharp` (NecEChsh.v).

Theorem 20.5 (A raised flag stands on the check)**.** On the machine above, let τ be a trace from a clean start s_0 that ends with the flag up. Then, for some counter c,

$$\tau \;=\; pre \cdot \text{CHECK PCHSH } c \cdot mid_1 \cdot \text{COMMIT PCHSH } c \cdot mid_2 \cdot \text{CERTIFY} \cdot post,$$

and with v the value of c after *pre*: the tally of v passes the seven integer facts; every pair was sampled and the pinned matrix is symmetric and PSD; its score satisfies $S^2 \leq 8$ and $|S| \leq 2\sqrt{2}$, and in fact $S^2 < 8$, though passing tallies come as close to 8 as you like; and c still held v when the COMMIT ran. For any stored program run from a fresh start (counters set, every version 0, line 1, the ledger at 0) that raises the flag, the trace it executed contains a CHECK PCHSH c that passed, and the tally c held when it ran passes the seven integer facts, has every pair sampled and the pinned matrix symmetric and PSD, and has $S^2 \leq 8$ and $|S| \leq 2\sqrt{2}$. Conversely, from a fresh start in which a counter c holds the packing of a tally t, the three instructions CHECK PCHSH c, COMMIT PCHSH c, CERTIFY raise the flag if and only if t passes the check, and then the ledger reads exactly 3; if t fails, the trap goes up at the CHECK and no number of further steps raises the flag.

Proof. A raised flag from a clean start has the earned chain (Theorem 16.9, which holds for this machine as for the small one): a passing CHECK, then a COMMIT of the same claim with the counter's edition unchanged, then a CERTIFY. A passing CHECK of PCHSH at c means the checker says yes at c's value, which is the meaning by Theorem 20.4(2), and the bound is (4) of the same theorem. An unchanged edition means an unchanged value. For the converse, the CHECK passes exactly when the tally passes; if it does, COMMIT and CERTIFY find their facts and the three acts cost three; if it fails, the trap latch goes up, and a run takes no step once the trap is up (as in Theorem 16.17), so the flag never rises. □

Checked as `small_chsh_flag_implies_tsirelson`, `small_chsh_program_flag_implies_tsirelson`, `small_chsh_chain_certifies_iff`, `small_chsh_tally_certifies_iff`, `small_chsh_tally_certifies` and `small_chsh_tally_refused_forever` (SmallChshMachine.v); the first is printed in Section E. The strict bound on the passing tally's score is `nec_e_check_strict`, and `nec_e_check_sharp` gives passing tallies as close to 8 as you like (NecEChsh.v). That's the one thing the machine does with the check, as the section's opening said: a raised flag can stand on it. Five tallies, each a theorem with its counts written out, show the check isn't vacuous and where its edges are.

Proposition 20.6 (Five tallies)**.** Counts are given as (same, diff) for the pairs 00, 01, 10, 11. (4,1), (4,1), (4,1), (1,4) has $S = 12/5$: the check passes, the flag rises, the ledger reads 3. (17,3), (17,3), (17,3), (3,17) has $S = 14/5 = 2.8$, just under $2\sqrt{2}$, with $S^2 = 7.84$: it passes, the ledger reads 3. (9,1), (9,1), (9,1), (1,9) has $S = 16/5$ with $S^2 > 8$: refused forever, whatever follows. (1,0), (1,0), (1,0), (0,1), the PR box, has $S = 4$: refused forever. (1,0), (1,0), (1,0), (1,0), the all-ones classical plan, has $S = 2$: refused forever.

Checked as `small_chsh_demo_12_5_certifies`, `small_chsh_demo_14_5_certifies`, `small_chsh_demo_16_5_refused_forever`, `small_chsh_demo_pr_box_refused_forever` and `small_chsh_demo_all_ones_refused_forever` (SmallChshMachine.v). The last one is the check being stricter than the bound, as a named theorem: a perfectly legal classical play with $S = 2$ is turned away.

What the theorem doesn't say stays what it was. The tally is *placed*. Nobody plays it on the machine: the theorem starts from a counter that already holds the packing, and no instruction plays a round, so a program can put any code in a counter for free. There are no marginals. The pinned matrix is stricter than the quantum set, so failing says nothing about whether the play was quantum, and there is no margin for sampling noise at Tsirelson's point.

A tally stays free. Each count is a counter, a round is an INC on one of them, and INC costs nothing, so a pile of trials costs nothing to write down; the packing into one counter is a program's business, and in the theorem the packed counter is simply there at the start. What costs is the claim about the pile. Statistics are free; the claim about them is what costs.

So here, in one paragraph, is what the check guarantees. If it passes, then every question pair was sampled, the four empirical ratios satisfy three polynomial inequalities, hence the fixed five-by-five is PSD, hence S squared is at most eight. It does *not* guarantee that the tally came from honest play (a program can INC whatever counts it likes for nothing, including four rounds that put S at four). It does not guarantee any marginal is zero (no marginals are recorded and none are tested). It does not put the players' state behind the counts. What it does give is a quantum strategy with the same correlators: a passing tally's correlators have a PSD completion (`zero_marginal_implies_elliptope`), and every point with one is the correlator table of a strategy on two eight-level systems (`tr_representation`). And it fails the tally of every single plan, honest classical play included. It has no margin for sampling noise either. Honest play at Tsirelson's point aims at correlators of plus or minus one over root two, and there every pair of squared correlators the check adds up comes to exactly one, the edge of what it accepts. So an honest tally aimed at that point sits right on the line, and ordinary luck over a finite number of rounds pushes it over most of the time. I simulated it, drawing each round's answers at Tsirelson's correlators: the check fails 92 percent of the time at 10 rounds per question pair, 87 percent at 100, and about 85.5 percent from 1,000 to 100,000. Fair coins pass every time from 100 rounds per pair on, and fail about 2.5 percent of the time at 10. The limit has a closed form. To first order the largest singular value of the noisy correlator matrix is one plus the larger eigenvalue of a random symmetric two-by-two with independent Gaussian entries, and the check passes only when that eigenvalue is at most zero, which happens with probability $(2-\sqrt{2})/4$. So honest play at Tsirelson's point fails with probability $(2+\sqrt{2})/4 \approx 0.854$ in the long run. That's simulation and arithmetic. No theorem in the files says it.

So the files also have the check with a tolerance. For $\varepsilon = p/q$ it tests the same seven facts in integers with the 1 in the contraction inequalities replaced by $(1+\varepsilon)^2$. At $\varepsilon = 0$ it is the check above. A pass means the two-by-two correlator matrix has operator norm at most $1+\varepsilon$, and $S^2 \le 8(1+\varepsilon)^2$. A tally of 100 rounds per pair with correlators $0.72, 0.70, 0.72, -0.72$ fails the exact check and passes at $\varepsilon = 1/10$. Checked as `nec_e_tol_check_zero`, `nec_e_tol_check_sound`, `nec_e_tol_check_norm`, `nec_e_tol_check_bound` and `nec_e_tol_check_accepts_noisy_tsirelson` (NecEChsh.v). In the same simulation, with $\varepsilon = 3/\sqrt{n}$ for n rounds per pair, honest Tsirelson play fails it about five times in a hundred thousand. The price is in the

bound: a pass caps S at $2\sqrt{2}\,(1+\varepsilon)$, a little outside Tsirelson's number. The exact check remains the default. With p and q fixed for a machine instance, the tolerance check also runs as CHECK on the same versioned registers: its CHECK, COMMIT, CERTIFY chain certifies exactly when the integer tolerance test passes, pays three marks on success, and traps permanently on failure. From a clean start, every raised flag has that earned chain behind it, with the same register value at CHECK and COMMIT, and establishes the relaxed bound above. Checked as `small_chsh_tol_chain_iff`, `small_chsh_tol_chain_pays_three`, `small_chsh_tol_refused_forever` and `small_chsh_tol_flag_bound` (SmallChshTolerance.v). The Python host in `thiele_small/chsh.py` implements the same integer check and instruction chain. The check certifies a *shape* of the tally. What the shape means is a question for whoever supplied the rounds.

And notice which way the fence faces. The most practical use anyone has found for a Bell test, certified randomness, needs the opposite bound. Stefano Pironio and colleagues (2010) argued, and backed with an experiment, that a Bell violation, with settings the devices can't choose, certifies that the outputs are random, and a group of quantum cryptographers, Rotem Arnon-Friedman, Frédéric Dupuis, Omar Fawzi, Renato Renner and Thomas Vidick (2018), made the accounting tight with what they call entropy accumulation. A string printed in advance passes no such test, so it's the cleanest case I know, outside this book, of a check that can fail, separating output that was established from output that was printed. That needs S bounded from below, with an error probability. This check bounds S from above and fails every single plan, so it's a fence that keeps out the super-quantum and the fixed plan. As a nonlocality test it's no use. The deployed version exists, too: Gautam Kavuri and colleagues, physicists at NIST and the University of Colorado (2025), run a public randomness beacon whose outputs each trace back through a Bell test to a public record, the nearest working system I know to a certified claim written down for good. And records cut both ways there. Jonathan Barrett, Roger Colbeck and Adrian Kent, working on quantum cryptography (2013), pointed out that a device which keeps its own memory of earlier rounds can leak it through outputs it makes public later, so reusing a device can undo a guarantee the user thought was settled. A permanent record in the wrong box is an attack.

20.7 A rational stand-in for an irrational number

One number you'll see in the files: 5657 over 2000, which is 2.8285, and two root two is 2.82842... It's the four-decimal round-up, chosen so that rational code can test "above the Tsirelson bound" without square roots, and it's safe because 5657 squared is 32,001,649 and eight times 2000 squared is 32,000,000, so the fraction sits above root eight by about seven hundred-thousandths (`rational_tsirelson_boun`

d). It's used in a few rational theorems. It is *not* used in the integer check, and the real-number theorems use root eight.

20.8 Two more tests, as mathematics

The second rung: a nine-by-nine table on I, the four questions, and the four Alice-Bob *products*, with the correlators and five extra free numbers for three- and four-body entries. Its two four-body entries are tied, one the negative of the other. That tie is a constraint, $\langle \{A_0, A_1\} \otimes \{B_0, B_1\} \rangle = 0$: the zero entries of the first rung don't imply it, and a classical strategy breaks it (`q1ab_tie_is_a_constraint`, QuantumPartitionPSD_1AB.v). So the second rung is level 1+AB on that slice, and that's what I mean by level 1+AB everywhere in this book. Its PSD is again equivalent to a contraction-like condition (`q1ab_psd_iff_column_contractive`). The check for it proves something stronger, that the matrix is positive definite, by a cascade of two Schur steps down to a four-by-four that Sylvester's criterion closes. Sylvester's name on a test about matrices fits in a way I enjoy, because the word *matrix* is his. In 1850, in the *Philosophical Magazine*, the English mathematician James Joseph Sylvester used it for an "oblong arrangement of terms" out of which you cut square blocks and take their determinants, and *matrix* was already an old word for the place something grows from. That's exactly what the test does: it cuts the top-left square blocks out of the big grid, one row deep, two, three, four (the leading minors), and asks each one's determinant to be positive. Was he the first to state the test? The name doesn't settle that; names stick to people for all kinds of reasons, and I'd treat this one as a lead. And a fact that's easy to list without noticing what it means: with all five extra numbers at zero, the condition certifies a strict *subset of the classical set* (the ball where the four correlators' squares sum to at most one), forcing S at most two (`q1ab_check_at_gzero_implies_classical_bound`); it can't certify anything above the classical line. To get a point above two you have to supply the extras (`q1ab_certifies_a_superclassical_chsh_point`, S equals 2.4 with one extra at minus one half). Appendix F has the algebra.

The two sizes are one test. A single positive-semidefinite predicate takes the size as a parameter, and the five-by-five and the nine-by-nine are it at two sizes.

Definition 20.3 (The size-generic PSD test)**.** A *claim* is a top index n and a matrix M indexed by naturals. It is *realizable* when $M_{ij} = M_{ji}$ for all $i, j \leq n$ and $\sum_{i \leq n} \sum_{j \leq n} v_i M_{ij} v_j \geq 0$ for every real vector v. The CHSH claim of E is top index 4 with the matrix $M(E)$ of Definition 20.1; the level-1+AB claim uses top index 8 and the nine-by-nine matrix.

Corollary 20.7 (The level-1 test is the size-five case)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$: the three contraction inequalities hold if and only if the CHSH claim of E is realizable. The same holds at size nine for the level-1+AB contraction condition and its claim.

Proof. Realizability at top index 4 unfolds to symmetry and PSD of the five-by-five, by computing the finite sums, and that is equivalent to contraction by Corollary F.13. At size nine the same unfolding meets Theorem F.19. Nothing new is proved about the matrices; the point is that one predicate covers both. □

Checked as `column_contractive_iff_general_realizable` (GenRealizability.v) and `column_contractive_q1ab_iff_general_realizable`.

The third thing: the *elliptope*, the set of correlators for which *some* choice of the two missing entries (A0 with A1, B0 with B1) makes the five-by-five PSD. Every one of the sixteen plans is inside it (`deterministic_strategy_elliptope`), mixtures stay inside (`elliptope_convex`), every point inside obeys $S^2 \leq 8$ (`elliptope_tsirelson`), Tsirelson's point is inside and reaches it (`nec_e_pinned_tsirelson_tight`, NecEChsh.v), the PR box is out (`pr_box_not_elliptope`), and it's strictly bigger than the classical set (`beyond_classical_elliptope`). There's a decider for it with a soundness proof, in two branches: a strict Sylvester check with the completion supplied, and a rational factorisation certificate of the completed matrix, checked in integers. What's proved is that a pass puts the correlators inside the set and that no choice of inputs gets the PR box through (`elliptope_check_full_sound`, `elliptope_full_gate_never_accepts_pr_box`), and, by computation, that some named points pass, the all-ones plan and a point that the arc-sine boundary puts exactly on the quantum edge (`am_pythagorean_boundary`) among them (`gate_accepts_all_ones`, `gate_accepts_pythagorean_boundary`). The file's header also argues that every point with a rational PSD completion has such a certificate; that is a comment, and no theorem states it. It's a decider with a soundness proof, as mathematics; none of the small machine's properties is the elliptope check.

20.9 The quantum set, checked

Three descriptions of the same set have been running through this part: the tables some quantum strategy can produce, the tables whose five-by-five matrix has a PSD completion, and the tables inside Landau and Masanes's curved edge. They are the same set, and that's checked, for strategies on finitely many levels. Here's how each piece goes, because the constructions are the part worth seeing.

Quantum tables are inside the completion set. A strategy's correlator is the inner product of two unit vectors: Alice's observable applied to the shared state, and Bob's

applied to it (`qs_corr_dot`). Four unit vectors always have a PSD table of inner products, so every quantum table has a completion, and Tsirelson's bound follows from that: $S^2 \leq 8$ for every strategy (`qs_tsirelson`, QuantumStrategies.v), and its level-one moment matrix passes the PSD test with the marginals in (`qs_npa_psd`). Quantum mechanics uses complex amplitudes, and the complex case goes through by a plain trick: a complex vector of length n is a real vector of length $2n$, its real parts followed by its imaginary parts, and the real part of a complex inner product is the ordinary inner product of those longer real vectors. So the real argument runs unchanged (`qc_tsirelson` and `qc_npa_psd`, QuantumStrategiesComplex.v). The two-qubit strategy from the start of this part reaches $2\sqrt{2}$ in both readings (`qs_tsirelson_reached`, `qc_tsirelson_reached`).

The completion set is inside the quantum tables. This is Tsirelson's representation, and the construction is his. Take a table with a PSD completion. A PSD matrix is a table of inner products of vectors, always: peel off one row at a time, and a zero on the diagonal forces its whole row to zero (`tr_gram`). So the completion hands over four unit vectors in four dimensions, a_0, a_1 for Alice and b_0, b_1 for Bob, with $a_x \cdot b_y$ equal to the table's entries. Now the trick. There are four real eight-by-eight matrices, built from the two-by-two ones that swap two entries or flip the sign of one, that each square to the identity and anticommute in pairs: multiply any two of them in either order and the answers differ only in sign (`tr_gamma_clifford`, checked by computation). Use a vector's four components to mix those four matrices, and you get an observable that squares to the identity, a legitimate measurement. Give Alice and Bob a maximally entangled pair of eight-level systems, and the correlator of two such observables comes out as exactly the inner product of their vectors (`tr_corr_inner`). So the strategy built from the four vectors produces the table (`tr_representation` for real amplitudes and `tr_representation_complex` for complex, TsirelsonRepresentation.v). Eight levels on each side is always enough.

The curved edge is the same set again. Each correlator is the cosine of an angle, the angle between one of Alice's vectors and one of Bob's. Walk the loop a_0, b_0, a_1, b_1 and back to a_0: the angle across any one leg can't exceed the sum of the other three, because angles on a sphere obey the triangle inequality (`am_sphere_triangle`, ArcsineBoundary.v). Flip one vector to its opposite and you get the other four inequalities, with 2π in them. Written in arcsines, those eight inequalities are Landau and Masanes's four. Going the other way, any four angles that pass them can be met by actual vectors: put Bob's two at the right angle to each other, build each of Alice's from the two angles it has to make with them, and check that the determinant that says such a triangle exists comes out nonnegative. So a table has a PSD completion exactly when its arcsines pass the four inequalities (`am_arcsine`), and those are exactly the quantum tables (`am_quantum_arcsine`). The point $(3/5, 4/5, 4/5, -3/5)$

sits on the edge, because $\arcsin\frac{3}{5} + \arcsin\frac{4}{5} = \frac{\pi}{2}$ by the 3-4-5 triangle (`am_pythagorean_boundary`), and so does Tsirelson's point (`am_tsirelson_boundary`).

20.10 What isn't proved

One step on paper, and then nothing about dimension. Quantum mechanics also allows systems with infinitely many levels, and the checks don't need finitely many. Whatever the dimension, a shared state v gives an expectation on the operators the players measure with: $x \mapsto \mathrm{Re}\,\langle v, x\,v\rangle$. That one line is the step on paper. The proofs use only what that expectation always has: you can add, multiply, scale and take adjoints, the product spreads over sums, and the expectation of x^*x is never negative. Each measurement is its own adjoint with the expectation of its square at most one, which covers unsharp measurements too, and Alice's commute with Bob's. That last demand is weaker than putting them on two separate systems, so the result covers the wider, commuting model as well. On that footing Tsirelson's sum of squares goes through, and the CHSH value is at most $2\sqrt{2}$ (`ta_tsirelson`). The same nonnegative form, with each diagonal entry raised to one, completes the correlator table to a PSD matrix (`ta_realizable`), so the four arcsine inequalities hold (`ta_arcsine`), and with a unit the full first-level moment matrix, marginals and all, is PSD (`ta_npa_psd`, all in TsirelsonAlgebraic.v). The other direction needs no help from infinity: every point inside the arcsine edge is reached on two eight-level systems.

Also: the finite-mixture theorem (`fine_theorem` in the source) proves only one direction, that a finite local model satisfies the bound, and its converse as stated is false: a table with $S = 0$ that no local model produces (`nec_e_fine_converse_false`). Fine's own theorem, in the form that speaks of correlations, is proved separately as an equivalence: a table of four correlators comes from a local model exactly when each is between -1 and 1 and all eight CHSH inequalities hold (`nec_e_fine_iff`, NecEFine.v). And one more thing about tallies, which the guarantee paragraph above already used: a free tally is not thereby classical (the four rounds with S equal to four). The correlators (one, one, one, minus one) are the PR box, which no classical play produces; the classical vertex this section used, which fails the check like every plan, is (one, one, one, one).

That's the game, the number, the two walls, and the check. Last of the machine sections: what all this costs against physics.

21

The counter against physics

Section 4 did the physics argument once: on a finite machine a permanent flag merges states, Landauer's principle (a named premise) prices merges, and the toll follows. I'm not going to do it again. This section is about what a machine's counter, the small machine's μ included, has to do with any of that, and the answer is: less than the word "Landauer" in a file name suggests, and I want you to hear the exact amount from me.

21.1 The calibration is a unit conversion

There's a function in the source that multiplies the counter by a chosen energy per mark, and a theorem that a state with one mark, at the scale kT ln 2 per mark, carries kT ln 2 of energy (`calibrated_mu_landauer_energy`, proving the predicate `mu_landauer_calibration` for every k and T). Read that for exactly what it is: a unit conversion, true by definition, with the scale as a free input. Nothing in it picks the scale. The theorem would be just as true with the scale set to a dollar a mark. I'd say that out loud, so the name "Landauer" in it does no work. What *would* connect marks to joules is the premise in Section 4, `landauer_heat`, and that one is an assumption with a name that nothing proves. And even as a scale, $k_B T \ln 2$ a mark is the floor for one particular squeeze, a halving. Corollary 4.9's floor for a flip that brings k states into a room already holding m, starting from the even spread on those $m + k$ states, is $k_B T \ln \frac{m+k}{m}$, and for one newcomer into a big room that's far below a halving, roughly $k_B T / m$. Price every mark at $k_B T \ln 2$ and you can overstate the floor of a single narrow flip by a factor that grows with m.

Remember the constant from Boltzmann's grave in Part II, the exchange rate between a count and heat? It has a newer chapter, and it makes the point sharper than I can. The k_B in $k_B T \ln 2$ turns a temperature into an energy, and for most of its history it was a measured number, with error bars, pinned down by careful experiments. Since 20 May 2019 it isn't. That day the international system of units was redefined so that Boltzmann's constant is exactly 1.380649×10^{-23} joules per kelvin, by definition, and the kelvin is whatever makes that true. So the conversion from temperature to energy has been a convention since then, in the strictest sense: nobody can measure it wrong any more. What's still physics is everything around it,

whether wiping a bit at a temperature really sheds that much heat, which is what the Lyon bead was measuring. The calibration theorem is the same kind of move as that redefinition. It fixes a conversion, and the physics, if there is any, is somewhere else.

Definition 21.1 (Energy at a scale)**.** For a real number c (joules per mark) and a state s, the energy of s at scale c is $\mu(s) \cdot c$. The *Landauer calibration* at k_B and T is the statement that every state with $\mu(s) = 1$ has energy $k_B T \ln 2$ at scale $k_B T \ln 2$.

Theorem 21.1 (The calibration is a unit conversion)**.** (1) For all real k_B and T, the Landauer calibration at k_B and T holds. (2) For any counter, there are two real-valued functions of the state that give 1 and 2 on every state with one mark, so the counter alone fixes no energy value.

Proof. (1) Substitute $\mu(s) = 1$: $1 \cdot k_B T \ln 2 = k_B T \ln 2$, by ring arithmetic, for any k_B and T, positive or not. (2) Take the two functions to be the energy at scales 1 and 2. At $\mu(s) = 1$ they give 1 and 2. □

Checked as `calibrated_mu_landauer_energy` (PricingPhysicsAudit.v), which proves `mu_landauer_calibration` (PricingPhysicsTarget.v) for all k_B and T, and `mu_has_no_intrinsic_joule_value` (PricingPhysicsAudit.v). The physics is the premise of Section 4, restated here once in the form the next theorems test. In words first: a *dissipation* hands each instruction a whole number of heat units, each unit worth kT ln 2. The floor premise says an instruction that takes at least one state where some yes-or-no property is false to a state where it's true, never sends a true one back to false, and sends two different states to one, dissipates at least one unit. That's a different statement from Landauer's principle as Premise 4.5 has it. That one depends on the spread over states, $Q \geq k_B T \ln 2 \cdot \Delta H$, and gives nothing for a known state. This one gives every such instruction a floor of one whole unit, whatever the spread. To get the second from the first you'd need more, at the least a spread with a full bit of uncertainty removed at every such step, and the known-state theorem and the narrow squeeze of Section 4 show that isn't automatic. So I give it its own name, *structural dissipation pricing*. The calibration says no instruction dissipates more units than the counter charges it. One word about that word: "dissipation" here means heat handed to the surroundings, in the sense Section 4 took from Sagawa: a bill for where the forgotten bit goes. It has nothing to do with waste, or with doing the job badly. Two premises, nothing proved about either, and the only question is whether they can sit in the same room:

Premise 21.2 (Structural dissipation pricing)**.** A *dissipation* is a function δ from instructions to naturals, counted in units of $k_B T \ln 2$. The *floor*, a hypothesis separate from Premise 4.5 of Section 4: every instruction that collapses a yes-or-no property has $\delta(i) \geq 1$, where *i collapses P* when some state with P false is sent to a state with P true, every state with P true is sent to one with P true, and some two different states are sent to the same state. The *calibration*: $\delta(i) \leq \text{cost}(i)$ for every instruction. Neither is proved anywhere; the theorems below ask whether they can hold together.

The last clause of the floor is there because the first two don't make a step forget anything on a machine with infinitely many states. INC A sends a state with $A = 0$ to one with $A = 1$ and keeps every positive A positive, so it raises "$A \geq 1$" one way, and it is one-to-one. Any one-to-one map that misses some state does the same to the property "is in my image". Such a step can be run backward, and Landauer's principle charges it nothing; a floor read without the last clause would charge it a whole unit. On a finite machine the clause comes for free. A step that sends one false state into the true ones and keeps every true state true puts one more state into the true set than the set holds, so two of them land on the same state. Checked as `nec_f_one_way_injective` and `nec_f_finite_one_way_merges` (NecFPhysics.v).

21.2 The premises have to be able to hold together

Here's the one piece of this section that I think is good physics discipline, and it's a consistency test. Suppose you want to claim the counter tracks dissipation. You need two premises: that every step you've chosen to count dissipates at least one unit, and an upper calibration that bounds dissipation by the charge. Can those two hold at once? There's a theorem saying exactly when (`calibrated_model_exists_iff_positive_cost`): a model satisfying both exists precisely when every operation you've chosen to count (the statement below calls those *eligible*) costs at least one. I should say how cheap that is. The model the proof builds takes the dissipation to be the cost itself, so the theorem shows the two premises don't contradict each other. It doesn't show that the counter tracks any dissipation. Still, it does bite on a schedule: if you count a zero-cost step, no such model can exist. One place the checked files show the premises holding is a one-bit reset, where resetting costs one and idling costs nothing (`bit_model_has_satisfiable_calibration`); the small machine's flag-raising steps, below, are another.

Now point the test at the small machine of Section 16, counting every step that collapses some yes-or-no property, and it fails. Read without the last clause of the floor, it would fail at a step that forgets nothing: INC A raises "$A \geq 1$" one way, is one-to-one, and costs zero (`nec_f_small_one_way_pair_fails`, NecFPhysics.v). With the clause in place the failure sits at a merge. Its free instructions merge states

(the last item of the list at the end of that chapter has the pair: two cores that one DEC sends to the same core, for nothing), so DEC A 2, which collapses the property "A is zero or the line number is 2" by sending a state where it's false and a state where it's true to the same state, costs zero there, and (1) below says no dissipation can charge it one and stay under its cost. That's checked as `nec_f_small_premise_pair_fails`, with the free merge itself as `nec_f_small_free_collapse_is_merge` (NecFPhysics.v). The machine prices the certifying chain and leaves its other merges free, so the premise pair holds for the steps the toll is about (`nec_f_small_premise_pair_on_flag`) and fails for the whole instruction set. That's the honest shape.

Theorem 21.2 (When the premises can hold together)**.** (1) For any set of operations, any predicate "eligible" on them, and any cost, a dissipation with $\delta(i) \geq 1$ on every eligible i and $\delta(i) \leq \mathrm{cost}(i)$ on every i exists if and only if every eligible operation costs at least one. (2) On a one-bit machine with a reset of cost 1 and an idle of cost 0, taking "eligible" to mean "sends two different states to one", such a dissipation exists.

Proof. (1) If such a δ exists, $1 \leq \delta(i) \leq \mathrm{cost}(i)$. Conversely, take $\delta = \mathrm{cost}$. (2) The reset is the only operation that sends two states to one, and it costs 1; apply (1). □

Checked as `calibrated_model_exists_iff_positive_cost`, with the definition `calibrated_positive_model`, and `bit_model_has_satisfiable_calibration` (ObservationPolicy.v).

Three statements are in play, and each points its own way. Energy at a scale, $\mu \cdot c$, is a conversion and points nowhere. The calibration, $\delta(i) \leq \mathrm{cost}(i)$, puts the charge above the dissipation: an upper bound. Landauer's principle puts the heat above an entropy loss: a lower bound. The theorem above says the assumptions can hold together. That's compatibility, and it isn't evidence. It doesn't make the ledger the least energy anything needs, and it doesn't make it the heat anything gives off.

But now the history escape from Section 4, pointed at a machine. Keep the previous state in a history list (push it onto the front each step; below, $s :: h$ is the list h with s put on the front) and every fixed-instruction step becomes injective, while the step you can see stays the same; the line number can collapse to one place while the bigger state keeps its past. The free merge, lifted to a machine with history, isn't a merge. So "this step merged, therefore it dissipated" can't be said of the visible step alone. It's a sentence about the whole state including whatever history is retained, and a physical model has to say what's retained. The permanent-certificate theorem in Section 4 supplies that for the certifying step on finite memory. For the rest of a machine's steps it's unsaid.

Theorem 21.3 (Keeping history makes every step injective)**.** Let the history step send a pair (s, h), a state and a list of past states, to $(\text{apply}(s, i), s :: h)$. For any step function, the history step with a fixed instruction is injective, and its first component is the original step.

Proof. Undoing the history step, by popping the list, returns the old pair, so two pairs with the same image are equal. □

Checked as `retained_history_step_injective` (ObservationPolicy.v).

21.3 The calorimeter protocol, as a specification

Here is how one *would* measure this, and it's here because it makes the boundary exact. Take a two-state register with energies zero and a gap, a population p in the excited state, and the simplest possible rate equation for how p changes in one time step (a rate for climbing into the excited state, a rate for falling out of it, and the length of the step). For the canonical reset (population one half to zero, with the rates chosen so the equation closes), the heat delivered to the bath is exactly half the gap (`canonical_reset_heat_exact`). Choose the gap to be twice kT ln 2 and that heat is exactly kT ln 2 (`selected_gap_gives_landauer_heat`).

Read that twice, because the temptation is to read it as a derivation of Landauer's number and it's the opposite. The gap was *chosen* to make the heat come out as Landauer's number. The same populations and rates with half that gap give half the Landauer value, and there's a theorem exhibiting identical population dynamics with different heat (`master_equation_does_not_fix_heat_scale`). The rate equation fixes the *shape*; the energy scale is a selected Hamiltonian (the physicist's name for the rule that says what energy each state has), and selecting it is not deriving it.

The canonical reset is a bookkeeping toy, and its rates show it. A register that trades heat with a bath at temperature T has rates in *detailed balance*: the rate up over the rate down is $e^{-\Delta/k_BT}$, which at the gap $2k_BT\ln 2$ is $1/4$. The canonical rates are 0 and 1, and the factor $e^{-\Delta/k_BT}$ is never 0, so no gap and no temperature put them in that ratio (`canonical_rates_break_detailed_balance`). With rates that are in it, a relaxation at a fixed gap moves toward the equilibrium population $1/(1+e^{\Delta/k_BT})$ and, with steps short enough not to overshoot, never goes below it (`detailed_balance_settles_at_gibbs`, `detailed_balance_never_empties`). At the gap $2k_BT\ln 2$ that population is $1/5$ (`landauer_gap_settles_at_one_fifth`). The toy has no work term either; nothing in it pays for the squeeze. Part (4) of the theorem below says what that costs: at the gap $k_BT\ln 2$ the toy books less heat than Landauer's bound asks for the same squeeze from one half to zero, so the toy has no second law.

Definition 21.3 (The two-state protocol)**.** For a gap Δ, the register's mean energy at excited population p is $\Delta \cdot p$. One step of the rate equation is

$$p' = p + \delta t \left(k_{01}(1-p) - k_{10}\, p\right),$$

and the heat to the bath over a step from p to p' is defined as $\Delta p - \Delta p'$. The canonical reset takes $p = \frac{1}{2}$, $\delta t = 1$, $k_{01} = 0$, $k_{10} = 1$, ending at $p' = 0$, and counts one logical event.

Theorem 21.4 (The heat is fixed by the chosen gap)**.** (1) The canonical numbers satisfy the rate equation. (2) For every Δ, the canonical reset's heat is $\Delta/2$. (3) For all k_B, T, the gap $\Delta = 2k_BT\ln 2$ gives heat $k_BT\ln 2$. (4) If $k_B > 0$ and $T > 0$, the gap $k_BT\ln 2$ gives heat strictly below $k_BT\ln 2$. (5) There are two different gaps for which the same canonical populations satisfy the rate equation and the heats differ.

Proof. All five are field arithmetic on the definitions: $\frac{1}{2} + 1\cdot(0-\frac{1}{2}) = 0$; $\Delta\cdot\frac{1}{2} - \Delta\cdot 0 = \Delta/2$; and $\ln 2 > 0$ for (4). For (5) take gaps 1 and 2. □

Checked as `canonical_reset_satisfies_master_equation`, `canonical_reset_heat_exact`, `selected_gap_gives_landauer_heat`, `canonical_reset_heat_below_landauer_at_small_gap` (given $k_B > 0$ and $T > 0$) and `master_equation_does_not_fix_heat_scale` (CalorimeterProtocol.v), with the definitions in CalorimeterProtocolTarget.v.

A reset that pays its way is driven. Raise the gap a little, let the register settle with rates in detailed balance, raise it again, and so on, starting from a gap of zero, where the two states are alike and the population is one half. Raising the gap while the population holds is work done on the register; settling at the new gap is heat to the bath. That's the textbook way to wipe a bit slowly, and here it's checked.

Definition 21.4 (The driven reset)**.** Write kT for k_BT. At gap Δ the equilibrium excited population is $\pi(\Delta) = 1/(1+e^{\Delta/kT})$ and the register's free energy is $F(\Delta) = -kT\ln(1+e^{-\Delta/kT})$. Rates satisfy *detailed balance* at gap Δ when $k_{01} = k_{10}\, e^{-\Delta/kT}$. A *driven reset* along a schedule of gaps $g_0, g_1, \ldots, g_n$ takes, at step m, the gap from g_m to g_{m+1} with the population held at $\pi(g_m)$, which is work $\pi(g_m)(g_{m+1} - g_m)$, and then lets the register settle to $\pi(g_{m+1})$, which is heat $g_{m+1}\big(\pi(g_m) - \pi(g_{m+1})\big)$ to the bath. The reset's work and heat are the sums over its steps. The uniform schedule from 0 to D in N steps has $g_m = Dm/N$.

Theorem 21.5 (A driven reset pays $kT \ln 2$ in work)**.** (1) With rates in detailed balance at gap Δ, one step of the rate equation takes p to $\pi(\Delta) + \big(1 - \delta t\,(k_{01} + k_{10})\big)\big(p - \pi(\Delta)\big)$; when $\delta t\,(k_{01} + k_{10}) = 1$ it lands on $\pi(\Delta)$ from any p. (2) For any schedule, work minus heat is $g_n \pi(g_n) - g_0 \pi(g_0)$, the change in the register's mean energy. (3) If $kT > 0$ and the schedule never falls, the work is at least $F(g_n) - F(g_0)$; if also every step raises the gap by at most h, the work is at most $F(g_n) - F(g_0) + h\big(\pi(g_0) - \pi(g_n)\big)$. (4) If $kT > 0$, $D \geq 0$ and $N \geq 1$, the uniform reset from 0 to D in N steps does work between $kT \ln 2 - kTe^{-D/kT}$ and $kT \ln 2 + D/(2N)$, and leaves an excited population of at most $e^{-D/kT}$.

Proof. (1) Under detailed balance $k_{01} = (k_{01} + k_{10})\,\pi(\Delta)$, so the step is linear in $p - \pi(\Delta)$ with that factor. (2) The terms telescope. (3) The slope of F is π, which falls as the gap rises, so the mean value theorem gives $\pi(b)(b - a) \leq F(b) - F(a) \leq \pi(a)(b - a)$ for $a \leq b$; add over the steps. (4) $F(0) = -kT \ln 2$, $\pi(0) = \frac{1}{2}$, $\ln(1 + x) \leq x$ for $x \geq 0$, and $\pi(D) \leq e^{-D/kT}$; put these into (3) with $h = D/N$. □

Checked as `detailed_balance_settles_at_gibbs`, `detailed_balance_thermalizing_step`, `driven_first_law`, `driven_work_second_law`, `driven_work_near_free_energy` and `driven_reset_work_window` (CalorimeterProtocol.v), with the definitions in CalorimeterProtocolTarget.v.

Here nobody picked the gap to make the number come out. Take D large and N much larger than D: the work closes in on $kT \ln 2$ from both sides, and the population left in the excited state goes to zero. The bit is wiped, and Landauer's number is the work it took. The temperature gets in through detailed balance, which is the thermal premise the toy didn't have. It's still a model. The settling step is the one part (1) describes, and the work and heat are the standard bookkeeping for a slowly driven two-state system.

What ties it to a machine: the protocol counts one logical event (the file defines that count as the constant one), and one is what the small machine's CERTIFY charges, the last of the three paid acts behind a raised flag (Section 16). That's a match of two numbers, both one by choice, and nothing more. There is no theorem mapping the protocol's two register states to any machine state. The file's "heat to the bath" is the register's mean energy before minus after, so it also assumes the bath receives what the register loses, and no Coq says that. So a device test would have to measure the gap, the populations, the rates, and the bath heat independently; separately establish that the device's rates are in detailed balance (that the up and down rates stand in the ratio the temperature dictates), which the driven reset assumes and the canonical one breaks; and separately establish that the device's physical reset *is* the machine's certify event. The driven reset is an executable experimental specification.

The canonical reset is a bookkeeping toy. Neither is a measured device, and neither is an unconditional conversion of marks to joules, and anyone who tells you otherwise is reading a file name.

21.4 What permanence adds to Landauer: nothing new, said plainly

Here's a fair question: is there a prediction the permanent-certificate theorems make that Landauer's principle alone doesn't? No, and the reason is short. The smallest case, one certified state and one flipping in, is "set a bit to one whatever it held," which is the reset Landauer described and Bérut measured; Landauer's own name for it is "restore to one," and Section 3 put it beside the toll at the start. The general bound, kT ln((m plus k) over m) for squeezing m plus k equally likely states into m, is what Landauer's principle gives for that squeeze, at kT ln 2 for each bit removed. What the permanence theorems add is the *derivation*: which premises about a machine (finite, permanent) make the certifying step a reset, so that Landauer applies to it. That's a result about the machine. Physics gets nothing new from it, and that's the right size for it.

21.5 What this section is

A calibration that's a unit conversion; a pair of premises, consistent on the steps the toll is about and inconsistent on the whole instruction set; a history that makes any step injective; and a measurement protocol that's a specification with a selected scale. The physics in this book is in Section 4, as a named premise with its critics named. Here there's accounting, and I've tried to call every line by its right name. That's the end of the machine. What's left is what it took to check any of this, and where the checking stops.

v

Checking it, and its edge

22

What Coq is, and why

Coq is a proof assistant, which is a phrase I had to go and look up. Underneath the phrase it's a programming language, but one whose type system is strict enough to do a thing ordinary languages can't: you write a proof down as a program, and the compiler refuses it unless the program genuinely matches the statement you claimed. You can't talk it into a maybe.

The name is a joke three ways at once, and it's worth knowing before you say it out loud in English. *Coq* is French for rooster. It's also a nod to Thierry Coquand, a French computer scientist and mathematician, at the University of Gothenburg as of 2026, who worked out the logic it started from with Gérard Huet, a French computer scientist at INRIA, and to that logic's initials, CoC, for the Calculus of Constructions; and there's a French habit of naming research tools after animals. The English slang reading did what you'd guess. By the development team's own account, some people, women students especially, ended up never learning the tool because of its name. So in March 2025 it got a new one, the Rocq Prover, after Rocquencourt, the Inria site where it was first built, and after the Roc, the giant bird of legend, so the bird stayed. Everything in this book was checked with Coq 8.18, from before the rename, so that's the name I use. The rooster changes nothing about what got checked.

22.1 Types as claims

Here's the idea, and it's one idea. In a normal language a type says what kind of value a variable holds: in Python, "x is an int" means x is a whole number. In Coq, types are *claims*. The type "n is greater than zero" is the type of *proofs* that n is positive. A function from "n is greater than zero" to "n plus one is greater than zero" is a proof that if n is positive, n plus one is too, and the compiler checks that the function really has that type, the way it would check that a function from integers to strings really returns strings. This correspondence between proofs and programs has a name, Curry-Howard, after two people who noticed it separately decades apart, and Coq's foundation, the Calculus of Inductive Constructions, is a type system built to exploit it. Coquand worked it out with Christine Paulin-Mohring, a logician and computer scientist best known for developing Coq, on top of the Calculus of

Constructions he and Huet had made; their paper on it was given in 1988 and printed in 1990, so the late 1980s is as close as I'd date it. The "inductive" part means you define your own recursive types (numbers, lists, trees, machine states) inside the system. The numbers, lists and states this book uses are all defined that way, inside the system, so the foundation is explicit; nothing is smuggled.

"Decades apart" undersells it, and the gap is worth a side trip. Haskell Curry, an American logician at Penn State best known for combinatory logic, noticed in 1934 that the type of a function from A to B reads like the claim "A implies B," and that the types his functions could have lined up with the claims you can prove; he and Robert Feys, a Belgian logician at Leuven, pushed it further in 1958. William Howard, a proof theorist at the University of Illinois at Chicago, wrote the sharper version in 1969, and then it went around as a photocopied manuscript until 1980, when it was finally printed in a collection of essays for Curry. (The Edinburgh computer scientist Philip Wadler's 2015 survey, "Propositions as Types," tells this story.) So the sharpest statement of the idea under the tool this book is checked with spent its first eleven years as a stack of photocopies. Is that the bottom? No. Reading a proof of "if A then B" as a method that turns any proof of A into a proof of B is older still: the intuitionists had it in the 1920s and 1930s, Brouwer, Heyting and Kolmogorov, which is why people also call it the BHK interpretation. Where it really starts I can't tell you. It changes nothing in what the checker checks. It changes how I'd tell the story: nobody sat down one day and invented "proofs are programs." It kept getting noticed, by people coming from different directions, until somebody wrote it down plainly.

The compiler checks whether the types line up. If your proof has a hole, it rejects the proof, unless you write "trust me" (the keyword is Admitted), and then it marks the theorem as unproved. But here's the part that matters: the compiler checks that the proof matches the *statement*. It does not check that the statement says what you *meant*. Hand it a statement that means nothing and it'll prove it for you, three characters, no complaint. So a proof being accepted is not the same as the thing being true. It means this proof matches this statement, and whether that's the right statement is on me. The machine can't tell. The point said a short version of this and I'm saying it again because every section since has leaned on it: the checked names in this book are a promise that the *proof* is there, and the sentences around them are my claim about what the statement *says*, and the second can be wrong while the first is fine, which is why so many sections of this book stop to say exactly what a statement's hypotheses are.

22.2 Why I used it

Not to make anything true. I used it because I don't trust my own eye to catch a gap in an argument I want to believe. I spent months convinced by my own reasoning, pushed the proof through, and watched a key step come apart in my hands. The machine caught where I'd fooled myself. That's what it's for. Gap-catcher. Not the proof. The proof is the argument, and an argument has to close in a way a person can follow, or it hasn't closed at all, which is why this book is written in prose. A list of theorem names can't close an argument for anyone.

Two precedents show that a proof assistant is a serious tool. Neither of them is a toy. CompCert, Xavier Leroy's C compiler, written and proved correct in Coq: the assembly it emits behaves as the source does, for programs with defined behaviour. And Georges Gonthier's 2005 proof of the four-colour theorem, a century-old problem whose first proof (in 1976) needed a computer to check fewer than two thousand configurations, by its own announcement's count, and was distrusted for exactly that reason. Gonthier redid it in Coq, working from a later version of the proof by Neil Robertson, Daniel Sanders, Paul Seymour and Robin Thomas (1997), so that the *checker* was the small, auditable thing and the cases were the program. That's de Bruijn's criterion applied to mathematics itself: the thing you have to trust is the small kernel. The big proof only has to get past it. The name was given by Henk

Barendregt, a Dutch logician known for his work on the lambda calculus and type theory, after Nicolaas de Bruijn, whose Automath project, begun in 1967, kept the part of a proof checker you have to trust small: a system meets the criterion when its proofs can be checked by a small independent program that a person could read.

The 1976 proof deserves its own side trip. Kenneth Appel and Wolfgang Haken did it at the University of Illinois, and the mathematics department changed the slogan on its postage meter to "Four colors suffice." The slogan it replaced had been on the department's mail since soon after 1963, and it said that $2^{11213} - 1$ is prime, a prime Don Gillies, a professor at the same university who helped design its ILLIAC II computer, had found in 1963 while checking that machine out. Think about that for a second. For years on end a mathematics department stamped its mail with results nobody could have checked by hand, first one and then the other, and both were right. So was the distrust of the 1976 proof wrong? I don't think so. The worry was never that the computer lied. It was that nobody could read what had been checked, and a check you can't look at is hard to lean on. Gonthier's answer was de Bruijn's: make the part you have to read small enough to read.

22.3 What "zero Admitted" means

"Zero Admitted" is literal. Nowhere in the proof tree did I write "trust me" and walk on. Every step the checker could see, it saw, end to end, with no "and the rest is obvious." It's a hygiene fact about the gap-catcher. It passes no verdict on the argument: it says I didn't let myself skip a step where the machine was watching. It does not say the statements are the right statements.

What it doesn't mean, listed. It doesn't mean the theorems prove physical reality; they prove things about the machine as formalised, and whether that describes the world is answered by experiments (Section 4, Section 21). It doesn't mean Coq's own foundation is proved consistent; it's believed consistent, and that can't be proved from inside, by Gödel (Section 13), so the foundation is trusted, and "trusted" has substance: it's the same kernel CompCert and the four-colour proof stand on, and a bug in it would take all of them down together, which is a kind of safety in numbers. And it doesn't mean no axioms: the proofs use a handful of standard-library assumptions, and the assumption audit names them: functional extensionality on dependent functions (two functions that agree everywhere are equal); two decidability principles that come with the standard library's real numbers, used wherever real analysis is, for example in the matrix proofs behind the game check and in the real-valued results of Sections 4, 12 and 21; and, in some theorems, the law of excluded middle (every proposition is true or false). The count, from the full audit: the full-corpus probe covers 5,997 named theorems across 341 files: 5,198

close under the global Coq context outright, and 799 use only Coq standard-library assumptions. Zero project-local axioms appear in any of the 5,997 dependency trees. The small machine's files, under minimal/, are inside that probe too, and every one of their 1,417 probed theorems comes back closed under Coq's own rules, with no axiom at all. Among them are the small machine, its version for any property language, the simpler host, the definition of Thiele-complete and its window theorem, the host machine with a register for every whole number, the numbers that write a guest program down, and the pigeonhole (Section 16), and the files that run Parts II and III on that machine and the diagonal on its programs (Sections 16.17 and 16.18). Each also ends by printing the assumptions of every headline theorem in it, and every one of those lines says the theorem is closed. A lemma a closed theorem uses is closed with it. So for those files every headline theorem is closed under the global context, resting on no axiom of mine or of the standard library, and two separate printouts say so. That isn't the same as having no premises: each theorem keeps the hypotheses written in its own statement. Only two small files under minimal/, MuCore.v and Napkin.v, sit outside the big audit, and they are built and checked on their own by `bash scripts/verify_core.sh`, which compiles each from a clean slate with `coqc`, fails on any `Axioms:` line, and expects ten closed theorems from MuCore.v and seven from Napkin.v. The universal program's own files, which lean on the vendored library's counter-machine blocks, sit in the kernel tree with the other link files, and they also end by printing their headline theorems' assumptions; so do the files of the priced host, the compiler from computable functions and the theorem for every computably presented Thiele machine. In plain words: every theorem in the audit either uses no axiom at all or uses only those standard-library axioms, none of mine. The hypotheses in each statement are a separate matter, and they stay. Those counts are of the sources at this edition. Add a theorem and the counts move. The shape, no axiom of mine anywhere, is what every edition has to keep.

22.4 What you're trusting, in one place

If you accept every checked theorem in this book, here is what you're still taking on trust, all of it, so you don't have to collect it from five chapters.

1. The statements. The checker confirms a proof proves its statement. It doesn't confirm the statement says what my sentence about it says. That's why the book prints the important ones in full, premises and all, and why Appendix E prints the Coq text of the load-bearing ones beside a line on whether my sentence says exactly that, more, or less. A machine check supplements that reading.
2. Coq's kernel and its foundation, and the standard assumptions above.

3. Coq's bytecode machine, on the build's side. Coq can hand a computation inside a proof to a virtual machine: a compiler from Coq terms to bytecode and an interpreter for that bytecode, written in OCaml and C, which sit beside the kernel's ordinary reduction and are trusted when used. A proof reaches it through the tactic `vm_compute` or through a cast marked for it. Of the 332 files in the audit, 44 call `vm_compute` themselves, and 267 reach the machine one way or the other, because the standard library's arithmetic deciders (`lia`, `nia`, `lra`, `nra`, `psatz`) check their certificates on it; the vendored library's tactics for its lambda-calculus files call `vm_compute` too. When `coqc` builds those files, it trusts the bytecode machine. Coq's independent checker, `coqchk`, keeps its bytecode machine switched off unless it's told otherwise (its option `-bytecode-compiler` defaults to no), the procedure in Section 22.7 doesn't tell it otherwise, and with the machine off the kernel checks each such step again by its ordinary reduction. So the bytecode machine is trusted by the build, and the final pass doesn't lean on it. No proof uses `native_compute`, the variant that compiles a computation to machine code through the OCaml compiler, so native compilation adds nothing to this list.
4. The tools under Coq. Coq's own program, the compiler it was built with, the operating system it ran on and the processor under all of them are trusted. This is the list Ken Thompson's lecture from Section 13 was about. I didn't write Coq, or OCaml, or any of the rest, so by his moral I can't fully trust any of them, and neither can you. What I can do is keep the list short and print it, so you know exactly which tools you're being asked to believe have no spare key cut into them.
5. The tested parts. The tests are evidence that the programs around the proofs do what's claimed on the runs tried. They can't show the programs always do. Edsger Dijkstra had a sentence for this, and the version in his 1976 book starts by admitting he'd already said it many times: program testing "can be quite effective for showing the presence of bugs, but is hopelessly inadequate for showing their absence." (I'm quoting it as John Harrison, who was then proving floating-point hardware correct at Intel, quotes it in a 2007 survey, from that book.) That's why this item says *tested*. It never says *proved*.
6. The physics. Anywhere a theorem has Landauer's principle or a calorimeter in it, the physical premise is named and assumed (Section 4, Section 21). Nothing here derives it.
7. Whoever sets the start and whoever reads the record. Every guarantee about the record is about programs run from a clean start on a step that does what its definition says. The operator who chooses the start, a reset, a device running the

machine, and anyone reading a report they didn't watch being made are outside it (Section 25 lists the whole fence).

One program on the install list is missing from this list on purpose. CSDP, a numerical solver for semidefinite programs written by Brian Borchers, a mathematician at New Mexico Tech, is called by one proof: `algebraically_coherent_tsirelson_general` (category/AlgebraicCoherence.v) ends with the tactic `psatzQ4`, which hands the inequality to CSDP. CSDP only proposes a certificate, a sum-of-squares identity with rational coefficients. Coq then checks it again: a checker written in Coq, whose soundness is proved in Coq's standard library, evaluates the certificate, and the kernel checks that evaluation. A wrong certificate makes the tactic fail. It can't make a false statement pass. So CSDP is needed to build that proof, and it's outside the trusted base.

22.5 Thompson's knife, again

Every link on that list is a place where Ken Thompson's compiler from Section 13 could sit unseen: a translator that plants something and re-plants it when it translates itself. Coq's own binary, the compiler that built it, the operating system under both. I don't believe any of them does. I can't prove it, and the discipline of this section is to say so. "Formally verified" doesn't cover a chain whose bottom is trust.

And a planted knife isn't the only way a trusted link goes wrong. Sometimes nobody plants anything and the thing at the bottom is just wrong. In 1994 Thomas Nicely, a mathematics professor at Lynchburg College who was computing with twin primes and their reciprocals, found that his new Intel Pentium got some divisions wrong. The standard example is 4,195,835 divided by 3,145,727, which the bad chips got wrong in the fourth decimal place. Intel's explanation was missing entries in a table inside the divider: five of them, it said, dropped by a script. Intel set aside about $475 million to cover it.

Push on the five. In 2024 Ken Shirriff, who reverse-engineers old chips from photographs of their silicon and writes them up on his blog, went to die photographs of an affected chip, read the table's layout off the silicon, and counted sixteen missing entries, left out by a mistake in the table's mathematical definition, of which only five ever mattered. If Shirriff's reading of the silicon is right, and I haven't checked it, Intel's own explanation, repeated for thirty years, had the effect right and the cause off, and it took somebody reading the actual silicon to say so. Floating-point arithmetic went on to become one of the places where formal verification earned its keep in industry; Harrison, who did that work at Intel, brings up this very bug in his 2007 survey of it. Does any of that touch this book? Only at the bottom of the list above. Every proof here was checked by a program running on some processor, and

nobody here has read that processor's tables off the silicon either. That's why the processor is on the list as trusted. I can't give it a kinder word.

22.6 The auditor on top

Beyond the type checker, the project has a custom proof auditor I called the Inquisitor, because a thing that reads every proof looking for ways you've cheated deserves a name you won't forget. It reads every proof file, and its source names about eighty distinct rules. The main families: no holes (no Admitted, no admit, no give-up, no unproved axiom or parameter in project code); vacuity heuristics (no theorem whose conclusion is literally True or a tautology; no theorem of the shape "P implies P"; existence claims backed by a constant witness get flagged, and a flag is a reason to look and nothing more, since some constructions like a fixed interpreter program are *supposed* to be fixed and get judged on what they compute); physics-definition heuristics (no physical quantity defined as a placeholder constant, no physics claim without an invariance condition); scope enforcement (the proof tree is split into tiers, the kernel tier may not import from research-side files, and every active file must reach the cost or semantics foundation through its imports, or it doesn't get to claim the kernel's pedigree); and comment hygiene (no unfinished or historical markers in maintained source). Its rules are heuristics. They point at places to look. They don't prove a theorem means what it should, and they can't promise every empty premise got caught.

So none of this is foolproof, the checker and the auditor included, and I don't treat it as if it were. That's the reason some of the theorems are runs. The worked examples in Part IV, the earned run that pays exactly three and the one that traps for good, and the two guests run on the simpler host, are computed by Coq from the definitions. The demonstration machine on the priced host is different: its numbers are proved, and nobody has run it. A proof can be hollow in a way nobody spots. A definition that has to produce the numbers on the page, run by the checker itself, is harder to fool, and when it disagrees, it tells you where to look.

22.7 Checking the companion formalization

The mathematical claims are the statements printed in this book, with their stated hypotheses and the arguments given here. Running software is an additional way to check their formal counterparts. It isn't a prerequisite for reading the argument.

A reproducible check needs a fixed copy of the proof sources, dependencies and probes. Changing that copy changes the object being checked; a later software result cannot silently revise a theorem in this edition. The procedure below describes the companion proof build. With Python 3, GNU make, Coq 8.18 and its standard library,

CSDP, OCaml with its ocamlfind tool, and MetaCoq 1.2.1 for Coq 8.18 (the vendored lambda-calculus files need it) installed, run this from the root of a checkout of the sources at a fixed revision. The revision this edition was checked against is this one:

```
Companion sources:
  https://github.com/sethirus/The-Thiele-Machine
  tag v4.0.0, this edition's release tag
Revision used for this edition:
  the commit the tag v4.0.0 names; print its hash with
  git rev-list -n 1 v4.0.0
Dependency snapshot:
  Coq 8.18.0 (Ubuntu 24.04 packages), MetaCoq 1.2.1
  (scripts/install_metacoq.sh), vendored Coq Library of
  Undecidability Proofs at upstream commit
  8880e198bdc44cba1bd901f1fee701a95fc440ae (coq-8.18
  branch; vendor/coq-undecidability/UPSTREAM.md)
Build command:
  python3 scripts/reproduce_coq.py
Audit report for this revision:
  artifacts/print_assumptions_all_proofs.json
  SHA-256 85cf21cc932def2abaa49d25cb1447d5
          4a5adffeacf314f53b3fce32e83a0752
  corpus digest 59afe7ccb7d651aedc8e98a557931e46
                ee10d511706d4a0d4956a922737baefd
Commit the audit report was built from:
  git log -1 --format=%H v4.0.0 -- \
    artifacts/print_assumptions_all_proofs.json
  prints the commit that wrote the report; its proof
  sources are the ones the report was built from, and
  python3.12 scripts/assumption_receipt_fingerprint.py
  run at the tag prints the corpus digest above
Manuscript source for this edition:
  monograph/monograph.tex at the release commit; a file
  can't print its own checksum, so take it with
  git rev-parse v4.0.0:monograph/monograph.tex
```

A commit can't print its own hash either, and that's why the block names the release by its tag and gives the command that turns the tag into a hash. The corpus digest ties the report to the sources without trusting the tag. It is a SHA-256 hash, computed by `scripts/assumption_receipt_fingerprint.py`, over this input, in this order: a fixed label; the first line Coq's compiler prints for its version; then,

for every proof file the project lists in `coq/_CoqProject` and every proof file of the vendored library, sorted by path, the file's path and its text with comments removed, each run of white space shortened to one space, and the wording of the strings in its `claim_not_imply` audit notes erased; then the project file `coq/_CoqProject` as written, and the five scripts that scope, generate, run, aggregate and fingerprint the audit, each read as a stream of Python tokens with its comments dropped. The version-two fingerprint records token names instead of Python's internal numeric token IDs; the same corpus gives the same digest under Python 3.12 and 3.13. Each item is followed by a zero byte. So an edit to a Coq comment or to its layout leaves the digest alone, and any other edit to a proof file moves it, apart from the audit-note wording described above.

The counts this book reports from the assumption audit (5,997 theorems in 341 files) are the ones in that report, and they belong to that revision. A change to the proof sources beyond their comments and layout changes the corpus digest; the audit then has to be run again and its counts read afresh. With those installed, run:

```
python3 scripts/reproduce_coq.py
```

That copies the sources into a fresh directory and works only from the copy: it builds the part of the vendored Coq Library of Undecidability Proofs the proofs use, regenerates the build file, rebuilds every proof from source, and runs Coq's independent checker over every module the project lists. Extra probe files that recheck named theorems can be added with `-probe`; none run by default. It reuses no compiled proof from the checkout, and it writes its pass file only after every stage passes. A failed run resumes from the snapshot it captured, and a change to the sources needs a fresh run:

```
python3 scripts/reproduce_coq.py --resume \
  --output artifacts/reproduction/RUN_DIRECTORY
```

The run has to pass. Don't stop at the pass: read the theorem types and their global dependencies, because a closed report doesn't make a theorem's own premises go away.

Each other result this book reports has its own command, run from the root of the same checkout. The tests that need Coq skip without coqc 8.18 and a compiled tree; the two environment variables that point them at one are named in tests/realize_harness.py.

```
Full assumption inventory (regenerate, then compare):
  make assumption-receipt
  make assumption-receipt-check
The custom auditor (its warnings are heuristics):
  python3 scripts/inquisitor.py \
```

```
    --report INQUISITOR_REPORT.md
The two files outside the audit:
  bash scripts/verify_core.sh
Extraction to OCaml:
  scripts/realize_extract.sh build/realize
Python against Coq, with the deliberate breaks of
the small machine (the test named
test_mutations_of_the_small_machine_are_caught):
  python3 -m pytest tests/test_realize.py
Python against the extracted OCaml:
  python3 -m pytest tests/test_realize_ocaml.py
The compiler runs:
  python3 -m pytest tests/test_cmp_compiler.py
The U and U_P runs in Running it, with steps and times:
  python3 scripts/realize_universal_runs.py \
    build/realize/realize_driver \
    --case all --json runs.json
The edition's numbers against the receipt and the Coq:
  python3 -m pytest tests/test_edition_numbers.py
The semantic boundaries, runs that show what a pass means:
  python3 -m pytest tests/test_semantic_boundaries.py
```

The auditor's warnings are heuristics. A constant witness or a plain tautology can be the right construction, and a warning isn't a defect until somebody shows one.

That's the checker. What's left is to tell you how to break what it checked.

23

How to break this

Every main claim in this book comes with the exact shape of the thing that would kill it. I went to the trouble on purpose. A claim you can't break is a claim that wasn't saying anything. So I'm handing you a loaded gun pointed at my own work, with every sentence sharp enough to aim at. These are written as instructions: here is the trace to find, the object to build, the theorem to exhibit. Build one and the proof tree falls, and I mean that as an invitation.

The invitation is an old one, and people have put prices on it. Donald Knuth gives the first person to find each error in his books one hexadecimal dollar, 0x$1.00, which is $2.56, deposited these days at the Bank of San Serriffe, a bank that doesn't exist, since real checks turned out to be an open door for fraud. Why pay strangers to find your mistakes? His own page says it plainly enough: a careful reader makes the book better for everyone who reads it next. This chapter doesn't come with a bank, real or made up. What it comes with is, for every claim, a description of the exact object that would make it fall, so you know before you start what counts as winning.

Before the list, the honest part about it. Most targets below are theorems Coq has checked, and knocking one of those over head-on would take a bug in Coq's kernel. That can happen, but it isn't where I'd put my money. The places a reader can actually win are aimed at me, and there are three. None of them goes through Coq. A sentence of mine that says more than the Coq statement it cites (Appendix E puts the load-bearing ones side by side to make that easy). A Coq definition that doesn't capture the English word it's named for: on the small machine, "is at least 0" is a check that can't fail, and the step certifies it anyway, so "certified" there doesn't mean "found something out" (Section 16 says so, and that's the kind of gap to hunt). And a list of premises that only its own example satisfies.

23.1 The abstract claims (Parts II and III)

The universal floor (Section 3). Build a certification system that satisfies the one-step toll and give a run from uncertified to certified with total cost zero. `universal_nfi_any_substrate` falls. If you instead drop the toll, you've built a free-certification system. That's a different system, and it counts for nothing against the theorem.

The toll from premises (Section 4). Exhibit a finite, duplicate-free list of every state, a certificate no step revokes, and an *injective* step that switches it on. `permanent_flip_is_not_injective` falls, and the derivation of the toll with it. An infinite state space, a revocable certificate, or an unpriced merge is not a counterexample; the file already builds one of each. For the squeeze bound, a finite permanent machine with a cost meeting the halving premise where one step switches k states on beside m certified ones and two to the cost times m is less than m plus k; `permanent_flips_compression_bound` falls. For merge-or-revoke, a finite machine and an injective instruction that switches a reading on without switching it off anywhere; `flip_merges_or_revokes` falls. A move that merges and also revokes counts against nothing. It's the three cards. For the eight-state machine, a move of it that merges two states and costs nothing; `fin_merging_priced` falls.

The bill in bits (Section 4). Exhibit a finite machine with a permanent certificate, a move that switches k states on beside m certified ones (k at least one), and a spread of probability over those m plus k states whose image under the move has entropy above log base two of m. `permanent_step_entropy_ceiling` falls. For the general form, a step and a spread on a finite list of every state under which two different states, each with some probability, land on the same state while the entropy doesn't drop; `entropy_drop_pos_of_support_merge` falls. A heat measurement on a real device below what the bound asks doesn't touch either theorem. It refutes the premise `landauer_heat` for that device, which is Landauer's principle, and that's a fight with physics. The counting stays where it was. And a certifying step from a state already known that gives off no heat isn't a counterexample: `known_state_flip_forces_no_heat` says the premise sets no positive floor there.

Learning is free (Section 5). Show that the four-state demon's measure step merges two states, or that its observer's candidate list doesn't shrink. `observer_narrowing_can_be_free` falls. A different price schedule that charges the measure step is allowed and isn't a counterexample. The claim is "not forced." It never says "always free."

The counter (Section 6; on the small machine, Section 16). Find a state of the small machine and an instruction such that the ledger after is anything other than the ledger before plus that instruction's cost; the loud version is a ledger that goes down. `mu_conservation` falls. The uniqueness half, that only one bookkeeping starts at zero and follows the schedule, is checked over any counter machine as `nec_f_initiality` (Section 6); a reachable state where two such bookkeepings disagree makes it fall.

Windows (Section 7). Exhibit a window and a price computed from the observed before and after that meets the floor and never overcharges, on a machine where two steps share their before and after pictures and only one certifies. `shadow_cannot_`

`price_exactly` falls. A window that shows certification is not a counterexample; the theorem says exactly that such a window prices exactly. For the three escapes, a verifier of one of the three kinds that's wrong about the pair, under its own relation; the argument after Definition 7.1 falls.

Entitlement (Section 8). Produce a strict narrowing of a candidate list, with a distinguishing observation, that does not yield a strictly stronger membership test; the card argument falls. Or exhibit a run with a tree no deeper than its paid steps, covering the narrowing, whose counter rose by less than the index bits lost; Theorem 8.2 falls. On every Thiele-complete machine: a run from a clean start that ends with the record up, meets the hypotheses of Theorem 16.43, and has fewer record moves than the index bits it narrowed, or has no earned chain in it; `ent_representation` falls, and the weighted form (`ent2_weighted_representation`) falls the same way. A narrowing with a big drop and a small bill is not a counterexample unless a tree no deeper than the record moves covers it: the covering is a hypothesis, and `ent2_uncovered_claim_false` says why it has to stay one. On the host machine, a run from a clean start that raises the record only after more than sixteen CHECKs; `ent2_questions_cap` falls. On the small machine, a program of INC and DEC that runs fewer than m moves and whose final line number and B determine whether $A \geq m$ at every start; `ent2_free_cannot_decide_fast` falls.

The latch (Section 10). Exhibit an extension of a base whose record is driven by the computation and never switches off, and which is not a latch on any event. `record_axis_is_latch_holds` falls. A record that switches off, or one driven by a hidden clock, is the file's own pair of counterexamples. The file built those already.

The diagonal (Section 13). Two routes. A decider for some substrate with a shortcut predicate whose flip the substrate can represent and which answers correctly on every program; the generic proof fails at a named step. For the nat family, a d that correctly decides the predicate of the substrate built from d. L has its recursion theorem by proof, so it gets a target of its own: a map on L's terms, computable by an L term, with no term that behaves like its own image; `L_recursion_theorem` falls. On the host machine: a host program that decides an extensional property of host programs that some program has and some lacks, or a map of host programs that a host program carries out, with no host program that computes the same function as its image; `sm_host_rice` or `sm_kleene` falls. On the small machine: a decider for a property of what two-counter programs do from the clean start that some program has and some lacks; `sm_guest_rice` falls. A decider for such a property over every input of two-counter programs; `tc_rice` falls. A two-counter program that multiplies every plain input by the number of Theorem 16.60; `tc2_no_mult` falls. A map of host programs that a host program carries out, with no host program that stops when its image stops and agrees with it on register 0, the trap latch, the ledger,

the flag, the number of facts and whether the channel is empty, is a counterexample to `sm2_kleene_obs`. A map whose fixed point disagrees with its image on some other register is not one: some computable map has no fixed point that agrees with its image on every register (`sm2_no_exact`).

The pointer thesis (Section 15). It's a thesis, so the three routes are its own: a deployed discipline pricing a record that parties who didn't produce it don't copy and rely on; a rival event copied and relied on as well as certification in a real ecosystem; an honest formalisation, with independence as defined there, where certification fails the test.

One concession goes with it, and I make it in full. The window results, the separation of the two runs, and the counter's uniqueness don't single out certification. Their proofs use only that the step carries something the classical picture doesn't determine, so the same shape of result holds for any such field. Nothing in Parts II to IV proves that certification is the one event a model is *forced* to price; Section 4 forces a price on every merge, certifying or not. That's the thesis, and Section 15 is the case I make for it.

The five models (Section 14). Each fragment has a theorem with a named falsifier: a finality fork with nobody slashable; a gas schedule meeting floor and no-overcharge whose charging predicate differs from the commit predicate on some step; one of the RFC's own example proofs that the transcribed inclusion check rejects, or an input with leaf index equal to tree size that it accepts; a program the bounds checker accepts whose verification condition fails. Each kills one fragment. None of them, and I want this clear, says anything about the real system beyond the fragment.

23.2 The machine claims (Part IV)

The small machine (Section 16). Four targets. From a clean start, a run that raises the flag without an earlier passing CHECK, COMMIT and CERTIFY of the same claim, with the counter's version unchanged between the CHECK and the COMMIT; `ear ned_certification_provenance` falls, and `earned_commitment_provenance` for a COMMIT alone. From a clean start, a trace that leaves a fact in the table nobody checked; `no_forging` falls. From a clean start, a run that ends certified for a total charge under three; `certified_run_min_cost` falls, and a run at exactly three is not a counterexample: CHECK "A is 0", COMMIT, CERTIFY from a start where A is 0 is one, and the same program from any other start is refused forever (`earned_run_check_can_fail`, `earned_run_refused_forever`). A check that can't fail certifying for three isn't a counterexample either; the step doesn't police trivial checks, and the book says so. A two-counter machine and a program whose run on the small

machine disagrees with it step for step, or halts when it doesn't; `simulation_step` or `halting_correspondence` falls.

Thiele-complete (Section 16). A machine that charges at least one for every move and still meets the four clauses; `complete_has_free_move` falls, and with it `clock_not_thiele_complete`, whose proof uses it. A Thiele-complete machine and a function of its window that gives its record at the end of every run from every loaded state; `complete_no_record_oracle` falls. A clause none of the excluded machines needs: drop it and show the paid latch or the silent machine still fails, and `paid_latch_not_thiele_complete` or `silent_not_thiele_complete` has been proved by the wrong clause, which is worth a letter.

The universal program (Section 16.13). A program of the small machine and a start on which U halts and the guest doesn't, or the other way round; `universal_halting` falls. One on which the flags disagree, or the ledgers differ at a matching point; `universal_flag_iff` or `universal_ledger_exact` falls. A run of U whose raised flag doesn't stand on a passing CHECK, then COMMIT, of one slot, then CERTIFY; `universal_earned` falls.

Every computably presented machine (Section 16.14). A computably presented Thiele machine and a start on which U_P's flag goes up while the machine's reading never says yes, or the other way round; `presented_universal_flag_iff` falls. With the reading starting at no, a step the machine reaches at which every point of U_P's run that matches it has a ledger more than two above the machine's; `presented_universal_surcharge_le_two` falls. A program on the host machine, from a clean start, that matches exactly a guest certified with its whole ledger below three; `presented_universal_no_exact_below_three` falls. A machine whose reading no program decides isn't a counterexample. It's outside the class, and the book says so.

The classical CHSH bound (Section 20). Exhibit four fixed bits and a tally consistent with that one plan, every pair sampled, scoring above two. `chsh_violation_rules_out_locally_factorizable_coupling` falls, and so does known mathematics. A randomised plan landing past two by luck doesn't count; the theorem is about one fixed plan, and Section 20's coin-flip example shows the luck.

The integer check (Section 20). The game check has two walls, and here's how to knock each one over. A correlator that passes the three contraction inequalities whose five-by-five is not PSD (forward), or a PSD five-by-five of that shape whose correlators aren't contractive (reverse); `npa_psd_iff_column_contractive` falls. A correlator whose pinned five-by-five is PSD (contractive, by the equivalence) with S squared above eight; `npa_psd_implies_tsirelson_bound` falls. The PR box and the classical mixture (1, 0, 1, 0) are *not* counterexamples to any of these. Both fail contraction by plain arithmetic (the table in Appendix F.4), which is exactly what the theorems need them to do. On the machine: a raised flag from a clean start with

no passing CHECK of the packed-counts property before it, or whose counts give $S^2 > 8$; `small_chsh_flag_implies_tsirelson` falls. A tally that passes the seven facts while its pinned matrix isn't PSD, or one with every pair sampled and a PSD pinned matrix that fails them; `small_chsh_check_iff` falls. The all-ones tally and the PR box being refused are not counterexamples; those refusals are the point.

The nine-by-nine (Section 20). A point and five extra numbers where the contraction-like condition holds and the nine-by-nine isn't PSD, or the other way round; `q1ab_psd_iff_column_contractive` falls. A point where the condition holds with all five extras at zero and S is above two; `q1ab_check_at_gzero_implies_classical_bound` falls.

The elliptope completion (Section 20). This one's a shape, so you're trying to push a point in or out of it. Three targets, one per theorem, plus the decider beside it. Signs a_0, a_1, b_0, b_1, each squaring to one, and a vector on which the completed matrix of that plan goes negative; `deterministic_strategy_elliptope` falls. A correlator with some positive semidefinite completion and S squared above eight; `elliptope_tsirelson` falls. Real numbers x and y that make the PR box's completed matrix positive semidefinite; `pr_box_not_elliptope` falls. And for the decider that sits beside it: inputs on which it says yes while the correlators have no PSD completion at all (S squared above eight is one way to show that); `elliptope_check_full_sound` falls. The all-ones plan and (1, 0, 1, 0) aren't counterexamples here either, from the other side: the completion admits them (`turing_point_elliptope` for the second), which is what the theorems require.

The audit (Section 22). Run the assumption probe on any theorem the book cites and find a project-local axiom or parameter in its dependencies, and the claim of zero of them falls. For the small machine, its hosts and the universal programs, compile any of their files and find a printed assumption line that isn't closed. Run the Inquisitor and read what it says; it fails on high and medium findings. A new finding means somebody looks. It doesn't automatically mean a theorem is false.

23.3 What's mine and what isn't

This is not a claim to have been first. Appendix C does the sorting item by item, with sources: what was known, what's added, and what kind of addition it is. Here's the short form, and I'd welcome correction on any boundary.

Borrowed, with names. As the opening chapter warned, I usually had the hunch first and found these names after, and that order doesn't make any of it mine. Everything in Part I (every source is cited where it's used), the category theory (Eilenberg, Mac Lane) included. Minsky's two-counter machines and the Coq library that proves their halting undecidable and compiles computable functions into them

(Forster, Larchey-Wendling and coauthors). Turing, Church, Kleene, Rice, Cantor, Gödel, the diagonal pattern, Lawvere's unification, and the mechanized versions of them: Michael Norrish's in HOL4, Mario Carneiro's in Lean, and Yannick Forster, Fabian Kunze and Gert Smolka's in Coq. Bell, CHSH, Tsirelson, Stephanie Wehner's semidefinite proof of Tsirelson's bound (2006), NPA, Landau, Masanes, Fine, the elliptope, and the machine-checked proofs of Tsirelson's bound that came before mine (Echenim and Mhalla in Isabelle, and Lean's mathlib). Landauer, Bennett, Szilard, Maxwell, and their critics. The potential method (Tarjan) and its mechanizations (Nipkow, Charguéraud and Pottier), Lamport's clocks, Floyd and Hoare. Gettier, Necula, Cook, Hartley, Shannon, Rissanen. Zurek. The guarantee about every program: the reference monitor (Anderson), execution monitors (Schneider), and monitors checked at every instruction and verified in Coq (Azevedo de Amorim and coauthors). The counting under the permanence results: Poincaré's recurrence, as Zermelo used it. The seL4 and CompCert discipline. The facts under the permanence results are Landauer's own example and Shannon's bounds, and "set a bit to one whatever it held" is my way of saying his RESTORE TO ONE.

Formal reconstructions. The Coq development defines the state types, transition systems, real-matrix predicates, encodings, and proof terms this book uses. When I say a theorem is checked, I mean the stated proposition has a kernel-checked proof under its printed premises. That doesn't make the underlying idea mine, and it doesn't make it new.

What this project proved, inside its stated interfaces. The abstract floor, which is the potential method with one particular potential. The derivation of the toll from finiteness, permanence, and a named price, in counted states and in bits, with each premise's counterexample built. The eight-state machine on which every one of those premises is a theorem. The window that can't price the flag exactly, and the small machine's own colliding pair. The latch form over any base, with its two counterexamples and converse. The small machine's earned record from a clean start (checked, then committed, then certified, about the same edition, nothing forged, three at least), its exact ledger, its universality through Minsky, the definition of Thiele-complete that every clock fails and the window blindness of every machine meeting it, the universal program U with its guest's halting and flag matched exactly, its ledger equal to the guest's at every matching point, and its own flag earned, the priced program U_P that runs every computably presented Thiele machine with its reading latched, its flag earned on the guest's own reading and its ledger within two of the guest's when the reading starts at no, a surcharge no program on that host, run from a clean start, can avoid when the guest certifies with its whole ledger below three, and the simpler host's mirror, its isolation and its refusal to certify for free. The substrate diagonal and the L results, with Rice's and Kleene's theorems

on the machine's host and Rice's from the clean start on the two-counter machine. The integer check, exact for every pair being sampled and the pinned matrix being PSD (and so a sufficient test for the bound, which tells you nothing about quantum behaviour), the equivalence with contraction, Tsirelson's bound from PSD, which is known mathematics checked again here, and the check earned on the host, a raised flag standing on it. Structural entitlement for every Thiele-complete machine, with its worked members, its floor that takes no supplied tree and its proof that the covering can't be dropped; the eight-state machine run by the small machine; the time a check saves; and the collision of every bare-transcript verifier on every Thiele-complete machine. All of it with premises printed. Of that list, the toll derivation as a rule on a ledger (Ouldridge and Wolpert priced the same merge in entropy first), the pricing form of the window fact, and the latch form are the only pieces I'd call new statements, and only in the sense that I didn't find them stated where I looked. The fourth thing I'd name is built and checked, and its idea isn't new: a record that's priced, and on the small machine earned, under the reference monitor's every-program move, which is old and isn't mine. Appendix C names the same four the same way, with the prior work beside each.

Interpretation, which is mine and not proved. That structure is what a computation establishes and the toll is the right thing to price. That certification is the pointer event. That the record axis "belongs in the step." Those are proposals, with the arguments I've given, and the proposals are where I'd aim first if I were you.

The claim ledger

Here's everything this book claims, one line each: what kind of claim it is, and where it lives. "Proved" means proved under the premises printed in that section, and nothing more. If a line here ever says more than its section does, the section is right and the line is wrong. "By unfolding" after it means the proof does little more than read the answer off the definitions, a case split whose every branch is a line: true, checked, and carrying almost nothing the definitions didn't already say. If you're counting results, count those differently. I mark them so they don't pad the list, and where they're stated in the book they carry the label *observation*.

Ledger is a good word for it, and it's older than bookkeeping. Around 1400 a *ledger* was a book that lay permanently in one place, a big church breviary, say, from the old verb *leggen*, to lay; the account-book sense turns up in the 1500s. A book that stays put. That's the job here too. The home sections are long and they wander, and this is the one place where every claim lies still, so you can come back and find it where you left it.

24.1 Claims about the model

First the model. Every line in this table is about any system that has the pieces the line names: a state and a step, plus whatever the result asks for, a reading and the toll, a counter and a price list, a window, programs. None of them is about my machine, except the lines that run these results on it, which say so and point to Section 16.17. The table runs over several pages, in reading order.

Here's one way to use it, and it takes five minutes. Pick the sentence in this book you trust least. Find its line. Read the Kind column before anything else. Then go to the home section and check that the line says no more than the section does. If it says more, you've found my mistake, and the end of this section says what I'd like done about it.

Claim	Kind	Home
The early engine derived physics	not claimed; it ran a simulation	Section 1

Claim	Kind	Home
Structure: relations, kept under chosen changes, carried by any stuff, relative to a frame	choice (my overlay of the sources; they never proved it together)	Section 2
Turing's four parts have no slot for "these belong together"	not claimed; I looked and didn't find one, which is weaker	Section 2
Equivalence relation, partition	definition	Section 2, Definition 2.1
Sameness and sorting are one object	proved (textbook, Halmos; `nec_t_rel_of_partition_equiv`, `nec_t_blocks_of_equiv_partition`, `nec_t_rel_round_trip` and `nec_t_blocks_round_trip`, NecTPartition.v)	Section 2, Theorem 2.1
A partition is the start, smallest by the count of names	choice	Section 2
Nobody prices vouching for a sorting	not claimed; only that I found no one who does	Section 2

Claim	Kind	Home
Certification system: states, moves, step, cost, reading	definition	Section 3, Definition 3.1
The toll	choice	Section 3, Definition 3.1
The universal floor: no to yes costs at least one	proved (premise: the toll), by unfolding	Section 3, Observation 3.1
The floor speaks about the world, or computing power	not claimed	Section 3
A computation is in scope just by being one	not claimed; someone has to supply the five parts and prove the toll	Section 3
On a finite machine, a permanent flip merges	definition; proved (no price premise)	Section 4, Definition 4.1, Theorem 4.1

Claim	Kind	Home
In the worst case over spreads a permanent flip removes a full bit, whatever the number of yes-states; for an even spread it removes $\log_2 \frac{m+k}{m}$	proved (`permanent_flip_spread_loses_a_bit`, `merge_pair_removes_a_bit`, `permanent_flip_uniform_entropy_drop`, PermanentCertificationEntropy.v)	Section 4
Merge pricing	assumed (premise)	Section 4, Premise 4.2
Finite, permanent and merge pricing give the toll	proved (premise: merge pricing), by unfolding	Section 4, Observation 4.2
Each premise has a counterexample	proved (the strip, the coin, the stamp); "free" is proved too (`history_escape`, `toggle_escape` and `free_merge_escape`, AxMerge.v)	Section 4
On a finite machine, a flip merges or revokes	proved	Section 4, Theorem 4.3
Forced to be priced exactly when it merges	definition; proved (premise: move names decidable)	Section 4, Definition 4.3, Theorem 4.4
On a finite machine with decidable move names, permanent records are forced; the converse fails	proved (the three cards)	Section 4, Corollary 4.5
Halving price	assumed (premise)	Section 4, Premise 4.4
The squeeze bound	proved (premise: halving price)	Section 4, Theorem 4.6
An eight-state machine meets every premise, so the toll holds on it outright	proved	Section 4
The small machine runs the eight-state machine through a window, move for move, paying exactly its charges	proved (from live states; the setup's claim can't fail)	Section 16.17, Theorem 16.41
The small machine as a whole is not merge-priced	proved	Section 16.17, Theorem 16.42

Claim	Kind	Home
The halving price holds exactly when no pile-up exceeds 2^{cost}; the eight-state price list is the least that meets it; the small machine is not halving-priced	proved (finite machines; Landauer's price premise stays named)	Section 4

Claim	Kind	Home
Forgetting nothing removes no bits	proved	Section 4, Theorem 4.7
A permanent flip from the even spread removes at least $\log_2 \frac{m+k}{m}$ bits	proved	Section 4, Theorem 4.8
Landauer's principle	assumed (premise)	Section 4, Premise 4.5
Heat floor of a permanent flip	proved (premises: Landauer, $k_B T \geq 0$, the even spread)	Section 4, Corollary 4.9
Positive heat for any full spread	proved (premises: Landauer, $k_B T > 0$)	Section 4, Corollary 4.10
Entropy pricing gives the toll	definition; proved (premise: entropy pricing)	Section 4, Definition 4.6, Theorem 4.11
Merge pricing, the halving price and entropy pricing are one premise: the most bits a move removes from any spread is $\log_2$ of its worst pile-up, entropy pricing is the halving price, and merge pricing is both asked of two states	proved	Section 4, Theorem 4.12
On a known state the premise sets no positive heat floor	proved	Section 4, Theorem 4.13
The bits removed, one term per state	proved	Section 4, Theorem 4.14

Claim	Kind	Home
On the small machine CERTIFY and COMMIT merge, every merge-priced cost obeys the toll, and record-layer merge pricing holds for its own prices and gives the toll, with no finiteness premise	definition; proved (premise: record-layer merge pricing, for the toll)	Section 4, Premise 4.7, Theorem 4.15
Landauer's principle is true of the world	not claimed; a named premise, with its critics named	Section 4
Any machine worth building is of this kind	not claimed	Section 4
Permanence is the job to pick	choice	Section 4
The machine's own narrowing is priced	proved (premise: halving price)	Section 5, Theorem 5.1
An observer can learn for free	definition; proved (one schedule that allows it)	Section 5, Definition 5.1, Theorem 5.2
Learning is free on every schedule	not claimed	Section 5

Claim	Kind	Home
Three states are the minimum for free teaching	proved	Section 5, Theorem 5.3
"No Free Insight" names the committed kind	choice (the name kept, the scope stated)	Section 5
Watching narrows like a quantum measurement	not claimed	Section 5
A counter kept by a price list	definition	Section 6, Definition 6.1
Conservation	proved (`nec_f_conservation` and `nec_f_counter_monotone`, NecFCounter.v)	Section 6, Theorem 6.1
Only one honest count	proved (`nec_f_initiality`, NecFCounter.v)	Section 6, Theorem 6.2

Claim	Kind	Home
The fold on lists is unique; it reaches states only under a condition	proved (`nec_f_fold_unique`, `nec_f_descent_iff_functional`, `nec_f_factor_gives_descent` and `nec_f_descent_fails_for_history`, NecFCounter.v)	Section 6
Initiality picks the prices, or makes marks physical	not claimed	Section 6
The toll is the potential method	definition; proved	Section 6, Definition 6.2, Theorem 6.3
The potential method is new here	not claimed (it's Tarjan's)	Section 6
The counter is the cost-semantics frameworks' writer	proved for the writer (`run_writer_is_run_and_cost`): a run in the writer over whole numbers returns the final state and the total cost	Section 6
The counter embeds the cost-semantics frameworks' type systems and their before-the-run checking of grades	not claimed; by design the grade is written by the step during the run	Section 6
A window that forgets the counter can't recover it	proved (premise: the counter can be changed on its own; `nec_w_forgetful_window_no_counter` and, for the flag, `nec_w_forgetful_window_no_flag`, NecWArgued.v); the premise is needed (`nec_w_forget_needs_settable`)	Section 6
A decoder needs constant fibres	proved (any types)	Section 7, Theorem 7.1
No exact window price across a collision	proved (premise: a collision), by unfolding	Section 7, Observation 7.2

Claim	Kind	Home
On the toy, no verifier on bare transcripts is sound and complete, and none that reads only the bare part of a richer transcript is either, for any pair like it	proved, over arbitrary types	Section 7
Full-state, commitment and response verifiers are sound and complete for their explicit explanation relations; commitment requires binding and honesty	proved; verification cost is stipulated, and no proof reads it	Section 7, Definition 7.1, Theorem 7.3
On every Thiele-complete machine, no verifier on bare transcripts is sound and complete for the record, or for a ledger rise of 3	proved	Section 7, Theorem 7.4
The one-round "ask" models a cheating prover	not claimed	Section 7
The window fact is mysticism, or Rice	not claimed	Section 7
Entitlement: object, checked relation, paid event	definition (the book's word; Coq has no such term)	Section 8, Definition 8.1
A decision tree has at most two to its depth leaves	definition; proved	Section 8, Definition 8.2, Lemma 8.1
Paid narrowing, counted	proved with the bill of the charges (`nec_e_cs_count_any`, NecEEnt.v), which equals the counter's rise by conservation (`nec_f_conservation`); both are checked (premises: a covering tree, paid steps)	Section 8, Theorem 8.2

Claim	Kind	Home
Structural entitlement over any certification system: a strictly stronger test, a paid crossing, a bill that covers the index bits	proved (premises: covering tree, paid steps)	Section 8, Theorem 8.3
A reading that a grouping was checked and committed sees only its partition and entitles exactly the groupings that refine it	proved	Section 8, Theorem 8.5
Structural entitlement for every Thiele-complete machine: stronger test, earned chain, index bits at most the record moves, ledger rise at least 3	proved (premises: a clean start, and M1 to M6)	Section 16.17, Theorem 16.43
A floor with no supplied tree, from the run's own CHECKs	proved (answers taken at the rivals as they start)	Section 16.17, Theorem 16.44
Structural entitlement with the narrowing made by the run, over any certification system whose test steps are named: every drop made by a test the run passed, the exact product count, index bits at most k times the bill	proved (premises: test steps charged at least one, a prior containing the start, a 2^{-k} share kept by each test)	Section 8, Theorem 8.4

Claim	Kind	Home
The same for every Thiele-complete machine: index bits at most k times the passing CHECKs, at most k times the ledger's rise; the rounded sum with no share assumed; the share can't be dropped	proved (premises: a clean start, the record up at the end, a prior containing the start, the share for part (4))	Section 16.17, Theorem 16.45, Proposition 16.46
What the record adds to the price bound from a clean start: the earned chain and a floor of 3	proved	Section 16.17, Theorem 16.47
The weighted count and the partition form	proved; the weights are whole numbers, and no probabilities enter	Section 16.17
The eight-start member, and a search through n bits with up to 16 questions	proved; the 16 is the machine's fact table	Section 16.17, Proposition 16.48, Theorem 16.49
The covering hypothesis can't be dropped	proved, by a counterexample on the small machine	Section 16.17
The time tax: no free program of fewer than m moves decides $A \geq m$, one paid move decides it, and three paid moves certify it	proved, for programs of INC and DEC	Section 16.17, Theorem 16.50
The lists, tree and fibres are read off the run	not claimed, except in the worked members	Section 16.17
Entitlement is truth, or permission	not claimed	Section 8
The count prices why candidates were dropped	not claimed; it prices the size of the drop	Section 8
Certified insight event, raw observation	definition	Section 9, Definition 9.1

Claim	Kind	Home
The toll prices only the crossing, flat in depth	fact read off the definitions	Section 9, Remark 9.1
Sorting moves by whether the reading moves	choice	Section 9
Nothing makes the crossing wait for a check	fact read off the definitions	Section 9, Remark 9.2
A paid flag proves its claim true	not claimed	Section 9

Claim	Kind	Home
Base, cover, base-driven record, latch	definition	Section 10, Definition 10.1, Definition 10.2
Over any base, a base-driven record is a latch	proved, by unfolding	Section 10, Observation 10.1
Every reachable latch is a base-driven record	proved	Section 10, Theorem 10.2
A finite reversible base can't write a record	proved; an endless history can	Section 10, Theorem 10.3
Two records are two latches	proved	Section 10, Lemma 10.4
The toggle: driven, can switch off, no latch	proved	Section 10, Lemma 10.5
The clock: permanent, undriven	proved	Section 10, Lemma 10.6
The clock is undriven on the states its runs reach, while the clock started in sync is a latch there and is rejected only by (R1) over every state	proved	Section 10, Lemma 10.7

Claim	Kind	Home
Base-driven records agree up to the schedule	proved (`nec_w_billed_honest`, `nec_w_billed_same_latch`, `nec_w_billed_same_up_to_schedule` and `nec_w_billed_step_cost`, NecWArgued.v)	Section 10
The event is a free choice	proved, the converse applied twice (`nec_w_event_free_choice`, NecWLatch.v)	Section 10, Corollary 10.8
The schedule doesn't fix the probabilities	proved, with the branching latch refuted	Section 10, Theorem 10.9
One state space up to renaming	not claimed	Section 10
The latch on a Turing machine, a register machine, L	definition; proved (and the relation is an equivalence)	Section 10, Definition 10.4, Theorem 10.10
Weak base equivalence connects those bases	not claimed	Section 10
A growing record is a family of latches	definition; proved, and one latch isn't enough	Section 10, Definition 10.5, Theorem 10.11

Claim	Kind	Home
Substrate; shortcut predicate	definition	Section 13, Definition 13.1, Definition 13.2
No inside decider with a representable flip	proved (premise: the substrate's recursion theorem)	Section 13, Theorem 13.1
The nat family: every decider misses program two	proved	Section 13, Theorem 13.2
An outside decider gets the nat family right	proved (`nec_w_outside_decider`, NecWArgued.v)	Section 13
Recursion, Rice and halting for L	definition; proved	Section 13, Definition 13.3, Theorem 13.3
Undecidable means unknowable; the wall separates Turing degrees	not claimed	Section 13

Claim	Kind	Home
Casper setting; one storage slot; a gas schedule	definition (transcribed; the gas schedule's charge premise stipulated)	Section 14, Definition 14.1, Definition 14.2, Definition 14.3
Accountable safety	proved (premises: quorum intersection, one parent)	Section 14, Theorem 14.1
The quorum premise can hold; one vote finalizes free	proved	Section 14, Theorem 14.2
A persistent write to a fresh slot nets at least 20000	proved, on the transcription	Section 14, Theorem 14.3
Gas is exact exactly when commits pay one	proved (premise: the stipulated charge predicate)	Section 14, Theorem 14.4
A permanent write on a finite machine merges, and the merge lands on the record's yes side	proved	Section 14, Theorem 14.5
Platform and quote; inclusion check; bounds fragment	definition	Section 14, Definition 14.4, Definition 14.5, Definition 14.6
The quote can't attest what it never measured	proved	Section 14, Theorem 14.6

Claim	Kind	Home
The RFC's inclusion examples pass	proved (symbolic hash)	Section 14, Theorem 14.7
The proof-carrying checker is the verification condition	proved	Section 14, Theorem 14.8
A transcription is the real system	not claimed	Section 14
A toll is a better design than a wall	not claimed	Section 14

Claim	Kind	Home
Ecosystem, proliferation, pointer; copied and relied on, independent carriers; observation game	definition	Section 15, Definition 15.1, Definition 15.2, Definition 15.3
Independent copies on every state: the event is equivalent at any two states	proved	Section 15, Theorem 15.1
In a replicated header, gas used is copied as finality is, and only finality is relied on	proved	Section 15, Theorem 15.2
Certification is the event whose records are copied and relied on by parties who didn't produce them	thesis	Section 15, Thesis 15.3
In the toy, "certified" is the unique pointer	proved	Section 15, Theorem 15.4
The survey; the swap shows the verdict follows the observer maps	proved	Section 15, Theorem 15.5
One blind observer stops proliferation	proved	Section 15, Theorem 15.6
Agreement doesn't buy permanence; durable views do	proved	Section 15, Theorem 15.7
The formal objects prove the thesis	not claimed; "independent" is defined, "deployed" and "faithfully model" are defined nowhere	Section 15
The floor raised to a threshold	proved (premises: the witness starts at zero, grows by at most the cost, and meets the threshold)	Appendix B, Theorem B.1

24.2 Claims about the machine

Part IV built one small machine and the hosts that run it, and the ledger keeps them apart from the model. The machine is the small one of Section 16, two counters and six instruction kinds, where commitments are earned and universality is proved, with the definition of Thiele-complete that every machine can be held against, the universal program U on a host made of the same instruction kinds, and the priced program U_P that runs every computably presented Thiele machine. Every line in the next tables is about those machines, or about every machine meeting that definition, or about the game check and the physics section that close Part IV, and nothing else, under the premises printed in its section.

Claim	Kind	Home
Checker and arithmetic meaning agree	proved on the small machine, under the stated premises	Section 16, Lemma 16.1
Two-counter simulation and halting correspondence	proved on the small machine, under the stated premises	Section 16, Theorem 16.2 (step), Theorem 16.3 (halting)
Halting is undecidable by the vendored two-counter reduction	proved on the small machine, under the stated premises	Section 16, Theorem 16.4
Exact ledger, permanent certification and toll	proved on the small machine, under the stated premises, by unfolding	Section 16, Theorem 16.5
Instance of the abstract certification system	proved on the small machine, under the stated premises	Section 16, Theorem 16.6
Certification factors as a latch over the core base	proved on the small machine, under the stated premises	Section 16, Theorem 16.7
A passing commitment has a prior matching check at the same version	proved on the small machine, under the stated premises	Section 16, Theorem 16.8
Certification has a check, commitment and certification in that order	proved on the small machine, under the stated premises	Section 16, Theorem 16.9

Claim	Kind	Home
Unearned commitment and uncommitted certification trap	proved on the small machine, under the stated premises	Section 16, Theorem 16.10
Current-version checked claims are true	proved on the small machine, under the stated premises	Section 16, Theorem 16.11
No stored fact is forged on a clean-start trace	proved on the small machine, under the stated premises	Section 16, Theorem 16.12
Bounded table keeps facts and traps when full	proved on the small machine, under the stated premises	Section 16, Theorem 16.13
Equal computational windows can hide different records	proved on the small machine, under the stated premises	Section 16, Theorem 16.14
The window cannot recover the ledger, flag or commitment permission	proved on the small machine, under the stated premises	Section 16, Theorem 16.15
From a clean start, a run ending with the flag up adds at least three to the ledger	proved on the small machine, under the stated premises, by unfolding	Section 16, Observation 16.16
That minimum of three is reached by a run whose check could have failed, and when that check fails the program is refused forever	proved on the small machine, under the stated premises	Section 16, Theorem 16.17
The step refuses trivial checks	not claimed; “is at least 0” passes every value and certifies for three too	Section 16

Claim	Kind	Home
From a clean start, provenance, checker soundness, no forging and the floor of three hold for any property language with an exact equality test and a checker proved to mean what it says, a sorted list read out of a counter included	proved on the generic machine (`generic_earned_certification_provenance`, `generic_checker_soundness`, `generic_no_forging`, `generic_certified_run_min_cost`; the sorted list as `sorted_run_certifies_iff` and `sorted_run_refused_forever`, EarnedGeneric.v)	Section 16
With the flag up from a clean start, the channel names a fact an earlier passing COMMIT committed, and a later commit can replace it	proved on the small machine (`certified_channel_earned` and `channel_names_last_commit`, EarnedCore.v)	Section 16
The guarantees hold against whoever sets the start, a reset, or a second writer that doesn't bump versions	not claimed; they are about programs from a clean start, one step at a time	Section 16, Section 25
Thiele-complete means a universal base that is free and can't touch the record, a record that stays up once it's up, an earned record with a checker proved to mean its claims, an exact toll of one mark per check, commit and certificate, and a claim whose check can fail	definition	Section 16, Definition 16.7

Claim	Kind	Home
Every Thiele-complete machine obeys the toll, runs every two-counter program, counts its record moves on its ledger, pays at least three on any run from a clean state that ends with the record up, has a check that fails and a run that certifies, and from a clean state raises its record only by a CERTIFY	proved, for every machine and every interface meeting the definition	Section 16, Theorem 16.19
A clock (every move charged, any reading or any latch as the record) obeys the toll with a universal base and is not Thiele-complete; nor is any machine whose every move costs something	proved, by unfolding	Section 16, Observation 16.21
Each clause does work: a paid latch meets the base and toll clauses and fails, and a silent machine meets three clauses and fails	proved	Section 16, Theorem 16.22
The four clauses of Thiele-complete are independent: for each, a weakly Thiele-complete machine meets the other three, fails it, and is Thiele-complete through no interface	proved	Section 16, Theorem 16.23

Claim	Kind	Home
Thiele-complete at any unit: with record moves at one fixed price $c \geq 1$ the class is the unit class with its prices multiplied by c, and the clock fails at every unit	proved	Section 16, Theorem 16.24
The small machine, the same machine over any property language with an exact equality test, a proved checker and some property true of one number and false of another, and the sorted-list machine are Thiele-complete	proved on the small machine and on the generic machine	Section 16, Theorem 16.25
Every Thiele-complete machine hides its record and its ledger from its base window	proved, for every machine and every interface meeting the definition, by unfolding	Section 16, Observation 16.26
No fixed host program can keep the guest's facts on an exact copy of a guest counter	proved, for any host property language and any translation into a finite list of host properties	Section 16.13, Theorem 16.27
One fixed program U, in the small machine's own instruction kinds and with no guest step built in, runs every small-machine program from every clean start start(x, y), step for step, with halting and answers matched	proved on the host machine	Section 16.13, Theorem 16.28, Theorem 16.29

Claim	Kind	Home
U's flag rises at some step if and only if the guest's does, and its ledger equals the guest's at every matching point	proved on the host machine	Section 16.13, Theorem 16.30
U's raised flag stands on U's own check, commit and certificate on a slot, and on the guest's own earned chain for the same claim	proved on the host machine	Section 16.13, Theorem 16.31
U's host is Thiele-complete, and whether U halts on a loaded guest is undecidable	proved on the host machine; undecidable in the library's sense	Section 16.13, Theorem 16.32
The simpler host runs any small-machine program as a guest, and halts with any two-counter program	proved on the small machine, by construction: its guest step runs the guest's own step as a primitive; the guest program is held as a list	Section 16.15, Theorem 16.35
From a loaded host, or any state where the two agree, the simpler host's mirror always equals the guest's flag	proved on the small machine, by unfolding	Section 16.15, Observation 16.36
From a host that agrees, the simpler host's step collects the guest's toll; on every trace its ledger covers the guest's	proved on the small machine	Section 16.15, Theorem 16.37
The simpler host's mirror rises only in a guest step that passed CERTIFY	proved on the small machine, for every sequence of host moves	Section 16.15, Theorem 16.38
The simpler host's own instructions never touch the guest	proved on the small machine	Section 16.15, Theorem 16.38

Claim	Kind	Home
From a loaded host, the simpler host's raised mirror means the guest's own run checked, committed and certified the same claim	proved on the small machine (`host_mirror_earned`, UniversalThiele.v)	Section 16.15
The simpler host is a certification system	proved on the small machine	Section 16.15, Theorem 16.39
The simpler host's ledger equals the guest's	not claimed; a guest step with budget b costs b whatever the guest instruction cost, so its ledger only covers the guest's; U's is exact	Section 16.15, Theorem 16.37

Claim	Kind	Home
One fixed program U_P, U with one paid move added, runs every computably presented Thiele machine: halting matched, the state decoded at every matching point, the host's flag up exactly when the guest's reading has been yes, and the host's ledger the guest's plus a surcharge	proved on the priced host	Section 16.14, Theorem 16.33
U_P's raised flag stands on its own check, commit and certificate on a slot whose claim is the guest's own reading of a state where it said yes	proved on the priced host	Section 16.14, Theorem 16.33

Claim	Kind	Home
The surcharge is at most two when the reading starts at no, and zero when the first raising move costs at least three	proved	Section 16.14, Theorem 16.34
No program on the host machine, run from a clean start, exactly matches a guest certified with its whole ledger below three	proved, for every program from a clean start, by unfolding	Section 16.14, Theorem 16.34
The priced host is Thiele-complete, with the paid move read as a check that never passes	proved	Section 16.14
A demonstration machine (one move of price three, reading "at least three") runs on U_P, with the host's ledger exactly $3n$ at the host time matching move n	proved; never run	Section 16.14
A machine whose reading no program decides, or whose next move no program computes, runs on U_P	not claimed; outside the class	Section 16.14
A guest's reading that goes back to no is carried as no	not claimed; the host's flag is the latch of the reading	Section 16.14
A counter beside a Turing machine changes nothing	proved	Section 16.16, Theorem 16.40
A Turing machine can't price a reading	not claimed	Section 16.16

Claim	Kind	Home
Rice's theorem on the host machine, for properties that depend on everything a program ends with (registers, table, channel, trap, ledger, flag)	proved; undecidable in the library's sense	Section 16.18, Theorem 16.51
Rice's theorem for two-counter programs, from the clean start	proved; the fact table isn't compared	Section 16.18, Theorem 16.52
s-m-n, and a universal host program for host programs	proved, with no premise on register 2	Section 16.18, Theorem 16.53
Kleene's recursion theorem on the host machine	proved; the fixed point is of the function	Section 16.18, Theorem 16.54
Kleene's recursion theorem up to the record on the host machine: stopping, register 0, trap latch, ledger exactly, flag, number of facts, whether the channel is empty	proved; the host program simulates the program T that carries out the transformation, marks and all, and pays for the whole record in one go once it has computed	Section 16.18, Theorem 16.55
A fixed point of full behaviour (every register, what the facts name) on the host machine	refuted for registers (`sm2_no_exact`), and for the fact table, the channel and the versions (`nec_e_no_fixed_point_facts` and `nec_e_no_fixed_point_versions`, NecEHost.v)	Section 16.18, Theorem 16.56
No inside decider with a representable flip, and no host decider, for functional properties of host programs	proved (the recursion premise is a theorem there)	Section 16.18, Theorem 16.57

Claim	Kind	Home
Rice's theorem for two-counter programs on every input (stop, counters, trap latch, ledger, flag, and what each fact and the channel are about)	proved	Section 16.18, Theorem 16.58
Kleene's recursion theorem for two-counter programs, inputs written as powers of two, for the function	proved; refuted once the recorded facts count	Section 16.18, Theorem 16.59
Kleene's recursion theorem for two-counter programs on plain inputs	refuted: a carried-out transformation has no fixed point	Section 16.18, Theorem 16.60

Claim	Kind	Home
The integer check makes the pinned matrix PSD	definition; proved	Section 20, Definition 20.1, Theorem 20.1
PSD forces $S^2 \leq 8$	proved (algebra only)	Section 20, Theorem 20.2
A passed check means $S^2 < 8$, and no smaller bound holds	proved: $S^2 \leq 8$ as the composition of the two, and in one statement as `small_chsh_check_tsirelson` (SmallChshCheck.v); strictly below 8, and as close to 8 as you like, by `nec_e_check_strict` and `nec_e_check_sharp` (NecEChsh.v)	Section 20, Corollary 20.3
A PSD pinned matrix has a PSD completion	proved	Section 20, Remark 20.1

Claim	Kind	Home
Every deterministic plan fails the check	proved (`nec_e_check_refuses_all_plans` and `nec_e_check_refuses_deterministic`, NecEChshInt.v); no such plan's pinned matrix is PSD (`nec_e_deterministic_not_pinned_psd`, NecEChsh.v)	Section 20, Remark 20.1
The level-1 test is the size-five case	definition; proved	Section 20, Definition 20.3, Corollary 20.7
The 1+AB matrix ties its four-body cells, the constraint $\langle\{A_0, A_1\} \otimes \{B_0, B_1\}\rangle = 0$; the level-1 pinnings don't imply it, and a classical strategy breaks it	proved (`q1ab_tie_is_a_constraint`, QuantumPartitionPSD_1AB.v)	Section 20, Appendix F
A tally above two isn't one plan replayed	proved	Appendix F, Theorem F.3
A tally above two proves nonlocality; a pass proves the data quantum	not claimed	Section 20
A finite local model meets the bound	proved; Fine's theorem in correlation form is proved as an iff (`nec_e_fine_iff`), and the converse of `fine_theorem` as stated is false (`nec_e_fine_converse_false`, NecEFine.v)	Appendix F, Theorem F.2
The elliptope gate is sound; the PR box fails it	proved	Appendix F, Theorem F.29
The arc-sine boundary: a table has a PSD completion, and so is quantum, exactly when its arcsines pass the four inequalities (if and only if)	proved	Appendix F

Claim	Kind	Home
Every PSD completion comes from a quantum strategy, and the completion set is exactly the set of quantum correlator tables in finite dimension, real or complex	proved	Appendix F
A two-qubit strategy reaches $2\sqrt{2}$; every strategy in finite dimension, with real or complex amplitudes, has $S^2 \leq 8$ and a PSD level-1 moment matrix; every deterministic plan is such a strategy	proved	Appendix F
The integer route to the bound is new mathematics	not claimed; the route is NPA's	Section 20
The integer check passes exactly when every pair was sampled and the pinned matrix is PSD	proved	Section 20, Theorem 20.4
Honest play at Tsirelson's point fails the exact check about 85 percent of the time	simulated, with a first-order limit of $(2+\sqrt{2})/4$; no theorem	Section 20
The check with a tolerance ε is the exact check at $\varepsilon = 0$; a pass means operator norm at most $1+\varepsilon$ and $S^2 \leq 8(1+\varepsilon)^2$	proved (`nec_e_tol_check_zero`, `nec_e_tol_check_norm`, `nec_e_tol_check_bound`, NecEChsh.v); earned CHECK integration: `small_chsh_tol_flag_bound` (SmallChshTolerance.v)	Section 20

Claim	Kind	Home
A flag raised on the host machine from a clean start stands on a passing CHECK of the packed counts, a COMMIT and a CERTIFY; the chain run from a fresh start certifies for 3 exactly when the check passes	proved	Section 20, Theorem 20.5
Five tallies: 12/5 and 14/5 certify; 16/5, the PR box and the all-ones plan are refused forever	proved	Section 20, Proposition 20.6
The tally was played, or was honest, or puts a bound on nature	not claimed; it is placed in a counter for free	Section 20
The calibration is a unit conversion	definition; proved	Section 21, Definition 21.1, Theorem 21.1
Structural dissipation pricing (a whole-unit floor on collapsing instructions, separate from Landauer's principle); dissipation at most cost	assumed (premise)	Section 21, Premise 21.2
The premises fit when every counted step costs at least one, and fit a one-bit reset	proved	Section 21, Theorem 21.2
A collapse needs the merge clause: INC A raises "$A \geq 1$" one way, one-to-one and free; on a finite machine a one-way step always merges	proved (`nec_f_one_way_injective`, `nec_f_small_one_way_pair_fails`, `nec_f_finite_one_way_merges`, NecFPhysics.v)	Section 21, Premise 21.2

Claim	Kind	Home
The small machine's whole instruction set fails the premise pair	proved (`nec_f_small_premise_pair_fails`); its free steps merge (`nec_f_small_free_collapse_is_merge`, NecFPhysics.v)	Section 21
Keeping history makes every step injective	proved	Section 21, Theorem 21.3
The heat is fixed by the chosen gap	definition; proved	Section 21, Definition 21.3, Theorem 21.4
The canonical reset's rates are in detailed balance at no gap; with rates that are, a fixed-gap relaxation settles at $1/(1+e^{\Delta/k_BT})$, which is $1/5$ at the gap $2k_BT\ln 2$	proved (`canonical_rates_break_detailed_balance`, `detailed_balance_never_empties`, `landauer_gap_settles_at_one_fifth`, CalorimeterProtocol.v)	Section 21
A driven reset with rates in detailed balance pays between $kT\ln 2 - kTe^{-D/kT}$ and $kT\ln 2 + D/(2N)$ in work	definition; proved	Section 21, Definition 21.4, Theorem 21.5
The counter is physical dissipation, in joules	not claimed	Section 21
Permanence predicts more than Landauer	not claimed	Section 21
Zero Admitted; a handful of standard assumptions	fact from the assumption audit, true of the sources at this edition	Section 22
The small machine's files, the Thiele-complete files and the universal programs' files use no axioms	fact from each file's own printed assumptions, every one closed, and from the full audit, which finds every probed theorem under minimal/ closed	Section 22

The theorems prove physical reality, or Coq's consistency	not claimed	Section 22

24.3 What I propose

And then there's what I'm proposing. These are mine. They reach further than any theorem in here, and nothing in the tables puts an unavoidable physical price on every observable change.

- That the axis belongs in the step (Section 25, Section 26).
- That the usual account of a computation is a shadow of the step (Section 26).
- That certification is the event worth recording, which is the pointer thesis (Section 15).
- That this is a structural account of computation, with a cost model as one part of it (Section 25).
- That the toll is the right thing to price. The alternatives are a wall or a penalty for contradiction (Section 14, Section 26).

And three things this book doesn't settle, with what would settle each. Whether Landauer's critics are right, which would leave every theorem standing and one premise unowned by the world: that takes measurements of real devices (Section 4). Whether certification really is the pointer event: that takes a way to tie the definitions to the systems the thesis is about, and Section 15 gives three ways to kill it. And whether there's a smaller start than a partition: that takes a ruler someone can state and a smaller structure measured by it (Section 2).

That's the whole ledger. If you find a claim in this book that isn't on it, or a line that says more than its home does, that's my mistake, and I'd want to hear about it.

25

Scope

This is the line around what I've proved. Inside it: an accounting law for one event, a derivation of that law from premises each with its own counterexample, a set of windows that provably lose it, a record axis that's a latch over any base, a diagonal that holds by proof on L, a machine of six instruction kinds on which commitments are earned and which computes what two counters can, one fixed program in its instruction kinds, run on a host with more counters and one paid move, that runs every computably presented Thiele machine, and an integer check that implies a bound by algebra. Outside it: everything that follows. This is the fence. I built it myself, so lean on it before you trust it.

G. K. Chesterton, the English writer who has been called the prince of paradox, has a famous fence, in a 1929 book called *The Thing*. A reformer finds a fence across a road, can't see what it's for, and wants it cleared away; the wiser one tells him to go off and find out why it was put there, and only then come back and argue for tearing it down. This section runs the other way round. Every post below comes labelled with what it's there for, the overclaim it stops, so you don't have to go off and find out first. Pull up any post you like. Just look at what walks through the gap when you do.

Physical interpretation of the counter. The ledger is unique once the start and schedule are chosen. That doesn't make it entropy, and it doesn't make it energy. A physical bridge needs a model of which states carry information, premises that can all hold at once, and a real argument tying the formal charge to dissipation. The permanent-certificate theorems supply the first for an abstract finite machine with a permanent reading. The two premises are shown able to hold together whenever every counted step costs at least one, and for a one-bit reset. The small machine's free steps merge states and are charged nothing, which breaks the bridge for its whole instruction set (Section 21), and the calorimeter protocol shows that population dynamics don't fix the energy scale. No joule value attaches to one mark without thermal and device premises (Section 4, Section 21). Once you pick the start and the price list, the number is forced by arithmetic, and it isn't energy. What I'd need before it could be is the list at the top of this paragraph.

Class separation. The witnesses show specific windows losing specific fields. They don't stop a conventional simulator from keeping those fields in its encoding. No

separation of standard complexity classes follows from the collisions (Section 7, Section 16). I'll be blunt about why I'm careful here. Whether P equals NP is one of the seven problems the Clay Mathematics Institute put a million dollars on, at a meeting in Paris in May 2000. One of the seven has been solved since, the Poincaré conjecture, by the mathematician Grigori Perelman, who posted the proof as preprints in 2002 and 2003. P versus NP hasn't. If a two-state collision on my machine settled it, I'd be owed a million dollars. Nobody owes me anything. A window that loses a field is a fact about that window. A complexity class is a fact about what every possible program can do within a budget, and nothing in these collisions reaches that far.

Trace initiality, and what it doesn't settle. The universal property of the trace fold is about evaluating lists of instructions: pick where a target starts, and there's exactly one way to fold a list into it (Section 6, checked as `nec_f_fold_unique`). The counter's uniqueness assumes a schedule. Rise-only pricing assumes an event, and a reference count that says what charging too much means (Section 7). Those results make the construction precise. None of them shows that the schedule I chose, or the event I chose, is physically necessary.

Other observables. Certification is one worked example. A family of other observations might turn up other structural laws, but each would need its own events, its own evidence, and its own cost rules, justified on their own terms. The window and oracle results are about what specific maps keep, and nothing more.

Program predicates versus state observations. The state-level collision (Section 7's wall) and the program-level diagonal (Section 13's wall) live in different domains. Linking them would take a theorem of its own. The nat family and L (Section 13) keep their explicit quantifiers and representability premises.

Entitlement bounds. The count is handed its tree and its covering, along with the payment condition. It doesn't dig the tree out of an arbitrary run, and it isn't a Shannon bound on every trace (Section 8). On every Thiele-complete machine the payment is the record moves, each exactly one mark, and a floor that takes no tree exists when the questions are the run's own CHECKs; but its answers are taken at the rivals as they start, a free branch can still narrow for nothing, the lists, tree and fibres are supplied except in the worked members, and the covering can't be dropped. The search through n bits is capped at 16 questions by the machine's fact table, and the time tax is a lower bound for programs of INC and DEC only. Nothing here extends it to every Thiele-complete machine (Section 16.17).

Quantum behaviour. The fixed-matrix equivalence is about one completion with zero marginals and zero within-party moments. The elliptope lets the within-party moments move, at the level of correlators. Full behaviours with marginals, and operator realisations, need identifications this book doesn't make. A sound check

isn't complete for those bigger sets, and the elliptope decider is mathematics. None of the small machine's properties is it (Section 20).

The record axis. The proved structural result is: a permanent record driven by a base computation is a latch on an event; on a finite machine its write merges states; and the named windows don't recover it. Certification is one instance. No theorem selects it as the unique event every system must record (Section 10, Section 15). A finite monotone multi-valued record decomposes into threshold latches. One Boolean latch can't hold it. Revocation, including revocation priced from the other side as in Casper, lies outside the permanent-record axis; a price on revocation doesn't transfer to a price on the write (Section 14). Probabilistic records don't inherit the uniqueness result either. Two honest random records can have the same branches and the same charges and still give those branches different weights, one choosing evenly and one two to one, so the schedule doesn't fix the probabilities (`schedule_determines_probabilities_refuted`). And no single deterministic latch explains a record that can branch both ways from the same observation (`deterministic_latch_handles_branching_refuted`). Weak base equivalence is defined and is an equivalence; no two of the three bases of Definition 10.3, and none of them and the small machine, are related by it in the checked work, and the latch form holds on each separately, so it doesn't tell them apart (Section 10). A record driven by the computation that stays up is a latch, and on a finite machine its write merges states; which event deserves the latch isn't proved; records that can come back down lie outside all of this (they're limit computability's, and Section 10 points at Ershov's ladder), and random ones lose the uniqueness.

Who it's safe against. Every guarantee about the record is about programs: any list of instructions, run from a clean start on a step that does what its definition says, one writer stepping at a time. That covers the buggy program and the hostile one. It doesn't cover whoever sets the start (a snapshot put back, a table loaded with facts nobody checked), a reset or a power cycle, whoever picks the program, the wire that carries the report, someone who flips the flag with a probe, the tools that built the step, or a second writer that changes a counter without bumping its edition. Those are the operator, the device and the tool chain, and they're outside this fence. A record read by someone who didn't watch it being made is only as good as their reason to believe the box that shows it. The ways across that I know of, a quote over measured state and a proof that travels on its own, are in Section 7 and Section 14, and this book builds neither. What the guarantee gives inside the fence is safety: no program forges the chain. Availability is a different promise, and it isn't given. A hostile program can put the trap up or fill the table, and after that nothing useful gets done. The proofs also use Coq's whole numbers, which never overflow, so versions and the ledger grow without bound. A build with fixed-width counters,

versions or ledger has to stop before any of them wraps, failing safe with the trap up, because a wrapped version could make an old fact look live again and a wrapped ledger would fall. Starting, restoring, resetting and that overflow stop belong to the implementation, and none of them is proved here.

The diagonal on the machine. What's proved for the small machine's programs is that halting is undecidable in the library's synthetic sense, for them and for the universal program (Section 16). L has its recursion theorem, Rice and halting by proof (Section 13). On the host machine, the one with a counter for every number, Rice's theorem holds for properties that depend on everything a program ends with (every register, the table, the channel, the trap, the ledger and the flag; the line number and the versions are left out), for programs run on an input in register 1; s-m-n holds with no premise on register 2; and Kleene's recursion theorem holds for the transformations a host program carries out on numbers, with a fixed point that agrees with its image on stopping, register 0, the trap latch, the ledger, the flag, the number of facts and whether the channel is empty; for some of these transformations no fixed point agrees on every register. On the two-counter machine itself Rice's theorem is proved on every input, with facts compared by what they're about, their versions left out of the comparison; Kleene's is proved with inputs written as powers of two and refuted with plain inputs (Section 16.18).

The five models. Each is a fragment transcribed from a specification, with a named theorem and a named limit: the Casper transcription has no deposits or fork choice; the gas fragment omits the refund cap; the attestation fragment has no signatures; the transparency verifier covers control flow and leaves collision resistance out; the proof-carrying checker is a three-instruction memory-bounds language. None is a claim about the real system beyond its fragment (Section 14).

The machine. Part IV has one small machine, with the hosts that run it, and here's its fence. The small machine of Section 16 (two counters, six instruction kinds) is where commitments are earned: from a clean start, a raised flag went through a passing check, a commitment and a certification of the same claim, nothing is forged, and the cheapest such run adds exactly three to the ledger, a price reached by a check that could have failed (the step doesn't police trivial checks: a check that can't fail certifies for three too). It's also where universality is proved, by matching Minsky's two-counter machines step for step and borrowing their undecidable halting from the Coq library. It's Thiele-complete under a definition every clock-style machine fails, and every machine meeting that definition hides its record and ledger from its own window. The universal Thiele machine is one fixed program, U, on a host with the small machine's instruction kinds and more counters, with no guest step built in: it runs every program of the small machine from every clean start start(x, y), halts when the guest halts, raises its own flag by its own check, commit and certificate if

and only if the guest's flag rises somewhere in the guest's run, and keeps a ledger equal to the guest's at every matching point. With one paid move added, one fixed program, U_P, runs every computably presented Thiele machine: its step, prices and reading computed by programs, the reading decided on every state, the run fixed by a driver. The host halts when the guest stops, its flag is the latch of the guest's reading, raised on its own earned chain whose claim is that reading, and its ledger is the guest's plus a surcharge of at most two when the reading starts at no, a surcharge no program on that host, run from a clean start, can avoid when the guest certifies with its whole ledger below three. A machine whose reading no program decides is outside. A simpler host with a guest step built in runs guests by construction and keeps a mirror of the guest's flag; its ledger only covers the guest's. The small machine's check price is flat, its fact table holds sixteen, and its certificate is historical (Section 16 lists the rest).

The comparison machines. The small machines I built to compare against are fragments too. The random-access memory result covers an addressed list memory with load, increment and decrement on cells (`concrete_ram_write_reads_back` and its neighbours), and it's about that machine. The register machine Section 10 puts the latch on (`record_axis_is_latch_on_ram_holds`) is a different small machine, the one with indirect addressing, and that theorem is about it. The reversible-machine result covers arithmetic update cores: add and subtract, each undone by the other, with or without wrapping at a modulus (`janus_like_unbounded_inverse`, `janus_like_bounded_inverse`). None is a theorem about real memories or full reversible languages. The cost-semantics results are local correspondences, short of full cost-framework embeddings: the trace's cost is a writer cost and a grade, and the toll is the potential method with one potential (Section 6). Nobody's whole type system gets embedded either.

Outward consequences. I looked for results about real systems that would need the record axis. Five narrow ones are proved, each as a two-state countermodel: a certificate-transparency client's local tree head can't decide whether the world's log is consistent (`ct_local_view_insufficient`); a quote check that matches only the composite digest accepts a mismatched selection (`tpm_selection_binding_is_necessary`); a local suffix can't decide a trusted anchor (`weak_subjective_suffix_insufficient`); an acknowledgement without a durable commit record can be lost (`wal_ack_requires_durability`); and a current local log can't decide what happened before it (`audit_local_snapshot_insufficient`). Each reduces to a known indistinguishability or durability argument. Five stronger candidates would need protocol or hardware semantics I haven't formalised, and I've proved nothing about them. So none of this yields a new outside result that both needs the record axis and is ready to use.

Where each result lives. Every system that follows the toll, whatever its states: the floor, at most as many certifications as cost, and the potential-method reading. Every finite machine whose price covers merges: the toll for permanent records, and, when its move names can be decided equal or different, which instructions every such price must charge. Every finite machine paying per halving: the machine's own narrowing bound. Every machine extending a base with a driven, permanent record: the latch. Every window through which a certifying and a non-certifying step look alike: no exact price from the window. Every machine meeting the definition of Thiele-complete: a free move, at least three marks on any run from a clean state that ends certified, and a final window that doesn't determine the record or the ledger. Any eight counts: the integer check and its PSD bridge, which are mathematics. This machine only: its six instruction kinds and their prices, its sixteen slots, its own colliding pair, and the encodings its universal programs use.

What I'm proposing beyond that. That the axis belongs in the step; that certification is the event worth recording; that this is a structural account of computation, with a cost model as one part of it. Those reach further than any theorem here, and no theorem here puts an unavoidable physical price on every observable change. They're the argument, and the argument is what the rest of the book was. What kind of claim is that? A proposal about which distinctions an account of computation ought to draw, and no proof settles an "ought". The proofs show the proposal hangs together, follows from premises I've named, and costs little (six instruction kinds on the small machine); whether anyone should adopt it is something I argue for and can't prove. Here's what would weaken it: if every use people have for the record turned out to be recoverable from a log the program writes plus a correctness proof about the program, or if the real systems keep choosing walls over tolls for good reasons, as the five in Section 14 mostly did.

26

Conclusion

I started with one word and a machine I couldn't explain. Here's where the word ended up, and where the machine did.

Structure, by the end of Part I, was relations among things, kept under chosen changes, carried by whatever stuff, always relative to a frame. The start I chose, the smallest one by the count of names, was a sorting of a pile, and the thing nobody I read had priced was the moment you stand behind the sorting. That was Part I, and it ended with a question.

The question got an exact form in Part II: five parts, one rule, one theorem you proved at the table. Then the rule stopped being only my preference. With finitely many yes-states and a flag that stays up, every move that raises it merges states, and if forgetting is charged for, the way Landauer's principle charges for it in its worst case, the toll follows. That's a derivation under three premises you can refuse one at a time, each with its counterexample built. It isn't a proof that every computation has to adopt the rule; and the charge on forgetting is a named premise with its critics named. Learning can be free; committing to a record that stays put can't be, on those premises. That's as far as physics goes in this book, and I've tried to keep it exactly that far.

Part III was the logic, and here's what it earned, in the order it was earned. The counter is arithmetic once the schedule is chosen. A window in which a raising step and a quiet one look the same can't price the raise exactly, by a fact about functions far older than the machine. Entitlement is defined as an object, a checked relation, and a paid event; given lists, a tree and a covering somebody supplies, a run that paid a charged step for every level of the tree has paid for the index bits it commits to (no run is shown to have to pay them, the run doesn't recover the tree, the lists count repeats, and the index bits aren't the questions anybody asked); on every Thiele-complete machine the same count, for a run from a clean start, is a theorem about its record moves, with the earned chain inside the run, and the count can't do without the covering; and it is a license to lean on a claim. It doesn't prove the claim true. Insight, in the sense the toll prices, is a committed relation. A feeling doesn't count. Its floor is flat: at least one, however deep. The claim behind the flag is something a check could have refused, and in the bare model nothing makes the commit wait for the check. Over any base, a base-driven record is a latch, with

two free choices, the event and the price list; "only one substrate" is false and "one structure, many price lists" is true once the event is fixed. No procedure the substrate itself can represent decides the shortcut predicate, given a recursion theorem, by the same diagonal that broke Frege's floor, and the machine's answer is to record the commitment. It leaves the judgement unautomated. Of five record disciplines I modelled in fragments, one charges a fee at the write. And which event to record is a thesis, with its refutations listed.

Part IV built one small machine, two counters and six instruction kinds, with hosts to run it, and on it the model stands whole: from a clean start, a raised flag was checked, then committed, then certified, on the same edition of the same counter, nothing gets forged, the cheapest run to a raised flag adds exactly three to the ledger, and a run whose check could have failed reaches that three (the step doesn't police trivial checks), and it computes everything Minsky's two-counter machines can, so its halting is undecidable. Earned commitments and universality are proved there. So is what Thiele-complete has to mean: a universal base that's free and can't touch the record, a record that stays up once it's up, an earned record, one mark for each check, commit and certificate, and some claim whose check can fail. Every clock, every machine that charges for computing, fails it; the small machine meets it; and every machine that meets it hides its record and its ledger from its own window. And so is a universal Thiele machine, with its scope said plainly: one fixed program, U, written in the small machine's own six kinds of instruction on a host with more counters, with no guest step built in, that runs every small-machine program from every clean start start(x, y), halts exactly when its guest halts, raises its own flag by its own check, commit and certificate if and only if the guest's rises somewhere in its run, and keeps a ledger equal to the guest's at every matching point. And then the bigger version of the same thing: with one paid move added, one fixed program, U_P, runs every computably presented Thiele machine, a machine from anywhere whose driver, step, prices and reading are computed by programs. It halts when the guest stops, raises its own flag on its own earned chain exactly when the guest's reading has said yes, with the guest's own reading as the claim it checked, and, when the guest's reading starts at no, keeps the guest's prices to within a surcharge of at most two, which no program on that host, run from a clean start, can avoid when the guest certifies with its whole ledger below three. An ordinary programmable computer, as built, doesn't do that for the programs it runs; U and U_P show a fixed interpreter can. All of that is proved in Coq with no axioms: each file prints the assumptions of every headline theorem, and every line comes back closed.

Then a check worth earning, the two-player game with an integer test whose pass forces Tsirelson's bound for the tally it's handed. That one does use axioms, the standard library's: functional extensionality and the two decidability principles of

its real numbers, because the bound is a statement about real numbers. And what the counter has to do with physics, which is less than the word Landauer in a file name suggests. The places where the machine came up short of the idea stayed on the page: its free steps merge states, so the chairs don't force the toll on it, and only the merge premise asked of the record layer does; and a check that can't fail certifies for three like any other. Part V was how to check it and how to break it. Every result carries the shape of the thing that would kill it, gathered in one place (Section 23): a state of the small machine and an instruction after which the ledger goes down; a contractive correlator whose pinned matrix isn't positive semidefinite; a certification system that follows the toll and certifies at total cost zero; a function on numbers that correctly decides the shortcut predicate of the substrate built from it. Any one of them brings down the theorem it matches. I hunted. I came up empty. For a theorem Coq has checked, that's no surprise: finding one would take a bug in the checker. The places a reader can really win are where my sentence says more than the statement does, or a definition doesn't catch the word it's named for, and Section 23 starts with those.

If you asked what's new here, I wouldn't give you one thing, because there isn't one. There are four, and each comes with its fence and its neighbours. First, the toll isn't only my preference: on a finite machine a reading nothing turns off can only be turned on by a step that merges states, so if merges are priced, raising it costs (Theorem 4.1, Observation 4.2). The counting under it is the finite cousin of the recurrence argument Zermelo threw at Boltzmann in 1896, and the price on a merge is Landauer's, as a premise. Ouldridge and Wolpert (2023) had already put a physical price on that merge for finite automata, so what's here is the toll as a rule on a ledger. Second, a window that shows a raising step and a quiet one the same can't price the raise exactly (Observation 7.2); that a price computed from what a window shows can't tell apart two steps the window shows the same is elementary, and what's here is the pricing form. Third, a record driven by the computation that never switches off is a latch on one event, over any base (Observation 10.1); latches and history variables are old, and the theorem covers only records that never come down. Fourth, a record that's priced, and on the small machine earned (Theorem 16.9), under the move of guaranteeing something about every program. That move isn't mine at all. It's the reference monitor's (Anderson, 1972), made precise for monitors that watch a run (Schneider, 2000), and verified in Coq for monitors checked at every instruction (Azevedo de Amorim and coauthors, 2015). What's put together under it here is built and checked, and its idea isn't new: the check bound to the edition it looked at, the toll paid on top, and universal programs that carry both to every program of the small machine and to every computably presented Thiele machine, earning their own flag as they go. Appendix C sorts all four with sources.

And two things I'd point at if you asked how far the checking reaches. Neither is a fifth new thing.

In the game: an integer check on eight counts forces a five-by-five matrix to be positive semidefinite, and from there the Tsirelson bound follows, and the chain never once reaches for physics. No Hilbert space, no operators, no observables. The route through the matrix is the first rung of a ladder Navascués, Pironio and Acín built in 2007. None of that route is mine, and the semidefinite road to Tsirelson's bound is older than that. What's here is the check that runs in whole numbers, and the checked link from "the check passed" to the bound. It's algebra you can follow line by line, and the assumption audit on that link finds exactly three standard axioms from Coq's own library (two that come with its real numbers, and functional extensionality) and nothing of mine, which is how anyone can see that no physics was smuggled in. And on the small machine's host a raised flag can stand on that check: from a clean start a raised flag contains a passing CHECK of the packed counts, a COMMIT and a CERTIFY, and the check passes exactly when every pair was sampled and the five-by-five is positive semidefinite (Section 20).

In the model: the diagonal rules out a correct decider whenever its flip can be expressed inside the substrate and the substrate supplies the fixed point; in L the fixed point is proved from the reduction rules and the diagonal is Rice's theorem. On the small machine's host, Kleene's recursion theorem and Rice's theorem are proved for its own programs, so the wall stands there by proof, with its premise a theorem: no host program decides a property of what a program computes that some program has and some lacks; such a property is undecidable in the library's sense (Section 16.18).

What this book doesn't settle, by my count. Whether the toll is the *right* thing to price. The five real systems mostly built a wall or a penalty for contradiction. Whether certification is the pointer event. Whether Landauer's critics are right, which would leave every theorem standing and one premise unowned by the world. And the question I'd most like someone to take from me: is there a smaller start than a partition, by a ruler they can state? I said in Section 2 it would change this book. It would.

And the two computers from the first pages, the sorter and the copier, holding the same bits. Walk the warehouse again. On an ordinary machine there's still no lamp for what the sorter did, and nothing in this book changes that. On the small machine there is one: a flag and a counter you can read off the state, and a theorem that, from a clean start, the flag can't be up unless some claim went through a passing check, a commit about the same edition and a certificate, in that order, where a claim can be any property, in a language with an exact equality test, whose checker is proved to mean what it says, "this counter, read as a list, is sorted" among them (`sorted_run_certifies_iff` and `sorted_run_refused_forever`, EarnedGeneric.v). That's the one lamp I set out to light. It's a lamp for whoever can read this machine's state. A "tests passed" sheet fits it only when the test is a checker proved to mean what it says; the sticker and the very sure AI from the first pages ask something else, whether to believe a report you didn't watch being made, and nothing in this book answers that (Section 25). Whether every machine ought to have that lamp is the argument. No proof here settles it.

So that's the Thiele Machine. An abstract model with five parts and one rule, an argument made with it that the usual account of a computation is a shadow and the axis it leaves out belongs in the step, and one small machine to check the argument against, with one fixed program, written in its instruction kinds plus one paid move and run on a host with more counters, that runs every machine of the model whose workings a program can present. The theorems are what they are, with their premises printed. The interpretation is mine, and I've tried to say at every turn which is which, because the first time through I was the only one checking, and I've met me, and that's not enough people. Now there's you.

A

Things I had to learn first

I came in with none of the formal fields this work turned out to need. By the time the proofs compiled I'd needed enough of half a dozen of them that I had to put down somewhere what those fields had words for. This appendix is that place. None of it is the argument. It's what I needed to be able to make it. Everything below is what I understood after I went and read about it; I'm not claiming to teach you these subjects, and if you already know a part, skip it.

A.1 Bits, bytes, and what a CPU sees

A bit is one digit that's either zero or one, the smallest fact a machine can hold. Everything else is bits stacked up. Eight of them make a byte, which lands you any number from 0 to 255, and the first thing that genuinely surprised me is that the same byte is also a letter: a convention called ASCII simply agrees to call 65 the letter A, 48 the digit 0, 32 a space. Nothing in the byte knows which it is. The meaning is a convention laid on top.

You can see the convention's seams with a pencil. Write 65 in binary: 1000001. Knock off the first two digits and what's left, 00001, is 1, and A is the first letter. B is 66, which ends in 00010, two. Now lower case: a is 97, which is 1100001, the same pattern with one more bit switched on, the one worth 32. Every capital and its small letter differ in exactly that bit. That's on purpose. The first version of the code, in 1963, had no lower case at all; when lower case arrived, in the 1967 revision, it was placed so the two cases differ by one bit, which made matching them, and building keyboards and printers, simpler. Nothing in the byte knows it's a letter, but somebody made the letters easy for a circuit to tell apart.

Everything in this subsection is about real CPUs. None of it belongs to the abstract model, which has a kind of state and a step rule and nothing about widths or slots, and the small machine of Part IV needs very little of it: its two counters have no width at all, and hold any whole number.

A word is a chunk of bits the CPU treats as one unit, in 2026 usually 64 bits (0 to about 18 quintillion) on a desktop or server and 32 (0 to about 4 billion) on many small devices. The abstract model doesn't care which width.

A register is a named storage slot inside the CPU that holds one word, the fastest place to read or write. Memory is a longer list of word-sized slots accessed by index, the address. The program counter is one specific register that tracks where in the program the machine is; after every instruction it moves on to the next instruction unless the instruction says jump. On the small machine it's called the line number. A stack pointer is a register pointing at the top of a region used for nested calls. A port is a named channel for talking to the outside world.

Modular arithmetic means counting wraps around: counting modulo 16, the numbers run 0 to 15, and 17 wraps to 1. Bitwise operations work on the bits of a value one position at a time: AND is 1 only when both bits are, OR when either is, XOR (exclusive or) exactly when they differ. A shift moves every bit left or right, which multiplies or divides by a power of two. Popcount is the count of one-bits in a value. Hexadecimal is base 16, digits 0 to 9 then A to F, written with a 0x prefix: 0xFF is 255.

A trap, on a CPU, means execution jumps to a fixed handler and the program stops advancing, the chip's "this went wrong." On the small machine a trap is one bit that goes up when an instruction refuses, and then the core stops moving. An ISA, instruction set architecture, is the list of every instruction a CPU recognises, each identified by an opcode, a numeric tag the CPU decodes. The small machine's has six. The abstract model has none; it has a step rule.

A.2 The notation the formal parts use

I don't think in these symbols, and I had to learn to read them. The natural numbers are 0, 1, 2, and so on, written with a double-struck N; the integers add the negatives; the reals are the whole number line. A function from a set A to a set B is written with an arrow. For a finite set, vertical bars around it mean its size; for a number, its absolute value. A big sigma is a sum over every member of a set. A delta before a quantity means the change in it. An upside-down A means "for every"; a backwards E means "there exists." A double arrow is implication; a double-headed one is "if and only if." A small circle between two functions is composition, apply the right one first. Angle brackets mean one of three things, and the sentence around them says which: a pair of numbers packed into one, $\langle m, n \rangle$ (Section 16.13); a dot product of two vectors; or, around a random quantity, its long-run average, which I picked up from the Bell-test papers. The quantum sense of a circled times never appears, because the game section does its work in plain integer arithmetic. A superscript T on a matrix is its transpose, rows and columns swapped, and I with a subscript n is the identity matrix.

A.3 Coq, in just enough detail

Coq is a proof assistant: a programming language whose type system is expressive enough that proofs are programs and the compiler checks them (Part V has the long version). A Theorem is a named statement followed by a proof; a Lemma is the same thing used for an intermediate result; a Definition introduces a named value; an Inductive type is one with a finite list of constructors, the only ways to build a value of it (Coq's natural numbers have two: zero, and successor of another natural). A tactic is a step inside a proof: induction (base case, then step), destruct (split into cases), reflexivity (prove a thing equals itself), simpl (compute), lia and lra and nra (arithmetic solvers for integers, linear reals, and polynomial reals), psatz (a certificate search for polynomial inequalities). Qed ends a proof and asks Coq to check the whole chain. Admitted is the keyword for "I claim this without proof," and it appears nowhere in the active tree. A Hypothesis inside a Section is a named premise that becomes an argument of every theorem that uses it when the Section closes; "closed under the global context" doesn't make it go away. Print Assumptions lists the global axioms a theorem depends on; on the kernel tier it shows only standard-library ones. A total function always returns a value, and Coq requires every function to be total; the small machine's step function is, traps included. "Parametric in K" means the proof works for any K; the small machine's table size, sixteen, is a literal constant, and the proofs are about that literal.

A.4 Category theory beyond Part I

Section 2 sketches objects, arrows, composition, identities, associativity, and the identity laws. Beyond that: a functor is a structure-preserving map from one category to another, sending objects to objects and arrows to arrows while respecting composition and identities; think of it as a translation rule between two worlds of structure. An embedding is a functor that throws nothing away, and faithful is the precise version for arrows: two distinct arrows never land on one. Conservative is the version for inverses: if the image of an arrow has an inverse, the arrow already had one, so the functor can't manufacture an isomorphism (an arrow with an inverse) that wasn't there. The two properties are different, and neither gives the other. Read the natural numbers under addition as a category with one object and an arrow for each number, and the same for the integers. The inclusion of the first into the second is faithful, since distinct naturals stay distinct integers. It isn't conservative: 1 has no inverse among the naturals, and its image has one, minus 1. The other way round, the functor from the integers onto the trivial one-arrow group is conservative, since every arrow of the integers already has an inverse. It isn't faithful: it sends every integer to the same arrow. An embedding needs faithfulness specifically. A

monoidal product combines two objects or two arrows into one, with a unit and laws that make the combining behave. An initial object has exactly one arrow into every object; the counter's uniqueness in this book is a uniqueness-on-reachable-states fact, and no category-theoretic initial object is involved. Yoneda's lemma is where the categorical engine first clicked for me, the night I quit borrowing other people's runtimes (Section 1), and nothing in the formal development uses it.

A.5 Bell-test vocabulary

A correlation between two outcomes is the average of their product; with outcomes plus or minus one it lies between minus one and one, plus one meaning always match, minus one always differ. A correlator is one such average in a labelled setting, E00 for Alice's setting 0 and Bob's 0. The marginal of a joint distribution is the distribution of one side alone with the other averaged out; zero-marginal means each side's outcome is plus or minus one equally often on average. A local hidden-variable model is one where both outcomes are fixed by a shared random variable agreed before the game plus each side's own setting; Bell's theorem says no such model produces certain correlations quantum mechanics does. No-signalling means neither side's marginal depends on the other's setting. Quantum-realisable means compatible with some quantum state and measurements; the NPA hierarchy gives a ladder of necessary tests, each a positive-semidefiniteness condition on a moment matrix, and no single rung is known to be sufficient in general (Chaturvedi's 2026 preprint argues none is, even for this game). The PR box is the no-signalling correlation that wins the game every time, stronger than quantum mechanics allows. Cauchy-Schwarz is the inequality that a dot product is bounded by the product of the lengths. The Tsirelson bound is two root two, the largest game score quantum mechanics allows; the classical bound is two and the PR bound is four. (If you go looking for Boris Tsirelson's 1980 paper, look under Cirel'son, the spelling the journal printed, or you may not find it.) The elliptope is the set of correlators that can be filled out into a positive-semidefinite matrix.

A.6 Other terms I use without rebuilding

Provenance is the recorded chain of where an object came from; on the small machine, a commitment's provenance is the passing check that came before it (Section 16). Simulation, in computer science, is one system imitating another's input-output behaviour; a Turing machine simulates any system of the model whose step a program can compute, my machine included, by writing the whole state out on its tape, and the difference this book is about is what a window on that tape shows. That's real, and it raises a fair question: if a tape can stand in for the state, what was

the difference? The tape carries a picture of the state, the way a photograph carries a fire, and a photograph of a fire keeps you exactly as warm as any other photograph. The counter and the toll live in the step rule. An encoding copies the bytes, and the step rule isn't in them. A Turing machine built with the toll in its own transition table is a different case: it's an instance of the model, and it simulates nothing. What no machine can do is make a simulated rule binding just by writing it down on tape.

Projection is a function that drops information on purpose. The window on the small machine keeps three things, the line number and the two counters, which are what Minsky's machine already has, and that's why projecting lands you back in ordinary computation. It drops everything else, including the parts doing the new work: the versions, the table of checked facts, the channel, the ledger and the flag. Nothing is wrong with it. It's a perfectly good function. It just costs you what made the state more than a pair of counters. A quotient is the operation of declaring two things the same when they agree on what you chose to look at and working with the classes instead. A projection does exactly that. Dropping fields one at a time also glues together states that the dropped fields kept apart. The fibre of a function over a value is everything the function sends to that value. An injection sends no two inputs to the same output; a surjection covers every output. Decidable means an algorithm always halts with a yes-or-no answer; keep the claim and the algorithm apart, since the algorithm decides the claim only once there's a proof that its answer matches. Polynomial time means running time bounded by a power of the input size; P is the problems solvable that way, NP the problems where a candidate solution can be *checked* that way, and whether the two are the same is open.

"Kernel" wears three hats in this book, so here they are before one of them ambushes you. First, mathematics: the kernel of a map is what it sends to the identity, and for a projection the useful version is the relation "these two states project to the same thing." Second, the proofs, where the hat comes in two sizes: the kernel tier, the part of the Coq development where the model is written down and its main theorems live (Section 22), with the link files that tie the small machine to them; and Coq's own kernel, the small checker that accepts or rejects every proof. Third, the operating-system kernel, which I never use and mention only so you know it isn't meant. The sentence around the word always says which one is.

A.7 Matrices

A matrix is positive semidefinite (PSD) when $v^{\mathsf{T}} M v \geq 0$ for every vector v; the game check is all about one matrix being PSD. The determinant of a square matrix is a single number that is zero exactly when the matrix squashes some direction flat. For a two-by-two matrix with rows (a, b) and (c, d) it's $ad - bc$. For a bigger one,

expand along a row: for each entry, delete its row and column, take the determinant of what's left, multiply by the entry, alternate the signs, and add up; the recursion bottoms out at two by two. A principal submatrix is what you get by choosing some of the indices and keeping only those rows and the same columns; its determinant is a principal minor. A symmetric matrix is PSD exactly when every principal minor is at least zero, every one of them, and the game check uses the three-by-three ones (Appendix F.4). The Schur complement turns a question about a block matrix into a question about a smaller one. For a symmetric block matrix with corner block P, off-diagonal block Q and other corner R, it's $R - Q^{\mathsf{T}} P^{-1} Q$, and when P is positive definite the whole matrix is PSD exactly when that complement is (Lemma F.8). Positive definite matters; invertible isn't enough. Take $P = -1$ and every other entry zero: the complement is zero, which is PSD, and the whole matrix isn't.

A.8 Borrowed ideas the book pushes on

These get their full story in the chapter that uses them; here's the one-line version for when you've forgotten which one Landauer was.

The pigeonhole principle: put more pigeons than holes and some hole holds two. Section 4's room with too few chairs is that, and so is the proof that a permanent flag can only be raised by a step that forgets something. Landauer's principle (Rolf Landauer, 1961): a step that wipes information, lowering the entropy of what a machine holds by some number of bits, gives off at least Boltzmann's constant times the temperature times the natural log of two in heat per bit. In this book it's always a premise with its name on it. Nothing here derives it (Section 4, Section 21). Entropy, in Claude Shannon's sense: for a spread of chances over states, roughly the average number of yes-or-no questions you'd need to pin the state down, so an even spread over n states has $\log_2 n$ bits. A spread, in this book, is just a list of chances that add up to one. Maxwell's demon (Section 5): a little creature that watches molecules and sorts fast from slow, seemingly for free; Charles Bennett's answer was that the measuring can be free and the wiping of the demon's notes is what costs. The potential method (Robert Tarjan, 1985): pay for a run with a quantity stored in the state, so a step's bill plus the change in the stored quantity is what you count; Section 6 reads the toll as that method with one particular potential.

The halting problem (Alan Turing, 1936, who essentially proved it without stating it in this form; Stephen Kleene stated it in 1952, and Martin Davis seems to have been the first to name it, in 1958, Section 13): no program decides, for every program and input, whether it stops. Kleene's recursion theorem (Stephen Kleene, 1938): for any transformation of programs that can itself be computed, some program behaves exactly like its own transform. Rice's theorem (Henry Rice, 1953): any property that

depends only on what a program does, and isn't true of all programs or of none, can't be decided by a program. The diagonal is the move under all three: build the thing that does the opposite of what the table says about it (Section 13). L is a small lambda calculus, a language made of nothing but functions applied to functions, which Yannick Forster and Gert Smolka built in Coq and which Section 13 uses as a tool. A hash is a short fingerprint of a longer piece of data, made so that finding two inputs with the same fingerprint is supposed to be hard; a hash chain folds each new record into the fingerprint of the last, and a Merkle tree folds them pairwise into one root (Section 3, Section 14).

A.9 Words this book builds

These are mine, or mine in the sense that I pinned them down for this book. Each is built properly in the chapter named; this is only a reminder.

A few words first, because everything else hangs off them and each has exactly one meaning. The *model* (or the abstract model) is a kind of state and a rule for stepping, plus exactly the extra pieces a given theorem asks for: a certification reading and the toll, or a counter and a price list, or a window, or programs written as states and a recursion theorem. The name Thiele Machine belongs to the model. Parts I to III live there, and every claim in them is about any system with the pieces it names. A *substrate* is the model's shape with whatever extension a theorem names, and nothing else; the word never means my machine, and never the side of a simulation that does the stepping (Let me be straight with you). *My machine* (also the machine I build, the build, the small machine, the smallest honest machine, Section 16) is the concrete machine of Part IV that fills in those pieces at once: two counters with their versions, a table of checked facts, a channel, a trap, a ledger and a flag, six instruction kinds and their prices, with its hosts and universal programs. The *scaffolding* is that machine and everything built on it, and Part IV lives there. *A Thiele machine*, lower case, is any system proved to be an instance of the certification system (Definition 3.1), five parts and the toll; the paper toys are, and so is the small machine. *Weakly Thiele-complete* means a Thiele machine whose base is Turing-universal; that bar is nearly empty, since a clock that charges one for every move pays the toll for any record. *Thiele-complete* means more (Definition 16.7): a universal base whose moves are free and can't touch the record, a record that stays up once it's up and is raised from a clean state only after a passing check, a commitment to the same claim and a certificate, one mark for each of those three acts, and a claim whose check, commit and certificate raise the record on some loaded starts and leave it down on others. Every clock-style machine fails it; the small machine meets it; and every machine that meets it hides its record and ledger from its own window. A *universal* Thiele machine is one fixed program that runs any machine of a stated class as a

guest, with the guest's record earned again and its toll paid by the host's own steps. The ones in this book are U (Section 16.13), a fixed program in the small machine's instruction kinds, with no guest step built in, that runs every small-machine program with halting and flag exact and its ledger equal to the guest's at every matching point, and U_P (Section 16.14), the same program with one paid move added, that runs every *computably presented* Thiele machine, one whose driver, step, prices and reading are computed by programs with the reading decided on every state, keeping its reading as a latch and, when the reading starts at no, its prices to within two, a surcharge no program on that host, run from a clean start, can avoid when the guest certifies with its whole ledger below three. The simpler host with a built-in guest step runs guests by construction and keeps a mirror of the guest's flag. A computer running the small machine isn't a Thiele machine; it implements one, inside bounds, the way a laptop implements a Turing machine. When a word below belongs to only one of the two layers, the entry says which.

A *partition* is a sorting of a pile into groups, nothing in two groups and nothing left over (Section 2). A *certification system* is a state type, moves, a step, a cost per move, and a yes-or-no reading, plus one rule, the *toll*: a move that turns the reading from no to yes costs at least one (Section 3). The files call that rule A2 in the numbering their certification records use (Appendix B). A move *merges* when two different starting states land on the same end; one that never does is *injective*, it forgets nothing. A reading is *permanent* when no move turns it from yes back to no (Section 4).

The *counter*, which I also call the *ledger*: in the model, a number carried in the state that starts at zero and that every step raises by exactly that step's charge (Section 6). My machine has exactly one, the ledger μ (Section 16). Two other things get called counters, and each chapter says which: Minsky's two bins on the small machine, and the eight counts that tally the game's rounds (Section 20). Neither is the ledger. A *price list* says what each move costs; the model takes it as given, and my machine's table of what each instruction costs is in Section 16. A *receipt* is a total that lives in the state, only climbs, equals the sum of the charges on every path including the failures, and is the only such total for its price list (Section 6 proves it for any system with a counter and a price list, `nec_f_conservation`, `nec_f_counter_monotone` and `nec_f_initiality`; on my machine the sum is a checked theorem too, Section 16). In the model there is just a *certification reading*, yes or no (Section 3); on the small machine it's the *flag*, set by a CERTIFY that passes and never cleared.

These belong to the small machine only (Section 16). A *two-counter machine* is Minsky's: two counters holding whole numbers with no upper limit, and two instructions: add one, and subtract one and jump if the counter is above zero, or else go on to the next line; already enough that nobody can decide in general whether one halts. The small machine is a two-counter machine plus HALT and three instructions

for the record, six in all: INC, DEC, HALT, CHECK, COMMIT and CERTIFY. A counter's *version* is its edition number: it starts at 0 and goes up by one every time the counter is written, so an edition never comes back. A *fact* is a property, a counter and a version together, such as "A is even, as of edition 3"; the properties come from a fixed language of three, *is zero*, *is even* and *is at least n*. The same theorems also hold for any property language with an exact equality test whose checker is proved to mean what it says, and one such language reads a counter as a list and asks whether it's sorted (`sorted_run_certifies_iff` and `sorted_run_refused_forever`, EarnedGeneric.v). The *fact table* holds up to sixteen facts, and the only way in is a CHECK that passed; nothing ever leaves it. The small machine's *channel* holds one committed fact or nothing, and COMMIT puts a fact there only if the table has it about the counter's current edition. With the flag up from a clean start it names a commitment earned that way, and a later COMMIT replaces it, as a two-commit run shows (`certified_channel_earned` and `channel_names_last_commit`, EarnedCore.v). The *trap* is a bit that goes up when an instruction refuses, and then nothing in the core moves again. A commitment is *earned* when a passing check put its fact in the table first, about the same edition, with no write to that counter in between; on the small machine every flag raised from a clean start is earned, and that's a theorem. CHECK, COMMIT and CERTIFY cost one each and everything else costs nothing, so from a clean start a run ending with the flag up adds at least three to the ledger, and a run whose check could have failed adds exactly three: CHECK *is zero* on A, COMMIT, CERTIFY, from a start where A is 0. The step doesn't police trivial checks; *is at least 0* passes every value and certifies for three too. The *host machine* is the small machine with a counter, called a *register*, for every whole number, and one property, Slot, true of a number that packs a property of the small machine together with a value it holds of. A *slot* is a register of the host used for nothing but holding one checked fact about the guest, and its *mirror* is a register beside it, safe to read, that says which claim the slot holds while its fact is live. U is the fixed program of 3728 host instructions that runs every small-machine program; it reads guest instructions out of one register, keeps the guest's counters in two others, and puts every checked fact in a slot, because on this machine reading a counter writes it. The universal program's *guest* is a program of the small machine, held as one number in a register of the host and run one instruction at a time by U's fetch and decode. U_P is U with one more handler, for PAY, an instruction that costs one and does nothing else; its guests are programs compiled from computably presented Thiele machines (Section 16.14).

A *window* is any function that shows you part of a state (Section 7); a *shadow*, said loosely, is what a window leaves you with. A *collision* is two steps that look the same through a window, before and after, where one raises the flag and the other

doesn't (Section 7). *Entitlement* is what a run has when it carries an object, a checked relation between that object and a claim, and a paid event that put the object in place: a license to lean on the claim. It doesn't prove the claim true (Section 8). A *latch* is a record that switches on the first time some event holds and never switches off (Section 10). The diagonal's *substrate* (Section 13) is the model's shape extended with programs, a run that may give nothing back, a counter that never falls, a way to write programs as states, the transformations the substrate counts as its own, and its own recursion theorem for those; the proof of the diagonal uses neither the counter nor the toll.

B

Every assumption in one place

This book leans on a small number of rules that nothing proves, plus a handful of record types that carry rules as fields. I've named each one where it's used. Here they are side by side, because a reader who wants to break this book should be able to see every place where it says "suppose," in one list, with what each "suppose" buys.

One thing to undo first, and it's my fault. The checked work numbers the rules its records carry with bare labels, A2, A3 and so on, as if they were axioms. The book uses "the toll" for the rule that matters and avoids the bare labels; this appendix maps the labels for anyone reading the files.

B.1 The records that carry a rule

Definition B.1 (Certification system)**.** The record `CertificationSystem` (UniversalCertificationCost.v) has five data fields, a state type, a move type, a step function, a cost per move in $\mathbb{N}$, and a yes/no reading, and one proof field, `cs_cert_costs`: a move that turns the reading from no to yes costs at least one. This proof field is the toll of Section 3, and in this record's numbering it is called A2. Every result stated "for any certification system" assumes it, because the record cannot be built without it.

Definition B.2 (Quantitative certification system)**.** The record `QuantitativeCertificationSystem` (QuantitativeNoFI.v) extends a certification system with a *witness*, a function W from states to $\mathbb{N}$, and a threshold $N \in \mathbb{N}$, and adds two proof fields: (A3, `qcs_cost_bounds_witness`) $W(s) + \mathrm{cost}(i) \geq W(\mathrm{step}(s, i))$, the witness grows by at most what the move cost; and (A5, `qcs_cert_threshold_witness`) a state that reads yes has $W(s) \geq N$. An optional extension, `QuantitativeCertificationSystem_full`, adds (A4, `qcsf_witness_nondecreasing`) $W(s) \leq W(\mathrm{step}(s, i))$.

Theorem B.1 (The floor raised to a threshold)**.** In a quantitative certification system, every run from a state with witness 0 (this starting condition is called A6) to a state that reads yes has total cost at least N.

Proof. Adding A3 along the run telescopes: the witness at the end is at most the witness at the start plus the total cost, so at most the total cost. A5 says the end witness is at least N. Neither A4 nor the toll is used. □

Checked as `universal_nfi_quantitative` (QuantitativeNoFI.v). A3 and A4 bound opposite directions, and neither implies the other: witness 1, cost 0, witness 0 satisfies A3 and breaks A4, and witness 0, cost 0, witness 1 does the reverse. The record is a general shape. The small machine's own floor of three is proved directly (Observation 16.16) and never goes through it.

Here are the labels side by side, so none of them is left without a meaning.

Label	In the certification records
A2	the toll: field `cs_cert_costs` of `CertificationSystem`
A3	the witness grows by at most the cost: field `qcs_cost_bounds_witness`
A4	the witness never drops: field `qcsf_witness_nondecreasing` of the full record
A5	yes needs a witness at the threshold: field `qcs_cert_threshold_witness`
A6	the run starts with witness zero: a hypothesis of `universal_nfi_quantitative`

B.2 The premises the derivations take

These are definitions used as hypotheses. Nothing proves them in general; each theorem that uses one says so in its statement.

Premise	What it says	Where it's used	Checked name
Finite	a list with no repeats contains every state	the room argument, Section 4	`finite_states`
Permanent	no move turns the reading from yes to no	the room argument	`permanent` (and `permanent_at` for one move)
Merge pricing	every move that merges costs at least one	the toll derived, Section 4	`merging_steps_priced`

Premise	What it says	Where it's used	Checked name
Halving price	each unit of cost pays for at most one halving of distinct states	the squeeze bound; narrowing along a run, Section 5	`compression_priced`
Entropy pricing	each move costs at least the bits it removes from any spread	the third road to the toll	`entropy_priced`
Landauer's principle	removing ΔH bits gives off heat $\geq k_B T \ln 2 \cdot \Delta H$	the heat floor; with $k_B T \geq 0$ (or > 0) as a separate hypothesis on each theorem	`landauer_heat`
Recursion (generic)	every transformation the substrate can represent has a fixed point	the diagonal, Section 13	field `recursion_theorem` of the class `Substrate`
Commitment contract	a commitment bit is set only when the claim holds, and whenever it holds	the commitment escape, Section 7	stated in Definition 7.1
Computably presented	four programs compute the driver, the step, the price and the reading, and the reading's program answers on every state	the universal host for presented machines, Section 16.14	`cg_presentation`
Clean start	an empty fact table, an empty channel and the flag down	every provenance theorem about the small machine, Section 16	`clean_start`

Three remarks on the table. First, the recursion premise holds by construction for the nat family and by proof for L (Section 13) and for the programs of the small machine's host (Section 16.18). Second, the class `Substrate` (Substrate.v) carries a different rule from the toll, `mu_monotone`: the counter never goes down on a step. Neither implies the other, and the diagonal uses neither. Third, merge pricing is Landauer's principle taken in its worst case over what the machine might be holding: it charges every merging move, while the principle in its general form charges only

for uncertainty actually removed, when the step can be driven by a protocol timed from outside it (Theorem 4.13). Under time-symmetric control, Riechers, Boyd, Wimsatt and Crutchfield (2020) argue, even a known state pays for a move that doesn't undo itself.

B.3 The axioms of the checker itself

Beyond the premises above, the checked proofs rest on Coq's own foundation and on a handful of standard-library axioms, which the assumption audit lists theorem by theorem: functional extensionality on dependent functions; two decidability principles for the real numbers (used wherever real analysis is, for example in the game check of Section 20); and the law of excluded middle, in some theorems. No axiom or parameter of the project's own appears in the audited theorems. Section 22 says what that does and doesn't mean.

C

What's mine, and what isn't

Every chapter of this book leans on somebody, and I've named them where I lean. This appendix does a different job, and it's the one where I'd most like to be caught out, and caught out early. For each thing the book could be accused of claiming as its own, it says what was already known, what this work actually defines, builds, proves, or connects, and what kind of contribution that is. Six kinds, and they aren't the same claim: a *new result* (a statement nobody had); a *new proof* of something known; a *mechanization* (a known result checked by a machine, here Coq); an *implementation* (something built that runs); a *synthesis* (known pieces put together under one question); and a *conjecture* or *thesis* (a claim stated and not proved; a thesis ties defined notions to the world, so no proof can settle it). Every kind can be worth doing. None of them is the others. And if you read down the list and notice that "new result" hardly shows up, good. You're reading it right.

Two warnings about my own evidence before the list. First, "I didn't find it" is a fact about my searches. The literature is bigger than my searches. I searched with every tool I had, and some tools will cheerfully invent a citation if you let them, so where it matters I say where I looked, and you should assume it may well be sitting in a paper I'd need a few more lifetimes of homework to read. Second, nothing below is inferred from how many theorems there are, from the audit's clean bill, or from a good story about who came before. Those tell you the work was checked. They don't tell you it's new. Thousands of checked theorems could, in principle, be thousands of careful ways of agreeing with Tarjan.

C.1 The universal floor and the potential method

Known: the potential method of amortized analysis, a function on states whose drop bounds the cost of a run, is Tarjan's (1985), and it has been mechanized more than once (Nipkow in Isabelle/HOL, 2015 and 2019; Charguéraud and Pottier's time credits in Coq, 2019), built into type systems for automatic resource analysis (from Hofmann and Jost, 2003, and Hoffmann and coauthors, 2011, onward), and recast in cost-aware type theories (Niu, Sterling, Grodin and Harper, 2022). Mine: choosing the potential that is one when a designated reading says no and zero when it says yes, so that "the crossing costs" (the toll) and "every run that crosses paid" (the floor)

are the potential method's two halves (Section 6). Kind: a mechanization of a known technique with a particular choice of potential. The choice is the point: it makes "was certified" something the account must carry. The theorem is not new mathematics. If it looks clever, that's Tarjan's cleverness; my part was picking which number to plug in.

Go back to Tarjan's paper and there's a sentence that sits right on the line this book draws. He opens with the dictionary (to amortize is to put money aside at intervals, as in a sinking fund) and then gives two views of the method. In the banker's view the computer is coin-operated, and each operation is handed credits to spend or save; in the physicist's view, which he credits to Daniel Sleator, every state has a potential and you count its changes. Then he stops to say that the credits are only an accounting device: the programs that actually run contain no mention of credits or debits. That's exactly right for what he wanted, a bound proved about a program from outside. It's also exactly the gap. In his method the coins live in the proof. The one move I chose was to put them in the step, where no program takes a step without it being counted. Same arithmetic, different place.

C.2 The toll from finiteness, permanence, and a price on merges

Known: that steps which merge states (no single-valued inverse) are what Landauer's principle charges for is Landauer's (1961), sharpened by Bennett (1973, 1982, 2003; the words "the erasure of a bit or the merging of two computation paths" are from the 2003 abstract); generalizations and the dependence on the input distribution are Maroney's (2009), Sagawa's (2014), Wolpert's; the critique is Earman and Norton's and Norton's. Write-once memory is an old topic in coding theory (Rivest and Shamir, 1982), where the question is reuse and price never comes up, and Chris Heegard (1985) worked out the capacity of permanent memory with noise, again a question of how much fits. What it costs doesn't come into it. Statisticians priced permanence on finite memory in their own currency: in Hellman and Cover's analysis of hypothesis testing with finite memory (1970), a machine that settles its answer by falling into states it never leaves does worse than one that doesn't, so a decision held for good costs memory states. Heat never enters their analysis. And the counting under the theorem is older than any of that, twice over. Theorem 4.1 is the finite, discrete cousin of the recurrence argument Zermelo, using Poincaré's theorem of 1890, turned on Boltzmann in 1896, here aimed at a reading. Zermelo aimed it at entropy, and Bocchieri and Loinger gave the quantum recurrence in 1957. In automata form it is classical: a finite machine whose moves are all one-to-one reshuffles recognises only group languages, and the inputs that raise a permanent reading form a language that is never one of those unless it is empty (Pin's 1992 paper on reversible automata is the reference I'd check; automata theorists treat the

step as folklore), and going quantum doesn't change it (Brodsky and Pippenger, 2002: quantum finite automata that measure once, at the end, accept with bounded error exactly the group languages). The closest of all is recent. Ouldridge and Wolpert (2023) give a proof that if the smallest automaton for a language has a step that merges states, every automaton for it does, and argue that recognising it on a device that repeats one local step produces entropy, while a language whose smallest automaton only reshuffles can be recognised for free; one of their worked examples is the strings with no b, the complement of the latch's language. That's the core with a physical price already on it. And the logarithm in the heat floor matches the absolute-irreversibility bound of Murashita, Funo and Ueda (2014), which Manzano, Kardeş, Roldán and Wolpert (2024) carried over to computation. The match is a mirror image. It isn't a special case of their bound: the squeeze is ordinary Landauer heat, whose logarithm equals their bound for the free expansion that undoes it, and absolute irreversibility reaches the commit itself only on a fixed-rate device starting from a known state. Constructor theory states a cousin of the same core for devices that run in a cycle: Marletto, Vedral and coauthors (2022) and Violaris and Marletto (2022) built an irreversible task out of the quantum homogeniser and argued that erasure may cost more than Landauer's bound. Section 4 tells all of it. Mine: the price as a named rule on a ledger, a premise that merges cost at least a mark, which turns "the flip merges" into "the flip costs" on any machine whether or not anybody's physics is right, and the toll derived from it (Observation 4.2); what comes with that form, namely the characterisation of the moves every merge-pricing schedule must charge, when move names can be decided equal or different (the merging moves, so the core again in pricing words), the merge-or-revoke dichotomy for a single move, the squeeze and entropy forms for a permanent flip, and a built counterexample for each premise; and the mechanization of all of it (Section 4). Kind: the core is classical, and somebody else's: Zermelo's in physics, folklore in automata theory, with Pin's paper as the place to check it. The pricing form isn't wholly mine either: priced in entropy, for automata, it's Ouldridge and Wolpert's. They derive the price from the physics. I assume mine. What's mine is the price as a named rule on a ledger, the toll derived from it, its single-move and counted forms, the counterexamples, and the mechanized proofs, and the physics is a named premise and is Landauer's. Searches: the core turned up in the places just named, and its physical price in Ouldridge and Wolpert; I didn't find the ledger form or the derived toll stated anywhere I looked, and nobody's search covers a whole field, mine least of all; I found no mechanization of Landauer's principle in Coq, Isabelle, or Lean.

C.3 Learning free, committing priced

Known: that measurement can be done at no fundamental cost and the cost sits in erasing the record is Bennett's resolution of Maxwell's demon (1982), with the quantitative trade-off by Sagawa and Ueda (2009). The closest physics I found to the line between learning and committing is Ouldridge and ten Wolde's fundamental costs of persistent copies (2017; with Govern, 2017), copies that stay correlated with their originals after the two come apart; theirs is heat for a correlation in chemistry, and Section 5 says how it differs. The sharp chemical form, that a persistent copy grown at equilibrium carries no information about its template, is Poulton, ten Wolde and Ouldridge's (2019), and that the bill can be split between measuring and erasing in more than one way is Fahn's (1996). Mine: the four-state demon and the three-state ring as checked models, and the two-state impossibility (Section 5). Kind: mechanization and illustration of a known physical point inside my ledger. Not a new physical claim. Bennett did the hard part in 1982; I built a toy with four dots on it and got Coq to agree with him.

C.4 Windows that can't price what they hide

Known: a function of an observation is constant on the observation's fibres; that's elementary, and the systems version of it is Moore's distinguishing experiments (1956), Kalman's observability (1960), and the completeness question of abstract interpretation (Cousot and Cousot, 1977; Giacobazzi, Ranzato and Scozzari, 2000); in control theory it is diagnosability and opacity of discrete-event systems (Sampath and coauthors, 1995; Saboori and Hadjicostis, 2007), where a collision is a hidden event no observer can diagnose without delay; in economics it is the hard-evidence view of mechanism design (Green and Laffont, 1986; Bull and Watson, 2007), where what a mechanism can tell apart is limited by the evidence on offer; and its sharp form for fingerprints is Naor and Rothblum's lower bound for checking a stored file (2005); in distributed systems it is Haeberlen and Kuznetsov's (2009), that only omission and commission faults can be detected from outside, and in philosophy it is Easwaran's transferability (2009). Mine: stating the pricing version (no window price is exact across a collision), exhibiting a colliding pair for the small machine's window, and the theorem that every Thiele-complete machine has two runs from one clean start that end looking the same through the window its base shows, one certified and one not (`complete_hides_record`; Section 7, Section 16). Kind: mechanization of an elementary lemma, plus specific facts about the machine and the definition.

C.5 Structural entitlement and its bound

Known: decision-tree counting bounds (a tree that tells N cases apart has depth at least $\log_2 N$); certifying algorithms that hand over checkable certificates (McConnell, Mehlhorn, Näher and Schweitzer, 2011); proof-carrying code (Necula, 1997); the gap between justified true belief and knowledge (Gettier, 1963). Mine: the definition of entitlement as an object, a checked relation, and a paid event on the record, and the count that, given lists, a tree and a covering, a run that paid for the tree's depth pays at least the index bits it commits to (Section 8). Kind: synthesis; the counting idea is standard, the assembly and its vocabulary are this work's, the pricing policy is assumed (nothing derives it), and the count is mechanized: in its bill form over any certification system, and for the record moves of every Thiele-complete machine on a run from a clean start, with the earned chain in the run, a floor that takes the run's own checks in place of a tree, and a counterexample showing the covering can't be dropped (Section 16.17); and in a form where the run makes the posterior from a prior, every drop is made by a check the run passed, and the index bits are at most k times the bill when each passing check keeps a 2^{-k} share, a hypothesis the same counterexample shows is needed (Theorems 8.4 and 16.45).

C.6 The record axis is a latch

Known: monotone, write-once state is old: the latch (Eccles and Jordan, patent filed 1918, paper 1919), sticky bits (Plotkin, 1989), grow-only state in replicated data types (Shapiro and coauthors), history variables and stuttering in refinement (Abadi and Lamport, 1991), append-only authenticated sets for transparency logs, which fit revocation as a once-only event (Tomescu and coauthors, 2019), and, in program logic, monotonic state, where a property witnessed once on state that only moves forward can be recalled at any later point, with a soundness proof and a working home in F* (Ahman, Fournet, Hriţcu, Maillard, Rastogi and Swamy, 2018): the type-theory name for a record that stays up. Past the finite case, inductive and quasi-inductive definitions (Richter and Aczel, 1974; Burgess, 1986) and the flag trick of infinite time Turing machines (Hamkins and Lewis, 2000) are the transfinite versions of a record that only grows and a toggle, and Danos and Krivine's (2005) reversible calculus needs an action declared irreversible in order to commit. Mine: the exact statement that a record driven by the computation that never switches off is, over any base, a latch on one event, with its converse and a counterexample for each of the two conditions, and the argument that base-driven records agree up to the price list while the stronger uniqueness fails (Section 10). Kind: a small structural lemma, I think not stated before in this form (I found no prior statement

in the searches I ran), built from old notions; the latch form is mechanized, and so is the agreement up to the price list (`nec_w_billed_same_up_to_schedule`).

C.7 The diagonal and L

Known: Kleene's recursion theorem and Rice's theorem; Lawvere's fixed-point form (1969) and Yanofsky's retelling (2003); mechanized recursion theorems and Rice (Norrish in HOL4, 2011; Carneiro in Lean; Forster, Kunze, Smolka and coauthors in Coq, including L itself, 2017, and the Coq Library of Undecidability). Mine: the diagonal stated once over any substrate that supplies a recursion theorem; and Rice and halting proved again for L from its reduction rules (Section 13); and the same two theorems built for the small machine's host, with Rice's on every input of the two-counter machine and Kleene's with inputs written as powers of two, and Kleene's refuted for plain inputs (Section 16.18). Kind: checked instances of known theorems of computability. They hold for any model with universality and s-m-n, and the work is in building those for this machine.

C.8 The game check

Known: that positive semidefiniteness of a correlation matrix gives the Tsirelson bound is Tsirelson's (1980) and Stephanie Wehner's semidefinite proof (2006), from a paper on Tsirelson bounds for generalized CHSH inequalities; the moment-matrix hierarchy, the Schur-complement analysis of this very level-one matrix and the level $1 + AB$ are Navascués, Pironio and Acín's (2007, 2008), with both four-body cells free where my matrix ties them; that every point with a PSD completion is the correlator table of a quantum strategy, built from anticommuting observables in a maximally entangled state, is Tsirelson's too (1980); the arcsine characterization is Landau's (1988) and Masanes's (2003); and Tsirelson's bound has already been checked by machine, in Isabelle/HOL (Echenim and Mhalla, 2023) and in Lean's mathlib. Rational sum-of-squares certificates are Helfried Peyrl and Pablo A. Parrilo's (2008), from a paper on computing sum-of-squares decompositions with rational coefficients. Mine: a decision procedure in integer arithmetic on outcome counts; a checked chain from "the counts pass" to "the pinned level-one matrix is positive semidefinite" to "$S^2 \leq 8$"; checked proofs of the representation in finite dimension, real or complex, on two eight-level systems, and of the arcsine characterization; proofs of the contraction equivalence by explicit test vectors; and the level-$1 + AB$ cascade (on the slice $\langle \{A_0, A_1\} \otimes \{B_0, B_1\} \rangle = 0$) and an elliptope gate in whole numbers, with a soundness proof (Section 20 and its derivation appendix). Kind: mechanization, with a new proof of a known equivalence in one slice. I found no prior formalization taking counts as input, and that's a report on where I looked,

and the literature is bigger than where I looked. The same check earned on the small machine's host, a raised flag standing on it, is built and checked; the idea isn't new (Section 20).

C.9 Five record disciplines, one question

Known: each system and its guarantee belongs to its designers: Casper (Buterin and Griffith, 2017), Ethereum's storage gas and refunds (EIP-2200, EIP-3529), extend-only TPM registers, Certificate Transparency (RFC 9162), proof-carrying code. Mine: asking all five whether the write is priced, with a small checked fragment of each (Section 14). Kind: synthesis and an organizing reading; the fragments are not verifications of those systems.

C.10 The pointer thesis

Known: Zurek's quantum Darwinism (2009), with Ollivier, Poulin and Zurek (2004) and Blume-Kohout and Zurek (2006): preferred states are the ones whose records proliferate redundantly in the environment. Mine: borrowing that shape for the question of which event deserves a record, with definitions (copied and relied on by parties who didn't produce it, independent carriers), two toys, and refutations of two stronger forms (Section 15). Kind: thesis. No claim about quantum physics.

C.11 The counter as a receipt

Known: one-way counters that give order and termination are old: Turing's 1949 note on checking a large routine, Floyd's ranking functions (1967), Lamport's clocks (1978). Carrying a run's cost beside its result is the writer cost of Danner, Licata and Ramyaa (2015) and of Danner and Licata (2022), and the graded effect semantics of Katsumata (2014), with a working-language version in Orchard, Liepelt and Eades (2019). The receipt side of time credits is Mével, Jourdan and Pottier's (2019): in their Coq-checked extension of the Iris logic every step uses up a credit and produces a receipt, receipts prove lower bounds, and enough of them prove that a counter can't overflow in any feasible time, which is the closest published neighbour of a total that only climbs. Mine: putting three properties side by side (the total only climbs, it equals the sum of the charges on every path including the failures, and it is the only such total for a given price list), the argument for any system with a counter and a price list, and the checked sum on the small machine (Section 6, Section 16). Kind: synthesis; the abstract statements are checked over any counter machine (`nec_f_conservation`, `nec_f_counter_monotone` and `nec_f_initialit y`), and on the small machine the sum is too. Against the graded frameworks the

answer is split: the counter is their writer exactly (`run_writer_is_run_and_cost`), and their type systems, bounding relations and before-the-run checking of grades are not embedded, because the grade here is written by the step during the run (Section 6). On loop-free programs the counter agrees with a fixed grade exactly, and where a run's cost depends on its data it holds the exact number that a fixed grade can only bound; that's argued in Section 6, with no checked theorem behind it, and cost-aware type theories such as calf get exactness back with grades that depend on the inputs.

C.12 The small machine

Known: two-counter machines are Minsky's (1961, 1967), and the undecidability of their halting is checked in the Coq Library of Undecidability (Forster, Larchey-Wendling and coauthors, 2019, 2020). The guarantee about every program is older than any of this, and it isn't mine either. It's the reference monitor (Anderson, 1972): a check that sits in the path of every operation, so no program gets around it. Schneider (2000) worked out what a monitor that watches a run can enforce: only safety properties, the kind a bad run breaks at some finite moment, and "no CERTIFY without an earlier earned COMMIT" is one of those. Azevedo de Amorim and coauthors (2015) verified, in Coq, monitors that check tags at every instruction. The shape of "earned," a certificate nothing can make except through the rules, is Milner's LCF, where a theorem is a value only the logic's rules can build (Gordon, Milner and Wadsworth, 1979; Gordon, 2000); the small machine is that architecture at the level of single instructions. "No certify without an earlier commit" is also a correspondence assertion in the sense of Woo and Lam (1993), which Gordon and Jeffrey (2003) check with a type system, and a store of commitments that the rules of a dialogue keep, each speaker's updated by the rules as the moves are made, is Hamblin's (1970, chapter 8, on formal dialectic); the difference from all three is again that here the step enforces it. A check that blesses a value and a write that takes the blessing away is close kin to dynamic taint tracking, whose labels go back to Denning's lattice model of information flow (1976), with the blessing itself, controlled endorsement, given a type system and a security theorem by Cecchetti, Myers and Arden (2017), and what a monitor can enforce with bounded memory is Talhi, Tawbi and Debbabi's question (2008). A small monotone record inside each replica of a replicated service is A2M (Chun and coauthors, 2007), TrInc (Levin and coauthors, 2009), MinBFT (Veronese and coauthors, 2013) and Damysus (Decouchant and coauthors, 2022), and Clement, Junqueira, Kate and Rodrigues (2012) worked out, with proofs, what such a record buys and what it doesn't. The signature route to the same goal, which needs no trusted step, is IA-CCF (Shamis and coauthors, 2021): replicas sign and log their protocol messages in the ledger, so blame lands

on individual members however many of them misbehave. And "check once, issue a capability the system trusts, rely on it later" is built and deployed in Garg and Pfenning's proof-carrying file system (2010), whose logic lets a policy's consequences change with time and state, and in Sirer and coauthors' logical attestation for the Nexus operating system (2011), unforgeable statements about programs used as credentials; those are the closest systems I've found to check, commit, certify. Mine: a machine of six instruction kinds on two counters, with a version for each counter, a bounded fact table and a channel; its theorems that, from a clean start, a committed fact was checked first about the same edition, that nothing is forged, that nothing is evicted, that with the flag up from a clean start the channel names a fact an earlier passing COMMIT committed, and a later commit can replace it, as a two-commit run shows (`certified_channel_earned` and `channel_names_last_commit`, EarnedCore.v), and that a run from a clean start to a raised flag adds at least three to the ledger, with three reached by a run whose check could have failed (the step doesn't police trivial checks, so one that can't fail costs three too); the same theorems for any property language with an exact equality test whose checker is proved to mean what it says, a sorted list read out of a counter included (`sorted_run_certifies_iff` and `sorted_run_refused_forever`, EarnedGeneric.v); and the step-for-step match with the library's machine, so halting is undecidable for it (Section 16). Kind: an instance of monitor enforcement, implemented and mechanized, with two things on top that I didn't find put together where I looked: the check is bound to the edition of the thing it checked, and a toll is paid for the commitment. The first of those alone is old: checking at commit time that nothing you read has changed since is optimistic concurrency control (Kung and Robinson, 1981), and each counter's version is its version number. What I didn't find is that check put together with a toll. The universality and the undecidability are borrowed, and the other properties are short consequences of the step rule. Whether the design is new I can't say, and I make no claim beyond the proofs.

C.13 The universal host

Known: Turing's universal machine (1936), which runs any machine given as data, and the universal register machines that run two-counter programs. Keeping a guest and the code running beside it out of each other's state is what a virtual machine monitor promises (Popek and Goldberg, 1974). Compiling computable functions into counter machines is checked in the Coq Library of Undecidability. Mine: one fixed program, U, written in the small machine's own six kinds of instruction on a host with a register for every whole number, with no guest step built in, that runs every program of the small machine from every clean start start(x, y), halts exactly when its guest does, raises its own flag by its own check, commit and certificate if

and only if the guest's rises somewhere in its run, and keeps a ledger equal to the guest's at every matching point; its host machine is Thiele-complete, and whether U halts on a loaded guest is undecidable (Section 16.13). Two small facts force its shape, and they're stated there: reading a counter on this machine writes it, and a finite program can't keep the guest's facts on a copy of the guest's counter, by the pigeonhole. Beside it, a simpler host with a built-in guest step runs guests by construction and keeps a mirror of the guest's flag, with the mirror, isolation and earned-chain theorems. An ordinary programmable computer, as built, doesn't do this for the programs it runs, because the code beside a program shares its memory and can flip anything; U is a fixed interpreter that does. Kind: implementation and mechanization of a design whose shape is Turing's universal machine, with the record and the toll carried through. The universality is borrowed from the small machine and from Minsky. And the same program with one paid move added, U_P, runs every computably presented Thiele machine, with its halting matched, its reading carried as a latch on the host's own earned flag whose claim is that reading, and its prices to within a surcharge of at most two when the reading starts at no, which no program on the host, run from a clean start, can avoid when the guest certifies with its whole ledger below three (Section 16.14); the compiler that carries the guest onto two counters is the library's. Kind for that part: implementation and mechanization again. That a universal machine runs any computable machine is Turing's; what's carried through here is the record, earned again, and the prices, exact up to the surcharge, and I didn't find that put together where I looked.

C.14 Thiele-complete, defined so a clock fails

Known: Turing completeness, and the observation every theorist makes in four seconds, that a toll alone is cheap: a machine that charges for every step pays any toll. Enforcement mechanisms have been compared and sorted before, and they aren't one notion seen several ways: James Morris (1973) separated access limitation from authentication in programming languages, and Kevin Hamlen, Greg Morrisett and Fred Schneider (2006) classified which policies execution monitors, program rewriters and static analysis can each enforce, and they differ; Wai Wong (1995), on a neighbouring track, recorded and checked HOL proofs so a theorem could carry its derivation. Mine: a definition of Thiele-complete with four clauses (a free universal base that can't touch the record and a record that stays up once it's up, an earned record, an exact toll on the three record acts, and some claim whose check can fail), the proofs that every clock-style machine and a paid latch and a silent machine fail it while the small machine, the same machine over any property language with an exact equality test, a proved checker and a property that holds of one number and fails of another, and the universal program's host meet it, and the theorem that every

machine meeting it hides its record and ledger from its own window (Section 16). Kind: a definition with short consequences. The window theorem is the fibre fact of item 4 plus one observation, that a universal base prints any window for free. I'm not claiming the definition is the only reasonable one, or that LCF kernels, reference monitors and proof-carrying code are all Thiele-complete machines in disguise; Hamlen, Morrisett and Schneider's sorting says such mechanisms genuinely differ, and the book doesn't have a theorem showing the definition holds up across such differences.

C.15 What that adds up to

Read straight down the list. The mathematics underneath this book is mostly known, and some of it is very old. What's mine is the question (should an account of computation carry what was certified and what certifying cost, as part of how it steps), and the answer isn't one result. It's four, and I name them the same way wherever they come up. The toll derived from finiteness, permanence and a price on merges (item 2; the counting is classical, the finite cousin of Zermelo's recurrence objection and folklore about permutation automata, and Ouldridge and Wolpert already priced it in entropy for automata, so what's mine is the price as a rule on a ledger and the toll it gives, and the price is Landauer's, as a premise). The pricing form of the window fact (item 4; the fact itself is elementary). The latch form of a record that never comes down (item 6; latches and history variables are old). And a record that's priced, and on the small machine earned, under the reference monitor's every-program move (items 12, 13 and 14; the move is Anderson's, Schneider's and the micro-policy authors', and what's added is the check bound to its edition, the toll on top, a definition of Thiele-complete that a clock fails, and universal programs that carry both to every small-machine program and to every computably presented Thiele machine, earning their own flag). The first is a ledger form of a core that's classical and already priced in physics; the second and third are small exact statements I didn't find stated where I looked; the fourth is built and checked, and its idea isn't new. Under all four sits a large body of mechanized proof that ties those statements to one small machine and the programs that run on it. Around it is my best attempt at saying which is which, which you should check as hard as anything else in here. Item 2's core, the finite cousin of Zermelo's recurrence objection, was folklore among automata theorists, with Pin's paper as the place to check it, long before I found it, and its price in physics was Ouldridge and Wolpert's. If someone shows me its ledger form, the pricing form in item 4, or item 6 stated earlier, I'll move it up the list and be glad to, and a little embarrassed, in that order.

D

Every result and where it is checked

Every numbered result in this book is in the table below, in the order it appears, with the Coq names that carry it, the file each lives in, and what the last full assumption audit printed for it. Four groups of rows are placed out of that order, beside the rows they belong with: Definition 16.11 and Theorems 16.41 and 16.42 beside the eight-state machine of Section 4, the rows from Theorem 16.43 to Theorem 16.50 beside Theorem 8.3, Theorem 16.33 and the two rows after it below the host's rows, and Remark 20.1 below the small machine's five tallies. "none" means the theorem is closed: its proof uses nothing beyond Coq's own rules. The other entries name the standard-library assumptions it uses: FE is functional extensionality on dependent functions, RD is one or both of the two decidability principles that come with the real numbers, and EM is excluded middle. "def" marks a definition, which has nothing to assume. The audit covers every file the project builds, the small machine's files under minimal/ and the kernel tree under coq/kernel alike; only MuCore.v and Napkin.v, which are built and checked on their own, sit outside it. The small machine's files, the universal program's files and the priced host's files also end by printing the assumptions of their own headline theorems, and every one of those lines says closed. "+N" after a file means *N* other files under coq/ and minimal/ define or declare a name spelled the same way, section variables and hypotheses included (notations, module aliases and record fields aren't counted); the row gives the file this result uses. A premise is assumed and never proved, so its row gives at most the Coq definition that states it, marked "def". A thesis has no Coq name, because the book states it and doesn't claim it; its formal schema is a definition, marked "def". The section that states each one says so.

This table is an index of companion formalizations. The statements, hypotheses and arguments are in the numbered sections it points to. An audit status reports the recorded check; running a later version does not change a claim in this book. Each row has one of three relations to its Coq names. Most are *equivalent*: one Coq statement, or one per numbered part, says what the numbered statement says. A few are *assembled*: the numbered statement follows from the listed Coq results by a step done in the text, and the row says so, as Corollary 20.3's does with "the composition

of." No row is meant to say more than its Coq names, and where the prose does, the prose is wrong. A name and its file fix the constant, the revision is the one in Section 22, and Section E prints twenty-one of the statements with their premises and that comparison made.

Result	Sec.	Coq name and file	Axioms
Definition 2.1: Equivalence relation, partition	§2	stated in the text	
Theorem 2.1: Sameness and sorting are one object	§2	`nec_t_rel_of_partition_equiv` (`NecTPartition.v`) `nec_t_blocks_of_equiv_partition` (`NecTPartition.v`) `nec_t_rel_round_trip` (`NecTPartition.v`) `nec_t_blocks_round_trip` (`NecTPartition.v`)	none none none none
Definition 3.1: Certification system	§3	`CertificationSystem` (`UniversalCertificationCost.v`, +1)	def
Observation 3.1: The universal floor	§3	`universal_nfi_any_substrate` (`UniversalCertificationCost.v`) `every_toll_system_pays_the_floor` (`UniversalCertificationCost.v`) `cs_state` (`UniversalCertificationCost.v`) `cs_instr` (`UniversalCertificationCost.v`) `cs_step` (`UniversalCertificationCost.v`) `cs_cost` (`UniversalCertificationCost.v`) `cs_cert` (`UniversalCertificationCost.v`) `cs_cert_costs` (`UniversalCertificationCost.v`)	none none def def def def def def
Definition 4.1: Finite, permanent, injective	§4	`permanent` (`PermanentCertification.v`) `step_injective` (`PermanentCertification.v`)	def def
Theorem 4.1: A permanent flip merges	§4	`permanent_flip_is_not_injective` (`PermanentCertification.v`)	none
Premise 4.2: Merge pricing	§4	`merging_steps_priced` (`PermanentCertification.v`)	def

Result	Sec.	Coq name and file	Axioms
Observation 4.2: The toll from finiteness, permanence, and merge pricing	§4	`a2_from_merging_price_and_permanence` (`PermanentCertification.v`) `permanent_certification_trace_floor` (`PermanentCertification.v`) `certification_system_from_merging_price` (`PermanentCertification.v`)	none none def
Theorem 4.3: Merge or revoke	§4	`flip_merges_or_revokes` (`PermanentRecordPricing.v`) `permanent_at_flip_is_not_injective` (`PermanentRecordPricing.v`)	none none
Definition 4.3: Forced to be priced	§4	`forced_priced` (`PermanentRecordPricing.v`)	def
Theorem 4.4: Forced price is merging	§4	`forced_priced_iff_merges` (`PermanentRecordPricing.v`)	none
Corollary 4.5: Permanent records are forced	§4	`forced_priced_iff_merges` (`PermanentRecordPricing.v`) `permanent_record_write_is_forced_priced` (`PermanentRecordPricing.v`) `forced_price_without_permanent_record` (`PermanentRecordPricing.v`)	none none none
Premise 4.4: Halving price	§4	`compression_priced` (`PermanentRecordPricing.v`)	def
Theorem 4.6: The squeeze bound	§4	`a2_from_compression_price_and_permanence` (`PermanentRecordPricing.v`) `permanent_flips_compression_bound` (`PermanentRecordPricing.v`) `permanent_flips_log_bound` (`PermanentRecordPricing.v`) `permanent_flips_collapse_at_least` (`PermanentCertification.v`)	none none none none
In the text: an eight-state machine meets every premise	§4	`fin_finite`, `fin_permanent`, `fin_merging_priced`, `fin_a2_from_merging_price`, `fiber_bound_compression` (`FiniteCertMachine.v`)	none

Result	Sec.	Coq name and file	Axioms
In the text: on the eight-state machine, each move's price is the halvings in its largest squeeze; on a finite machine, the halving price holds exactly when no pile-up exceeds 2^{cost}	§4	`frag_price_is_squeeze` (`FragmentSmall.v`) `ent2_compression_priced_iff_fibres` (`CompressionSmall2.v`) `ent2_frag_compression_priced` (`CompressionSmall2.v`) `ent2_frag_toll_by_compression` (`CompressionSmall2.v`) `ent2_frag_cost_minimal` (`CompressionSmall2.v`) `ent2_frag_bound_attained` (`CompressionSmall2.v`)	none none none none none none
Definition 16.11: The eight-state machine, compiled	§16	stated in the text	
Theorem 16.41: The small machine runs the eight-state machine	§16	`frag_runs_finite_machine` (`FragmentSmall.v`) `frag_pays_finite_price` (`FragmentSmall.v`) `frag_runs_finite_trace` (`FragmentSmall.v`) `frag_certification_paid` (`FragmentSmall.v`) `frag_setup_live` (`FragmentSmall.v`) `frag_program_runs` (`FragmentSmall.v`) `frag_program_earned` (`FragmentSmall.v`) `frag_fin_toll` (`FragmentSmall.v`) `frag_cert_system` (`FragmentSmall.v`)	none none none none none none none none def
Theorem 16.42: The small machine's merges	§16	`frag_small_certify_merges` (`FragmentSmall.v`) `frag_small_certifying_step_priced_merge` (`FragmentSmall.v`) `frag_small_dec_free_merge` (`FragmentSmall.v`) `frag_small_not_merge_priced` (`FragmentSmall.v`)	none none none none
Theorem 4.7: A move that forgets nothing removes no bits	§4	`step_entropy_invariant_if_injective` (`PermanentCertificationEntropy.v`)	FE, RD, EM

Result	Sec.	Coq name and file	Axioms
Theorem 4.8: Entropy after a permanent flip	§4	`permanent_step_entropy_ceiling` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`permanent_flip_uniform_entropy_drop` (`PermanentCertificationEntropy.v`)	FE, RD, EM
Premise 4.5: Landauer's principle, as a premise	§4	`landauer_heat` (`PermanentCertificationEntropy.v`)	def
Corollary 4.9: Heat floor of a permanent flip	§4	`permanent_step_entropy_ceiling` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`permanent_flip_uniform_entropy_drop` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`entropy_le_log_support` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`permanent_flip_heat_floor` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`permanent_flip_heat_positive` (`PermanentCertificationEntropy.v`)	FE, RD, EM
In the text: in the worst case over spreads a permanent flip removes a full bit, whatever m is	§4	`merge_pair_removes_a_bit` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`permanent_flip_spread_loses_a_bit` (`PermanentCertificationEntropy.v`)	FE, RD, EM
Corollary 4.10: Positive heat for any full spread	§4	`permanent_flip_full_support_entropy_drop_positive` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`permanent_flip_full_support_heat_positive` (`PermanentCertificationEntropy.v`)	FE, RD, EM
Definition 4.6: Entropy-priced	§4	stated in the text	
Theorem 4.11: The toll from entropy pricing	§4	`a2_from_entropy_price_and_permanence` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`entropy_priced_trace_floor` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`certification_system_from_entropy_price` (`PermanentCertificationEntropy.v`)	def

Result	Sec.	Coq name and file	Axioms
Theorem 4.12: One premise, three forms	§4	`entropy_drop_le_log_fibre` (`PricingOnePremise.v`)	FE, RD, EM
		`entropy_drop_of_fibre` (`PricingOnePremise.v`)	FE, RD, EM
		`worst_case_entropy_drop` (`PricingOnePremise.v`)	FE, RD, EM
		`entropy_priced_iff_compression_priced` (`PricingOnePremise.v`)	FE, RD, EM
		`merging_priced_iff_pair_halving` (`PricingOnePremise.v`)	none
		`merging_priced_iff_pair_entropy` (`PricingOnePremise.v`)	FE, RD, EM
		`one_premise_three_forms` (`PricingOnePremise.v`)	FE, RD, EM
Theorem 4.13: A known state gets no heat floor from the premise	§4	`known_state_step_removes_no_entropy` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`known_state_flip_forces_no_heat` (if and only if) (`PermanentCertificationEntropy.v`)	FE, RD, EM
Theorem 4.14: The bits removed, one term per state	§4	`entropy_drop_as_point_sum` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`entropy_drop_nonneg` (`PermanentCertificationEntropy.v`)	FE, RD, EM
		`entropy_drop_pos_of_support_merge` (`PermanentCertificationEntropy.v`)	FE, RD, EM
Premise 4.7: Record-layer merge pricing	§4	`record_merges_priced` (`RecordMerge.v`)	def

Result	Sec.	Coq name and file	Axioms
Theorem 4.15: The toll on the machine I built	§4	`frag_small_certify_merges` (`FragmentSmall.v`)	none
		`frag_small_not_merge_priced` (`FragmentSmall.v`)	none
		`rm_commit_merges` (`RecordMerge.v`)	none
		`rm_merge_price_charges_record_moves` (`RecordMerge.v`)	none
		`rm_toll_from_any_merge_price` (`RecordMerge.v`)	none
		`rm_cert_system_from_merge_price` (`RecordMerge.v`)	none
		`rm_merge_price_gives_record_price` (`RecordMerge.v`)	none
		`rm_certify_record_merge` (`RecordMerge.v`)	def
		`rm_commit_record_merge` (`RecordMerge.v`)	none
		`rm_base_moves_no_record_merge` (`RecordMerge.v`)	none
		`rm_small_record_priced` (`RecordMerge.v`)	none
		`rm_toll_from_record_price` (`RecordMerge.v`)	none
		`rm_cert_system_from_record_price` (`RecordMerge.v`)	none
		`rm_record_price_and_exact_toll` (`RecordMerge.v`)	none
In the text: a repeated move, a closed machine or a repeating word, raising the flag for good from a start whose flag is down, merges two states the run visits; moves picked from outside need not; the strip machine has no finite list of its states	§4	`nec_f_orbit_merges_visited` (`NecFRepeat.v`)	none
		`nec_f_repeat_merges_visited` (`NecFRepeat.v`)	none
		`nec_f_closed_merges_visited` (`NecFRepeat.v`)	none
		`nec_f_word_merges_visited` (`NecFRepeat.v`)	none
		`nec_f_outside_moves_no_merge` (`NecFRepeat.v`)	none
		`nec_f_history_infinite` (`NecFPaperToys.v`)	none

Result	Sec.	Coq name and file	Axioms
In the text: on a finite chain with detailed balance, one step's entropy production is the drop in relative entropy and is nonnegative; from a known start the total is at most $-\ln \pi(s)$; the flows balance	§4	`re_sigma_is_drop` (`RelaxationEntropy.v`)	FE, RD
		`re_sigma_nonneg` (`RelaxationEntropy.v`)	FE, RD
		`re_relative_entropy_monotone` (`RelaxationEntropy.v`)	FE, RD
		`re_known_start_total` (`RelaxationEntropy.v`)	FE, RD
		`re_known_start_bound` (`RelaxationEntropy.v`)	FE, RD
		`re_known_start_vs_set` (`RelaxationEntropy.v`)	FE, RD
		`re_flux_balance` (`RelaxationEntropy.v`)	
		`re_stretch_ratio` (`RelaxationEntropy.v`)	
In the text: under Doeblin's condition the chain converges to its steady spread and the entropy from a known start converges to $-\ln \pi(s)$; in continuous time the relative entropy falls at the production rate	§4	`rc_contract` (`RelaxationConvergence.v`)	FE, RD
		`rc_converges` (`RelaxationConvergence.v`)	FE, RD
		`rc_known_start_gap` (`RelaxationConvergence.v`)	FE, RD, EM
		`rc_known_start_limit` (`RelaxationConvergence.v`)	FE, RD, EM
		`rcd_sigma_nonneg` (`RelaxationContinuous.v`)	FE, RD, EM
		`rcd_D_derivative` (`RelaxationContinuous.v`)	FE, RD, EM
		`rcd_D_monotone` (`RelaxationContinuous.v`)	FE, RD, EM
		`rcd_produced_window` (`RelaxationContinuous.v`)	FE, RD, EM
In the text: the mean length of a stretch with the flag up, counted in steps, is the steady mass of the up-states over the flow into them	§4	`rs_split` (`RelaxationStretch.v`)	FE, RD
		`rs_partial` (`RelaxationStretch.v`)	FE, RD
		`rs_mean_stretch` (`RelaxationStretch.v`)	FE, RD

Result	Sec.	Coq name and file	Axioms
In the text: in continuous time, the mean length of a stretch with the flag up, in seconds, is the steady mass of the up-states over the flow into them; the leaving times exist and the chance a stretch is still running decays exponentially	§4	`rsc_mass_over_flow` (`RelaxationStretchContinuous.v`) `rsc_leave_exists` (`RelaxationStretchContinuous.v`) `rsc_S_decay` (`RelaxationStretchContinuous.v`) `rsc_window_value` (`RelaxationStretchContinuous.v`) `rsc_mean_stretch_time` (`RelaxationStretchContinuous.v`) `rsc_stretch_identity` (`RelaxationStretchContinuous.v`)	FE, RD FE, RD FE, RD, EM FE, RD FE, RD, EM FE, RD, EM
In the text: under the continuous-time Doeblin condition the production rate is at least c times the relative entropy, which then decays like e^{-ct}; from a known start the entropy produced over the whole relaxation is $-\ln \pi(s)$	§4	`rcl_sigma_ge` (`RelaxationContinuousLimit.v`) `rcl_D_decay` (`RelaxationContinuousLimit.v`) `rcl_D_limit` (`RelaxationContinuousLimit.v`) `rcl_produced_limit` (`RelaxationContinuousLimit.v`) `rcl_D_at_zero` (`RelaxationContinuousLimit.v`) `rcl_known_start_total` (`RelaxationContinuousLimit.v`)	FE, RD, EM FE, RD, EM FE, RD, EM FE, RD, EM FE, RD, EM FE, RD, EM
Theorem 5.1: The machine's own narrowing is priced	§5	`run_narrowing_priced_log` (`KnowledgeNarrowing.v`)	none
Definition 5.1: What an observer knows	§5	`knowledge` (`KnowledgeNarrowing.v`)	def
Theorem 5.2: An observer can learn for free	§5	`observer_narrowing_can_be_free` (`KnowledgeNarrowing.v`) `demon_observer_learns` (`KnowledgeNarrowing.v`) `demon_machine_spread_kept` (`KnowledgeNarrowing.v`) `measure_forgets_nothing` (`KnowledgeNarrowing.v`) `wipe_merges` (`KnowledgeNarrowing.v`) `wipe_costs_at_least_one` (`KnowledgeNarrowing.v`)	none none none none none none

Result	Sec.	Coq name and file	Axioms
Theorem 5.3: Three states are the minimum for free teaching	§5	`free_incremental_narrowing_with_three` (`KnowledgeNarrowingMinimal.v`) `no_free_incremental_narrowing_below_three` (`KnowledgeNarrowingMinimal.v`) `demon_refutes_incremental` (`KnowledgeNarrowingMinimal.v`)	none none none
Definition 6.1: A counter kept by a price list	§6	stated in the text	
Theorem 6.1: Conservation	§6	`nec_f_conservation` (`NecFCounter.v`) `nec_f_counter_monotone` (`NecFCounter.v`)	none none
Theorem 6.2: Only one honest count	§6	`nec_f_initiality` (`NecFCounter.v`)	none
Definition 6.2: Potential, amortized cost	§6	`amortized_nonneg` (`CostSemanticsComparison.v`) `cert_potential` (`CostSemanticsComparison.v`)	def def
Theorem 6.3: The toll is the potential method	§6	`a2_iff_nonnegative_amortized_cost` (`CostSemanticsComparison.v`) `potential_telescoping` (`CostSemanticsComparison.v`) `nfi_by_potential` (`CostSemanticsComparison.v`) `certification_system_is_potential_method` (`CostSemanticsComparison.v`) `flips_le_cost` (`CostFrameworks.v`) `a2_and_aara_iff_exact` (`CostFrameworks.v`)	none none none none none none
In the text: on loop-free programs over a counter that conserves, a fixed grade equals the counter's rise on every run that doesn't trap, the small machine included; a loop has no fixed exact grade and an input-dependent one	§6	`nec_f_grade_exact` (`NecFGrade.v`) `nec_f_grade_exact_small` (`NecFGrade.v`) `nec_f_loop_cost` (`NecFGrade.v`) `nec_f_loop_no_fixed_grade` (`NecFGrade.v`) `nec_f_loop_input_grade` (`NecFGrade.v`)	none none none none none

Result	Sec.	Coq name and file	Axioms
Theorem 7.1: Decoders need constant fibres	§7	`decoding_requires_fiber_constancy` (`ObservationPolicy.v`)	none
		`selected_representatives_give_decoder` (`ObservationPolicy.v`)	none
Observation 7.2: No exact window price across a collision	§7	`shadow_floor_overcharges` (`ShadowPricing.v`)	none
		`shadow_cannot_price_exactly` (`ShadowPricing.v`)	none
		`window_showing_reading_prices_exactly` (`ShadowPricing.v`)	none
		`window_showing_reading_has_no_collision` (`ShadowPricing.v`)	none
Definition 7.1: Three richer transcripts and their explanations	§7	stated in the text	
Theorem 7.3: Three conditional escapes	§7	`ver_full_state_escape` (`VerifierSmall.v`)	none
		`ver_commitment_escape` (`VerifierSmall.v`)	none
		`ver_honest_meets_contract` (`VerifierSmall.v`)	none
		`ver_unchecked_breaks_contract` (`VerifierSmall.v`)	none
		`ver_response_escape` (`VerifierSmall.v`)	none
In the text: the same collision survives into any richer transcript that keeps it	§7	`ver_collision_blocks` (`VerifierSmall.v`)	none
		`ver_no_factor` (`VerifierSmall.v`)	none
Theorem 7.4: No bare verifier on a Thiele-complete machine	§7	`ver_complete_no_record_verifier` (`VerifierSmall.v`)	none
		`ver_complete_no_ledger_verifier` (`VerifierSmall.v`)	none
		`ver_small_no_ledger_verifier` (`VerifierSmall.v`)	none
		`ver_corollary` (`VerifierSmall.v`)	def
		`ver_small_corollary` (`VerifierSmall.v`)	
Definition 8.1: Entitlement	§8	stated in the text	
Definition 8.2: Decision tree	§8	stated in the text	

Result	Sec.	Coq name and file	Axioms
Lemma 8.1: A tree is no wider than two to its depth	§8	`decision_tree_leaves_le_pow2_depth` (`DecisionTreeBound.v`)	none
		`decision_tree_log2_up_leaf_bound` (`DecisionTreeBound.v`)	none
Theorem 8.2: Paid narrowing, counted	§8	`ent_cs_count` (`EntitlementSmall.v`)	none
		`nec_e_cs_count_any` (`NecEEnt.v`)	none
		`nec_e_cs_tight` (`NecEEnt.v`)	none
		`nec_f_conservation` (`NecFCounter.v`)	none
Theorem 8.3: Structural entitlement, over any certification system	§8	`ent_narrowing_strengthens` (`EntitlementSmall.v`)	none
			none
		`ent_cs_count` (`EntitlementSmall.v`)	none
		`ent_cs_entitlement` (`EntitlementSmall.v`)	none
			none
		`ent_leaves_le_pow2` (`EntitlementSmall.v`)	
		`ent_log2_leaves_le_depth` (`EntitlementSmall.v`)	
Theorem 8.4: Structural entitlement, with the narrowing made by the run	§8	`ent_cs_run_entitlement` (`EntitlementSmall.v`)	none
			none
		`ent_stages_product` (`EntitlementSmall.v`)	none
		`ent_stages_share_bits` (`EntitlementSmall.v`)	
Theorem 16.43: Structural entitlement represented	§16	`ent_representation` (`EntitlementSmall.v`)	none
			none
		`ent_every_shortcut_lands_here` (`EntitlementSmall.v`)	none
		`ent_complete_tree_bound` (`EntitlementSmall.v`)	
Theorem 16.44: A floor with no supplied tree	§16	`ent_questions_floor` (`EntitlementSmall.v`)	none
Theorem 16.45: Structural entitlement, with the narrowing made by the run	§16	`ent_run_entitlement` (`EntitlementSmall.v`)	none
			none
		`ent_run_round_bits` (`EntitlementSmall.v`)	none
		`ent_stages_round_bits` (`EntitlementSmall.v`)	
Proposition 16.46: One CHECK can take four bits	§16	`ent_share_needed` (`EntitlementSmall.v`)	none

Result	Sec.	Coq name and file	Axioms
Theorem 16.47: What the record adds	§16	`ent2_record_adds` (`EntitlementMore2.v`)	none
		`ent2_upgrade_iff` (`EntitlementMore2.v`)	none
		`ent2_observed_representation` (`EntitlementMore2.v`)	none
In the text: weights and the partition form	§16	`ent2_weighted_bits` (`EntitlementMore2.v`)	none
			none
		`ent2_weighted_cs` (`EntitlementMore2.v`)	none
		`ent2_weighted_representation` (`EntitlementMore2.v`)	none
			none
		`ent2_weighted_expected_bound` (`EntitlementMore2.v`)	none
			none
		`ent2_partition_to_reduction` (`EntitlementMore2.v`)	none
			none
		`ent2_wreduction_covers` (`EntitlementMore2.v`)	none
			none
		`ent2_rise_independent` (`EntitlementMore2.v`)	none
		`ent2_expected_rise_exact` (`EntitlementMore2.v`)	
		`ent2_cover_sum` (`EntitlementMore2.v`)	
		`ent2_partition_old_iff` (`EntitlementMore2.v`)	
		`ent2_partition_bits` (`EntitlementMore2.v`)	
		`ent2_reduction_not_partition` (`EntitlementMore2.v`)	
Proposition 16.48: The eight-start member	§16	`ent_small_certifies_iff` (`EntitlementSmall.v`)	none
			none
		`ent_small_posterior_is_certified` (`EntitlementSmall.v`)	none
			none
		`ent_small_instance` (`EntitlementSmall.v`)	none
			none
		`ent_small_questions` (`EntitlementSmall.v`)	none
		`ent2_observed_small_instance` (`EntitlementMore2.v`)	
		`ent_earned_complete_with` (`EntitlementSmall.v`)	
		`ent_small_run_narrowing` (`EntitlementSmall.v`)	

Result	Sec.	Coq name and file	Axioms
Theorem 16.49: A search through n bits	§16	`ent2_mmachine_complete` (`MultiThiele2.v`)	none
		`ent2_bit_search` (`BitSearchMember2.v`)	none
		`ent2_search_certifies_iff` (`BitSearchMember2.v`)	none
		`ent2_search_questions_floor` (`BitSearchMember2.v`)	none
		`ent2_questions_cap` (`BitSearch2.v`)	none
		`ent2_search_posterior_is_certified` (`BitSearchMember2.v`)	none
		`ent2_search_lands` (`BitSearchMember2.v`)	
		`ent2_search_observed` (`BitSearchObserved2.v`)	
		`ent2_no_search_past_16` (`BitSearchMember2.v`)	
In the text: the covering can't be dropped	§16	`ent2_uncovered_posterior` (`CoveringNeeded2.v`)	none
		`ent2_uncovered_claim_false` (`CoveringNeeded2.v`)	none
		`ent2_no_cheap_covering` (`CoveringNeeded2.v`)	
Theorem 16.50: The time tax	§16	`ent2_free_run_shift` (`TimeTax2.v`)	none
		`ent2_free_cannot_decide_fast` (`TimeTax2.v`)	none
		`ent2_ladder_decides` (`TimeTax2.v`)	none
		`ent2_chain_decides` (`TimeTax2.v`)	none
		`ent2_time_tax` (`TimeTax2.v`)	none
		`ent2_machine_free_cannot_decide` (`TimeTax2.v`)	none
		`ent2_machine_ladder_decides` (`TimeTax2.v`)	none
		`ent2_ladder_free` (`TimeTax2.v`)	
		`nec_e_time_tax_one_paid_move` (`NecESearch.v`)	
		`nec_e_time_tax_free_needed` (`NecESearch.v`)	

Result	Sec.	Coq name and file	Axioms
Theorem 8.5: A committed grouping entitles exactly its refinements	§8	`pr_grouping_partition` (`PartitionReading.v`) `pr_reads_structural` (`PartitionReading.v`) `pr_entitled_iff_refines` (`PartitionReading.v`) `pr_reading_entitles` (`PartitionReading.v`) `pr_reading_entitles_only` (`PartitionReading.v`) `pr_toll` (`PartitionReading.v`) `pr_reading_costs_three` (`PartitionReading.v`)	none none none none none none none
Definition 9.1: Certified insight event, raw observation	§9	stated in the text	
Remark 9.1: What the toll prices, and what it doesn't	§9	`nec_f_board_raw_free` (`NecFPaperToys.v`) `nec_f_board_floor` (`NecFPaperToys.v`) `nec_f_board_floor_flat` (`NecFPaperToys.v`)	none none none
Remark 9.2: Nothing makes the crossing wait	§9	`nec_f_board_no_wait` (`NecFPaperToys.v`)	none
Definition 10.1: Base, record-carrying machine, cover	§10	stated in the text	
Definition 10.2: Base-driven record, latch	§10	stated in the text	

Result	Sec.	Coq name and file	Axioms
Observation 10.1: The record is a latch	§10	`record_axis_is_latch_r1_r4` (`StructuralClockSync.v`)	none
		`record_axis_is_latch_holds` (`StructuralRecordAxis.v`)	none
		`BaseMachine` (`StructuralCoreAnyBase.v`)	def
		`BaseCover` (`StructuralCoreAnyBase.v`)	def
		`computation_driven` (`StructuralCoreAnyBase.v`)	def
		`HonestBaseExtension` (`StructuralCoreAnyBase.v`)	def
		`latch_factorization` (`StructuralCoreAnyBase.v`)	def
		`ledger_carried` (`StructuralCore.v`)	def
		`rc_a2` (`StructuralCore.v`)	def
		`record_permanent` (`StructuralCoreCover.v`)	def
		`reachable_record_write` (`StructuralCoreCover.v`)	def
Theorem 10.2: Every reachable latch is a base-driven record	§10	`latch_core_honest` (`RecordAxisDiscrimination.v`)	none
Theorem 10.3: Records on reversible machines	§10	`history_latch_injective` (`RecordAxisDiscrimination.v`)	none
		`history_latch_honest` (`RecordAxisDiscrimination.v`)	none
		`finite_reversible_cannot_write` (`RecordAxisDiscrimination.v`)	none
		`permanent_flip_is_not_injective` (`PermanentCertification.v`)	none
Lemma 10.4: Two records are two latches	§10	`record_pair_is_two_latches_holds` (`StructuralRecordAxis.v`)	none
Lemma 10.5: The toggle	§10	`toggle_computation_driven` (`StructuralRecordAxis.v`)	none
		`toggle_not_permanent` (`StructuralRecordAxis.v`)	none
		`toggle_not_latch` (`StructuralRecordAxis.v`)	none
Lemma 10.6: The clock	§10	`clock_record_permanent` (`StructuralRecordAxis.v`)	none
		`clock_record_not_driven` (`StructuralRecordAxis.v`)	none

Result	Sec.	Coq name and file	Axioms
Lemma 10.7: The clock, on the states runs reach	§10	`record_axis_is_latch_reachable` (`StructuralClockSync.v`) `clock_not_reachably_driven` (`StructuralClockSync.v`) `sync_clock_record_permanent` (`StructuralClockSync.v`) `sync_clock_not_driven` (`StructuralClockSync.v`) `sync_clock_reachably_driven` (`StructuralClockSync.v`) `sync_clock_reachable_latch` (`StructuralClockSync.v`)	none none none none none none
Corollary 10.8: The event is a free choice	§10	`nec_w_event_free_choice` (`NecWLatch.v`)	none
Theorem 10.9: The schedule doesn't fix the probabilities	§10	`schedule_determines_probabilities_refuted` (`ProbabilisticRecord.v`) `deterministic_latch_handles_branching_refuted` (`ProbabilisticRecord.v`)	none none
Definition 10.3: Three concrete bases	§10	stated in the text	
Definition 10.4: Weak base equivalence	§10	stated in the text	
Theorem 10.10: Outside bases, and the relation	§10	`weak_base_equiv_refl_holds` (`CrossBaseGranularity.v`) `weak_base_equiv_sym_holds` (`CrossBaseGranularity.v`) `weak_base_equiv_trans_holds` (`CrossBaseGranularity.v`) `weak_match_left_runs` (`CrossBaseGranularity.v`) `weak_match_right_runs` (`CrossBaseGranularity.v`) `weak_equiv_preserves_record_latch_holds` (`CrossBaseGranularity.v`) `record_axis_is_latch_on_tm_holds` (`CrossBaseGranularity.v`) `record_axis_is_latch_on_ram_holds` (`CrossBaseGranularityRAM.v`) `record_axis_is_latch_on_l_holds` (`CrossBaseGranularityL.v`)	none none none none none none none none none
Definition 10.5: A growing record	§10	stated in the text	

Result	Sec.	Coq name and file	Axioms
Theorem 10.11: Growing records are families of latches	§10	`growing_record_decomposes_holds` (`GrowingRecord.v`)	none
		`thresholds_determine_record_holds` (`GrowingRecord.v`)	none
		`record_price_iff_threshold_price_holds` (`GrowingRecord.v`)	none
		`one_latch_refuted` (`GrowingRecord.v`)	none
		`chain_needs_bits_holds` (`GrowingRecord.v`)	none
Theorem 11.1: The cups fix the place, and only with antisymmetry	§11	`bp_thresholds_determine` (`AxCore.v`)	none
		`thresholds_fail_on_preorder` (`AxCore.v`)	none
		`indiscrete_loses_record` (`AxLatch.v`)	none
In the text: two places each at or below the other leave the same cups up, so the cups fix the place if and only if the order is partial	§11	`nec_f_same_rung_same_cups` (`NecFCups.v`)	none
		`nec_f_non_partial_cups_fail` (`NecFCups.v`)	none
		`nec_f_cups_iff_partial` (`NecFCups.v`)	none
Theorem 11.2: The toll over any direction	§11	`ax_floor_iff_a2` (`AxCore.v`)	none
		`ax_cost_ge_exits` (`AxCore.v`)	none
		`ax_view_a2` (`AxCore.v`)	none
		`ax_a2_iff_views` (`AxLatch.v`)	none
		`two_point_nfi` (`AxCore.v`)	none
		`ax_exit_is_strict_rise` (`AxCore.v`)	none
Theorem 11.3: Lines a direction needs	§11	`chain_bits_exact` (`AxLatch2.v`)	none
		`chain3_not_join_latch` (`AxLatch.v`)	none
Theorem 11.4: Turning a cup over merges	§11	`ax_exit_merges_or_revokes` (`AxMerge.v`)	none
		`ax_growth_exit_merges` (`AxMerge.v`)	none
		`ax_a2_iff_flipping_merges_priced` (`AxMerge.v`)	none
		`ax_a2_from_merge_price` (`AxMerge.v`)	none
Theorem 11.5: Three ways out	§11	`history_escape` (`AxMerge.v`)	none
		`toggle_escape` (`AxMerge.v`)	none
		`free_merge_escape` (`AxMerge.v`)	none

Result	Sec.	Coq name and file	Axioms
Theorem 12.1: Window prices over a direction	§12	`ax_window_collision_overcharges` (`AxWindow.v`)	none
		`ax_window_no_exact_price` (`AxWindow.v`)	none
		`ax_window_exact_iff_no_collision` (`AxWindow.v`)	none
		`ax_overcharge_lower` (`AxWindow.v`)	none
		`ax_overcharge_tight` (`AxWindow.v`)	none
		`ax_exact_without_position` (`AxWindow.v`)	
		`ax_threshold_window_blind` (`AxWindow.v`)	
Theorem 12.2: The toll prices a collapse inside a fibre	§12	`ax_exit_collision` (`AxShadow.v`)	none
		`ax_fibre_collapse_exit` (`AxShadow.v`)	none
		`ax_step_fibre_in_fibre` (`AxShadow.v`)	none
		`shadow_merge_escape` (`AxShadow.v`)	none
		`ax_compression_iff_fibres` (`AxShadow.v`)	none
		`ax_priced_spots` (`AxInfinite.v`)	none
		`ax_spots_priced` (`AxInfinite.v`)	
Theorem 12.3: The halving price in bits bounds the average loss	§12	`ax_fibre_gibbs` (`AxProb.v`)	FE, RD, EM
		`ax_loss_le` (`AxProb.v`)	FE, RD, EM
		`ax_priced_loss_bits` (`AxProb.v`)	FE, RD, EM
		`ax_priced_loss_tight` (`AxProb.v`)	FE, RD, EM
		`ax_inf_loss_bits` (`AxInfinite.v`)	FE, RD, EM
Theorem 12.4: An infinite pile	§12	`ax_collapse_unpriceable` (`AxInfinite.v`)	none
		`ax_geometric_loss` (`AxInfinite.v`)	FE, RD, EM
		`ax_block_mass_n` (`AxInfinite.v`)	FE, RD
		`ax_block_loss_unbounded` (`AxInfinite.v`)	FE, RD, EM
		`ax_block_no_toll` (`AxInfinite.v`)	FE, RD, EM

Result	Sec.	Coq name and file	Axioms
In the text: under Landauer's premise, on a machine with the halving price, the toll priced at $kT \ln 2$ per unit covers the premise's minimum, for every spread and every finite family of landing piles	§12	`ax_toll_covers_landauer_minimum` (`AxInfinite.v`)	FE, RD, EM
Definition 13.1: Substrate	§13	stated in the text	
Definition 13.2: Shortcut predicate	§13	stated in the text	
Theorem 13.1: No inside decider	§13	`structural_shortcut_undecidable` (if and only if) (`StructuralUndecidability.v`) `Substrate` (`Substrate.v`) `WithShortcutPredicate` (`StructuralUndecidability.v`)	none def def
Theorem 13.2: The nat family	§13	`nat_self_undecidable` (`NatSubstrateInstance.v`) `nat_recursion_theorem` (`NatSubstrateInstance.v`) `nat_structural_shortcut_undecidable` (if and only if) (`NatSubstrateInstance.v`)	none none none
Definition 13.3: L, as built here	§13	stated in the text	
Theorem 13.3: Recursion, Rice and halting for L	§13	`second_recursion` (`LRecursion.v`) `L_recursion_theorem` (`LRecursion.v`) `L_rice` (`LRecursion.v`) `L_halting_undecidable` (`LRecursion.v`) `Q_spec` (`LRecursion.v`) `L_structural_shortcut_undecidable` (if and only if) (`LRecursion.v`)	none none none none none none
Definition 14.1: Casper setting and votes	§14	stated in the text	
Theorem 14.1: Accountable safety	§14	`accountable_safety` (`CasperFFG.v`) `conflicting_records_are_priced` (`CasperRecordReading.v`)	none none

Result	Sec.	Coq name and file	Axioms
Theorem 14.2: The premise can hold, and the write is free	§14	`casper_fork_exists` (`CasperForkWitness.v`) `casper_fork_slashable` (`CasperForkWitness.v`) `finalization_without_slashing` (`CasperRecordReading.v`)	none none none
Definition 14.2: One slot in one transaction	§14	stated in the text	
Theorem 14.3: Pricing the persistent write and the revoked one	§14	`persistent_write_priced` (`EVMStorageGas.v`) `revoked_write_nearly_free` (`EVMStorageGas.v`)	none none
Definition 14.3: A gas schedule	§14	stated in the text	
Theorem 14.4: Gas schedule exactness	§14	`gas_schedule_exactness` (if and only if) (`GasMetering.v`) `undercharged_opcode_admits_free_commitment` (`GasMetering.v`) `undercharged_opcode_breaks_certification_floor` (`GasMetering.v`) `overcharge_breaks_exactness` (`GasMetering.v`) `exact_commitment_pricing_characterization` (`CommitmentPredicateAdequacy.v`)	none none none none none
Theorem 14.5: A permanent write forgets on its own yes side	§14	`permanent_write_has_logical_payment` (`PricingPhysicsAudit.v`) `permanent_at_flip_is_not_injective` (`PermanentRecordPricing.v`) `permanent_flip_merges_on_yes_side` (`PermanentRecordPricing.v`)	none none none
Definition 14.4: Platform and quote	§14	stated in the text	
Theorem 14.6: The quote cannot attest unmeasured state	§14	`quote_cannot_attest_unmeasured_state` (`TPMQuoteGap.v`) `quote_decides_measured_claims` (`TPMQuoteGap.v`) `tpm_interface_authenticity_refuted` (`TPMQuoteAuthenticity.v`)	none none none
Definition 14.5: Inclusion check, RFC 9162 section 2.1.3.2	§14	stated in the text	

Result	Sec.	Coq name and file	Axioms
Theorem 14.7: The RFC's own examples pass	§14	`rfc9162_example_inclusion_d3` (`RFC9162Merkle.v`) `rfc9162_example_inclusion_d0` (`RFC9162Merkle.v`) `rfc9162_example_inclusion_d4` (`RFC9162Merkle.v`) `rfc9162_example_inclusion_d6` (`RFC9162Merkle.v`) `rfc9162_example_consistency_4_7` (`RFC9162Merkle.v`) `rfc9162_inclusion_boundary_safe` (`RFC9162Merkle.v`) `ct_extension_preserves_entries` (`RFC9162Merkle.v`)	none none none none none none
Definition 14.6: The bounds fragment	§14	stated in the text	
Theorem 14.8: The checker is the verification condition	§14	`pcc_checker_accepts_iff_vc` (`NeculaPCC.v`) `pcc_certificate_implies_vc` (`NeculaPCC.v`) `pcc_unsafe_program_rejected` (`NeculaPCC.v`)	none none none
Definition 15.1: Ecosystem, records, proliferates, pointer	§15	stated in the text	
Definition 15.2: Copied and relied on, independent carriers, the pointer	§15	`copied_and_relied_on` (`PointerObservable.v`) `pointer_on` (`PointerObservable.v`) `independent_carriers` (`PointerObservable.v`) `meterable` (`PointerObservable.v`) `pointer_on_unique` (`PointerObservable.v`)	def def def none
Theorem 15.1: Independent copies on every state: the event is equivalent at any two states	§15	`independent_copies_everywhere_are_constant` (`PointerObservable.v`)	none

Result	Sec.	Coq name and file	Axioms
Theorem 15.2: A replicated header: copying doesn't separate, reliance does	§15	`ledger_carriers_independent` (`PointerObservable.v`) `ledger_copying_alone_does_not_separate` (`PointerObservable.v`) `ledger_finalized_is_pointer` (`PointerObservable.v`) `ledger_gas_not_relied_on` (`PointerObservable.v`)	none none none none
Thesis 15.3: The pointer thesis	§15	a thesis, stated and not claimed; its schema is `pointer_thesis_schema` (`PointerObservable.v`)	def
Theorem 15.4: The toy	§15	`toy_cert_unique_pointer` (`PointerObservable.v`) `toy_work_not_proliferating` (`PointerObservable.v`)	none none
Theorem 15.5: The survey, and the swap	§15	`twelve_candidate_measurements_checked` (`RecordProliferationSurvey.v`) `swapped_event_is_pointer_checked` (`RecordProliferationSurvey.v`) `five_labeled_models_have_selected_pointer` (`PointerObservableReductions.v`) `pointer_criterion_holds_indexed` (`PointerObservableReductions.v`)	none none none def
Theorem 15.6: One blind observer is enough	§15	`blind_observer_blocks_proliferation` (`PointerObservableCounterexamples.v`) `labeled_model_verdicts` (`PointerObservableCounterexamples.v`)	none none
Definition 15.3: Observation game	§15	stated in the text	
Theorem 15.7: What buys permanence	§15	`toggle_game_refutes_strong_pointer_necessity` (`EcosystemGame.v`) `durable_consensus_implies_permanence` (`EcosystemGame.v`)	none none

Result	Sec.	Coq name and file	Axioms
In the text: on the three states, the six requirements that name no reading admit both the toll and the price on merging; the ceiling on raises with the floor fixes one mark per raise	§15	`nec_f_three_toll_meets` (`NecFThreeState.v`) `nec_f_three_merge_meets` (`NecFThreeState.v`) `nec_f_three_bills_differ` (`NecFThreeState.v`) `nec_f_three_automorphism_id` (`NecFThreeState.v`) `nec_f_record_ceiling_unique` (`NecFThreeState.v`) `nec_f_three_ceiling_picks_toll` (`NecFThreeState.v`) `nec_f_three_merge_fails_ceiling` (`NecFThreeState.v`) `nec_f_three_cs_floor` (`NecFThreeState.v`)	none none none none none none none none
Definition 16.1: Properties, checker, meaning	§16	stated in the text	
Lemma 16.1: The checker says what it means	§16	`eval_iff` (`EarnedCore.v`, +7)	none
Definition 16.2: Core, state, claim, start	§16	stated in the text	
Definition 16.3: Instructions, costs, the step	§16	`cexec` (`EarnedCore.v`, +2) `exec` (`EarnedCore.v`, +2) `run` (`EarnedCore.v`, +7) `total_cost` (`EarnedCore.v`, +3) `run_prog` (`EarnedCore.v`, +2) `start` (`EarnedCore.v`, +2) `clean_start` (`EarnedCore.v`, +2)	def def def def def def def
Definition 16.4: Two-counter program, compile, window	§16	stated in the text	
Theorem 16.2: Step for step	§16	`simulation_step` (`EarnedCore.v`) `simulation_run` (`EarnedCore.v`)	none none
Theorem 16.3: Halting matches	§16	`halting_correspondence` (`EarnedCore.v`)	none

Result	Sec.	Coq name and file	Axioms
Theorem 16.4: Halting is undecidable for this machine	§16	`mm2_step_iff` (`EarnedCoreLinks.v`) `mm2_stop_iff` (`EarnedCoreLinks.v`) `mm2_terminates_iff` (`EarnedCoreLinks.v`) `mm2_halting_iff` (if and only if) (`EarnedCoreLinks.v`) `earned_core_halting_undecidable` (`EarnedCoreLinks.v`)	none none none none none
Theorem 16.5: Ledger, toll, permanence	§16	`mu_conservation` (`EarnedCore.v`, +1) `mu_conservation_trace` (`EarnedCore.v`, +1) `only_certify_certifies` (`EarnedCore.v`, +1) `a2` (`EarnedCore.v`, +3) `cert_permanent` (`EarnedCore.v`)	none none none none none
Theorem 16.6: An instance of the certification system	§16	`earned_core_floor` (`EarnedCoreLinks.v`) `universal_nfi_any_substrate` (`UniversalCertificationCost.v`) `nfi_floor` (`EarnedCore.v`, +1) `earned_cs` (`EarnedCoreLinks.v`)	none none none def
Theorem 16.7: The flag is a latch of the base	§16	`earned_core_honest` (`EarnedCoreLinks.v`) `earned_core_is_latch` (`EarnedCoreLinks.v`) `record_axis_is_latch_holds` (`StructuralRecordAxis.v`) `cert_latch` (`EarnedCore.v`) `base_blind` (`EarnedCore.v`)	none none none none none
Definition 16.5: Untouched	§16	`untouched` (`EarnedCore.v`, +2)	def
Theorem 16.8: A commitment was checked first	§16	`earned_commitment_provenance` (`EarnedCore.v`)	none
Theorem 16.9: A raised flag was checked, then committed, then certified	§16	`earned_certification_provenance` (`EarnedCore.v`) `earned_certification_same_claim` (`EarnedCore.v`)	none none
Theorem 16.10: Unearned steps trap and change no claim	§16	`unearned_commit_traps` (`EarnedCore.v`) `uncommitted_certify_traps` (`EarnedCore.v`)	none none

Result	Sec.	Coq name and file	Axioms
Theorem 16.11: Checker soundness	§16	`checker_soundness` (`EarnedCore.v`)	none
		`committed_claim_holds` (`EarnedCore.v`, +1)	none
		`sound_step` (`EarnedCore.v`)	none
		`sound` (`EarnedCore.v`, +2)	def
Definition 16.6: Earned fact; no forgery	§16	stated in the text	
Theorem 16.12: No forging	§16	`facts_step` (`EarnedCore.v`)	none
		`no_forging_step` (`EarnedCore.v`)	none
		`no_forging` (`EarnedCore.v`)	none
Theorem 16.13: Bounded table, no eviction	§16	`facts_keep` (`EarnedCore.v`)	none
		`full_table_traps` (`EarnedCore.v`)	none
		`facts_bounded_step` (`EarnedCore.v`)	none
Theorem 16.14: Same window, different record	§16	`receipt_separation` (`EarnedCore.v`, +1)	none
Theorem 16.15: No window oracle	§16	`no_mu_oracle` (`EarnedCore.v`, +2)	none
		`no_cert_oracle` (`EarnedCore.v`, +2)	none
		`no_commit_oracle` (`EarnedCore.v`)	none
Observation 16.16: The minimum price of a certificate	§16	`certified_run_min_cost` (`EarnedCore.v`)	none
		`program_certified_min_cost` (`EarnedCore.v`)	none
Theorem 16.17: Three is tight	§16	`earned_run_check_can_fail` (`EarnedCore.v`)	none
		`earned_run_refused_forever` (`EarnedCore.v`)	none
		`earned_run` (`EarnedCore.v`)	def
		`min_cost_tight` (`EarnedCore.v`)	none
Definition 16.7: Thiele-complete	§16	`thiele_interface` (`ThieleComplete.v`)	def
		`thiele_complete_with` (`ThieleComplete.v`)	def
		`thiele_complete` (`ThieleComplete.v`)	def
		`weakly_thiele_complete` (`ThieleComplete.v`)	def
		`claim_language` (`ThieleComplete.v`)	def
		`lang_interface` (`ThieleComplete.v`)	def
		`thiele_complete_over` (`ThieleComplete.v`)	def
Theorem 16.18: CHECK is a unit-cost test	§16	`check_ge_unit_cost` (`ThieleComplete.v`)	none

Result	Sec.	Coq name and file	Axioms
Theorem 16.19: What every Thiele-complete machine has	§16	`thiele_complete_is_weak` (`ThieleComplete.v`) `base_halting_correspondence` (`ThieleComplete.v`) `certificate_costs_three` (`ThieleComplete.v`) `complete_has_free_move` (`ThieleComplete.v`) `base_runs_every_program` (`ThieleComplete.v`) `ledger_counts_record_moves` (`ThieleComplete.v`) `check_can_fail` (`ThieleComplete.v`) `some_run_certifies` (`ThieleComplete.v`) `only_certify_raises` (`ThieleComplete.v`) `complete_costs_at_most_one` (`ThieleComplete.v`)	none none none none none none none none none none
Theorem 16.20: The language first, and what a certificate then means	§16	`thiele_complete_over_complete` (`ThieleComplete.v`) `thiele_complete_with_over` (`ThieleComplete.v`) `over_certificate_means` (`ThieleComplete.v`)	none none none
Observation 16.21: A clock is a Thiele machine and isn't Thiele-complete	§16	`clock_weakly_thiele_complete` (`ThieleComplete.v`) `clock_not_thiele_complete` (`ThieleComplete.v`) `latch_clock_weakly_thiele_complete` (`ThieleComplete.v`) `latch_clock_not_thiele_complete` (`ThieleComplete.v`) `complete_has_free_move` (`ThieleComplete.v`)	none none none none none

Result	Sec.	Coq name and file	Axioms
Theorem 16.22: Each clause does work	§16	paid_latch_meets_base_and_toll (ThieleComplete.v) paid_latch_not_thiele_complete (ThieleComplete.v) silent_meets_base_record_toll (ThieleComplete.v) silent_not_thiele_complete (ThieleComplete.v) paid_latch_weakly_thiele_complete (ThieleComplete.v) one_move_record_excluded (ThieleComplete.v) silent_weakly_thiele_complete (ThieleComplete.v) never_certifies_excluded (ThieleComplete.v)	none none none none none none none none
Theorem 16.23: The four clauses are independent	§16	drop_meets_b_c_d (ThieleCompleteIndependent.v) drop_fails_a (ThieleCompleteIndependent.v) drop_not_thiele_complete (ThieleCompleteIndependent.v) drop_weakly_thiele_complete (ThieleCompleteIndependent.v) free_meets_a_c_d (ThieleCompleteIndependent.v) free_fails_b (ThieleCompleteIndependent.v) free_not_thiele_complete (ThieleCompleteIndependent.v) free_weakly_thiele_complete (ThieleCompleteIndependent.v) paid_meets_a_b_d (ThieleCompleteIndependent.v) paid_fails_c (ThieleCompleteIndependent.v) paid_not_thiele_complete (ThieleCompleteIndependent.v) paid_weakly_thiele_complete (ThieleCompleteIndependent.v) thiele_complete_clauses_independent (ThieleCompleteIndependent.v)	none none none none none none none none none none none none none

Result	Sec.	Coq name and file	Axioms
Theorem 16.24: Any unit	§16	thiele_complete_at_one (ThieleCompleteScaled.v)	none
		scale_complete_at (ThieleCompleteScaled.v)	none
		unscale_complete (ThieleCompleteScaled.v)	none
		scaled_has_free_move (ThieleCompleteScaled.v)	none
		clock_not_thiele_complete_at_any_unit (ThieleCompleteScaled.v)	none
		latch_clock_not_thiele_complete_at_any_unit (ThieleCompleteScaled.v)	none
		paid_moves_not_thiele_complete_at_any_unit (ThieleCompleteScaled.v)	none
		scaled_certificate_costs_three_c (ThieleCompleteScaled.v)	
		scaled_committed_claim_holds (ThieleCompleteScaled.v)	
		scaled_only_certify_raises (ThieleCompleteScaled.v)	
		doubled_earned_scaled (ThieleCompleteScaled.v)	
		doubled_earned_not_thiele_complete (ThieleCompleteScaled.v)	
Theorem 16.25: The small machine, the generic one and the sorted one are Thiele-complete	§16	earned_core_thiele_complete (ThieleComplete.v)	none
		earned_generic_thiele_complete (ThieleComplete.v)	none
		sorted_machine_thiele_complete (ThieleComplete.v)	none
		thiele_complete_floor (EarnedGenericLinks.v)	none
		earned_complete_agrees (EarnedGenericLinks.v)	none
		generic_complete_agrees (EarnedGenericLinks.v)	
		earned_core_thiele_complete_over (ThieleComplete.v)	
		earned_generic_thiele_complete_over (ThieleComplete.v)	
		sorted_machine_thiele_complete_over (ThieleComplete.v)	

Result	Sec.	Coq name and file	Axioms
Observation 16.26: Every Thiele-complete machine hides its record and its ledger from its window	§16	`complete_every_window_printed` (`ThieleCompleteWindow.v`)	none
		`complete_no_record_oracle` (`ThieleCompleteWindow.v`)	none
		`complete_no_ledger_oracle` (`ThieleCompleteWindow.v`)	none
		`complete_cs_window_blind` (`EarnedGenericLinks.v`)	none
		`complete_hides_record` (`ThieleCompleteWindow.v`)	
		`complete_hides_ledger` (`ThieleCompleteWindow.v`)	
		`thiele_complete_hides` (`ThieleCompleteWindow.v`)	
Definition 16.8: The host machine	§16	`pair` (`UniversalCodes.v`)	def
		`pcode` (`UniversalCodes.v`)	def
		`heval` (`UniversalCodes.v`, +1)	def
		`hholds` (`UniversalCodes.v`)	def
Theorem 16.27: No host fact on an exact copy of the guest's counter	§16	`pigeonhole_collision` (`UniversalNoCopy.v`)	none
		`no_exact_copy_host` (`UniversalNoCopy.v`)	none
		`multi_cs_no_exact_copy` (`UniversalInterpreterLinks.v`)	none
Definition 16.9: The program U, and loading a guest	§16	`U` (`UniversalLayout.v`, +4)	def
		`hload` (`UniversalSim.v`, +1)	def
		`prog_code` (`UniversalCodes.v`)	def
		`U_paid_sites` (`UniversalLayout.v`)	none
Theorem 16.28: U keeps pace with its guest	§16	`hload_rel` (`UniversalSim.v`)	none
		`U_step` (`UniversalSim.v`)	none
		`U_simulation` (`UniversalRun.v`)	none
Theorem 16.29: Halting and answers match	§16	`universal_halting` (`UniversalRun.v`)	none
		`universal_output` (`UniversalRun.v`)	none
Theorem 16.30: The host's flag is the guest's, and its ledger is exact	§16	`universal_flag_iff` (`UniversalRun.v`)	none
		`universal_ledger_exact` (`UniversalRun.v`)	none
Theorem 16.31: The host earns its own flag	§16	`universal_earned` (`UniversalRun.v`)	none

Result	Sec.	Coq name and file	Axioms
Theorem 16.32: The host is Thiele-complete, and *U*'s halting is undecidable	§16	`universal_thiele_complete` (`UniversalRun.v`)	none
		`U_run_on_host_machine` (`UniversalRun.v`)	none
		`interp_halting_undecidable` (`UniversalInterpreterLinks.v`)	none
		`interp_U_certified_floor` (`UniversalInterpreterLinks.v`)	none
		`multi_cs_floor` (`UniversalInterpreterLinks.v`)	none
		`interp_U_complete_floor` (`UniversalInterpreterLinks.v`)	
		`interp_complete_agrees` (`UniversalInterpreterLinks.v`)	
		`interp_halting_iff` (`UniversalInterpreterLinks.v`)	
		`universal_thiele_complete_over` (`UniversalRun.v`)	
Definition 16.10: The host	§16	`hexec` (`UniversalThiele.v`)	def
		`U` (`UniversalThiele.v`, +4)	def
Theorem 16.35: The host runs any guest	§16	`universal_simulation` (`UniversalThiele.v`)	none
		`universal_program_simulation` (`UniversalThiele.v`)	none
		`host_thiele_complete` (`UniversalThiele.v`)	
Observation 16.36: The copy is the record	§16	`record_agreement_step` (`UniversalThiele.v`)	none
		`record_agreement` (`UniversalThiele.v`)	none
		`simulated_record` (`UniversalThiele.v`)	none
		`record_agreement_program` (`UniversalThiele.v`)	none
		`hload_agrees` (`UniversalThiele.v`)	
Theorem 16.37: The host's step collects the guest's toll	§16	`toll_enforced_by_host_step` (`UniversalThiele.v`)	none
		`every_guest_crossing_is_a_host_crossing` (`UniversalThiele.v`)	none
		`host_pays_for_guest` (`UniversalThiele.v`)	none
		`host_cost_covers_guest` (`UniversalThiele.v`)	
		`simulated_cost` (`UniversalThiele.v`)	

Result	Sec.	Coq name and file	Axioms
Theorem 16.38: No free host certification	§16	`no_free_host_certification_step` (`UniversalThiele.v`)	none
		`no_free_host_certification` (`UniversalThiele.v`)	none
		`no_free_host_certification_program` (`UniversalThiele.v`)	none
		`guestless_runs_leave_mirror` (`UniversalThiele.v`)	none
		`own_record_only_by_certify` (`UniversalThiele.v`)	none
Theorem 16.39: The host is a Thiele machine	§16	`host_toll` (`UniversalThiele.v`)	none
		`host_own_toll` (`UniversalThiele.v`)	none
		`host_mirror_toll` (`UniversalThiele.v`)	none
		`host_nfi` (`UniversalThieleLinks.v`)	none
		`host_cs` (`UniversalThieleLinks.v`)	def
		`host_mirror_cs` (`UniversalThieleLinks.v`)	def
		`guest_cs` (`UniversalThieleLinks.v`)	def
Remark 16.1: Two guests, run	§16	`demo_certifies` (`UniversalThiele.v`)	none
		`demo_program_certifies` (`UniversalThiele.v`)	none
		`demo_forgery_fails` (`UniversalThiele.v`)	none
In the text: the same chain for any property language with an exact equality test and a proved checker, “sorted” included	§16	`generic_earned_certification_provenance` (`EarnedGeneric.v`)	none
		`generic_checker_soundness` (`EarnedGeneric.v`)	none
		`generic_no_forging` (`EarnedGeneric.v`)	none
		`generic_certified_run_min_cost` (`EarnedGeneric.v`)	none
		`sorted_run_certifies_iff` (`EarnedGeneric.v`)	none
		`sorted_run_refused_forever` (`EarnedGeneric.v`)	none
In the text: with the flag up from a clean start, the channel names a fact an earlier passing COMMIT committed, and a later commit can replace it, as a two-commit run shows	§16	`certified_channel_earned` (`EarnedCore.v`)	none
		`channel_names_last_commit` (`EarnedCore.v`)	none

Result	Sec.	Coq name and file	Axioms
In the text: after a run from a loaded host, a raised mirror means the guest's own run earned it	§16	`host_mirror_earned` (`UniversalThiele.v`)	none
Theorem 16.33: One host runs every computably presented Thiele machine	§16	`presented_universal` (`PresentedUniversal.v`) `presented_universal_halting` (if and only if), `presented_universal_points`, `presented_universal_halt_point`, `presented_universal_flag_iff` (if and only if), `presented_universal_earned` (`PresentedUniversal.v`) `pu_universal_halting` (if and only if), `pu_universal_flag_iff` (if and only if), `pu_universal_ledger_exact`, `pu_universal_earned`, `pu_universal_thiele_complete` (`UniversalPRun.v`) `pu_universal_thiele_complete_over` (`UniversalPRun.v`)	none none none none
Theorem 16.34: Exact, within two, and no better	§16	`presented_universal_surcharge_le_two`, `presented_universal_exact`, `presented_universal_no_exact_below_three`, `presented_universal_U_no_exact_below_three` (`PresentedUniversal.v`)	none
In the text: a demonstration machine on the priced host	§16	`pu_demo_computably_presented`, `pu_demo_never_halts`, `pu_demo_flag_rises`, `pu_demo_exact`, `pu_demo_earned` (`PresentedDemo.v`)	none
Theorem 16.40: A counter beside a Turing machine changes nothing	§16	`thiele_simulates_turing_gen` (`ProperSubsumption.v`) `thiele_simulates_turing` (`ProperSubsumption.v`)	none none

Result	Sec.	Coq name and file	Axioms
In the text: in the framework of Åman Pohjola and Parrow, a monotonic composition has no consensus process; the small machine, threads pooled on one shared state, has one, so its composition is not monotonic	§16	`cs_no_consensus_monotonic` (`ConsensusSeparation.v`) `cs_machine_consensus` (`ConsensusSeparation.v`) `cs_machine_not_monotonic` (`ConsensusSeparation.v`)	none FE FE
In the text: the shared small machine brings any number of processors to wait-free agreement; with everything growing, no protocol brings two to agreement	§16	`sc_agreement` (`SmallConsensus.v`) `sc_validity` (`SmallConsensus.v`) `sc_wait_free` (`SmallConsensus.v`) `mc_no_consensus` (`MonotoneConsensus.v`) `mc_mono_interfere` (`MonotoneConsensus.v`) `mc_mono_no_consensus` (`MonotoneConsensus.v`)	none none none none FE FE
In the text: without PAY, a run that never traps and ends with the flag down bills at most 32 plus its repeated COMMITs	§16	`pf_bound` (`PayFree.v`) `pf_bound_start` (`PayFree.v`) `pf_no_repeat_bound` (`PayFree.v`)	none none none
Definition 16.12: Behaving the same, on the host	§16.18	`sm_hequiv` (`SmHostBlocks.v`) `sm_hext` (`SmHostRice.v`)	def def
Theorem 16.51: Rice's theorem on the host machine	§16.18	`sm_host_rice` (`SmHostRice.v`) `sm_rice_prog_halts` (`SmHostRice.v`) `sm_rice_prog_diverges` (`SmHostRice.v`) `sm_host_rice_fun` (`SmKleene.v`)	none none none none
Theorem 16.52: Rice's theorem for the small machine, from the clean start	§16.18	`sm_guest_rice` (`SmGuestRice.v`) `sm_guest_halting_clean_undecidable` (`SmGuestRice.v`)	none none
Theorem 16.53: s-m-n, and a universal program for host programs (each an if and only if)	§16.18	`sm2_smn_all` (`SmSmnAll.v`) `sm_smn` (`SmKleene.v`) `sm_host_universal` (`SmKleene.v`) `sm_hdecode_hcode` (`SmCodes.v`)	none none none none

Result	Sec.	Coq name and file	Axioms
Theorem 16.54: Kleene's recursion theorem on the host machine (equivalence of what the two compute)	§16.18	`sm_second_recursion` (`SmKleene.v`)	none
		`sm_kleene` (`SmKleene.v`)	none
		`sm_kleene_codes` (if and only if) (`SmKleene.v`)	none
Theorem 16.55: The recursion theorem, up to the record (for every carried-out F, some e agrees with $F(e)$ on register 0, the trap latch, the ledger, the flag, the number of facts and whether the channel is empty)	§16.18	`sm2_kleene_obs` (`SmFixedPoint.v`)	none
		`sm2_kleene_fun` (`SmFixedPoint.v`)	none
		`sm2_final_numbers` (`SmTally.v`)	none
		`sm2_ev_spec` (`SmTally.v`)	none
		`sm2_VG_forward` (if and only if) (`SmFixed.v`)	none
		`sm2_VG_halt` (`SmFixed.v`)	none
		`sm2_while` (`SmLoops.v`)	none
		`sm2_mma_ctx` (`SmMMAOff.v`)	none
Theorem 16.56: No fixed point of full behaviour	§16.18	`sm2_no_exact` (`SmNoExact.v`)	none
		`sm2_reg_bound` (`SmNoExact.v`)	none
Theorem 16.57: The diagonal on the host machine (for a functional property some program has and some lacks, no decider whose flip a host program carries out says yes exactly when a program has the property, and no host program decides it)	§16.18	`sm_no_inside_decider` (`SmKleene.v`)	none
		`sm_no_host_decider` (`SmDecider.v`)	none
Theorem 16.58: Rice's theorem for the two-counter machine, on any inputs	§16.18	`tc_rice` (`TcRice.v`)	none
		`tc_rice_dichotomy` (`TcRice.v`)	none
Theorem 16.59: Kleene for the two-counter machine, inputs written as powers of two (for every carried-out F, some e is equivalent to $F(e)$: it computes the same function)	§16.18	`tc_universal` (`TcPacked.v`)	none
		`tc_smn` (`TcPacked.v`)	none
		`tc_second_recursion` (`TcPacked.v`)	none
		`tc_kleene` (`TcPacked.v`)	none
		`tc_no_fine_kleene` (`TcNoFine.v`)	none

Result	Sec.	Coq name and file	Axioms
Theorem 16.60: No plain recursion on two counters (some carried-out F has no e equivalent to $F(e)$ as a plain function)	§16.18	`tc2_no_mult` (`Tc2Mult.v`)	none
		`am_no_multiplier` (`Tc2Chain.v`)	none
		`tc2_plain_recursion_false` (`Tc2Plain.v`)	none
In the text: Schroeppel's theorem, no program of the small machine started on n computes 2^n, nor any machine with a finite control read through zero and parity above a threshold	§16.18	`tc2_no_pow` (`Tc2Pow.v`)	none
		`am_no_power_of_two` (`Tc2Pow.v`)	none
Theorem 17.1: Every universal base lifts	§17	`lift_thiele_complete` (`LiftCore.v`)	none
		`lift_window_nonvacuous` (`LiftCore.v`)	none
		`lift_window_thiele_complete` (`LiftCore.v`)	none
			none
		`lift_reduct_universal` (`LiftConverse.v`)	none
		`lift_projection_run` (`LiftConverse.v`)	
Theorem 17.2: What doesn't lift	§17	`lift_finite_moves_not_a_base` (`LiftConverse.v`)	none
			none
		`lift_ub_not_finitely_branching` (`LiftConverse.v`)	none
			none
		`lift_finite_branching_not_complete` (`LiftConverse.v`)	none
			none
		`lift_stateless_not_complete` (`LiftConverse.v`)	
		`lift_ax_indiscrete_no_machine` (`LiftAxis.v`)	
		`lift_canonical_nonvac_iff` (`LiftConverse.v`)	
Theorem 17.3: The one-lever presentation loses nothing	§17	`ub_to_sim_inhabited` (`LiftMacro.v`)	none
		`sim_to_macro_inhabited` (`LiftMacro.v`)	none
		`sim_macro_runs` (`LiftMacro.v`)	none
		`sim_base_lifts` (`LiftMacro.v`)	none
		`fl_has_sim_base` (`LiftMacro.v`)	none
		`fl_no_universal_base` (`LiftMacro.v`)	none
		`fl_macro_lifts` (`LiftMacro.v`)	none

Result	Sec.	Coq name and file	Axioms
Theorem 17.4: The base doesn't matter	§17	`lift_num_models_agree` (`LiftModelsAll.v`)	none
		`lift_ex_in_all_models` (`LiftModelsAll.v`)	none
		`lift_ex_thiele_complete` (`LiftModels.v`)	none
		`lift_ram_thiele_complete` (`LiftModels.v`)	none
		`lift_numeric_base` (`LiftModelsAll.v`)	
		`lift_classic_bases` (`LiftModelsAll.v`)	
Theorem 17.5: One counter is too few	§17	`lift_oc_halts_bound` (`LiftOneCounter.v`)	none
		`lift_oc_halts_dec` (`LiftOneCounter.v`)	none
		`lift_cm_halting_undecidable` (`LiftModels.v`)	none
		`lift_no_oc_translation` (`LiftModels.v`)	none
		`lift_no_oc_translation_undecidability` (`LiftModels.v`)	none
Theorem 18.1: Side by side	§18	`cmpz_cost_prod` (`CzProd.v`)	none
		`cmpz_interleaving_invariant` (`CzCounter.v`)	none
		`cmpz_prod_a2_iff` (`CzProd.v`)	none
		`cmpz_cs_or_floor` (`CzCS.v`)	none
		`cmpz_cs_and_floor` (`CzCS.v`)	none
		`cmpz_cs_and_tight` (`CzCS.v`)	none
		`cmpz_prod_tc` (`CzProdTC.v`)	none
		`cmpz_prod_joint_certificate` (`CzProdTC.v`)	none
		`cmpz_prod_joint_certificate_tight` (`CzProdTC.v`)	none
		`cmpz_prod_universal` (`CzCat.v`)	
Theorem 18.2: One after the other	§18	`cmpz_seq_tc` (`CzSeq.v`)	none
		`cmpz_seq_composes` (`CzSeq.v`)	none
		`cmpz_ld_a2_if` (`CzSeq.v`)	none
		`cmpz_ld_unclean` (`CzSeq.v`)	none

Result	Sec.	Coq name and file	Axioms
Theorem 18.3: What breaks	§18	`cmpz_and_costs_six` (`CzProdTC.v`)	none
		`cmpz_and_not_complete` (`CzProdTC.v`)	none
		`cmpz_or_clocks_not_complete` (`CzCounter.v`)	none
		`cmpz_shared_raises` (`CzShared.v`)	none
		`cmpz_shared_not_earned` (`CzShared.v`)	none
		`cmpz_exactness_not_preserved` (`CzWin.v`)	none
		`cmpz_sum_record_loses` (`CzCounter.v`)	
Theorem 18.4: Nesting	§18	`cmpz_sim_exit_cost` (`CzCat.v`)	none
		`cmpz_follow_exists` (`CzFollow.v`)	none
		`cmpz_follow_reaches` (`CzFollow.v`)	none
		`cmpz_follow_cost_ge_exits` (`CzFollow.v`)	none
			none
		`cmpz_end_only_not_enough` (`CzFollow.v`)	none
		`cmpz_sim_cost_ge_exits` (`CzCat.v`)	none
		`cmpz_priced_link` (`CzTower.v`)	none
		`cmpz_host_link` (`CzTower.v`)	none
		`cmpz_host_link_le_two` (`CzTower.v`)	none
		`cmpz_host_link_exact` (`CzTower.v`)	none
		`cmpz_host_link_start_up` (`CzTower.v`)	none
		`cmpz_link_compose` (`CzLink.v`)	none
		`cmpz_tower` (`CzLink.v`)	none
		`cmpz_tower_presented_exact` (`CzTower.v`)	
Theorem 19.1: The compiler is correct (for a source program with no recursion, the compiled program halts with y exactly when the source computes y)	§19	`cmp_final` (`CmpFinal.v`)	none
		`cmp_exec_machine` (`CmpRun.v`)	none
		`cmp_exec_sound` (`CmpRun.v`)	none
		`cmp_exec_complete` (`CmpRun.v`)	none
		`cmp_exec_unhalted` (`CmpRun.v`)	none
		`cmp_interp_sound` (`CmpLang.v`)	none
		`cmp_interp_complete` (`CmpLang.v`)	none
Definition 20.1: Correlators from the counts; the pinned matrix	§20	stated in the text	
Theorem 20.1: The integer check implies the pinned matrix is PSD	§20	`column_contractive_check_witness_sound` (`CHSHColumnCheck.v`)	FE, RD
		`column_contractive_check_witness_npa_psd` (`CHSHColumnCheck.v`)	FE, RD
		`npa_psd_iff_column_contractive` (if and only if) (`CHSHColumnCheck.v`)	FE, RD

Result	Sec.	Coq name and file	Axioms
Theorem 20.2: PSD forces the Tsirelson bound	§20	`npa_psd_implies_tsirelson_bound` (`CHSHColumnCheck.v`)	FE, RD
		`npa_psd_zero_marginal_implies_row_bounds` (`CHSHColumnCheck.v`)	FE, RD
		`chsh_gap_is_sum_of_squares` (`TsirelsonFromAlgebra.v`)	FE, RD
Corollary 20.3: A passed check means the bound	§20	the composition of Theorem 20.1 and Theorem 20.2	FE, RD
		`nec_e_check_strict` (`NecEChsh.v`)	FE, RD
		`nec_e_check_never_tsirelson` (`NecEChsh.v`)	none
		`nec_e_no_sqrt8` (`NecEChsh.v`)	FE, RD
		`nec_e_check_sharp` (`NecEChsh.v`)	
Definition 20.2: The packed tally, and the property PCHSH	§20	`small_chsh_tally` (`SmallChshCheck.v`)	def
		`small_chsh_tally_of` (`SmallChshMachine.v`)	def
		`small_chsh_meaning` (`SmallChshCheck.v`)	def
Theorem 20.4: The check is exact	§20	`small_chsh_check_iff` (`SmallChshCheck.v`)	FE, RD
		`small_chsh_clear_denominators` (`SmallChshCheck.v`)	FE, RD
		`small_chsh_psd_iff_contractive` (`SmallChshCheck.v`)	FE, RD
		`small_chsh_check_tsirelson` (`SmallChshCheck.v`)	FE, RD
		`small_chsh_eval_iff` (`SmallChshMachine.v`)	FE, RD
		`small_chsh_psd_tsirelson` (`SmallChshCheck.v`)	FE, RD
		`small_chsh_tally_of_code` (`SmallChshMachine.v`)	
		`nec_e_check_strict` (`NecEChsh.v`)	
		`nec_e_check_never_tsirelson` (`NecEChsh.v`)	
		`nec_e_check_sharp` (`NecEChsh.v`)	

Result	Sec.	Coq name and file	Axioms
Theorem 20.5: A raised flag stands on the check	§20	`small_chsh_flag_implies_tsirelson` (`SmallChshMachine.v`) `small_chsh_program_flag_implies_tsirelson` (`SmallChshMachine.v`) `small_chsh_chain_certifies_iff` (`SmallChshMachine.v`) `small_chsh_tally_certifies` (`SmallChshMachine.v`) `small_chsh_tally_refused_forever` (`SmallChshMachine.v`) `small_chsh_tally_certifies_iff` (`SmallChshMachine.v`) `nec_e_check_strict` (`NecEChsh.v`) `nec_e_check_sharp` (`NecEChsh.v`)	FE, RD FE, RD FE, RD FE, RD FE, RD FE, RD FE, RD FE, RD FE, RD
In the text: the check with a tolerance $\varepsilon = p/q$; at $\varepsilon = 0$ it is the exact check; a pass means operator norm at most $1 + \varepsilon$ and $S^2 \leq 8(1+\varepsilon)^2$; a noisy tally near Tsirelson's point fails the exact check and passes at $\varepsilon = 1/10$	§20	`nec_e_tol_check_zero` (`NecEChsh.v`) `nec_e_tol_check_sound` (`NecEChsh.v`) `nec_e_tol_check_norm` (`NecEChsh.v`) `nec_e_tol_check_bound` (`NecEChsh.v`) `nec_e_tol_check_accepts_noisy_tsirelson` (`NecEChsh.v`)	none FE, RD FE, RD FE, RD none
Definition 21.3: The two-state protocol	§21	stated in the text	
Theorem 21.4: The heat is fixed by the chosen gap	§21	`canonical_reset_satisfies_master_equation` (`CalorimeterProtocol.v`) `canonical_reset_heat_exact` (`CalorimeterProtocol.v`) `selected_gap_gives_landauer_heat` (`CalorimeterProtocol.v`) `canonical_reset_heat_below_landauer_at_small_gap` (given $0 < k_B$ and $0 < T$) (`CalorimeterProtocol.v`) `master_equation_does_not_fix_heat_scale` (`CalorimeterProtocol.v`)	FE, RD FE, RD FE, RD, EM FE, RD, EM FE, RD

Result	Sec.	Coq name and file	Axioms
In the text: the canonical rates break detailed balance; with rates in detailed balance a fixed-gap relaxation settles at the equilibrium population, $1/5$ at the gap $2k_BT\ln 2$, and never goes below it	§21	`canonical_rates_break_detailed_balance` (`CalorimeterProtocol.v`) `detailed_balance_never_empties` (`CalorimeterProtocol.v`) `landauer_gap_settles_at_one_fifth` (`CalorimeterProtocol.v`)	FE, RD FE, RD FE, RD, EM
Definition 21.4: The driven reset	§21	stated in the text	
Theorem 21.5: A driven reset pays $kT\ln 2$ in work	§21	`detailed_balance_settles_at_gibbs` (`CalorimeterProtocol.v`) `detailed_balance_thermalizing_step` (`CalorimeterProtocol.v`) `driven_first_law` (`CalorimeterProtocol.v`) `driven_work_second_law` (given $0 < kT$) (`CalorimeterProtocol.v`) `driven_work_near_free_energy` (given $0 < kT$) (`CalorimeterProtocol.v`) `driven_reset_work_window` (given $0 < k_BT$) (`CalorimeterProtocol.v`)	FE, RD FE, RD FE, RD FE, RD, EM FE, RD, EM FE, RD, EM
Definition B.1: Certification system	§B	`CertificationSystem` (`UniversalCertificationCost.v`, +1) `cs_cert_costs` (`UniversalCertificationCost.v`)	def def
Definition B.2: Quantitative certification system	§B	`QuantitativeCertificationSystem` (`QuantitativeNoFI.v`) `qcs_cost_bounds_witness` (`QuantitativeNoFI.v`) `qcs_cert_threshold_witness` (`QuantitativeNoFI.v`) `QuantitativeCertificationSystem_full` (`QuantitativeNoFI.v`)	def def def def
Theorem B.1: The floor raised to a threshold	§B	`universal_nfi_quantitative` (`QuantitativeNoFI.v`)	none
Definition F.1: Settings, outcomes, correlators	§F	stated in the text	
Definition F.2: Counters	§F	`WitnessCounts` (`CHSHColumnCheck.v`)	def

Result	Sec.	Coq name and file	Axioms
Lemma F.1: Deterministic strategies	§F	`local_S_2_deterministic` (`BoxCHSH.v`)	none
		`deterministic_strategy_chsh_bounded` (`MinorConstraints.v`)	FE, RD
Definition F.3: Finite factorizable model, `is_factorizable`	§F	`is_factorizable` (`MinorConstraints.v`)	def
Theorem F.2: `fine_theorem`, one direction only	§F	`fine_theorem` (`MinorConstraints.v`)	FE, RD
		`factorizable_CHSH_classical_bound` (`MinorConstraints.v`)	FE, RD
Remark F.1: What the name does and does not cover	§F	`fine_theorem` (`MinorConstraints.v`)	FE, RD
		`local_box_CHSH_bound` (`MinorConstraints.v`)	FE, RD
			FE, RD
		`nec_e_fine_converse_false` (`NecEFine.v`)	FE, RD
		`nec_e_fine_iff` (`NecEFine.v`)	
Definition F.4: `WCLocallyConsistent`	§F	`WCLocallyConsistent` (`CHSHStatisticalBridge.v`)	def
Theorem F.3: A tally above 2 is not one replayed strategy	§F	`chsh_violation_rules_out_locally_factorizable_coupling` (`CHSHCouplingBridge.v`)	none
			none
		`locally_consistent_classical_bound` (`CHSHCouplingBridge.v`)	def
		`chsh_stat_from_wc` (`CHSHStatisticalBridge.v`)	
Lemma F.4: The six-square identity, `chsh_gap_is_sum_of_squares`	§F	`chsh_gap_is_sum_of_squares` (`TsirelsonFromAlgebra.v`)	FE, RD
Theorem F.5: Row-bound form of Tsirelson's bound	§F	`tsirelson_squared` (`TsirelsonFromAlgebra.v`)	FE, RD
			FE, RD
		`tsirelson_from_row_bounds` (`TsirelsonGeneral.v`)	FE, RD
		`tsirelson_bound_abs` (`TsirelsonGeneral.v`)	
Theorem F.6: Tightness	§F	`tsirelson_tight` (`TsirelsonFromAlgebra.v`)	FE, RD
			FE, RD
		`tsirelson_achievable` (`TsirelsonGeneral.v`)	

Result	Sec.	Coq name and file	Axioms
Remark F.2: Where equality can happen	§F	`nec_e_check_strict` (`NecEChsh.v`)	FE, RD
		`nec_e_check_never_tsirelson` (`NecEChsh.v`)	FE, RD
		`nec_e_check_sharp` (`NecEChsh.v`)	FE, RD
		`nec_e_no_sqrt8` (`NecEChsh.v`)	none
		`nec_e_equality_forces_point` (`NecEChshEquality.v`)	FE, RD
		`nec_e_equality_two_points` (`NecEChshEquality.v`)	FE, RD
		`nec_e_no_rational_half_square` (`NecEChshEquality.v`)	FE, RD
In the text: quantum strategies with real amplitudes in finite dimension: every one has $S^2 \leq 8$ and a PSD level-1 moment matrix, a two-qubit one reaches $2\sqrt{2}$, and deterministic plans are strategies	§F	`qs_tsirelson` (`QuantumStrategies.v`)	FE, RD
		`qs_tsirelson_reached` (`QuantumStrategies.v`)	FE, RD
		`qs_deterministic_plan` (`QuantumStrategies.v`)	FE, RD
		`qs_corr_dot` (`QuantumStrategies.v`)	FE, RD
		`qs_npa_psd` (`QuantumStrategies.v`)	FE, RD
In the text: quantum strategies with complex amplitudes in finite dimension: every one has $S^2 \leq 8$ and a PSD level-1 moment matrix, and the two-qubit one reaches $2\sqrt{2}$	§F	`qc_tsirelson` (`QuantumStrategiesComplex.v`)	FE, RD
		`qc_tsirelson_reached` (`QuantumStrategiesComplex.v`)	FE, RD
		`qc_corr_dot` (`QuantumStrategiesComplex.v`)	FE, RD
		`qc_npa_psd` (`QuantumStrategiesComplex.v`)	FE, RD
		`qc_gram_psd` (`QuantumStrategiesComplex.v`)	FE, RD

Result	Sec.	Coq name and file	Axioms
In the text: Tsirelson's representation: the completion set is exactly the set of correlator tables of quantum strategies in finite dimension, real or complex (if and only if); every PSD matrix is a Gram matrix; four anticommuting real symmetric involutions on eight levels	§F	`tr_representation` (`TsirelsonRepresentation.v`) `tr_representation_complex` (`TsirelsonRepresentation.v`) `tr_elliptope_quantum` (`TsirelsonRepresentation.v`) `tr_quantum_elliptope` (`TsirelsonRepresentation.v`) `tr_complex_elliptope` (`TsirelsonRepresentation.v`) `tr_gram` (`TsirelsonRepresentation.v`) `tr_gamma_clifford` (`TsirelsonRepresentation.v`) `tr_corr_inner` (`TsirelsonRepresentation.v`)	FE, RD FE, RD FE, RD FE, RD FE, RD FE, RD FE, RD FE, RD
In the text: in any dimension, for any expectation on an algebra of observables where Alice's commute with Bob's, Tsirelson's bound holds, the correlators have a PSD completion and satisfy the arcsine inequalities, and the first-level moment matrix with marginals is PSD	§20	`ta_quad_nonneg` (`TsirelsonAlgebraic.v`) `ta_tsirelson` (`TsirelsonAlgebraic.v`) `ta_realizable` (`TsirelsonAlgebraic.v`) `ta_arcsine` (`TsirelsonAlgebraic.v`) `ta_npa_psd` (`TsirelsonAlgebraic.v`) `ta_classical_instance` (`TsirelsonAlgebraic.v`)	FE, RD FE, RD FE, RD FE, RD, EM FE, RD FE, RD
In the text: the arcsine characterization of Landau and Masanes: a table has a PSD completion, and is a quantum table, exactly when its entries lie in $[-1, 1]$ and its arcsines pass four inequalities (if and only if); the angle on the sphere obeys the triangle inequality; two boundary points	§F	`am_arcsine` (`ArcsineBoundary.v`) `am_quantum_arcsine` (`ArcsineBoundary.v`) `am_necessary` (`ArcsineBoundary.v`) `am_sufficient` (`ArcsineBoundary.v`) `am_sphere_triangle` (`ArcsineBoundary.v`) `am_pythagorean_boundary` (`ArcsineBoundary.v`) `am_tsirelson_boundary` (`ArcsineBoundary.v`)	FE, RD, EM FE, RD, EM FE, RD, EM FE, RD, EM FE, RD, EM FE, RD, EM FE, RD, EM

Result	Sec.	Coq name and file	Axioms
Lemma F.7: A rational constant above the bound, `rational_tsirelson_bound`	§F	`rational_tsirelson_bound` (`TsirelsonFromAlgebra.v`)	FE, RD
Definition F.5: The pinned level-1 matrix	§F	`zero_marginal_npa` (`NPAMomentMatrix.v`) `NPAMomentMatrix` (`NPAMomentMatrix.v`) `npa_to_matrix` (`NPAMomentMatrix.v`)	def def def
Lemma F.8: Schur complement, the standard form (if and only if)	§F	`schur_identity` (`SchurComplement.v`) `schur_complement_psd` (`SchurComplement.v`)	FE, RD FE, RD
Definition F.6: Column contractive, `zero_marginal_column_contractive`	§F	`zero_marginal_column_contractive` (`CHSHColumnCheck.v`)	def
Lemma F.9: The 2x2 test, `psd2_quadratic_form_nonneg`	§F	`psd2_quadratic_form_nonneg` (`CHSHColumnCheck.v`)	FE, RD
Theorem F.10: Contractive implies PSD	§F	`zero_marginal_npa_column_contractive_implies_psd` (`CHSHColumnCheck.v`) PSD5 (`ConstructivePSD.v`)	FE, RD def
Lemma F.11: Discriminant, `quadratic_nonneg_discriminant`	§F	`quadratic_nonneg_discriminant` (`ConstructivePSD.v`)	FE, RD
Theorem F.12: PSD implies contractive	§F	`npa_psd_implies_column_contractive` (`CHSHColumnCheck.v`) `npa_quad5_test_col0` (`CHSHColumnCheck.v`) `npa_quad5_test_col1` (`CHSHColumnCheck.v`) `npa_quad5_test_schur` (`CHSHColumnCheck.v`)	FE, RD FE, RD FE, RD FE, RD
Corollary F.13: The equivalence	§F	`npa_psd_iff_column_contractive` (if and only if) (`CHSHColumnCheck.v`) `column_contractive_iff_npa_psd` (`CHSHColumnCheck.v`) PSD5 (`ConstructivePSD.v`) `npa_psd` (`NPAMomentMatrix.v`)	FE, RD FE, RD def def

Result	Sec.	Coq name and file	Axioms
Theorem F.14: PSD implies Tsirelson	§F	`npa_psd_implies_tsirelson_bound` (`CHSHColumnCheck.v`)	FE, RD
		`npa_psd_implies_tsirelson_bound_abs` (`CHSHColumnCheck.v`)	FE, RD
		`psd_3x3_determinant_nonneg` (`ConstructivePSD.v`)	FE, RD
		`npa_psd_zero_marginal_implies_row_bounds` (`CHSHColumnCheck.v`)	FE, RD
Remark F.3: What PSD of the pinned matrix means	§F	`zero_marginal_implies_elliptope` (`ElliptopeCompletion.v`)	RD
Definition F.7: `column_contractive_check_witness`	§F	`column_contractive_check_witness` (`CHSHColumnCheck.v`)	def
Lemma F.15: The seven facts are the three conditions with denominators cleared	§F	`small_chsh_clear_denominators` (`SmallChshCheck.v`)	FE, RD
Theorem F.16: Soundness of the integer check	§F	`column_contractive_check_witness_sound` (`CHSHColumnCheck.v`)	FE, RD
		`column_contractive_check_witness_npa_psd` (`CHSHColumnCheck.v`)	FE, RD
Corollary F.17: A passed check implies the Tsirelson bound	§F	the composition of Theorem F.16, Corollary F.13 and Theorem F.14; on the small machine it is one theorem, `small_chsh_check_tsirelson` (`SmallChshCheck.v`)	FE, RD
Definition F.8: `q1ab_moment_matrix`	§F	`q1ab_moment_matrix` (`QuantumPartitionPSD_1AB.v`)	def
In the text: the four-body tie is the slice $\langle\{A_0, A_1\} \otimes \{B_0, B_1\}\rangle = 0$, which a classical strategy breaks	§F	`q1ab_tie_is_a_constraint` (`QuantumPartitionPSD_1AB.v`)	FE, RD
		`q1ab_four_body_cells_are_conjugate` (`QuantumPartitionPSD_1AB.v`)	none
Definition F.9: The residual, `q1ab_residual`	§F	`q1ab_residual` (`QuantumPartitionPSD_1AB.v`)	def
		`column_contractive_q1ab` (`QuantumPartitionPSD_1AB.v`)	def
Lemma F.18: Three squares plus the residual, `quad9_q1ab_sos_decomposition`	§F	`quad9_q1ab_sos_decomposition` (`QuantumPartitionPSD_1AB.v`)	FE, RD

Result	Sec.	Coq name and file	Axioms
Theorem F.19: PSD at level 1+AB, q1ab_psd_iff_column_contractive	§F	q1ab_psd_iff_column_contractive (if and only if) (QuantumPartitionPSD_1AB.v) column_contractive_q1ab_implies_psd9 (QuantumPartitionPSD_1AB.v) psd9_implies_column_contractive_q1ab (QuantumPartitionPSD_1AB.v) PSD9 (QuantumPartitionPSD_1AB.v) column_contractive_q1ab (QuantumPartitionPSD_1AB.v)	FE, RD FE, RD FE, RD def def
Lemma F.20: Fraction-free Schur step	§F	sym6_Schur_identity (QuantumPartitionPSD_1AB.v) sym5_Schur_identity (QuantumPartitionPSD_1AB.v) sym6_qf_nonneg_from_pd (QuantumPartitionPSD_1AB.v) sym5_qf_nonneg_from_pd (QuantumPartitionPSD_1AB.v)	FE, RD FE, RD FE, RD FE, RD
Lemma F.21: Sylvester for 4x4, sym4_qf_nonneg_from_pd	§F	sym4_qf_nonneg_from_pd (QuantumPartitionPSD_1AB.v) sym4_LDLT_identity (QuantumPartitionPSD_1AB.v) sym4_d1 (QuantumPartitionPSD_1AB.v) sym4_d4 (QuantumPartitionPSD_1AB.v) sym4_Q2 (QuantumPartitionPSD_1AB.v)	FE, RD FE, RD def def def
Definition F.10: What the integer check computes	§F	g12345_COMMON_Z (CHSHColumnCheck.v) cleared_g12345_H11_Z (CHSHColumnCheck.v) cleared_g12345_H66_Z (CHSHColumnCheck.v) schur_step_Z (CHSHColumnCheck.v) q1ab_g12345_check_z_kernel (CHSHColumnCheck.v)	def def def def def
Theorem F.22: The full 1+AB check implies PSD	§F	q1ab_g12345_caller_witness_z_abs_sound (QuantumPartitionPSD_1AB.v) q1ab_g12345_minors_witness_implies_psd9 (QuantumPartitionPSD_1AB.v)	FE, RD FE, RD
Remark F.4: Interior only	§F	nec_e_singular_level1_passes (NecEChshEquality.v) nec_e_singular_full_fails (NecEChshEquality.v)	none none

Result	Sec.	Coq name and file	Axioms
In the text: the worked cascade passes the full 1+AB check, fails it at a zero last moment, and passes the level-1 check	§F	`nec_e_worked_cascade_passes` (`NecEChshEquality.v`) `nec_e_worked_cascade_fails_at_zero` (`NecEChshEquality.v`) `nec_e_worked_level1_passes` (`NecEChshEquality.v`)	none none none
Theorem F.23: The zero-moment check sits inside the classical bound	§F	`q1ab_check_at_gzero_forces_unit_ball` (`QuantumPartitionPSD_1AB.v`) `q1ab_check_at_gzero_implies_classical_bound` (`QuantumPartitionPSD_1AB.v`) `q1ab_residual_g_zero_decomp` (`QuantumPartitionPSD_1AB.v`) `column_contractive_check_q1ab_sound_at_g_zero` (`QuantumPartitionPSD_1AB.v`) `column_contractive_check_q1ab` (`QuantumPartitionPSD_1AB.v`)	FE, RD FE, RD FE, RD FE, RD def
Definition F.11: Completion and membership	§F	`completed_npa` (`EllitopeCompletion.v`) `elliptope_realizable` (`EllitopeCompletion.v`)	def def
Theorem F.24: Deterministic plans are inside, `deterministic_strategy_elliptope`	§F	`deterministic_strategy_elliptope` (`EllitopeCompletion.v`)	FE, RD
Theorem F.25: Convexity and finite mixtures	§F	`elliptope_convex` (`EllitopeCompletion.v`) `elliptope_finite_mixture` (`EllitopeCompletion.v`) `lhv_mixture_elliptope` (`EllitopeCompletion.v`) `elliptope_zero` (`EllitopeCompletion.v`)	FE, RD FE, RD FE, RD FE, RD
Theorem F.26: Tsirelson's bound holds on the completion set, `elliptope_tsirelson`	§F	`elliptope_tsirelson` (`EllitopeCompletion.v`) `psd_cauchy_schwarz` (`EllitopeCompletion.v`)	FE, RD FE, RD
Corollary F.27: The PR box is outside, `pr_box_not_elliptope`	§F	`pr_box_not_elliptope` (`EllitopeCompletion.v`)	FE, RD
Theorem F.28: Strictly beyond the classical bound, `beyond_classical_elliptope`	§F	`beyond_classical_elliptope` (`EllitopeCompletion.v`)	FE, RD

Result	Sec.	Coq name and file	Axioms
Definition F.12: The two branches	§F	elliptope_pd_check (EllipticGate.v)	def
		elliptope_ldl_check (EllipticGate.v)	def
		ldl_equations (EllipticGate.v)	def
		elliptope_check_full (EllipticGate.v)	def
Theorem F.29: Soundness of the gate, elliptope_check_full_sound	§F	elliptope_check_full_sound (EllipticGate.v)	FE, RD
			FE, RD
		elliptope_full_gate_never_accepts_pr_box (EllipticGate.v)	FE, RD
			FE, RD
		elliptope_pd_check_sound (EllipticGate.v)	def
		elliptope_ldl_check_sound (EllipticGate.v)	
		elliptope_check_full (EllipticGate.v)	

E

The exact words of twenty-one results

This book states its theorems in its own words, and Section 22 says why that's a risk: the checker confirms a proof matches its statement. Whether my sentence matches the statement is a question the checker never asks. So here are the statements themselves for twenty-one of the results the argument leans on hardest. Each is copied from its file with the comments taken out and a few long lines broken to fit the page, which Coq doesn't notice, and nothing else changed, and each comes with its file, a plain reading, and a line on whether the book's printed version says exactly that, more, or less. You don't need to read Coq to use this. Put the printed theorem next to the block and check that every premise in the block turns up in the sentence. That's the whole test. These are the statements this book was checked against; if a file differs later, the book means the one printed here.

E.1 The definitions they rest on

A statement is only as plain as the definitions inside it, and the readings below describe several of those definitions in words. Here are the ones they lean on hardest, copied the same way: the small machine's step, the meaning of the universal reading checker, the simulations that nesting is about, the meaning of the CHSH property, and the universal base with the interface that carries it.

The step of the small machine, with the checks it runs and `untouched`, minimal/EarnedCore.v. `prop`, `eval`, `fact`, `fact_eqb`, `cost` and `run` are as in Definitions 16.1, 16.2 and 16.3.

```
Inductive instr : Type :=
| INC (c : ctr)
| DEC (c : ctr) (j : nat)
| HALT
| CHECK (p : prop) (c : ctr)
| COMMIT (p : prop) (c : ctr)
| CERTIFY.

Record core : Type := mkcore {
  ca : nat; cb : nat;
  va : nat; vb : nat;
```

```
  pc : nat;
  facts : list fact;
  chan : option fact;
  err : bool
}.

Definition val (k : core) (c : ctr) : nat :=
  match c with CA => ca k | CB => cb k end.

Definition ver (k : core) (c : ctr) : nat :=
  match c with CA => va k | CB => vb k end.

Definition claim (k : core) (p : prop) (c : ctr) : fact := mkfact p c (ver k c).

Definition write (k : core) (c : ctr) (n j : nat) : core :=
  match c with
  | CA => mkcore n (cb k) (S (va k)) (vb k) j (facts k) (chan k) (err k)
  | CB => mkcore (ca k) n (va k) (S (vb k)) j (facts k) (chan k) (err k)
  end.

Definition goto (k : core) (j : nat) : core :=
  mkcore (ca k) (cb k) (va k) (vb k) j (facts k) (chan k) (err k).

Definition trap (k : core) : core :=
  mkcore (ca k) (cb k) (va k) (vb k) (pc k) (facts k) (chan k) true.

Definition record_fact (k : core) (f : fact) : core :=
  mkcore (ca k) (cb k) (va k) (vb k) (S (pc k)) (f :: facts k) (chan k) (err k).

Definition commit_to (k : core) (f : fact) : core :=
  mkcore (ca k) (cb k) (va k) (vb k) (S (pc k)) (facts k) (Some f) (err k).

Definition fact_cap : nat := 16.

Definition check_ok (k : core) (p : prop) (c : ctr) : bool :=
  negb (err k) && eval p (val k c) && Nat.ltb (length (facts k)) fact_cap.

Definition commit_ok (k : core) (p : prop) (c : ctr) : bool :=
  negb (err k) && existsb (fact_eqb (claim k p c)) (facts k).

Definition certify_ok (k : core) : bool :=
  negb (err k) && match chan k with Some _ => true | None => false end.

Definition cexec (k : core) (i : instr) : core :=
  if err k then k else
  match i with
  | INC c => write k c (S (val k c)) (S (pc k))
  | DEC c j =>
      match val k c with
      | 0 => goto k (S (pc k))
      | S n => write k c n j
      end
  | HALT => k
  | CHECK p c => if check_ok k p c then record_fact k (claim k p c) else trap k
  | COMMIT p c => if commit_ok k p c then commit_to k (claim k p c) else trap k
```

```
  | CERTIFY => if certify_ok k then goto k (S (pc k)) else trap k
  end.

Definition fires (k : core) (i : instr) : bool :=
  match i with CERTIFY => certify_ok k | _ => false end.

Record state : Type := mkst { core_of : core; mu : nat; cert : bool }.

Definition exec (s : state) (i : instr) : state :=
  mkst (cexec (core_of s) i) (mu s + cost i) (cert s || fires (core_of s) i).

Definition untouched (s : state) (mid : list instr) (c : ctr) : Prop :=
  forall t1 i t2, mid = t1 ++ i :: t2 ->
    let k := core_of (run t1 s) in
    ver (cexec k i) c = ver k c /\ val (cexec k i) c = val k c.
```

In plain words: a core holds the two counters, their versions, the line number, the fact table, the channel and the trap bit. `claimkpc` is the fact made of `p`, `c` and `c`'s version now. CHECK passes when the trap is down, the checker says yes of the counter's value, and the table holds fewer than 16 facts, and it adds the claim to the table. COMMIT passes when the trap is down and that exact claim, version and all, is in the table, and it puts the claim in the channel in place of whatever was there. CERTIFY takes no operand: it passes when the trap is down and the channel holds a fact, any fact. A failing one sets the trap and changes nothing else, and a core with the trap up stays as it is. `exec` runs the core's step, adds the instruction's cost to the ledger, and raises the flag when a passing CERTIFY runs. `untouchedsmidc` says each instruction of `mid`, at the point where it runs from `s`, leaves `c`'s value and version as they were. Definitions 16.3 and 16.5 say exactly this.

`cg_uholds`, the meaning of the universal reading checker, with the claims it reads and the routine's start, coq/kernel/foundation/CompilerChecker.v.

```
Inductive cg_uprop : Type :=
| UBase (q : G.cprop)
| URun (r : nat).

Definition cg_env_chk (m v : nat) : env nat nat :=
  fun x => if Nat.ltb x m then cg_expo (qs x) v else 0.

Definition cg_uholds (p : cg_uprop) (v : nat) : Prop :=
  match p with
  | UBase q => G.cholds q v
  | URun r =>
      exists ig R xS xB xT m k st,
        cg_rdec r = Some (ig, R, xS, xB, xT, m) /\
        k <= cg_expo (qs xT) v /\
        sss_steps (mm_sss_env eq_nat_dec) (ig, R) k (ig, cg_env_chk m v) st /\
        out_code (fst st) (ig, R) /\
        get_env (snd st) xB = 1
  end.
```

In plain words: a claim is `UBaseq`, a word of the counter language, or `URunr`, a number naming a reading routine. A word means what it meant on the small machine. A routine claim holds of a value `v` when `r` decodes to a routine (its first line, its code, and three register numbers and a register count), and the routine, started at its first line with each register below the count holding the exponent of that register's prime in `v`, reaches in some number of steps no larger than the exponent of the fuel register's prime in `v` a state outside its code with 1 in its answer register. `cg_ueval_iff`, printed below, says the checker's yes is exactly this, and Section 16.14 says the same.

`cmpz_hom`, the simulations Theorem 18.4 is about, with the objects they join, coq/kernel/foundation/AxCore.v, AxComplete.v and CzCat.v. `T.run` is `run` of minimal/ThieleComplete.v, printed further down.

```
Record BPre (A : Type) : Type := {
  bp_leb : A -> A -> bool;
  bp_refl : forall x, bp_leb x x = true;
  bp_trans : forall x y z, bp_leb x y = true -> bp_leb y z = true ->
    bp_leb x z = true
}.

Definition bp_le {A} (P : BPre A) (x y : A) : Prop := bp_leb A P x y = true.

Record amachine (A : Type) (P : BPre A) : Type := mk_am {
  am_state : Type;
  am_move : Type;
  am_step : am_state -> am_move -> am_state;
  am_cost : am_move -> nat;
  am_rec : am_state -> A
}.

Definition am_pt {A P} (AM : amachine A P) (r : am_state AM -> bool) : T.machine :=
  T.mk_machine (am_state AM) (am_move AM) (am_step AM) (am_cost AM) r.

Definition am_bare {A P} (AM : amachine A P) : T.machine :=
  am_pt AM (fun _ => false).

Definition am_run {A P} (AM : amachine A P) (tr : list (am_move AM))
    (s : am_state AM) : am_state AM :=
  T.run (am_bare AM) tr s.

Record cmpz_obj : Type := mk_obj {
  ob_A : Type;
  ob_P : BPre ob_A;
  ob_M : amachine ob_A ob_P
}.

Record cmpz_hom (X Y : cmpz_obj) : Type := mk_hom {
  h_state : am_state (ob_M X) -> am_state (ob_M Y);
  h_rec : ob_A X -> ob_A Y;
  h_mono : forall a b, bp_le (ob_P X) a b -> bp_le (ob_P Y) (h_rec a) (h_rec b);
  h_rec_nat : forall s, am_rec (ob_M Y) (h_state s) = h_rec (am_rec (ob_M X) s);
```

```
  h_sim : forall s m, exists tr : list (am_move (ob_M Y)),
            h_state (am_step (ob_M X) s m) = am_run (ob_M Y) tr (h_state s)
}.
```

In plain words: an object is a type of records with an order given by a yes-or-no test, and a machine whose record lives in that type. A morphism from X to Y is a map on states and a map on records such that the record map keeps the order, reading the record after mapping the state gives the record map of the old record, and each move of X is matched by some list of moves of Y that lands on the image of the new state. That last field asks for some list and names none, which is why Theorem 18.4(1) and the statement printed for it below speak of some host trace.

`small_chsh_meaning`, the meaning of the property PCHSH, with the correlators and the score it's read with, coq/kernel/quantum/SmallChshCheck.v, and the three definitions it uses from TsirelsonFromAlgebra.v and NPAMomentMatrix.v. `npa_to_matrix` lays the ten entries out as the five-by-five matrix $M(E)$ of Section 20.

```
Definition CHSH_value (e00 e01 e10 e11 : R) : R :=
  e00 + e01 + e10 - e11.

Definition npa_psd (npa : NPAMomentMatrix) : Prop :=
  let M := nat_matrix_to_fin5 (npa_to_matrix npa) in
  symmetric5 M /\ PSD5 M.

Definition zero_marginal_npa (E00 E01 E10 E11 : R) : NPAMomentMatrix := {|
  npa_EA0 := 0;
  npa_EA1 := 0;
  npa_EB0 := 0;
  npa_EB1 := 0;
  npa_E00 := E00;
  npa_E01 := E01;
  npa_E10 := E10;
  npa_E11 := E11;
  npa_rho_AA := 0;
  npa_rho_BB := 0;
|}.

Definition small_chsh_corr (same diff : nat) : R :=
  if Nat.eqb (same + diff) 0 then 0
  else (INR same - INR diff) / INR (same + diff).

Definition small_chsh_e00 (t : small_chsh_tally) : R :=
  small_chsh_corr t.(small_chsh_same00) t.(small_chsh_diff00).

Definition small_chsh_e01 (t : small_chsh_tally) : R :=
  small_chsh_corr t.(small_chsh_same01) t.(small_chsh_diff01).

Definition small_chsh_e10 (t : small_chsh_tally) : R :=
  small_chsh_corr t.(small_chsh_same10) t.(small_chsh_diff10).

Definition small_chsh_e11 (t : small_chsh_tally) : R :=
  small_chsh_corr t.(small_chsh_same11) t.(small_chsh_diff11).
```

```
Definition small_chsh_score (t : small_chsh_tally) : R :=
  CHSH_value (small_chsh_e00 t) (small_chsh_e01 t)
             (small_chsh_e10 t) (small_chsh_e11 t).

Definition small_chsh_sampled (t : small_chsh_tally) : Prop :=
  (0 < t.(small_chsh_same00) + t.(small_chsh_diff00))%nat /\
  (0 < t.(small_chsh_same01) + t.(small_chsh_diff01))%nat /\
  (0 < t.(small_chsh_same10) + t.(small_chsh_diff10))%nat /\
  (0 < t.(small_chsh_same11) + t.(small_chsh_diff11))%nat.

Definition small_chsh_psd (t : small_chsh_tally) : Prop :=
  npa_psd (zero_marginal_npa (small_chsh_e00 t) (small_chsh_e01 t)
                             (small_chsh_e10 t) (small_chsh_e11 t)).

Definition small_chsh_meaning (t : small_chsh_tally) : Prop :=
  small_chsh_sampled t /\ small_chsh_psd t.
```

In plain words: the correlator of a question pair is the number of rounds that agreed minus the number that disagreed, over the number of rounds, read as 0 when the pair had none. The score is the CHSH combination of the four correlators. The meaning is that every question pair has at least one round and the pinned matrix of the four correlators, with every one-sided average and both same-side entries set to 0, is symmetric and positive semidefinite. Definition 20.2 says exactly this.

The universal base and the interface that carries it, minimal/ThieleComplete.v. `machine` has the fields `m_state`, `m_move`, `m_step`, `m_cost` and `m_record`.

```
Inductive reg : Type := RA | RB.

Inductive cm_instr : Type := CINC (r : reg) | CDEC (r : reg) (j : nat).

Definition cm_conf : Type := (nat * (nat * nat))%type.

Definition cm_get (x : cm_conf) (r : reg) : nat :=
  match r with RA => fst (snd x) | RB => snd (snd x) end.

Definition cm_set (x : cm_conf) (r : reg) (n j : nat) : cm_conf :=
  match r with RA => (j, (n, snd (snd x))) | RB => (j, (fst (snd x), n)) end.

Definition cm_exec (i : cm_instr) (x : cm_conf) : cm_conf :=
  match i with
  | CINC r => cm_set x r (S (cm_get x r)) (S (fst x))
  | CDEC r j =>
      match cm_get x r with
      | 0 => (S (fst x), snd x)
      | S n => cm_set x r n j
      end
  end.

Record universal_base (M : machine) : Type := mk_ub {
  ub_window : m_state M -> cm_conf;
  ub_live : m_state M -> Prop;
  ub_compile : cm_instr -> m_move M;
  ub_load : nat -> nat -> m_state M;
```

```
  ub_load_window : forall a b, ub_window (ub_load a b) = (1, (a, b));
  ub_load_live : forall a b, ub_live (ub_load a b);
  ub_sim : forall s i, ub_live s ->
    ub_window (m_step M s (ub_compile i)) = cm_exec i (ub_window s) /\
    ub_live (m_step M s (ub_compile i))
}.

Inductive kind (C : Type) : Type :=
| KBase
| KCheck (c : C)
| KCommit (c : C)
| KCertify.

Record thiele_interface (M : machine) : Type := mk_ti {
  ti_base : universal_base M;
  ti_claim : Type;
  ti_kind : m_move M -> kind ti_claim;
  ti_meaning : ti_claim -> m_state M -> Prop;
  ti_check : m_state M -> ti_claim -> bool;
  ti_same : ti_claim -> m_state M -> m_state M -> Prop;
  ti_clean : m_state M -> Prop;
  ti_ledger : m_state M -> nat
}.
```

In plain words: a two-counter configuration is a line number and two registers, and `cm_exec` is what one instruction does to it, the same INC and DEC-or-fall-through as the small machine's base. A universal base for a machine reads a configuration off each state (its window), picks out the live states, gives a move for each instruction and a loaded state for each start; each loaded state shows $(1, a, b)$ and is live, and from every live state the compiled move changes the window exactly as `cm_exec` says and leaves the state live. The interface lists its parts and asks nothing of them: the base, the claims, a kind for each move, each claim's meaning, the checker, "unchanged," the clean states and the ledger. The demands are the four clauses printed below under Thiele-complete. This is part (a) of Definition 16.7 and the interface it describes.

E.2 The universal floor

`universal_nfi_any_substrate`, coq/kernel/nfi/UniversalCertificationCost.v, with the record it quantifies over.

```
Record CertificationSystem := mk_cert_system {
  cs_state : Type;
  cs_instr : Type;
  cs_step  : cs_state -> cs_instr -> cs_state;
  cs_cost  : cs_instr -> nat;
  cs_cert  : cs_state -> bool;
  cs_cert_costs :
    forall (s : cs_state) (i : cs_instr),
      cs_cert s = false ->
```

```
      cs_cert (cs_step s i) = true ->
      cs_cost i >= 1;
}.

Theorem universal_nfi_any_substrate :
  forall (CS : CertificationSystem)
         (trace : list (cs_instr CS))
         (s0 : cs_state CS),
    cs_cert CS s0 = false ->
    cs_cert CS (cs_run CS trace s0) = true ->
    cs_total_cost CS trace >= 1.
```

In plain words: a certification system is a record of a state type, a move type, a step, a cost and a yes-or-no reading, plus one more field, `cs_cert_costs`, which is the toll: a move that takes the reading from false to true costs at least one. Since the toll is a field, every system the theorem talks about has it, which is why the same file also states it as `every_toll_system_pays_the_floor`. The theorem: start at a state reading false, run any list of moves (`cs_run` applies them in order, `cs_total_cost` adds their costs), end at a state reading true, and the costs add up to at least one. Observation 3.1 says exactly this; the rule that Definition 3.1 calls the toll is the record's last field here.

E.3 A permanent flip merges

`permanent_flip_is_not_injective`, coq/kernel/nfi/PermanentCertification.v, inside a section that fixes the states, the moves, the step and the reading.

```
Variables (S I : Type).
Variable step : S -> I -> S.
Variable cert : S -> bool.

Definition finite_states (all : list S) : Prop :=
  NoDup all /\ forall s, In s all.

Definition permanent : Prop :=
  forall s i, cert s = true -> cert (step s i) = true.

Definition step_injective (i : I) : Prop :=
  forall a b, step a i = step b i -> a = b.

Theorem permanent_flip_is_not_injective :
  forall all s i,
    finite_states all ->
    permanent ->
    cert s = false ->
    cert (step s i) = true ->
    ~ step_injective i.
```

In plain words: `finite_states` *all* says the list *all* has no repeats and every state is on it, so there are finitely many. `permanent` says once the reading is true, every move keeps it true. `step_injective i` says move *i* never sends two different states to the same one. The theorem: on a finite machine with a permanent reading, a move that takes some state from false to true isn't injective, so it merges two states. Theorem 4.1 says exactly this, with "merges" meaning "isn't injective".

E.4 The toll from finiteness, permanence and merge pricing

`a2_from_merging_price_and_permanence`, the same file and section, with a cost on moves added.

```
Variable cost : I -> nat.

Definition merging_steps_priced : Prop :=
  forall i, ~ step_injective i -> cost i >= 1.

Definition a2_holds : Prop :=
  forall s i, cert s = false -> cert (step s i) = true -> cost i >= 1.

Theorem a2_from_merging_price_and_permanence :
  forall all,
    finite_states all ->
    permanent ->
    merging_steps_priced ->
    a2_holds.
```

In plain words: `merging_steps_priced` says every move that isn't injective costs at least one, and `a2_holds` is the toll. The theorem: finite, permanent and merges priced give the toll. Observation 4.2 says this and one thing more, that the machine with that cost is then a certification system the universal floor applies to. That second half isn't in this statement. It's the definition right after it in the file (`certification_system_from_merging_price`), which fills the record's toll field with this theorem, so the printed form is true, and it leans on two Coq items: this theorem and that definition.

E.5 No exact window price across a collision

`shadow_cannot_price_exactly`, coq/kernel/nfi/ShadowPricing.v, inside a section that fixes the states, moves, step, reading and a window `obs`.

```
Variables (S I O : Type).
Variable step : S -> I -> S.
Variable cert : S -> bool.
Variable obs : S -> O.

Definition flips (s : S) (i : I) : bool :=
```

```
  negb (cert s) && cert (step s i).

Definition shadow_cost (price : O -> O -> nat) (s : S) (i : I) : nat :=
  price (obs s) (obs (step s i)).

Definition meets_floor (cost : S -> I -> nat) : Prop :=
  forall s i, flips s i = true -> cost s i >= 1.

Definition never_overcharges (cost : S -> I -> nat) : Prop :=
  forall s i, flips s i = false -> cost s i = 0.

Definition shadow_collision : Prop :=
  exists s1 i1 s2 i2,
    obs s1 = obs s2 /\
    obs (step s1 i1) = obs (step s2 i2) /\
    flips s1 i1 = true /\
    flips s2 i2 = false.

Theorem shadow_cannot_price_exactly :
  shadow_collision ->
  forall price,
    ~ (meets_floor (shadow_cost price) /\
       never_overcharges (shadow_cost price)).
```

In plain words: `flips s i` says move *i* takes *s* from false to true. A shadow price is any function of what the window shows before and after a step. `meets_floor` says every flip costs at least one, `never_overcharges` says every step that isn't a flip costs nothing, and a `shadow_collision` is two steps that look the same through the window, before and after, one a flip and one not. The theorem: given a collision, no shadow price does both. Observation 7.2 bundles this with three neighbours from the same file: that a price meeting the floor charges some non-flip at least one (`shadow_floor_overcharges`), that a window showing the reading prices exactly (`window_showing_reading_prices_exactly`), and that such a window has no collision (`window_showing_reading_has_no_collision`). This statement is the second of the four clauses, and the book says exactly what it says.

E.6 The record is a latch

`record_axis_is_latch_holds`, coq/kernel/foundation/StructuralRecordAxis.v; the definitions come from StructuralCoreAnyBase.v and StructuralCoreCover.v.

```
Definition record_permanent (M : RCM) : Prop :=
  forall s, rc_cert M s = true -> rc_cert M (rc_next M s) = true.

Definition computation_driven (M : RCM) (B : BaseMachine) (C : BaseCover M B)
  : Prop :=
  exists f : b_state B -> bool -> bool,
    forall m, rc_cert M (rc_next M m) = f (base_state M B C m) (rc_cert M m).
```

```
Definition HonestBaseExtension (M : RCM) (B : BaseMachine) (C : BaseCover M B)
  : Prop :=
  computation_driven M B C /\ ledger_carried M /\ rc_a2 M /\
  record_permanent M /\ reachable_record_write M.

Definition latch_next (B : BaseMachine) (h : b_state B -> bool)
    (x : b_state B * bool) : b_state B * bool :=
  (b_next B (fst x), orb (snd x) (h (fst x))).

Definition latch_factorization (M : RCM) (B : BaseMachine) (C : BaseCover M B)
    (h : b_state B -> bool) : Prop :=
  forall m,
    (base_state M B C (rc_next M m), rc_cert M (rc_next M m)) =
    latch_next B h (base_state M B C m, rc_cert M m).

Definition record_axis_is_latch : Prop :=
  forall M B C, HonestBaseExtension M B C ->
    exists h, latch_factorization M B C h.

Theorem record_axis_is_latch_holds : record_axis_is_latch.
```

In plain words: here a machine M steps by itself (`rc_next` gives the one next state), carries a reading `rc_cert` and a ledger, and sits over a base B through a cover `C`, a map down to the base's states that commutes with the step. `computation_driven` says the reading's next value is some function `f` of the base state and the reading's current value. `HonestBaseExtension`, the book's base-driven record, means driven, a ledger that never falls, the toll, a permanent reading, and some reachable step that writes it. `latch_next` is the latch of an event `h`: the base steps, and the record becomes "already on, or `h` holds now." The theorem: every base-driven record evolves exactly as the latch of some event. Observation 10.1 says a little more than this statement: it names the event, $h(b) = f(b, \text{off})$, where the statement only says one exists. The proof builds exactly that event (its first move is `exists (fun b => f b false)`), so it shows the theorem's stronger form too. And the proof uses two of the five conditions, driven and permanent; the other three are carried along and never used. The form with only those two as hypotheses is `record_axis_is_latch_r1_r4` (StructuralClockSync.v), and that is the form Observation 10.1 states.

E.7 A raised flag was checked, then committed, then certified

`earned_certification_same_claim`, minimal/EarnedCore.v.

```
Theorem earned_certification_same_claim : forall s0 tr,
  clean_start s0 -> cert (run tr s0) = true ->
  exists pre1 p c mid1 mid2 post,
    tr = pre1 ++ CHECK p c :: mid1 ++ COMMIT p c :: mid2 ++ CERTIFY :: post /\
    check_ok (core_of (run pre1 s0)) p c = true /\
    commit_ok (core_of (run (pre1 ++ CHECK p c :: mid1) s0)) p c = true /\
    certify_ok (core_of (run (pre1 ++ CHECK p c :: mid1 ++ COMMIT p c :: mid2) s0))
      = true /\
    chan (core_of (run (pre1 ++ CHECK p c :: mid1 ++ COMMIT p c :: mid2) s0))
      = Some (claim (core_of (run (pre1 ++ CHECK p c :: mid1) s0)) p c) /\
    claim (core_of (run (pre1 ++ CHECK p c :: mid1) s0)) p c
```

```
    = mkfact p c (ver (core_of (run pre1 s0)) c) /\
  ver (core_of (run pre1 s0)) c = ver (core_of (run (pre1 ++ CHECK p c :: mid1) s0)) c /\
  untouched (run (pre1 ++ [CHECK p c]) s0) mid1 c.
```

In plain words: the clean-start premise on `s0` says the fact table and the channel are empty and the flag is down, and `run tr s0` runs a list of instructions. The theorem: from a clean start, if the flag is up at the end, the list splits as some start, CHECK `p c`, a middle, COMMIT of the same `p c`, another middle, CERTIFY, and the rest; the CHECK, the COMMIT and the CERTIFY each pass where they run; when the CERTIFY runs, the channel holds the claim the COMMIT made, which is the fact (`p`, `c`, the version `c` had at the CHECK); the counter's version at the CHECK equals its version at the COMMIT; and nothing in the first middle touched that counter. CERTIFY takes no operand, and the channel clause is what ties it to the claim. Theorem 16.9 says exactly this. The same file keeps the statement without the channel clause as `earned_certification_provenance`.

E.8 Checker soundness

`checker_soundness`, minimal/EarnedCore.v.

```
Theorem checker_soundness : forall s0 tr f,
  clean_start s0 ->
  let k := core_of (run tr s0) in
  In f (facts k) -> f_ver f = ver k (f_ctr f) -> holds (f_prop f) (val k (f_ctr f)).
```

In plain words: from a clean start, after any list of instructions, every fact in the table that's about its counter's current version is true of that counter's current value, where `holds` is the property's meaning written without the checker. Theorem 16.11 says exactly this in its first sentence; its "in particular" is the next corollary in the file, `committed_claim_holds`.

E.9 No forging

`no_forging`, minimal/EarnedCore.v, with the two definitions it uses.

```
Definition earned (s0 : state) (tr : list instr) (f : fact) : Prop :=
  exists pre mid,
    tr = pre ++ CHECK (f_prop f) (f_ctr f) :: mid /\
    check_ok (core_of (run pre s0)) (f_prop f) (f_ctr f) = true /\
    f = claim (core_of (run pre s0)) (f_prop f) (f_ctr f) /\
    (f_ver f = ver (core_of (run tr s0)) (f_ctr f) ->
     untouched (run (pre ++ [CHECK (f_prop f) (f_ctr f)]) s0) mid (f_ctr f)).

Definition no_forgery (s0 : state) (tr : list instr) : Prop :=
  forall f, In f (facts (core_of (run tr s0))) -> earned s0 tr f.

Theorem no_forging : forall s0 tr, clean_start s0 -> no_forgery s0 tr.
```

In plain words: a fact is `earned` when a passing CHECK of exactly that property and counter wrote it somewhere in the list, it is exactly the claim that CHECK made,

and, if it's still about the counter's current version, nothing touched the counter since. `no_forgery` says every fact in the table at the end is earned. The theorem: from every clean start, every list of instructions leaves no forgery. That's the last clause of Theorem 16.12, with Definition 16.6 saying the same as `earned`; the two one-step clauses before it are `facts_step` (one instruction at a core) and `no_forging_step` (a trace with no forgery stays that way when one instruction is added).

E.10 The minimum price of a certificate

`certified_run_min_cost`, minimal/EarnedCore.v.

```
Theorem certified_run_min_cost : forall s0 tr,
  clean_start s0 -> cert (run tr s0) = true ->
  total_cost tr >= 3 /\ mu (run tr s0) >= mu s0 + 3.
```

In plain words: from a clean start, a list of instructions that ends with the flag up cost at least three in total, and the ledger rose by at least three. Observation 16.16 says exactly this in its first sentence; its form for stored programs is `program_certified_min_cost`.

E.11 Three, reached by a check that could have failed

`earned_run_check_can_fail` and `earned_run_refused_forever`, minimal/EarnedCore.v, with the program they run.

```
Definition earned_run : list instr := [CHECK PZero CA; COMMIT PZero CA; CERTIFY].

Theorem earned_run_check_can_fail : forall a b,
  (a = 0 ->
     cert (run_prog 4 earned_run (start a b)) = true /\
     mu (run_prog 4 earned_run (start a b)) = 3) /\
  (a <> 0 ->
     cert (run_prog 4 earned_run (start a b)) = false /\
     err (core_of (run_prog 4 earned_run (start a b))) = true).

Theorem earned_run_refused_forever : forall n a b,
  a <> 0 -> cert (run_prog n earned_run (start a b)) = false.
```

In plain words: `earned_run` is CHECK "A is 0", COMMIT the same, CERTIFY, and `start a b` puts a in A and b in B with an empty table. The first theorem: run it for four program steps; if a is 0 the flag is up and the ledger is exactly 3, and if a isn't 0 the flag is down and the trap is up. The second: if a isn't 0, the flag is down after any number of steps. So three is reached by a check that could have failed, and when it fails nothing ever certifies. Theorem 16.17 says this. The older witness in the same file, `min_cost_tight`, reaches three with "A is at least 0", a check that can't fail; the step doesn't police that, and Section 16 says so.

E.12 Thiele-complete, the four clauses

`thiele_complete`, minimal/ThieleComplete.v, with the clauses it is made of, and its form over a claim language fixed first, `thiele_complete_over`. `load a b` is the interface's loaded state for the start $(1, a, b)$, written inside a section that fixes `I`; `run M` runs a list of moves from a state.

```
Definition universal_base_clause : Prop :=
  (forall i, ti_kind I (ub_compile (ti_base I) i) = KBase) /\
  (forall a b, ti_clean I (load a b)) /\
  (forall s m, ti_kind I m = KBase -> m_record M (m_step M s m) = m_record M s) /\
  (forall s m, m_record M s = true -> m_record M (m_step M s m) = true).

Definition earned_chain (s0 : m_state M) (tr : list (m_move M)) : Prop :=
  exists pre c chk mid1 cmt mid2 crt post,
    tr = pre ++ chk :: mid1 ++ cmt :: mid2 ++ crt :: post /\
    ti_kind I chk = KCheck c /\ ti_kind I cmt = KCommit c /\
      ti_kind I crt = KCertify /\
    ti_check I (run M pre s0) c = true /\
    (forall t1 t2, mid1 = t1 ++ t2 ->
       ti_same I c (run M pre s0) (run M (pre ++ chk :: t1) s0)) /\
    m_record M (run M (pre ++ chk :: mid1 ++ cmt :: mid2) s0) = false /\
    m_record M (run M (pre ++ chk :: mid1 ++ cmt :: mid2 ++ [crt]) s0) = true.

Definition earned_record_clause : Prop :=
  (forall s, ti_clean I s -> m_record M s = false) /\
  (forall s0 tr, ti_clean I s0 -> m_record M (run M tr s0) = true ->
    earned_chain s0 tr) /\
  (forall s c, ti_check I s c = true -> ti_meaning I c s) /\
  (forall c s s', ti_same I c s s' -> ti_meaning I c s -> ti_meaning I c s').

Definition record_move (m : m_move M) : nat :=
  match ti_kind I m with KBase => 0 | _ => 1 end.

Definition exact_toll_clause : Prop :=
  (forall m, m_cost M m = record_move m) /\
  (forall s m, ti_ledger I (m_step M s m) = ti_ledger I s + m_cost M m).

Definition non_vacuity_clause : Prop :=
  exists c chk cmt crt,
    ti_kind I chk = KCheck c /\ ti_kind I cmt = KCommit c /\
      ti_kind I crt = KCertify /\
    (forall a b, m_record M (run M [chk; cmt; crt] (load a b)) = true <->
                 ti_meaning I c (load a b)) /\
    (exists a b, ti_meaning I c (load a b)) /\
    (exists a b, ~ ti_meaning I c (load a b)).

Definition thiele_complete_with : Prop :=
  universal_base_clause /\ earned_record_clause /\ exact_toll_clause /\
    non_vacuity_clause.

Definition thiele_complete (M : machine) : Prop :=
  exists I : thiele_interface M, thiele_complete_with I.
```

```
Record claim_language (M : machine) : Type := mk_cl {
  cl_claim : Type;
  cl_eqb : cl_claim -> cl_claim -> bool;
  cl_eqb_spec : forall c d, cl_eqb c d = true <-> c = d;
  cl_meaning : cl_claim -> m_state M -> Prop;
  cl_same : cl_claim -> m_state M -> m_state M -> Prop;
  cl_same_keeps : forall c s s', cl_same c s s' ->
    cl_meaning c s -> cl_meaning c s'
}.

Record lang_interface (M : machine) (L : claim_language M) : Type := mk_lang {
  lang_base : universal_base M;
  lang_kind : m_move M -> kind (cl_claim L);
  lang_check : m_state M -> cl_claim L -> bool;
  lang_sound : forall s c, lang_check s c = true -> cl_meaning L c s;
  lang_clean : m_state M -> Prop;
  lang_ledger : m_state M -> nat
}.

Definition lang_ti {M : machine} {L : claim_language M}
  (J : lang_interface M L) : thiele_interface M :=
  mk_ti M (lang_base J) (cl_claim L) (lang_kind J) (cl_meaning L)
    (lang_check J) (cl_same L) (lang_clean J) (lang_ledger J).

Definition thiele_complete_over (M : machine) (L : claim_language M) : Prop :=
  exists J : lang_interface M L, thiele_complete_with (lang_ti J).

Theorem clock_not_thiele_complete : forall rd, ~ thiele_complete (clock rd).
```

In plain words: `M` is a machine (states, moves, step, cost, record) and `I` an interface for it (a universal base, claims with meanings, a checker, "unchanged," clean states, a ledger, and a kind for each move). The first clause says the compiled two-counter moves are base moves, loaded states are clean, base moves leave the record alone, and the record never comes down; the universal base's own fields, printed in Section E.1, say the compiled moves act on the window exactly as the two-counter instructions act. `earned_chain` is a passing check of a claim, then a commit of the same claim with the thing it's about unchanged at every point between, then the certify that raises the record. The earned-record clause says clean states have the record down, every run from a clean state that raises it contains that chain, a passing check means the claim holds, and a claim that holds keeps holding while its thing is unchanged. The exact toll: base moves cost 0, record moves 1, and the ledger adds each move's cost. Non-vacuity: some claim's bare chain raises the record from a loaded state exactly when the claim holds there, and the claim holds at one loaded state and fails at another. `thiele_complete` asks for one interface meeting all four. A `claim_lang uage` fixes the claims, an exact equality test on them, their meanings, "unchanged," and the proof that a meaning keeps holding while its thing is unchanged; a `lan g_interface` over it supplies only the base, the kinds, the checker with its proof of soundness against the language's meanings, the clean states and the ledger; `l`

ang_ti makes it a plain interface whose claims, meanings and "unchanged" are the language's; and thiele_complete_over asks that interface to meet all four. clock is the two-counter machine with every move costing 1 and any reading of its configuration as the record. Definition 16.7 says this, the language-first form included, and adds the weaker notion, weakly Thiele-complete (weakly_thiele_complete, not printed here). Observation 16.21 says the last line and more: the clock is weakly Thiele-complete (clock_weakly_thiele_complete), a flag that switches on at the first yes behaves the same way (latch_clock_weakly_thiele_complete and latch_clock_not_thiele_complete), and no machine whose every move costs at least 1 is Thiele-complete, which comes from complete_has_free_move. Clause (c) fixes the unit at one. Theorem 16.24 states the definition with every record move at one fixed price $c \geq 1$ (thiele_complete_at, ThieleCompleteScaled.v), shows that class is this one with its prices multiplied by c (scale_complete_at and unscale_complete), and shows the clock fails at every unit (clock_not_thiele_complete_at_any_unit).

E.13 The universal program's flag and ledger

universal_halting, universal_flag_iff, universal_ledger_exact and universal_thiele_complete, coq/kernel/foundation/UniversalRun.v.

```
Theorem universal_halting : forall P x y,
  (exists m, E.halted P (E.core_of (grun P x y m))) <->
  (exists n, M.halted U (M.core_of (hrun P x y n))).

Theorem universal_flag_iff : forall P x y,
  (exists m, E.cert (grun P x y m) = true) <->
    (exists n, M.cert (hrun P x y n) = true).

Theorem universal_ledger_exact : forall P x y,
  (forall m, exists n,
     ((rel P (grun P x y m) (hrun P x y n) /\ m <= n) \/
      rel_halt P (grun P x y m) (hrun P x y n)) /\
     M.mu (hrun P x y n) = E.mu (grun P x y m)) /\
  (forall n, exists m,
     E.mu (grun P x y m) <= M.mu (hrun P x y n) <= E.mu (grun P x y (S m))).

Theorem universal_thiele_complete : TC.thiele_complete host_machine.
```

In plain words: grun P x y m is m program steps of the small machine's program P from start x y; hrun P x y n is n program steps of the one fixed host program U from the host loaded with P, x and y. The first theorem: the guest halts at some step exactly when U does. The second: the guest's flag is up at some step exactly when the host's is. The third: for every guest step count there's a host step count where the two are related, with the host count at least the guest count, or both stopped, with equal ledgers; and every host point's ledger lies between the guest's at some step and at the

step after. `rel` and `rel_halt` are the relations Theorem 16.28 spells out. The fourth: the host machine, the small machine with a register for every whole number and the one property `PSlot`, read one move per instruction, is Thiele-complete in the sense of the previous subsection. Theorem 16.30 says exactly the second and the third. Theorem 16.29 says the first and more: at halting the registers, the traps, the ledgers and the flags match too (`universal_output`, not printed here). Theorem 16.32 says the fourth and more: U's run is a run of the host, the floor on the cost of a run that raises the flag, and the undecidability of U's halting, each with its own Coq theorem in the table of Section D.

E.14 The universal reading checker

`cg_run_check`, `cg_ueval` and `cg_ueval_iff`,
coq/kernel/foundation/CompilerChecker.v.

```
Definition cg_run_check (rc : cg_routine_code) (v : nat) : bool :=
  match rc with
  | (ig, R, xS, xB, xT, m) =>
      let st := cg_mme_run_fuel (cg_expo (qs xT) v) (ig, R) (ig, cg_env_chk m v) in
      cg_out_codeb (ig, R) (fst st) && Nat.eqb (get_env (snd st) xB) 1
  end.

Definition cg_ueval (p : cg_uprop) (v : nat) : bool :=
  match p with
  | UBase q => G.ceval q v
  | URun r =>
      match cg_rdec r with
      | Some rc => cg_run_check rc v
      | None => false
      end
  end.

Theorem cg_ueval_iff : forall p v, cg_ueval p v = true <-> cg_uholds p v.
```

In plain words: a claim is either one of the counter language's words, checked as before, or a number r naming a reading routine. For the second kind the checker decodes r; a code that names no routine gets false. Otherwise it reads the registers and a fuel number t out of the value, runs the routine for at most t steps, and says yes exactly when the routine has left its code with 1 in its answer register. The theorem says the checker's yes is the claim's meaning, which for a routine is that bounded run. Section 16.14 says exactly this. The checker answers on every input because it's a Coq function that always returns (`cg_ueval_total`).

E.15 The surcharge can't always be avoided

`presented_universal_no_exact_below_three`,
coq/kernel/foundation/PresentedUniversal.v.

```
Theorem presented_universal_no_exact_below_three :
  forall (Mp : presented_machine) (s0 : T.cs_state (pm_sys Mp)) (n : nat),
  mlatch Mp s0 n = true -> mledger Mp s0 n < 3 ->
  forall (h0 : hstate) (tr : list (@M.pu_instr pu_hprop)),
  M.pu_clean_start h0 ->
  ~ (M.cert (hrun_l tr h0) = mlatch Mp s0 n /\
     M.mu (hrun_l tr h0) = M.mu h0 + mledger Mp s0 n).
```

In plain words: if a presented machine's latch is up after n moves while its ledger is below three, then no list of host instructions, run from a clean host start, ends with the host's flag equal to that latch and the host's ledger raised by exactly that ledger. Theorem 16.34(3) says exactly this. It's about the guest's whole ledger at that point. A guest whose one raising move is cheap but whose earlier moves already paid three or more isn't covered.

E.16 Nesting pays for every rise, on every host run that follows the guest

`cmpz_hom`, coq/kernel/foundation/CzCat.v, and `cmpz_follows`, `cmpz_follow_cost_ge_exits` and `cmpz_end_only_not_enough`, coq/kernel/foundation/CzFollow.v; `cmpz_follows` and `cmpz_follow_cost_ge_exits` are written inside a section that fixes `X`, `Y` and `f : cmpz_hom X Y`.

```
Record cmpz_hom (X Y : cmpz_obj) : Type := mk_hom {
  h_state : am_state (ob_M X) -> am_state (ob_M Y);
  h_rec : ob_A X -> ob_A Y;
  h_mono : forall a b, bp_le (ob_P X) a b -> bp_le (ob_P Y) (h_rec a) (h_rec b);
  h_rec_nat : forall s, am_rec (ob_M Y) (h_state s) = h_rec (am_rec (ob_M X) s);
  h_sim : forall s m, exists tr : list (am_move (ob_M Y)),
            h_state (am_step (ob_M X) s m) = am_run (ob_M Y) tr (h_state s)
}.

Fixpoint cmpz_follows (tr : list (am_move (ob_M X))) (s : am_state (ob_M X))
    (segs : list (list (am_move (ob_M Y)))) : Prop :=
  match tr, segs with
  | [], [] => True
  | m :: tr', seg :: segs' =>
      h_state f (am_step (ob_M X) s m) = am_run (ob_M Y) seg (h_state f s) /\
      cmpz_follows tr' (am_step (ob_M X) s m) segs'
  | _, _ => False
  end.

Theorem cmpz_follow_cost_ge_exits :
  (forall a b, bp_le (ob_P Y) (h_rec f a) (h_rec f b) -> bp_le (ob_P X) a b) ->
  ax_a2 (X := am_axsys (ob_M Y)) ->
  forall tr s segs, cmpz_follows tr s segs ->
    cmpz_cost (ob_M Y) (concat segs) >= cmpz_exit_count (ob_M X) tr s.

Theorem cmpz_end_only_not_enough :
  exists (X Y : cmpz_obj) (f : cmpz_hom X Y),
    (forall a b, bp_le (ob_P Y) (h_rec f a) (h_rec f b) -> bp_le (ob_P X) a b) /\
    ax_a2 (X := am_axsys (ob_M Y)) /\
    exists (tr : list (am_move (ob_M X))) s (tr' : list (am_move (ob_M Y))),
```

```
h_state f (am_run (ob_M X) tr s) = am_run (ob_M Y) tr' (h_state f s) /\
cmpz_cost (ob_M Y) tr' < cmpz_exit_count (ob_M X) tr s.
```

In plain words: a morphism f from a guest X to a host Y is a map of states and a monotone map of records such that the record of an image state is the image of the record, and every move of the guest is simulated by some list of the host's moves landing on the image of the state after the move. `cmpz_followstrssegs` says the host's run is split into one stretch per guest move, each stretch carrying the image of the state before that move to the image of the state after it. The first theorem: if the record map reflects the order and Y pays its toll, then every host run that follows a guest run in that sense costs at least the number of guest steps whose new record isn't at or below the old one. The second: for host runs that only reach the image of the guest's end state, no such bound holds in general. On a one-bit machine whose move b sets the bit to b, the guest run "up, then down" has one such step, and the empty host run reaches the image of its end for nothing. Theorem 18.4(1) says exactly these, with "faithfully" meaning that f is a morphism, and with `cmpz_follow_exis ts` and `cmpz_follow_reaches` (CzFollow.v) for the existence of a following run and where it ends. The earlier form, `cmpz_sim_cost_ge_exits` (CzCat.v), finds one host run reaching the end image at that cost. It bounds the host's cost by the guest's rises. It says nothing about the guest's other charges.

E.17 The copy is the record, on the simpler host

`record_agreement`, minimal/UniversalThiele.v.

```
Definition agrees (h : hstate) : Prop := mrec h = E.cert (gst h).

Theorem record_agreement : forall tr h, agrees h -> agrees (hrun tr h).
```

In plain words: the host keeps a mirror bit `mrec` beside the guest's state `gst`, and `agrees h` says the mirror equals the guest's own flag. The theorem: a host state that agrees still agrees after any list of host moves, whatever they are. Observation 16.36 says this for lists of moves and adds single moves, stored programs, loading, and the mirror after n moves GSTEP 1, which are `record_agreement_step`, `record_agr eement_program`, `hload_agrees` and `simulated_record`.

E.18 No free host certification, on the simpler host

`no_free_host_certification`, minimal/UniversalThiele.v, with the host's moves and their costs.

```
Inductive hinstr : Type :=
| OWN (i : E.instr)
| GSTEP (b : nat).

Definition hcost (i : hinstr) : nat :=
```

```
  match i with OWN j => E.cost j | GSTEP b => b end.

Definition crossing (g g' : E.state) : bool := negb (E.cert g) && E.cert g'.

Theorem no_free_host_certification : forall tr h,
  mrec h = false -> mrec (hrun tr h) = true ->
  exists pre b post, tr = pre ++ GSTEP b :: post /\ b >= 1 /\
    crossing (gst (hrun pre h)) (gst (hexec (hrun pre h) (GSTEP b))) = true.
```

In plain words: a host move is either `OWN i`, one of the small machine's own instructions on the host's core, or `GSTEP b`, one guest step with budget `b`, which costs `b` whatever the guest instruction cost. `crossing g g'` says the guest's flag went from down to up. The theorem: if the mirror is down before a list of host moves and up after it, then somewhere in the list there's a `GSTEP b` with `b` at least one at which the guest's flag crossed. Theorem 16.38 prints the one-move form, `no_free_host_certification_step`, which also says the host wasn't trapped and the guest's next instruction was CERTIFY; this is the form for whole lists that it lifts to. The printed theorem says two things more, each with its own Coq theorem: a trace of OWN moves alone leaves the mirror and the guest state where they were (`guestless_runs_leave_mirror`), and the host's own flag rises only by an OWN CERTIFY that passes (`own_record_only_by_certify`). Neither the one-move form nor the list form says what the guest did before its CERTIFY. That, after a run from a loaded host, a raised mirror means the guest's own run checked and committed first is the next theorem.

E.19 A raised mirror stands on the guest's own earned chain, on the simpler host

`host_mirror_earned`, minimal/UniversalThiele.v.

```
Theorem host_mirror_earned : forall tr a b P x y,
  mrec (hrun tr (hload a b P x y)) = true ->
  exists n gtr pre1 p c mid1 mid2 post,
    gtr = E.trace_of n P (E.start x y) /\
    gst (hrun tr (hload a b P x y)) = E.run gtr (E.start x y) /\
    gtr = pre1 ++ E.CHECK p c :: mid1 ++ E.COMMIT p c :: mid2 ++ E.CERTIFY :: post /\
    E.check_ok (E.core_of (E.run pre1 (E.start x y))) p c = true /\
    E.commit_ok (E.core_of (E.run (pre1 ++ E.CHECK p c :: mid1) (E.start x y))) p c
      = true /\
    E.certify_ok (E.core_of (E.run (pre1 ++ E.CHECK p c :: mid1 ++ E.COMMIT p c :: mid2)
                             (E.start x y))) = true /\
    E.ver (E.core_of (E.run pre1 (E.start x y))) c
      = E.ver (E.core_of (E.run (pre1 ++ E.CHECK p c :: mid1) (E.start x y))) c /\
    E.untouched (E.run (pre1 ++ [E.CHECK p c]) (E.start x y)) mid1 c.
```

In plain words: load the host with a guest program P and a guest start `(x, y)` (`hload` gives the guest a clean start), run any list of host moves, and suppose the mirror is up at the end. Then for some number of steps `n`, the guest's state is its own program's run of `n` steps, and the instructions that run executed split exactly the way Theorem 16.9 splits a trace: a passing CHECK, a COMMIT of the same claim at the same version with the counter untouched between, then a passing CERTIFY. So

the host's mirror can't be up unless the guest's own program earned it. The book's sentences citing `host_mirror_earned` say this in words; the Coq says a little more than they need, since the chain sits inside the instructions the guest program itself executed. Any list containing the chain would have been enough for them.

E.20 Structural entitlement, represented

`ent_representation`, minimal/EntitlementSmall.v, with the definitions it leans on.

```
Definition ent_strict_sublist (post prior : list X) : Prop :=
  incl post prior /\ exists x, In x prior /\ ~ In x post.

Definition ent_distinguishes (obs : X -> O) (w : X) (post : list X) : Prop :=
  forall t, In t post -> obs w <> obs t.

Definition ent_member (eqb : O -> O -> bool) (obs : X -> O) (L : list X) (o : O) : bool :=
  existsb (fun t => eqb (obs t) o) L.

Definition ent_stronger (P Q : O -> bool) : Prop :=
  forall o, P o = true -> Q o = true.

Definition ent_strictly_stronger (P Q : O -> bool) : Prop :=
  ent_stronger P Q /\ exists o, P o = false /\ Q o = true.

Definition ent_reduction (r : X -> O) (T : ent_tree) (prior post : list X) : Prop :=
  exists F : X -> list X,
    (forall x, In x prior -> exists t, In t post /\ In x (F t) /\ r x = r t) /\
    length prior <= fold_right Nat.add O (map (fun t => length (F t)) post) /\
    (forall t, In t post -> length (F t) <= ent_leaves T).

Definition ent_certified {M : machine} (I : thiele_interface M) {O : Type}
    (s0 : m_state M) (tr : list (m_move M))
    (d : list (m_move M) -> O) (P : O -> bool) : Prop :=
  m_record M (run M tr s0) = true /\ P (d tr) = true.

Theorem ent_representation :
  forall (M : machine) (I : thiele_interface M) (X O : Type)
         (s0 : m_state M) (tr : list (m_move M))
         (d : list (m_move M) -> O) (obs r : X -> O) (eqb : O -> O -> bool)
         (prior post : list X) (T : ent_tree),
    thiele_complete_with I ->
    (forall o1 o2, eqb o1 o2 = true <-> o1 = o2) ->
    ent_strict_sublist post prior ->
    (exists w, In w prior /\ ~ In w post /\ ent_distinguishes obs w post) ->
    ti_clean I s0 ->
    ent_certified I s0 tr d (ent_member eqb obs post) ->
    ent_depth T <= record_moves I tr ->
    0 < length post ->
    ent_reduction r T prior post ->
    ent_strictly_stronger (ent_member eqb obs post) (ent_member eqb obs prior) /\
    earned_chain I s0 tr /\
    (exists pre c chk mid1 cmt rest,
       tr = pre ++ chk :: mid1 ++ cmt :: rest /\
       ti_kind I chk = KCheck c /\ ti_kind I cmt = KCommit c /\
       ti_meaning I c (run M pre s0) /\ ti_meaning I c (run M (pre ++ chk :: mid1) s0)) /\
    Nat.log2_up (length prior) - Nat.log2_up (length post) <= record_moves I tr /\
    record_moves I tr = ti_ledger I (run M tr s0) - ti_ledger I s0 /\
    ti_ledger I s0 + 3 <= ti_ledger I (run M tr s0).
```

In plain words: take a machine and an interface through which it is Thiele-complete, any type of rivals X and of observations O with an exact equality test, a run t from a clean state s_0, and a decoder d that reads the run's receipts, its list of moves. Hypotheses in order: the posterior list is inside the prior and drops at least one rival; some dropped rival shows an observation no survivor shows; the start is clean; the run ends with the record up and the posterior's membership test accepts the decoded run; the tree is no deeper than the number of record moves in the run; the posterior is non-empty; and the covering is by fibres, as `ent_redu`

ction spells out. Conclusions in order: the posterior's test is strictly stronger; the run contains the earned chain; some CHECK of a claim and a later COMMIT of the same claim appear in the run, with the claim holding at both; the index bits are at most the record moves; the record moves are exactly the ledger's rise; and the rise is at least 3. Theorem 16.43 says this, with the hypotheses numbered (M1) to (M6) and the conclusions numbered (1) to (5), and it says one thing more in (2): there the meanings hold at the chain's own CHECK and COMMIT. The Coq's third conclusion is a separate existential that doesn't name the chain; the printed form follows from it with the last two parts of clause (b), a passing check means the claim holds and an unchanged thing keeps it holding.

And its form with the narrowing made by the run, `ent_run_entitlement`, same file, with the definitions it leans on.

```
Fixpoint ent_narrow (fs : list (X -> bool)) (L : list X) : list X :=
  match fs with [] => L | f :: fs' => ent_narrow fs' (filter f L) end.

Fixpoint ent_stages (fs : list (X -> bool)) (L : list X) : list (nat * nat) :=
  match fs with
  | [] => []
  | f :: fs' => (length L, length (filter f L)) :: ent_stages fs' (filter f L)
  end.

Definition ent_prod (l : list nat) : nat := fold_right Nat.mul 1 l.

Fixpoint ent_passes {M : machine} (I : thiele_interface M) (s : m_state M)
    (done tr : list (m_move M)) : list (list (m_move M) * ti_claim I) :=
  match tr with
  | [] => []
  | m :: rest =>
      let tail := ent_passes I (m_step M s m) (done ++ [m]) rest in
      match ti_kind I m with
      | KCheck c => if ti_check I s c then (done, c) :: tail else tail
      | _ => tail
      end
  end.

Definition ent_run_tests {M : machine} (I : thiele_interface M) (s0 : m_state M)
    (tr : list (m_move M)) : list (m_state M -> bool) :=
  map (fun pc x => ti_check I (run M (fst pc) x) (snd pc)) (ent_passes I s0 [] tr).

Definition ent_run_posterior {M : machine} (I : thiele_interface M) (s0 : m_state M)
    (tr : list (m_move M)) (prior : list (m_state M)) : list (m_state M) :=
  ent_narrow (ent_run_tests I s0 tr) prior.

Theorem ent_run_entitlement :
  forall (M : machine) (I : thiele_interface M),
    thiele_complete_with I ->
    forall (s0 : m_state M) (tr : list (m_move M)) (prior : list (m_state M)) (k : nat),
    ti_clean I s0 ->
    m_record M (run M tr s0) = true ->
    In s0 prior ->
    Forall (fun st => fst st <= 2 ^ k * snd st) (ent_stages (ent_run_tests I s0 tr) prior) ->
    incl (ent_run_posterior I s0 tr prior) prior /\
    In s0 (ent_run_posterior I s0 tr prior) /\
    (forall x, In x prior -> ~ In x (ent_run_posterior I s0 tr prior) ->
       exists pre c m rest, tr = pre ++ m :: rest /\ ti_kind I m = KCheck c /\
         ti_check I (run M pre s0) c = true /\ ti_check I (run M pre x) c = false) /\
    length prior * ent_prod (map snd (ent_stages (ent_run_tests I s0 tr) prior))
      = length (ent_run_posterior I s0 tr prior)
          * ent_prod (map fst (ent_stages (ent_run_tests I s0 tr) prior)) /\
    Nat.log2_up (length prior) - Nat.log2_up (length (ent_run_posterior I s0 tr prior))
      <= k * length (ent_passes I s0 [] tr) /\
    length (ent_passes I s0 [] tr) <= record_moves I tr /\
    record_moves I tr = ti_ledger I (run M tr s0) - ti_ledger I s0 /\
    Nat.log2_up (length prior) - Nat.log2_up (length (ent_run_posterior I s0 tr prior))
      <= k * (ti_ledger I (run M tr s0) - ti_ledger I s0) /\
    earned_chain I s0 tr /\
    ti_ledger I s0 + 3 <= ti_ledger I (run M tr s0).
```

In plain words: ent_passes walks the run and keeps, for each CHECK that passes at the state the run is in, the moves before it and its claim; ent_run_tests turns each into a test of a candidate start x, replay those moves from x and check the same claim; the posterior keeps the candidates that pass every test, in order; and ent_stages records the number of candidates standing before and after each test. Hypotheses: the machine is Thiele-complete through I, the start is clean, the run ends with the record up, the start is among the candidates, and every test keeps at least a 2^{-k} share. Conclusions in order: the posterior is inside the prior and keeps the start; every dropped candidate fails, replayed, a CHECK the run made and passed; the exact product count; the index bits are at most k times the tests passed; the tests passed are at most the record moves; the record moves are the ledger's rise; the index bits are at most k times that rise; the earned chain; and a rise of at least 3. Theorem 16.45 (1) to (5) says exactly this; its part (6) is ent_run_round_bits, not printed here.

E.21 A raised flag stands on the CHSH check

small_chsh_flag_implies_tsirelson, coq/kernel/quantum/SmallChshMachine.v, over the host machine with the one property small_chsh_PCHSH.

```
Theorem small_chsh_flag_implies_tsirelson :
  forall (s0 : @state small_chsh_prop) (tr : list (@instr small_chsh_prop)),
  clean_start s0 ->
  cert (run small_chsh_prop_eqb small_chsh_eval tr s0) = true ->
  exists pre1 c mid1 mid2 post,
    tr = pre1 ++ CHECK small_chsh_PCHSH c :: mid1
           ++ COMMIT small_chsh_PCHSH c :: mid2 ++ CERTIFY :: post /\
    let v := vals (core_of (run small_chsh_prop_eqb small_chsh_eval pre1 s0)) c in
    let t := small_chsh_tally_of v in
    small_chsh_check t = true /\
    small_chsh_meaning t /\
    (small_chsh_score t * small_chsh_score t <= 8)%R /\
    (Rabs (small_chsh_score t) <= 2 * sqrt 2)%R /\
    vals (core_of (run small_chsh_prop_eqb small_chsh_eval
                   (pre1 ++ CHECK small_chsh_PCHSH c :: mid1) s0)) c = v.
```

In plain words: start from a clean start and run any list of instructions of the host machine whose only property is the CHSH one, and suppose the flag is up at the end. Then the list splits as some instructions, a CHECK of the property at some counter c, some more, a COMMIT of the same property at the same counter, some more, a CERTIFY, and the rest. Take the value v that counter c held just before the CHECK, and read it as a tally of eight counts. That tally passes the seven integer facts (small_chsh_check); it means what the check is for: every question pair sampled and the pinned five-by-five symmetric and positive semidefinite (small_chsh_meaning); its score S has $S^2 \leq 8$ and $|S| \leq 2\sqrt{2}$; and counter c still held v when the COMMIT ran. Because that tally passes the check, its score is in fact strictly below the bound,

$S^2 < 8$ (`nec_e_check_strict`), and passing tallies come as close to 8 as you like (`nec_e_check_sharp`), both in NecEChsh.v. Theorem 20.5 says exactly this in its first part. It says more in two places that have their own Coq theorems, the stored-program form and the converse that the three instructions certify exactly when the check passes, and the real numbers in the statement are why it carries FE and RD, three standard-library assumptions in all, as printed in Section D.

E.22 The recursion theorem up to the record, but not on every register

`sm2_kleene_obs` and `sm2_no_exact`, coq/kernel/foundation/SmFixedPoint.v and coq/kernel/foundation/SmNoExact.v, with the relation they use. Here `M` is the host machine's module, `hendsPxs` says program `P`, started with x in register 1 and everything else clean, stops (a trap counts as stopping) in state `s`, and `sm_computes_mapTF` says that on the number of every program `p`, `T` stops with the number of `Fp`.

```
Definition sm2_obs (s t : hstate) : Prop :=
  M.vals (M.core_of s) 0 = M.vals (M.core_of t) 0 /\
  M.err (M.core_of s) = M.err (M.core_of t) /\
  M.mu s = M.mu t /\ M.cert s = M.cert t /\
  length (M.facts (M.core_of s)) = length (M.facts (M.core_of t)) /\
  (M.chan (M.core_of s) = None <-> M.chan (M.core_of t) = None).

Definition sm2_obs_equiv (P Q : list hinstr) : Prop :=
  forall x,
    (forall s, hends P x s -> exists t, hends Q x t /\ sm2_obs s t) /\
    (forall t, hends Q x t -> exists s, hends P x s /\ sm2_obs s t).

Theorem sm2_kleene_obs : forall (F : list hinstr -> list hinstr) (T : list hinstr),
  sm_computes_map T F -> exists e, sm2_obs_equiv e (F e).

Theorem sm2_no_exact :
  exists (F : list hinstr -> list hinstr) (T : list hinstr),
    sm_computes_map T F /\ forall e, ~ hequiv e (F e).
```

In plain words: `sm2_obsst` says two final states agree on register 0, the trap latch, the ledger, the flag, the number of facts and whether the channel is empty, and nothing else. `sm2_obs_equivPQ` says that on every input, any state `P` ends in has a state `Q` ends in that agrees in that sense, and the same the other way round, which includes that `P` stops on an input exactly when `Q` does. The first theorem: for every transformation F of host programs that some host program `T` carries out on numbers, there is a program `e` that agrees with `Fe` in that sense. `T` is any host program, and may itself check, commit and certify. The second: there is a transformation F, carried out by a host program, such that no program `e` behaves the same as `Fe` in the sense of Definition 16.12, which compares every register, the fact table, the channel, the trap latch, the ledger and the flag. Theorems 16.55 and 16.56 say these two. The book's printed version says exactly this, with the line number and versions left out of both comparisons, as the definition does.

F

The CHSH derivation, in full

This appendix supports Section 20. That section tells the story; this one does the algebra. Each item gives the exact statement with all its hypotheses, the steps a reader needs to check it, and the Coq name with its file; the file paths in this appendix are under coq/. Nothing here identifies any matrix with a quantum state. Where the reading "quantum" appears, it is a cited result from outside the kernel.

F.1 Setting and notation

Definition F.1 (Settings, outcomes, correlators)**.** Alice receives a setting $x \in \{0,1\}$ and Bob a setting $y \in \{0,1\}$. Each answers a bit, recoded as an outcome $a, b \in \{+1,-1\}$ (bit 0 becomes $+1$, bit 1 becomes -1). The correlator E_{xy} is the average of the product ab over the rounds with settings (x,y). The CHSH quantity is

$$S = E_{00} + E_{01} + E_{10} - E_{11}.$$

I collect the four correlators into the 2×2 matrix

$$E = \begin{pmatrix} E_{00} & E_{01} \\ E_{10} & E_{11} \end{pmatrix},$$

whose rows are indexed by Alice's setting and whose columns by Bob's. The rows of E are (E_{00}, E_{01}) and (E_{10}, E_{11}); the columns are (E_{00}, E_{10}) and (E_{01}, E_{11}).

The same polynomial S appears under three names in the source: `CHSH` (kernel/quantum/TsirelsonGeneral.v), `CHSH_value` (kernel/quantum/TsirelsonFromAlgebra.v) and `chsh_S` (kernel/quantum/ElliptopeCompletion.v).

Definition F.2 (Counters)**.** Eight counts sit in the record `WitnessCounts` (kernel/quantum/CHSHColumnCheck.v): for each setting pair, same_{xy} counts rounds whose two answer bits matched and diff_{xy} counts rounds whose bits differed. A round adds one to exactly one of the eight. Write

$$N_{xy} = \text{same}_{xy} + \text{diff}_{xy}, \qquad D_{xy} = \text{same}_{xy} - \text{diff}_{xy}, \qquad E_{xy} = \frac{D_{xy}}{N_{xy}}.$$

A matched round contributes $ab = +1$ and a differing round $ab = -1$, so D_{xy} is the sum of the products and D_{xy}/N_{xy} is their average. Since $|D_{xy}| \le N_{xy}$, every correlator from counters lies in $[-1, 1]$. In Coq the integers are `chsh_n_z` and `chsh_d_z` (kernel/quantum/CHSHColumnCheck.v). When a bucket is empty, the real-valued correlator `state_bucket_correlation` (kernel/quantum/CHSHColumnCheck.v) and the rational one `chsh_correlator_q` (kernel/quantum/CHSHStatisticalBridge.v) both return 0 by convention. The CHSH score of a tally, `chsh_stat_from_wc` (CHSHStatisticalBridge.v), adds the first three correlators and obtains $-E_{11}$ by swapping the two counters of the last bucket.

F.2 The classical bound

Lemma F.1 (Deterministic strategies)**.** Let $a_0, a_1, b_0, b_1 \in \{+1, -1\}$. Then $-2 \le a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1 \le 2$.

Proof. Factor: $a_0b_0 + a_0b_1 + a_1b_0 - a_1b_1 = a_0(b_0 + b_1) + a_1(b_0 - b_1)$. If $b_0 = b_1$ then $b_0 + b_1 = \pm 2$ and $b_0 - b_1 = 0$; if $b_0 \neq b_1$ the other way round. In both cases exactly one of the two brackets is ± 2 and the other is 0, so the expression is ± 2 times a sign, which is $+2$ or -2. There are sixteen sign choices, and every one of them gives exactly ± 2. None gives 0 or ± 4. □

The Coq proofs do not factor; they split into the sixteen sign cases and compute. The rational statement is `local_S_2_deterministic` (kernel/quantum/BoxCHSH.v): for $f_A, f_B : \mathbb{N} \to \mathbb{Q}$ with every value ± 1, the absolute value of $f_A(0)f_B(0) + f_A(0)f_B(1) + f_A(1)f_B(0) - f_A(1)f_B(1)$ is at most 2. The real statement is `deterministic_strategy_chsh_bounded` (kernel/quantum/MinorConstraints.v), with conclusion $-2 \le \cdots \le 2$.

Definition F.3 (Finite factorizable model, `is_factorizable`)**.** A real-valued correlation function $E(a, b, x, y)$ is factorizable (kernel/quantum/MinorConstraints.v) when there exist response functions $A(x, \lambda)$, $B(y, \lambda)$, weights $p(\lambda)$ and a natural number $\lambda_{\max}$ such that: $0 \le p(\lambda) \le 1$ for $\lambda \le \lambda_{\max}$; $\sum_{\lambda=0}^{\lambda_{\max}} p(\lambda) = 1$; every $A(x, \lambda)$ and every $B(y, \lambda)$ is -1 or $+1$; and for all a, b, x, y, $E(a, b, x, y) = \sum_{\lambda=0}^{\lambda_{\max}} p(\lambda)A(x, \lambda)B(y, \lambda)$. The outcome arguments a, b are dummies; only correlations are modelled.

Theorem F.2 (`fine_theorem`, one direction only)**.** If E is factorizable, then $-2 \le E(0,0,0,0) + E(0,0,0,1) + E(0,0,1,0) - E(0,0,1,1) \le 2$ (kernel/quantum/MinorConstraints.v; restated as `factorizable_CHSH_classical_bound`).

Proof. Substitute the factorization: $S = \sum_\lambda p(\lambda) S_\lambda$, where S_λ is the deterministic expression of Lemma F.1 at the signs $A(\cdot, \lambda), B(\cdot, \lambda)$. Each $S_\lambda \in [-2, 2]$, the weights are nonnegative and sum to 1, so $-2 = \sum_\lambda p(\lambda)(-2) \le S \le \sum_\lambda p(\lambda) \cdot 2 = 2$. □

Remark F.1 (What the name does and does not cover)**.** Fine's 1982 theorem is an equivalence between several descriptions of local models and the Bell inequalities. `fine_theorem` proves one direction: a finite factorizable model obeys the bound. Its converse, as it's stated, is false (`nec_e_fine_converse_false`); the equivalence in the form Fine meant, for correlations, with each correlator between -1 and 1 and all eight inequalities, is `nec_e_fine_iff`. And `fine_theorem` covers only finitely many hidden states ($\lambda \le \lambda_{\max}$). An arbitrary probability space is outside it. The transfer to rational boxes is `local_box_CHSH_bound` (kernel/quantum/MinorConstraints.v): if `box_correlations B` is factorizable then the real image of $|S(B)|$ is at most 2.

The finite-tally statement concerns one fixed strategy replayed on every recorded round.

Definition F.4 (`WCLocallyConsistent`)**.** For bits $a_0, a_1, b_0, b_1 \in \{0, 1\}$ and a counter record w (kernel/quantum/CHSHStatisticalBridge.v), w is locally consistent with (a_0, a_1, b_0, b_1) when, for each setting pair (x, y): if $a_x = b_y$ then $\mathrm{diff}_{xy} = 0$, otherwise $\mathrm{same}_{xy} = 0$; and every bucket is sampled, $N_{00}, N_{01}, N_{10}, N_{11} > 0$.

Theorem F.3 (A tally above 2 is not one replayed strategy)**.** Let w be a counter record whose rational score `chsh_stat_from_wc` w is greater than 2. Then for all bits a_0, a_1, b_0, b_1, w is not locally consistent with (a_0, a_1, b_0, b_1). This is `chsh_violation_rules_out_locally_factorizable_coupling` (kernel/quantum/CHSHCouplingBridge.v). A stronger companion, `locally_consistent_classical_bound`, states that a locally consistent record has score of absolute value at most 2; read through its contrapositive, it gives the first statement.

Proof. Under local consistency each sampled bucket is pure: if $a_x = b_y$ every round in the bucket matched, so $E_{xy} = N_{xy}/N_{xy} = +1$; otherwise every round differed and $E_{xy} = -1$. In signs, $E_{xy} = \alpha_x \beta_y$ with $\alpha_x = (-1)^{a_x}$ and $\beta_y = (-1)^{b_y}$. The score is then the deterministic expression of Lemma F.1, which is at most 2 in absolute value. A score above 2 contradicts this. □

The theorem rules out one explanation, a single four-bit plan used on every round. It says nothing about a local model that redraws its hidden state every round; Section 20 gives a four-round coin-flip tally with score 4.

F.3 Tsirelson's bound from algebra

Lemma F.4 (The six-square identity, `chsh_gap_is_sum_of_squares`)**.** For all real a, b, c, d,

$$4(a^2+b^2+c^2+d^2) - (a+b+c-d)^2 \\ = (a-b)^2 + (a-c)^2 + (a+d)^2 + (b-c)^2 + (b+d)^2 + (c+d)^2.$$

(kernel/quantum/TsirelsonFromAlgebra.v, closed by `ring`.)

Proof. Expand the square on the left: $(a+b+c-d)^2 = a^2+b^2+c^2+d^2+2ab+2ac-2ad+2bc-2bd-2cd$. So the left side is

$$3(a^2+b^2+c^2+d^2) - 2ab - 2ac + 2ad - 2bc + 2bd + 2cd.$$

On the right, each variable appears in exactly three of the six brackets (a in $a-b$, $a-c$, $a+d$; b in $a-b$, $b-c$, $b+d$; c in $a-c$, $b-c$, $c+d$; d in $a+d$, $b+d$, $c+d$), so the squared terms add to $3(a^2+b^2+c^2+d^2)$. The cross terms are $-2ab$ from $(a-b)^2$, $-2ac$ from $(a-c)^2$, $+2ad$ from $(a+d)^2$, $-2bc$ from $(b-c)^2$, $+2bd$ from $(b+d)^2$ and $+2cd$ from $(c+d)^2$. Both sides agree term by term. □

Since the right side is a sum of squares, $S^2 \le 4\sum_{xy} E_{xy}^2$ for any four reals; this is `cauchy_schwarz_chsh` (TsirelsonFromAlgebra.v).

Theorem F.5 (Row-bound form of Tsirelson's bound)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$ with $E_{00}^2 + E_{01}^2 \le 1$ and $E_{10}^2 + E_{11}^2 \le 1$, $S^2 \le 8$. This is `tsirelson_squared` (kernel/quantum/TsirelsonFromAlgebra.v) and `tsirelson_from_row_bounds` (kernel/quantum/TsirelsonGeneral.v). The absolute form $|S| \le \sqrt{8} = 2\sqrt{2}$ is `tsirelson_bound_abs` (TsirelsonGeneral.v).

Proof. Adding the two row bounds gives $\sum_{xy} E_{xy}^2 \le 2$. By Lemma F.4, $S^2 \le 4\sum_{xy} E_{xy}^2 \le 8$. □

Theorem F.6 (Tightness)**.** The point $E_{00} = E_{01} = E_{10} = 1/\sqrt{2}$, $E_{11} = -1/\sqrt{2}$ satisfies both row bounds and has $S = \sqrt{8}$. This is `tsirelson_tight` (TsirelsonFromAlgebra.v) and `tsirelson_achievable` (TsirelsonGeneral.v).

Proof. Each row has squared length $1/2 + 1/2 = 1$, and $S = 4/\sqrt{2} = 2\sqrt{2} = \sqrt{8}$. □

Remark F.2 (Where equality can happen)**.** The step about where equality happens is checked as `nec_e_equality_forces_point` and `nec_e_equality_two_points`, and

the fact that no ratio of whole numbers solves $2a^2 = 1$ as `nec_e_no_rational_half_square` (NecEChshEquality.v); the conclusion about tallies at the end is checked as `nec_e_no_sqrt8`, `nec_e_check_strict`, `nec_e_check_never_tsirelson` and `nec_e_check_sharp` (NecEChsh.v). Equality $S^2 = 8$ under the row bounds forces equality in both steps of the proof above: the six squares must all vanish, which gives $a = b = c = -d$, and the rows must be tight, which gives $2a^2 = 1$. So the point of Theorem F.6 and its negative are the only places the bound is reached. A tally's correlators are ratios of whole numbers and $2a^2 = 1$ has no such solution, so no tally reaches the bound; the integer check's passing tallies stay strictly below it and come as near as you like.

Lemma F.7 (A rational constant above the bound, `rational_tsirelson_bound`)**.** $\sqrt{8} < 5657/2000$ (kernel/quantum/TsirelsonFromAlgebra.v).

Proof. Both numbers are positive, so it suffices to compare squares. $5657^2 = 32{,}001{,}649$ and $8 \cdot 2000^2 = 32{,}000{,}000$, so $(5657/2000)^2 = 32{,}001{,}649/4{,}000{,}000 > 8 = (\sqrt{8})^2$. If $\sqrt{8} \geq 5657/2000$ held, squaring both nonnegative sides would give $8 \geq (5657/2000)^2$, a contradiction. □

The constant is the four-decimal round-up of $2\sqrt{2} = 2.8284271\ldots$; the gap is about 7.3×10^{-5}. It lets rational code test "above the Tsirelson bound" without a square root. It plays no part in the integer check of item E.

F.4 The zero-marginal 5x5 moment matrix

Definition F.5 (The pinned level-1 matrix)**.** `zero_marginal_npa` (kernel/quantum/NPAMomentMatrix.v) fills the general record `NPAMomentMatrix` with all four one-party entries and both within-party entries set to zero, and `npa_to_matrix` lays it out on rows and columns I, A_0, A_1, B_0, B_1:

$$M(E) \;=\; \begin{array}{c} \\ I \\ A_0 \\ A_1 \\ B_0 \\ B_1 \end{array} \begin{array}{c} \begin{array}{ccccc} I & A_0 & A_1 & B_0 & B_1 \end{array} \\ \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & E_{00} & E_{01} \\ 0 & 0 & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & 0 \\ 0 & E_{01} & E_{11} & 0 & 1 \end{pmatrix} \end{array}$$

Eigenvalues never come into the source's definition of positive semidefiniteness. It is defined by quadratic forms: `PSD5` (kernel/quantum/ConstructivePSD.v) says

$v^{\mathsf{T}} M v \geq 0$ for every $v \in \mathbb{R}^5$, and `npa_psd` (NPAMomentMatrix.v) is "symmetric and `PSD5`". In block form,

$$M(E) \;=\; \begin{pmatrix} 1 & 0 \\ 0 & G \end{pmatrix}, \qquad G \;=\; \begin{pmatrix} I_2 & E \\ E^{\mathsf{T}} & I_2 \end{pmatrix}.$$

The corner 1 is decoupled, so $M(E)$ is PSD exactly when G is.

Lemma F.8 (Schur complement, the standard form)**.** Let P be symmetric positive definite. The symmetric block matrix $\begin{pmatrix} P & Q \\ Q^{\mathsf{T}} & R \end{pmatrix}$ is PSD if and only if $R - Q^{\mathsf{T}} P^{-1} Q$ is PSD.

Proof. For u, w of the right sizes,

$$\begin{pmatrix} u \\ w \end{pmatrix}^{\mathsf{T}} \begin{pmatrix} P & Q \\ Q^{\mathsf{T}} & R \end{pmatrix} \begin{pmatrix} u \\ w \end{pmatrix} = (u + P^{-1}Qw)^{\mathsf{T}} P \,(u + P^{-1}Qw) + w^{\mathsf{T}} (R - Q^{\mathsf{T}} P^{-1} Q)\, w.$$

The first term is never negative because P is positive definite, and it is zero at $u = -P^{-1}Qw$. So the form is nonnegative for all (u, w) exactly when the second term is nonnegative for all w. □

The lemma is checked in general form, for finite index sets and any symmetric P with a symmetric inverse, as `schur_identity` and `schur_complement_psd` (SchurComplement.v); the forward direction needs no positivity at all, and the converse needs P positive semidefinite. The proofs of the checks don't invoke it abstractly. They carry out its one instance with $P = I_2$, $Q = E$, $R = I_2$, where the complement is

$$I_2 - E^{\mathsf{T}} E = \begin{pmatrix} 1 - E_{00}^2 - E_{10}^2 & -(E_{00}E_{01} + E_{10}E_{11}) \\ -(E_{00}E_{01} + E_{10}E_{11}) & 1 - E_{01}^2 - E_{11}^2 \end{pmatrix}.$$

The diagonal of $E^{\mathsf{T}} E$ holds the squared lengths of the columns of E, which is why the conditions below are column conditions.

Definition F.6 (Column contractive, `zero_marginal_column_contractive`)**.** (kernel/quantum/CHSHColumnCheck.v) Write $p = 1 - E_{00}^2 - E_{10}^2$, $q = 1 - E_{01}^2 - E_{11}^2$ and $s = E_{00}E_{01} + E_{10}E_{11}$. The correlators are column contractive when

$$p \geq 0, \qquad q \geq 0, \qquad p\,q - s^2 \geq 0.$$

Lemma F.9 (The 2x2 test, `psd2_quadratic_form_nonneg`)**.** (kernel/quantum/CHSHColumnCheck.v) If $\alpha \geq 0$, $\delta \geq 0$ and $\alpha\delta - \beta^2 \geq 0$, then $\alpha u^2 + 2\beta uw + \delta w^2 \geq 0$ for all real u, w.

Proof. If $w = 0$ the form is $\alpha u^2 \geq 0$. Otherwise put $t = u/w$, so the form is $w^2(\alpha t^2 + 2\beta t + \delta)$. If $\alpha = 0$, then $-\beta^2 \geq 0$ forces $\beta = 0$ and the bracket is $\delta \geq 0$. If $\alpha > 0$, $\alpha t^2 + 2\beta t + \delta = \alpha(t + \beta/\alpha)^2 + (\alpha\delta - \beta^2)/\alpha$, a sum of two nonnegative terms. □

Theorem F.10 (Contractive implies PSD)**.** If $p \geq 0$, $q \geq 0$ and $pq - s^2 \geq 0$, then `PSD5` holds of $M(E)$. This is `zero_marginal_npa_column_contractive_implies_psd` (kernel/quantum/CHSHColumnCheck.v).

Proof. Write $v = (v_0, x_0, x_1, y_0, y_1)$. Reading off the entries of $M(E)$,

$$v^{\mathsf{T}} M(E)\, v = v_0^2 + x_0^2 + x_1^2 + y_0^2 + y_1^2 + 2x_0(E_{00}y_0 + E_{01}y_1) + 2x_1(E_{10}y_0 + E_{11}y_1).$$

Complete the squares in x_0 and x_1:

$$\begin{aligned} v^{\mathsf{T}} M(E)\, v = v_0^2 &+ (x_0 + E_{00}y_0 + E_{01}y_1)^2 + (x_1 + E_{10}y_0 + E_{11}y_1)^2 \\ &+ p\, y_0^2 - 2s\, y_0 y_1 + q\, y_1^2. \end{aligned}$$

To check the last three terms, expand the two squares: they contribute $(E_{00}y_0 + E_{01}y_1)^2 + (E_{10}y_0 + E_{11}y_1)^2$ beyond the cross terms, which is $(E_{00}^2 + E_{10}^2)y_0^2 + 2s\, y_0 y_1 + (E_{01}^2 + E_{11}^2)y_1^2$; subtracting it from $y_0^2 + y_1^2$ leaves $p\, y_0^2 - 2s\, y_0 y_1 + q\, y_1^2$. The first three terms are squares. The last is the 2×2 form with $\alpha = p$, $\beta = -s$, $\delta = q$, nonnegative by Lemma F.9. □

Lemma F.11 (Discriminant, `quadratic_nonneg_discriminant`)**.** (kernel/quantum/ConstructivePSD.v) If $a + 2bt + ct^2 \geq 0$ for every real t, then $b^2 \leq ac$.

Proof. If $c < 0$ the quadratic tends to $-\infty$, so $c \geq 0$. If $c = 0$, the line $a + 2bt$ is nonnegative for every t only if $b = 0$, and then $b^2 = 0 = ac$. If $c > 0$, evaluate at $t = -b/c$: $a - b^2/c \geq 0$, so $b^2 \leq ac$. (The Coq proof is the same case analysis, with explicit witnesses t for the contradictions.) □

Theorem F.12 (PSD implies contractive)**.** If `PSD5` holds of $M(E)$, then $p \geq 0$, $q \geq 0$ and $pq - s^2 \geq 0$. This is `npa_psd_implies_column_contractive` (kernel/quantum/CHSHColumnCheck.v).

Proof. Feed the quadratic form three test vectors, written in the order (I, A_0, A_1, B_0, B_1); their values are the lemmas `npa_quad5_test_col0`, `npa_quad5_test_col1` and `npa_quad5_test_schur` (CHSHColumnCheck.v). Each is the choice $x = -Ey$ that zeroes both completed squares in the proof of Theorem F.10, so only $y^{\mathsf{T}}(I_2 - E^{\mathsf{T}}E)y$ is left.

First, $v = (0, -E_{00}, -E_{10}, 1, 0)$ gives $v^{\mathsf{T}}Mv = p$, so $p \geq 0$.

Second, $v = (0, -E_{01}, -E_{11}, 0, 1)$ gives $v^{\mathsf{T}}Mv = q$, so $q \geq 0$.

Third, for each real t, $v(t) = (0, -(E_{00}t + E_{01}), -(E_{10}t + E_{11}), t, 1)$ gives $v(t)^{\mathsf{T}}Mv(t) = p\,t^2 - 2s\,t + q \geq 0$. Apply Lemma F.11 with $a = q$, $b = -s$, $c = p$: $s^2 \leq pq$. □

Corollary F.13 (The equivalence)**.** For all real $E_{00}, E_{01}, E_{10}, E_{11}$, `PSD5` of $M(E)$ holds if and only if the correlators are column contractive: `npa_psd_iff_column_contractive` (kernel/quantum/CHSHColumnCheck.v). With symmetry packaged in, `column_contractive_iff_npa_psd` states contractivity if and only if `npa_psd` holds of `zero_marginal_npa` at the four correlators.

The equivalence above is about the columns. The bound itself goes through the rows.

Theorem F.14 (PSD implies Tsirelson)**.** If `npa_psd` holds of `zero_marginal_npa` at $E_{00}, E_{01}, E_{10}, E_{11}$, then $S^2 \leq 8$, and $|S| \leq \sqrt{8}$. These are `npa_psd_implies_tsirelson_bound` and `npa_psd_implies_tsirelson_bound_abs`.

Proof. `psd_3x3_determinant_nonneg` (kernel/quantum/ConstructivePSD.v) says a symmetric PSD matrix with unit diagonal has every 3×3 principal minor $1 - x^2 - y^2 - z^2 + 2xyz \geq 0$. On rows A_0, B_0, B_1 the entries are $x = E_{00}$, $y = E_{01}$, $z = 0$, so $E_{00}^2 + E_{01}^2 \leq 1$. On rows A_1, B_0, B_1 the same gives $E_{10}^2 + E_{11}^2 \leq 1$. This is `npa_psd_zero_marginal_implies_row_bounds`. Then Theorem F.5 applies, through `tsirelson_from_minors` (kernel/quantum/TsirelsonGeneral.v). □

Here are the three conditions and the score at a few points, all by direct arithmetic. In this table, $a = 3/5$, $b = 4/5$ and $t = 1/\sqrt{2}$.

point $(E_{00}, E_{01}, E_{10}, E_{11})$	S	p	q	$pq - s^2$	**contractive**
$(0, 0, 0, 0)$	0	1	1	1	yes
$(0.6, 0.6, 0.6, -0.6)$	2.4	0.28	0.28	0.0784	yes
$(a, b, b, -a)$	2.8	0	0	0	yes, with no slack
$(t, t, t, -t)$	$2\sqrt{2}$	0	0	0	yes, with no slack
$(1, 1, 1, 1)$, all-ones plan	2	-1	-1		no
$(1, 0, 1, 0)$, a classical mixture	2	-1	1		no
$(1, 1, 1, -1)$, PR box	4	-1	-1		no

Every deterministic plan has all four correlators equal to ± 1, so $p = 1 - 1 - 1 = -1$ and every one of the sixteen fails.

Remark F.3 (What PSD of the pinned matrix means)**.** $M(E)$ is the level-1 moment matrix with the two within-party entries $\langle A_0 A_1 \rangle$ and $\langle B_0 B_1 \rangle$ pinned to zero. Its PSD is sufficient for the correlators to admit some PSD completion (item G, `zero_marginal_implies_elliptope`), and it is not necessary: the all-ones plan has a completion but fails here. So a check built on $M(E)$ is sound for the completion set and not complete for it, and it rejects some honest classical tallies. That correlators with a PSD completion are exactly those reachable by quantum measurements is Tsirelson's theorem, and both halves are proved here in finite dimension. For real and for complex amplitudes, a strategy's correlators are inner products of unit vectors (`qs_corr_dot`, `qs_uvec_unit` and `qs_vvec_unit`; with complex amplitudes, `qc_corr_dot`, `qc_uvec_unit` and `qc_vvec_unit`, the vectors read as real ones of twice the length) and its level-1 moment matrix is PSD (`qs_npa_psd`, `qc_npa_psd`), so they have a PSD completion (`tr_quantum_elliptope`, `tr_complex_elliptope`). The other half, that every PSD completion comes from a quantum strategy, is `tr_representation`: the strategy is built on two eight-level systems from the completion's Gram vectors (item H).

F.5 The integer check

Definition F.7 (`column_contractive_check_witness`)**.** (kernel/quantum/CHSHColumnCheck.v) From the counters of Definition F.2, with all arithmetic in $\mathbb{Z}$, let

$$A = N_{00}^2 N_{10}^2 - D_{00}^2 N_{10}^2 - D_{10}^2 N_{00}^2,$$
$$B = N_{01}^2 N_{11}^2 - D_{01}^2 N_{11}^2 - D_{11}^2 N_{01}^2,$$
$$C = D_{00} D_{01} N_{10} N_{11} + D_{10} D_{11} N_{00} N_{01}.$$

The check is the conjunction of seven facts, in this order: $0 < N_{00}$, $0 < N_{01}$, $0 < N_{10}$, $0 < N_{11}$, $0 \le A$, $0 \le B$, and $C \cdot C \le A \cdot B$.

There is no division, no real number, no square root and no S in it.

Lemma F.15 (The seven facts are the three conditions with denominators cleared)**.** Assume $N_{00}, N_{01}, N_{10}, N_{11} > 0$ and put $E_{xy} = D_{xy}/N_{xy}$. Then, with p, q, s as in Definition F.6,

$$p = \frac{A}{N_{00}^2 N_{10}^2}, \qquad q = \frac{B}{N_{01}^2 N_{11}^2}, \qquad s = \frac{C}{N_{00} N_{01} N_{10} N_{11}},$$
$$pq - s^2 = \frac{AB - C^2}{(N_{00} N_{01} N_{10} N_{11})^2}.$$

Because every denominator is positive, $0 \le A \iff p \ge 0$, $0 \le B \iff q \ge 0$ and $C^2 \le AB \iff s^2 \le pq$.

Proof. For p: $1 - D_{00}^2/N_{00}^2 - D_{10}^2/N_{10}^2$; multiply by $N_{00}^2 N_{10}^2$ to get $N_{00}^2 N_{10}^2 - D_{00}^2 N_{10}^2 - D_{10}^2 N_{00}^2 = A$. For q the same with columns 01 and 11, multiplied by $N_{01}^2 N_{11}^2$. For s:

$$\frac{D_{00}}{N_{00}} \frac{D_{01}}{N_{01}} + \frac{D_{10}}{N_{10}} \frac{D_{11}}{N_{11}} = \frac{D_{00} D_{01} N_{10} N_{11} + D_{10} D_{11} N_{00} N_{01}}{N_{00} N_{01} N_{10} N_{11}} = \frac{C}{N_{00} N_{01} N_{10} N_{11}}.$$

Then $pq = AB/(N_{00}^2 N_{10}^2 N_{01}^2 N_{11}^2) = AB/(N_{00} N_{01} N_{10} N_{11})^2$ and $s^2 = C^2/(N_{00} N_{01} N_{10} N_{11})^2$; subtract. Multiplying an inequality by a positive number preserves it in both directions. □

The Coq source states the direction it needs:

Theorem F.16 (Soundness of the integer check)**.** If `column_contractive_check_witness` w = `true`, then the four correlators `state_bucket_correlation` same_{xy} diff_{xy} of w are column contractive. This is `column_contractive_check_witness_sound` (kernel/quantum/CHSHColumnCheck.v), and `column_contractive_check_witness_npa_psd` (same file) carries it on to PSD of the pinned matrix.

Proof. The first four conjuncts make every N_{xy} positive, so the empty-bucket convention never applies and each correlator is D_{xy}/N_{xy}. The integer facts lift to the reals by the injection $\mathbb{Z} \to \mathbb{R}$, which preserves order. Lemma F.15 turns them into the three conditions. □

Corollary F.17 (A passed check implies the Tsirelson bound)**.** If `column_contractive_check_witness` w = `true`, the correlators of w satisfy $S^2 \leq 8$. It is the composition of Theorem F.16, Corollary F.13 and Theorem F.14.

On the small machine the same conclusion is one checked theorem, `small_chsh_check_tsirelson` (SmallChshCheck.v), and its earned form is `small_chsh_flag_implies_tsirelson` (SmallChshMachine.v); see Section 20.

A worked tally. Take $(\text{same}, \text{diff}) = (4,1),(4,1),(4,1),(1,4)$ for the pairs $00, 01, 10, 11$. Then $N_{xy} = 5$ for all four, $D_{00} = D_{01} = D_{10} = 3$, $D_{11} = -3$, and the correlators are $0.6, 0.6, 0.6, -0.6$ with $S = 2.4$. Compute:

$$A = 25 \cdot 25 - 9 \cdot 25 - 9 \cdot 25 = 625 - 225 - 225 = 175,$$
$$B = 25 \cdot 25 - 9 \cdot 25 - 9 \cdot 25 = 175,$$
$$C = 3 \cdot 3 \cdot 5 \cdot 5 + 3 \cdot (-3) \cdot 5 \cdot 5 = 225 - 225 = 0.$$

All seven facts hold: four positive N, $0 \leq 175$ twice, and $0 \leq 175 \cdot 175$. Dividing back, $p = 175/625 = 0.28$ and $s = 0$, the second row of the table in item D.

Two more, for contrast. One round per pair from the all-ones plan, $(1,0)$ four times, gives $N = D = 1$ everywhere and $A = 1 - 1 - 1 = -1 < 0$: the check fails. The tally $(4,1),(9,1),(9,1),(1,4)$ gives correlators $3/5, 4/5, 4/5, -3/5$, with $N = 5, 10, 10, 5$ and $D = 3, 8, 8, -3$: $A = 25 \cdot 100 - 9 \cdot 100 - 64 \cdot 25 = 0$, $B = 100 \cdot 25 - 64 \cdot 25 - 9 \cdot 100 = 0$ and $C = 3 \cdot 8 \cdot 10 \cdot 5 + 8 \cdot (-3) \cdot 5 \cdot 10 = 0$. It passes with no slack, at $S = 2.8$.

F.6 Level 1+AB

The source's comments index Alice's observables A_1, A_2 and Bob's B_1, B_2, so its $\langle A_1 B_1 \rangle$ is E_{00} here. The Coq variables themselves are already named in the book's order, from e_{00} to e_{11}. The five extra parameters are named in the record `Q1ABMoments` (kernel/quantum/QuantumPartitionPSD_1AB.v): γ_1 for $\langle A_0 A_1 B_0 \rangle$, γ_2 for $\langle A_0 A_1 B_1 \rangle$, γ_3 for $\langle A_0 B_0 B_1 \rangle$, γ_4 for $\langle A_1 B_0 B_1 \rangle$, γ_5 for $\langle A_0 A_1 B_0 B_1 \rangle$. These are labels only; the file builds no operators.

Definition F.8 (`q1ab_moment_matrix`)**.** (kernel/quantum/QuantumPartitionPSD_1AB.v) The 9×9 real matrix on rows and columns $I, A_0, A_1, B_0, B_1, A_0B_0, A_0B_1, A_1B_0, A_1B_1$, with the entries exactly as the Coq definition places them:

	I	A_0	A_1	B_0	B_1	A_0B_0	A_0B_1	A_1B_0	A_1B_1
I	1	0	0	0	0	E_{00}	E_{01}	E_{10}	E_{11}
A_0	0	1	0	E_{00}	E_{01}	0	0	γ_1	γ_2
A_1	0	0	1	E_{10}	E_{11}	γ_1	γ_2	0	0
B_0	0	E_{00}	E_{10}	1	0	0	γ_3	0	γ_4
B_1	0	E_{01}	E_{11}	0	1	γ_3	0	γ_4	0
A_0B_0	E_{00}	0	γ_1	0	γ_3	1	0	0	γ_5
A_0B_1	E_{01}	0	γ_2	γ_3	0	0	1	$-\gamma_5$	0
A_1B_0	E_{10}	γ_1	0	0	γ_4	0	$-\gamma_5$	1	0
A_1B_1	E_{11}	γ_2	0	γ_4	0	γ_5	0	0	1

Two constraints are fixed by this layout. First, the four-body cells carry opposite signs: $+\gamma_5$ at (A_0B_0, A_1B_1) and $-\gamma_5$ at (A_0B_1, A_1B_0), guarded by `q1ab_four_body_cells_are_conjugate` (QuantumPartitionPSD_1AB.v). In a real moment matrix those two cells are $\langle A_0A_1B_0B_1\rangle$ and $\langle A_0A_1B_1B_0\rangle$, so the tie says $\langle A_0A_1(B_0B_1 + B_1B_0)\rangle = 0$, which for a real matrix is $\langle\{A_0, A_1\} \otimes \{B_0, B_1\}\rangle = 0$. The level-1 pinnings don't imply it. Take a classical strategy: Alice's two answers a_0, a_1 are independent fair signs, and Bob copies them, $b_0 = a_0$ and $b_1 = a_1$. Every pinned cell of its moment matrix is 0 and both four-body cells are 1, so no choice of the nine numbers gives its matrix (`q1ab_tie_is_a_constraint`, QuantumPartitionPSD_1AB.v). The matrix here is therefore level 1+AB on the slice $\langle\{A_0, A_1\} \otimes \{B_0, B_1\}\rangle = 0$, and that is what "level 1+AB" names in this book. The slice holds whenever A_0 and A_1 anticommute. The soundness theorems below are about this matrix, so they stand as stated. Second, the matrix keeps the level-1 pinning: every cell whose label reduces to a marginal or to $\langle A_0A_1\rangle$ or $\langle B_0B_1\rangle$ is zero, for instance (A_0, A_0B_0), (A_0B_0, A_0B_1) and (A_0B_0, A_1B_0). Symmetry is `q1ab_moment_matrix_symmetric`.

Write a vector as $(v_I, v_{A_0}, v_{A_1}, v_{B_0}, v_{B_1}, v_{00}, v_{01}, v_{10}, v_{11})$, where v_{xy} is the coordinate of A_xB_y. Put

$$L_1 = E_{00}v_{00} + E_{01}v_{01} + E_{10}v_{10} + E_{11}v_{11},$$
$$L_2 = E_{00}v_{B_0} + E_{01}v_{B_1} + \gamma_1 v_{10} + \gamma_2 v_{11},$$
$$L_3 = E_{10}v_{B_0} + E_{11}v_{B_1} + \gamma_1 v_{00} + \gamma_2 v_{01}.$$

Definition F.9 (The residual, `q1ab_residual`)**.** (QuantumPartitionPSD_1AB.v)

$$\begin{aligned} R = {} & v_{B_0}^2 + v_{B_1}^2 + v_{00}^2 + v_{01}^2 + v_{10}^2 + v_{11}^2 \\ & + 2v_{B_0}(\gamma_3 v_{01} + \gamma_4 v_{11}) + 2v_{B_1}(\gamma_3 v_{00} + \gamma_4 v_{10}) \\ & + 2\gamma_5(v_{00}v_{11} - v_{01}v_{10}) - L_1^2 - L_2^2 - L_3^2. \end{aligned}$$

The predicate `column_contractive_q1ab` says $R \geq 0$ for all six real inputs.

Lemma F.18 (Three squares plus the residual, `quad9_q1ab_sos_decomposition`)**.** (QuantumPartitionPSD_1AB.v, closed by `ring`.) For every vector,

$$v^{\mathsf{T}} M_9\, v = (v_I + L_1)^2 + (v_{A_0} + L_2)^2 + (v_{A_1} + L_3)^2 + R.$$

Proof. The coordinates v_I, v_{A_0}, v_{A_1} appear in the form only through their diagonal squares and the cross terms $2v_I L_1$, $2v_{A_0} L_2$, $2v_{A_1} L_3$, read off rows I, A_0, A_1 of the matrix. Completing each square adds $L_1^2 + L_2^2 + L_3^2$, which the residual subtracts. What is left in R is the diagonal of the remaining six coordinates and the cross terms among them, read off the lower right 6×6 block. □

Theorem F.19 (PSD at level 1+AB, `q1ab_psd_iff_column_contractive`)**.** (QuantumPartitionPSD_1AB.v) For all real E_{xy} and $\gamma_1, \ldots, \gamma_5$, `PSD9` of the matrix of Definition F.8 holds if and only if `column_contractive_q1ab` holds. The directions are `column_contractive_q1ab_implies_psd9` and `psd9_implies_column_contractive_q1ab`.

Proof. If $R \geq 0$ everywhere, Lemma F.18 writes the form as three squares plus R. Conversely, given the six residual inputs, choose $v_I = -L_1$, $v_{A_0} = -L_2$, $v_{A_1} = -L_3$; the three squares vanish and PSD gives $R \geq 0$. This is the level-1 argument of item D one size up. □

As a quadratic form in $(v_{B_0}, v_{B_1}, v_{00}, v_{01}, v_{10}, v_{11})$, numbered 1 to 6, the residual has the symmetric coefficient matrix H with these upper entries. They are the `q12345_H11` to `q12345_H66` definitions (QuantumPartitionPSD_1AB.v), and `q12345_sym6_qf_equals_residual` proves the form equals R. Each entry can be read back off R by collecting the coefficient of $v_i v_j$.

$$
\begin{aligned}
H_{11} &= 1 - E_{00}^2 - E_{10}^2, & H_{22} &= 1 - E_{01}^2 - E_{11}^2,\\
H_{12} &= -(E_{00}E_{01} + E_{10}E_{11}), & H_{13} &= -E_{10}\gamma_1,\\
H_{14} &= \gamma_3 - E_{10}\gamma_2, & H_{15} &= -E_{00}\gamma_1,\\
H_{16} &= \gamma_4 - E_{00}\gamma_2, & H_{23} &= \gamma_3 - E_{11}\gamma_1,\\
H_{24} &= -E_{11}\gamma_2, & H_{25} &= \gamma_4 - E_{01}\gamma_1,\\
H_{26} &= -E_{01}\gamma_2, & H_{33} &= 1 - E_{00}^2 - \gamma_1^2,\\
H_{34} &= -(E_{00}E_{01} + \gamma_1\gamma_2), & H_{35} &= -E_{00}E_{10},\\
H_{36} &= \gamma_5 - E_{00}E_{11}, & H_{44} &= 1 - E_{01}^2 - \gamma_2^2,\\
H_{45} &= -\gamma_5 - E_{01}E_{10}, & H_{46} &= -E_{01}E_{11},\\
H_{55} &= 1 - E_{10}^2 - \gamma_1^2, & H_{56} &= -(E_{10}E_{11} + \gamma_1\gamma_2),\\
H_{66} &= 1 - E_{11}^2 - \gamma_2^2.
\end{aligned}
$$

The top left 2×2 block is $I_2 - E^{\mathsf{T}}E$ from item D.

The full check does not decide $R \geq 0$ directly. It proves the stronger fact that H is positive definite, by two Schur steps and a Sylvester test, all fraction-free.

Lemma F.20 (Fraction-free Schur step)**.** For a symmetric $n \times n$ matrix h and any vector v, with $S_{ij} = h_{11}h_{ij} - h_{1i}h_{1j}$ for $i, j \geq 2$ (here S is the Schur matrix; the score plays no part in this lemma),

$$
h_{11}\, v^{\mathsf{T}} h\, v = \Big(\sum_j h_{1j} v_j\Big)^2 + \sum_{i,j\geq 2} S_{ij} v_i v_j.
$$

So if $h_{11} > 0$ and the $(n-1) \times (n-1)$ matrix S gives a nonnegative form, then h does too. The instances used are `sym6_Schur_identity` (QuantumPartitionPSD_1AB.v) and `sym5_Schur_identity`, with the conclusions `sym6_qf_nonneg_from_pd` and `sym5_qf_nonneg_from_pd`.

Proof. Expand the square: $h_{11}^2 v_1^2 + 2h_{11}v_1 \sum_{j\geq 2} h_{1j}v_j + (\sum_{j\geq 2} h_{1j}v_j)^2$. The left side is $h_{11}^2 v_1^2 + 2h_{11}v_1 \sum_{j\geq 2} h_{1j}v_j + h_{11}\sum_{i,j\geq 2} h_{ij}v_i v_j$. The difference is $\sum_{i,j\geq 2}(h_{11}h_{ij} - h_{1i}h_{1j})v_i v_j$. Dividing by $h_{11} > 0$ keeps the sign. □

Lemma F.21 (Sylvester for 4x4, `sym4_qf_nonneg_from_pd`)**.** (QuantumPartitionPSD_1AB.v) Let h be symmetric 4×4 with leading principal minors d_1, d_2, d_3, d_4 (`sym4_d1` to `sym4_d4`). If all four are strictly positive, then $v^{\mathsf{T}} h\, v \geq 0$ for all v.

Proof. `sym4_LDLT_identity` (closed by `ring`) is the fraction-free factorization

$$d_1^2 d_2 d_3\, v^{\mathsf{T}} h\, v = d_1 d_2 d_3\, P_1^2 + d_1 d_3\, Q_1^2 + Q_2^2 + d_1^2 d_2 d_4\, v_4^2,$$

where $P_1 = \sum_j h_{1j} v_j$, $Q_1 = (h_{11}h_{22} - h_{12}^2)v_2 + (h_{11}h_{23} - h_{12}h_{13})v_3 + (h_{11}h_{24} - h_{12}h_{14})v_4$, and Q_2 is the third linear form (`sym4_Q2`). With every $d_k > 0$ the right side is nonnegative and the factor on the left is positive. □

The cascade for the full check runs on H. Step one: $H_{11} > 0$ and the 5×5 matrix $S_{ij}^{(6)} = H_{11}H_{ij} - H_{1i}H_{1j}$, $2 \leq i, j \leq 6$, gives a nonnegative form. Step two: $S_{22}^{(6)} > 0$ and the 4×4 matrix $S_{ij}^{(5)} = S_{22}^{(6)} S_{ij}^{(6)} - S_{2i}^{(6)} S_{2j}^{(6)}$, $3 \leq i, j \leq 6$, gives a nonnegative form. Step three: the four leading minors of $S^{(5)}$ are positive. The real predicates are `sym5_pd_interior`, `sym6_pd_interior` and the witness `q1ab_g12345_minors_witness`, which implies `column_contractive_q1ab` and then `PSD9` (`q1ab_g12345_minors_witness_implies_psd9`).

Definition F.10 (What the integer check computes)**.** The supplied moments arrive as bucket pairs: $\gamma_k = N_{g_k}/D_{g_k}$ with $N_{g_k} = \text{same}_{g_k} - \text{diff}_{g_k}$ and $D_{g_k} = \text{same}_{g_k} + \text{diff}_{g_k}$. Watch the letters: for these five buckets the source swaps them, so here N is the difference and D the count, the reverse of item A. I keep the source's letters so the names below match the files. Let $K = (N_{00}N_{01}N_{10}N_{11}D_{g_1}D_{g_2}D_{g_3}D_{g_4}D_{g_5})^2$ (`g12345_COMMON_Z`, kernel/quantum/CHSHColumnCheck.v). The check computes the 21 integers KH_{ij} (`cleared_g12345_H11_Z` to `cleared_g12345_H66_Z`), the 15 integers of $S^{(6)}$ at scale K^2 and the 10 integers of $S^{(5)}$ at scale K^4, each by `schur_step_Z`, $h_{11}h_{ij} - h_{1i}h_{1j}$ on integers. The decider `q1ab_g12345_check_z_kernel` is the conjunction of: $N_{xy} > 0$ for the four pairs; $D_{g_k} > 0$ for $k = 1..5$; $-D_{g_k} < N_{g_k} < D_{g_k}$ for $k = 1..5$; and six strict positivities, namely $KH_{11} > 0$, $K^2 S_{22}^{(6)} > 0$, and the four leading minors of the integer $S^{(5)}$. The full check `q1ab_g12345_full_integer_check_kernel` is `column_contractive_check_witness` of item E and this decider.

The bridge lemmas, for example `cleared_g12345_H11_Z_bridge` (QuantumPartitionPSD_1AB.v), prove that each cleared integer, read as a real, equals the corresponding real quantity times the stated positive power of K. A $k \times k$ leading minor of a matrix scaled by $c > 0$ scales by c^k. So every integer positivity holds exactly when the real one does.

Theorem F.22 (The full 1+AB check implies PSD)**.** Let the counts make every N_{xy} and every D_{g_k} positive, and let `q1ab_g12345_check_z_kernel` pass on them. Then the matrix of Definition F.8 at the four correlators and $\gamma_k = N_{g_k}/D_{g_k}$ is symmetric and `PSD9`. This is `q1ab_g12345_caller_witness_z_abs_sound` followed by `q1ab_g12345_minors_witness_implies_psd9` (kernel/quantum/QuantumPartitionPSD_1AB.v).

Proof. The bucket conjuncts make every denominator positive, the bridges carry the six integer positivities to the real cascade, and Lemma F.20 twice plus Lemma F.21 give $R \geq 0$, hence `PSD9` by Theorem F.19. □

Remark F.4 (Interior only)**.** Every test in the cascade is strict, so a passing check certifies that H is positive definite, which is stronger than semidefinite. A point where H is PSD but singular cannot pass, and neither can a moment with $|\gamma_k| = 1$. For example, the tally of item E with correlators $(3/5, 4/5, 4/5, -3/5)$ has $H_{11} = 1 - 9/25 - 16/25 = 0$, so the full 1+AB check fails on it for every choice of γ, though the level-1 check of item E accepts it (`nec_e_singular_full_fails` and `nec_e_singular_level1_passes`, NecEChshEquality.v). The source proves soundness only; no converse is stated.

A worked example of the cascade, in exact rational arithmetic, checked by computation as `nec_e_worked_cascade_passes`, `nec_e_worked_cascade_fails_at_zero` and `nec_e_worked_level1_passes` (NecEChshEquality.v). Take the correlators $(0.6, 0.6, 0.6, -0.6)$ and $\gamma = (0, 0, 0, 0, -1/2)$; as buckets, γ_1 to γ_4 are $(1, 1)$, giving $0/2$, and γ_5 is $(1, 3)$, giving $-2/4$, both inside the strict bounds. Then $H_{11} = H_{22} = 7/25$, $H_{12} = 0$, and every entry H_{1j}, H_{2j} with $j \geq 3$ is zero. So $S_{22}^{(6)} = (7/25)^2 = 49/625$, and $S^{(5)}$ is a positive multiple of the lower right block of H:

$$S^{(5)} = \frac{343}{781250} \begin{pmatrix} 32 & -18 & -18 & -7 \\ -18 & 32 & 7 & 18 \\ -18 & 7 & 32 & 18 \\ -7 & 18 & 18 & 32 \end{pmatrix},$$

where $50H_{33} = 32$, $50H_{34} = 50H_{35} = -18$, $50H_{36} = 50(-1/2 + 0.36) = -7$, $50H_{45} = 50(1/2 - 0.36) = 7$, and $50H_{46} = 50H_{56} = 18$. The leading minors of the integer matrix are 32, 700, 15000 and 140625, all positive. With $H_{11} = 7/25 > 0$ and $S_{22}^{(6)} > 0$, all six tests pass, and with item E's $A = B = 175$, $C = 0$ the full check passes. At $\gamma_5 = 0$ instead, the same block becomes $I_4 - \mathbf{e}\mathbf{e}^{\mathsf{T}}$ with $\mathbf{e} = (0.6, 0.6, 0.6, -0.6)$, whose third leading minor is negative ($\frac{1}{25^3}$ times -1250), and the check fails.

That last fact is general. The zero-moment check `column_contractive_check_q1ab_kernel` (kernel/quantum/CHSHColumnCheck.v) takes no γ buckets; it is the level-1 check together with `sum_E_sq_check_witness`, the integer form of $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \le 1$ with denominators cleared by $(N_{00}N_{01}N_{10}N_{11})^2$.

Theorem F.23 (The zero-moment check sits inside the classical bound)**.** If `column_contractive_check_q1ab` w = `true` (QuantumPartitionPSD_1AB.v, the same function), then the four correlators of w satisfy $E_{00}^2 + E_{01}^2 + E_{10}^2 + E_{11}^2 \le 1$ (`q1ab_check_at_gzero_forces_unit_ball`) and $|S| \le 2$ (`q1ab_check_at_gzero_implies_classical_bound`).

Proof. At $\gamma = 0$ the residual splits into the level-1 form in (v_{B_0}, v_{B_1}) and $|\mathbf{v}|^2 - \langle \mathbf{e}, \mathbf{v}\rangle^2$ in $\mathbf{v} = (v_{00}, v_{01}, v_{10}, v_{11})$, where $\mathbf{e} = (E_{00}, E_{01}, E_{10}, E_{11})$ (`q1ab_residual_g_zero_decomp`). The check gives column contractivity at $\gamma = 0$ (`column_contractive_check_q1ab_sound_at_g_zero`). Test at $v_{B_0} = v_{B_1} = 0$ and $\mathbf{v} = \mathbf{e}$: with $r = |\mathbf{e}|^2 \ge 0$, the residual is $r - r^2 \ge 0$, so $r \le 1$. Then Lemma F.4 gives $S^2 \le 4r \le 4$. □

So the zero-moment check certifies a strict part of the classical region: no point above $|S| = 2$, and none of the sixteen deterministic points either, since each has $|\mathbf{e}|^2 = 4$. A point above 2 needs supplied moments. `q1ab_certifies_a_superclassical_chsh_point` (QuantumPartitionPSD_1AB.v) proves `PSD9` at correlators $(3/5, 3/5, 3/5, -3/5)$, $\gamma = (0, 0, 0, 0, -1/2)$, with $S = 12/5$, through the γ_5 slice: its real condition `q1ab_g5_caller_witness` asks $-1 < \gamma_5 < 1$ and

$$(1-\gamma_5)\left[(E_{00}+E_{11})^2 + (E_{01}-E_{10})^2\right] + (1+\gamma_5)\left[(E_{00}-E_{11})^2 + (E_{01}+E_{10})^2\right] \le 2(1-\gamma_5^2).$$

At this point the left side is $\frac{3}{2}\cdot 0 + \frac{1}{2}\cdot(\frac{36}{25} + \frac{36}{25}) = \frac{36}{25} = 1.44$ and the right side is $2\cdot\frac{3}{4} = 1.5$. The integer version, `q1ab_g5_kernel_check_accepts_superclassical`, runs the γ_5 check on the counters $(4,1), (4,1), (4,1), (1,4)$ and the bucket $(1,3)$ and returns `true` by computation.

Each of the four 1+AB checks (CHSHColumnCheck.v) contains `column_contractive_check_witness` as a conjunct, so a passing 1+AB check also passes the level-1 check and inherits $S^2 \le 8$ from item E. That is a composition I did by reading the definitions; the source has no single theorem stating it.

F.7 The elliptope completion

Definition F.11 (Completion and membership)**.** `completed_npa` (kernel/quantum/EllitopeCompletion.v) is the record of item D with the within-party entries set to free reals: x in the (A_0, A_1) cells and y in the (B_0, B_1) cells, marginals still zero. The correlators are elliptope realizable (`elliptope_realizable`) when some x, y make the completed matrix symmetric and PSD.

$$M(E; x, y) \;=\; \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & x & E_{00} & E_{01} \\ 0 & x & 1 & E_{10} & E_{11} \\ 0 & E_{00} & E_{10} & 1 & y \\ 0 & E_{01} & E_{11} & y & 1 \end{pmatrix}$$

Its quadratic form at $v = (v_0, \ldots, v_4)$ is $\sum_k v_k^2 + 2xv_1v_2 + 2yv_3v_4 + 2E_{00}v_1v_3 + 2E_{01}v_1v_4 + 2E_{10}v_2v_3 + 2E_{11}v_2v_4$ (`completed_quad_expand`). The pinned matrix is the case $x = y = 0$, so PSD of $M(E)$ implies membership: `zero_marginal_implies_elliptope`.

Theorem F.24 (Deterministic plans are inside, `deterministic_strategy_elliptope`)**.** (EllitopeCompletion.v) If $a_0^2 = a_1^2 = b_0^2 = b_1^2 = 1$, then $(a_0b_0, a_0b_1, a_1b_0, a_1b_1)$ is elliptope realizable, with $x = a_0a_1$ and $y = b_0b_1$.

Proof. With these entries the form equals $v_0^2 + (a_0v_1 + a_1v_2 + b_0v_3 + b_1v_4)^2$. Expanding the second square gives $a_0^2v_1^2 + \cdots$ on the diagonal, which is $v_1^2 + \cdots$ by the unit squares, and the cross terms $2a_0a_1v_1v_2 = 2xv_1v_2$, $2a_0b_0v_1v_3 = 2E_{00}v_1v_3$, and so on for all six pairs. A sum of two squares is never negative. □

The all-ones plan is the instance `classical_tightness_witness_elliptope`, and the classical mixture $(1, 0, 1, 0)$ is `turing_point_elliptope`, at $x = 1$, $y = 0$, with form $v_0^2 + (v_1 + v_2 + v_3)^2 + v_4^2$.

Theorem F.25 (Convexity and finite mixtures)**.** For $0 \le \lambda \le 1$, if E and F are elliptope realizable, so is $\lambda E + (1 - \lambda)F$ (`elliptope_convex`, EllitopeCompletion.v). For finitely many members with nonnegative weights summing to at most 1, the weighted sum is realizable (`elliptope_finite_mixture`). Every finite mixture of deterministic plans is realizable (`lhv_mixture_elliptope`).

Proof. The form is affine in (E, x, y) for each fixed v. Take the completion $(\lambda x_1 + (1 - \lambda)x_2, \lambda y_1 + (1 - \lambda)y_2)$; the form at the mixture is the same mixture of two nonnegative forms. The finite version is induction on the number of members,

with the shortfall in weight absorbed by the zero point (`elliptope_zero`). The last statement combines this with Theorem F.24. □

Theorem F.26 (Tsirelson's bound holds on the completion set, `elliptope_tsirelson`)**.** (EllipteopeCompletion.v) If E is elliptope realizable then $S^2 \leq 8$.

Proof. For a symmetric PSD matrix the bilinear form obeys Cauchy-Schwarz, $\mathcal{B}(u,w)^2 \leq \mathcal{Q}(u)\,\mathcal{Q}(w)$ (`psd_cauchy_schwarz`), because $\mathcal{Q}(u+tw) = \mathcal{Q}(u) + 2t\,\mathcal{B}(u,w) + t^2\mathcal{Q}(w) \geq 0$ for all t and Lemma F.11 applies. Take $u_1 = e_{A_0}$ and $w_+ = e_{B_0} + e_{B_1}$: $\mathcal{B}(u_1, w_+) = E_{00} + E_{01}$, $\mathcal{Q}(u_1) = 1$, $\mathcal{Q}(w_+) = 2 + 2y$. Take $u_2 = e_{A_1}$ and $w_- = e_{B_0} - e_{B_1}$: $\mathcal{B}(u_2, w_-) = E_{10} - E_{11}$, $\mathcal{Q}(w_-) = 2 - 2y$. So $c_1^2 \leq 2 + 2y$ and $c_2^2 \leq 2 - 2y$ with $c_1 = E_{00} + E_{01}$, $c_2 = E_{10} - E_{11}$, and both budgets are nonnegative. Then $c_1^2 c_2^2 \leq (2+2y)(2-2y) = 4 - 4y^2 \leq 4$, so $c_1 c_2 \leq 2$, and $S^2 = (c_1 + c_2)^2 = c_1^2 + 2c_1c_2 + c_2^2 \leq (2+2y) + 4 + (2-2y) = 8$. □

The completion can make a row of E longer than 1 (the all-ones plan has rows of squared length 2), which is why this proof does not use the row bounds of item D.

Corollary F.27 (The PR box is outside, `pr_box_not_elliptope`)**.** (EllipteopeCompletion.v) $(1, 1, 1, -1)$ is not elliptope realizable, since its $S^2 = 16 > 8$.

Theorem F.28 (Strictly beyond the classical bound, `beyond_classical_elliptope`)**.** (EllipteopeCompletion.v) $(3/5, 3/5, 3/5, -3/5)$ is elliptope realizable and has $S = 12/5 > 2$.

Proof. At $x = y = 0$ the form equals $v_0^2 + (v_1 + \frac{3}{5}v_3 + \frac{3}{5}v_4)^2 + (v_2 + \frac{3}{5}v_3 - \frac{3}{5}v_4)^2 + \frac{7}{25}v_3^2 + \frac{7}{25}v_4^2$. Check: the two middle squares give v_3^2 coefficient $\frac{9}{25} + \frac{9}{25} = \frac{18}{25}$, plus $\frac{7}{25}$ is 1; the v_3v_4 terms are $+\frac{18}{25}$ and $-\frac{18}{25}$ and cancel; the cross terms are $2 \cdot \frac{3}{5}v_1v_3$, $2 \cdot \frac{3}{5}v_1v_4$, $2 \cdot \frac{3}{5}v_2v_3$ and $-2 \cdot \frac{3}{5}v_2v_4$, as the entries require. □

By Theorem F.2 no finite factorizable model has $S > 2$, so this point is not a finite mixture of plans, while Theorem F.25 puts every such mixture inside. The completion set therefore strictly contains the finite classical set, and the PR box lies outside it. The source does not state that the completion set lies inside the no-signalling set.

The gate (kernel/quantum/ElliptopeGate.v) is an integer decider with a soundness proof. Its completion entries arrive as bucket pairs, and an empty bucket is read as $0/1$ (`bden`), so unlike item E it does not require every pair to be sampled.

Definition F.12 (The two branches)**.** `elliptope_pd_check` (EllitopeGate.v): with the completion supplied as buckets and all entries scaled to integers by a positive common factor K, the four leading minors of the 4×4 block on A_0, A_1, B_0, B_1 are strictly positive. `elliptope_ldl_check`: with the completion and a rational certificate supplied (six subdiagonal entries over a common positive denominator, four pivots $d_k \geq 0$ over another), the ten entry equations of $M = LDL^{\mathsf{T}}$ hold in cross-multiplied integer arithmetic (`ldl_equations`). `elliptope_check_full` is their disjunction.

Theorem F.29 (Soundness of the gate, `elliptope_check_full_sound`)**.** (EllitopeGate.v) If `elliptope_check_full` returns `true` on a counter record, any completion buckets and any certificate, then the four correlators of the record are elliptope realizable. And `elliptope_full_gate_never_accepts_pr_box`: on the PR box tally the gate returns `false` for every choice of the remaining arguments.

Proof. The first branch reduces to Lemma F.21 through the same fraction-free identity (`elliptope_pd_check_sound`). The second is a weighted sum of squares: given the ten equations, the form is $\sum_k d_k P_k^2$ with $d_k \geq 0$ (`elliptope_ldl_check_sound`). The PR statement is soundness followed by Corollary F.27. □

Computed examples, each closed by `reflexivity`: `gate_accepts_all_ones`, the all-ones plan, through the second branch at $x = y = 1$; `gate_accepts_turing_point`; `gate_accepts_beyond_classical`, the $12/5$ point, through the first branch at $x = y = 0$; `gate_accepts_pythagorean_boundary`, the point $(3/5, 4/5, 4/5, -3/5)$, through the second branch with pivots $(1, 1, 0, 0)$.

The gate is a decider with a soundness proof, and that's all it is: no property of the small machine is built from it.

F.8 What is not proved

The following are cited results and open questions. Where part of one is proved, the paragraph names the theorem and the part that stays cited.

The arcsine boundary. For two settings and two outcomes per side, Tsirelson, Landau (1988) and Masanes (2003) give theorems characterizing the correlators reachable by quantum measurements: $|\arcsin E_{00} + \arcsin E_{01} + \arcsin E_{10} - \arcsin E_{11}| \leq \pi$ and its sign variants. The characterization is checked in ArcsineBoundary.v as an if and only if with the completion set (`am_arcsine`) and, through Tsirelson's representation, with the quantum set (`am_quantum_arcsine`). The point $(3/5, 4/5, 4/5, -3/5)$ is on the boundary: $\arcsin \frac{3}{5} + \arcsin \frac{4}{5} = \frac{\pi}{2}$ by the 3-4-5 triangle, so the sum is π (a

`m_pythagorean_boundary`); Tsirelson's point is on it too (`am_tsirelson_boundary`). The necessary direction holds in any dimension, for any expectation on an algebra of observables in which Alice's commute with Bob's (`ta_arcsine`, TsirelsonAlgebraic.v).

Equality with the quantum set. That the completion set of item G is exactly the set of quantum correlators is Tsirelson's representation theorem, and it is proved in finite dimension: `tr_representation` for real amplitudes and `tr_representation_complex` for complex ones (TsirelsonRepresentation.v), with two eight-level systems enough for every point. That PSD of the level-1 moment matrix is necessary for quantum behaviour is proved for strategies in finite dimension, with real amplitudes (`qs_npa_psd`, QuantumStrategies.v) and with complex ones (`qc_npa_psd`, QuantumStrategiesComplex.v), and in any dimension, marginals included, for any expectation on an algebra of observables (`ta_npa_psd`, TsirelsonAlgebraic.v).

Achievability. Theorem F.6 is a point of $\mathbb{R}^4$. A state and measurements that reach it are built in QuantumStrategies.v: the two-qubit state $(|00\rangle + |11\rangle)/\sqrt{2}$ with $A_0 = Z$, $A_1 = X$, $B_0 = (Z + X)/\sqrt{2}$ and $B_1 = (Z - X)/\sqrt{2}$ has correlators $1/\sqrt{2}$, $1/\sqrt{2}$, $1/\sqrt{2}$, $-1/\sqrt{2}$ and $S = 2\sqrt{2}$ (`qs_tsirelson_reached`), and every strategy in finite dimension has $S^2 \leq 8$, with real amplitudes (`qs_tsirelson`) and with complex ones (`qc_tsirelson`, QuantumStrategiesComplex.v). No experiment stands behind either.

Completeness of the checks. The 1+AB cascade and the elliptope gate are proved sound, and neither has a proved converse. The level-1 integer check is the exception: on the small machine the same seven integer facts hold exactly when every pair was sampled and the pinned matrix is positive semidefinite (`small_chsh_check_iff`, Theorem 20.4), so that check is complete for what it tests. The 1+AB cascade is strict and misses boundary points. The comment in ElliptopeGate.v that the second branch accepts every point with a rational PSD completion is a mathematical remark about LDL^{T} factorizations, and the file has no theorem for it; nor is there a theorem that the four 1+AB checks accept nested sets.

Finite levels. For full behaviours, marginals included, the NPA levels are relaxations. Chaturvedi (arXiv 2607.14569, 2026) argues, in a preprint, that no finite NPA level characterizes the complete quantum set even in this simplest Bell scenario. Nothing in this book checks that proof.

Statistics and physics. Nothing here says how many rounds make a tally informative, and nothing says the counters came from honest play; a program can INC whatever counts it likes.

F.9 Prior work

Tsirelson's 1980 paper gives the bound $2\sqrt{2}$ for quantum correlations, and his theorem says that quantum two-outcome correlators are inner products of real unit vectors. Landau (1988) and Masanes (2003, arXiv quant-ph/0309137) each give a condition that, by their proofs, characterizes the quantum correlators exactly for two settings and two outcomes. Wehner (2006, arXiv quant-ph/0510076) gets Tsirelson bounds, CHSH among them, from a semidefinite program on the 4×4 Gram matrix of those vectors. Navascués, Pironio and Acín (2007, arXiv quant-ph/0607119; 2008, arXiv 0803.4290) built the hierarchy of moment-matrix tests; their 2008 paper applies a Schur complement to the level-1 matrix of this scenario, recovers the arcsine condition, and discusses the 1+AB level. I went back to each of those papers for these points, and that is what I found in them. The results are theirs. Two of them are proved again here in Coq, in finite dimension: Tsirelson's representation (TsirelsonRepresentation.v) and the arcsine characterization (ArcsineBoundary.v); the rest of their proofs are not redone. Formal proofs of Tsirelson's bound already exist, by Echenim and Mhalla in Isabelle/HOL (arXiv 2306.12535) and in Lean's mathlib (tsirelson_inequality in Mathlib.Algebra.Star.CHSH), and exact rational sum-of-squares certificates go back at least to Peyrl and Parrilo (2008). So the moment-matrix route to $2\sqrt{2}$ is standard. What is specific here is narrower: an integer certificate computed from outcome counts, and a mechanized chain from "the counts pass" to PSD of the pinned matrix and on to $S^2 \leq 8$. I did not find that chain elsewhere, and I make no claim beyond it. On the host machine, the same chain is earned: a raised flag from a clean start stands on a passing check of the packed counts, and the check passes exactly when the pinned matrix is positive semidefinite and every pair was sampled (Section 20, Theorems 20.4 and 20.5). The converse direction of the integer check, integers from PSD, is checked there too, and there too I make no claim beyond the chain.

G

Constructing the quoting operation

The proof in Section 13 uses a program that takes the code of a term and returns the code of that code. Here is its construction. Nothing in the operation lets a program inspect itself by magic: its input is an ordinary tree encoded as a closed lambda term.

The construction follows Forster and Smolka (2017), whose Coq development of L has the same three syntax builders on Scott-encoded children, an auxiliary procedure for numerals, and a quote procedure Q with the same specification, defined by the same recursion on codes; the self-application that uses Q in Section 13 is their fixed-point lemma, after Barendregt's (1984) proof for the full lambda calculus. What follows is my rebuilding of it, with every step checked on the standard library.

G.1 Numerals and syntax as data

For readability, the formulas below give names to bound variables. Renaming a bound variable changes nothing; the underlying syntax uses the de Bruijn indices of Definition 13.3. Applications associate to the left. Write $X \Downarrow Y$ here for a finite sequence of the weak call-by-value steps ending in the value Y.

Scott numerals are the following closed values:

$$\overline{0} = \lambda z.\lambda s.z, \qquad \overline{n+1} = \lambda z.\lambda s.s\,\overline{n}.$$

Thus a numeral selects either the supplied zero case or the supplied successor case, passing its predecessor to the latter. The cases below are values, so choosing one does not prematurely run the other.

The three syntax constructors, operating on already encoded children, are

$$\begin{aligned} \mathrm{mkvar} &= \lambda n.\lambda v.\lambda a.\lambda l.v\,n, \\ \mathrm{mkapp} &= \lambda p.\lambda q.\lambda v.\lambda a.\lambda l.a\,p\,q, \\ \mathrm{mklam} &= \lambda p.\lambda v.\lambda a.\lambda l.l\,p. \end{aligned}$$

Each builder is a term of the language L of Definition 13.3. For a variable of index n, an application and an abstraction, respectively, define their codes by the following

resulting values:

$$\ulcorner \#n \urcorner = \lambda v.\lambda a.\lambda l.v\,\overline{n},$$
$$\ulcorner p\,q \urcorner = \lambda v.\lambda a.\lambda l.a\ulcorner p\urcorner\ulcorner q\urcorner,$$
$$\ulcorner \lambda p \urcorner = \lambda v.\lambda a.\lambda l.l\ulcorner p\urcorner.$$

In particular, mkvar $\overline{n}$, mkapp $\ulcorner p\urcorner\ulcorner q\urcorner$ and mklam $\ulcorner p\urcorner$ reduce to these three codes. A code is closed even when the term it describes has a free variable: the index of that variable is stored as a numeral. It is data inside the code, and the code itself has no free variable.

G.2 Recursion without running under a lambda

The usual self-application trick has to delay its recursive call in a call-by-value language. Use

$$W = \lambda x.\lambda f.\lambda v.\ f\,(\lambda a.x\,x\,f\,a)\,v, \qquad \mathrm{rec}(F) = \lambda v.W\,W\,F\,v.$$

When F and u are closed values, beta reduction gives

$$\mathrm{rec}(F)\,u \longrightarrow W\,W\,F\,u \longrightarrow^{*} F\,(\lambda a.W\,W\,F\,a)\,u = F\,(\mathrm{rec}(F))\,u.$$

The recursive argument is a lambda, hence already a value. It starts work only when F applies it to an argument. That is the whole reason this construction survives the call-by-value rule.

G.3 Quoting numerals

Let $E_j = \ulcorner \#j \urcorner$ for $j = 0, 1, 2$, and let $Z = \ulcorner \overline{0} \urcorner$, obtained directly from the finite syntax of $\overline{0} = \lambda\lambda.1$. Write $\mathrm{mklam}^k(X)$ for k nested applications of the syntax builder mklam to X; it is an abbreviation for constructing k lambda nodes, and the language gains no new operation from it.

Define

$$F_N = \lambda r.\lambda n.\ n\,Z\,(\lambda k.\mathrm{mklam}^2(\mathrm{mkapp}\,E_0\,(r\,k))), \qquad Q_N = \mathrm{rec}(F_N).$$

Then $Q_N\,\overline{n} \Downarrow \ulcorner \overline{n} \urcorner$, by induction on n. At zero the Scott numeral selects Z. At a successor it passes $\overline{n}$ to the second case; the induction hypothesis supplies the code of $\overline{n}$, and the builders produce the code of $\lambda\lambda.(\#0\,\overline{n})$, which is exactly the syntax of $\overline{n+1}$.

G.4 Quoting terms

For a proposed recursive call r, define three handlers:

$$\begin{aligned} H_V &= \lambda n.\mathrm{mklam}^3(\mathrm{mkapp}\, E_2\, (Q_N\, n)),\\ H_A(r) &= \lambda p.\lambda q.\mathrm{mklam}^3(\mathrm{mkapp}\, (\mathrm{mkapp}\, E_1\, (r\, p))\, (r\, q)),\\ H_L(r) &= \lambda p.\mathrm{mklam}^3(\mathrm{mkapp}\, E_0\, (r\, p)). \end{aligned}$$

Now set

$$F_Q = \lambda r.\lambda t.t\, H_V\, (H_A(r))\, (H_L(r)), \qquad Q = \mathrm{rec}(F_Q).$$

The handlers are lambdas and so are values. The Scott code of the input chooses one of them and hands it the encoded children. Induction on the represented term proves $Q\ulcorner t\urcorner \Downarrow \ulcorner\ulcorner t\urcorner\urcorner$:

- For $t = \#n$, the first handler constructs three lambda nodes over an application of variable 2 to the quoted numeral. That is the syntax of $\lambda\lambda\lambda.(\#2\,\overline{n})$, the code of $\#n$.
- For $t = p\,q$, the induction hypotheses quote both child codes. The second handler constructs three lambdas over variable 1 applied to those two codes. That is the syntax of the application code.
- For $t = \lambda p$, the last handler constructs three lambdas over variable 0 applied to the quoted child code. That is the syntax of the abstraction code.

Each recursive call works on a proper child of the finite input tree, or on a smaller numeral. Each builder takes finitely many beta steps. The induction therefore establishes termination as well as the value returned.

The mkapp used in Section 13 is this builder. With Q and that builder written out, its self-referential term has every piece on the page. The companion names are `mk_app_enc`, `rec_spec`, `Qn_spec` and `Q_spec` (LRecursion.v); the construction above is the argument those names identify.

Sources and mathematical dependencies

These works identify the principal mathematical dependencies and the intellectual background named in the book. The definitions, statements and worked examples remain on its pages. A published result about thermodynamic cost, quantum correlations or universal computation is not an additional theorem proved by this manuscript.

The universal programs and the compiler that puts every computably presented Thiele machine on the universal host use effective computability and counter-machine simulation; their construction is given in Section 16.13 and Section 16.14, and the counter and lambda references identify the established machinery under them. The thermodynamic references do not remove the explicit physical premise of the heat bound. The environmental-record papers motivate Section 15; they do not prove this book's pointer thesis.

H.1 Structure, information and knowledge

Edwin A. Abbott (1884). Flatland: A Romance of Many Dimensions, by A Square. Seeley and Co., London.

Rudolf Ahlswede and Gunter Dueck (1989). Identification via channels. IEEE Transactions on Information Theory 35(1), pp. 15–29. DOI: 10.1109/18.42172.

Hans Albert (1968). Traktat über kritische Vernunft. J. C. B. Mohr (Paul Siebeck), Tübingen. English translation: Treatise on Critical Reason, Princeton University Press, 1985. The Münchhausen trilemma.

Hans Albert. Wikipedia. Article. German philosopher (1921–2023), with Popper a leading critical rationalist; Traktat über kritische Vernunft (1968).

American Institute of Physics. Labor status of women computers. In Harvard College Observatory history pages. AIP page. Hourly pay of the women computers, and of the men, nearly five times as much.

American Standards Association (1963). American Standard Code for Information Interchange, ASA X3.4-1963.

Aristotle. Physics II.1; Metaphysics VII.3. Matter, form and substance. Also Metaphysics I.5 (985b–986b), on the Pythagoreans: "they supposed the elements of numbers to be the elements of all things" (trans. W. D. Ross, MIT Classics text); and Prior Analytics I.23 (41a23–30), the proof by contradiction that if the diagonal were commensurable with the side, odd numbers would equal even ones.

Sergei Artemov and Roman Kuznets (2014). Logical omniscience as infeasibility. Annals of Pure and Applied Logic 165(1), pp. 6–25.

Sergei N. Artemov (1995). Operational Modal Logic. Technical Report MSI 95-29, Cornell University. The Logic of Proofs, the first justification logic, surveyed in the Stanford Encyclopedia of Philosophy entry on justification logic. Entry.

Sergei N. Artemov (2001). Explicit provability and constructive semantics. Bulletin of Symbolic Logic 7(1), pp. 1–36. Explicit evidence ("this is a proof") against implicit provability ("there is a proof"), and why reflection is harmless for the first.

Mark van Atten (2018). The Creating Subject, the Brouwer–Kripke Schema, and infinite proofs. Indagationes Mathematicae 29(6), pp. 1565–1636. DOI: 10.1016/j.indag.2018.06.005; arXiv: 1805.00404. The reading of the creating subject is in Section 4.2, as read in the arXiv preprint; I didn't see the journal page.

Robert J. Aumann (1976). Agreeing to disagree. The Annals of Statistics 4(6). DOI: 10.1214/aos/1176343654. The partition model of knowledge, Section 7.

Steve Awodey (1996). Structure in mathematics and logic: a categorical perspective. Philosophia Mathematica 4(3), pp. 209–237. DOI: 10.1093/philmat/4.3.209.

A. J. Ayer (1956). The Problem of Knowledge. Macmillan, London, p. 34. The other definition Gettier targets.

Alexandru Baltag, Bryan Renne and Sonja Smets (2014). The logic of justified belief, explicit knowledge, and conclusive evidence. Annals of Pure and Applied Logic 165(1), pp. 49–81. DOI: 10.1016/j.apal.2013.07.005.

Thomas F. Banchoff (n.d.). Edwin Abbott Abbott (biographical sketch). PDF.

Michael J. Barany (2019). Making a name in mid-century mathematics: individuals, institutions and the open secret of Nicolas Bourbaki. Talk, History and Philosophy of Science, University of Cambridge, 17 October 2019. Abstract. Bourbaki's 1948 application to the American Mathematical Society and the second one a year later.

Pietro Baroni and Massimiliano Giacomin (2007). On principle-based evaluation of extension-based argumentation semantics. Artificial Intelligence 171(10–15), pp. 675–700. DOI: 10.1016/j.artint.2007.04.004.

Paul Benacerraf (1965). What numbers could not be. The Philosophical Review 74(1), pp. 47–73. DOI: 10.2307/2183530. Structuralism, Section 2.

Charles H. Bennett (1988). Logical Depth and Physical Complexity. In Rolf Herken (ed.), The Universal Turing Machine: A Half-Century Survey, Oxford University Press, pp. 227–257.

Max Black (1952). The Identity of Indiscernibles. Mind 61(242), pp. 153–164.

Max Black. Wikipedia. Article. The Russian-born British-American philosopher, at Cornell from 1946, and his 1952 paper on two spheres.

Roberto Bonola (1912). Non-Euclidean Geometry: A Critical and Historical Study of Its Development. Open Court; Dover reprint, 1955. The attempts on the parallel postulate from Proclus through the Islamic geometers and Saccheri, and Farkas Bolyai's letter to his son.

Boston University (2019). Leonid Levin, BU computer scientist, elected to National Academy of Sciences. Article. Kolmogorov as his advisor, his leaving the Soviet Union, and Boston University from 1980.

Nicolas Bourbaki (1950). The Architecture of Mathematics. American Mathematical Monthly 57(4), pp. 221–232. The mother structures. "Whose nature has not been specified" I have secondhand, from J. L. Bell's "Reflections on Bourbaki's notion of structure," which reproduces the passage without a page.

Robert B. Brandom (1994). Making It Explicit: Reasoning, Representing, and Discursive Commitment. Cambridge, MA: Harvard University Press.

Robert Brandom. Course pages, University of Pittsburgh. Home page.

Tyler Burge (1993). Content preservation. The Philosophical Review 102(4), pp. 457–488. DOI: 10.2307/2185680. The acceptance principle.

Walter Burkert (1972). Lore and Science in Ancient Pythagoreanism. Translated by Edwin L. Minar Jr. Harvard University Press. Cited here through Huffman's entry.

Martin Caminada (2014). Strong admissibility revisited. In Simon Parsons, Nir Oren, Chris Reed and Federico Cerutti (eds.), Computational Models of Argument: Proceedings of COMMA 2014, Frontiers in Artificial Intelligence and Applications

266, IOS Press, pp. 197–208. DOI: 10.3233/978-1-61499-436-7-197. Journal version with Paul E. Dunne (2019), Argument & Computation 10(3), pp. 277–300, DOI: 10.3233/AAC-190463.

Gregory J. Chaitin (1966). On the Length of Programs for Computing Finite Binary Sequences. Journal of the ACM 13(4), pp. 547–569.

G. K. Chesterton (1929). The Thing. The fence passage is quoted in the American Chesterton Society's Lecture 57: The Thing.

Roderick M. Chisholm (1957). Perceiving: A Philosophical Study. Cornell University Press, p. 16. One of the two definitions Gettier targets.

R. Cockett, I. J. Geng, C. M. Scandolo and P. V. Srinivasan (2023). Extending resource monotones using Kan extensions. In Proceedings of the Fifth International Conference on Applied Category Theory (ACT 2022), Electronic Proceedings in Theoretical Computer Science 380, pp. 203–223. arXiv:2206.09784.

Leo Corry (1992). Nicolas Bourbaki and the Concept of Mathematical Structure. Synthese 92(3), pp. 315–348. The sentence quoted, "plays no actual role in the presentation of theories within the treatise," I read in Chapter 7 of his book Modern Algebra and the Rise of Mathematical Structures (Birkhäuser), p. 338 of the chapter as posted on his website.

Claude Crépeau, Rafael Dowsley and Anderson C. A. Nascimento (2020). On the commitment capacity of unfair noisy channels. IEEE Transactions on Information Theory. arXiv: 1905.10921.

Ho Ngoc Duc (1997). Reasoning about rational, but not logically omniscient, agents. Journal of Logic and Computation 7(5), pp. 633–648. DOI: 10.1093/logcom/7.5.633.

Karl Duncker (1935; English translation 1945). Zur Psychologie des produktiven Denkens. Julius Springer, Berlin. English translation by Lynne S. Lees: On Problem-Solving, Psychological Monographs 58(5), whole no. 270, American Psychological Association, Washington, 1945. The candle problem.

Phan Minh Dung (1995). On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. Artificial Intelligence 77(2), pp. 321–357. DOI: 10.1016/0004-3702(94)00041-X.

Julien Dutant (2015). The Legend of the Justified True Belief Analysis. Philosophical Perspectives 29(1), pp. 95–145. DOI: 10.1111/phpe.12061. Known

here from its abstract and summaries. Dutant's page at King's College London: kcl.ac.uk/people/julien-dutant.

Kenny Easwaran (2009). Probabilistic proofs and transferability. Philosophia Mathematica (III) 17(3), pp. 341–362. DOI: 10.1093/philmat/nkn032.

Samuel Eilenberg and Saunders Mac Lane (1945). General Theory of Natural Equivalences. Transactions of the American Mathematical Society 58(2), pp. 231–294. Original paper.

Euclid. Elements, Books I and X. Definitions, postulates, common notions and commensurability. Quoted in T. L. Heath's translation (Definition 1, Postulate 1, Common Notion 1).

Ronald Fagin, Joseph Y. Halpern, Yoram Moses and Moshe Y. Vardi (1995). Reasoning About Knowledge. Cambridge, MA: MIT Press.

Dean P. Foster and Robert A. Stine (2008). α-investing: a procedure for sequential control of expected false discoveries. Journal of the Royal Statistical Society, Series B 70(2), pp. 429–444. DOI: 10.1111/j.1467-9868.2007.00643.x.

Gottlob Frege (1884). Die Grundlagen der Arithmetik, section 64. Directions and parallel lines.

Gottlob Frege (1903). Grundgesetze der Arithmetik, vol. II. Hermann Pohle, Jena. The appendix answering Russell's paradox.

Milton Friedman (1975). There's No Such Thing as a Free Lunch: Essays on Public Policy. Open Court, LaSalle, Illinois.

The Galileo Project. Galileo Galilei. Rice University. Biography.

Peter Gärdenfors (1986). Belief revisions and the Ramsey test for conditionals. The Philosophical Review 95(1), pp. 81–93. The impossibility theorem.

Aurélien Garivier, Tor Lattimore and Emilie Kaufmann (2016). On explore-then-commit strategies. Advances in Neural Information Processing Systems 29 (NeurIPS 2016), pp. 784–792. arXiv: 1605.08988.

Murray Gell-Mann (1964). A schematic model of baryons and mesons. Physics Letters 8(3), pp. 214–215. The paper that named the quark. DOI: 10.1016/S0031-9163(64)92001-3.

Edmund L. Gettier (1963). Is Justified True Belief Knowledge? Analysis 23(6), pp. 121–123. JSTOR: 3326923. The remark on Plato is in footnote 1, p. 121.

Sam Glucksberg (1962). The influence of strength of drive on functional fixedness and perceptual recognition. Journal of Experimental Psychology 63(1), pp. 36–41. DOI: 10.1037/h0044683.

Joseph A. Goguen and José Meseguer (1982). Security policies and security models. Proceedings of the 1982 IEEE Symposium on Security and Privacy, pp. 11–20. DOI: 10.1109/SP.1982.10014. Noninterference.

Sanford C. Goldberg (2010). Relying on Others: An Essay in Epistemology. Oxford University Press.

Alvin I. Goldman (1976). Discrimination and Perceptual Knowledge. Journal of Philosophy 73(20), pp. 771–791. The fake-barn case, credited there to Carl Ginet.

Oded Goldreich, Brendan Juba and Madhu Sudan (2012). A theory of goal-oriented communication. Journal of the ACM 59(2), article 8.

I. J. Good (1979). Studies in the History of Probability and Statistics XXXVII: A. M. Turing's statistical work in World War II. Biometrika 66(2), pp. 393–396. JSTOR: 2335677.

Gösta Grahne (1998). Updates and counterfactuals. Journal of Logic and Computation 8(1), pp. 87–117. DOI: 10.1093/logcom/8.1.87. Revision of a static world versus update of a changing one.

Peter D. Grünwald (2005). A Tutorial Introduction to the Minimum Description Length Principle. In Peter D. Grünwald, In Jae Myung and Mark A. Pitt (eds.), Advances in Minimum Description Length: Theory and Applications, MIT Press. arXiv: math/0406077.

Paul R. Halmos (1960). Naive Set Theory. D. Van Nostrand. Section 7, Relations: equivalence relations and partitions.

C. L. Hamblin (1970). Fallacies. Methuen, London. Chapter 8, formal dialectic, where each speaker has a commitment store.

G. H. Hardy (1940). A Mathematician's Apology. Cambridge University Press. The essay as a defence of mathematics pursued for its own sake; summary at Wikipedia, A Mathematician's Apology.

Ralph V. L. Hartley (1928). Transmission of Information. Bell System Technical Journal 7(3), pp. 535–563. DOI: 10.1002/j.1538-7305.1928.tb01236.x.

Hartley (unit). IEC 80000-13:2008, Quantities and units—Part 13: Information science and technology, items 13-24 and 13-24.b, pp. 18–19: the hartley (Hart) is the

base-ten information unit. IEC publication record; the table is available in the distributor's standard preview. This cites the 2008 edition, superseded in 2025. The alternative names ban and dit, and naming after Ralph Hartley: Wikipedia, Hartley (unit).

Chris Heegard (1985). On the capacity of permanent memory. IEEE Transactions on Information Theory 31(1), pp. 34–42. DOI: 10.1109/TIT.1985.1056994.

Geoffrey Hellman (1989). Mathematics without Numbers: Towards a Modal-Structural Interpretation. Oxford: Clarendon Press. DOI: 10.1093/0198240341.001.0001.

Martin E. Hellman and Thomas M. Cover (1970). Learning with finite memory. The Annals of Mathematical Statistics 41(3), pp. 765–782. DOI: 10.1214/aoms/1177696958.

Jaakko Hintikka (1962). Knowledge and Belief: An Introduction to the Logic of the Two Notions. Cornell University Press. Logical omniscience is surveyed in the Stanford Encyclopedia of Philosophy entry on epistemic logic. Entry.

Wesley H. Holliday (2009). Dynamic testimonial logic. In Xiangdong He, John Horty and Eric Pacuit (eds.), Logic, Rationality, and Interaction, LORI 2009, Lecture Notes in Computer Science 5834, Springer, pp. 161–179. DOI: 10.1007/978-3-642-04893-7_13. Authority graphs for trust in testimony, with complete reduction axioms.

Carl Huffman (2006, revised 2024). Pythagoreanism. Stanford Encyclopedia of Philosophy. Entry. The source for the remark on Hippasus.

David Hume (1748). An Enquiry Concerning Human Understanding, Section X, "Of Miracles," paragraph 4: "A wise man, therefore, proportions his belief to the evidence." Hume Texts Online, E 10.4. Hume as a figure of the Scottish Enlightenment: Stanford Encyclopedia of Philosophy, "David Hume," Entry.

Iamblichus (c. AD 300). On the Pythagorean Life. Translated by Thomas Taylor as Iamblichus' Life of Pythagoras (1818), chapters XVIII and XXXIV. Project Gutenberg 63300.

Jonathan Jenkins Ichikawa and Matthias Steup (2001, substantive revision 2026). The Analysis of Knowledge. Stanford Encyclopedia of Philosophy. Entry. Dharmottara's mirage and Peter of Mantua's running Plato.

Christoph Jäger and Nicholas Shackel (2025). Testimonial authority and knowledge transmission. Social Epistemology 39(6), pp. 636–651. DOI: 10.1080/02691728.2025.2449608.

Hirofumi Katsuno and Alberto O. Mendelzon (1991). On the difference between updating a knowledge base and revising it. Proceedings of the Second International Conference on Principles of Knowledge Representation and Reasoning (KR'91), pp. 387–394; reprinted in Peter Gärdenfors (ed.), Belief Revision, Cambridge University Press (1992), pp. 183–203. DOI: 10.1017/CBO9780511526664.007.

Arthur M. Keller and Marianne Winslett Wilkins (1985). On the use of an extended relational model to handle changing incomplete information. IEEE Transactions on Software Engineering 11(7), pp. 620–633.

Felix Klein (1872). Vergleichende Betrachtungen über neuere geometrische Forschungen (the Erlangen program). Erlangen: A. Deichert.

Wolfgang Köhler (1917). Intelligenzprüfungen an Anthropoiden. Abhandlungen der Königlich Preussischen Akademie der Wissenschaften, Physikalisch-mathematische Klasse, 1917. English translation by Ella Winter, The Mentality of Apes, London: Kegan Paul, Trench, Trubner, 1925.

Andrei N. Kolmogorov (1965). Three approaches to the quantitative definition of information. Problems of Information Transmission 1(1), pp. 1–7; reprinted in International Journal of Computer Mathematics 2 (1968), pp. 157–168. DOI of the reprint: 10.1080/00207166808803030.

Lewis A. Kornhauser. Faculty page, New York University School of Law. Page; Lawrence G. Sager, faculty publications, University of Texas School of Law, Unpacking the Court (1986).

Lewis A. Kornhauser and Lawrence G. Sager (1986). Unpacking the court. Yale Law Journal 96(1), beginning p. 82. DOI: 10.2307/796436.

Oliver Kosut and Jörg Kliewer (2018). Authentication capacity of adversarial channels. 2018 IEEE Information Theory Workshop (ITW), Guangzhou, pp. 1–5. Authentication capacity is zero or the capacity without an adversary, according to overwritability.

Erik C. W. Krabbe (2001). The problem of retraction in critical discussion. Synthese 127(1–2), pp. 141–159. DOI: 10.1023/A:1010318403544.

Georg Kreisel (1967). Informal rigour and completeness proofs. In Imre Lakatos (ed.), Problems in the Philosophy of Mathematics, North-Holland, Amsterdam, pp. 138–186. DOI: 10.1016/S0049-237X(08)71525-8. The creating subject's evidence at a stage.

Saul Kripke (1975). Outline of a theory of truth. Journal of Philosophy 72(19), pp. 690–716. Grounded and ungrounded sentences.

Ralf Krömer (2006). La “machine de Grothendieck” se fonde-t-elle seulement sur des vocables métamathématiques? Bourbaki et les catégories au cours des années cinquante. Revue d’histoire des mathématiques 12(1), pp. 119–162. Numdam. Why categories never entered the Éléments.

Jennifer Lackey (2008). Learning from Words: Testimony as a Source of Knowledge. Oxford University Press. The case of the teacher who doesn’t believe what she teaches.

James Ladyman (1998). What is structural realism? Studies in History and Philosophy of Science Part A 29(3), pp. 409–424. DOI: 10.1016/S0039-3681(98)80129-5.

Gottfried Wilhelm Leibniz (1679). Letter to Christiaan Huygens, 8 September 1679, with the enclosed essay on the analysis of situation. In C. I. Gerhardt (ed.), Mathematische Schriften, vol. 2, Halle, 1850; the essay is on pp. 20–27. Huygens replied on 22 November 1679. Discussion with the dated correspondence.

Gottfried Wilhelm Leibniz (1716). Fourth paper to Samuel Clarke, 2 June 1716, section 4. In A Collection of Papers which passed between the late learned Mr. Leibnitz and Dr. Clarke, trans. Samuel Clarke, James Knapton, London, 1717. Text.

Isaac Levi (1980). The Enterprise of Knowledge. MIT Press. Where routine and deliberate expansion first appear.

Isaac Levi (1991). The Fixation of Belief and Its Undoing: Changing Beliefs through Inquiry. Cambridge University Press. Chapter 3, routine and deliberate expansion. DOI: 10.1017/CBO9780511663819.

Leonid A. Levin (1984). Randomness Conservation Inequalities; Information and Independence in Mathematical Theories. Information and Control 61(1), pp. 15–37. The line about the uninformed witness is Levin’s, quoted as the epigraph of Luis Antunes, Lance Fortnow, Dieter van Melkebeek and N. V. Vinodchandran, Computational Depth: Concept and Applications, Theoretical Computer Science 354(3), 2006, pp. 391–404. Paper.

Dennis V. Lindley (1956). On a Measure of the Information Provided by an Experiment. Annals of Mathematical Statistics 27(4), pp. 986–1005. DOI: 10.1214/aoms/1177728069.

Christian List and Philip Pettit (2002). Aggregating sets of judgments: an impossibility result. Economics and Philosophy 18(1), pp. 89–110. DOI: 10.1017/S0266267102001098.

John Locke (1690). An Essay Concerning Human Understanding, Book I, chapter iv, section 23. The attack on taking belief from authority. Modernized text of Book I.

MacTutor History of Mathematics. Arend Heyting. University of St Andrews. Biography. Doctorate in 1925 under Brouwer.

MacTutor History of Mathematics. Emmy Amalie Noether. University of St Andrews. Biography. Habilitation closed to women; courses advertised under Hilbert's name; habilitation in 1919.

Petrus Mantuanus (d. 1400). Logica. Medieval manuscripts in Oxford libraries, Bodleian Library. Catalogue entry. Peter of Mantua as a logician of the late fourteenth century.

Per Martin-Löf (1996). On the Meanings of the Logical Constants and the Justifications of the Logical Laws. Nordic Journal of Philosophical Logic 1(1), pp. 11–60. Lectures given in Siena in 1983. The passage on proof as an act is on p. 28.

Robert K. Merton (1957). Priorities in Scientific Discovery: A Chapter in the Sociology of Science. American Sociological Review 22(6), pp. 635–659.

Meta Platforms (2021). Introducing Meta: A Social Technology Company. Company announcement, 28 October 2021. Announcement.

Janet Metcalfe (1986). Premonitions of insight predict impending error. Journal of Experimental Psychology: Learning, Memory, and Cognition 12(4), pp. 623–634. DOI: 10.1037/0278-7393.12.4.623. High warmth before an insight answer goes with the answer being wrong.

Janet Metcalfe and David Wiebe (1987). Intuition in insight and noninsight problem solving. Memory and Cognition 15(3), pp. 238–246. The warmth ratings.

Benjamin Morison (2014, revised 2026). Sextus Empiricus. Stanford Encyclopedia of Philosophy. Entry. The source for his dates, his medicine and his skepticism.

Emmy Noether (1918). Invariante Variationsprobleme. Nachrichten von der Gesellschaft der Wissenschaften zu Göttingen, Mathematisch-Physikalische Klasse, pp. 235–257. Original paper, EuDML 59024.

Robert Nozick (1981). Philosophical Explanations. Harvard University Press. Chapter 3, the sensitivity condition on knowledge.

J. J. O'Connor and E. F. Robertson (n.d.). Theaetetus of Athens. MacTutor History of Mathematics. Entry.

John J. O'Connor and Edmund F. Robertson. Paul Richard Halmos. MacTutor History of Mathematics. Biography.

James R. O'Shea (2010). Review of Bernhard Weiss and Jeremy Wanderer (eds.), Reading Brandom: On Making It Explicit. Notre Dame Philosophical Reviews, 13 December 2010. Review. Brandom's deontic scorekeeping of commitments and entitlements, with default, inferential and testimonial (deferential) authority.

David L. Parnas (1972). On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM 15(12), pp. 1053–1058. DOI: 10.1145/361598.361623.

Charles S. Peirce (1877). The Fixation of Belief. Popular Science Monthly 12, pp. 1–15. Quoted: p. 6.

Plato. Meno 97d–98a: Daedalus's statues, and true opinions tied down by reasoning; trans. W. R. M. Lamb, Plato in Twelve Volumes, vol. 3, Harvard University Press, 1967. Republic III, 403e: Glaucon on a guardian needing a guardian; trans. Paul Shorey, Plato in Twelve Volumes, vols. 5–6, Harvard University Press, 1969, text. Republic, Book VII, 514a–517c, translated by Benjamin Jowett: the cave; the prisoners' honours for whoever is quickest at the passing shadows and best at guessing what comes next (516c–d); Socrates' own reading of the cave as the world of sight (517b); MIT Internet Classics Archive. Timaeus; Theaetetus: the receptacle and the discussion of knowledge as true judgement with an account. Philosophical antecedents. None of the formal results takes them as a premise.

Karl R. Popper (1959). The Logic of Scientific Discovery. Hutchinson. Translation of Logik der Forschung (1934). Section 30: basic statements, the jury, and the piles driven into a swamp.

Henry Prakken (2005). Coherence and flexibility in dialogue games for argumentation. Journal of Logic and Computation 15(6), pp. 1009–1040. DOI: 10.1093/logcom/exi046.

Michael D. Resnik (1997). Mathematics as a Science of Patterns. Oxford: Clarendon Press. DOI: 10.1093/0198250142.001.0001.

Jorma Rissanen (1978). Modeling by shortest data description. Automatica 14(5), pp. 465–471. DOI: 10.1016/0005-1098(78)90005-5.

Benoît Rittaud and Albrecht Heeffer (2014). The Pigeonhole Principle, Two Centuries before Dirichlet. The Mathematical Intelligencer 36(2), pp. 27–29. Jean Leurechon's use of the principle in 1622.

Bertrand Russell (1902). Letter to Gottlob Frege, 16 June 1902, and Frege's reply. In Jean van Heijenoort (ed.), From Frege to Gödel: A Source Book in Mathematical Logic, 1879–1931. Harvard University Press, 1967, pp. 124–125 and 127–128. Stanford Encyclopedia of Philosophy, Russell's paradox. Russell as a British philosopher and logician: Stanford Encyclopedia of Philosophy, "Bertrand Russell," Entry.

Bertrand Russell (1948). Human Knowledge: Its Scope and Limits. George Allen and Unwin, London. The stopped clock: a true belief about the time from a clock that has stopped, which is not knowledge. Quoted in the Stanford Encyclopedia of Philosophy entry "The Analysis of Knowledge" (which misdates the book 1923). SEP entry.

Girolamo Saccheri (1733). Euclides ab omni naevo vindicatus. Milan.

Krister Segerberg (1998). Irrevocable Belief Revision in Dynamic Doxastic Logic. Notre Dame Journal of Formal Logic 39(3). DOI: 10.1305/ndjfl/1039182247.

Sextus Empiricus. Outlines of Pyrrhonism I.164–177. The five modes attributed to Agrippa.

Claude E. Shannon (1948). A Mathematical Theory of Communication. Bell System Technical Journal 27, pp. 379–423 and 623–656. Bell Labs publication record. Quoted: "irrelevant to the engineering problem," p. 379; "allows one to make a saving in time," p. 385; half of English "determined by the structure of the language," p. 399.

Claude E. Shannon, interviewed by Robert Price (1982). Oral history, 28 July 1982, IEEE History Center. Transcript.

Stewart Shapiro (1997). Philosophy of Mathematics: Structure and Ontology. New York: Oxford University Press.

Ray J. Solomonoff (1964). A Formal Theory of Inductive Inference, Parts I and II. Information and Control 7(1), pp. 1–22, and 7(2), pp. 224–254.

Sophia of Hanover. Wikipedia. Article. Her friendship with Leibniz from 1676 to 1714 and her part in laying out the Herrenhausen Gardens.

Stanford Encyclopedia of Philosophy. Gottlob Frege. Entry. German logician and mathematician at Jena.

Stephen M. Stigler (1980). Stigler's Law of Eponymy. Transactions of the New York Academy of Sciences 39(1), pp. 147–157. DOI: 10.1111/j.2164-0947.1980.tb02775.x.

Göran Sundholm (1998). Proofs as Acts and Proofs as Objects: Some questions for Dag Prawitz. Theoria 64(2–3), pp. 187–216. DOI: 10.1111/j.1755-2567.1998.tb00187.x.

Göran Sundholm (2014). Constructive Recursive Functions, Church's Thesis, and Brouwer's Theory of the Creating Subject: Afterthoughts on a Parisian Joint Session. In Constructivity and Computability in Historical and Philosophical Perspective, Logic, Epistemology, and the Unity of Science, Springer, pp. 1–35. DOI: 10.1007/978-94-017-9217-2_1.

Alfred Tarski (1954). Contributions to the theory of models, I and II. Indagationes Mathematicae 16, pp. 572–581 and 582–588. The first part proposes the name theory of models.

Alfred Tarski (1986). What are logical notions? Edited by John Corcoran. History and Philosophy of Logic 7(2), pp. 143–154. DOI: 10.1080/01445348608837096. From a lecture given at Bedford College, University of London, on 16 May 1966; Klein's Erlangen program is its starting point.

Neil Tennant (2008). Belief-revision, the Ramsey test, monotonicity, and the so-called impossibility results. The Review of Symbolic Logic 1(4), pp. 402–423. DOI: 10.1017/S1755020308090023.

Anvith Thudi, Hengrui Jia, Ilia Shumailov and Nicolas Papernot (2022). On the Necessity of Auditable Algorithmic Definitions for Machine Unlearning. Proceedings of the 31st USENIX Security Symposium. arXiv: 2110.11891.

Myron Tribus and Edward C. McIrvine (1971). Energy and Information. Scientific American 225(3), pp. 179–188.

Trinity College, Cambridge. Godfrey Harold Hardy. Page. Hardy as Sadleirian Professor at Cambridge, and the 1940 essay.

Anne S. Troelstra (1969). Principles of Intuitionism. Lecture Notes in Mathematics 95, Springer, Berlin. DOI: 10.1007/BFb0080643. The creating subject who never forgets.

Nikolai K. Vereshchagin and Paul M. B. Vitányi (2004). Kolmogorov's Structure Functions and Model Selection. IEEE Transactions on Information Theory 50(12), pp. 3265–3290. arXiv: cs/0204037; DOI: 10.1109/TIT.2004.838346. "Meaningless information, random noise" is in Section II.D, as read in the arXiv preprint; I didn't see the journal page.

Heinrich Weber (1893). Leopold Kronecker. Jahresbericht der Deutschen Mathematiker-Vereinigung 2, pp. 5–23. The remark attributed to a lecture of 1886 is on p. 19.

Robin Wilson (2002). Four Colours Suffice? Gresham College lecture, 21 October 2002. Lecture. De Morgan's letter of 23 October 1852 to the Irish mathematician William Rowan Hamilton.

Andreas Winter, Anderson C. A. Nascimento and Hideki Imai (2003). Commitment capacity of discrete memoryless channels. Cryptography and Coding 2003, LNCS 2898, pp. 35–51. DOI: 10.1007/978-3-540-40974-8_4; arXiv: cs/0304014.

Timothy Yenter and Ezio Vailati (2003, revised 2024). Samuel Clarke. Stanford Encyclopedia of Philosophy. Entry. Clarke as the British philosopher and theologian who defended Newton's natural philosophy in his letters with Leibniz.

Linda Zagzebski (1994). The inescapability of Gettier problems. The Philosophical Quarterly 44(174), pp. 65–73. DOI: 10.2307/2220147.

Ernst Zermelo (1908). Untersuchungen über die Grundlagen der Mengenlehre I. Mathematische Annalen 65(2), pp. 261–281. DOI: 10.1007/BF01449999.

H.2 Cost, records and thermodynamics

János Aczél, Bruno Forte and C. T. Ng (1974). Why the Shannon and Hartley entropies are "natural." Advances in Applied Probability 6(1), pp. 131–146. DOI: 10.2307/1426210. Ng's 1971 Waterloo doctorate under Aczél: Mathematics Genealogy Project.

Roberto M. Amadio, Nicolas Ayache, François Bobot, Jaap P. Boender, Brian Campbell, Ilias Garnier, Antoine Madet, James McKinna, Dominic P. Mulligan, Mauro Piccolo, Randy Pollack, Yann Régis-Gianas, Claudio Sacerdoti Coen, Ian Stark and Paolo Tranquilli (2014). Certified Complexity (CerCo). Foundational and Practical Aspects of Resource Analysis, FOPARA 2013, LNCS 8552, pp. 1–18. DOI: 10.1007/978-3-319-12466-7_1.

American Physical Society (2006). May, 1911: Rutherford and the Discovery of the Atomic Nucleus. APS News 15(5), This Month in Physics History. Article.

Kenneth Appel and Wolfgang Haken (1976). Every planar map is four colorable. Bulletin of the American Mathematical Society 82(5), pp. 711–712.

Will Arthur and David Challener, with Kenneth Goldman (2015). A Practical Guide to TPM 2.0: Using the Trusted Platform Module in the New Age of Security.

Apress. Chapter 11, NV indexes (bit-field indexes and counters); Chapter 12, Platform Configuration Registers (read, extend and reset; an extend can't be undone), DOI: 10.1007/978-1-4302-6584-9_12.

J. L. Austin (1962). How to Do Things with Words. Clarendon Press, Oxford. The William James Lectures, Harvard, 1955.

Adam Back (2002). Hashcash: A Denial of Service Counter-Measure. Technical report, 1 August 2002; the scheme was first proposed in 1997. Report.

John C. Baez, Tobias Fritz and Tom Leinster (2011). A characterization of entropy in terms of information loss. Entropy 13(11), pp. 1945–1957. DOI: 10.3390/e13111945; arXiv: 1106.1791.

Mihir Bellare and Bennet S. Yee (1997). Forward integrity for secure audit logs. Technical Report CS98-580, Department of Computer Science and Engineering, University of California, San Diego.

Charles H. Bennett (1973). Logical Reversibility of Computation. IBM Journal of Research and Development 17(6), pp. 525–532. Original paper.

Charles H. Bennett (1982). The thermodynamics of computation: a review. International Journal of Theoretical Physics 21, pp. 905–940. DOI: 10.1007/BF02084158.

Charles H. Bennett (2003). Notes on Landauer's principle, reversible computation, and Maxwell's Demon. Studies in History and Philosophy of Modern Physics 34(3), pp. 501–510. arXiv: physics/0210005v2; DOI: 10.1016/S1355-2198(03)00039-X. Quoted from the arXiv preprint: "the erasure of a bit or the merging of two computation paths" (abstract) and "in my opinion the one of greatest merit" (p. 1); "the most important logically reversible operations" and "erasure of one of two copies of data known to be identical" (p. 2). I didn't see the journal pages.

Antoine Bérut, Artak Arakelyan, Artyom Petrosyan, Sergio Ciliberto, Raoul Dillenschneider and Eric Lutz (2012). Experimental verification of Landauer's principle linking information and thermodynamics. Nature 483, pp. 187–189. DOI: 10.1038/nature10872.

Laurent Bindschaedler, Quentin Botha and Christoph Siebenbrunner (2026). Agent Flight Recorder: Tamper-Evident Audit Trails with On-Chain Anchoring for Long-Horizon Tool-Using Agents. Preprint. arXiv: 2609.01931.

Matt Bishop and Michael Dilger (1996). Checking for race conditions in file accesses. Computing Systems 9(2), pp. 131–152. PDF.

Thomas Blanchard (2024). Causation and the Time-Asymmetry of Knowledge. Australasian Journal of Philosophy 102(4), pp. 959–977. DOI: 10.1080/00048402.2024.2328629.

P. Bocchieri and A. Loinger (1957). Quantum Recurrence Theorem. Physical Review 107, 337. DOI: 10.1103/PhysRev.107.337.

Ludwig Boltzmann (1896). Entgegnung auf die wärmetheoretischen Betrachtungen des Hrn. E. Zermelo. Annalen der Physik und Chemie, Neue Folge 57, pp. 773–784.

Walter J. Boyne (2013). The Checklist. Air Force Magazine, August 2013, pp. 52–56. The Model 299 crash of 30 October 1935: the locked controls, the earlier written routines from 1911 on, and the judgement that "it took the Model 299 crash to institutionalize its use." Article.

Walter J. Boyne. Wikipedia entry: United States Air Force officer, aviation historian, director of the National Air and Space Museum (acting from 1982, director 1983–1986). Entry.

Léon Brillouin (1951). Maxwell's Demon Cannot Operate: Information and Entropy. I. Journal of Applied Physics 22(3), pp. 334–337. DOI: 10.1063/1.1699951.

Jeffrey Bub (2001). Maxwell's Demon and the Thermodynamics of Computation. Studies in History and Philosophy of Modern Physics 32(4), pp. 569–579. Szilard and Brillouin on the cost of acquiring information, at least $k \log 2$ per bit.

Eric Budish (2025). Trust at Scale: The Economic Limits of Cryptocurrencies and Blockchains. The Quarterly Journal of Economics 140(1), pp. 1–62. DOI: 10.1093/qje/qjae033.

Jesse Bull and Joel Watson (2007). Hard evidence and mechanism design. Games and Economic Behavior 58(1), pp. 75–93. DOI: 10.1016/j.geb.2006.03.003.

Vitalik Buterin (2016). Transaction spam attack: next steps. Ethereum Foundation Blog, 22 September 2016. Post.

Vitalik Buterin and Martin Swende (2020). EIP-2929: Gas cost increases for state access opcodes. Ethereum Improvement Proposals. eips.ethereum.org/EIPS/eip-2929.

Vitalik Buterin and Martin Swende (2021). EIP-3529: Reduction in refunds. Ethereum Improvement Proposals. eips.ethereum.org/EIPS/eip-3529.

Vitalik Buterin and Virgil Griffith (2017). Casper the Friendly Finality Gadget. arXiv: 1710.09437v1.

Vitalik Buterin, Diego Hernandez, Thor Kamphefner, Khiem Pham, Zhi Qiao, Danny Ryan, Juhyeok Sin, Ying Wang and Yan X. Zhang (2020). Combining GHOST and Casper. arXiv: 2003.03052.

Arthur Charguéraud and François Pottier (2019). Verifying the Correctness and Amortized Complexity of a Union-Find Implementation in Separation Logic with Time Credits. Journal of Automated Reasoning 62, pp. 331–365; first published online in 2017. DOI: 10.1007/s10817-017-9431-7.

Wen Tsing Chow and William H. Henrich (1962). Storage matrix. US patent 3,028,659, filed 27 December 1957, granted 10 April 1962. Patent text. The invention in 1956 by Chow, of the Arma Division of the American Bosch Arma Corporation in Garden City, New York, at the request of the United States Air Force, to store the targeting constants of the Atlas E/F's airborne digital computer: Wikipedia, Programmable ROM, History section, entry.

Byung-Gon Chun, Petros Maniatis, Scott Shenker and John Kubiatowicz (2007). Attested append-only memory: making adversaries stick to their word. Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07), pp. 189–204. DOI: 10.1145/1294261.1294280.

Michael T. Clanchy (1979). From Memory to Written Record: England 1066–1307. London: Edward Arnold; third edition, Wiley-Blackwell, 2012.

Allen Clement, Flavio Junqueira, Aniket Kate and Rodrigo Rodrigues (2012). On the (limited) power of non-equivocation. Proceedings of the 2012 ACM Symposium on Principles of Distributed Computing (PODC '12), pp. 301–308. DOI: 10.1145/2332432.2332490.

Bob Coecke, Tobias Fritz and Robert W. Spekkens (2016). A mathematical theory of resources. Information and Computation 250, pp. 59–86. DOI: 10.1016/j.ic.2016.02.008.

Scott A. Crosby and Dan S. Wallach (2009). Efficient data structures for tamper-evident logging. Proceedings of the 18th USENIX Security Symposium, Montreal. Conference page.

Nicholas of Cusa (1433). De concordantia catholica. Where he calls the Donation of Constantine a forgery.

Norman Danner and Daniel R. Licata (2022). Denotational semantics as a foundation for cost recurrence extraction for functional languages. Journal of Functional Programming 32. DOI: 10.1017/S095679682200003X.

Norman Danner, Daniel R. Licata and Ramyaa Ramyaa (2015). Denotational cost semantics for functional languages with inductive types. Proceedings of ICFP 2015, pp. 140–151. DOI: 10.1145/2784731.2784749.

Joe DeBlasio (2025). Discussion: removal of one-6962-log requirement. Chromium ct-policy mailing list, 17 November 2025. Post.

Jérémie Decouchant, David Kozhaya, Vincent Rahli and Jiangshan Yu (2022). Damysus: streamlined BFT consensus leveraging trusted components. Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22), pp. 1–16. DOI: 10.1145/3492321.3519568.

Deuteronomy 19:15. Two or three witnesses for a conviction. NRSV text.

David Deutsch and Chiara Marletto (2015). Constructor theory of information. Proceedings of the Royal Society A 471(2174), 20140540. DOI: 10.1098/rspa.2014.0540; arXiv: 1405.5563. Section 8.6, the irreducible perturbation caused by measurement.

David Deutsch and Chiara Marletto (2025). Constructor theory of time. Preprint. arXiv: 2505.08692.

Cynthia Dwork and Moni Naor (1992). Pricing via Processing or Combatting Junk Mail. Advances in Cryptology, CRYPTO '92, LNCS 740 (Springer, 1993), pp. 139–147. DOI: 10.1007/3-540-48071-4_10.

John Earman and John D. Norton (1998, 1999). Exorcist XIV: The Wrath of Maxwell's Demon. Part I: From Maxwell to Szilard, Studies in History and Philosophy of Modern Physics 29, pp. 435–471; Part II: From Szilard to Landauer and Beyond, 30, pp. 1–40. Part I; Part II.

Aram Ebtekar and Marcus Hutter (2024). Modeling the Arrows of Time with Causal Multibaker Maps. Entropy 26(9), article 776. DOI: 10.3390/e26090776.

Arthur S. Eddington (1928). The Nature of the Physical World. Cambridge University Press. Chapter IV, The Running-Down of the Universe. Quotation with chapter.

Encyclopaedia Britannica. Donation of Constantine. Web page. Composed in the eighth century; the consensus view puts it in the 750s or 760s.

Encyclopaedia Britannica. Leo Szilard. Web page. Born in Budapest in 1898; Berlin doctorate 1922 and the Institute of Theoretical Physics there.

Massimiliano Esposito and Christian Van den Broeck (2010). Three detailed fluctuation theorems. Physical Review Letters 104, 090601. DOI: 10.1103/PhysRevLett.104.090601. The split of entropy production into adiabatic and nonadiabatic parts.

Ethereum Foundation (n.d.). The Merge. ethereum.org roadmap page, on the transition of 15 September 2022. Page.

Ethereum Foundation (n.d.). Timeline of all Ethereum forks (Tangerine Whistle, EIP-150, and the Berlin upgrade, block 12,244,000). ethereum.org. Page.

Ethereum Foundation. Devcon editions (Devcon 0, Berlin 2014; Devcon 1, London 2015; Devcon 2, Shanghai 2016). devcon.org. Page.

Ethereum Foundation. Ethereum execution specifications, Prague fork, src/ethereum/forks/prague/vm/instructions/storage.py and vm/gas.py, at commit ecb68f21a7d28db101c769878184f6facb319da1. Repository.

Paul N. Fahn (1996). Maxwell's demon and the entropy cost of information. Foundations of Physics 26, pp. 71–93. DOI: 10.1007/BF02058888.

Philippe Faist, Frédéric Dupuis, Jonathan Oppenheim and Renato Renner (2015). The minimal work cost of information processing. Nature Communications 6, article 7669. arXiv: 1211.1037.

Federal Aviation Administration (2008). Revisions to Cockpit Voice Recorder and Digital Flight Data Recorder Regulations. Final rule, Federal Register 73(46), 7 March 2008, pp. 12541–12573. The two-hour cockpit voice recorder.

Federal Aviation Administration (2026). 25-Hour Cockpit Voice Recorder (CVR) Requirement, New Aircraft Production. Final rule, Federal Register 91(21), 2 February 2026. Federal Register text. The thirty-minute recorders of 1966, the two-hour rule, and the 25-hour standard of ICAO (2016) and EASA (from 2021).

FIDE (2023). FIDE Laws of Chess taking effect from 1 January 2023, Article 9.2.3. FIDE Handbook.

Richard FitzNigel (c. 1180). Dialogus de Scaccario (The Dialogue of the Exchequer). Text and translation ed. Charles Johnson, Nelson's Medieval Classics, 1950. The widths of the cuts on Exchequer tallies.

Robert W. Floyd (1967). Assigning Meanings to Programs. In J. T. Schwartz (ed.), Mathematical Aspects of Computer Science, Proceedings of Symposia in Applied Mathematics 19, American Mathematical Society, pp. 19–32.

Forrester (n.d.). Books as Evidence, before and after Pacioli (1494 AD). Paper on the University of Strathclyde accounting history pages; the page gives no initials or date. Mirror of the paper.

Fox-IT (2012). Black Tulip: Report of the investigation into the DigiNotar Certificate Authority breach. Version 1.0. Report.

Edward Fredkin and Tommaso Toffoli (1982). Conservative logic. International Journal of Theoretical Physics 21(3–4), pp. 219–253. DOI: 10.1007/BF01857727.

Tobias Fritz (2017). Resource convertibility and ordered commutative monoids. Mathematical Structures in Computer Science 27(6), pp. 850–938. arXiv:1504.03661.

Tom Furtwangler (2017). The vaccine vial monitor: the world's smartest sticker. PATH, 6 November 2017. Article. The stickers first sold in 1996, for oral polio vaccine.

Gaius. Institutes, book 2, sections 104, 119 and 147 (second century). The will by bronze and balance before five witnesses, a scale-holder and the buyer of the estate (104); a will sealed by seven witnesses (119, 147). Text and translation.

Carl Friedrich Gauss (1831). Untersuchungen über die Eigenschaften der positiven ternären quadratischen Formen von Ludwig August Seeber. Göttingische gelehrte Anzeigen, 9 July 1831; reprinted in Journal für die reine und angewandte Mathematik 20 (1840), pp. 312–320.

Atul Gawande (2007). The Checklist. The New Yorker, 10 December 2007. The earliest place the line "too much airplane for one man to fly" turns up that I could find; it names no newspaper. Article.

General Conference on Weights and Measures (2018). Resolution 1 of the 26th meeting: On the revision of the International System of Units (SI), effective 20 May 2019.

Sebastian Goldt and Udo Seifert (2017). Stochastic thermodynamics of learning. Physical Review Letters 118, 010601. DOI: 10.1103/PhysRevLett.118.010601.

Tomáš Gonda and Robert W. Spekkens (2023). Monotones in general resource theories. Compositionality 5, article 7. DOI: 10.32408/compositionality-5-7.

Michael J. C. Gordon (2000). From LCF to HOL: a short history. In Gordon Plotkin, Colin Stirling and Mads Tofte (eds.), Proof, Language, and Interaction: Essays in Honour of Robin Milner, MIT Press, pp. 169–185. Paper.

Jim Gray (1981). The Transaction Concept: Virtues and Limitations. Proceedings of the 7th International Conference on Very Large Data Bases, pp. 144–154. Paper.

Jerry R. Green and Jean-Jacques Laffont (1986). Partially verifiable information and mechanism design. Review of Economic Studies 53(3), pp. 447–456.

Stuart Haber and W. Scott Stornetta (1991). How to Time-Stamp a Digital Document. Journal of Cryptology 3(2), pp. 99–111.

Andreas Haeberlen and Petr Kuznetsov (2009). The fault detection problem. Principles of Distributed Systems, OPODIS 2009, Lecture Notes in Computer Science, Springer, pp. 99–114. DOI: 10.1007/978-3-642-10877-8_10. Faults divided into classes; only omission and commission faults are detectable.

Andreas Haeberlen, Petr Kouznetsov and Peter Druschel (2007). PeerReview: practical accountability for distributed systems. Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07), pp. 175–188. DOI: 10.1145/1294261.1294279.

Thomas C. Hales (2005). A proof of the Kepler conjecture. Annals of Mathematics 162(3), pp. 1065–1185. DOI: 10.4007/annals.2005.162.1065.

Thomas C. Hales (2008). Formal proof. Notices of the American Mathematical Society 55(11), pp. 1370–1379.

Thomas Hales (2024). The Formal Proof of the Kepler Conjecture: a critical retrospective. arXiv: 2402.08032. "Growing out of my frustration" is on p. 2.

Thomas Hales and others (2017). A formal proof of the Kepler conjecture. Forum of Mathematics, Pi 5, e2. DOI: 10.1017/fmp.2017.1.

Douglas Harper (n.d.). Online Etymology Dictionary, entry "ledger". Entry.

Percy John Heawood (1890). Map-colour theorem. Quarterly Journal of Pure and Applied Mathematics 24, pp. 332–338. I haven't seen the original; "the now apparently recognised proof" is as quoted in secondary accounts, one of which spells it "recognized."

Joseph M. Hellerstein and Peter Alvaro (2020). Keeping CALM: When Distributed Consistency Is Easy. Communications of the ACM 63(9), pp. 72–81. DOI: 10.1145/3369736.

Maurice Herlihy (1991). Wait-free synchronization. ACM Transactions on Programming Languages and Systems 13(1), pp. 124–149. DOI: 10.1145/114005.102808.

David Hilbert (1900). Mathematische Probleme. Nachrichten von der Königlichen Gesellschaft der Wissenschaften zu Göttingen, pp. 253–297. English translation by Mary Winston Newson, Mathematical Problems, Bulletin of the American Mathematical Society 8 (1902), pp. 437–479.

Yoichi Hirai (2017). CasperOneMessage.thy: accountable safety of one-message Casper, in Isabelle/HOL. Repository.

Anson Ho, Ege Erdil and Tamay Besiroglu (2023). Limits to the Energy Efficiency of CMOS Microprocessors. arXiv: 2312.08595. Table I: switching a CMOS gate, 100 aJ to 100 fJ.

C. A. R. Hoare (1969). An Axiomatic Basis for Computer Programming. Communications of the ACM 12(10), pp. 576–580. DOI: 10.1145/363235.363259.

Geoffrey M. Hodgson (2025). The Secret History of the Tally Stick. Web page. Henry I and tallies from about 1100; the stock and the stockholder.

Jan Hoffmann, Klaus Aehlig and Martin Hofmann (2011). Multivariate Amortized Resource Analysis. Proceedings of POPL 2011, pp. 357–370. DOI: 10.1145/1926385.1926427. Journal version: ACM Transactions on Programming Languages and Systems 34(3), 2012, article 14, DOI: 10.1145/2362389.2362393.

Martin Hofmann and Steffen Jost (2003). Static prediction of heap space usage for first-order functional programs. Proceedings of the 30th ACM Symposium on Principles of Programming Languages (POPL 2003), pp. 185–197. DOI: 10.1145/604131.604148. The start of type-based amortized analysis.

Horace (c. 20 BC). Epistles, book 1, epistle 1, line 14. The source of *Nullius in verba*.

Radha Jagadeesan, Alan Jeffrey, Corin Pitcher and James Riely (2009). Towards a theory of accountability and audit. Computer Security, ESORICS 2009, Lecture Notes in Computer Science 5789, Springer, pp. 152–167. DOI: 10.1007/978-3-642-04444-1_10.

Edwin T. Jaynes (1965). Gibbs vs Boltzmann entropies. American Journal of Physics 33(5), pp. 391–398. DOI: 10.1119/1.1971557. "An anthropomorphic concept" is in Section VI, p. 398, and the credit to Wigner in the acknowledgments on the same page.

Edwin T. Jaynes (1989). Clearing up Mysteries: The Original Goal. In John Skilling (ed.), Maximum Entropy and Bayesian Methods, Kluwer, pp. 1–27. DOI: 10.1007/978-94-015-7860-8_1. Where the mind projection fallacy is named.

C. H. Jenkinson (1910). Early wooden tallies relating to Surrey. Surrey Archaeological Collections 23, pp. 203–207. PDF. Hazel; the payer kept the stock, the Exchequer the foil; abolished by statute in 1826; the burning that started the fire.

E. Jensen, G. Hagensen and J. Broughton (1987). A New Approach to Exclusive Data Access in Shared Memory Multiprocessors. Technical Report UCRL-97663, Lawrence Livermore National Laboratory, November 1987. The load-linked and store-conditional pair.

Joint Committee for Guides in Metrology (2012). International vocabulary of metrology: basic and general concepts and associated terms (VIM), 3rd edition, JCGM 200:2012, entry 2.41, "metrological traceability." The definition is on p. 29; "does not ensure … that there is an absence of mistakes" is Note 5 to 2.41, p. 30.

Ruth E. Kastner (2025). Maxwell's demon is foiled by the entropy cost of measurement, not erasure. Foundations 5(2), article 16. DOI: 10.3390/foundations5020016; arXiv: 2503.18186.

Shin-ya Katsumata (2014). Parametric effect monads and semantics of effect systems. Proceedings of POPL 2014, pp. 633–645. DOI: 10.1145/2535838.2535846.

Alfred Bray Kempe (1879). On the Geographical Problem of the Four Colours. American Journal of Mathematics 2(3), pp. 193–200. DOI: 10.2307/2369235. Quoted: p. 194.

Johannes Kepler (1611). Strena seu de Nive Sexangula. Frankfurt am Main: Godefrid Tampach. English: The Six-Cornered Snowflake, translated by Colin Hardie, with essays by L. L. Whyte and B. J. Mason, Oxford: Clarendon Press, 1966. I haven't seen the Hardie translation; the words quoted from it are as given in secondary accounts.

Narayana R. Kocherlakota (1998). Money Is Memory. Journal of Economic Theory 81(2), pp. 232–251. DOI: 10.1006/jeth.1997.2357.

Artemy Kolchinsky and David H. Wolpert (2017). Dependence of dissipation on the initial distribution over states. Journal of Statistical Mechanics: Theory and Experiment 2017, 083202. DOI: 10.1088/1742-5468/aa7ee1.

H. T. Kung and John T. Robinson (1981). On optimistic methods for concurrency control. ACM Transactions on Database Systems 6(2), pp. 213–226.

Robert Künnemann, Ilkan Esiyok and Michael Backes (2019). Automated verification of accountability in security protocols. 2019 IEEE 32nd Computer Security Foundations Symposium (CSF), IEEE. DOI: 10.1109/CSF.2019.00034; arXiv: 1805.10891.

Ralf Küsters, Tomasz Truderung and Andreas Vogt (2010). Accountability: definition and relationship to verifiability. Proceedings of the 17th ACM Conference on Computer and Communications Security (CCS '10), pp. 526–535. DOI: 10.1145/1866307.1866366.

Mikhail Kuter, Marina Gurskaya and Albert Gurskii (2025). Genoa 1340, Double Entry, and 'Gold Coasting'. De Computis, Revista Española de Historia de la Contabilidad 22(1), pp. 1–33. DOI: 10.26784/issn.1886-1881.22.1.11927.

James Ladyman, Stuart Presnell, Anthony J. Short and Berry Groisman (2007). The connection between logical and thermodynamic irreversibility. Studies in History and Philosophy of Modern Physics 38(1), pp. 58–79. DOI: 10.1016/j.shpsb.2006.03.007.

Leslie Lamport (1978). Time, Clocks, and the Ordering of Events in a Distributed System. Communications of the ACM 21(7), pp. 558–565. DOI: 10.1145/359545.359563. "Implemented by counters with no actual timing mechanism" is on p. 559.

Leslie Lamport (1983). What Good Is Temporal Logic? In R. E. A. Mason (ed.), Information Processing 83, Elsevier, pp. 657–668. Author paper. The paper that introduces invariance under stuttering.

Leslie Lamport (n.d.). The Writings of Leslie Lamport, note on "Time, Clocks and the Ordering of Events in a Distributed System." Web page. The Johnson and Thomas note and the comments of Jim Gray.

Rolf Landauer (1961). Irreversibility and Heat Generation in the Computing Process. IBM Journal of Research and Development 5(3), pp. 183–191. DOI: 10.1147/rd.53.0183. Quoted: "requires a minimal heat generation," "typically of the order of kT," "serves the purpose of standardizing signals…" and "show, or at least very strongly suggest," all p. 183; "no guarantee that this minimum is in fact achievable," p. 187.

Ben Laurie, Adam Langley and Emilia Kasper (2013). Certificate Transparency. RFC 6962. RFC 6962; DOI: 10.17487/RFC6962.

Ben Laurie, Eran Messeri and Rob Stradling (2021). Certificate Transparency Version 2.0. RFC 9162. RFC 9162; DOI: 10.17487/RFC9162.

Yves Lecerf (1963). Machines de Turing réversibles. Récursive insolubilité en $n \in \mathbb{N}$ de l'équation $u = \theta^n u$, où θ est un "isomorphisme de codes." Comptes Rendus de l'Académie des Sciences de Paris 257, pp. 2597–2600.

Geoffrey A. Lee (1977). The Coming of Age of Double Entry: The Giovanni Farolfi Ledger of 1299–1300. Accounting Historians Journal 4(2), pp. 79–96. DOI: 10.2308/0148-4184.4.2.79.

Tom Leinster. Wikipedia. Article. A British mathematician known for his work on category theory, at Glasgow and then Edinburgh. John C. Baez, Wikipedia, Article: mathematical physicist at the University of California, Riverside, working on higher categories and applied category theory. Tobias Fritz's category-theoretic approach to probability: arXiv listing.

Dave Levin, John R. Douceur, Jacob R. Lorch and Thomas Moscibroda (2009). TrInc: small trusted hardware for large distributed systems. Proceedings of the 6th USENIX Symposium on Networked Systems Design and Implementation (NSDI '09), pp. 1–14.

A. Liapounoff (1907). Problème général de la stabilité du mouvement, traduit du russe par Édouard Davaux. Annales de la Faculté des Sciences de Toulouse, 2e série, 9, pp. 203–474. Numdam. The French translation the 1992 English one was made from.

Elliott H. Lieb and Jakob Yngvason (1999). The physics and mathematics of the second law of thermodynamics. Physics Reports 310(1), pp. 1–96. DOI: 10.1016/S0370-1573(98)00082-9.

Jacques-Louis Lions (chair) et al. (1996). ARIANE 5 Flight 501 Failure: Report by the Inquiry Board. Paris, 19 July 1996. Copy of the report.

Josef Loschmidt (1876). Über den Zustand des Wärmegleichgewichtes eines Systems von Körpern mit Rücksicht auf die Schwerkraft. Sitzungsberichte der Kaiserlichen Akademie der Wissenschaften in Wien, Mathematisch-Naturwissenschaftliche Classe 73, Abt. II, pp. 128–142.

Aleksandr M. Lyapunov (1892). The general problem of the stability of motion. Doctoral thesis, Kharkov Mathematical Society (in Russian). English translation by A. T. Fuller, International Journal of Control 55(3) (1992), pp. 531–773.

Henry Sumner Maine (1861). Ancient Law. John Murray, London, chapter VI. The seven seals of the praetor's will as the seven people of the older mock sale. Project Gutenberg 22910.

Gonzalo Manzano, Gülce Kardeş, Édgar Roldán and David Wolpert (2024). Thermodynamics of Computations with Absolute Irreversibility, Unidirectional Transitions, and Stochastic Computation Times. Physical Review X 14, 021026. DOI: 10.1103/PhysRevX.14.021026.

Chiara Marletto (2016). Constructor theory of thermodynamics. Preprint, first posted in 2016. arXiv: 1608.02625.

Chiara Marletto, Vlatko Vedral, Laura T. Knoll, Fabrizio Piacentini, Ettore Bernardi, Enrico Rebufello, Alessio Avella, Marco Gramegna, Ivo Pietro Degiovanni and Marco Genovese (2022). Emergence of constructor-based irreversibility in quantum systems: theory and experiment. Physical Review Letters 128, 080401. DOI: 10.1103/PhysRevLett.128.080401; arXiv: 2009.14649.

O. J. E. Maroney (2009). Generalizing Landauer's Principle. Physical Review E 79, 031105. arXiv: quant-ph/0702094v3; DOI: 10.1103/PhysRevE.79.031105.

Owen Maroney (2009). Information Processing and Thermodynamic Entropy. Stanford Encyclopedia of Philosophy, first published 15 September 2009. Entry. It ends without a verdict on Landauer's principle.

Sinisa Matetic, Mansoor Ahmed, Kari Kostiainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels and Srdjan Capkun (2017). ROTE: rollback protection for trusted execution. Proceedings of the 26th USENIX Security Symposium, pp. 1289–1306. USENIX page.

James Clerk Maxwell (1865). A Dynamical Theory of the Electromagnetic Field. Philosophical Transactions of the Royal Society of London 155, pp. 459–512. DOI: 10.1098/rstl.1865.0008.

James Clerk Maxwell (1867, 1871). Letter to Peter Guthrie Tait, 11 December 1867, printed in Cargill Gilston Knott, Life and Scientific Work of Peter Guthrie Tait, Cambridge University Press, 1911, pp. 213–214 ("very observant and neat-fingered being," p. 214), with the undated note "Concerning Demons" on pp. 214–215 ("Call him no more a demon but a valve," p. 215); and Theory of Heat, London: Longmans, Green, 1871, the section on the limitation of the second law of thermodynamics. Text of the letter.

Ralph C. Merkle (1979). Secrecy, Authentication, and Public Key Systems. PhD thesis, Stanford University. Thesis. Where tree authentication, now called the Merkle tree, is introduced.

Clyde Lee Miller (2009; revised 2021). Cusanus, Nicolaus [Nicolas of Cusa]. Stanford Encyclopedia of Philosophy. SEP entry.

Alfred Mitchell-Innes (1913). What is Money? The Banking Law Journal, May 1913, pp. 377–408. The exchequer tally and "raising a tally."

Kenichi Morita (1996). Universality of a reversible two-counter machine. Theoretical Computer Science 168(2), pp. 303–320. DOI: 10.1016/S0304-3975(96)00081-3.

Mozilla. Certificate Transparency. MDN Web Docs. Page.

Yûto Murashita, Ken Funo and Masahito Ueda (2014). Nonequilibrium equalities in absolutely irreversible processes. Physical Review E 90, 042110. DOI: 10.1103/PhysRevE.90.042110. The bound on average entropy production by $-\ln(1-\lambda)$, with free expansion as the first example.

Satoshi Nakamoto (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. Paper.

Moni Naor and Guy N. Rothblum (2005). The complexity of online memory checking. Proceedings of the 46th IEEE Symposium on Foundations of Computer Science (FOCS 2005), pp. 573–584; journal version in Journal of the ACM 56(1), 2009. IACR ePrint 2006/091.

National Institute of Standards and Technology (2019). The International System of Units (SI). NIST Special Publication 330, 2019 edition. NIST page on the kelvin. The 2019 redefinition fixing the Boltzmann constant.

National Museum of Australia. Defining Moments: Black box flight recorder invented. Museum page. David Warren's flight memory recorder, Aeronautical Research Laboratories, Melbourne: the idea in 1953, the first prototype in 1956, the combined voice and data recorder in 1958.

Tobias Nipkow (2015). Amortized Complexity Verified. Interactive Theorem Proving, ITP 2015, LNCS 9236, pp. 310–324. Author paper. Extended treatment: Tobias Nipkow and Hauke Brinkop (2019), same title, Journal of Automated Reasoning 62(3), pp. 367–391. DOI: 10.1007/s10817-018-9459-3.

Yue Niu, Jonathan Sterling, Harrison Grodin and Robert Harper (2022). A Cost-Aware Logical Framework. Proceedings of the ACM on Programming Languages 6(POPL), article 9. DOI: 10.1145/3498670.

John D. Norton (2005). Eaters of the Lotus: Landauer's Principle and the Return of Maxwell's Demon. Studies in History and Philosophy of Modern Physics 36, pp. 375–411. Author's papers and discussion.

John D. Norton (2013). The end of the thermodynamics of computation: a no-go result. Philosophy of Science 80(5), pp. 1182–1192. DOI: 10.1086/673714.

John D. Norton (2018). Maxwell's demon does not compute. In M. E. Cuffaro and S. C. Fletcher (eds.), Physical Perspectives on Computation, Computational Perspectives on Physics, Cambridge University Press, pp. 240–256. PhilSci-Archive 14808. The demon exorcised with Liouville's theorem alone.

John D. Norton (2026). Too good to be true: entropy, information, computation, and the uninformed thermodynamics of erasure. Philosophy of Science, published online 22 May 2026; manuscript dated 2025. DOI: 10.1017/psa.2026.10221.

Devon O'Brien (2018). Certificate Transparency Enforcement in Google Chrome. Chromium ct-policy mailing list, 6 February 2018. Announcement.

J. J. O'Connor and E. F. Robertson (n.d.). The four colour theorem. MacTutor History of Mathematics. Entry. (De Morgan's letter to Hamilton of 23 October 1852 is quoted from here.)

Garson O'Toole (2016). There Ain't No Such Thing as a Free Lunch, TANSTAAFL. Quote Investigator, 27 August 2016. Article.

Daniel Oberhaus (2018). The World's Oldest Blockchain Has Been Hiding in the New York Times Since 1995. Vice (Motherboard), 27 August 2018. Article. Surety's weekly hash under "Notices & Lost and Found."

Dominic Orchard, Vilem Liepelt and Harley Eades III (2019). Quantitative program reasoning with graded modal types. Proceedings of the ACM on Programming Languages 3(ICFP), article 110. DOI: 10.1145/3341714.

Thomas E. Ouldridge and David H. Wolpert (2023). Thermodynamics of deterministic finite automata operating locally and periodically. New Journal of Physics 25, 123013. DOI: 10.1088/1367-2630/ad1070; arXiv: 2208.06895. The sixth main result: if a language's minimal automaton is non-invertible, every automaton for it is, and recognising it with a local, periodic process produces entropy; worked example, the strings with no b.

Thomas E. Ouldridge and Pieter Rein ten Wolde (2017). Fundamental Costs in the Production and Destruction of Persistent Polymer Copies. Physical Review Letters 118, 158103. DOI: 10.1103/PhysRevLett.118.158103.

Thomas E. Ouldridge, Christopher C. Govern and Pieter Rein ten Wolde (2017). Thermodynamics of Computational Copying in Biochemical Systems. Physical Review X 7, 021004. DOI: 10.1103/PhysRevX.7.021004.

Luca Pacioli (1494). Summa de arithmetica, geometria, proportioni et proportionalità. Venice. The bookkeeping treatise is Distinctio IX, Tractatus XI, Particularis de computis et scripturis.

Pavel Pakshin, Elena Panteley and Natalia Pakshina (2017). The Legacy of A. M. Lyapunov to Systems and Control Theory: 110 years of the Toulouse edition. IFAC World Congress 2017, open-track proposal. Davaux's translation, reviewed and corrected by Lyapunov.

Bryan Parno, Jacob R. Lorch, John R. Douceur, James Mickens and Jonathan M. McCune (2011). Memoir: practical state continuity for protected modules. Proceedings of the 2011 IEEE Symposium on Security and Privacy, pp. 379–394. DOI: 10.1109/SP.2011.38.

Juan M. R. Parrondo, Jordan M. Horowitz and Takahiro Sagawa (2015). Thermodynamics of information. Nature Physics 11(2), pp. 131–139. DOI: 10.1038/nphys3230.

Max Planck (1920). The genesis and present state of development of the quantum theory. Nobel Lecture, 2 June 1920. Quoted in the translation by H. T. Clarke and L. Silberstein, The Origin and Development of the Quantum Theory, Oxford: Clarendon Press, 1922, p. 11.

Henri Poincaré (1890). Sur le problème des trois corps et les équations de la dynamique. Acta Mathematica 13, pp. 1–270.

Luciano Pomatto, Philipp Strack and Omer Tamuz (2023). The cost of information: the case of constant marginal costs. American Economic Review 113(5), pp. 1360–1393. DOI: 10.1257/aer.20190185.

Jenny M. Poulton, Pieter Rein ten Wolde and Thomas E. Ouldridge (2019). Nonequilibrium correlations in minimal dynamical models of polymer copying. Proceedings of the National Academy of Sciences 116(6), p. 1946. arXiv: 1805.08502.

Peter E. Prosser (2001). Church History's Biggest Hoax. Christian History 72. Article. Cusa in 1433, before Valla.

Hans Reichenbach (1956). The Direction of Time. University of California Press; Dover reprint. Stanford Encyclopedia of Philosophy entry on Reichenbach. Cited by Rovelli (2024) as the first to notice that any measurement is dissipative.

Richard Rhodes (1986). The Making of the Atomic Bomb. Simon and Schuster, New York.

Paul M. Riechers, Alexander B. Boyd, Gregory W. Wimsatt and James P. Crutchfield (2020). Balancing error and dissipation in computing. Physical Review Research 2, 033524. DOI: 10.1103/PhysRevResearch.2.033524; arXiv: 1909.06650. Time-symmetric control and nonreciprocal transitions.

Lídia del Rio, Johan Åberg, Renato Renner, Oscar Dahlsten and Vlatko Vedral (2011). The thermodynamic meaning of negative entropy. Nature 474, pp. 61–63. DOI: 10.1038/nature10123; arXiv: 1009.1630.

James Ritty and John Ritty (1879). Cash Register and Indicator. US patent 221,360, 4 November 1879. See also Ritty's Incorruptible Cashier, American Table. The Dayton saloon, the 1878 steamboat trip and the propeller counter, the dial with no drawer, and the sale through Jacob H. Eckert to John H. Patterson in 1884: Wikipedia, James Ritty.

Ronald L. Rivest and Adi Shamir (1982). How to reuse a write-once memory. Information and Control 55, pp. 1–19. DOI: 10.1016/S0019-9958(82)90344-8.

Neil Robertson, Daniel Sanders, Paul Seymour and Robin Thomas (1997). The four-colour theorem. Journal of Combinatorial Theory, Series B 70(1), pp. 2–44.

Carlo Rovelli (2022). Memory and Entropy. Entropy 24(8), article 1022. DOI: 10.3390/e24081022; arXiv: 2003.06687.

Carlo Rovelli (2024). The Thermodynamic Cost of Choosing. Foundations of Physics 54, article 28. arXiv: 2309.08557; DOI: 10.1007/s10701-024-00765-7. "Stabilization must be irreversible" is on p. 3 of the arXiv preprint.

The Royal Society. History of the Royal Society. Web page. First meeting 28 November 1660; the motto adopted in the First Charter, 1662.

Runtime Verification, Inc. and The University of Texas at Austin (2018). casper-proofs: Coq proofs of Casper FFG accountable safety and plausible liveness, file Core/AccountableSafety.v at commit d8fe05df57e59909bf5c3392579e66ac1b78dab8. Repository.

Takahiro Sagawa (2014). Thermodynamic and Logical Reversibilities Revisited. Journal of Statistical Mechanics: Theory and Experiment, P03025. arXiv: 1311.1886v2.

Takahiro Sagawa and Masahito Ueda (2009). Minimal Energy Cost for Thermodynamic Information Processing: Measurement and Information Erasure. Physical Review Letters 102, 250602. arXiv: 0809.4098v3; DOI: 10.1103/PhysRevLett.102.250602. See also the erratum, Physical Review Letters 106, 189901 (2011).

Denise Schmandt-Besserat (1992). Before Writing, volume 1: From Counting to Cuneiform. University of Texas Press.

Denise Schmandt-Besserat. Tokens and Writing: the Cognitive Development. Author's page, University of Texas at Austin. Web page. Hollow clay envelopes with the tokens impressed on the outside, around 3300 BC.

Bruce Schneier and John Kelsey (1999). Secure audit logs to support computer forensics. ACM Transactions on Information and System Security 2(2), pp. 159–176. DOI: 10.1145/317087.317089.

Securities and Exchange Commission (2022). Electronic Recordkeeping Requirements for Broker-Dealers, Security-Based Swap Dealers, and Major Security-Based Swap Participants. Release No. 34-96034, 12 October 2022. Final rule. Adds an audit-trail alternative to the write-once requirement of Rule 17a-4(f) in place since 1997.

Alex Shamis and coauthors (2021). IA-CCF: individual accountability for permissioned ledgers. Preprint. arXiv: 2105.13116.

Steven Shapin (1984). Pump and Circumstance: Robert Boyle's Literary Technology. Social Studies of Science 14(4), pp. 481–520. DOI: 10.1177/030631284014004001.

Caroline Shenton (2012). The Day Parliament Burned Down. Oxford University Press. The tallies burned in the furnaces under the House of Lords, 16 October 1834.

James Six (1782). Account of an improved Thermometer. Philosophical Transactions of the Royal Society of London 72, pp. 72–81. DOI: 10.1098/rstl.1782.0012. Quoted: "if it be not immediately noticed, it can never after be known," p. 72.

Rob Smart (2021). Anti-rollback explained. PSA Certified, 27 April 2021. Article.

Michael Stock, Pauline Barat, Richard S. Davis, Alain Picard and Martin J. T. Milton (2015). Calibration campaign against the international prototype of the kilogram in anticipation of the redefinition of the kilogram part I: comparison of the international prototype with its official copies. Metrologia 52(2), pp. 310–316. DOI: 10.1088/0026-1394/52/2/310.

William E. Story (1879). Note on the Preceding Paper. American Journal of Mathematics 2(3), from p. 201. DOI: 10.2307/2369236. "Substantially proved" is on p. 201.

Raoul Strackx and Frank Piessens (2016). Ariadne: a minimal approach to state continuity. Proceedings of the 25th USENIX Security Symposium, pp. 875–892. USENIX page.

Supreme Court of the United States (2005). Arthur Andersen LLP v. United States, 544 U.S. 696. The conviction reversed over the jury instructions.

Leo Szilard (1929). Über die Entropieverminderung in einem thermodynamischen System bei Eingriffen intelligenter Wesen. Zeitschrift für Physik 53, pp. 840–856. DOI: 10.1007/BF01341281.

George G. Szpiro (2003). Does the proof stack up? Nature 424, pp. 12–13. The Annals editor's letter as Hales reported it, ending "run out of energy . . . ".

George G. Szpiro (2006). The Secret Life of Numbers. Washington, DC: Joseph Henry Press. Chapter 12, "Proving the Proof."

Omer Tamuz, Caltech, faculty page; Philipp Strack, Yale, faculty page; Luciano Pomatto, Caltech, HSS new faculty 2016–17. The economists behind the 2023 cost-of-information paper.

Wei Tang (2019). EIP-2200: Structured Definitions for Net Gas Metering. Ethereum Improvement Proposals. eips.ethereum.org/EIPS/eip-2200.

Robert Endre Tarjan (1985). Amortized Computational Complexity. SIAM Journal on Algebraic and Discrete Methods 6(2), pp. 306–318. DOI: 10.1137/0606031.

This Day in Aviation (n.d.). 30 October 1935: Boeing Model 299 crash at Wright Field. Page.

William Thomson (1874). Kinetic Theory of the Dissipation of Energy. Nature 9, pp. 441–444. Nature page.

Tommaso Toffoli (1980). Reversible computing. Automata, Languages and Programming (ICALP 1980), Lecture Notes in Computer Science, pp. 632–644. DOI: 10.1007/3-540-10003-2_104.

Alin Tomescu, Vivek Bhupatiraju, Dimitrios Papadopoulos, Charalampos Papamanthou, Nikos Triandopoulos and Srinivas Devadas (2019). Transparency logs via append-only authenticated dictionaries. Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19), pp. 1299–1316. DOI: 10.1145/3319535.3345652.

László Fejes Tóth (1953). Lagerungen in der Ebene, auf der Kugel und im Raum. Berlin: Springer.

László Fejes Tóth (1964). Regular Figures. Oxford: Pergamon Press.

Shoichi Toyabe, Takahiro Sagawa, Masahito Ueda, Eiro Muneyuki and Masaki Sano (2010). Experimental demonstration of information-to-energy conversion

and validation of the generalized Jarzynski equality. Nature Physics 6, pp. 988–992. DOI: 10.1038/nphys1821.

Timo Treitz and Robert Künnemann (2026). Accountability in Certificate Transparency and variants. Preprint. arXiv: 2609.11552.

Trusted Computing Group (n.d.). Trusted Platform Module Library Specification, Family 2.0, Part 1: Architecture. The platform configuration registers and the extend operation. Specification page.

Alan M. Turing (1949). Checking a large routine. In Report of a Conference on High Speed Automatic Calculating Machines, University Mathematical Laboratory, Cambridge, pp. 67–69. Turing Digital Archive. A corrected text was published by F. L. Morris and C. B. Jones, Annals of the History of Computing 6 (1984). "Definite assertions which can be checked individually" is in the opening paragraph; I read it in the reprint by F. L. Morris and C. B. Jones, Annals of the History of Computing 6(2), 1984, p. 139.

U.S. Department of Justice (2002). Press release, 15 June 2002, on the jury verdict against Arthur Andersen LLP. Press release.

U.S. Department of Justice (2002). Press release, 31 October 2002, on the indictment of Andrew Fastow; Enron's special purpose entities and off-balance-sheet treatment. Press release.

Jos Uffink (revised 2024). Boltzmann's Work in Statistical Physics. Stanford Encyclopedia of Philosophy. SEP entry. Zermelo as Planck's assistant in 1896; Boltzmann's reply "drenched in sarcasm and bitterness."

University of Waterloo, Department of Pure Mathematics (2020). We remember Distinguished Professor Emeritus János Aczél. Notice. Aczél at Waterloo from 1965, founder of Aequationes Mathematicae. Bruno Forte at Waterloo: Mathematics Genealogy Project.

Lorenzo Valla (c. 1440). De falso credita et ementita Constantini donatione declamatio. The Donation of Constantine. English translation: The Treatise of Lorenzo Valla on the Donation of Constantine, trans. Christopher B. Coleman, Yale University Press, 1922.

Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, Lau Cheuk Lung and Paulo Verissimo (2013). Efficient Byzantine fault-tolerance. IEEE Transactions on Computers 62(1), pp. 16–30. DOI: 10.1109/TC.2011.221.

Maria Violaris and Chiara Marletto (2022). Irreversibility beyond Landauer's principle: is it possible to perform erasure using the quantum homogenizer? New Journal of Physics 24, 113030. DOI: 10.1088/1367-2630/ac9e13.

Kent Weare (2016). Ethereum DOS Attack. InfoQ, 19 September 2016. Article.

David H. Wolpert (1992). Memory systems, computation, and the second law of thermodynamics. International Journal of Theoretical Physics 31(4), pp. 743–785. DOI: 10.1007/BF00670829.

David H. Wolpert (2019). The stochastic thermodynamics of computation. Journal of Physics A: Mathematical and Theoretical 52, 193001. DOI: 10.1088/1751-8121/ab0850; arXiv: 1905.05669. Review of distribution-dependent costs and restrictions imposed by computational architecture.

David H. Wolpert and Jens Kipper (2024). Memory Systems, the Epistemic Arrow of Time, and the Second Law. Entropy 26(2), article 170. DOI: 10.3390/e26020170.

David H. Wolpert and William G. Macready (1997). No free lunch theorems for optimization. IEEE Transactions on Evolutionary Computation 1(1), pp. 67–82. DOI: 10.1109/4235.585893.

Gavin Wood (2014). Ethereum: A Secure Decentralised Generalised Transaction Ledger (the yellow paper). Current version.

World Health Organization (2011). PQS performance specification: Vaccine Vial Monitor. WHO/PQS/E06/IN05.2.

Ernst Zermelo (1896). Ueber einen Satz der Dynamik und die mechanische Wärmetheorie. Annalen der Physik und Chemie, Neue Folge 57, pp. 485–494. A separate work from the 1908 set-theory paper listed under the first heading.

H.3 Computation, self-reference and checking

Martín Abadi and Leslie Lamport (1991). The Existence of Refinement Mappings. Theoretical Computer Science 82(2), pp. 253–284. Earlier versions appeared at LICS 1988 and as SRC Research Report 29. Author report; DOI: 10.1016/0304-3975(91)90224-P.

Keith Adams and Ole Agesen (2006). A Comparison of Software and Hardware Techniques for x86 Virtualization. Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS XII), pp. 2–13. DOI: 10.1145/1168857.1168860. Binary translation

of instructions such as POPF, which on x86 runs in user mode without trapping. Intel's VT-x and AMD-V in 2005 and 2006: Wikipedia, x86 virtualization.

Alok Aggarwal and Jeffrey Scott Vitter (1988). The input/output complexity of sorting and related problems. Communications of the ACM 31(9), pp. 1116–1127. DOI: 10.1145/48529.48535. The I/O model.

Danel Ahman, Cédric Fournet, Cătălin Hriţcu, Kenji Maillard, Aseem Rastogi and Nikhil Swamy (2018). Recalling a witness: foundations and applications of monotonic state. Proceedings of the ACM on Programming Languages 2(POPL), 65:1–65:30. arXiv: 1707.02466.

Arthur Azevedo de Amorim, Maxime Dénès, Nick Giannarakis, Cătălin Hriţcu, Benjamin C. Pierce, Antal Spector-Zabusky and Andrew Tolmach (2015). Micro-policies: formally verified, tag-based security monitors. Proceedings of the 2015 IEEE Symposium on Security and Privacy, pp. 813–830.

James P. Anderson (1972). Computer Security Technology Planning Study. Technical Report ESD-TR-73-51, Electronic Systems Division, Air Force Systems Command, Hanscom Field, Bedford, MA. Early computer security papers.

Andrew W. Appel (2001). Foundational proof-carrying code. Proceedings of LICS 2001, pp. 247–256. DOI: 10.1109/LICS.2001.932501.

François Bancilhon and Nicolas Spyratos (1981). Update semantics of relational views. ACM Transactions on Database Systems 6(4), pp. 557–575. DOI: 10.1145/319628.319634.

Henk Barendregt (1984). The Lambda Calculus: Its Syntax and Semantics. Revised edition. Studies in Logic and the Foundations of Mathematics 103. Amsterdam: North-Holland. The second fixed-point theorem.

Henk Barendregt. Wikipedia. Article. A Dutch logician known for his work in lambda calculus and type theory.

Henk Barendregt and Freek Wiedijk (2005). The challenge of computer mathematics. Philosophical Transactions of the Royal Society A 363(1835), pp. 2351–2375.

Francisco Bavera and Eduardo Bonelli (2018). Justification logic and audited computation. Journal of Logic and Computation 28(5), pp. 909–934; first published online in 2015.

Gerd Behrmann, Ansgar Fehnker, Thomas Hune, Kim Larsen, Paul Pettersson, Judi Romijn and Frits Vaandrager (2001). Minimum-cost reachability for priced

timed automata. Hybrid Systems: Computation and Control (HSCC 2001), Lecture Notes in Computer Science 2034, pp. 147–161.

Guy Blanc, Caleb Koch, Jane Lange and Li-Yang Tan (2022). The query complexity of certification. Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing (STOC 2022), pp. 623–636. DOI: 10.1145/3519935.3519993; arXiv: 2201.07736.

Manuel Blum (1967). A machine-independent theory of the complexity of recursive functions. Journal of the ACM 14(2), pp. 322–336. DOI: 10.1145/321386.321395. The two axioms for a complexity measure; cited beside the definition of a model of computation in the opening chapter.

Manuel Blum (1983). Coin Flipping by Telephone: A Protocol for Solving Impossible Problems. ACM SIGACT News 15(1), pp. 23–27. Presented at CRYPTO 1981.

Manuel Blum and Sampath Kannan (1989). Designing programs that check their work. Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC '89), pp. 86–97. DOI: 10.1145/73007.73015. Journal version: Journal of the ACM 42(1), 1995, pp. 269–291, DOI: 10.1145/200836.200880.

George Boole (1847). The Mathematical Analysis of Logic, Being an Essay towards a Calculus of Deductive Reasoning. Macmillan, Barclay and Macmillan, Cambridge.

George Boole (1854). An Investigation of the Laws of Thought. Walton and Maberly, London. Chapter III.

Brian Borchers (1999). CSDP, A C Library for Semidefinite Programming. Optimization Methods and Software 11(1–4), pp. 613–623. DOI: 10.1080/10556789908805765. The solver that proposes the certificate for `psatz` in Section 22; Coq checks it again.

Alex Brodsky and Nicholas Pippenger (2002). Characterizations of 1-way quantum finite automata. SIAM Journal on Computing 31(5), pp. 1456–1478. DOI: 10.1137/S0097539799353443. Measure-once automata with bounded error accept exactly the group languages; measure-many automata accept every piecewise testable language.

N. G. de Bruijn (1970). The mathematical language AUTOMATH, its usage, and some of its extensions. In Symposium on Automatic Demonstration, Lecture Notes in Mathematics 125, Springer, pp. 29–61. The Automath project at Eindhoven ran from 1967; see the Automath archive.

Samuel Bucheli, Meghdad Ghari and Thomas Studer (2017). Temporal justification logic. Methods for Modalities (M4M9), EPTCS 243, pp. 59–74. arXiv: 1703.02195.

John P. Burgess (1986). The truth is never simple. Journal of Symbolic Logic 51(3), pp. 663–681. DOI: 10.2307/2274021. Quasi-inductive definitions, in the setting of revision theories of truth.

Stanley Burris (2010, substantive revision 2026). George Boole. Stanford Encyclopedia of Philosophy. Entry.

Stanley Burris and Javier Legris (2009, substantive revision 2021). The Algebra of Logic Tradition. Stanford Encyclopedia of Philosophy. Entry.

Michael Burrows, Martin Abadi and Roger Needham (1990). A logic of authentication. ACM Transactions on Computer Systems 8(1), pp. 18–36. DOI: 10.1145/77648.77649. First presented at the 12th ACM Symposium on Operating Systems Principles, 1989.

Chris K. Caldwell (ed.). Prime Curios! page for $2^{11213} - 1$. The Prime Pages. Web page. The Illinois mathematics department's postage meter changed to stamp "$2^{11213} - 1$ is prime" after the 1963 find.

Georg Cantor (1874). Über eine Eigenschaft des Inbegriffes aller reellen algebraischen Zahlen. Journal für die reine und angewandte Mathematik 77, pp. 258–262. DOI: 10.1515/crll.1874.77.258.

Georg Cantor (1891). Über eine elementare Frage der Mannigfaltigkeitslehre. Jahresbericht der Deutschen Mathematiker-Vereinigung 1, pp. 75–78. English: On an elementary question in the theory of manifolds, translated by Anthony Phillips, translation. The symbols m and w, and the comparison with the 1874 proof, are on p. 75 of the original.

Mario Carneiro (2019). Formalizing computability theory via partial recursive functions. Interactive Theorem Proving, ITP 2019, LIPIcs 141, article 12. arXiv: 1810.08380; DOI: 10.4230/LIPIcs.ITP.2019.12.

Ethan Cecchetti, Andrew C. Myers and Owen Arden (2017). Nonmalleable information flow control. CCS 2017, pp. 1875–1891. DOI: 10.1145/3133956.3134054.

K. Mani Chandy and Leslie Lamport (1985). Distributed snapshots: determining global states of distributed systems. ACM Transactions on Computer Systems 3(1), pp. 63–75. DOI: 10.1145/214451.214456. Stable properties.

G. K. Chesterton. Wikipedia. Article. An English writer, referred to as the "prince of paradox".

Alessandro Chiesa and Eran Tromer (2010). Proof-carrying data and hearsay arguments from signature cards. Proceedings of Innovations in Computer Science (ICS 2010), pp. 310–331.

Alonzo Church (1936). An Unsolvable Problem of Elementary Number Theory. American Journal of Mathematics 58(2), pp. 345–363. DOI: 10.2307/2371045.

Clay Mathematics Institute (n.d.). The Millennium Prize Problems. Web page.

Clay Mathematics Institute (n.d.). The Poincaré Conjecture. Web page. Perelman's preprints of 2002 and 2003, and the prize of 2010.

Edgar F. Codd (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM 13(6), pp. 377–387. DOI: 10.1145/362384.362685.

Thomas Colcombet and Daniela Petrişan (2020). Automata minimization: a functorial approach. Logical Methods in Computer Science 16(1), paper 32. DOI: 10.23638/LMCS-16(1:32)2020. Journal version of the CALCO 2017 paper.

Stephen A. Cook (1971). The complexity of theorem-proving procedures. Proceedings of STOC 1971, pp. 151–158. DOI: 10.1145/800157.805047.

Stephen A. Cook and Robert A. Reckhow (1973). Time bounded random access machines. Journal of Computer and System Sciences 7(4), pp. 354–375. DOI: 10.1016/S0022-0000(73)80029-7.

Coq development team (2021). Renaming Coq. Announcement, Rocq Prover Discourse forum, 8 April 2021. Announcement.

Thierry Coquand and Christine Paulin-Mohring (1990). Inductively defined types. In COLOG-88, International Conference on Computer Logic, Tallinn, 1988, Lecture Notes in Computer Science 417, Springer, pp. 50–66. DOI: 10.1007/3-540-52335-9_47.

Thierry Coquand and Gérard Huet (1988). The calculus of constructions. Information and Computation 76(2–3), pp. 95–120. DOI: 10.1016/0890-5401(88)90005-3. Huet as a French computer scientist at INRIA: Wikipedia, "Gérard Huet". Coquand as a French computer scientist and mathematician at the University of Gothenburg: Wikipedia, "Thierry Coquand".

Patrick Cousot and Radhia Cousot (1977). Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. Proceedings of POPL 1977, pp. 238–252. Author publication record and paper.

Haskell B. Curry (1934). Functionality in combinatory logic. Proceedings of the National Academy of Sciences 20(11), pp. 584–590.

Haskell B. Curry and Robert Feys (1958). Combinatory Logic, Volume I. With two sections by William Craig. Studies in Logic and the Foundations of Mathematics, North-Holland, Amsterdam.

Vincent Danos and Jean Krivine (2005). Transactions in RCCS. CONCUR 2005, Lecture Notes in Computer Science 3653, pp. 398–412. DOI: 10.1007/11539452_31.

Donald W. Davies (2004). Corrections to Turing's Universal Computing Machine. In B. Jack Copeland (ed.), The Essential Turing, pp. 103–124. Oxford University Press. Davies at the National Physical Laboratory from 1947 and the word *packet*: Wikipedia, Donald Davies.

Martin Davis (1958). Computability and Unsolvability. New York: McGraw-Hill. Reprinted by Dover, 1982.

Richard Dedekind (1888). Was sind und was sollen die Zahlen? Vieweg, Braunschweig.

Dorothy E. Denning (1976). A lattice model of secure information flow. Communications of the ACM 19(5), pp. 236–243.

Jack B. Dennis and Earl C. Van Horn (1966). Programming semantics for multiprogrammed computations. Communications of the ACM 9(3), pp. 143–155. DOI: 10.1145/365230.365252. Capabilities.

Department of Mathematics, University of Illinois at Urbana-Champaign (n.d.). Postmarks Used by Department of Mathematics. Document.

Harry Deutsch and Oliver Marshall (2021, revised 2022). Alonzo Church. Stanford Encyclopedia of Philosophy. Entry.

Charles Dickens (1843). A Christmas Carol. Chapman and Hall, London. Stave Three, the game of Yes and No.

Edsger W. Dijkstra (1976). A Discipline of Programming. Prentice-Hall. The sentence on testing is quoted as in Harrison (2007), p. 629.

Edsger W. Dijkstra (1982). Why numbering should start at zero. EWD 831, 11 August 1982. Transcription.

Wolfgang Doeblin. His life (birth in Berlin in 1915, doctorate at the Sorbonne in 1938, work on Markov chains, death in June 1940) is from J. J. O'Connor and E. F.

Robertson, “Wolfgang Doeblin,” MacTutor History of Mathematics archive, University of St Andrews, mathshistory.st-andrews.ac.uk.

Rod Downey, Carl Jockusch, Timothy H. McNicholl and Paul Schupp (2014). Asymptotic density and the Ershov hierarchy. arXiv: 1309.0137. States that the 1-c.e. sets (one change of guess, from no to yes) are precisely the c.e. sets.

Andrej Dudenhefner (2022). Certified Decision Procedures for Two-Counter Machines. FSCD 2022, LIPIcs 228, article 16. The successful-decrement jump convention used in the counter guest. DOI: 10.4230/LIPIcs.FSCD.2022.16.

W. H. Eccles and F. W. Jordan (1919). A trigger relay utilizing three-electrode thermionic vacuum tubes. The Electrician 83, p. 298; reprinted in Radio Review 1(3), pp. 143–146.

William Henry Eccles and Frank Wilfred Jordan (1920). Improvements in ionic relays. British patent GB148582A, filed 21 June 1918, published 5 August 1920. Patent text. The 1918 date in the text is the filing date. The circuit as the flip-flop, the basis for electronic memory: History of Information, entry; and Wikipedia, Flip-flop (electronics), on the Eccles–Jordan trigger circuit as the first electronic latch, later the basic storage element of digital circuitry.

Yu. L. Ershov (1968). A hierarchy of sets. I. Algebra and Logic 7, pp. 25–43; translated from Algebra i Logika 7(1), pp. 47–74. DOI: 10.1007/BF02218750.

Benja Fallenstein and Nate Soares (2015). Vingean reflection: reliable reasoning for self-improving agents. Technical Report 2015–2, Machine Intelligence Research Institute. Report.

Solomon Feferman (2006). Are There Absolutely Unsolvable Problems? Gödel’s Dichotomy. Philosophia Mathematica (III) 14. Author’s text; DOI: 10.1093/philmat/nkj003.

Solomon Feferman (2006). The nature and significance of Gödel’s incompleteness theorems. Lecture, Institute for Advanced Study, Gödel Centenary Program, 17 November 2006. Author’s text.

Robert Feys. Wikipedia. Article. Belgian logician at the University of Leuven; with Curry, Combinatory Logic (1958).

Jean-Christophe Filliâtre, Léon Gondelman and Andrei Paskevich (2014). The spirit of ghost code. Computer Aided Verification (CAV 2014), Lecture Notes in Computer Science 8559, pp. 1–16. DOI: 10.1007/978-3-319-08867-9_1.

Lester R. Ford Jr. and Selmer M. Johnson (1959). A Tournament Problem. American Mathematical Monthly 66(5), pp. 387–389.

Yannick Forster and Dominique Larchey-Wendling (2019). Certified undecidability of intuitionistic linear logic via binary stack machines and Minsky machines. Certified Programs and Proofs, CPP 2019, pp. 104–117. DOI: 10.1145/3293880.3294096.

Yannick Forster and Fabian Kunze (2019). A Certifying Extraction with Time Bounds from Coq to Call-By-Value Lambda Calculus. Interactive Theorem Proving, ITP 2019, LIPIcs 141, article 17. DOI: 10.4230/LIPIcs.ITP.2019.17.

Yannick Forster and Gert Smolka (2017). Weak Call-by-Value Lambda Calculus as a Model of Computation in Coq. Interactive Theorem Proving, ITP 2017, LNCS 10499, pp. 189–206. Author paper; DOI: 10.1007/978-3-319-66107-0_13.

Yannick Forster, Dominik Kirst and Gert Smolka (2019). On Synthetic Undecidability in Coq, with an Application to the Entscheidungsproblem. Certified Programs and Proofs, CPP 2019, pp. 38–51. DOI: 10.1145/3293880.3294091.

Yannick Forster, Dominique Larchey-Wendling, Andrej Dudenhefner, Edith Heiter, Dominik Kirst, Fabian Kunze, Gert Smolka, Simon Spies, Dominik Wehr and Maximilian Wuttke (2020). A Coq Library of Undecidable Problems. The Sixth International Workshop on Coq for Programming Languages, CoqPL 2020. Workshop record. The library itself is the Coq Library of Undecidability Proofs, repository.

Galileo Galilei (1586). La Bilancetta (The Little Balance). Described at the Galileo Project, Rice University: page. The words quoted are from the English translation in Laura Fermi and Gilberto Bernardini, Galileo and the Scientific Revolution, Basic Books, 1961, as reproduced at math.nyu.edu; I didn't see the printed page.

George Gamow (1947). One Two Three... Infinity. Viking Press, New York.

Deepak Garg and Frank Pfenning (2010). A proof-carrying file system. IEEE Symposium on Security and Privacy 2010, pp. 349–364.

Roberto Giacobazzi, Francesco Ranzato and Francesca Scozzari (2000). Making abstract interpretations complete. Journal of the ACM 47(2), pp. 361–416. Author publication record.

Donald B. Gillies (1964). Three new Mersenne primes and a statistical theory. Mathematics of Computation 18, pp. 93–97. DOI: 10.1090/S0025-5718-1964-0159774-6. Gillies as a professor at the University of Illinois from 1956 who designed the pipeline control of ILLIAC II and found these primes during its checkout in 1963, and the mathematics department's postal meter cancellation for

the largest of them, used until Appel and Haken's four-colour proof of 1976: Wikipedia, Donald B. Gillies and ILLIAC II.

Jean-Yves Girard (1987). Linear logic. Theoretical Computer Science 50(1), pp. 1–101. DOI: 10.1016/0304-3975(87)90045-4.

Kurt Gödel (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. Monatshefte für Mathematik und Physik 38, pp. 173–198. DOI: 10.1007/BF01700692.

Kurt Gödel (1951). Some basic theorems on the foundations of mathematics and their implications. Gibbs Lecture, American Mathematical Society, Brown University, 26 December 1951. In Kurt Gödel, Collected Works, Vol. III: Unpublished Essays and Lectures, S. Feferman et al. (eds.), Oxford University Press, 1995, pp. 304–323.

Joseph A. Goguen (1973). Realization is universal. Mathematical Systems Theory 6, pp. 359–374. DOI: 10.1007/BF01843493.

E. Mark Gold (1965). Limiting recursion. Journal of Symbolic Logic 30, pp. 28–48.

Shafi Goldwasser, Silvio Micali and Charles Rackoff (1985). The Knowledge Complexity of Interactive Proof-Systems. Proceedings of the 17th Annual ACM Symposium on Theory of Computing, pp. 291–304.

Georges Gonthier (2008). Formal proof: the four-color theorem. Notices of the American Mathematical Society 55(11), pp. 1382–1393. The Coq proof was completed in 2005.

Andrew D. Gordon and Alan Jeffrey (2003). Authenticity by typing for security protocols. Journal of Computer Security 11(4), pp. 451–519. DOI: 10.3233/JCS-2003-11402.

Michael J. C. Gordon, Robin Milner and Christopher P. Wadsworth (1979). Edinburgh LCF: A Mechanised Logic of Computation. Lecture Notes in Computer Science 78, Springer. DOI: 10.1007/3-540-09724-4.

David Alan Grier (2005). When Computers Were Human. Princeton University Press.

Nicholas Griffin, ed. (2003). The Cambridge Companion to Bertrand Russell. Cambridge University Press. G. G. Berry as a junior librarian at the Bodleian, p. 63.

Joseph Y. Halpern, Yoram Moses and Mark R. Tuttle (1988). A knowledge-based analysis of zero knowledge. Proceedings of the 20th Annual ACM Symposium on Theory of Computing (STOC '88), pp. 132–147. DOI: 10.1145/62212.62224.

Joel David Hamkins and Andy Lewis (2000). Infinite time Turing machines. Journal of Symbolic Logic 65(2), pp. 567–604. DOI: 10.2307/2586556; arXiv: math/9808093.

Joel David Hamkins and Theodor Nenu (2026). Did Turing prove the undecidability of the halting problem? Journal of Logic and Computation 36(1), exaf075. DOI: 10.1093/logcom/exaf075; arXiv: 2407.00680.

Kevin W. Hamlen, Greg Morrisett and Fred B. Schneider (2006). Computability classes for enforcement mechanisms. ACM Transactions on Programming Languages and Systems 28(1), pp. 175–205.

John Harrison (2007). Floating-Point Verification. Journal of Universal Computer Science 13(5), pp. 629–638. Paper.

Klaus Havelund and Grigore Roşu (2002). Synthesizing monitors for safety properties. Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2002), Lecture Notes in Computer Science 2280, pp. 342–356. DOI: 10.1007/3-540-46002-0_24.

Douglas R. Hofstadter (1979). Gödel, Escher, Bach: An Eternal Golden Braid. Basic Books.

Kohei Honda (1993). Types for dyadic interaction. CONCUR '93, LNCS 715, pp. 509–523. DOI: 10.1007/3-540-57208-2_35.

William A. Howard (1980). The formulae-as-types notion of construction. In To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism, Academic Press. Circulated as a typescript in 1969.

William Alvin Howard. Wikipedia. Article. Proof theorist at the University of Illinois at Chicago.

William Stanley Jevons (1864). Pure Logic, or the Logic of Quality apart from Quantity. Edward Stanford, London.

Juvenal. Satires, Satire VI, lines 346–348, and the Oxford fragment found by E. O. Winstedt in a Bodleian manuscript in 1899, published as E. O. Winstedt (1899), A Bodleian MS. of Juvenal, The Classical Review 13, pp. 201–205. The debate over whether the lines are Juvenal's is summarized at Wikipedia, Quis custodiet ipsos custodes?.

Rudolf E. Kalman (1960). A New Approach to Linear Filtering and Prediction Problems. Journal of Basic Engineering 82(1), pp. 35–45. DOI: 10.1115/1.3662552.

Rudolf E. Kalman (1960). Contributions to the Theory of Optimal Control. Boletín de la Sociedad Matemática Mexicana 5, pp. 102–119. Reprinted paper.

Paul A. Karger and Roger R. Schell (1974). Multics Security Evaluation: Vulnerability Analysis. Technical Report ESD-TR-74-193, Vol. II, Electronic Systems Division, Air Force Systems Command, June 1974.

Stephen Cole Kleene (1938). On notation for ordinal numbers. Journal of Symbolic Logic 3(4), pp. 150–155. DOI: 10.2307/2267778. The second recursion theorem is stated and proved at the end of section 2.

Stephen Cole Kleene (1952). Introduction to Metamathematics. North-Holland and D. Van Nostrand. Partial recursive functions, parameterisation and recursion theorems.

Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch and Simon Winwood (2009). seL4: formal verification of an OS kernel. Proceedings of SOSP 2009, pp. 207–220. DOI: 10.1145/1629575.1629596.

Donald E. Knuth (1998). The Art of Computer Programming, Volume 3: Sorting and Searching, second edition. Addison-Wesley. Section 5.3.1, merge insertion, pp. 184–186.

Donald E. Knuth (n.d.). Home page, Stanford University: "Professor Emeritus of The Art of Computer Programming at Stanford University." Page.

Donald E. Knuth (n.d.). The Art of Computer Programming (TAOCP) home page, on the reward for errors. Page. See also Knuth's news for 2008, page, on the Bank of San Serriffe.

Andrei N. Kolmogorov (1932). Zur Deutung der intuitionistischen Logik. Mathematische Zeitschrift 35, pp. 58–65.

Helge Kragh (2014). The True (?) Story of Hilbert's Infinite Hotel. arXiv: 1403.0059.

Lindsey Kuper and Ryan R. Newton (2013). LVars: lattice-based data structures for deterministic parallelism. Proceedings of the 2nd ACM SIGPLAN Workshop on Functional High-Performance Computing (FHPC 2013), pp. 71–84. DOI: 10.1145/2502323.2502326.

Joachim Lambek (1961). How to Program an Infinite Abacus. Canadian Mathematical Bulletin 4(3), pp. 295–302. Paper. "Our method is probably not new" is on p. 295.

Joachim Lambek and Philip J. Scott (1986). Introduction to Higher Order Categorical Logic. Cambridge Studies in Advanced Mathematics 7. Cambridge University Press.

F. William Lawvere (1969). Adjointness in foundations. Dialectica 23(3–4), pp. 281–296. DOI: 10.1111/j.1746-8361.1969.tb01194.x. The quantifiers as the two adjoints of substitution.

F. William Lawvere (1969). Diagonal arguments and cartesian closed categories. Lecture Notes in Mathematics 92, pp. 134–145. Reprinted with author commentary in Reprints in Theory and Applications of Categories 15 (2006), pp. 1–13. Authorized reprint.

Xavier Leroy (2009). Formal verification of a realistic compiler. Communications of the ACM 52(7), pp. 107–115. DOI: 10.1145/1538788.1538814.

Leonid A. Levin (1973). Universal sequential search problems. Problemy Peredachi Informatsii 9(3), pp. 115–116; English translation in Problems of Information Transmission 9(3), pp. 265–266. Record.

Orna Lichtenstein, Amir Pnueli and Lenore Zuck (1985). The glory of the past. Logics of Programs, Lecture Notes in Computer Science, pp. 196–218. DOI: 10.1007/3-540-15648-8_16. Past-time temporal logic.

Martin H. Löb (1955). Solution of a problem of Leon Henkin. Journal of Symbolic Logic 20(2), pp. 115–118.

J. R. Lucas (1961). Minds, Machines and Gödel. Philosophy 36(137), pp. 112–127. DOI: 10.1017/S0031819100057983.

Charles E. Mackenzie (1980). Coded Character Sets: History and Development. Addison-Wesley. The design history of ASCII, including the placing of the lowercase letters so that each differs from its capital by one bit.

Elizabeth J. Magie (1904). Game-Board (The Landlord's Game). US patent 748,626, filed 23 March 1903, issued 5 January 1904. Her description of the game as "a practical demonstration of the present system of land-grabbing" is from The Single Tax Review, Autumn 1902, as transcribed at landlordsgame.info; I didn't see the printed page. Parker Brothers' purchase of her 1923 patent for $500 in 1935: Wikipedia, The Landlord's Game.

Jean-Pierre Marquis (1996, substantive revision 2019). Category Theory. Stanford Encyclopedia of Philosophy. Entry. The borrowed words "category" and "functor".

John McCarthy (1979). History of Lisp. Author's version dated 12 February 1979. Section "The implementation of LISP".

Ross M. McConnell, Kurt Mehlhorn, Stefan Näher and Pascal Schweitzer (2011). Certifying algorithms. Computer Science Review 5(2), pp. 119–161. DOI: 10.1016/j.cosrev.2010.09.009.

Leonard A. McGee and Stanley F. Schmidt (1985). Discovery of the Kalman Filter as a Practical Tool for Aerospace and Industry. NASA Technical Memorandum 86847, Ames Research Center. NTRS.

Jason Megill. The Lucas-Penrose Argument about Gödel's Theorem. Internet Encyclopedia of Philosophy. Article.

Z. A. Melzak (1961). An Informal Arithmetical Approach to Computability and Computation. Canadian Mathematical Bulletin 4(3), pp. 279–293.

Glen Mével, Jacques-Henri Jourdan and François Pottier (2019). Time credits and time receipts in Iris. ESOP 2019, LNCS 11423, pp. 3–29. DOI: 10.1007/978-3-030-17184-1_1.

Jeff Miller. Earliest Uses of Symbols of Set Theory and Logic. MacTutor History of Mathematics. Page. Peano's epsilon for *est*, 1889.

Marvin L. Minsky (1961). Recursive Unsolvability of Post's Problem of Tag and Other Topics in Theory of Turing Machines. Annals of Mathematics 74(3), pp. 437–455. Original paper; DOI: 10.2307/1970290.

Marvin L. Minsky (1967). Computation: Finite and Infinite Machines. Prentice-Hall. Two-counter machines are universal, so their halting problem is undecidable.

Torben Æ. Mogensen (1992). Efficient Self-Interpretation in Lambda Calculus. Journal of Functional Programming 2(3), pp. 345–364. DOI: 10.1017/S0956796800000423. Scott's encoding carried from data to lambda terms themselves.

Liesbeth De Mol (2008). Tag systems and Collatz-like functions. Theoretical Computer Science 390(1), pp. 92–101. DOI: 10.1016/j.tcs.2007.10.020.

Edward F. Moore (1956). Gedanken-Experiments on Sequential Machines. In Claude E. Shannon and John McCarthy (eds.), Automata Studies, pp. 129–153. Princeton University Press. Original paper.

Augustus De Morgan (1864). On the Syllogism, No. IV, and on the Logic of Relations. Transactions of the Cambridge Philosophical Society 10. Sources disagree on the page range and on the date it was read.

F. L. Morris and C. B. Jones (1984). An early program proof by Alan Turing. Annals of the History of Computing 6(2), pp. 139–143. Paper.

James H. Morris, Jr. (1973). Protection in programming languages. Communications of the ACM 16(1), pp. 15–21. DOI: 10.1145/361932.361937.

Joan Moschovakis (1999, revised 2023). Intuitionistic Logic. Stanford Encyclopedia of Philosophy. Entry. The Brouwer–Heyting–Kolmogorov explanation, and Heyting's formalization of 1930.

Yiannis N. Moschovakis (2009). Kleene's Amazing Second Recursion Theorem. Computer Science Logic, LNCS 5771, pp. 24–39. "This little gem is stated unbilled and proved (completely) in the last two lines of §2 of the short note Kleene (1938)." Longer version: Bulletin of Symbolic Logic 16(2) (2010), pp. 189–239. Author lecture text.

George C. Necula (1997). Proof-Carrying Code. Proceedings of POPL 1997, pp. 106–119. Author publication list.

George C. Necula and Peter Lee (1996). Safe Kernel Extensions Without Run-Time Checking. Proceedings of the 2nd USENIX Symposium on Operating Systems Design and Implementation (OSDI '96), pp. 229–243. USENIX page.

Thomas R. Nicely (1995). Enumeration to 10^{14} of the twin primes and Brun's constant. Virginia Journal of Science 46(3), pp. 195–204. Author's page. The computation in which the Pentium's division flaw turned up.

Kyndylan Nienhuis, Alexandre Joannou, Thomas Bauereiss, Anthony Fox, Michael Roe, Brian Campbell, Matthew Naylor, Robert M. Norton, Simon W. Moore, Peter G. Neumann, Ian Stark, Robert N. M. Watson and Peter Sewell (2020). Rigorous engineering for hardware security: formal modelling and proof in the CHERI design and implementation process. 2020 IEEE Symposium on Security and Privacy, pp. 1003–1020. DOI: 10.1109/SP40000.2020.00055. Technical report version: University of Cambridge UCAM-CL-TR-940, which states the reachable-capability monotonicity property.

Michael Norrish (2011). Mechanised Computability Theory. Interactive Theorem Proving, ITP 2011, LNCS 6898, pp. 297–311. Author paper; DOI: 10.1007/978-3-642-22863-6_22.

J. J. O'Connor and E. F. Robertson (n.d.). Haskell Brooks Curry. MacTutor History of Mathematics. Biography.

Caspar Oesterheld (2019). Robust program equilibrium. Theory and Decision 86, pp. 143–159. DOI: 10.1007/s11238-018-9679-3.

Susan Owicki and David Gries (1976). An axiomatic proof technique for parallel programs I. Acta Informatica 6(4), pp. 319–340. DOI: 10.1007/BF00268134.

Catuscia Palamidessi, Vijay Saraswat, Frank D. Valencia and Björn Victor (2006). On the expressiveness of linearity vs persistence in the asynchronous pi-calculus. Proceedings of the 21st Annual IEEE Symposium on Logic in Computer Science (LICS 2006), pp. 59–68. DOI: 10.1109/LICS.2006.39.

Parker Brothers (Hasbro) (n.d.). Monopoly, official rules, the section on the Bank.

Bryan Parno, Jon Howell, Craig Gentry and Mariana Raykova (2013). Pinocchio: nearly practical verifiable computation. Proceedings of the 2013 IEEE Symposium on Security and Privacy, pp. 238–252. IACR ePrint 2013/279.

Christine Paulin-Mohring. Wikipedia. Article. A mathematical logician and computer scientist best known for developing the interactive theorem prover Rocq, formerly Coq.

Giuseppe Peano (1889). Arithmetices principia, nova methodo exposita. Bocca, Turin.

Charles S. Peirce (1870, printed 1873). Description of a Notation for the Logic of Relatives, Resulting from an Amplification of the Conceptions of Boole's Calculus of Logic. Memoirs of the American Academy of Arts and Sciences 9(2), pp. 317–378.

Roger Penrose (1989). The Emperor's New Mind: Concerning Computers, Minds, and the Laws of Physics. Oxford University Press.

Roger Penrose (1994). Shadows of the Mind: A Search for the Missing Science of Consciousness. Oxford University Press.

Oskar Pfungst (1907). Das Pferd des Herrn von Osten (Der Kluge Hans). J. A. Barth, Leipzig. English translation: Clever Hans (The Horse of Mr. von Osten), trans. C. L. Rahn, Henry Holt, New York, 1911. Wilhelm von Osten as a mathematics teacher, the exhibitions, Pfungst's tests, and von Osten never persuaded, as summarized at Wikipedia, Clever Hans.

Jean-Éric Pin (1992). On reversible automata. In LATIN '92, 1st Latin American Symposium on Theoretical Informatics, Lecture Notes in Computer Science 583, Springer, pp. 401–416. DOI: 10.1007/BFb0023844.

Serge A. Plotkin (1989). Sticky Bits and Universality of Consensus. Proceedings of PODC 1989, pp. 159–175. DOI: 10.1145/72981.72992.

Johannes Åman Pohjola and Joachim Parrow (2016). The expressive power of monotonic parallel composition. Programming Languages and Systems, ESOP 2016, Lecture Notes in Computer Science, Springer, pp. 780–803. DOI: 10.1007/978-3-662-49498-1_30.

Gerald J. Popek and Robert P. Goldberg (1974). Formal Requirements for Virtualizable Third Generation Architectures. Communications of the ACM 17(7), pp. 412–421.

Emil L. Post (1947). Recursive Unsolvability of a Problem of Thue. Journal of Symbolic Logic 12(1), pp. 1–11. The defect in Turing's appendix and its correction are described in Liesbeth De Mol, Turing Machines, Stanford Encyclopedia of Philosophy, entry.

Hilary Putnam (1965). Trial and error predicates and the solution to a problem of Mostowski. Journal of Symbolic Logic 30, pp. 49–57.

W. V. O. Quine (1962). Paradox. Scientific American 206(4), April 1962, beginning p. 84. Reprinted as "The Ways of Paradox" in The Ways of Paradox and Other Essays, Harvard University Press, 1966. I haven't seen the original printing; the quoted phrase is as the essay is usually quoted.

Jean-Jacques Quisquater, Myriam Quisquater, Muriel Quisquater, Michaël Quisquater, Louis Guillou, Marie Annick Guillou, Gaïd Guillou, Anna Guillou, Gwenolé Guillou, Soazig Guillou and Thomas A. Berson (1990). How to Explain Zero-Knowledge Protocols to Your Children. Advances in Cryptology, CRYPTO '89, LNCS 435, pp. 628–631. Springer. DOI: 10.1007/0-387-34805-0_60.

Panu Raatikainen (2013, revised 2025). Gödel's Incompleteness Theorems. Stanford Encyclopedia of Philosophy. Entry. Source for the cautions about what the Gödel sentence says.

Erich Reck. Dedekind's Contributions to the Foundations of Mathematics. Stanford Encyclopedia of Philosophy. Entry. Peano acknowledged Dedekind's priority for the axioms.

Henry Gordon Rice (1953). Classes of Recursively Enumerable Sets and Their Decision Problems. Transactions of the American Mathematical Society 74(2), pp. 358–366. DOI: 10.1090/S0002-9947-1953-0053041-6.

Jules Richard (1905). Les principes des mathématiques et le problème des ensembles. Revue générale des sciences pures et appliquées 16(12), pp. 541–543. English translation in Jean van Heijenoort (ed.), From Frege to Gödel, Harvard University Press, 1967, pp. 142–144. His life (school posts, doctorate, geometry) is from

J. J. O'Connor and E. F. Robertson, "Jules Antoine Richard," MacTutor History of Mathematics archive, University of St Andrews, mathshistory.st-andrews.ac.uk.

Wayne Richter and Peter Aczel (1974). Inductive definitions and reflecting properties of admissible ordinals. In J. E. Fenstad and P. G. Hinman (eds.), Generalized Recursion Theory, North-Holland, Studies in Logic and the Foundations of Mathematics 79, pp. 301–381. DOI: 10.1016/S0049-237X(08)70592-5.

John Scott Robin and Cynthia E. Irvine (2000). Analysis of the Intel Pentium's Ability to Support a Secure Virtual Machine Monitor. Proceedings of the 9th USENIX Security Symposium, Denver, pp. 129–144. Paper.

The Rocq Prover (2025). Rocq Prover 9.0.0 release notes, 12 March 2025. Release notes.

The Rocq Prover. About the Rocq Prover (the name: Inria Rocquencourt and the mythological bird Roc; the old name Coq: the Calculus of Constructions, Thierry Coquand and the rooster). Web page. The project's own answer to why it was called Coq, the French habit of naming research software after animals with Caml, Elan, Foc and Phox as examples, is on the Presentation page of its wiki. Coquand's work on the calculus of inductive constructions with Christine Paulin-Mohring: Wikipedia, Rocq.

Hartley Rogers, Jr. (1967). Theory of Recursive Functions and Effective Computability. New York: McGraw-Hill. Acceptable numberings and the recursion theorem.

Morgan Rogers (2019). Toposes of discrete monoid actions. arXiv:1905.10277.

Morgan Rogers (2021). Toposes of monoid actions. arXiv:2112.10198.

J. Barkley Rosser (1936). Extensions of some theorems of Gödel and Church. Journal of Symbolic Logic 1(3), pp. 87–91. DOI: 10.2307/2269028. Rosser's trick, which weakens Gödel's ω-consistency premise to consistency. Rosser as a student of Alonzo Church at Princeton: Wikipedia, J. Barkley Rosser.

James S. Royer and John Case (1994). Subrecursive Programming Systems. Boston: Birkhäuser. DOI: 10.1007/978-1-4612-0249-3.

Bertrand Russell (1903). The Principles of Mathematics. Cambridge University Press. Chapter X, The Contradiction, §100: "I was led to it in the endeavour to reconcile Cantor's proof that there can be no greatest cardinal number with the very plausible supposition that the class of all terms … has necessarily the greatest possible number of members." Text of §100.

Bertrand Russell (1906). Les paradoxes de la logique. Revue de métaphysique et de morale 14, pp. 627–650.

A. Saboori and C. N. Hadjicostis (2007). Notions of security and opacity in discrete event systems. Proceedings of the 46th IEEE Conference on Decision and Control, pp. 5056–5061. DOI: 10.1109/CDC.2007.4434515.

Anooshiravan Saboori (2011). Verification and Enforcement of State-Based Notions of Opacity in Discrete Event Systems. PhD thesis, University of Illinois at Urbana-Champaign; adviser Christoforos N. Hadjicostis. IDEALS.

M. Sampath, R. Sengupta, S. Lafortune, K. Sinnamohideen and D. C. Teneketzis (1996). Failure diagnosis using discrete-event models. IEEE Transactions on Control Systems Technology 4(2), pp. 105–124. Its first page gives Sampath, Sengupta, Lafortune and Teneketzis at the University of Michigan and Sinnamohideen at Johnson Controls. DOI: 10.1109/87.486338.

M. Sampath, R. Sengupta, S. Lafortune, K. Sinnamohideen and D. Teneketzis (1995). Diagnosability of discrete-event systems. IEEE Transactions on Automatic Control 40(9), pp. 1555–1575. DOI: 10.1109/9.412626.

Jürgen Schmidhuber (2003). Gödel machines: self-referential universal problem solvers making provably optimal self-improvements. Technical Report IDSIA-19-03. arXiv:cs/0309048.

Fred B. Schneider (2000). Enforceable security policies. ACM Transactions on Information and System Security 3(1), pp. 30–50. DOI: 10.1145/353323.353382.

Rich Schroeppel (1972). A Two Counter Machine Cannot Calculate 2^N. MIT Artificial Intelligence Laboratory, AI Memo 257, 1 May 1972. MIT DSpace.

Marcel-Paul Schützenberger (1961). On the definition of a family of automata. Information and Control 4(2–3), pp. 245–270.

Dana Scott (1962). A system of functional abstraction. Lectures delivered at the University of California, Berkeley, 1962–63; unpublished. The origin of the Scott encoding, as given in Mogensen–Scott encoding, Wikipedia, Article.

Dana Scott. Carnegie Mellon University's School of Computer Science, faculty Turing Award recipients: the 1976 award jointly with Michael Rabin for their work on nondeterministic machines. His doctorate at Princeton under Alonzo Church, domain theory and models of the lambda calculus: Wikipedia, Dana Scott.

Marc Shapiro, Nuno Preguiça, Carlos Baquero and Marek Zawirski (2011). A comprehensive study of Convergent and Commutative Replicated Data Types. INRIA Research Report RR-7506. HAL: inria-00555588.

J. C. Shepherdson and H. E. Sturgis (1963). Computability of recursive functions. Journal of the ACM 10(2), pp. 217–255. DOI: 10.1145/321160.321170. The unlimited register machine.

Ken Shirriff (2024). Intel's $475 million error: the silicon behind the Pentium division bug. Ken Shirriff's blog, December 2024. Post.

Ken Shirriff. Righto (blog). Blog. Reverse engineering of old chips from die photographs.

Siebel School of Computing and Data Science, University of Illinois. Donald B. Gillies Memorial Lecture. Web page. The three primes found while checking out ILLIAC II, "commemorated by the Mathematics Department with a postage mark stamped on all outgoing mail" for nearly a decade.

Emin Gün Sirer, Willem de Bruijn, Patrick Reynolds, Alan Shieh, Kevin Walsh, Dan Williams and Fred B. Schneider (2011). Logical attestation: an authorization architecture for trustworthy computing. SOSP 2011, pp. 249–264. DOI: 10.1145/2043556.2043580.

Geoffrey Smith (2009). On the foundations of quantitative information flow. Foundations of Software Science and Computational Structures, FoSSaCS 2009, LNCS 5504, pp. 288–302. DOI: 10.1007/978-3-642-00596-1_21.

Raymond M. Smullyan (1993). Recursion Theory for Metamathematics. New York: Oxford University Press. DOI: 10.1093/oso/9780195082326.001.0001.

Raymond M. Smullyan (1994). Diagonalization and Self-Reference. Oxford: Clarendon Press. DOI: 10.1093/oso/9780198534501.001.0001.

Dava Sobel (2016). The Glass Universe: How the Ladies of the Harvard Observatory Took the Measure of the Stars. Viking.

Chamseddine Talhi, Nadia Tawbi and Mourad Debbabi (2008). Execution monitoring enforcement under memory-limitation constraints. Information and Computation 206, pp. 158–184.

Ken Thompson (1984). Reflections on Trusting Trust. Communications of the ACM 27(8), pp. 761–763. Delivered as the 1983 Turing Award lecture. DOI: 10.1145/358198.358210.

Ken Thompson. Heidelberg Laureate Forum, laureate profile: the 1983 Turing Award with Dennis Ritchie for operating systems theory and the implementation of Unix. Thompson's design and implementation of the original Unix at Bell Labs: Wikipedia, Ken Thompson.

Amin Timany and Lars Birkedal (2021). Reasoning about monotonicity in separation logic. Proceedings of the 10th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2021), pp. 91–104. DOI: 10.1145/3437992.3439931.

Alan M. Turing (1936–1937). On Computable Numbers, with an Application to the Entscheidungsproblem. Proceedings of the London Mathematical Society, series 2, volume 42, pp. 230–265. DOI: 10.1112/plms/s2-42.1.230. The "note of instructions" is in Section 9, argument III, p. 253.

Alan M. Turing (1937). Computability and λ-definability. Journal of Symbolic Logic 2(4), pp. 153–163. DOI: 10.2307/2268280. The computable functions are the λ-definable functions of Church and the general recursive functions.

United States of America Standards Institute (1967). USA Standard Code for Information Interchange, USAS X3.4-1967. The revision that added the lower-case letters, one bit away from the capitals.

University of Illinois Distributed Museum. Four Colors Suffice. Web page. The "four colors suffice" postmark the Department of Mathematics stamped on its outgoing mail for years after the proof.

Leslie G. Valiant (1990). A bridging model for parallel computation. Communications of the ACM 33(8), pp. 103–111. DOI: 10.1145/79173.79181. The bulk-synchronous parallel (BSP) model.

Vitruvius. The Ten Books on Architecture, Book IX, Introduction. Trans. Morris Hicky Morgan, Harvard University Press, 1914. Passage.

Philip Wadler (2015). Propositions as Types. Communications of the ACM 58(12), pp. 75–84. Author's copy.

Hao Wang (1957). A variant to Turing's theory of computing machines. Journal of the ACM 4(1), pp. 63–92. DOI: 10.1145/320856.320867. Machines that mark a square and never erase one.

Mark B. Wells (1965). Applications of a language for computing in combinatorics. Information Processing 65, Proceedings of IFIP Congress 1965, pp. 497–498.

David A. Wheeler (2005). Countering Trusting Trust through Diverse Double-Compiling. Proceedings of the 21st Annual Computer Security Applications Conference (ACSAC), Tucson, pp. 28–40.

David A. Wheeler (2009). Fully Countering Trusting Trust through Diverse Double-Compiling. PhD dissertation, George Mason University. Author's page.

Alfred North Whitehead and Bertrand Russell (1910). Principia Mathematica, vol. I. Cambridge University Press. Introduction, chapter II.

Stephen Wolfram (2021). After 100 Years, Can We Finally Crack Post's Problem of Tag? A Story of Computational Irreducibility, and More. Stephen Wolfram Writings, March 2021. Article.

Wai Wong (1995). Recording and checking HOL proofs. In Higher Order Logic Theorem Proving and Its Applications (TPHOLs 1995), Lecture Notes in Computer Science 971, Springer, pp. 353–368.

Thomas Y. C. Woo and Simon S. Lam (1993). A semantic model for authentication protocols. Proceedings of the 1993 IEEE Computer Society Symposium on Research in Security and Privacy, pp. 178–194. DOI: 10.1109/RISP.1993.287633. Correspondence and secrecy as the two basic properties.

Noson S. Yanofsky (2003). A Universal Approach to Self-Referential Paradoxes, Incompleteness and Fixed Points. Bulletin of Symbolic Logic 9(3), pp. 362–386. arXiv: math/0305282; DOI: 10.2178/bsl/1058448677.

Andrew Chi-Chih Yao (1981). Should tables be sorted? Journal of the ACM 28(3), pp. 615–628. DOI: 10.1145/322261.322274. The cell-probe model; an earlier version appeared at FOCS 1978.

Eliezer Yudkowsky and Marcello Herreshoff (2013). Tiling agents for self-modifying AI, and the Löbian obstacle. Machine Intelligence Research Institute, draft report. Report.

H.4 Correlations and environmental records

Rotem Arnon-Friedman, Frédéric Dupuis, Omar Fawzi, Renato Renner and Thomas Vidick (2018). Practical device-independent quantum cryptography via entropy accumulation. Nature Communications 9, 459. DOI: 10.1038/s41467-017-02307-4.

Alain Aspect, Jean Dalibard and Gérard Roger (1982). Experimental Test of Bell's Inequalities Using Time-Varying Analyzers. Physical Review Letters 49, pp. 1804–1807. DOI: 10.1103/PhysRevLett.49.1804. The group was at the Institut d'Optique in Orsay, as Aspect tells it in his 2022 interview with Phillips and Dalibard (arXiv: 2212.04737).

Philip Ball (2019). Quantum Darwinism, an Idea to Explain Objective Reality, Passes First Tests. Quanta Magazine, 22 July 2019. Article.

Jonathan Barrett, Roger Colbeck and Adrian Kent (2013). Memory attacks on device-independent quantum cryptography. Physical Review Letters 110, 010503. DOI: 10.1103/PhysRevLett.110.010503.

John S. Bell (1964). On the Einstein Podolsky Rosen paradox. Physics Physique Fizika 1(3), pp. 195–200. DOI: 10.1103/PhysicsPhysiqueFizika.1.195.

John S. Bell (1975). On wave packet reduction in the Coleman-Hepp model. Helvetica Physica Acta 48, p. 93.

John S. Bell (1990). Against "measurement". Physics World 3(8), pp. 33–40.

Robin Blume-Kohout and Wojciech H. Zurek (2006). Quantum Darwinism: Entanglement, branches, and the emergent classicality of redundantly stored quantum information. Physical Review A 73, 062310. arXiv: quant-ph/0505031; DOI: 10.1103/PhysRevA.73.062310.

Niels Bohr (1958). Atomic Physics and Human Knowledge. Wiley. The essay Unity of Knowledge (1954). The page number, p. 73, is the one Mauro Dorato and Jan Faye (2026) give when they quote the line; I haven't seen the page myself.

Fernando G. S. L. Brandão, Marco Piani and Paweł Horodecki (2015). Generic emergence of classical features in quantum Darwinism. Nature Communications 6, 7908. DOI: 10.1038/ncomms8908.

Nicolas Brunner, Daniel Cavalcanti, Stefano Pironio, Valerio Scarani and Stephanie Wehner (2014). Bell nonlocality. Reviews of Modern Physics 86, pp. 419–478. arXiv: 1303.2849.

Anubhav Chaturvedi (2026). No Finite NPA Level Characterizes the Complete Quantum Set in the Simplest Bell Scenario. arXiv: 2607.14569.

M.-C. Chen et al. (2019). Emergence of classical objectivity of quantum Darwinism in a photonic quantum simulator. Science Bulletin 64, p. 580.

M. A. Ciampini, G. Pinna, P. Mataloni and M. Paternostro (2018). Experimental signature of quantum Darwinism in photonic cluster states. Physical Review A 98, 020101.

Boris S. Cirel'son (Tsirelson) (1980). Quantum generalizations of Bell's inequality. Letters in Mathematical Physics 4, pp. 93–100. DOI: 10.1007/BF00417500.

John F. Clauser, Michael A. Horne, Abner Shimony and Richard A. Holt (1969). Proposed Experiment to Test Local Hidden-Variable Theories. Physical Review Letters 23, pp. 880–884. DOI: 10.1103/PhysRevLett.23.880.

Wim van Dam (2005). Implausible consequences of superstrong nonlocality. arXiv: quant-ph/0501159.

Wim van Dam. Scientific biography. University of California, Santa Barbara. Web page. Research on the theory of quantum computation and quantum communication; professor of computer science and physics at UC Santa Barbara.

Wim van Dam, Richard D. Gill and Peter D. Grünwald (2005). The statistical strength of nonlocality proofs. IEEE Transactions on Information Theory 51(8), pp. 2812–2835. DOI: 10.1109/TIT.2005.851738. "Much stronger" is on p. 9 of the arXiv preprint quant-ph/0307125; I didn't see the journal page.

Mauro Dorato and Jan Faye (2026). Niels Bohr as a Physicist of Principle. arXiv: 2609.04297. Quotes Bohr's Unity of Knowledge (1954) on irreversible amplification and permanent marks on a photographic plate, citing p. 73.

Mnacho Echenim and Mehdi Mhalla (2023). A formalization of the CHSH inequality and Tsirelson's upper-bound in Isabelle/HOL. Journal of Automated Reasoning 68(1), article 2 (print 2024). arXiv: 2306.12535v1; DOI: 10.1007/s10817-023-09689-9.

Albert Einstein, Boris Podolsky and Nathan Rosen (1935). Can Quantum-Mechanical Description of Physical Reality Be Considered Complete? Physical Review 47, pp. 777–780. DOI: 10.1103/PhysRev.47.777.

Arthur Fine (1982). Hidden Variables, Joint Probability, and the Bell Inequalities. Physical Review Letters 48, pp. 291–295. DOI: 10.1103/PhysRevLett.48.291.

Arthur Fine (2004, substantive revision 2017). The Einstein-Podolsky-Rosen Argument in Quantum Theory. Stanford Encyclopedia of Philosophy. Entry. Einstein's "smothered by formalism" (letter to Schrödinger, 19 June 1935) is Fine's translation, quoted from Section 1.1 of this entry, which gives it as p. 35 of Fine's The Shaky Game (2nd edition, 1996); I have it secondhand.

Stuart J. Freedman and John F. Clauser (1972). Experimental Test of Local Hidden-Variable Theories. Physical Review Letters 28, pp. 938–941. DOI: 10.1103/PhysRevLett.28.938.

Murray Gell-Mann and James B. Hartle (1995, printed 1998). Strong decoherence. In D.-H. Feng and B.-L. Hu (eds.), Proceedings of the 4th Drexel Conference on Quantum Non-Integrability: The Quantum-Classical Correspondence, International Press, Hong Kong. arXiv: gr-qc/9509054. The permanence of the past.

Iulia Georgescu (2026). John Bell and the most obscure journal in physics. Physics World. Article.

Richard D. Gill. Biography, Mathematical Institute, Leiden University. Page. Chair of Mathematical Statistics at Leiden from 2006.

Marissa Giustina and 21 coauthors (2015). Significant-Loophole-Free Test of Bell's Theorem with Entangled Photons. Physical Review Letters 115, 250401. DOI: 10.1103/PhysRevLett.115.250401.

Philippe Allard Guérin, Veronika Baumann, Flavio Del Santo and Časlav Brukner (2021). A no-go theorem for the persistent reality of Wigner's friend's perception. Communications Physics 4, 93. DOI: 10.1038/s42005-021-00589-1.

J. J. Halliwell (1999). Arrival Times in Quantum Theory from an Irreversible Detector Model. Progress of Theoretical Physics 102(4), pp. 707–717. DOI: 10.1143/PTP.102.707.

J. J. Halliwell (1999). Somewhere in the universe: Where is the information stored when histories decohere? Physical Review D 60, 105031. DOI: 10.1103/PhysRevD.60.105031.

B. Hensen and 18 coauthors (2015). Loophole-free Bell inequality violation using electron spins separated by 1.3 kilometres. Nature 526, pp. 682–686. DOI: 10.1038/nature15759.

Klaus Hepp (1972). Quantum theory of measurement and macroscopic observables. Helvetica Physica Acta 45, p. 237. The Coleman-Hepp model.

Gautam A. Kavuri and coauthors (2025). Traceable random numbers from a non-local quantum advantage. Nature 642, pp. 916–921. DOI: 10.1038/s41586-025-09054-3.

L. A. Khalfin. Person page, Math-Net.Ru. Entry. Steklov Mathematical Institute, St. Petersburg (Leningrad) branch.

Leonid A. Khalfin and Boris S. Tsirelson (1985). Quantum and quasi-classical analogs of Bell inequalities. In P. Lahti and P. Mittelstaedt (eds.), Symposium on the Foundations of Modern Physics: 50 Years of the Einstein-Podolsky-Rosen Gedankenexperiment, Joensuu, Finland, 16–20 June 1985. World Scientific, Singapore, 1985, p. 441.

Lawrence J. Landau (1988). Empirical two-point correlation functions. Foundations of Physics 18, pp. 449–460. DOI: 10.1007/BF00732549. Landau's 1969 Berkeley thesis, "Some Aspects of the Quantum Theory of Local Observables": Mathematics Genealogy Project.

Lluís Masanes (2003). Necessary and sufficient condition for quantum-generated correlations. arXiv: quant-ph/0309137v1. The paper gives his affiliation as the Universitat de Barcelona.

The mathlib Community (2020). The Lean mathematical library. Certified Programs and Proofs, CPP 2020, pp. 367–381. DOI: 10.1145/3372885.3373824. The CHSH and Tsirelson inequalities for star-algebras are in Mathlib.Algebra.Star.CHSH.

Miguel Navascués, Stefano Pironio and Antonio Acín (2007). Bounding the Set of Quantum Correlations. Physical Review Letters 98, 010401. DOI: 10.1103/PhysRevLett.98.010401; arXiv: quant-ph/0607119v2.

Miguel Navascués, Stefano Pironio and Antonio Acín (2008). A convergent hierarchy of semidefinite programs characterizing the set of quantum correlations. New Journal of Physics 10, 073013. arXiv: 0803.4290v1.

Harold Ollivier, David Poulin and Wojciech H. Zurek (2004). Objective Properties from Subjective Quantum States: Environment as a Witness. Physical Review Letters 93, 220401. arXiv: quant-ph/0307229v2.

Marcin Pawłowski, Tomasz Paterek, Dagomir Kaszlikowski, Valerio Scarani, Andreas Winter and Marek Żukowski (2009). Information causality as a physical principle. Nature 461, pp. 1101–1104. Pawłowski's affiliation, the University of Gdańsk, is on the paper: arXiv: 0905.2292.

Helfried Peyrl and Pablo A. Parrilo (2008). Computing sum of squares decompositions with rational coefficients. Theoretical Computer Science 409(2), pp. 269–281. Author paper; DOI: 10.1016/j.tcs.2008.09.025.

William D. Phillips and Jean Dalibard (2022). Experimental tests of Bell's inequalities: A first-hand account by Alain Aspect. European Physical Journal D 77, article 8 (2023). arXiv: 2212.04737; DOI: 10.1140/epjd/s10053-022-00557-6.

Stefano Pironio, Antonio Acín, Serge Massar, A. Boyer de la Giroday, Dzmitry N. Matsukevich, Peter Maunz, Steven Olmschenk, David Hayes, Le Luo, T. Andrew Manning and Christopher Monroe (2010). Random numbers certified by Bell's theorem. Nature 464, pp. 1021–1024. DOI: 10.1038/nature09008.

Sandu Popescu and Daniel Rohrlich (1994). Quantum nonlocality as an axiom. Foundations of Physics 24, pp. 379–385. DOI: 10.1007/BF02058098.

Xiao-Liang Qi and Daniel Ranard (2021). Emergent classicality in general multipartite states and channels. Quantum 5, article 555. DOI: 10.22331/q-2021-09-28-555.

Peter Rastall (1985). Locality, Bell's theorem, and quantum mechanics. Foundations of Physics 15, pp. 963–972. DOI: 10.1007/BF00739036.

C. Jess Riedel, Wojciech H. Zurek and Michael Zwolak (2016). Objective past of a quantum universe: Redundant records of consistent histories. Physical Review A 93, 032126. DOI: 10.1103/PhysRevA.93.032126; arXiv: 1312.0331.

Erwin Schrödinger. Wikipedia. Article. Austrian physicist; the wave equation of 1926.

Lynden K. Shalm and 33 coauthors (2015). Strong Loophole-Free Test of Local Realism. Physical Review Letters 115, 250402. DOI: 10.1103/PhysRevLett.115.250402.

James Joseph Sylvester (1850). Additions to the articles "On a new class of theorems" and "On Pascal's theorem". Philosophical Magazine, pp. 363–370; reprinted in The Collected Mathematical Papers of James Joseph Sylvester, vol. 1, Cambridge University Press, 1904, pp. 145–151. The "oblong arrangement of terms" passage is quoted in Jeff Miller's Earliest Known Uses of Some of the Words of Mathematics (M), entry "matrix". "Oblong arrangement of terms" is on p. 150 of the Collected Papers, vol. 1.

B. S. Tsirel'son (1987). Quantum analogues of the Bell inequalities. The case of two spatially separated domains. Journal of Soviet Mathematics 36(4), pp. 557–570. DOI: 10.1007/BF01663472.

Boris Tsirelson. Wikipedia. Article. Born in Leningrad in 1950; professor at Tel Aviv University from 1991 to 2017.

T. Unden, D. Louzon, M. Zwolak, W. H. Zurek and F. Jelezko (2019). Revealing the emergence of classicality using nitrogen-vacancy centers. Physical Review Letters 123, 140402.

Jean Ville (1939). Étude critique de la notion de collectif. Gauthier-Villars, Paris. The maximal inequality for nonnegative martingales.

Stephanie Wehner (2006). Tsirelson bounds for generalized Clauser-Horne-Shimony-Holt inequalities. Physical Review A 73, 022110. arXiv: quant-ph/0510076v2; DOI: 10.1103/PhysRevA.73.022110.

Reinhard F. Werner and Michael M. Wolf (2001). Bell inequalities and entanglement. Quantum Information and Computation 1(3), pp. 1–25. DOI: 10.26421/QIC1.3-1.

John A. Wheeler (1990). Information, Physics, Quantum: The Search for Links. In Wojciech H. Zurek (ed.), Complexity, Entropy and the Physics of Information, Addison-Wesley. First given at the 3rd International Symposium on Foundations of Quantum Mechanics, Tokyo, 1989.

Andrew Whitaker (2016). John Stewart Bell and Twentieth Century Physics: Vision and Integrity. Oxford University Press.

Yanbao Zhang, Scott Glancy and Emanuel Knill (2011). Asymptotically optimal data analysis for rejecting local realism. Physical Review A 84, 062118. DOI: 10.1103/PhysRevA.84.062118.

Yanbao Zhang, Scott Glancy and Emanuel Knill (2013). Efficient quantification of experimental evidence against local realism. Physical Review A 88, 052119. DOI: 10.1103/PhysRevA.88.052119.

Wojciech H. Zurek (1981). Pointer basis of quantum apparatus: Into what mixture does the wave packet collapse? Physical Review D 24(6), pp. 1516–1525. DOI: 10.1103/PhysRevD.24.1516.

Wojciech H. Zurek (2009). Quantum Darwinism. Nature Physics 5, pp. 181–188. DOI: 10.1038/nphys1202; arXiv: 0903.5082v1.